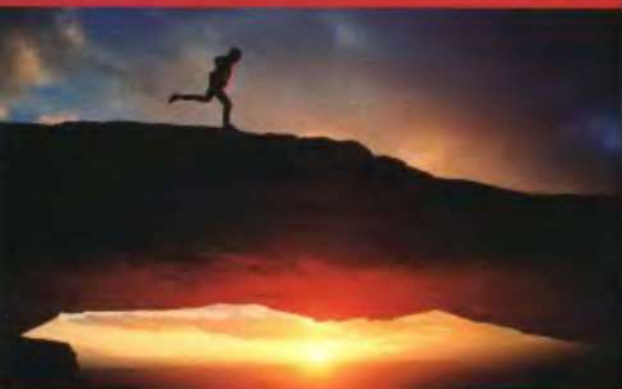


Эффективное программирование на C++

*Практическое программирование
на примерах*

Эндрю Кёниг
Барбара Му



Серия C++ In-Depth ♦ Бьярн Страуструп

Accelerated C++

Practical Programming by Example

**Andrew Koenig
Barbara E. Moo**



ADDISON-WESLEY

Boston • San Francisco • New York • Toronto • Montreal
London • Munich • Paris • Madrid
Capetown • Sydney • Tokyo • Singapore • Mexico City

Эффективное программирование на C++

Практическое программирование на примерах

**Эндрю Кёниг
Барбара Му**



Издательский дом "Вильямс"
Москва • Санкт-Петербург • Киев
2002

ББК 32.973.26-018.2.75

К35

УДК 681.3.07

Издательский дом "Вильямс"

Зав. редакцией А.В. Слепцов

Перевод с английского и редакция Н.М. Румко

По общим вопросам обращайтесь в Издательский дом "Вильямс" по адресу:
info@williamspublishing.com, http://www.williamspublishing.com

Кёниг, Эндрю, Му, Барбара, Э.

К35 Эффективное программирование на C++. Серия C++ *In-Depth*, т. 2. :
Пер. с англ. — М. : Издательский дом "Вильямс", 2002. — 384 с. : ил. —
Парал. тит. англ.

ISBN 5-8459-0350-5 (рус.)

Эта книга, в первую очередь, предназначена для тех, кому хотелось бы быстро научиться писать настоящие программы на языке C++. Зачастую новички в C++ пытаются освоить язык чисто механически, даже не попытавшись узнать, как можно эффективно применить его к решению каждодневных проблем. Цель данной книги — научить программированию на C++, а не просто изложить средства языка, поэтому она полезна не только для новичков, но и для тех, кто уже знаком с языком C++ и хочет использовать его в более натуральном, естественном стиле.

ББК 32.973.26-018.2.75

Все названия программных продуктов являются зарегистрированными торговыми марками соответствующих фирм.

Ни какая часть настоящего издания ни в каких целях не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами, будь то электронные или механические, включая фотокопирование и запись на магнитный носитель, если на это нет письменного разрешения издательства Addison-Wesley Publishing Company, Inc.

Authorized translation from the English language edition published by Addison-Wesley Publishing Company, Inc, Copyright ©2000

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from the Publisher.

Russian language edition published by Williams Publishing House according to the Agreement with R&I Enterprises International, Copyright © 2002

ISBN 5-8459-0350-5 (рус.)

ISBN 0-201-77581-6 (англ.)

© Издательский дом "Вильямс", 2002

© Addison-Wesley Publishing Company, Inc

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	15
0. ИТАК, НАЧНЕМ	21
1. РАБОТА СО СТРОКАМИ	29
2. ОРГАНИЗАЦИЯ ЦИКЛОВ И ВЫЧИСЛЕНИЙ	39
3. РАБОТА С ГРУППАМИ ДАННЫХ	59
4. ОРГАНИЗАЦИЯ ПРОГРАММ И ДАННЫХ	75
5. ИСПОЛЬЗОВАНИЕ ПОСЛЕДОВАТЕЛЬНЫХ КОНТЕЙНЕРОВ	101
6. ИСПОЛЬЗОВАНИЕ БИБЛИОТЕЧНЫХ АЛГОРИТМОВ	129
7. ИСПОЛЬЗОВАНИЕ АССОЦИАТИВНЫХ КОНТЕЙНЕРОВ	153
8. СОЗДАНИЕ ОБОБЩЕННЫХ ФУНКЦИЙ	171
9. ОПРЕДЕЛЕНИЕ НОВЫХ ТИПОВ	189
10. УПРАВЛЕНИЕ ПАМЯТЬЮ И ИСПОЛЬЗОВАНИЕ СТРУКТУР ДАННЫХ НИЗКОГО УРОВНЯ	205
11. ОПРЕДЕЛЕНИЕ АБСТРАКТНЫХ ТИПОВ ДАННЫХ	225
12. СОЗДАНИЕ ОБЪЕКТОВ КЛАССОВ, ИСПОЛЬЗУЕМЫХ КАК ЗНАЧЕНИЯ	253
13. НАСЛЕДОВАНИЕ И ДИНАМИЧЕСКОЕ СВЯЗЫВАНИЕ	271
14. ПОЧТИ АВТОМАТИЧЕСКОЕ УПРАВЛЕНИЕ ПАМЯТЬЮ	299
15. ВОЗВРАЩАЯСЬ К СИМВОЛЬНЫМ ИЗОБРАЖЕНИЯМ	317
16. КУДА ТЕПЕРЬ ДЕРЖАТЬ НАМ ПУТЬ	339
ПРИЛОЖЕНИЕ А. ЯЗЫК C++ (ПОДРОБНО)	343
ПРИЛОЖЕНИЕ Б. СТАНДАРТНАЯ БИБЛИОТЕКА (КРАТКИЙ ОБЗОР)	361
ПРЕДМЕТНЫЙ УКАЗАТЕЛЬ	377

ВВЕДЕНИЕ	15
Новый подход к программированию на C++	15
Наша книга полезна как для новичков, так и для опытных программистов	15
Абстракция	16
Охват материала	16
Несколько слов для опытных C- и C++-программистов	17
Структура книги	18
Как получить максимальную пользу от этой книги	18
Благодарности	19
0. ИТАК, НАЧНЕМ	21
0.1. Комментарии	21
0.2. Директива #include	22
0.3. Функция main	22
0.4. Фиgurные скобки	22
0.5. Использование стандартной библиотеки для вывода данных	23
0.6. Инструкция goto	23
0.7. Коннем чуть-чуть глубже	24
0.8. Резюме	25
Упражнения	27
1. РАБОТА СО СТРОКАМИ	29
1.1. Ввод данных	29
1.2. Выделение текста с помощью рамки	32
1.3. Резюме	35
Упражнения	36
2. ОРГАНИЗАЦИЯ ЦИКЛОВ И ВЫЧИСЛЕНИЙ	39
2.1. В чем суть проблемы	39
2.2. Общая структура программы	40
2.3. Вывод неизвестного числа строк	40
2.3.1. Понятие о while-инструкции	41
2.3.2. Разработка while-инструкции	42
2.4. Вывод строки	44
2.4.1. Вывод символов рамки	45
2.4.2. Вывод символов, не относящихся к рамке	48
2.5. Полная программа вывода приветствия в рамочке	49
2.5.1. Устранение многократного повторения префикса std::	49
2.5.2. Использование log-инструкции для компактности кода	50
2.5.3. Объединение нескольких проверок в одну	51

2.5.4. Полная программа вывода приветствия в рамочке	52
2.6. С чего начать отсчет	53
2.7. Резюме	54
Упражнения	57
3. РАБОТА С ГРУППАМИ ДАННЫХ	59
3.1. Вычисление оценок студентов	59
3.1.1. Как определить конец ввода данных	64
3.1.2. Инвариант цикла	65
3.2. Использование медианы вместо среднеарифметического	65
3.2.1. Сохранение коллекции данных в векторе	66
3.2.2. Генерирование выходных данных	67
3.2.3. Еще несколько замечаний	72
3.3. Резюме	73
Упражнения	74
4. ОРГАНИЗАЦИЯ ПРОГРАММ И ДАННЫХ	75
4.1. Организация вычислений	75
4.1.1. Вычисление медиан	77
4.1.2. Пересмотр политики вычисления оценок	78
4.1.3. Считывание оценок за выполнение домашних заданий	80
4.1.4. Три вида параметров функции	82
4.1.5. Использование функций для вычисления итоговой оценки студента	84
4.2. Организация данных	86
4.2.1. Соберем-ка все данные о студентах в одну кучу!	87
4.2.2. Управление занятиями с данными о студентах	87
4.2.3. Построение отчета	89
4.3. А теперь соберем все вместе	91
4.4. Декомпозиция программы вычисления итоговых оценок	94
4.5. Исправленная версия программы вычисления итоговых оценок	96
4.6. Резюме	97
Упражнения	99
5. ИСПОЛЬЗОВАНИЕ ПОСЛЕДОВАТЕЛЬНЫХ КОНТЕЙНЕРОВ	101
5.1. Разделение студентов на категории	101
5.1.1. Удаление элементов из вектора	102
5.1.2. Последовательный и произвольный доступ к данным	105
5.2. Итераторы	105
5.2.1. Типы итераторов	106
5.2.2. Операции, выполняемые итераторами	107
5.2.3. О некоторых синтаксических нюансах	108
5.2.4. Что означает выражение <code>students.erase(students.begin() + i)</code>	108
5.3. Использование итераторов вместо индексов	109
5.4. Изменение структуры данных для повышения производительности	111
5.5. Тип <code>list</code>	111
5.5.1. На некоторые различия стоит обратить особое внимание	113
5.5.2. Зачем так беспокоиться?	113
5.6. Разберем <code>string</code> -объект на части	114

5.7. Тестирование функции split	117
5.8. Сборка string-объектов	119
5.8.1. Опять рамочка	119
5.8.2. Вертикальная конкатенация	121
5.8.3. Горизонтальная конкатенация	122
5.9. Резюме	124
Упражнения	127
6. ИСПОЛЬЗОВАНИЕ БИБЛИОТЕЧНЫХ АЛГОРИТМОВ	129
6.1. Анализ string-объектов	129
6.1.1. Еще один вариант функции split	131
6.1.2. Палиндромы	133
6.1.3. Поиск URL-адресов	134
6.2. Сравнение схем вычисления оценок	138
6.2.1. Обработка записей с оценками студентов	139
6.2.2. Анализ оценок	140
6.2.3. Вычисление итоговых оценок на основе среднего арифметического значения оценок за домашние задания	143
6.2.4. Медиана оценок, полученных за выполненные домашние задания	144
6.3. Новый вариант классификации студентов	145
6.3.1. Решение в два прохода	146
6.3.2. Решение в один проход	148
6.4. Алгоритмы, контейнеры и итераторы	149
6.5. Резюме	150
Упражнения	152
7. ИСПОЛЬЗОВАНИЕ АССОЦИАТИВНЫХ КОНТЕЙНЕРОВ	153
7.1. Контейнеры, поддерживающие эффективный поиск	153
7.2. Подсчет слов	154
7.3. Генерирование таблицы перекрестных ссылок	156
7.4. Генерирование предложений	160
7.4.1. Представление правил	161
7.4.2. Чтение грамматических правил	162
7.4.3. Генерирование предложения	163
7.4.4. Выбор случайного элемента	166
7.5. Вспомним о производительности	168
7.6. Резюме	168
Упражнения	170
8. СОЗДАНИЕ ОБОБЩЕННЫХ ФУНКЦИЙ	171
8.1. Что такое обобщенная функция	171
8.1.1. Медианы неизвестного типа	172
8.1.2. Реализация шаблонов	174
8.1.3. Обобщенные функции и типы	175
8.2. Независимость структур данных	176
8.2.1. Алгоритмы и итераторы	177
8.2.2. Последовательный доступ только для чтения	178
8.2.3. Последовательный доступ только для записи	179

8.2.4. Последовательный доступ для чтения и записи	180
8.2.5. Двухнаправленный доступ	180
8.2.6. Произвольный доступ	181
8.2.7. Диапазоны итераторов и окончные значения	182
8.3. Входные и выходные итераторы	183
8.4. Использование итераторов для повышения гибкости программирования	185
8.5. Резюме	186
Упражнения	187
9. ОПРЕДЕЛЕНИЕ НОВЫХ ТИПОВ	189
9.1. Вернемся к структуре Student_info	189
9.2. Типы классов	190
9.2.1. Функции-члены	191
9.2.2. Функции-не-члены	193
9.3. Средства защиты	194
9.3.1. Функции доступа	195
9.3.2. Тестирование на пустоту	197
9.4. Класс Student_info	197
9.5. Конструкторы	198
9.5.1. Конструктор по умолчанию	200
9.5.2. Конструкторы с аргументами	201
9.6. Использование класса Student_info	201
9.7. Резюме	202
Упражнения	203
10. УПРАВЛЕНИЕ ПАМЯТЬЮ И ИСПОЛЬЗОВАНИЕ СТРУКТУР ДАННЫХ НИЗКОГО УРОВНЯ	205
10.1. Указатели и массивы	205
10.1.1. Указатели	206
10.1.2. Указатели на функции	208
10.1.3. Массивы	210
10.1.4. Арифметические операции с указателями	211
10.1.5. Индексирование	212
10.1.6. Инициализация массивов	212
10.2. Снова о строковых литералах	213
10.3. Инициализация массивов указателей на символы	214
10.4. Аргументы для функции main	216
10.5. Чтение и запись файлов	217
10.5.1. Стандартный поток ошибок	217
10.5.2. Использование нескольких входных и выходных файлов	217
10.6. Три вида управления памятью	219
10.6.1. Размещение объекта в памяти и освобождение этой памяти	220
10.6.2. Размещение в памяти массива	221
10.7. Резюме	222
Упражнения	224
11. ОПРЕДЕЛЕНИЕ АБСТРАКТНЫХ ТИПОВ ДАННЫХ	225
11.1. Класс Vec	225
11.2. Реализация класса Vec	226

11.2.1. Распределение памяти	227
11.2.2. Конструкторы	228
11.2.3. Определение типов	229
11.2.4. Индексирование и определение размера	231
11.2.5. Операции, возвращающие итераторы	232
11.3. Управление копированием	233
11.3.1. Конструктор копирования	234
11.3.2. Присваивание	235
11.3.3. Присваивание — это не инициализация	238
11.3.4. Деструктор	239
11.3.5. Операции по умолчанию	240
11.3.6. Тройное правило	241
11.4. Динамические Vec-объекты	242
11.5. Гибкое управление памятью	243
11.5.1. Конечный вариант класса Vec	245
11.6. Резюме	249
Упражнения	250
12. СОЗДАНИЕ ОБЪЕКТОВ КЛАССОВ, ИСПОЛЪЗУЕМЫХ КАК ЗНАЧЕНИЯ	253
12.1. Простой класс string	254
12.2. Автоматические преобразования	255
12.3. Операции над Str-объектами	256
12.3.1. Операторы ввода-вывода	257
12.3.2. “Друзья”	258
12.3.3. Другие бинарные операторы	260
12.3.4. Выражения смешанного типа	262
12.3.5. Разработка бинарных операторов	263
12.4. Некоторые преобразования просто опасны	264
12.5. Операторы преобразования	264
12.6. Преобразования и управление памятью	266
12.7. Резюме	268
Упражнения	269
13. НАСЛЕДОВАНИЕ И ДИНАМИЧЕСКОЕ СВЯЗЫВАНИЕ	271
13.1. Наследование	271
13.1.1. Снова о защите	273
13.1.2. Операции	273
13.1.3. Наследование и конструкторы	275
13.2. Полиморфизм и виртуальные функции	277
13.2.1. Получение значения при неизвестном типе объекта	278
13.2.2. Динамическое связывание	279
13.2.3. Подведем некоторые итоги	281
13.3. Использование наследования для решения нашей “старой” задачи	282
13.3.1. Контейнеры (фактически) неизвестного типа	284
13.3.2. Виртуальные деструкторы	287
13.4. Простой дескрипторный класс	288
13.4.1. Считывание дескриптора	290
13.4.2. Копирование дескрипторных объектов	291

13.5. Использование дескрипторного класса	293
13.6. Вникнем в некоторые подробности	294
13.6.1. Наследование и контейнеры	294
13.6.2. Какая функция вам нужна	295
13.7. Резюме	295
Упражнения	297
14. Почти АВТОМАТИЧЕСКОЕ УПРАВЛЕНИЕ ПАМЯТЬЮ	299
14.1. Дескрипторы, которые копируют свои объекты	300
14.1.1. Обобщенный дескрипторный класс	300
14.1.2. Использование обобщенного дескриптора	304
14.2. Дескрипторы с подсчетом количества ссылок	306
14.3. Дескрипторы для решения проблемы совместного использования данных	309
14.4. Усовершенствование управляемых дескрипторов	311
14.4.1. Копирование типов, которыми мы не можем управлять	312
14.4.2. Когда в копии есть необходимость	314
14.5. Резюме	315
Упражнения	315
15. ВОЗВРАЩАЯСЬ К СИМВОЛЬНЫМ ИЗОБРАЖЕНИЯМ	317
15.1. Проект системы	317
15.1.1. Использование наследования при моделировании структуры	318
15.1.2. Класс Pic_base	321
15.1.3. Производные классы	323
15.1.4. Управление копированием	326
15.2. Реализация	326
15.2.1. Реализация пользовательского интерфейса	326
15.2.2. Класс String_Pic	329
15.2.3. Дополнение выходных данных пробелами	331
15.2.4. Класс VCat_Pic	332
15.2.5. Класс HCat_Pic	333
15.2.6. Класс Frame_Pic	334
15.2.7. Не забывайте о друзьях	335
15.3. Резюме	337
Упражнения	338
16. КУДА ТЕПЕРЬ ДЕРЖАТЬ НАМ ПУТЬ	339
16.1. Используйте уже освоенные абстракции	339
16.2. Стремитесь узнать больше	341
Упражнения	342
ПРИЛОЖЕНИЕ А. ЯЗЫК C++ (ПОДРОБНО)	343
A.1. Объявления	343
A.1.1. Спецификаторы	345
A.1.2. Описатели	346
A.2. Типы	348
A.2.1. Целые типы	349
A.2.2. Тип значений с плавающей точкой	352

А.2.3. Константные выражения	352
А.2.4. Преобразования	353
А.2.5. Перечислимые типы	354
А.2.6. Перегрузка	354
А.3. Выражения	355
А.3.1. Операторы	358
А.4. Инструкции	359
Приложение Б. Стандартная библиотека (краткий обзор)	361
Б.1. Ввод-вывод информации	362
Б.2. Контейнеры и итераторы	364
Б.2.1. Общие контейнерные операции	364
Б.2.2. Последовательные контейнеры	365
Б.2.3. Дополнительные последовательные операции	367
Б.2.4. Ассоциативные контейнеры	367
Б.2.5. Итераторы	368
Б.2.6. Класс vector	370
Б.2.7. Класс list	370
Б.2.8. Класс string	371
Б.2.9. Класс pair	372
Б.2.10. Класс map	372
Б.3. Алгоритмы	373
Предметный указатель	377

Нашим студентам,
благодаря которым мы поняли,
как нужно учить других

Новый подход к программированию на C++

Если вы читаете эти строки, то вам, вероятно, хотелось бы быстро научиться писать стоящие программы на языке C++. Посему сразу же возьмем “быка за рога” и рассмотрим самые полезные разделы C++. Такая стратегия, возможно, станет понятной в процессе осуществления; основной ее принцип состоит в том, что мы не будем начинать с изучения языка C, несмотря на то что C++ строится на C. Мы сразу же начнем с использования структур данных высокого уровня и только позднее рассмотрим детали, на которых они базируются. Такой подход позволит вам немедленно приступить к написанию правильно организованных программ на C++.

Данный подход имеет еще одну необычную особенность. Мы делаем акцент не на изучении возможностей самого языка и библиотек, а на решении проблем. Конечно же, мы не уходим от разъяснения применяемых возможностей, но делаем это для поддержки программ, а не для того, чтобы использовать программы в качестве демонстрации этих возможностей.

Цель этой книги — научить программированию на C++, а не просто изложить средства языка; она особенно полезна читателям, которые уже знакомы с C++ и хотят использовать этот язык более эффективно. Ведь довольно часто приходится сталкиваться с тем, что новички в C++ пытаются освоить язык чисто механически, т.е. не пытаясь узнать, как применить его к решению каждодневных проблем.

Наша книга полезна как для новичков, так и для опытных программистов

Каждое лето мы практиковали недельный курс интенсивного изучения языка C++ в Станфордском университете (Stanford University). Для этого курса изначально был принят традиционный подход. Предполагая, что студенты уже знают C, мы начинали с демонстрации использования классов, а затем систематически “проходили” остальные разделы языка. Мы обнаружили, что наши студенты пребывали в растерянности в течение почти двух дней — до тех пор, пока не узнавали достаточно для того, чтобы приступить к написанию программ. С этого момента обучение продвигалось быстрее.

При переходе к C++-среде, которая поддерживала совершенно новую на тот момент стандартную библиотеку, мы тщательно пересмотрели свой курс. В новом курсе с самого начала использовалась эта библиотека, и внимание студентов акцентировалось именно на написании программ, а углубленное рассмотрение деталей мы практиковали только после того, как студенты получали достаточно информации, чтобы продуктивно их использовать.

Результаты оказались просто ошеломляющими. Через день после начала занятий наши студенты могли писать программы, в отличие от предыдущего курса, когда они приходили к этому лишь к концу недели. Более того, от растерянности слушателей наших курсов не осталось и следа.

Абстракция

Наш подход стал возможен только благодаря усовершенствованию C++ и нашему пониманию этого языка, что позволило нам игнорировать многие второстепенные темы, которые ставились во главу угла программистами раннего периода “жизни” C++.

Способность игнорировать детали — характерная особенность “зрелости” любых технологий. Например, первые автомобили ломались так часто, что каждому автомобилисту одновременно приходилось быть и автомехаником. Было полным безрассудством в то время отправляться в дорогу, не зная деталей устройства автомобиля. Современным водителям не нужны глубокие знания автомеханика, чтобы использовать автомобиль в качестве транспортного средства. Они, конечно, могут вникать в детали своего “железного коня” по какому-то иному причинам, но это совершенно другая история.

Мы определяем абстракцию как избирательное неведение, которое выражается в сосредоточении внимания на идеях текущей задачи и игнорировании всего остального. На наш взгляд, это самое важное в современном программировании. Ключ к написанию успешно работающей программы лежит в знании того, какие части проблемы следует принять во внимание, а какие — игнорировать. Каждый язык программирования предлагает инструменты создания полезных абстракций, и каждый успешно работающий программист знает, как использовать эти инструменты наилучшим образом.

Мы считаем абстракции настолько полезными, что буквально заполнили ими эту книгу. Безусловно, мы не называем их абстракциями в прямом смысле этого слова, поскольку они принимают различные формы. Мы оперируем такими понятиями, как функции, структуры данных, классы и наследование; ведь все это абстракции, которые мы не просто упоминаем здесь — мы их активно используем.

Если абстракции хорошо разработаны и удачно выбраны, то их можно использовать даже в том случае, когда вы не понимаете все их детали. Ведь нам необязательно быть автомеханиками, чтобы водить автомобиль, и точно так же нам не нужно понимать до конца работу C++-среды, чтобы использовать ее.

Охват материала

Если у вас серьезные намерения в отношении программирования на C++, вам просто необходимо прочесть эту книгу, несмотря на то что она не включает всего, что вам нужно знать.

Это утверждение вовсе не так уж парадоксально, как может показаться на первый взгляд. Ни одна книга такого объема не сможет уместить всей информации о C++, поскольку разным программистам для создания различных программ потребуются различные знания. Поэтому любая более полная книга о C++, например *Язык программирования C++* Бьярни Страуструпа (Addison-Wesley, 2000), при необходимости может вам уточнить интересующие детали.

С другой стороны, многие разделы C++ настолько важны (в универсальном смысле), что без их понимания просто невозможно написать эффективно работающие программы. Нашу книгу составляют именно такие разделы. Используя информацию

только из этой книги, можно написать огромное множество программ. Один из наших рецензентов, ведущий программист по обслуживанию большой коммерческой системы, написанной на C++, сказал, что в этой книге описаны все основные средства, которые он использует в своей работе.

С помощью этих средств вы можете писать настоящие C++-программы, а не C++-программы в стиле C или любого другого языка. Освоив материал этой книги, вы поймете, что еще вам нужно узнать и как приступить к работе. У любителей создавать телескопы есть поговорка, что легче вначале сделать 3-дюймовый отражатель, а затем 6-дюймовый, чем с самого начала приступить к созданию 6-дюймового отражателя.

Мы описываем только стандарт языка C++ и игнорируем всяческие расширения. Преимущество такого подхода в том, что программы, которые вы научитесь писать, будут работать везде. Кроме того, мы не уделяем внимание тому, как писать программы, предназначенные для работы в оконных средах, поскольку такие программы обязательно привязаны к конкретной среде. Постойте! Не спешите откладывать книгу в сторону! Поскольку наш подход универсален, в будущем вы сможете использовать все, что узнаете из этой книги, в любых средах. И потом, вы еще успеете прочитать, к примеру, о GUI-приложениях, но только *после того*, как освоите нашу книгу.

Несколько слов для опытных C- и C++-программистов

При изучении нового языка программирования часто приходится наблюдать, как многие пытаются писать программы в стиле языков, которые уже им знакомы. Наш подход позволяет избежать подобного, благодаря использованию абстракций высокого уровня из стандартной библиотеки C++ с самого начала обучения. Если вы имеете опыт работы на C или C++, то при таком подходе вас ожидает ряд хороших и не очень хороших новостей (причем это одни и те же новости).

Вероятно, новостью окажется для вас уже то, сколь небольшой уровень ваших знаний позволит понять язык C++ так, как он представлен в нашей книге. Возможно, на первых порах вам придется узнать больше, чем вы того ожидали (плохая новость), но вы освоите незнакомый материал быстрее, чем ожидали (хорошая новость). В частности, если вы уже знакомы с C++, то, скорее всего, вы сначала научились писать программы на C; это означает, что ваш стиль программирования построен на фундаменте языка C. В этом нет ничего дурного, но наш подход настолько отличается от упомянутого выше, что, как нам кажется, с нашей помощью вы увидите C++ с совершенно иной точки зрения, с которой вам еще не приходилось на него смотреть.

Конечно, многие детали синтаксиса вам будут уже знакомы, но ведь это всего лишь детали! Важные понятия языка представлены в книге, возможно, в совершенно неожиданном для вас порядке. Например, мы не касаемся массивов и указателей аж до главы 10 и даже не собираемся рассматривать такие любимцы многих программистов, как функции `printf` и `malloc`. Однако прямо в главе 1 мы рассматриваем библиотечный класс `string`. Заявляя о совершенно новом подходе в изучении C++, мы знаем, что говорим!

Структура книги

Нам кажется, что эту книгу удобно разделить на две части. В первой части (главы 0–7) акцент делается на программах, которые используют абстракции стандартной библиотеки. Во второй части (главы 8–16) речь идет об определении собственных абстракций.

Безусловно, такое раннее знакомство с библиотечными средствами может показаться неожиданным, но нам такое решение кажется правильным. Значительная часть синтаксиса языка C++ — особенно более трудные его разделы — обязана своим существованием авторам библиотеки. Пользователям библиотеки совсем не обязательно знать эти разделы языка. Игнорируя их до начала второй части книги, мы позволяем нашим читателям приступить к написанию C++-программ гораздо быстрее, чем при более традиционном подходе.

Если вы поймете, как использовать библиотеку, то будете готовы усвоить материал, посвященный средствам низкого уровня, на которых построена библиотека, и научитесь применять эти средства для написания собственных библиотек. Более того, вы начнете понимать, когда библиотека будет полезной, а когда стоит избегать написания нового библиотечного кода.

Хотя эта книга по объему меньше многих других книг о C++, мы все же попытались использовать в ней каждую важную идею, по крайней мере, дважды, а ключевые идеи — и того чаще. В результате многие части нашей книги ссылаются на другие части. Употребляя (и поясняя) впервые какой-нибудь важный термин, мы выделяем его *полужирным курсивом*, чтобы обратить на него ваше внимание. Кроме того, *так* его будет легче отыскать в тексте, если вы захотите вернуться к нему, чтобы при необходимости перечитать приведенные пояснения.

Все главы (за исключением последней) завершаются разделами “Резюме”. Эти разделы служат двум целям. Они позволяют вспомнить идеи, изложенные в данной главе, а также ознакомиться с дополнительным материалом, который, по нашему мнению, вам рано или поздно необходимо узнать. Мы предполагаем, что при первом чтении достаточно быстрого ознакомления с этими разделами, но они вам еще сослужат хорошую службу, если вы захотите вернуться и освежить в памяти некоторые моменты.

В двух приложениях изложены важные аспекты языка и библиотеки на таком уровне детализации, который, мы надеемся, окажется для вас весьма полезным при написании собственных программ.

Как получить максимальную пользу от этой книги

Любая книга по программированию содержит примеры программ, и эта не является исключением. Чтобы понять, как работают эти программы, у вас нет другого выхода, как выполнить их на компьютере. Компьютерный парк постоянно обновляется, поэтому все, что мы могли бы сказать о нем сейчас, окажется неточным к тому времени, когда вы прочтете эти слова. Поэтому, если вы еще не знаете, как скомпилировать и выполнить C++-программу, обратитесь, пожалуйста, по адресу: <http://www.acceleratedcpp.com> и прочитайте все, что мы приготовили для вас. Время от времени мы будем обновлять этот Web-сайт, и на нем вы всегда сможете получить актуальную информацию и советы по выполнению C++-программ. На этом сайте вы также найдете “готовые к употреблению” версии некоторых примеров программ и другую информацию, которая может оказаться для вас интересной.

Благодарности

Мы хотели бы поблагодарить тех людей, без помощи которых создание этой книги было бы невозможно. Особо мы признательны рецензентам, которые внесли свой вклад в написание этой книги: Роберту Бергеру (Robert Berger), Дагу Бруку (Dag Brück), Адаму Бучсбауму (Adam Buchsbaum), Стефану Клеймеджу (Stephen Clamage), Джону Кэлбу (John Kalb), Джеффри Олдхэму (Jeffrey Oldham), Дэвиду Слейтону (David Slayton), Бьярни Страуструпу (Bjarne Stroustrup), Альберту Тенбушу (Albert Tenbusch), Брюсу Тителману (Bruce Tetelman) и Кловису Тондо (Clovis Tondo). В издании этой книги принимали участие многие сотрудники издательства Addison-Wesley, поэтому огромное спасибо Тиреллу Албаху (Tytrell Albaugh), Банни Эймс (Bunny Ames), Майку Хендриксону (Mike Hendrickson), Деборе Лаферти (Deborah Lafferty), Кэси Охала (Cathy Ohala) и Симоне Пеймент (Simone Payment). Александр Цирис (Alexander Tsis) проверил греческую этимологию, используемую в разделе 13.2.2. Наконец, идея начать книгу с создания высокоуровневых программ зрела в наших головах в течение многих лет, но окончательно сформировалась благодаря сотням студентов, которые посещали наши курсы, и тысячам желающих научиться программировать, которые слушали наши лекции.

Эндрю Кознинг (Andrew Koenig)
Барбара Му (Barbara E. Moo)

Жилетт, Нью-Джерси
июнь 2000 года

ИТАК, НАЧНЕМ

Давайте-ка начнем с рассмотрения небольшой C++-программы.

```
// Маленькая C++-программа.  
#include <iostream>  
  
int main()  
{  
    std::cout << "Привет, мир!" << std::endl;  
    return 0;  
}
```

Программисты часто обращаются к такой программе. Несмотря на ее лаконизм, вам, прежде чем продолжить чтение, все равно понадобится затратить некоторое время на то, чтобы скомпилировать и запустить ее на своем компьютере. Программа должна записать текст

Привет, мир!

в стандартный выходной поток, которым обычно является окно на экране дисплея. Если при выполнении этой программы у вас возникли какие-либо проблемы, обратитесь за помощью к тому, кто уже знаком с C++, или воспользуйтесь Web-сайтом по следующему адресу.

<http://www.acceleratedcpp.com>

Особенность этой простой программы состоит в том, что если (несмотря на ее простоту) у вас возникли проблемы с ее выполнением, то наиболее вероятными причинами недоразумений являются опечатки или неправильное представление о том, как взаимодействовать с этой программой в процессе ее выполнения. Кроме того, полное понимание даже такой маленькой программы означает знание довольно существенной части основ языка C++. Чтобы добиться необходимого уровня понимания, рассмотрим подробно каждую строку этой программы.

0.1. Комментарии

Первая строка программы имеет следующий вид.

```
// маленькая C++-программа.
```

Комментарий (comment) начинается символами “//” и продолжается до конца строки. Компилятор игнорирует комментарии; их назначение — разъяснять составные части программы. В этой книге комментарии отмечаются *курсивом*, чтобы было проще отличить их от других частей программы.

0.2. Директива #include

В C++ многие основные средства языка (такие, как средства ввода-вывода) являются частью *стандартной библиотеки* (standard library), а не частью *базового языка* (core language). Это различие очень существенно, поскольку базовый язык всегда доступен для всех C++-программ, а что касается стандартной библиотеки, то в отношении тех ее частей, которые вы собираетесь использовать, необходимо в явном виде сделать запрос на их включение.

Средства стандартных библиотек программы запрашивают с помощью *директив* #include, которые обычно размещаются в начале программы. Единственная часть стандартной библиотеки, которая используется в нашей программе, связана с обеспечением операций ввода-вывода, и запрос на ее использование выражается следующей строкой.

```
#include <iostream>
```

Имя `iostream` предлагает поддержку последовательного, или потокового, ввода-вывода данных, а не ввода-вывода с произвольным доступом. Поскольку имя `iostream` входит в состав директивы #include и заключено в *угловые скобки* (< и >), оно указывает на часть библиотеки C++, именуемую *стандартным заголовком* (standard header).

Стандарт C++ точно не определяет, что представляет собой стандартный заголовок, но он указывает имя и поведение каждого заголовка. Включение стандартного заголовка делает доступными для программы средства соответствующей библиотеки, но точное функционирование этих средств определяется конкретной реализацией языка, а не вами.

0.3. Функция main

Функция (function) — это часть программы с именем, по которому некоторая другая часть программы может ее *вызвать* (call), т.е. заставить выполниться. Каждая C++-программа должна содержать функцию с именем `main`. При запуске программы на выполнение происходит вызов именно этой функции.

При вызове функции `main` в качестве результата ее выполнения возвращается некоторое целое число, назначение которого — сообщить C++-среде, в которой работает данная программа, насколько успешно прошла ее выполнение. Нулевое значение говорит об успешном выполнении, любое другое — о наличии проблем. Таким образом, строка

```
int main()
```

означает, что мы определяем функцию с именем `main`, которая возвращает значение типа `int`, где `int` — имя, используемое базовым языком для описания целых чисел. В круглые скобки, стоящие после имени `main`, заключаются параметры, которые передаются функции из среды. В нашем примере параметры отсутствуют, поэтому круглые скобки пустуют. О том, как используются параметры, передаваемые функции `main`, мы поговорим в разделе 10.4.

0.4. Фигурные скобки

Определение нашей функции `main` (с парой круглых скобок) продолжает последовательность *инструкций* (statements), заключенных в фигурные скобки.

```
int main()
{
    // Левая фигурная скобка.
    // Здесь должны быть инструкции.
}
// Правая фигурная скобка.
```

В C++ фигурные скобки уведомляют среду о следующем: все, что находится внутри них, следует воспринимать (и обрабатывать) как **блок**. В данном примере левая фигурная скобка отмечает начало последовательности инструкций нашей функции `main`, а правая — их конец. Другими словами, фигурные скобки означают, что все инструкции, расположенные между ними, являются частью одной и той же функции.

Если между фигурными скобками заключены две или больше инструкций (как в данной функции), они выполняются в порядке следования.

0.5. Использование стандартной библиотеки для вывода данных

Первая инструкция в фигурных скобках выполняет, наконец-то, реальное действие в нашей программе.

```
std::cout << "Привет, мир!" << std::endl;
```

Эта инструкция использует *оператор вывода* (output operator) стандартной библиотеки, "<<", сначала для вывода в стандартный выходной поток текста `Привет, мир!`, а затем для записи значения `std::endl`.

Префикс `std::` означает, что само имя является частью *пространства имен* (namespace), или именованной областью видимости, `std`. Пространство имен — это коллекция связанных имен. Стандартная библиотека использует пространство имен `std` для хранения всех определяемых ею имен. Например, стандартный заголовок `iostream` определяет имена `cout` и `endl`, а мы обращаемся к этим именам, используя их более "полную" форму: `std::cout` и `std::endl`.

Имя `std::cout` принадлежит стандартному выходному потоку, который представляет собой средство, используемое C++-средой для обычного вывода данных из программ. В типичной C++-среде, работающей в операционной системе управления полэкраным отображением информации, имя `std::cout` означает окно, которое среда связывает с программой на время ее выполнения. Под управлением такой системы выходные данные, записанные в поток `std::cout`, появятся в соответствующем окне.

Запись значения `std::endl` завершает текущую строку выходных данных, в результате чего очередные выходные данные (если таковые будут сгенерированы программой) должны отобразиться на новой строке.

0.6. Инструкция return

Инструкция `return` в виде

```
return 0;
```

завершает выполнение функции, в которой она находится, и передает значение, указанное между словом `return` и точкой с запятой (в данном примере — это число 0), программе, вызвавшей эту функцию. Возвращаемое значение должно иметь тип, который соответствует типу, указанному функцией. При использовании функции `main` типом возвращаемого ею значения является `int`, а программой, которой функция `main` возвращает значение, — сама C++-среда. Следовательно, инструкция

return в функции main должна включать целочисленное выражение, которое передается назад среде.

Конечно, в программе может существовать несколько (а не одно) мест, в которых имело бы смысл завершить выполнение программы; в этом случае программа должна иметь несколько инструкций return. И если определение функции “обещает”, что функция возвратит значение конкретного типа, тогда каждая инструкция return в этой функции должна возвращать значение “обещанного” типа.

0.7. Копнем чуть-чуть глубже

В этой программе используются еще два понятия, которыми “пропитан” язык C++: выражения и области видимости. Более подробно мы обсудим их ниже, а пока рассмотрим лишь основные моменты.

Выражение (expression) предлагает C++-среде выполнить заданное вычисление. Вычисление имеет некоторый результат, а также может иметь побочные эффекты, т.е. может оказать влияние на состояние программы или даже среды, причем это влияние не является непосредственной частью результата. Например, 3+4 представляет собой выражение, результат которого равен значению 7 при отсутствии побочных эффектов, а следующая запись

```
std::cout << "Привет, мир!" << std::endl;
```

является выражением, которое в качестве побочного эффекта выводит текст Привет, мир! в стандартный выходной поток и завершает текущую строку.

Любое выражение содержит *операторы* и *операнды*, которые могут выступать в различных формах. В приведенном выше выражении символы “<<” являются операторами, а std::cout, “Привет, мир!” и std::endl — операндами.

Каждый операнд имеет *тип* (type). Подробное обсуждение типов еще впереди, но заметим, что под типом подразумевается структура данных и значения операндов, которые имеют смысл для этой структуры данных. Результат выполнения операции зависит от типа участвующих в ней операндов.

Часто типы имеют имена. Например, базовый язык определяет int как имя типа для представления целых значений, а стандартная библиотека определяет std::ostream как тип, обеспечивающий вывод данных, основанный на концепции потоков. В нашей программе операнд std::cout имеет тип std::ostream.

При выполнении оператора “<<” используются два операнда. Но в нашей программе мы записали два таких оператора и три операнда. Как же такое может быть? Дело в том, что оператор “<<” является *левоассоциативным* (left-associate). Это значит, что если оператор “<<” встречается в одном и том же выражении дважды (или чаще), то каждый знак “<<” будет использовать максимально возможную часть выражения, которую он способен обработать в качестве левого операнда, и минимально возможную, которую он способен обработать в качестве правого операнда. В нашем примере первый оператор “<<” примет “Привет, мир!” в качестве правого операнда и std::cout в качестве левого, а второй знак “<<” примет std::endl в качестве правого операнда и std::cout << “Привет, мир!” в качестве левого. Если использовать круглые скобки для “выяснения отношений” между операндами и операторами, то мы увидим, что наше выражение вывода данных эквивалентно следующему.

```
(std::cout << "Привет, мир!") << std::endl;
```

Поведение каждого оператора “<<” зависит от типов его операндов. Напомним, первый знак “<<” слева принимает операнд `std::cout`, который имеет тип `std::ostream`. Правым операндом является строковый литерал, имеющий несколько загадочный тип, который мы не будем обсуждать, пока не доберемся до раздела 10.2. Принимая операнды таких типов, оператор “<<” записывает символы своего правого операнда в поток, указанный его левым операндом, причем результат операции сохраняется в его левом операнде.

Следовательно, левым операндом второго оператора “<<” является выражение, содержащееся в операнде `std::cout` типа `std::ostream`, а правым операндом — выражение `std::endl`, которое представляет собой *манипулятор* (*manipulator*). Ключевое свойство манипуляторов состоит в том, что запись манипулятора в поток воздействует на него определенным образом, а не просто записывает в него какие-то символы. Если левый операнд оператора “<<” имеет тип `std::ostream`, а правым операндом является манипулятор, то выполнение оператора “<<” заключается в совершении действия, заданного манипулятором, над заданным потоком, причем в качестве результата операции возвращается опять-таки поток. В случае использования манипулятора `std::endl` совершаемое действие состоит в завершении текущей строки выводимых данных.

Таким образом, полное выражение в качестве результата имеет значение `std::cout`, а побочными эффектами являются запись в стандартный выходной поток текста “Привет, мир!” и завершение выводимой строки. Завершая это выражение точкой с запятой, мы тем самым просим C++-среду не учитывать значение результата, поскольку нас интересуют здесь только побочные эффекты.

Область видимости (*scope*) имени — это часть программы, в которой это имя имеет значение. В C++ есть несколько различных видов областей видимости (с двумя из них мы успели познакомиться в этой программе).

Первой, используемой нами областью видимости является пространство имен, которое представляет собой коллекцию связанных имен. Стандартная библиотека определяет все свои имена в пространстве с именем `std`, что позволяет избежать конфликтов с именами, которые вы могли бы определить сами (если вам не придет в голову также определить “свое” пространство имен, используя имя `std`). При использовании любого имени из стандартной библиотеки необходимо указывать, что это имя взято из библиотеки. Например, имя `std::cout` означает, что имя `cout` определено в пространстве имен `std`.

Имя `std::cout` является *составным*, или *уточненным именем* (*qualified name*), использующим разрешающий оператор “::”, который также называется *оператором разрешения области видимости* (*scope operator*). Слева от оператора “::” находится (возможно, составное) имя области видимости (в нашем примере составного имени `std::cout` именем области видимости является `std`), а справа от оператора “::” — имя, определенное в области видимости, указанной слева. Итак, составное имя `std::cout` означает имя `cout`, которое определено в области видимости (пространстве имен) `std`.

Еще один вид области видимости образуется в результате использования фигурных скобок. Тело функции `main` — и тело любой функции — само является областью видимости. В такой маленькой программе этот факт не слишком интересен, но почти в каждой функции, которую нам предстоит еще написать, это весьма важно.

0.8. Резюме

Несмотря на простоту написанной нами программы, с ее помощью мы охватили ряд важных аспектов языка C++. Поэтому, прежде чем продолжить чтение книги, вам необходимо удостовериться в полном понимании материала этой главы.

Чтобы помочь вам в проведении своего рода самотестирования, эту главу — да и каждую последующую, за исключением главы 16, — мы завершаем разделом “Резюме” и набором упражнений. В разделах “Резюме” кратко излагаются основные идеи соответствующей главы, а подчас дается более широкое их толкование. Эти разделы можно использовать в качестве конспекта, позволяющего быстро освежить в памяти содержимое данной главы.

Структура программы. C++-программы обычно пишутся в *свободном формате* (free form). Это означает, что пробелы необходимы только там, где нужно предотвратить слитное написание смежных символов. В частности, символы новой строки (т.е. средство, используемое средой для перехода с одной строки на другую) представляют собой лишь еще один вид пробела и обычно не несут никакой дополнительной нагрузки. Вставка пробелов может как улучшить, так и ухудшить читабельность программы. Однако в следующих трех случаях свободный формат недопустим.

- **Строковые литералы.** Символы, заключенные в двойные кавычки. Не могут занимать несколько строк.
- **#include имя.** Эта директива должна занимать отдельную строку (возможен лишь комментарий).
- **// комментарии.** За символами “//” можно расположить любой текст комментария, который завершается с окончанием текущей строки.

Комментарий, который начинается парой символов “/*”, означает использование свободного формата; он завершается первой встретившейся парой символов “*/” и может занимать несколько строк.

Типы определяют структуры данных и операции, которые можно выполнять над этими структурами данных. В языке C++ предусмотрено два вида типов: встроенные в базовый язык (например, int) и определенные вне его (например, std::ostream).

Пространства имен обеспечивают механизм группирования связанных имен. Имена из стандартной библиотеки определяют в пространстве имен std.

Строковые литералы начинаются и оканчиваются двойными кавычками (“); каждый строковый литерал должен полностью помещаться на одной строке программы. Некоторые символы в строковых литералах имеют специальное значение, если им предшествует символ обратной косой черты (\).

\n	Символ новой строки
\t	Символ табуляции
\b	Знак возврата на одну позицию
\"	Этот символ обрабатывается как часть строки, а не как признак завершения строки
'	В строковых терминалах аналогично значению символа апострофа ('); используется для совместимости с символьными литералами (см. раздел 1.2)
\\	Включает в строку символ обратной косой черты (\), обрабатывая следующий символ как обычный

Подробнее о строковых литералах см. в разделах 10.2 и A.2.1.3.

Определения и заголовки. Каждое имя, используемое в C++-программе, должно иметь соответствующее определение. Стандартная библиотека определяет свои имена в заголовках, к которым программы получают доступ посредством директивы #include. Имена должны быть определены до их использования; следовательно, директива #include должна предшествовать использованию любого имени из этого заголовка. Заголовок <iostream> определяет библиотечные средства ввода-вывода.

Функция main. В каждой C++-программе должна быть определена ровно одна функция с именем `main`, которая возвращает `int`-значение. Программа запускается на выполнение путем вызова C++-средой функции `main`. Если функция `main` возвращает нуль, это означает, что программа выполнилась успешно; ненулевое значение свидетельствует о неудачном ее выполнении. В общем, функции должны включать, по крайней мере, одну инструкцию `return`, которую можно опустить, но в этом случае C++-среда предположит, что функция возвращает нулевое значение (в этом заключается ее особый статус). Однако хороший стиль программирования подразумевает явное включение в функцию `main` инструкции `return`.

Фигурные скобки и точки с запятой. Эти неприметные символы имеют большое значение в C++-программах. Их легко упустить по причине малого (физического) размера; они очень важны, поскольку их пропуск обычно приводит к получению от компилятора диагностических сообщений, которые трудно понять.

Некоторое количество (возможно, нулевое) инструкций, заключенных в фигурные скобки, представляет собой инструкцию, именуемую *блоком*, который является запросом на выполнение составляющих его инструкций в порядке их следования. Тело любой функции должно быть заключено в фигурные скобки, даже если оно состоит только из одной инструкции. Инструкции между соответствующими фигурными скобками образуют область видимости.

Выражение, предшествующее точке с запятой, представляет собой инструкцию, именуемую *инструкцией-выражением* (*expression statement*), которая является запросом на выполнение выражения ради его побочных эффектов с игнорированием результата. Наличие выражения необязательно, его можно опустить, и в этом случае мы получаем *пустую инструкцию* (*null statement*), у которой отсутствует результат.

Вывод данных. При вычислении выражения `std::cout << e` значение переменной `e` записывается в стандартный выходной поток, а результат становится равным значению `std::cout` типа `ostream`, которое позволяет выполнять сцепленные операции вывода данных.

Упражнения

- 0.1. Скомпилируйте и выполните программу, которая выводит текст Привет, мир!.
Что делает следующая инструкция?
`3 + 4;`
- 0.2. Напишите программу, которая выводит следующий текст.
это (") - кавычки, а это (\) - обратная косая черта.
- 0.3. Строковый литерал `"\t"` представляет символ табуляции; различные C++-среды отображают табуляцию по-разному. Чтобы узнать, как ваша среда обрабатывает табуляцию, проведите эксперимент, написав соответствующую программу.
- 0.4. Напишите программу, которая выводит текст программы, выводящей текст Привет, мир!.
- 0.5. Эта программа допустима? Почему?

```
#include <iostream>
int main() std::cout << "Привет, мир!" << std::endl;
```
- 0.6. А эта программа допустима? Почему?

```
#include <iostream>
int main() {
    std::cout << "Привет, мир!"
               << std::endl;
}
```

0.7. А что вы скажете об этой программе?

```
#include <iostream>
int main()
{
    /* Этот комментарий может занимать несколько строк, поскольку
    в качестве начального и конечного ограничителей он использует
    символы /* и */, */
    std::cout << "Этот вариант программы будет работать?"
               << std::endl;
    return 0;
}
```

0.8. А об этой?

```
#include <iostream>
int main()
{
    // Этот комментарий занимает несколько строк, благодаря
    // использованию символов // в начале каждой строки вместо
    // символов /* или */, служащих для ограничения многострочных
    // комментариев.
    std::cout << "Этот вариант программы будет работать?"
               << std::endl;
    return 0;
}
```

0.9. Приведите пример самой короткой допустимой программы?

0.10. Перепишите программу, выводящую текст Привет, мир!, чтобы вместо пробела в ней использовался символ новой строки.

РАБОТА СО СТРОКАМИ

В главе 0 подробно рассматривается крошечная программа, которую мы использовали для представления довольно многих фундаментальных понятий языка C++: комментариев, стандартных заголовков, областей действия, пространств имен, выражений, инструкций, строковых литералов и операторов вывода данных. В этой главе мы продолжаем рассмотрение основ языка путем написания простых программ, в которых используются символьные строки. Мы познакомимся с объявлениями, переменными и способами их инициализации, а также с вводом данных и C++-библиотечным классом `string`. Программы, представленные в этой главе, настолько просты, что для их выполнения не требуется никаких управляющих структур, о которых пойдет речь в главе 2.

1.1. Ввод данных

Коль мы умеем записывать (выводить) текст, логическим продолжением должна стать способность его прочитать (ввести). Например, мы можем превратить программу, выводящую текст Привет, мир!, в программу, которая приветствует конкретного человека.

```
// Запрашиваем имя человека и приветствуем его.
#include <iostream>
#include <string>
int main()
{
    // Запрашиваем имя человека.
    std::cout << "Введите, пожалуйста, свое имя: ";

    // Считываем введенное имя.
    std::string name;    // Определяем переменную name.
    std::cin >> name;    // Считываем данные в переменную name.

    // Выводим приветствие.
    std::cout << "Привет, " << name << "!" << std::endl;
    return 0;
}
```

При выполнении эта программа запишет текст

Введите, пожалуйста, свое имя:

в стандартный выходной поток. Если мы ответим, например, именем
Владимир,

то программа выведет приветствие в следующем виде.

Привет, Владимир!

Давайте разберемся в том, что здесь происходит. Чтобы прочитать вводимый текст, мы должны иметь место, в которое его можно поместить. Таким “местом” служит *переменная* (variable). Переменная — это *объект* (object), у которого есть имя. Объект, в свою очередь, представляет собой часть памяти компьютера, которая имеет тип. Важно понимать разницу между объектами и переменными, поскольку, как показано в разделах 3.2.2, 4.2.3 и 10.6.1, можно иметь объекты, у которых нет имен.

Если мы хотим использовать переменную, необходимо сообщить C++-среде ее имя и тип. Соблюдение требований, предъявляемых к поддержке как имени, так и типа переменных, позволяет среде сгенерировать эффективный машинный код для наших программ. Кроме того, соблюдение этих требований позволяет компилятору обнаружить орфографические ошибки в именах переменных — если, конечно, данная орфографическая ошибка не привела к совпадению с одним из имен, объявленных нашей программой.

В приведенном выше примере наша переменная называется `name` и имеет тип `std::string`. Как указано в разделах 0.5 и 0.7, использование префикса `std::` означает, что имя, `string`, которое стоит за ним, является частью стандартной библиотеки, а не частью базового языка или какой-нибудь нестандартной библиотеки. Как и при использовании любой части стандартной библиотеки, имя `std::string` имеет соответствующий заголовок, а именно заголовок `<string>`, поэтому мы и добавили в нашу программу соответствующую директиву `#include`.

С первой инструкцией программы

```
std::cout << "Введите, пожалуйста, свое имя: ";
```

вы должны быть уже знакомы. Она выводит сообщение, предлагающее пользователю ввести имя. Важная часть этой инструкции, а именно манипулятор `std::endl`, здесь отсутствует. Поскольку мы не использовали манипулятор `std::endl`, очередные выводимые данные (если таковые будут иметь место) не будут записаны с новой строки. После того как будет выведен запрос на ввод имени, компьютер перейдет в режим ожидания — на той же строке — вводимых данных.

Следующая инструкция

```
std::string name; // Определяем переменную name.
```

представляет собой *определение* (definition) нашей переменной с именем `name`, которая имеет тип `std::string`. Поскольку это определение находится внутри тела функции, переменная `name` является *локальной переменной* (local variable), которая существует только во время выполнения части программы, заключенной в фигурные скобки. Как только компьютер достигнет закрывающей фигурной скобки `}`, он *разрушит* (destroy) переменную `name` и возвратит системе память, которую она занимала. Ограниченное время существования локальных переменных — один из факторов, который весьма важен для проведения различия между переменными и другими объектами.

Неявно выраженным свойством типа объекта является его *интерфейс* (interface) — коллекция операций, которые возможны для объекта этого типа. Определяя `name` как переменную (именованный объект) типа `string`, мы неявно сообщаем среде, что желаем иметь возможность поступать с переменной `name` так, как (по “разрешению” библиотеки) мы можем поступать со строками, т.е. с переменными типа `string`.

Одной из таких операций является *инициализация* (initialization) `string`-переменной. При определении `string`-переменной неявно инициализируется, поскольку, согласно стандартной библиотеке, каждый `string`-объект начинает свое существование с некоторым значением. Вскоре вы увидите, что, создавая `string`-переменную, мы можем предос-

тавить ей собственное значение. Если мы этого не сделаем, “новорожденная” string-переменная начнет свое существование “голышом”, т.е. без символов. Такой string-объект мы называем *пустой* (empty) строкой, или *нуль-строкой* (null string).

Определив переменную name, мы выполняем следующую инструкцию.

```
std::cin >> name; // Считываем данные в переменную name.
```

Эта инструкция считывает данные из объекта std::cin в переменную name. По аналогии с оператором “<<” и операндом std::cout для вывода данных, библиотека использует оператор “>>” и операнд std::cin для ввода данных. В этом примере при выполнении операции “>>” из стандартного входного потока считываются string-данные, а результат чтения записывается в объект с именем name. Когда мы хотим с помощью библиотечных средств прочитать строку, чтение данных из потока начнется с отбрасывания пробелов и символов, подобных пробелу (табуляции, возврата на одну позицию или конца строки), затем в переменную name будут считываться символы до тех пор, пока не встретится пробел (либо символ, подобный пробелу) или символ, служащий признаком конца файла. Следовательно, результатом выполнения инструкции std::cin >> name является чтение слова из стандартного входного потока и сохранение в переменной name символов, которые составляют это слово.

Операция ввода данных имеет еще один побочный эффект: она заставляет приглашение на ввод имени отобразиться на выходном устройстве (устройстве вывода данных) компьютера. Необходимо отметить, что библиотечные средства ввода-вывода сохраняют выводимые данные во внутренней структуре данных, именуемой *буфером* (buffer), который используется для оптимизации операций вывода. В большинстве систем на запись символов в выходное устройство тратится значительное время, независимо от того, сколько символов подлежит выводу. Чтобы избежать затрат системных ресурсов в ответ на *каждый* запрос записи, библиотека использует буфер для накопления символов, которые должны быть записаны, и сбрасывает (т.е. записывает) его содержимое на выходное устройство только при необходимости. При таком подходе несколько операций вывода данных можно объединить в одну.

Существует несколько причин, которые заставляют систему “опорожнять” буфер. Во-первых, буфер может заполниться, и тогда библиотека перепишет его содержимое автоматически. Во-вторых, библиотека может получить запрос на чтение из стандартного входного потока. В этом случае библиотека немедленно переписывает содержимое выходного буфера, не ожидая, пока он заполнится до отказа. В-третьих, “сбросить” буфер в любой момент времени можно по соответствующей команде.

Когда наша программа запишет приглашение на ввод данных в поток cout, оно попадет в буфер, связанный со стандартным выходным потоком. Затем мы попытаемся прочитать данные с устройства cin. Именно эта операция чтения заставит переписать из cout-буфера его содержимое, поэтому мы твердо уверены в том, что пользователь обязательно увидит приглашение ввести данные.

Следующая инструкция, которая генерирует выходные данные, в явной форме указывает библиотеке сбросить содержимое буфера. Эта инструкция лишь чуть-чуть сложнее той, которая выводит приглашение для пользователя. Итак, мы пишем строковый литерал “Привет, ”, за ним — значение string-переменной name и, наконец, манипулятор std::endl. Запись значения std::endl завершает строку выводимых данных, а затем сбрасывает буфер, что вынуждает систему немедленно записать данные в выходной поток.

Очень важно организовать своевременную передачу содержимого выходных буферов в программах, выполнение которых требует значительного времени. В противном случае некоторые выходные данные могут надолго “застыть” в буферах системы.

1.2. Выделение текста с помощью рамки

До сих пор наши программы ограничивались простым выводом приветствий. Теперь можно поработать над оформлением приветствия, представив вводимые и выводимые данные, например, в таком виде.

```
Введите, пожалуйста, свое имя: Владимир
*****
*                               *
*   привет, владимир!   *
*                               *
*****
```

Как видите, наша программа должна вывести пять строк текста. Первая строка образует верхнюю границу рамки, которая состоит из ряда символов “звездочка” (*), причем количество символов “*” определяется длиной имени приветствуемого пользователя, сложенной с количеством символов в слове “привет, ” (с запятой и пробелом), а также с числом окаймляющих символов “*” (2) и пробелов (2) для каждой боковой стороны рамки. Вторая строка должна иметь соответствующее число пробелов с двумя символами “*” с каждой боковой стороны. Третья строка включает символ “*”, пробел, приветственное сообщение, пробел и символ “*”. Последние две строки повторяют вторую и первую строки соответственно.

Итак, стратегия решения этой задачи состоит в следующем. Сначала мы должны прочитать имя, использовать его для построения фразы приветствия, а затем применить фразу приветствия для построения каждой выводимой строки. Ниже приведена программа, которая использует описанную выше стратегию для решения нашей проблемы.

```
// Запрашиваем имя пользователя и
// генерируем приветствие в рамочке.
#include <iostream>
#include <string>
int main()
{
    std::cout << "Введите, пожалуйста, свое имя: ";
    std::string name;
    std::cin >> name;

    // Создаем сообщение, подлежащее выводу.
    const std::string greeting = "привет, " + name + "!";

    // Создаем вторую и четвертую строки приветствия.
    const std::string spaces(greeting.size(), ' ');
    const std::string second = "* " + spaces + " *";

    // Создаем первую и пятую строки приветствия.
    const std::string first(second.size(), '*');

    // Выводим все строки.
    std::cout << std::endl;
    std::cout << first << std::endl;
    std::cout << second << std::endl;
    std::cout << " * " << greeting << " * " << std::endl;
    std::cout << second << std::endl;
    std::cout << first << std::endl;

    return 0;
}
```

Прежде всего, наша программа запрашивает имя пользователя, потом читает его, сохраняя в переменной `name`. После этого программа определяет переменную с име-

нем `greeting`, которая содержит сообщение, подлежащее выводу. Затем она определяет переменную с именем `spaces`, содержащую столько пробелов, сколько содержится символов в переменной `greeting`. Переменная `spaces` используется для определения переменной `second`, которая будет содержать вторую строку “рамочного” приветствия. Наконец, программа создает переменную `first`, которая будет содержать столько символов “*”, сколько содержится в переменной `second`. Создав все необходимые переменные, программа построчно выводит их содержимое с соответствующими дополнениями.

Директивы `#include` и первые три инструкции в этой программе должны быть вам знакомы. Но определение переменной `greeting` обязывает нас представить три нововведения.

Первое состоит в том, что мы придали переменной значение одновременно с ее определением. Нам удалось это сделать, разместив между именем переменной и точкой с запятой символ “знак равенства” (=) с последующим значением, которое предназначается для этой переменной. Если переменная и присваиваемое ей значение имеют разные типы (как показано в разделе 10.2, `string`-переменные и строковые литералы имеют такие разные типы), C++-среда должна *преобразовать* (`convert`) это начальное значение в значение типа переменной.

Второе нововведение заключается в том, что мы можем использовать знак “плюс” (+) для *конкатенации*, т.е. сцепления (`concatenation`) `string`-переменной и строкового литерала или двух строк (но не двух строковых литералов). В главе 0 мы заметили, что 3+4 равно 7. Здесь же мы приводим пример, в котором знак “+” означает совершенную другую операцию. В каждом случае мы можем сказать, что его действия определяются в зависимости от типов операндов. Если некоторый оператор по-разному интерпретируется для операндов различных типов, мы говорим, что этот оператор *перегружен* (`overloaded`).

Третья новинка — использование слова `const` как части определения переменной. Это означает, что мы не собираемся изменять значение переменной в течение ее времени существования. Строго говоря, от использования слова `const` в этой программе никакой пользы не извлекается. Но одно лишь указание на то, значения каких переменных не будут изменяться, может сделать программу гораздо понятнее.

Обратите внимание на то, что если с помощью слова `const` мы объявляем переменную константной, необходимо тут же ее инициализировать, поскольку позже у нас такой возможности уже не будет. Заметьте также, что значение, которое мы используем для инициализации `const`-переменной, необязательно должно быть константой. В этом примере мы не знаем значения переменной `greeting` до тех пор, пока программа не прочтает введенное пользователем значение и не запомнит его в переменной `name`, что, очевидно, не может произойти, пока мы не выполним программу. Поэтому мы не можем сказать, что переменная `name` является `const`-переменной, поскольку изменяем ее значение, когда сохраняем в ней вводимые данные.

Одним из свойств оператора, который никогда не меняет своих действий, является его ассоциативность. В главе 0 мы узнали, что оператор вывода (<<) является левоассоциативным, поэтому инструкция `std::cout << s << t` означает то же самое, что `(std::cout << s) << t`. Аналогично оператор сложения “+” (и, коли на то пошло, оператор ввода “>>”) также левоассоциативный. Соответственно, значение выражения “Привет, ” + `name` + “!” равно результату первой конкатенации (строкового литерала “Привет, ” со значением переменной `name`), конкатенированному с символом “!” (т.е. конечному результату второй

конкатенации). Таким образом, если, например, переменная `name` содержит имя Владимир, то значение выражения "привет, " + `name` + "!" равно привет, Владимир!.

Итак, мы собрали воедино все, что собирались сказать, и сохранили эту информацию в переменной `greeting`. Следующий шаг — построение рамки, в которую будет заключено наше приветствие. Для этого рассмотрим еще три новинки, причем в одной-единственной инструкции.

```
std::string spaces(greeting.size(), ' ');
```

При определении переменной `greeting` мы использовали для ее инициализации символ `"="`. Здесь же за именем переменной `spaces` стоят два выражения, которые разделены запятой и заключены в круглые скобки. С помощью символа `"="` мы явно указываем, какое значение должна иметь определяемая нами переменная. Используя в определении круглые скобки (как в приведенной выше инструкции), мы велит C++-среде создать переменную (в данном случае переменную `spaces`) на основе указанных в скобках выражений, причем способ создания зависит от типа переменной. Другими словами, чтобы уяснить это определение, мы должны понять, что означает создать строку из двух выражений.

Как, возможно, удивитесь вы, неужели создание переменной полностью зависит от ее типа? В данном частном случае мы создаем строку из... из чего же мы ее создаем? Оба выражения записаны в форме, с которой мы еще не встречались. Что же они означают?

Первое выражение, `greeting.size()`, — это пример вызова функции-члена. На самом деле объект с именем `greeting` имеет компонент с именем `size`, который является функцией, которую, следовательно, мы можем вызвать, чтобы получить некоторое значение. Переменная `greeting` имеет тип `std::string`, который определен таким образом, что при вычислении выражения `greeting.size()` генерируется целое значение, представляющее количество символов в переменной `greeting`.

Второе выражение, `' '`, представляет собой символьный литерал. Символьные литералы совершенно отличаются от строковых. Символьный литерал всегда заключен в одинарные кавычки, а строковый — в двойные. Тип символьного литерала — это встроенный тип `char`, а тип строкового литерала гораздо сложнее, но мы не будем останавливаться на его разъяснении до раздела 10.2. Символьный литерал представляет одинарный символ. Символы со специальным значением внутри строкового литерала имеют специальное значение и в символьном литерале. Следовательно, если мы хотим задать символ "одинарная кавычка" (`'`) или "обратная косая черта" (`\`), необходимо предварить его обратной косой чертой (`\`). Специальные символы `'\n'`, `'\t'`, `'\"'` и связанные с ними формы аналогичны тем, которые применяются для строковых литералов (см. главу 0).

Чтобы покончить с разъяснением создания переменной `spaces`, необходимо отметить, что при построении `string`-переменной из целого значения и `char`-значения результат имеет столько копий этого `char`-значения, сколько задано целым значением. Так, например, после выполнения инструкции

```
std::string stars(10, '*');
```

выражение `stars.size()` будет равно числу 10, а сама переменная `stars` будет содержать десять символов "звездочка" (`*****`).

Следовательно, переменная `spaces` содержит столько символов, сколько их имеет в переменной `greeting`, но все символы переменной `spaces` являются пробелами.

Чтобы понять определение переменной `second`, никаких новых знаний не требуется. Для получения второй строки нашего приветствия в рамочке, мы конкатенируем

строковый литерал `"* "`, нашу строку пробелов (spaces) и строковый литерал `"*"`. Для определения переменной `first` также достаточно старых (теперь уже) знаний; переменная `first` приобретает значение, которое состоит из символов "звездочка" (*), количество которых равно количеству символов в переменной `second`.

С остальной частью программы вы должны быть знакомы: здесь выполняется запись строк в выходной поток, как описано в разделе 1.1.

1.3. Резюме

Типы

<code>char</code>	Встроенный тип, предназначенный для хранения одинарных символов, определенных C++-средой
<code>wchar_t</code>	Встроенный тип, предназначенный для хранения "широких" символов, например символов таких языков, как японского

Тип `string` определяется в стандартном заголовке `<string>`. Объект типа `string` содержит ряд символов, количество которых может быть даже нулевым. Если `n` — целое число, `s` — `char`-переменная, `is` — входной поток и `os` — выходной поток, то к `string`-операциям относятся следующие.

<code>std::string s;</code>	Определяет <code>s</code> как переменную типа <code>std::string</code> , которая изначально пуста
<code>std::string t = s;</code>	Определяет <code>t</code> как переменную типа <code>std::string</code> , которая изначально содержит копию символов, содержащихся в <code>s</code> , где <code>s</code> — любая <code>string</code> -переменная или строковый литерал
<code>std::string z(n, c);</code>	Определяет <code>z</code> как переменную типа <code>std::string</code> , которая изначально содержит <code>n</code> копий символа <code>c</code> . При этом <code>c</code> должен иметь тип <code>char</code> и не может быть <code>string</code> -переменной и строковым литералом
<code>os << s</code>	Записывает символы, содержащиеся в <code>s</code> (без каких-либо изменений в форматировании), в выходной поток, обозначенный <code>os</code> . Результатом выражения является <code>os</code>
<code>is >> s</code>	Считывает и отбрасывает символы из потока, обозначенного именем <code>is</code> , до тех пор, пока не встретится символ, не являющийся пробелом или символом, подобным пробелу. Затем считывает последовательные символы из потока <code>is</code> в переменную <code>s</code> (перезаписывая значение, которое она могла иметь) до тех пор, пока очередной прочитанный символ не окажется пробелом или подобным ему. Результатом выражения является <code>is</code>
<code>s + t</code>	Результат этого выражения имеет тип <code>std::string</code> и содержит копию символов, содержащихся в <code>s</code> , за которыми располагается копия символов, содержащихся в <code>t</code> . Либо <code>s</code> , либо <code>t</code> (но не оба сразу) может быть строковым литералом или значением типа <code>char</code>
<code>s.size()</code>	Количество символов в <code>s</code>

Переменные можно определить одним из трех способов.

```
std::string hello = "привет"; // Определяем переменную с явно
                               // заданным начальным значением.
std::string stars(100, '*');  // Создаем переменную в соответствии
                               // с типом и заданными выражениями.
```

```
std::string name;           // Определяем переменную с неявно
                             // заданным начальным значением,
                             // которое зависит от ее типа.
```

Переменные, определенные в фигурных скобках, являются локальными переменными, которые существуют только во время выполнения части программы внутри этих фигурных скобок. При достижении закрывающей фигурной скобки (}) локальные переменные разрушаются, а память, занимаемая ими, возвращается системе.

Определение переменной с помощью слова `const` гарантирует, что значение этой переменной не будет меняться в течение времени ее существования. Такая переменная должна быть инициализирована одновременно с ее определением, поскольку позже такой возможности не существует.

Ввод данных. При выполнении инструкции `std::cin >> v` из стандартного входного потока считываются и игнорируются пробелы и любые символы, подобные пробелу, затем отсюда же считываются символы, отличные от пробелов, и записываются в переменную `v`. Инструкция возвращает значение `std::cin`, которое имеет тип `istream`, позволяющий выполнить сцепленные операции ввода данных.

Упражнения

1.0. Скомпилируйте, выполните и протестируйте программы, представленные в этой главе.

1.1. Допустимы ли следующие определения? Почему?

```
const std::string hello = "Привет";
const std::string message = hello + ", мир" + "!";
```

1.2. Допустимы ли такие определения? Почему?

```
const std::string exclam = "!";
const std::string message = "Привет" + ", мир" + exclam;
```

1.3. Допустима ли следующая программа? Если да, то что она делает? Если нет, то почему?

```
#include <iostream>
#include <string>

int main()
{
    { const std::string s = "одна строка";
      std::cout << s << std::endl; }

    { const std::string s = "другая строка";
      std::cout << s << std::endl; }
    return 0;
}
```

1.4. Что можно сказать об этой программе? Что произойдет, если в третьей строке с конца две фигурные скобки `}}` заменить `};}`?

```
#include <iostream>
#include <string>

int main()
{
    { const std::string s = "одна строка";
      std::cout << s << std::endl;
    { const std::string s = "другая строка";
      std::cout << s << std::endl; }
}
```

```
    return 0;
}
```

1.5. Допустима ли такая программа? Если да, то что она делает? Если нет, то ответьте, почему, и предложите допустимый вариант.

```
#include <iostream>
#include <string>

int main()
{
    { std::string s = "одна строка";
      { std::string x = s + ", действительно";
        std::cout << s << std::endl; }
        std::cout << x << std::endl;
      }
    return 0;
}
```

1.6. Что делает следующая программа, если при запросе ввести данные вы вводите два имени (например, чарли чаплин). Предскажите поведение программы до ее выполнения, а затем проверьте свои предсказания.

```
#include <iostream>
#include <string>

int main()
{
    std::cout << "Как вас зовут? ";
    std::string name;
    std::cin >> name;
    std::cout << "Привет, " << name
              << std::endl << "А как вас зовут? ";
    std::cin >> name;
    std::cout << "Привет, " << name
              << "; с вами также было приятно познакомиться!"
              << std::endl;
    return 0;
}
```

ОРГАНИЗАЦИЯ ЦИКЛОВ И ВЫЧИСЛЕНИЙ

В разделе 1.2 мы разработали программу, которая выводит форматированную рамочку вокруг приветствия. В этой главе мы напишем более гибкую программу, которая позволит менять размер рамочки, не переписывая самой программы. По ходу дела мы изучим некоторые арифметические операторы и узнаем, как C++ поддерживает циклы и условия, а также ознакомимся с понятием инварианта цикла.

2.1. В чем суть проблемы

Программа, приведенная в разделе 1.2, выводит приветствие, заключенное в рамочку. Например, если пользователь введет имя Владимир, программа выведет приветствие в следующем виде.

```
*****  
*  
* Привет, Владимир! *  
*  
*****
```

В этой программе подготовленные данные выводились построчно. Но прежде были определены переменные `first` и `second`, содержащие наборы символов для первой и второй строк, а третья строка содержала само приветствие, окаймленное парами симметрично расположенных символов (пробел и “звездочка”). У нас не было необходимости создавать отдельные переменные для четвертой и пятой строк, поскольку они в точности совпадают со второй и первой строками соответственно.

Такой подход к построению программы имеет существенный недостаток: у каждой строки выводимых данных есть “своя” часть программы (и переменная), которая ей соответствует. Следовательно, даже простое изменение в формате вывода, например удаление пробелов между приветствием и рамкой, потребует переделки программы. Мы хотели бы создать более гибкую форму вывода данных, в которой не нужно сохранять каждую строку в локальной переменной.

В этой главе попробуем решить ту же задачу по-другому, генерируя каждый символ в отдельности, за исключением самого приветствия, которое уже имеется в нашем распоряжении в виде `string`-переменной. Ведь совсем необязательно хранить выводимые символы в переменных, поскольку выведенный символ нам больше не нужен.

2.2. Общая структура программы

Пожалуй, начнем с той части программы, которую нам не нужно переписывать.

```
#include <iostream>
#include <string>
int main()
{
    // Запрашиваем имя человека.
    std::cout << "Введите, пожалуйста, свое имя: ";

    // Читаем введенное имя.
    std::string name;
    std::cin >> name;

    // Создаем сообщение, подлежащее выводу.
    const std::string greeting = "Привет, " + name + "!";

    // А эту часть программы нам нужно переделать...
    return 0;
}
```

Если переписывать только ту часть программы, которая отмечена комментарием *// А эту часть программы нам нужно переделать...*, то мы знаем, что переменные `name`, `greeting` и другие нужные нам имена стандартной библиотеки уже определены. Новую версию программы мы будем строить по “фрагментам”, а затем, в разделе 2.5.4, соберем отдельные фрагменты в единое программное “сооружение”.

2.3. Вывод неизвестного числа строк

Мы можем представить свои выходные данные в виде прямоугольного массива символов, который необходимо вывести строка за строкой. И хотя мы заранее не знаем, сколько строк нужно вывести, нам известно, как вычислить количество строк, подлежащих выводу.

Само приветствие занимает одну строку, и также по одной строке приходится на верхнюю и нижнюю границы рамки. Итак, пока мы насчитали три строки. Если нам будет известно, сколько пустых (с пробелами) строк будет отделять приветствие от каждой из горизонтальных границ рамки, мы сможем удвоить это число и сложить результат с числом три, получив тем самым общее количество строк в наборе выходных данных.

```
// Количество пустых строк между приветствием и одной из
// горизонтальных границ рамки.
const int pad = 1;

// Общее количество выводимых строк.
const int rows = pad * 2 + 3;
```

Чтобы легче отыскать в программе место, в котором определяется количество пустых строк, мы решили прибегнуть к именам и этому количеству присвоили имя `pad`. Определив переменную `pad`, мы используем ее в вычислении значения переменной `rows`, которая будет хранить общее количество выводимых строк.

Встроенный тип `int` — наиболее естественный тип для работы с целыми числами, поэтому мы и выбрали его при определении переменных `pad` и `rows`. Мы также указали, что обе эти переменные являются константными (`const`-переменными); это, как мы узнали в разделе 1.2, связывает их “клятвой” ни при каких обстоятельствах не менять своих значений.

Если мы решили использовать одинаковое количество пробелов слева и справа от приглашения, равно как сверху и снизу, то для всех четырех сторон достаточно одной переменной `pad`. Если применять эту переменную при каждом обращении к количеству “околорабочных” пробелов, то для изменения размера рамки (точнее, ее “воздушного” пространства) понадобится внести в программу только одно изменение, присвоив этой переменной другое значение. Итак, мы вычислили, сколько строк нам нужно вывести. Теперь этот вывод осталось реализовать в виде программного кода.

```
// Отделяем вывод данных от ввода.
std::cout << std::endl;

// Выводим rows строк.
int r = 0;

// Инвариант: пока мы вывели r строк.
while (r != rows) {
    // Записываем строку выходных данных (как будет описано
    // в разделе 2.4).
    std::cout << std::endl;
    ++r;
}
```

Мы, как и в разделе 1.2, начали с вывода пустой строки, чтобы между вводимыми и выводимыми данными было некоторое пространство. Остальная часть этого фрагмента содержит много нововведений, на которых стоит остановиться подробнее. Если мы поймем, как работает эта часть, подумаем о том, как вывести каждую отдельную строку.

2.3.1. Понятие о *while*-инструкции

В нашей программе управление количеством выводимых строк организовано с помощью инструкции *while*. Данная инструкция повторяет выполнение заданной в ее теле инструкции до тех пор, пока истинно заданное условие. Инструкция *while* имеет следующую форму записи.

```
while (Условие)
    Инструкция
```

Элемент *инструкция* часто называют *телом while-цикла*.

Выполнение *while*-инструкции начинается с тестирования значения элемента *Условие*. Если условие ложно, тело цикла не выполняется вовсе. В противном случае оно выполняется один раз, после чего элемент *Условие* тестируется снова, и т.д. Инструкция *while* “мечется” между тестированием условия и выполнением тела цикла до тех пор, пока проверяемое условие не станет ложным, после чего выполнение программы продолжается уже за пределами *while*-инструкции.

Возвращаясь к нашему примеру, мы можем так “озвучить” действия *while*-инструкции: “до тех пор, пока значение переменной *r* не станет равным значению *rows*, выполняйте все, что находится внутри моих фигурных скобок `{}`”.

Чтобы программу было легче читать, удобно поместить *while*-тело в отдельную строку и сделать отступ от левого края. C++-среда не прекратит с нами сотрудничество, если мы запишем *while*-инструкцию следующим образом.

```
while (Условие) Инструкция
```

Но в этом случае мы должны подумать о том, имеем ли мы право усложнять жизнь другим программистам, которым, возможно, придется читать нашу программу.

Обратите внимание на то, что в описании синтаксиса после элемента *инструкция* нет точки с запятой. Элемент *инструкция* является либо обычной инструкцией, либо *блоком*, который представляет собой последовательность инструкций (количество которых может быть даже нулевым), заключенных в фигурные скобки (`{}`). Если элемент *инструкция* — обычная инструкция, то она завершится собственной точкой с запятой, и поэтому в записи еще одной точки с запятой нет никакой необходимости. Если мы имеем дело с блоком, его закрывающая фигурная скобка `}` отмечает конец элемента *инструкция*, т.е. снова-таки никакой дополнительной точки с запятой ставить не нужно. Поскольку блок представляет собой последовательность инструкций, заключенных в фигурные скобки, то, как мы знаем из раздела 0.7, он образует некоторую область видимости.

Как уже упоминалось, инструкция `while` начинает свою работу с тестирования условия (*condition*), которое представляет собой выражение, построенное в контексте, когда требуется получить значение истинности. Примером такого условия является выражение `г != rows`. В этом примере при сравнении переменных `г` и `rows` используется оператор неравенства `"!="`. Такое выражение имеет тип `bool`, который является встроенным типом и служит для представления истинных и ложных значений, обозначаемых `true` и `false` соответственно.

Еще одной новинкой программы является последняя инструкция в теле `while`-цикла.

```
++г;
```

Знак “двойной плюс” (`++`) служит для обозначения оператора *инкремента* (*increment*), действие которого заключается в сложении значения заданной переменной (в данном случае переменной `г`) с единицей (1). Вместо инструкции `++г`; мы могли бы записать следующую

```
г = г + 1;
```

но инкрементирование объекта (увеличение значения на 1) — настолько популярная операция, что оказалось полезным применить для нее специальное обозначение. Более того, как будет показано в разделе 5.1.2, идея преобразования значения в *непосредственно следующее за данным* (*immediate successor*), в отличие от вычисления произвольного значения, настолько существенна для структур абстрактных данных, что вполне заслуживает специального обозначения уже только по одной этой причине.

2.3.2. Разработка `while`-инструкции

Иногда трудно определить, какое в точности условие следует записать в `while`-инструкции. И точно также бывает трудно понять, что именно делает та или иная `while`-инструкция. Однако в конкретном примере несложно догадаться, что инструкция `while` в разделе 2.3 должна вывести некоторое количество строк, которое зависит от значения переменной `rows`. Но как заранее определить, сколько строк будет выводить программа? Например, как узнать, будет ли это число в точности равно значению `rows`, а может быть, `rows - 1`, `rows + 1` или чему-то еще? Мы могли бы “вручную” выполнить работу инструкции `while`, запоминая, как отражается результат выполнения каждой инструкции на состоянии программы, но как узнать, что где-то была допущена ошибка?

Существует проверенный метод построения и понимания `while`-инструкций, который опирается на две ключевые идеи: определение `while`-инструкции и общее поведение программ.

Первая идея состоит в том, что по завершении `while`-инструкции ее условие должно быть ложным — в противном случае она бы не завершилась. Поэтому, когда `while`-инструкция, приведенная в разделе 2.3, оканчивается, мы знаем, что значение выражения `r != rows` ложно и, следовательно, значение `r` равно значению `rows`.

Вторая идея заключается в том, что *инвариант цикла* (loop invariant), т.е. некоторое свойство, или утверждение должно быть истинным каждый раз, когда предполагается тестирование условия `while`-инструкции. Мы выбираем такой инвариант, который, по нашему мнению, вполне подходит, чтобы убедить самих себя в том, что программа ведет себя ожидаемым образом; мы и программу пишем так, чтобы сделать инвариант истинным в соответствующие моменты времени. Хотя инвариант не является частью текста программы, это — ценный интеллектуальный инструмент разработки программ. С каждой `while`-инструкцией, которую мы можем представить, можно связать некоторый инвариант. Описание инварианта в комментарии может сделать `while`-инструкцию более понятной.

Чтобы сделать наше обсуждение более конкретным, снова рассмотрим `while`-инструкцию, приведенную в разделе 2.3, и приведем комментарий, расположенный непосредственно перед инструкцией `while`.

```
// инвариант: пока мы вывели r строк.
```

Чтобы убедиться в том, что этот инвариант корректен для данного фрагмента программы, мы должны удостовериться, что он будет истинен каждый раз, когда предполагается тестирование условия `while`-инструкции. Это значит, что мы должны убедиться в истинности инварианта в двух конкретных местах программы.

Первое место находится как раз перед тем, как `while`-инструкция проверяет свое условие в первый раз. В этом месте нашего примера протестировать инвариант не составляет труда: поскольку мы не вывели пока еще ни одной строки, очевидно, что установка переменной `r`, равной значению 0, делает наш инвариант истинным.

Второе место находится как раз перед моментом достижения конца тела `while`-инструкции. Если инвариант будет “там” истинным, он будет истинным каждый раз, когда `while`-инструкция приступает к тестированию своего условия. Следовательно, инвариант будет истинным каждый раз.

Вместо того чтобы писать программу, отвечающую этим двум требованиям (вернее, требованию истинности инварианта в двух моментах: до начала работы `while`-инструкции и перед концом `while`-тела), мы должны быть уверены в том, что инвариант истинен не только каждый раз, когда `while`-инструкция тестирует условие, но также после того, как она завершится. В противном случае инвариант был бы истинным в начале одной из итераций `while`-тела и ложным впоследствии.

Теперь подытожим все, что мы знаем, в нашем фрагменте программы.

```
// инвариант: пока мы вывели r строк.
int r = 0;
// установка переменной r равной 0 делает инвариант истинным.
while (r != rows) {
    // мы можем допустить, что инвариант истинен здесь.

    // вывод строки делает инвариант ложным.
    std::cout << std::endl;
    // инкрементирование r делает инвариант истинным снова.
    ++r;
}
// мы можем заключить, что инвариант здесь истинен.
```


Инвариантом для нашей `while`-инструкции является утверждение, что мы вывели пока `r` строк. При определении переменной `r` мы присваиваем ей начальное значение 0. На этом этапе мы не вывели еще ни одной строки. Установка переменной `r` равной 0, бесспорно, делает инвариант истинным, поэтому мы выполнили первое требование.

Чтобы выполнить второе требование, мы должны убедиться в том, что инвариант будет истинным всегда, когда инструкция `while` “захочет” протестировать свое условие, т.е. “прогулка” вдоль условия и тела не повлияет на инвариант, оставив его истинным в конце тела.

Вывод строки заставляет инвариант стать ложным, поскольку переменная `r` больше не соответствует количеству строк, которое мы вывели. Но инкрементирование переменной `r`, учитывающее строку, которая была выведена, сделает инвариант истинным снова. Таким образом, к концу тела цикла инвариант опять становится истинным, поэтому мы удовлетворили и второе требование.

Поскольку оба требования удовлетворены, мы знаем, что после завершения `while`-инструкции мы выведем `r` строк. Более того, мы уже поняли, что `r == gows`. Оба эти факта свидетельствуют о том, что `gows` содержит общее количество выведенных программой строк.

Стратегия, которую мы использовали для понимания этого цикла, пригодится в различных контекстах. Общая идея — найти инвариант, который выражает некоторое важное свойство переменных, занятых в цикле (в нашем примере мы выводили `r` строк), и использовать условие, гарантирующее, что по завершении цикла эти переменные будут иметь некоторые значения (`r == gows`). В результате задача тела цикла — манипулировать соответствующими переменными, чтобы сделать в конечном счете условие ложным, поддерживая при этом истинность инварианта.

2.4. Вывод строки

Теперь, когда мы понимаем, как вывести заданное количество строк, можно переключить свое внимание на вывод одной строки. Другими словами, мы можем заняться реализацией той части программы, которая в предыдущем фрагменте отмечена следующим комментарием.

```
// Вывод строки делает инвариант ложным.
```

Начнем с того, что все выводимые строки имеют одинаковую длину. Если мы представляем выводимые данные в виде прямоугольного массива, то эта длина равна количеству столбцов в массиве. Мы можем вычислить это количество, прибавив к длине приветствия удвоенное значение “околорабочных” пробелов и число два (количество символов “звездочка”, используемых для “изготовления” самой рамочки).

```
const std::string::size_type cols = greeting.size() + pad * 2 + 2;
```

Обратите внимание на то, что переменная `cols` определяется здесь с использованием спецификатора `const`, “обещающего”, что ее значение не будет изменяться после такого определения. С такими “обещаниями” вы уже знакомы. Теперь же вам предстоит познакомиться еще с одним типом, с которым вам до сих пор не приходилось встречаться, а именно с типом `std::string::size_type`. Мы знаем, что первый символ “`::`” означает оператор разрешения области действия, а составное имя `std::string` — имя `string` из пространства имен `std`. Второй символ “`::`” говорит о том, что мы имеем в виду имя `size_type` из класса `string`. Подобно пространствам имен и блокам, классы определяют собственные области действия. Тип `std::string` определяет `size_type` как имя типа,

предназначенного для хранения числа символов в строке. Если нам когда-либо понадобится локальная переменная для хранения размера строки, мы должны использовать для этой переменной именно тип `std::string::size_type`.

Причина использования для переменной `cols` типа `std::string::size_type` заключается в гарантии того, что переменная `cols` сможет хранить количество символов в переменной `greeting`, каким бы большим оно ни оказалось. Скорее всего, если бы мы объявили переменную `cols` с использованием привычного типа `int`, такое решение не отразилось бы на работе программы. Но значение переменной `cols` зависит от размера данных, вводимых в нашу программу, а мы не можем управлять объемом вводимых данных. Ведь совсем не исключено, что кто-нибудь (ну, просто из озорства) введет такую длинную-предлинную строку, что тип `int` окажется недостаточным для хранения ее длины.

Тип `int` вполне подходит для переменной `rows`, поскольку количество строк зависит только от значения переменной `rad`, которое у нас под контролем. Каждая C++-среда должна обеспечить возможность любой `int`-переменной принимать значения, не превышающие число 32767, которое не такое уж и маленькое. Тем не менее, определяя переменную, которая должна содержать размер некоторой структуры данных, имеет смысл использовать тип, определяемый библиотекой как наиболее соответствующий такому узкому назначению.

Строка не может содержать отрицательное количество символов. Следовательно, тип `std::string::size_type` — это *тип без знака* (unsigned type), и объекты этого типа неспособны содержать отрицательные значения. Это свойство никак не влияет на программы, приведенные в данной главе, но, как мы увидим в разделе 8.1.3, оно может быть крайне важным.

Вычислив, сколько символов нужно вывести, мы можем использовать еще одну инструкцию `while` для их вывода.

```
std::string::size_type c = 0;
```

```
// инвариант: пока мы вывели c символов текущей строки.  
while (c != cols) {  
    // выводим один или несколько символов.  
    // изменяем значение c для поддержки истинности инварианта.  
}
```

Эта инструкция `while` ведет себя аналогично инструкции, приведенной в разделе 2.3, за исключением одного нюанса в ее теле: на этот раз мы решили вывести *один или несколько символов* вместо вывода ровно одной строки, как отмечено в разделе 2.3. В данном случае не существует причины, по которой мы обязаны организовать *посимвольный* вывод. Но если мы выведем хотя бы один символ, сделаем шаг вперед на пути к нашей цели. Вот что мы действительно должны сделать — так это гарантировать, что общее количество выведенных нами символов в этой строке в точности равно значению переменной `cols`.

2.4.1. Вывод символов рамки

Нам осталось решить, какие же вывести символы. Мы решим часть этой проблемы, если будем помнить о том, что при выводе первой или последней строки либо первого или последнего столбца нужно вывести символ “звездочка”. Более того, чтобы определить “момент” для вывода символа “звездочка”, мы можем применить свои познания в области инвариантов цикла.

Например, если значение переменной `r` равно нулю, то из заявленного инварианта следует, что мы не вывели еще ни одной строки, а это означает, что мы выводим

часть первой строки. Аналогично, если значение `r` равно значению `rows - 1`, мы знаем, что уже выведено `rows - 1` строк, поэтому мы, должно быть, выводим часть последней строки. Аналогичные рассуждения помогут нам заключить, что если значение переменной `c` равно нулю, мы выводим часть первого столбца, а если `c` равно `cols - 1`, — часть последнего столбца. Используя эти знания, мы можем закодировать еще часть нашей программы.

```
// инвариант: пока мы вывели с символов текущей строки.
while (c != cols) {
    if (r == 0 || r == rows - 1 || c == 0 || c == cols - 1) {
        std::cout << " ";
        ++c;
    } else {
        // выводим один или несколько символов.
        // изменяем значение c для поддержки истинности
        // инварианта.
    }
}
```

В этой инструкции представлен ряд новшеств, которые требуют детального разъяснения.

2.4.1.1. Инструкции if

Тело `while`-инструкции, приведенной в разделе 2.3.1, состоит из блока, который содержит инструкцию `if`. Эту инструкцию `if` мы используем для выяснения, не пора ли вывести символ “звездочка”. Инструкция `if` может иметь две формы.

- Без ключевого слова `else`

```
if (условие)
    инструкция
```

- С ключевым словом `else`

```
if (условие)
    инструкция1
else
    инструкция2
```

Как и в случае записи `while`-инструкции, элемент *условие* представляет собой выражение, которое может быть истинным или ложным. Если заданное условие истинно, программа выполняет инструкцию, которая следует за директивой `if` (элемент *инструкция* в первой форме записи или *инструкция1* во второй форме записи). Если условие ложно, то при использовании второй формы записи `if`-инструкции программа выполняет инструкцию, которая следует за ключевым словом `else` (элемент *инструкция2*).

2.4.1.2. Логические операторы

Что представляет собой следующее условие?

```
r == 0 || r == rows - 1 || c == 0 || c == cols - 1
```

Это условие истинно, если значение переменной `r` равно 0 или `rows - 1` либо значение переменной `c` равно 0 или `cols - 1`. В этом условии использовано два новых оператора: “==” и “||”. В C++-программах оператор *проверки равенства* записывается с помощью символа “==”, чтобы отличить его от оператора присваивания (=). Следовательно, выражение `r == 0`, имеющее тип `bool`, означает, равно ли значение переменной `r` числу

0. Оператор *логическое ИЛИ* (logical-or operator), записываемый с помощью символа “||”, генерирует истинное значение, если любой из его операндов истинен.

Операторы отношений имеют более низкий приоритет, чем арифметические. В выражениях, содержащих несколько операторов, именно *приоритет* операторов (precedence) определяет, как группируются операнды. Например, выражение

```
г == rows - 1
```

означает такое группирование операндов

```
г == (rows - 1),
```

а не такое

```
(г == rows) - 1,
```

поскольку арифметический оператор “минус” (-) имеет более высокий приоритет, чем оператор отношения “==”. Другими словами, мы вычитаем 1 из значения переменной rows и сравниваем результат со значением переменной г, что и требуется в нашей программе. Мы можем переопределить приоритет, заключив в круглые скобки подвыражение, которое мы хотели бы использовать как единый операнд. Например, если бы мы действительно хотели выполнить сначала сравнение, а затем вычитание, то использовали бы круглые скобки следующим образом: (г == rows) - 1. В этом выражении из результата сравнения значений переменных г и rows вычитается 1, и конечный результат будет равен 0 или -1, в зависимости от того, равно ли значение г значению rows.

Оператор логическое ИЛИ проверяет, является ли истинным хотя бы один из его операндов. Его форма записи имеет следующий вид.

```
условие1 || условие2
```

Здесь элементы *условие1* и *условие2* представляют собой условия, т.е. выражения, которые могут принимать ложные или истинные значения. Другими словами, ||-выражение принимает значение типа bool, которое истинно, если истинно хотя бы одно из заданных условий.

Оператор “||” имеет более низкий приоритет, чем операторы отношений, и, подобно большинству бинарных операторов в C++, является левоассоциативным. Более того, оператор “||” обладает одним удивительным свойством: если программа обнаруживает, что его левый операнд истинен, правый операнд не вычисляется вовсе. Это свойство часто называется *сокращенным вычислением* (short-circuit evaluation), и, как показано в разделе 5.6, оно может оказывать огромное влияние на процесс написания программы.

Поскольку оператор “||” является левоассоциативным, а также благодаря встроенным приоритетам операторов “||”, “==” и “-”, выражение

```
г == 0 || г == rows - 1 || с == 0 || с == cols - 1
```

не изменится, даже если бы мы поместили все его подвыражения в круглые скобки.

```
((г == 0 || г == (rows - 1)) || с == 0) || с == (cols - 1)
```

Чтобы вычислить это выражение (в его последнем варианте со скобками) с применением сокращенной стратегии, программа должна сначала вычислить крайний левый операнд внешнего оператора “||”, который имеет следующий вид.

```
(г == 0 || г == (rows - 1)) || с == 0
```

Для этого программа должна сначала вычислить левый операнд внутреннего оператора “||”, который выглядит следующим образом.

```
г == 0 || г == (rows - 1)
```

Вычисление этого операнда, в свою очередь, означает вычисление следующего выражения.

```
г == 0
```

Если значение г равно 0, каждое из следующих выражений

```
г == 0 || г == (rows - 1)
(г == 0 || г == (rows - 1)) || с == 0
((г == 0 || г == (rows - 1)) || с == 0) || с == (cols - 1)
```

должно быть истинным. Если значение переменной г отлично от нуля, следующим действием программы будет сравнение значения г со значением rows - 1. Если это сравнение также себя не оправдает, т.е. окажется ложным, то программа будет сравнивать значение переменной с с нулем и, в случае ложного результата, чтобы, наконец, получить конечный результат, перейдет к сравнению значения переменной с со значением выражения cols - 1.

Другими словами, при записи ряда условий, разделенных знаками операторов "||", мы предлагаем программе протестировать каждое из них по очереди. Если какое-нибудь из этих внутренних условий истинно, истинным окажется все условие целиком; в противном случае оно будет ложным. Вычисление каждого оператора "||" прекращается сразу же, как только можно определить его результат, поэтому, если любое из внутренних условий истинно, последующие условия уже не проверяются. Возвращаясь к нашей задаче, нетрудно увидеть, что эти четыре проверки равенства являются проверкой вывода первой строки, последней строки, первого столбца или последнего столбца. Следовательно, инструкция if выводит символ "звездочка", если мы таки попали на первую или последнюю строку либо на первый или последний столбец. В противном случае она (инструкция if) выполняет else-действие, которое нам предстоит еще определить.

2.4.2. Вывод символов, не относящихся к рамочке

Настало время записать инструкции, соответствующие следующим комментариям из фрагмента программы, приведенного в разделе 2.4.1

```
// Выводим один или несколько символов.
// Изменяем значение с для поддержки истинности инварианта.
```

Эти инструкции должны выводить символы, которые не являются частью рамочки. Нетрудно понять, что каждый из этих символов является либо пробелом, либо частью приветствия. Осталось лишь научиться их распознавать.

Вначале выясним, пора ли уже вывести первый символ приветствия. Для этого достаточно узнать, на нужной ли позиции мы "находимся", т.е. "вышли" ли мы на нужную строку и нужный столбец в этой строке. Искомая строка находится сразу после начальной строки "звездочек" и нескольких "пустых" строк, точное количество которых содержится в переменной rad. Соответствующий столбец (на соответствующей строке) идет сразу за начальным символом "звездочка" и пробелами, количество которых содержится в переменной rad. Наши знания об инвариантах подсказывают следующее: чтобы оказаться на нужной строке, значение переменной г должно быть равно значению выражения rad + 1, а чтобы оказаться на нужном столбце — значение переменной с должно быть равно значению выражения rad + 1.

Другими словами, чтобы точно определить момент для вывода первого символа приветствия, мы должны проверить, равны ли оба значения переменных г и с значе-

нию выражения `pad + 1`. Если мы точно определим позицию для вывода приветствия, уверенно выведем его; в противном случае мы выведем пробел. В обоих случаях мы не должны забыть об обновлении значения переменной `c`.

```
if (r == pad + 1 && c == pad + 1) {  
    std::cout << greeting;  
    c += greeting.size();  
} else {  
    std::cout << " ";  
    ++c;  
}
```

В условии внутри инструкции `if` используется оператор *логическое И* (logical-and operator). Подобно оператору `"||"`, оператор `"&&"` проверяет два условия и оценивает значение истинности. Он также левоассоциативен и использует стратегию сокращенного вычисления. Но, в отличие от оператора `"||"`, оператор `"&&"` генерирует истинное значение только в том случае, если оба условия окажутся истинными. Если одно из условий ложно, результат выполнения `&&`-оператора ложен. Более того, второе условие будет тестироваться только в том случае, если первое условие окажется истинным.

Если тестирование условий даст истинный результат, значит, самое время вывести приветствие. Делая это, мы искажаем наш инвариант, поскольку значение переменной `c` больше не будет равно количеству символов, записанных в этой строке. Чтобы инвариант снова стал истинным, значение переменной `c` нужно привести в соответствие с числом действительно выведенных символов. В выражении, которое обновляет значение переменной `c`, используется еще один новый оператор, именуемый *составным оператором присваивания* (compound-assignment operator). Такое составное присваивание означает сложение правого и левого операндов с сохранением результата в левом операнде. Таким образом, инструкция `c += greeting.size()` по своему результату идентична инструкции `c = c + greeting.size()`.

Оставшийся блок рассматриваемого фрагмента программы выполняется тогда, когда мы выводим не символы рамки и не символы приветствия. В этом случае мы должны вывести пробел и инкрементировать значение переменной `c`, чтобы вернуть инвариант "на путь истинный", что мы и делаем в `else`-ветви `if`-инструкции.

2.5. Полная программа вывода приветствия в рамочке

Теперь, когда мы проработали всю программу, нам осталось собрать воедино разрозненные ее фрагменты. Но прежде чем привести всю программу целиком, мы хотели бы показать, как можно сократить ее (причем тремя способами).

Первое сокращение — своего рода объявление, позволяющее заявить раз и навсегда о том, что данное имя "родом" из стандартной библиотеки. Это объявление разрешает нам избежать многократного повторения префикса `std::` в различных местах программы. Второе сокращение состоит в использовании сокращенного способа записи `while`-инструкции. Наконец, в результате третьего сокращения мы можем немного сократить программу, инкрементируя значение переменной `c` не в двух местах, а только в одном.

2.5.1. Устранение многократного повторения префикса `std::`

Вероятно, вы уже устали от созерцания перед каждым библиотечным именем префикса `std::`. Безусловно, использование `std::` в явном виде — хороший способ напоминания о том, что данные имена "берут свое начало" из стандартной библиотеки,

но к настоящему моменту вы должны уже иметь достаточно хорошее представление о библиотечных именах.

В C++ предусмотрена возможность оповещения о том, что некоторое имя необходимо всегда интерпретировать в качестве “выхода” из указанного пространства имен. Например, объявление

```
using std::cout;
```

говорит о том, что мы собираемся использовать имя `cout` исключительно в значении `std::cout` и что у нас нет намерения называть что-либо еще именем `cout` без указанного префикса. После этого объявления мы можем везде, вместо полного имени `std::cout`, использовать имя `cout`.

Неудивительно, что такое объявление называется *using-объявлением*. Имя, используемое в *using-объявлении*, ведет себя аналогично другим именам. Например, если подобное *using-объявление* находится внутри фигурных скобок, то указанное им значение будет действовать от момента объявления до закрывающей фигурной скобки.

Отныне, чтобы сделать нашу программу более стройной, мы будем использовать *using-объявления*.

2.5.2. Использование *for*-инструкций для компактности кода

Давайте снова взглянем на управляющую структуру, которую мы использовали в программе, приведенной в разделе 2.3. Если сконцентрировать внимание только на внешней структуре программы, то ее (программу) можно представить в следующем виде.

```
int r = 0;
while (r != rows) {
    // Действия, не изменяющие значение переменной r.
    ++r;
}
```

Такая форма *while*-инструкции встречается довольно часто. До начала ее работы мы определяем и инициализируем локальную переменную, которая тестируется в условии. В теле *while*-цикла значение этой переменной изменяется, чтобы условие, в конце концов, стало ложным. Поскольку управляющая структура такого вида очень распространена, для нее предусмотрена в языке специальная форма записи.

```
for (int r = 0; r != rows; ++r) {
    // Действия, не изменяющие значение переменной r.
}
```

В теле каждого из этих циклов переменная `r` будет принимать ряд значений, первое из которых равно 0, а последнее — `rows - 1`. Принято называть значение 0 началом диапазона, а `rows` — его *невключаемым концом* (off-the-end value). Такой диапазон называется *полуоткрытым* (half-open range) и часто записывается как [*Начало*, *Конец*). О том, что диапазон асимметричен, напоминают умышленно используемые неодинакового типа скобки `[]`. Так, например, диапазон `[1, 4)` содержит значения 1, 2 и 3, но не 4. Аналогично можно сказать, что переменная `r` принимает значения из диапазона `[0, rows)`.

Инструкция for имеет следующую форму записи.

```
for (инструкция_инициализации условие; выражение)
    инструкция
```

Первую строку такой записи часто называют *for-заголовком*. Он управляет элементом *инструкция*, который часто называют *телом for-цикла*. Элемент *инструк-*

ция_инициализации должен представлять собой либо определение (см. раздел 1.1), либо инструкцию-выражение (см. раздел 0.8). Поскольку инструкция любого из упомянутых видов завершается собственной точкой с запятой, в приведенной выше форме записи между элементами *инструкция_инициализации* и *Условие* дополнительная точка с запятой отсутствует.

Выполнение инструкции *for* начинается с элемента *инструкция_инициализации*, который выполняется только один раз, в начале работы *for*-цикла. Обычно посредством элемента *инструкция_инициализации* определяется и инициализируется управляющая переменная цикла, которая будет тестироваться при выполнении элемента *условие*. Если некоторая переменная определяется в элементе *инструкция_инициализации*, то при выходе из *for*-цикла она разрушается и поэтому недоступна для кода, расположенного за инструкцией *for*.

На каждом проходе (итерации) этого цикла, в том числе и первом, программа вычисляет элемент *условие*. Если заданное условие принимает истинное значение, выполняется тело *for*-цикла, а затем — элемент *выражение*. После этого повторяется тестирование элемента *условие* и (при его истинном результате) выполнение тела цикла и элемента *выражение* из *for*-заголовка до тех пор, пока тестирование условия не даст ложный результат.

В более общем виде суть *for*-инструкции можно представить следующим образом.

```
{  
    инструкция_инициализации  
    while (условие) {  
        инструкция  
        выражение;  
    }  
}
```

Здесь важно заключить элемент *инструкция_инициализации* и *while*-инструкцию в дополнительные фигурные скобки, ограничив тем самым время существования любых переменных, объявленных в элементе *инструкция_инициализации*. Обратите особое внимание на использование точки с запятой (ее присутствие и отсутствие). Точка с запятой не ставится после элементов *инструкция_инициализации* и *инструкция*, поскольку они являются инструкциями с собственными знаками (точка с запятой), если таковые им необходимы. Точка с запятой, поставленная после элемента *выражение*, превращает его в инструкцию.

2.5.3. Объединение нескольких проверок в одну

Код, соответствующий комментарию

```
// выводим один или несколько символов.
```

из фрагмента программы, приведенного в разделе 2.4, можно разбить на три случая: вывод одинарного символа “звездочка”, пробела или всего приветствия. Как отмечено в комментариях нашей программы, мы модифицируем значение переменной *s*, чтобы поддерживать истинность нашего инварианта после вывода символа “звездочка”, и снова делаем то же самое после вывода пробела. В этом нет ничего неверного, но зачастую, чтобы получить возможность объединить две или даже больше идентичных инструкций в одну, имеет смысл изменить порядок тестирования в программе.

Поскольку наши три случая взаимно исключающие, мы можем тестировать их в любом порядке. Если мы начнем с проверки на необходимость вывода приветствия, то точно будем знать, что в двух других случаях для поддержки инварианта вполне

достаточно инкрементирования переменной *c*, поэтому мы можем объединить две инструкции инкремента в одну.

```
if (Мы должны выводить приветствие) {
    cout << greeting;
    c += greeting.size();
} else {
    if (Мы должны выводить рамку)
        cout << "*";
    else
        cout << " ";
    ++c;
}
```

Объединив инструкции инкрементирования, мы обнаруживаем, что два наших блока включают лишь по одной инструкции, поэтому мы можем отказаться от двух пар фигурных скобок. Обратите также внимание на отступ, с которым написана инструкция `++c`. Он позволяет понять, что эта инструкция выполняется независимо от того, выводим ли мы символ рамки или пробел.

2.5.4. Полная программа вывода приветствия в рамочке

После объединения всех фрагментов программы и реализации методов сокращения кода получаем следующую программу.

```
#include <iostream>
#include <string>

// Уведомляем об использовании имен стандартной библиотеки.
using std::cin;      using std::endl;
using std::cout;     using std::string;

int main()
{
    // Запрашиваем имя человека.
    cout << "Введите, пожалуйста, свое имя: ";

    // Читаем введенное имя.
    string name;
    cin >> name;

    // Создаем сообщение, подлежащее выводу.
    const string greeting = "Привет, " + name + "!";

    // Количество пробелов, окружающих приветствие.
    const int pad = 1;

    // Количество выводимых строк и столбцов.
    const int rows = pad * 2 + 3;
    const string::size_type cols = greeting.size() + pad * 2 + 2;

    // Выводим пустую строку, чтобы отделить вывод от ввода.
    cout << endl;

    // Выводим rows строк.
    // Инвариант: пока мы вывели r строк.
    for (int r = 0; r != rows; ++r) {

        string::size_type c = 0;

        // Инвариант: пока мы вывели c символов в текущую строку.
        while (c != cols) {

            // Пора выводить приветствие (greeting)?
            if (r == pad + 1 && c == pad + 1) {
```

```

        cout << greeting;
        c += greeting.size();
    } else {
        // Мы должны выводить рамку?
        if (r == 0 || r == rows - 1 ||
            c == 0 || c == cols - 1)
            cout << "*";
        else
            cout << " ";
        ++c;
    }
    cout << endl;
}
return 0;
}

```

2.6. С чего начать отсчет

Многие опытные C++-программисты имеют привычку, которая может показаться на первый взгляд довольно странной: в их программах везде, где только можно, отсчет начинается не с 1, а с 0. Например, если свести внешний for-цикл приведенной выше программы к самой сути, получим следующее.

```

for (int r = 0; r != rows; ++r) {
    // Выводим строку.
}

```

Этот цикл можно было бы записать в следующем виде.

```

for (int r = 1; r <= rows; ++r) {
    // Выводим строку.
}

```

В одной версии отсчет начинается с 0 и при сравнении используется оператор отношения “не равно” ($\neq$); в другой же отсчет начинается с 1, а при сравнении применяется оператор отношения “меньше или равно” ($\leq$). Количество итераций одинаково в обоих случаях. Существует ли какая-то иная причина, чтобы предпочесть один вариант другому?

Одна из причин начала отсчета с 0 — это своего рода стимул при отображении интервалов использовать асимметричные диапазоны. Например, вполне естественно для описания первой for-инструкции использовать диапазон $[0, \text{rows})$, а для описания второй — диапазон $[1, \text{rows}]$.

Асимметричные диапазоны обычно легче использовать, чем симметричные, и все благодаря одному их важному свойству: диапазон формы $[m, n)$ содержит $n - m$ элементов, а диапазон формы $[m, n]$ — $n - m + 1$ элементов. Так, например, количество элементов в диапазоне $[0, \text{rows})$ очевидно (т.е. $\text{rows} - 0$, или просто rows), а количество элементов в диапазоне $[1, \text{rows}]$ менее очевидно.

Такое поведенческое различие между асимметричным и симметричным диапазонами особенно наглядно проявляется в случае пустых диапазонов: при использовании асимметричных диапазонов мы можем выразить любой пустой диапазон в форме $[n, n)$, в отличие от формы $[n, n1]$ для симметричных диапазонов. Ненулевая вероятность того, что конец диапазона может оказаться меньше его начала, способна стать причиной “головной боли” при разработке программ.

Еще одно “очко” в пользу начала отсчета с нуля привносят инварианты циклов, которые в этом случае попросту легче выразить. В нашем примере отсчет с нуля делает инвариант чрезвычайно простым. Вспомните соответствующий комментарий.

// инвариант: пока мы вывели г строк.

А какой бы был инвариант, если начать отсчет с 1?

Можно попытаться определить инвариант как заявление о том, что мы собираемся вывести g -ю строку, но такое утверждение нельзя квалифицировать как инвариант. Дело в том, что при последнем тестировании условия в `while`-инструкции переменная g будет содержать значение $rows + 1$, а ведь мы собираемся вывести лишь $rows$ строк. Следовательно, мы не имели намерения выводить g -ю строку, поэтому такой инвариант не истинен!

В этом случае в качестве инварианта можно было бы использовать заявление о том, что пока мы вывели $g - 1$ строк. Но если нам подходит такой инвариант, то почему бы не упростить его, используя для начального значения переменной g число 0?

Еще одной причиной начала отсчета с 0 является то, что в этом случае мы получаем возможность использовать для операции сравнения оператор “`!=`”, а не “`<=`”. Такое различие может показаться незначительным, но оно оказывает влияние на состояние программы при завершении цикла. Например, если проверяемое условие имеет вид $g != rows$, то по завершении цикла мы точно знаем, что $g == rows$. Выведя, согласно инварианту, g строк, мы знаем, что вывели *в точности* все $rows$ строк. Но если в качестве условия использовать выражение $g <= rows$, то все, что мы можем утверждать, сводится к тому, что мы вывели, *по меньшей мере*, $rows$ строк. Хотя и знаем, что могли бы вывести больше строк.

При отсчете с 0 мы можем в качестве условия применить выражение $g != rows$, если хотим быть уверены в выполнении ровно $rows$ итераций, или же можем использовать выражение $g < rows$, если нам необходимо, чтобы количество итераций было равно значению переменной $rows$ или больше него. Если мы начнем отсчет с 1, то сможем применить выражение $g <= rows$, если захотим выполнить, по крайней мере, $rows$ итераций, но как быть, если нам понадобится гарантия выполнения в точности $rows$ итераций? Тогда нам придется протестировать более сложное условие, а именно $g == rows + 1$. К сожалению, эта дополнительная сложность не влечет за собой никакого компенсирующего преимущества.

2.7. Резюме

Выражения. Язык C++ наследует богатый набор операторов из языка C (некоторые из них мы уже использовали). Кроме того, как мы уже видели на примере операторов ввода и вывода данных, C++-программы могут расширять базовый язык за счет применения встроенных операторов к объектам типа класса. Правильное понимание сложных выражений — необходимое условие для эффективного программирования на C++. Понимание таких выражений требует уяснения следующих аспектов программирования.

- Как можно группировать операнды на основе правил предшествования и ассоциативности операторов, используемых в выражении.
- Как происходит преобразование операндов в значения другого типа, если оно возможно.
- В каком порядке вычисляются операнды.

Различные операторы имеют различный приоритет. Большинство операторов являются левоассоциативными, хотя операторы присваивания и операторы, принимающие один аргумент, правоассоциативны. Ниже мы приводим самые распространенные операторы, вне зависимости от их использования в этой главе. Мы расположили их в порядке убывания приоритета, а двойные линии разделяют группы с одинаковым приоритетом.

<code>x.y</code>	Член <code>y</code> объекта <code>x</code>
<code>x[y]</code>	Элемент с индексом <code>y</code> в объекте <code>x</code>
<code>x++</code>	Инкрементирует <code>x</code> , возвращая исходное значение <code>x</code>
<code>x--</code>	Декрементирует <code>x</code> , возвращая исходное значение <code>x</code>
<code>++x</code>	Инкрементирует <code>x</code> , возвращая инкрементированное значение
<code>--x</code>	Декрементирует <code>x</code> , возвращая декрементированное значение
<code>!x</code>	Логическое отрицание. Если значение <code>x</code> истинно (<code>true</code>), то значение <code>!x</code> ложно (<code>false</code>)
<code>x * y</code>	Произведение <code>x</code> и <code>y</code>
<code>x / y</code>	Частное от деления <code>x</code> на <code>y</code> . Если оба операнда целые, C++-среда выбирает, в какую сторону округлять: до нуля или до $-\infty$
<code>x % y</code>	Остаток от деления <code>x</code> на <code>y</code> , эквивалент значению выражения <code>x - ((x / y) * y)</code>
<code>x + y</code>	Сумма <code>x</code> и <code>y</code>
<code>x - y</code>	Результат вычитания <code>y</code> из <code>x</code>
<code>x >> y</code>	Для целых значений <code>x</code> и <code>y</code> значение <code>x</code> сдвигается вправо на <code>y</code> разрядов; значение <code>y</code> при этом должно быть неотрицательным. Если <code>x</code> имеет тип <code>istream</code> , выполняется чтение из <code>x</code> в <code>y</code>
<code>x << y</code>	Для целых значений <code>x</code> и <code>y</code> значение <code>x</code> сдвигается влево на <code>y</code> разрядов; значение <code>y</code> при этом должно быть неотрицательным. Если <code>x</code> имеет тип <code>ostream</code> , выполняется запись <code>y</code> в <code>x</code>
<code>x Op y</code>	Операторы отношений (<code>Op</code>) генерируют значение типа <code>bool</code> , означающее истинность данного отношения. Операторы отношений: <code><</code> , <code>></code> , <code><=</code> и <code>>=</code>
<code>x == y</code>	Генерирует значение типа <code>bool</code> , означающее, равно ли значение <code>x</code> значению <code>y</code>
<code>x != y</code>	Генерирует значение типа <code>bool</code> , означающее, не равно ли значение <code>x</code> значению <code>y</code>
<code>x && y</code>	Генерирует значение типа <code>bool</code> , означающее, истинны ли значения обеих переменных <code>x</code> и <code>y</code> . Вычисляет <code>y</code> только в том случае, если <code>x</code> имеет значение <code>true</code>
<code>x y</code>	Генерирует значение типа <code>bool</code> , означающее, истинно ли значение хотя бы у одной из переменных <code>x</code> и <code>y</code> . Вычисляет <code>y</code> только в том случае, если <code>x</code> имеет значение <code>false</code>

<code>x = y</code>	Присваивает переменной <code>x</code> значение переменной <code>y</code> , генерируя значение <code>x</code> в качестве результата
<code>x op= x</code>	Составные операторы присваивания; эквивалент инструкции <code>x = x op y</code> , где <code>op</code> — арифметический оператор или оператор сдвига
<code>x ? y : z</code>	Результатом является значение <code>y</code> , если <code>x</code> равно <code>true</code> ; в противном случае <code>z</code> . Вычисляется только одно из значений <code>y</code> и <code>z</code>

Обычно программисты не полагаются на автоматическое соблюдение порядка вычисления операндов в выражении. Поскольку порядок вычислений не фиксирован, важно избегать написания выражения, в котором один операнд зависит от значения другого. Соответствующий пример рассматривается в разделе 4.1.5.

Операнды, где это возможно, преобразуются в значения соответствующего типа. Преобразования числовых операндов в выражениях (в том числе и в выражениях отношений) осуществляются в соответствии с обычными арифметическими преобразованиями, подробно описанными в разделе A.2.4.4. В основном, *обычные арифметические преобразования* тяготеют к сохранению точности. Более “мелкие” типы приводятся к более “крупным”, а типы со знаком — к типам без знака. Арифметические значения могут быть преобразованы в значения типа `bool`: значение 0 рассматривается как значение `false`, а любое другое — как значение `true`. Преобразование операндов типа класса рассматривается в главе 12.

Типы	
<code>bool</code>	Встроенный тип, представляющий значения истинности: <code>true</code> или <code>false</code>
<code>unsigned</code>	Целочисленный тип, который содержит только неотрицательные значения
<code>short</code>	Целочисленный тип, который должен хранить по меньшей мере 16 бит
<code>long</code>	Целочисленный тип, который должен хранить по меньшей мере 32 бит
<code>size_t</code>	Целочисленный тип без знака (из заголовка <code><cstdint></code>), который способен сохранить размер любого объекта
<code>string::size_type</code>	Целочисленный тип без знака, который способен хранить размер <code>string</code> -объекта

Полуоткрытые диапазоны включают только одну, а не обе свои конечные точки. Например, диапазон `[1, 3)` включает значения 1 и 2, но не 3.

Условие. Выражение, имеющее значение истинности. Арифметические значения, используемые в условиях, преобразуются в значения типа `bool`: ненулевые значения — в значение `true`, а нулевые — в значение `false`.

Инструкции

<code>using имя_пространства_имен : имя</code>	Определяет <i>имя</i> как синоним для элемента <i>имя_пространства_имен</i> : <i>имя</i> .
<code>имя_типа имя;</code>	Определяет переменную <i>имя</i> типа <i>имя_типа</i> .
<code>имя_типа имя = значение;</code>	Определяет переменную <i>имя</i> типа <i>имя_типа</i> , инициализированную в виде копии элемента <i>значение</i> .

имя_типа *Имя*(Аргументы);

Определяет переменную *Имя* типа *Имя_типа*, создаваемую в соответствии с аргументами, заданными с помощью элемента *Аргументы*.

Выражение;

Выполняет элемент *Выражение* для получения его побочных эффектов.

{ *инструкция*, или *инструкции* }

Называется блоком. Выполняет последовательность инструкций (их количество может быть даже нулевым) в порядке их следования. Может использоваться везде, где ожидается элемент *инструкция*. Переменные, определяемые внутри фигурных скобок, имеют область видимости, ограниченную блоком.

while (*условие*) *инструкция*

Если элемент *Условие* принимает значение false, ничего не выполняется; в противном случае выполняется элемент *инструкция*, а затем повторяется while-инструкция.

for (*инструкция_инициализации* *Условие*; *Выражение*) *инструкция*

Эквивалент инструкции { *инструкция_инициализации* while (*Условие*) { *инструкция* *Выражение*; } }, если элемент *инструкция* не содержит инструкцию continue (см. раздел А.4) или не является ею.

if (*условие*) *инструкция*

Выполняет элемент *инструкция*, если элемент *Условие* принимает значение true.

if (*условие*) *инструкция*; else *инструкция2*

Выполняет элемент *инструкция*, если элемент *Условие* принимает значение true, в противном случае выполняет элемент *инструкция2*.

Каждая else-ветвь соответствует ближайшей if-ветви.

return *значение*;

Осуществляет выход из функции и возвращает элемент *Значение* автору вызова функции.

Упражнения

- 2.0. Скомпилируйте и выполните программу, приведенную в этой главе.
- 2.1. Модифицируйте программу вывода приветствия в рамочке таким образом, чтобы она выводила рамочку, вплотную прилегающую к приветствию.
- 2.2. Модифицируйте программу вывода приветствия в рамочке таким образом, чтобы она использовала различное количество пробелов для отделения приветствия от боковых сторон, а также от верхней и нижней.
- 2.3. Модифицируйте программу вывода приветствия в рамочке таким образом, чтобы она предлагала пользователю ввести количество пробелов, отделяющих приветствие от рамки.
- 2.4. Программа вывода приветствия в рамочке, в основном, выводит строки с пробелами, отделяющие приветствие от горизонтальных границ рамочки, посимвольно. Модифицируйте программу так, чтобы она записывала все пробелы в одном выражении вывода данных.
- 2.5. Выведите ряд символов "*" так, чтобы они образовали квадрат, прямоугольник и треугольник.
- 2.6. Что делает следующий код?

```
int i = 0;
while (i < 10) {
    i += 1;
```

```
    std::cout << i << std::endl;  
}
```

2.7. Напишите программу, которая бы считала от 10 до -5.

2.8. Напишите программу для вычисления произведения чисел из диапазона [1, 10).

2.9. Напишите программу, которая бы предлагала пользователю ввести два числа, а затем сообщала, какое из них больше.

2.10. Поясните каждое из применений префикса `std::` в следующей программе.

```
int main()  
{  
    int k = 0;  
    while (k != n) { // инвариант: пока мы вывели  
                    // k символов "звездочка".  
        using std::cout;  
        cout << "*";  
        ++k;  
    }  
    std::cout << std::endl; // Здесь нужен префикс std::.  
    return 0;  
}
```

РАБОТА С ГРУППАМИ ДАННЫХ

Программы, которые мы рассматривали в главах 1 и 2, делали несколько больше, чем просто считывали строку и выводили ее в виде приветствия (с рамочкой или без). И все же большинство проблем, возникающих в реальной работе, гораздо сложнее. Одной из самых распространенных сложностей в программах является необходимость обработки массивов однородных данных.

В наших программах мы уже касались этой проблемы; в том смысле, что любая строка содержит несколько символов. И в самом деле, уже одно то, что нам удалось поместить заранее неизвестное количество символов в один объект (типа `string`), сделало эти программы простыми для написания.

В этой главе мы рассмотрим другие способы обработки групп данных, и в качестве примера возьмем программу, которая считывает оценки, полученные студентами на экзаменах и за выполнение домашних заданий, а затем вычисляет итоговые оценки. Кроме того, мы узнаем, как можно сохранить все введенные оценки, даже если нам заранее неизвестно, сколько их будет введено.

3.1. Вычисление оценок студентов

Представьте себе курс, в котором оценка по последнему экзамену каждого студента составляет 40% от итоговой оценки, оценка по экзамену в середине семестра — 20%, а усредненная оценка за выполнение домашних заданий — оставшиеся 40%. Вот как выглядит первый вариант программы, которая должна помочь студентам в вычислении их итоговых оценок.

```
#include <iomanip>
#include <ios>
#include <iostream>
#include <string>

using std::cin;      using std::setprecision;
using std::cout;     using std::string;
using std::endl;     using std::streamsize;

int main()
{
    // Запрашиваем и считываем имя студента.
    cout << "Пожалуйста, введите свое имя: ";
    string name;
    cin >> name;
    cout << "Привет, " << name << "!" << endl;

    // Запрашиваем и считываем оценки по экзаменам,
```



```

// проведенным в середине и в конце семестра.
cout << "Пожалуйста, введите оценки по экзаменам "
      << "в середине и в конце семестра: ";
double midterm, final;
cin >> midterm >> final;

// Запрашиваем оценки за выполнение домашних заданий.
cout << "Введите все оценки за выполнение домашних заданий, "
      << "завершив ввод знаком конца файла: ";

// Количество и сумма оценок, прочитанных до сих пор.
int count = 0;
double sum = 0;

// Переменная, в которую выполняется считывание данных.
double x;

// Инвариант:
// Мы прочитали пока count оценок, и
// переменная sum содержит сумму первых count оценок.
while (cin >> x) {
    ++count;
    sum += x;
}

// Выводим результат.
streamsize prec = cout.precision();
cout << "Ваша итоговая оценка равна " << setprecision(3)
      << 0.2 * midterm + 0.4 * final + 0.4 * sum / count
      << setprecision(prec) << endl;

return 0;
}

```

Как обычно, программа начинается с директив `#include` и `using`-объявлений, позволяющих использовать нужные нам библиотечные средства, а именно заголовки `<iomanip>` и `<iostream>`, с которыми мы пока еще не встречались. Заголовок `<iostream>` определяет тип `streamsize`, который библиотека ввода-вывода использует для представления размеров, а заголовок `<iomanip>` — манипулятор `setprecision`, который позволяет указать, сколько значащих цифр должны содержать выводимые нами данные.

Использование манипулятора `endl` в предыдущих программах не вынуждало нас включать заголовок `<iomanip>`, поскольку из-за частого применения его определение помещено в заголовок `<iostream>`, а не `<iomanip>`.

Программа начинается с запроса и считывания имени студента, а также оценок по экзаменам, проведенным в середине и конце семестра. Затем программа предлагает ввести оценки, полученные студентом за выполнение домашних заданий, ввод которых будет продолжаться до тех пор, пока не встретится признак конца файла. Различные C++-среды предлагают своим пользователям различные способы ввода в программу таких сигнальных значений, но чаще всего для ввода признака конца файла нужно перейти на новую строку и, удерживая клавишу `<Ctrl>`, нажать `<Z>` (для компьютеров, работающих под управлением операционной системы Microsoft Windows) или `<D>` (для компьютеров, работающих под управлением систем Unix или Linux).

При считывании оценок программа использует переменную `count` для подсчета их количества и сохраняет в переменной `sum` промежуточную их сумму. Завершив чтение всех оценок, программа выводит итоговую оценку студента, при вычислении которой используется среднее значение от всех оценок за домашние задания. Для вычисления этого среднего арифметического и пригодились значения, содержащиеся в переменных `count` и `sum`.

Многое в этой программе должно быть вам уже знакомо, но некоторые новшества требуют разъяснения.

Рассмотрим фрагмент программы, в котором выполняется чтение оценок студента.

```
cout << "пожалуйста, введите оценки по экзаменам "
      "в середине и в конце семестра: ";
double midterm, final;
cin >> midterm >> final;
```

В первой инструкции нет ничего нового: она выводит сообщение, которое в данном случае напоминает студенту о том, что ему нужно сделать. Следующая инструкция определяет переменные `midterm` и `final` типа `double` (это встроенный тип, который используется для представления чисел с плавающей точкой удвоенной точности). Для представления чисел с обычной точностью в языке C++ также предусмотрен тип, именуемый `float`, но, как правило, для вычислений с плавающей точкой используется именно тип `double`.

Наличие этих двух типов восходит к тем временам, когда такой системный ресурс, как память, был гораздо более дорогим, чем ныне. Более короткий тип представления вещественных чисел (`float`) позволяет хранить числа небольшой точности (до шести значащих, т.е. десятичных, цифр), что недостаточно для указания стоимости приличного дома. Тип `double` гарантирует, по крайней мере, десять значащих цифр. В современных компьютерах с помощью типа `double` числа представляются обычно более точно, чем с помощью типа `float`, и при этом их обработка происходит непонятно медленнее, а иногда даже и быстрее.

После определения переменных `midterm` и `final` наша программа считывает вводимые пользователем значения и запоминает их в этих "новоиспеченных" переменных. Подобно оператору вывода (см. раздел 0.7), оператор ввода возвращает левый операнд в качестве результата. Поэтому мы можем связать в цепочку операции ввода точно так же, как и операции вывода, т.е. инструкция

```
cin >> midterm >> final;
```

генерирует такой же результат, как и две следующие инструкции.

```
cin >> midterm;
cin >> final;
```

В каждом варианте из стандартного входного потока первое введенное число считывается в переменную `midterm`, а затем следующее число — в переменную `final`.

Следующая инструкция предлагает студенту ввести оценки, полученные за выполнение домашних заданий.

```
cout << "введите все оценки за выполнение домашних заданий, "
      "завершив ввод признаком конца файла: ";
```

Обратите внимание на то, что эта инструкция содержит только один оператор "<<", несмотря на наличие двух строковых литералов. Дело в том, что два или больше строковых литералов, разделенных лишь пробелами, автоматически конкатенируются. Следовательно, результат выполнения инструкции

```
cout << "введите число_1, "
      "затем число_2: ";
```

абсолютно совпадает с результатом выполнения следующей инструкции.

```
cout << "введите число_1, затем число_2: ";
```

Разделив один строковый литерал на два, мы тем самым отказываемся от слишком длинных и потому неудобных для чтения строк.

В следующей части программы определяются переменные, которые мы будем использовать для хранения новой порции входных данных.

```
int count = 0;
double sum = 0;
```

Обратите внимание на то, что мы присваиваем начальное значение 0 обоим переменным `sum` и `count`. Число 0 имеет тип `int`, а это значит, что для того, чтобы инициализировать переменную `sum`, C++-среда должна привести его к типу `double`. Мы могли бы избежать этого преобразования типов, инициализируя переменную `sum` с помощью значения 0.0, а не 0. Однако в данном контексте это не принесло бы нам никакой практической пользы, поскольку в любой среде подобные преобразования типов выполняются во время компиляции и не влияют на время выполнения программы, поэтому результат в обоих случаях был бы одинаковым.

В нашей же программе важнее то, что мы вообще присваиваем этим переменным какое-то начальное значение. Ведь когда мы не задаем для переменной никакого начального значения, это говорит о том, что мы неявно полагаемся на *инициализацию по умолчанию* (default-initialization). Инициализация, которая выполняется по умолчанию, зависит от типа инициализируемой переменной. Для объектов типа класса сам класс определяет, какой инициализатор следует использовать, если таковой не указан. Например, как отмечалось в разделе 1.1, если в программе некоторая `string`-переменная не инициализируется явным образом, то она инициализируется неявно и в качестве ее начального значения используется пустая строка. Однако для локальных переменных встроенного типа такой неявной инициализации не предусмотрено.

Локальные переменные встроенного типа, которые не инициализированы явным образом, остаются в *неопределенном состоянии*, или попросту *неопределенными* (undefined), а это говорит о том, что значение такой переменной состоит из “мусора”, т.е. случайных данных, оказавшихся в области памяти, в которой была создана эта переменная. Неопределенное значение можно заменить некоторым допустимым значением, иное же использование неопределенного значения неправомерно. Многие C++-среды не выявляют нарушений этого правила и позволяют получать доступ к неопределенным значениям. Это практически всегда приводит к аварийному отказу или получению неверного результата, поскольку “мусор” в памяти почти всегда оказывается не корректным значением, а совершенно неподходящим для объявленного типа.

Если бы мы не присвоили переменной `sum` (или переменной `count`) некоторое начальное значение, наша программа, вероятнее всего, завершилась бы отказом. Дело в том, что значения этих “новоиспеченных” переменных программа использует практически сразу же: считывает значение `count`, чтобы его инкрементировать, и значение `sum`, чтобы сложить его с только что принятым от пользователя. И в то же время мы отнюдь не беспокоимся о присваивании начального значения переменной `x`, поскольку такое присваивание выполняется посредством записи в нее вводимого пользователем данного, которое “затерло” бы любое значение, которое мы могли бы (ну, так, на всякий случай) присвоить переменной `x`.

Единственное новшество следующей `while`-инструкции связано с формой используемого здесь условия.

```
// инвариант:
// до сих пор мы прочитали count оценок, и
// значение sum равно сумме первых count оценок.
while (cin >> x) {
    ++count;
    sum += x;
}
```

Мы уже знаем, что `while`-цикл выполняется до тех пор, пока его условие (`cin >> x`) справедливо. Ниже (в разделе 3.1.1) мы подробнее остановимся на том, что означает выражение `cin >> x`, рассматриваемое как условие, но пока нам важно знать, что это условие справедливо, если удовлетворен новый запрос на ввод данных.

Внутри инструкции `while` мы используем инструкцию инкрементирования и составную инструкцию присваивания, которые мы уже применяли в главе 2. Поэтому вы уже должны знать, что выражение `++count` прибавляет 1 к значению переменной `count`, а выражение `sum += x` суммирует значение `x` с значением `sum`.

Теперь нам осталось лишь разобраться в том, как программа выполняет вывод данных.

```
streamsize prec = cout.precision();
cout << "Ваша итоговая оценка равна " << setprecision(3)
    << 0.2 * midterm + 0.4 * final + 0.4 * sum / count
    << setprecision(prec) << endl;
```

Наша задача — вывести итоговую оценку с тремя значащими цифрами. Достигнуть этого можно с помощью манипулятора `setprecision`. (Вспомните, мы уже знакомы с манипулятором `endl`.) Он манипулирует потоком, заставляя следующие выводимые данные “иметь при себе” заданное количество значащих цифр. Записав выражение `setprecision(3)`, мы тем самым формируем для C++-среды запрос на вывод значений оценок с тремя значащими цифрами (обычно двумя до десятичной точки и одной — после).

Используя манипулятор `setprecision`, мы изменяем точность любого следующего выводимого значения, которое может быть послано в поток `cout`. Поскольку эта инструкция находится в конце программы, мы знаем, что больше выходных данных в ней не будет. Тем не менее мы считаем, что было бы мудро (и в некотором смысле порядочно) оставить (после себя) точность потока `cout` в том состоянии, в котором она находилась до нашего вмешательства, т.е. до того, как мы ее изменили. Это достигается посредством вызова функции-члена (см. раздел 1.2) потока `cout` с именем `precision`. Эта функция сообщает значение точности, используемой потоком для вывода чисел с плавающей точкой. Итак, мы применяем манипулятор `setprecision` для установки значения точности равным 3, выводим итоговую оценку, а затем снова устанавливаем точность равной значению, полученному от функции `precision`. В выражении, с помощью которого вычисляется итоговая оценка, используется несколько арифметических операторов: “*” (для умножения), “/” (деления) и “+” (сложения).

Для установки точности мы могли бы следующим образом использовать функцию-член `precision`.

```
// Сохраняем предыдущее значение точности, устанавливая
// для нее новое значение, равное 3.
streamsize prec = cout.precision(3);

cout << "Ваша итоговая оценка равна "
    << 0.2 * midterm + 0.4 * final + 0.4 * sum / count << endl;

// Восстанавливаем исходное значение точности.
cout.precision(prec);
```

Однако все же мы предпочитаем манипулятор `setprecision`, который позволяет нам минимизировать ту часть программы, в которой восстанавливается исходное значение точности.

3.1.1. Как определить конец ввода данных

По сути, единственным настоящим новшеством в этой программе является условие `while`-инструкции. Здесь в качестве субъекта `while`-условия неявно используется объект класса `istream`.

```
while (cin >> x) { /* ... */ }
```

Результатом выполнения этой инструкции является попытка прочитать данные из потока `cin`. Если чтение пройдет успешно, переменная `x` будет содержать только что прочитанное значение и `while`-проверка также будет считаться успешной. Если чтение окажется неудачным (либо по причине отсутствия вводимых данных, либо из-за попытки пользователя ввести значение, не соответствующее типу переменной `x`), `while`-проверка будет признана неудачной, в результате чего мы не должны будем полагаться на значение переменной `x`.

Попробуем разобраться, как работает этот код. Для начала вспомните, что оператор `>>` возвращает свой левый операнд, поэтому оценка выражения `cin >> x` эквивалентна выполнению проверки `cin >> x` с последующей оценкой значения элемента `cin`. Например, мы можем ввести в переменную `x` одно значение и узнать, насколько успешной оказалась операция ввода, выполнив следующую инструкцию.

```
if (cin >> x) { /* ... */ }
```

Эта инструкция эквивалентна следующей.

```
cin >> x;  
if (cin) { /* ... */ }
```

При использовании выражения `cin >> x` в качестве условия мы не просто проверяем это условие, а считываем вводимое пользователем значение в переменную `x`, что является побочным эффектом этой проверки. Теперь нам осталось лишь выяснить, что означает использовать операнд `cin` в качестве условия `while`-инструкции.

Поскольку `cin` имеет тип `istream`, определенный в стандартной библиотеке, мы должны рассмотреть определение `istream`, чтобы понять значение таких элементов, как `if (cin)` или `while (cin)`. Некоторые детали этого определения довольно сложны, и подробно обсудить их мы сможем лишь после освоения материала из раздела 12.5. Но даже закрывая глаза на эти детали, мы можем понять суть дела.

Все условия, которые мы использовали в главе 2, включали операторы отношений, непосредственно генерирующие значения типа `bool`. Но в качестве условий можно также применять выражения, которые генерируют значения арифметического типа. В этом случае арифметическое значение преобразуется в значение типа `bool`: ненулевые значения соответствуют `bool`-значению `true`, а нулевые — `bool`-значению `false`. В данной ситуации нам важно знать, что класс `istream` обеспечивает преобразование, которое позволяет использовать `cin` в качестве значения, применяемого в условии. Мы пока не знаем, какой тип имеет это значение, но знаем, что оно может быть преобразовано в значение типа `bool`, а, следовательно, его можно использовать в условии. Значение, генерируемое этим преобразованием, зависит от внутреннего состояния объекта класса `istream`, который запомнит, была ли удачной последняя попытка ввести данные. Таким образом, использование `cin` в качестве условия эквивалентно проверке успешности последней попытки прочитать вводимые данные.

Существует несколько причин неудачного чтения из потока.

- Достигнут конец входного файла.
- Была попытка ввести значение, не совместимое с типом переменной, в которой предполагалось его сохранить; например, при попытке прочитать `int`-значение вдруг оказалось бы, что вводимое данное не является числом.
- Система обнаружила аппаратный отказ устройства ввода данных.

В любом из перечисленных случаев результат один и тот же: использование входного потока в качестве условия будет означать, что условие ложно. Более того, если нам однажды не удалось прочитать данные из потока, все дальнейшие попытки прочитать из него данные будут обречены на неудачу до тех пор, пока мы не восстановим поток, но как это сделать — узнаем в разделе 4.1.3.

3.1.2. Инвариант цикла

Инвариант (2.3.2) этого цикла требует особого внимания, поскольку условие в `while`-инструкции имеет побочные эффекты, которые влияют на истинность инварианта: успешное выполнение операции ввода `cin >> x` делает первую часть инварианта, в которой утверждается, что мы прочитали `count` оценок, ложной. Следовательно, мы должны так изменить наш анализ, чтобы условие само включало инвариант.

Мы знаем, что инвариант был истинным до оценки условия, поэтому уверены в том, что уже прочитали `count` оценок. Если ввод данных `cin >> x` окажется успешным, то мы прочитаем `count + 1` оценок. И эта часть инварианта будет истинной снова, если инкрементировать значение переменной `count`. Однако это исказит вторую часть инварианта, в которой утверждается, что значение переменной `sum` равно сумме первых `count` оценок, поскольку после инкремента `count` переменная `sum` будет содержать сумму первых `count - 1`, а не первых `count` оценок. К счастью, вторую часть инварианта мы можем сделать истинной, выполнив инструкцию `sum += x;`, и тогда инвариант целиком будет истинным на следующей итерации `while`-цикла.

Если условие ложно, это означает, что наша попытка ввода данных провалилась, т.е. мы не получили никаких новых данных, и инвариант при этом остался истинным. Следовательно, после завершения `while`-инструкции мы не должны учитывать побочные эффекты оценки ее условия.

3.2. Использование медианы вместо среднеарифметического

Наша программа имеет один недостаток: она отбрасывает каждую введенную пользователем оценку, полученную за выполнение домашнего задания, сразу после ее прочтения. Такой способ обработки прекрасно подходит для вычисления среднеарифметических значений, но что делать, если вместо среднеарифметической нам понадобится медианная оценка?

Самый простой способ вычисления медианы некоторой коллекции значений — отсортировать эти значения по возрастанию (или убыванию) и найти расположенное посередине или (при четном количестве значений) вычислить среднее арифметическое от двух “срединных” значений. Медианные значения часто оказываются полезнее, чем среднеарифметические, поскольку они не позволяют нескольким “несуветным” значениям “загубить” тенденцию, выраженную исследуемой коллекцией чисел.

И если мы убедились в необходимости вычисления медиан, нам придется полностью изменить свою программу. Чтобы найти медиану от неизвестного заранее количества зна-

чений, мы должны хранить каждое значение до тех пор, пока не прочитаем весь набор. Для вычисления среднего арифметического нам было достаточно хранить только промежуточные значения количества и суммы вводимых элементов, чтобы после завершения ввода просто разделить теперь уже итоговую сумму на итоговое количество.

3.2.1. Сохранение коллекции данных в векторе

Для вычисления медианы необходимо прочитать и сохранить все оценки за домашние задания, затем отсортировать их и, наконец, выбрать одну (или две) из них, занимающую срединное положение. Для того чтобы это вычисление было удобным и эффективным, нам необходимо знать способ решения следующих проблем.

- Как сохранить ряд значений, которые будут вводиться по одному, не зная заранее их количество.
- Как отсортировать значения после их прочтения.
- Как эффективно получить срединные значения.

Для решения этих проблем можно использовать тип, именуемый `vector`, который предоставляется стандартной библиотекой. Объект типа `vector` содержит последовательность значений некоторого заданного типа, причем при необходимости он может увеличиваться в размере, чтобы включить дополнительные значения, и позволяет получить эффективный доступ к каждому отдельному значению.

Переделку нашей программы начнем с того, что вместо накопления промежуточной суммы вводимых оценок и подсчета их количества (с последующей их потерей) попытаемся поместить их в вектор. Приведем фрагмент исходной версии рассматриваемой программы.

```
// исходная программа (фрагмент).
int count = 0;
double sum = 0;
double x;
// инвариант:
// мы прочитали пока count оценок, и
// переменная sum содержит сумму первых count оценок.
while (cin >> x) {
    ++count;
    sum += x;
}
```

В этом цикле подсчитывается количество прочитанных оценок и сохраняется промежуточная сумма этих значений. Необходимость сохранять эти переменные вместе со всеми вводимыми по одному значениями сделала бы инвариант цикла относительно сложным. Использование же такого типа, как `vector`, существенно упростит нашу задачу.

```
// Новая версия предыдущего фрагмента программы.
double x;
vector<double> homework;

// инвариант: объект homework содержит все оценки
// за домашние задания, введенные до сих пор.
while (cin >> x)
    homework.push_back(x);
```

Базовую структуру нашей программы мы не меняли: она по-прежнему считывает вводимые по очереди значения в переменную `x` до тех пор, пока не встретит признак конца файла или недостоверные данные. Новизна же заключается в обработке этих значений.

Итак, вы видите, что мы определили переменную `homework` типа `vector<double>`. Вектор — это *контейнер* (container), который содержит некоторую коллекцию значений. Все значения в конкретном векторе имеют одинаковый тип, но другие векторы могут хранить объекты иных типов. Определяя вектор, мы должны указать тип значений, которые будут храниться в этом векторе. Наше определение переменной `homework` говорит о том, что она представляет собой вектор, который будет содержать значения типа `double`.

Тип `vector` определен с использованием такого средства языка, как *классы шаблонов* (template classes). Подробнее об определении класса шаблона мы поговорим в главе II. А пока важно понять, что можно отделить понятие типа `vector` от конкретного типа объектов, содержащихся в векторе. Тип включаемых в вектор объектов мы указываем в угловых скобках. Например, объекты типа `vector<double>` — это векторы, которые содержат объекты типа `double`; объекты типа `vector<string>` — это векторы, которые содержат объекты типа `string` и т.д.

Цикл `while` из приведенного выше фрагмента программы считывает значения из стандартного входного устройства и запоминает их в векторе. Как и в исходной версии того же фрагмента программы, данные вводятся в переменную `x` до тех пор, пока не встретится признак конца файла или значение иного типа (не `double`). Новшеством в этом фрагменте является следующая инструкция.

```
homework.push_back(x);
```

Как и в случае с функцией `greeting.size()` (см. раздел 1.2), нетрудно догадаться, что `push_back` — это функция-член, которая определяется в классе `vector` и которую мы “просим”, чтобы она действовала “от имени” объекта `homework`. Мы вызываем эту функцию, передавая ей значение переменной `x`. Назначение функции `push_back` — добавить в конец вектора новый элемент, передаваемый ей в качестве аргумента. Итак, функция `push_back` “проталкивает” свой аргумент в конец вектора, а в качестве побочного эффекта она увеличивает размер данного вектора на единицу.

Поскольку функция `push_back` так удачно подходит к тому, что мы хотим сделать, нетрудно заметить, что ее вызов прекрасно поддерживает наш инвариант цикла, т.е. когда мы покинем `while`-цикл, все оценки за домашние задания будут прочитаны и сохранены в векторе `homework`, чего, собственно, мы и добивались.

Теперь можно подумать и о формировании результата, который должна вывести наша программа.

3.2.2. Генерирование выходных данных

В исходной версии программы (раздел 3.1) мы вычисляли итоговую оценку студента внутри самого выражения вывода результата.

```
streamsize prec = cout.precision();  
cout << "Ваша итоговая оценка равна " << setprecision(3)  
    << 0.2 * midterm + 0.4 * final + 0.4 * sum / count  
    << setprecision(prec) << endl;
```

Здесь переменные `final` и `midterm` содержат оценки по экзаменам, а `sum` и `count` — сумму и количество, соответственно, всех введенных оценок за домашние задания.

Как отмечалось в разделе 3.2.1, простейший способ вычисления медианы — упорядочить наши данные, а затем найти срединное значение или среднее арифметическое от двух срединных значений в случае четного числа элементов. Чтобы было проще понять суть вычисления, попробуем отделить вычисление медианы от кода вывода результата.

Чтобы найти медианное значение, необходимо сначала узнать размер вектора `homework`, причем сделать это нужно, по крайней мере, дважды: один раз для проверки равенства нулю, а затем для вычисления местоположения срединного элемента (элементов). Чтобы избежать необходимости узнавать размер дважды, мы сохраним его значение в локальной переменной.

```
typedef vector<double>::size_type vec_sz;  
vec_sz size = homework.size();
```

Тип `vector`, в свою очередь, определяет тип `vector<double>::size_type`, а также функцию с именем `size`. Эти члены действуют аналогично соответствующим членам в пространстве имен `string`: тип `size_type` — это тип, предназначенный для значений без знака и гарантирующий достаточный объем памяти для хранения размера максимально возможного вектора, а функция `size()` возвращает значение типа `size_type`, которое представляет количество элементов в векторе.

Поскольку нам нужно использовать размер в двух местах программы, мы запомним его значение в локальной переменной. В различных C++-средах для представления размеров используются различные типы, поэтому мы не можем записать соответствующий тип напрямую и быть уверенными, что мы создали независимую от C++-среды программу. Поэтому хорошим стилем программирования считается применение типа `size_type`, определенного библиотекой специально для представления размеров контейнеров, что мы и делаем, присваивая этому специальному типу имя `size`.

В нашем примере этот тип слишком громоздок как для написания, так и для чтения. Чтобы упростить программу, мы использовали языковой инструмент `typedef` (не упоминаемый нами до сих пор). Включая слово `typedef` как часть определения, мы тем самым заявляем, что хотим, чтобы определяемое нами имя было синонимом для указанного типа, а не переменной этого типа. А поскольку наше определение включает слово `typedef`, оно указывает имя `vec_sz` в качестве синонима для типа `vector<double>::size_type`. Имена, определенные посредством `typedef`, имеют ту же область видимости, что и любые другие имена, т.е. мы можем использовать имя `vec_sz` как синоним для типа `size_type` до тех пор, пока не закончится текущая область видимости.

Узнав, как называть тип значения, которое возвращает функция `homework.size()`, мы можем сохранить его в локальной переменной `size` того же типа. То, что мы используем имя `size` в различных целях, не вызывает двусмысленности. Единственный способ узнать размер вектора — поместить обращение к функции `size` с правой стороны от точки, а сам вектор — с левой. Иначе говоря, имя `size`, определенное для локальной переменной, и имя `size`, используемое в качестве операции на векторах, находятся в разных областях видимости. А поскольку эти имена находятся в разных областях видимости, компилятор (да и программист) сможет “понять”, какое именно имя `size` имеется в виду.

Вряд ли вы станете возражать, что бессмысленно искать медиану пустого набора данных, поэтому наш следующий шаг — удостовериться в наличии введенных данных.

```
if (size == 0) {  
    cout << endl << "Необходимо ввести оценки для расчета. "  
              "Пожалуйста, попробуйте снова." << endl;  
    return 1;  
}
```

Мы можем определить, как обстоят дела, проверив значение `size` на равенство нулю. Если наши опасения подтвердятся, самое благоразумное — сообщить об этом пользователю и прекратить выполнение программы. Именно так мы и поступаем,

возвращая значение 1, обозначающее отказ в работе. Как упоминалось в главе 0, система предполагает, что если функция `main` возвращает 0, программа выполнена успешно. Возврат любого другого значения хотя и интерпретируется “по-своему” конкретной C++-средой, но большинство C++-сред интерпретируют любое ненулевое значение как признак отказа.

Убедившись в наличии данных, можно приступить к вычислению медианы. Сначала нужно отсортировать данные, что мы и сделаем путем вызова библиотечной функции.

```
sort(homework.begin(), homework.end());
```

Функция `sort`, определенная в заголовке `<algorithm>`, так переставляет значения в контейнере, чтобы они стояли в порядке неубывания. Мы употребили термин “порядок неубывания” (вместо “порядок возрастания”), поскольку в контейнере могут быть одинаковые элементы.

Аргументы для функции `sort` задают диапазон элементов, подлежащих сортировке. В классе `vector` (как раз для такого случая) определены две функции-члена, `begin` и `end`. Подробнее о функциях `begin` и `end` мы поговорим в разделе 5.2.2, но пока для нас важно знать, что выражение `homework.begin()` означает первый элемент в векторе с именем `homework`, а `homework.end()` — “элемент”, расположенный непосредственно за последним (подробнее см. раздел 8.2.7).

Функция `sort` работает “на территории клиента”: она переставляет значения элементов исходного контейнера, а не создает новый контейнер для помещения в него результата своей работы.

После сортировки вектора `homework` нам нужно найти “серединный” элемент (элементы).

```
vec_sz mid = size/2;
double median;
median = size % 2 == 0 ? (homework[mid] + homework[mid-1]) / 2
                       : homework[mid];
```

Чтобы отыскать середину вектора, начнем с деления переменной `size` на 2. Если количество элементов вектора четное, это деление будет точным. В противном случае (при нечетном количестве) результат будет равен следующему меньшему целому значению.

Вычисление медианы зависит от того, четное или нечетное количество элементов в векторе. Если четное, медиана равна среднему арифметическому от двух элементов, ближайших к середине вектора. В противном случае в середине вектора будет находиться только один элемент, значение которого и является медианой.

В выражении, которое присваивает значение переменной `median`, используется два новых оператора: *оператор остатка* (`remainder operator`), “%”, и *условный оператор* (`conditional operator`), часто называемый “?:”-оператором.

Оператор остатка (%) возвращает остаток от деления своего левого операнда на правый. Если остаток от деления количества элементов на 2 равен 0, значит, программа прочитала четное количество элементов.

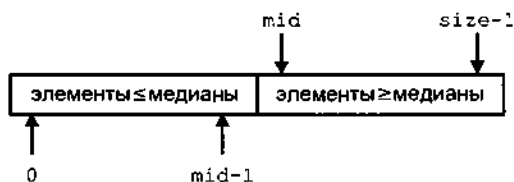
Условный оператор можно рассматривать как сокращенный вариант простого `if-else`-выражения. Сначала он вычисляет выражение, `size % 2 == 0`, которое предшествует символу “?” и служит условием для получения `bool`-значения. Если условие принимает значение `true`, то результатом является значение выражения, стоящего между символами “?” и “:”; в противном случае результат равен значению выражения, стоящего после символа “:”. Следовательно, если пользователь ввел четное количество элементов, программа установит переменную `median` рав-

ной среднему арифметическому от двух “срединных” элементов. Если же пользователь ввел нечетное количество элементов, программа установит переменную `median` равной значению `homework[mid]`. Подобно операторам “&&” и “||”, оператор “?” сначала вычисляет самый крайний слева операнд. А затем, в зависимости от полученного значения, он вычисляет только один из остальных своих операндов.

Ссылки на объекты `homework[mid]` и `homework[mid-1]` демонстрируют способ получения доступа к элементам вектора. Каждому элементу конкретного вектора соответствует некоторое целое значение, именуемое *индексом* (index). Так, например, выражение `homework[mid]` представляет элемент вектора `homework` с индексом `mid`. Первым элементом вектора `homework` является объект `homework[0]`, а последним — объект `homework[size - 1]`.

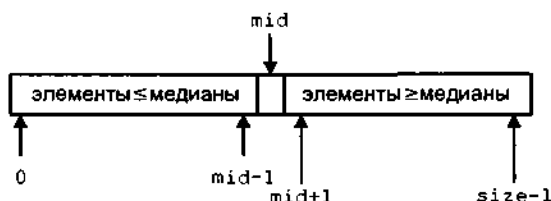
Каждый элемент конкретного вектора представляет собой (неименованный) объект типа, заданного для хранения в данном контейнере. Так, `homework[mid]` — это объект типа `double`, и поэтому он может участвовать в любых операциях, которые поддерживает тип `double`. В частности, чтобы получить среднее арифметическое от значений двух таких объектов, мы можем сложить два элемента вектора, а затем значение суммы разделить на 2.

Зная теперь, как получить доступ к элементам вектора `homework`, нетрудно догадаться, как вычисляется медианное значение. Для начала предположим, что значение переменной `size` четное, поэтому переменная `mid` будет содержать значение `size / 2`. Тогда с каждой стороны (от середины) должно находиться ровно по `mid` элементов вектора `homework`.



Поскольку мы знаем, что в каждой половине вектора `homework` должно находиться ровно по `mid` элементов, легко понять, что индексами двух ближайших к середине элементов являются `mid - 1` и `mid`, а медиана равна среднему арифметическому от этих элементов.

Если количество элементов нечетное, значение `mid`, благодаря усечению, на самом деле равно $(size - 1) / 2$. В этом случае наш отсортированный вектор `homework` можно представить в виде двух сегментов (по `mid` элементов в каждом), разделенных единственным элементом, находящимся посередине. Этот элемент и является медианой.



В любом случае наше вычисление медианы опирается на возможность доступа к любому элементу вектора по известному индексу.

Вычислив медиану, нам останется лишь вычислить и вывести итоговую оценку.

```
streamsize prec = cout.precision();
cout << "Ваша итоговая оценка равна " << setprecision(3)
    << 0.2 * midterm + 0.4 * final + 0.4 * median
    << setprecision(prec) << endl;
```

Конечный вариант нашей программы не намного сложнее версии, приведенной в разделе 3.1, несмотря на то что мы заставили ее проделать больший объем работы. В частности, хотя вектор `homework` должен при необходимости увеличивать свой размер, чтобы включить все полученные студентом оценки, нашей программе совсем не приходится “волноваться” о выделении памяти для хранения этих оценок. Всю эту работу выполняет за нас стандартная библиотека.

Итак, ниже приводится законченная программа в полном объеме. Отдельные ее фрагменты были рассмотрены выше, за исключением, может быть, директив `#include`, соответствующих `using`-объявлений и еще нескольких комментариев.

```
#include <algorithm>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

using std::cin;           using std::sort;
using std::cout;          using std::streamsize;
using std::endl;          using std::string;
using std::setprecision;  using std::vector;

int main()
{
    // Запрашиваем и читаем имя студента.
    cout << "Пожалуйста, введите свое имя: ";
    string name;
    cin >> name;
    cout << "привет, " << name << "!" << endl;

    // Запрашиваем и читаем оценки по экзаменам, проведенным
    // в середине и в конце семестра.
    cout << "Пожалуйста, введите оценки по экзаменам, "
        "проведенным в середине и в конце семестра: ";
    double midterm, final;
    cin >> midterm >> final;

    // Запрашиваем оценки за выполнение домашних заданий.
    cout << "Введите все оценки за выполнение домашних заданий, "
        "завершив ввод знаком конца файла: ";
    vector<double> homework;
    double x;
    // инвариант: объект homework содержит все оценки
    // за домашние задания, введенные до сих пор.
    while (cin >> x)
        homework.push_back(x);

    // Проверяем, ввел ли студент оценки за домашние задания.
    typedef vector<double>::size_type vec_sz;
    vec_sz size = homework.size();
    if (size == 0) {
        cout << endl << "Необходимо ввести оценки для расчета. "
            "Пожалуйста, попробуйте снова." << endl;
    }
}
```

```

    } return 1;

// Сортируем оценки.
sort(homework.begin(), homework.end());

// Вычисляем медианную оценку вектора homework.
vec_sz mid = size/2;
double median;
median = size % 2 == 0 ? (homework[mid] + homework[mid+1]) / 2
    : homework[mid];

// Вычисляем и выводим итоговую оценку.
streamsize prec = cout.precision();
cout << "Ваша итоговая оценка равна " << setprecision(3)
    << 0.2 * midterm + 0.4 * final + 0.4 * median
    << setprecision(prec) << endl;

return 0;
}

```

3.2.3. Еще несколько замечаний

Некоторые аспекты этой программы заслуживают особого внимания. Прежде всего, следует лучше пояснить, почему мы завершаем программу, если вектор `homework` оказывается пустым. Если медиана пустой коллекции значений не определена — тогда просто непонятно, что в этом случае она может означать. Следовательно, завершение программы можно считать справедливым решением — если мы не знаем, что делать, то лучше ретироваться. Но важно иметь представление о том, что может случиться, если продолжить выполнение программы. Если входные данные окажутся пустыми, а мы решим пренебречь проверкой этого факта, то код вычисления медианы будет обречен на неудачу. Почему?

Если мы не введем ни одного элемента, то выражение `homework.size()` и, следовательно, переменная `size` будут равны 0. Индекс `mid` тоже будет равен 0. Обращаясь к объекту `homework[mid]`, мы тем самым обращались бы к первому элементу (которому соответствует индекс 0) вектора `homework`. Однако вспомните: в векторе `homework` элементов не существует вовсе! Попытка обратиться к элементу `homework[0]` будет, мягко говоря, напрасной. Векторы не обеспечивают проверку того, находится ли индекс в допустимом диапазоне. Такая проверка — целиком на совести программиста.

Еще один момент, на который хотелось бы обратить ваше внимание, связан с тем, что тип `vector<double>::size_type`, подобно всем “размерным” типам стандартной библиотеки, является *целочисленным типом без знака* (unsigned integral type). Такие типы вообще неспособны хранить отрицательные значения; они хранят значения по модулю 2^n , где n зависит от C++-среды. Так, например, никогда не имеет смысла проверять выражение `homework.size() < 0`, поскольку это сравнение всегда будет генерировать значение `false`.

Более того, если обычные целые значения и целые без знака объединить в одном выражении, обычные целые будут преобразованы в значения типа без знака. Поэтому такое выражение, как `homework.size() - 100`, сгенерирует результат без знака, а это значит, что оно также не может быть меньше нуля, даже если на самом деле `homework.size() < 100`.

Наконец, стоит отметить неплохую эффективность выполнения нашей программы, несмотря на то что объект `vector<double>` увеличивает свой размер по мере поступления входных данных, а не использует заранее выделенную память нужного объема.

Мы можем быть уверены в эффективности этой программы, поскольку стандарт C++ накладывает соответствующие ограничения на реализацию библиотеки. Стандартная библиотека должна не только отвечать техническим требованиям по поведению, но и достигать определенного уровня производительности. Каждая согласующаяся со стандартом C++-среда должна выполнить следующее:

- реализовать класс `vector` таким образом, чтобы включение в вектор большого количества элементов отражалось на эффективности пропорционально (но не хуже) количеству элементов;
- реализовать функцию `sort` таким образом, чтобы она работала в среднем не медленнее вычисления выражения $n \log(n)$, где n — количество сортируемых элементов.

Итак, наша законченная программа гарантированно будет выполняться в любой согласующейся со стандартом C++-среде в течение $n \log(n)$ -времени (или быстрее). Действительно, стандартная библиотека была разработана с учетом высоких требований к производительности. Базовый язык C++ ориентирован на использование в приложениях, весьма критичных к производительности, при этом акцент на достижение скоростных характеристик распространяется также и на стандартную библиотеку.

3.3. Резюме

Локальные переменные инициализируются по умолчанию, если они определены без использования явного инициализатора. Инициализация по умолчанию для переменной любого встроенного типа означает, что ее значение не определено. Неопределенные значения можно использовать только в качестве левого операнда оператора присваивания.

Определения типов

`typedef Тип имя;` Определяет *имя* как синоним для элемента *Тип*

Тип `vector`, определенный в заголовке `<vector>`, представляет собой библиотечный тип, который служит контейнером, содержащим последовательность значений заданного типа. Размеры векторов увеличиваются динамически. Ниже перечислены следующие важные операции над векторами.

`vector<T>::size_type`

Тип, который способен содержать количество элементов в максимально возможном векторе

`v.begin()` Функция-член. Возвращает значение первого элемента в векторе `v`

`v.end()` Функция-член. Возвращает значение “элемента”, следующего за последним элементом в векторе `v`

`vector<T> v;` Создает пустой вектор, который может содержать элементы типа `T`

`v.push_back(e)` Функция-член. Увеличивает вектор на один элемент, инициализированный значением параметра `e`

`v[i]` Возвращает значение, содержащееся в позиции `i`

`v.size()` Функция-член. Возвращает количество элементов в векторе `v`

Другие библиотечные средства

`sort(b, e)` Переставляет элементы в диапазоне `[b, e)` в неубывающем порядке. Функция определена в заголовке `<algorithm>`

<code>max(e1, e2)</code>	Возвращает большее из значений выражений <code>e1</code> и <code>e2</code> , которые должны иметь одинаковый тип. Функция определена в заголовке <code><algorithm></code>
<code>while (cin >> x)</code>	Считывает значение соответствующего типа в переменную <code>x</code> и проверяет состояние потока. Если поток находится в состоянии неисправности, проверка признается неудачной; в противном случае — успешной, что позволяет выполнить тело <code>while</code> -цикла
<code>s.precision(n)</code>	Устанавливает точность потока <code>s</code> равной <code>n</code> для следующих выходных данных (или оставляет ее прежней, если <code>n</code> опущено). Возвращает предыдущее значение точности
<code>setprecision(n)</code>	Возвращает значение, которое, будучи записанным в выходной поток <code>s</code> , имеет эффект вызова функции <code>s.precision(n)</code> . Функция <code>setprecision(n)</code> определена в заголовке <code><iomanip></code>
<code>streamsize</code>	Тип значения, ожидаемого функцией <code>setprecision</code> и возвращаемого функцией <code>precision</code> . Определен в заголовке <code><iostream></code>

Упражнения

- 3.0. Скомпилируйте, выполните и протестируйте программы, приведенные в этой главе.
- 3.1. Предположим, нам нужно найти медиану некоторой коллекции значений. Допустим, мы уже прочитали несколько значений и не знаем, сколько еще осталось. Докажите, что мы не можем отбросить ни одно из уже прочитанных значений. *Подсказка:* допустим, что мы отбросили некоторое значение, а затем оказалось, что для значений из непрочитанной (а следовательно, неизвестной) части нашей коллекции *медианой* должно быть значение, которое мы отбросили.
- 3.2. Напишите программу, которая должна вычислить, сколько раз каждое отдельное слово содержится во введенных данных.
- 3.3. Напишите программу для вывода длины самого длинного и самого короткого `string`-значений во введенных данных.
- 3.4. Напишите программу, которая должна отслеживать оценки нескольких студентов сразу. Программа могла бы использовать синхронно два вектора: первый будет хранить имена студентов, а второй — итоговые оценки, вычисляемые после считывания введенных данных. Пока используйте фиксированное количество оценок за выполнение домашних заданий. В разделе 4.1.3 мы узнаем, как обрабатывать переменное количество оценок вперемешку с именами студентов.
- 3.5. При вычислении средней арифметической оценки, приведенном в разделе 3.1, можно столкнуться с проблемой деления на ноль, если студент не введет ни одной оценки. Деление на ноль не определено в C++, а это значит, что в каждой конкретной C++-среде эта проблема решается по-своему. Как поступает в этом случае ваша C++-среда? Перепишите упомянутую выше программу так, чтобы ее поведение не зависело от реакции конкретной C++-среды на деление на ноль.

ОРГАНИЗАЦИЯ ПРОГРАММ И ДАННЫХ

Программа, приведенная в разделе 3.2.2, больше по размеру, чем, возможно, вам бы того хотелось, но она могла быть еще больше, если бы мы не использовали вектор, `string`-переменную и функцию `sort`. Эти библиотечные средства, подобно другим, уже опробованным нами выше, имеют ряд общих свойств. Каждое из этих средств

- решает проблему конкретного типа;
- не зависит от большинства других;
- имеет имя.

Наши собственные программы обладают только первым из перечисленных трех свойств. Этот недостаток простителен лишь для маленьких программ, но по мере решения все более серьезных проблем вскоре становится ясно, что наши программные решения будут неуправляемыми, если их не разбить на независимые именованные части.

Подобно большинству языков программирования, C++ предлагает два фундаментальных способа организации больших программ: функции (иногда их называют подпрограммами) и структуры данных. Кроме того, C++ позволяет программистам объединять функции и структуры данных в одно целое, именуемое классом, знакомство с которым мы начнем в главе 9.

Узнав, как использовать функции и структуры данных для организации вычислений, мы также должны научиться разделять свои программы на файлы, которые можно компилировать отдельно, и снова объединять их уже после компиляции. В последней части этой главы показано, как в C++ поддерживается механизм раздельной компиляции.

4.1. Организация вычислений

Начнем, пожалуй, с написания функции вычисления итоговой оценки студента на основе полученных оценок на экзаменах в середине и конце семестра, а также обобщенной оценки за выполнение домашних заданий. При этом допустим, что мы уже вычислили обобщенную оценку, исходя из оценок за отдельные домашние задания, причем обобщенная оценка вычислялась как среднее арифметическое или как медианное значение. Помимо упомянутого допущения, эта функция должна использовать ту же стратегию вычисления итоговой оценки: домашние задания и результат последнего экзамена “вносят” по 40% в итоговый результат, а результат промежуточного экзамена — оставшиеся 20%.

Если общий процесс вычисления оказался разбитым на отдельные составляющие, то предпочтительнее собрать их под “крышей” одной функции. Очевидно, что в этом случае (вместо повторного выполнения отдельных частей) при необходимости решения аналогичной задачи мы можем просто использовать одну (уже готовую) функцию. Использование функций не только сокращает общие затраты на программирование, но также значительно облегчает процесс внесения изменений в вычисления. Предположим, мы хотим изменить политику расчета итоговой оценки. Если бы нам пришлось просматривать каждую программу, написанную нами ранее, чтобы найти ту часть, где рассчитывается итоговая оценка с учетом веса каждой ее составляющей, у нас бы быстро опустелись руки.

Для подобных вычислений необходимо отметить даже более существенное преимущество использования функций. Любая функция имеет имя. Если некоторому вычислению присвоить имя, мы в этом случае можем думать о нем более абстрактно, т.е. мы можем *больше* думать о том, *что* оно делает, и *меньше* — *как* оно работает. Если нам удастся идентифицировать важные части наших проблем, а затем создать именованные “кусочки” наших программ, которые соответствуют этим частям, то наши программы станут проще для понимания, а проблемы — легче для решения. Вот как выглядит функция, которая вычисляет итоговые оценки в соответствии с определенной выше политикой.

```
// вычисляем итоговую оценку студента на основе оценок,  
// полученных на экзаменах в середине и конце семестра,  
// а также на основе обобщенной оценки за выполнение домашних заданий.  
double grade(double midterm, double final, double homework)  
{  
    return 0.2 * midterm + 0.4 * final + 0.4 * homework;  
}
```

До сих пор все функции, которые мы определяли, имели имя `main`. Большинство других функций определяется аналогичным образом, т.е. посредством задания типа возвращаемого значения, за которым сначала указывается имя функции, затем *список параметров* (parameter list), заключенный в круглые скобки, и, наконец, тело функции, заключенное в фигурные скобки. Эти правила несколько усложняются для функций, которые возвращают значения, указывающие на другие функции (подробнее см. раздел A.1.2).

В этом примере параметрами являются `midterm`, `final` и `homework`, причем все они имеют тип `double`. Параметры ведут себя как переменные, которые локальны для данной функции, т.е. при вызове функции они создаются, а при выходе из нее — разрушаются.

Подобно любым другим переменным, параметры должны быть определены до их использования. В отличие от других переменных, их определение не означает немедленное создание: они создаются лишь при вызове функции. Следовательно, вызывая функцию, мы должны предоставить ей соответствующие *аргументы* (arguments), которые используются для инициализации параметров в момент, когда функция начинает выполняться. Например, в разделе 3.1 мы вычисляли оценку, используя следующую инструкцию.

```
cout << "ваша итоговая оценка равна " << setprecision(3)  
      << 0.2 * midterm + 0.4 * final + 0.4 * sum / count  
      << setprecision(prec) << endl;
```

Если бы в нашем распоряжении была функция `grade`, предыдущую инструкцию мы могли бы записать по-другому.

```
cout << "ваша итоговая оценка равна " << setprecision(3)  
      << grade(midterm, final, sum / count)  
      << setprecision(prec) << endl;
```

Мы должны так предоставить функции ее аргументы, чтобы они не только соответствовали по типу параметрам вызываемой нами функции, но и были указаны в таком же порядке. Следовательно, вызывая функцию `grade`, в качестве первого аргумента нужно передать значение оценки, полученной в середине семестра, в качестве второго — значение оценки, полученной в конце семестра, а в качестве третьего — значение обобщенной оценки за выполнение домашних заданий.

Аргументы могут быть выражениями, например `sum / count`, а не просто переменными. В общем случае каждый аргумент используется для инициализации соответствующего параметра, после чего поведение параметров ничем не отличается от поведения обычных локальных переменных внутри функции. Так, например, при вызове функции `grade(midterm, final, sum / count)` ее параметры не обращаются непосредственно к самим аргументам, а инициализируются копиями значений аргументов. Такой механизм часто называется *вызовом по значению* (call by value), поскольку параметр принимает копию значения аргумента.

4.1.1. Вычисление медиан

Еще одна проблема, которую мы решили в разделе 3.2.2 и которая, как нетрудно предположить, обязательно потребует решения в других контекстах, состоит в вычислении медианы вектора. В разделе 8.1.1 будет показано, как определить функцию в более общем виде, чтобы она смогла работать с вектором значений любого типа. А пока ограничимся рассмотрением вектора типа `vector<double>`.

Создание функции начнем с той части программы (см. раздел 3.2.2), в которой вычисляется медиана, и внесем в нее ряд изменений.

```
// вычисляем медиану вектора vector<double>.
// Обратите внимание на то, что вызов этой функции копирует
// аргумент vector целиком.
double median(vector<double> vec)
{
    typedef vector<double>::size_type vec_sz;
    vec_sz size = vec.size();
    if (size == 0)
        throw domain_error("медиана пустого вектора.");
    sort(vec.begin(), vec.end());
    vec_sz mid = size/2;
    return size % 2 == 0 ? (vec[mid] + vec[mid-1]) / 2 : vec[mid];
}
```

Одно изменение состоит в том, что мы присвоили нашему вектору имя `vec`, а не `homework`. Как-никак, наша функция может вычислять медиану от любого набора чисел, а не только от оценок за домашние задания. Мы также удалили переменную `median`, поскольку она нам больше не нужна: ведь мы можем вернуть медиану сразу после ее вычисления. Мы по-прежнему используем в качестве переменных `size` и `mid`, но сейчас они являются локальными для функции `median` и, следовательно, недоступными (и нерелевантными) вне ее. При вызове функции `median` эти переменные создаются, а при выходе из нее — разрушаются. Мы определяем `vec_sz` как локальное имя типа, поскольку не хотим никаких конфликтов ни с кем, кто захочет использовать это имя для другой цели.

Самое существенное изменение — обработка случая, когда вектор оказывается пустым. В разделе 3.2.2 в этом случае мы знали о необходимости выразить недовольство тому, кто выполняет нашу программу. Мы также знали, кем является этот пользователь, и поэтому нам было ясно, какая жалоба будет уместна в нашей программе. Однако в новой версии программы нам неизвестно, кто будет ее использовать и с какой целью, поэтому нужно подумать о более общей форме “жалобы” на пользователя программы. Такой более общей формой является *генерирование исключительной ситуации*, или *исключения* (throw an exception), если вектор окажется пустым.

Когда программа генерирует исключение, ее выполнение прекращается в той части, где встретилось слово throw, и продолжается в другой части, но уже с использованием *объекта исключения* (exception object), содержащего информацию, которую можно использовать для обработки этого исключения.

Самой существенной информацией, передаваемой в подобных случаях из одной части программы в другую, является сам факт генерирования исключения. Этого факта (вместе с типом объекта исключения) обычно достаточно, чтобы “понять”, что нужно сделать в такой ситуации. В данном примере исключение, генерируемое нашей программой, имеет тип domain_error. Этот тип определяется стандартной библиотекой в заголовке <stdexcept> и предназначен для сообщения о том, что аргумент функции не принадлежит набору значений, которые эта функция способна принять. При создании объекта типа domain_error для генерирования исключения мы можем предоставить ему string-значение с описанием произошедшей “неприятности”. Программа, которая перехватит исключение, может, как показано в разделе 4.2.3, использовать это string-значение в своем диагностическом сообщении.

Обратите внимание на еще один аспект, связанный с характером поведения функций. Вызывая функцию, мы можем считать ее параметры локальными переменными, начальными значениями которых являются значения соответствующих аргументов. Тогда выходит, что вызов функции включает копирование аргументов в ее параметры. В частности, при вызове функции median вектор, используемый в качестве аргумента, будет скопирован в параметр vec.

При использовании функции median полезно скопировать аргумент в параметр, даже если на это потребуются значительное время, поскольку функция median изменяет значение своего параметра благодаря вызову функции sort. При копировании аргумента изменения, внесенные функцией sort, не будут распространяться на автора вызова функции. Такое поведение имеет смысл, поскольку вызов функции median для заданного vector-значения не должен вносить изменения в сам вектор.

4.1.2. Пересмотр политики вычисления оценок

Функция grade, представленная в разделе 4.1, предполагает, что мы уже имеем дело с обобщенной оценкой студента за выполнение домашних заданий, а не просто с набором оценок за отдельные домашние задания. Способ получения этой оценки является частью нашей политики: в разделе 3.1 мы вычисляли среднее арифметическое значение, а в разделе 3.2.2 — медиану. Поэтому вполне логично будет выразить эту часть нашей оценочной политики в виде функции, используя те же строки кода, что и в разделе 4.1.

```
// Вычисляем итоговую оценку студента на основе оценок,  
// полученных на экзаменах в середине и конце семестра,  
// а также на основе вектора оценок за выполнение домашних заданий.  
// Эта функция не копирует свой аргумент, поскольку  
// функция median делает это за нас.  
double grade(
```

```

    double midterm, double final, const vector<double>& hw)
{
    if (hw.size() == 0)
        throw domain_error(
            "Студент не сделал ни одного домашнего задания ");
    return grade(midterm, final, median(hw));
}

```

В этой функции имеет смысл остановиться на трех моментах.

Прежде всего, обратите внимание на тип `const vector<double>&`, который мы задали для третьего аргумента. Этот тип часто называется “ссылкой на `const`-вектор `double`-значений”. Заявляя, что некоторое имя есть *ссылка* (reference) на некоторый объект, мы имеем в виду, что оно является еще одним именем для этого объекта. Например, следующие строки кода

```

vector<double> homework;
vector<double>& hw = homework; // hw – синоним для homework.

```

говорят о том, что `hw` — еще одно имя для объекта `homework`. С этого момента все, что мы делаем с объектом, именуемым `hw`, эквивалентно выполнению аналогичных действий с объектом `homework` и наоборот. Следующий код (с использованием слова `const`)

```

// chw – синоним для объекта homework,
// предназначенный только для чтения.
const vector<double>& chw = homework;

```

по-прежнему означает, что `chw` — еще одно имя для объекта `homework`, но модификатор `const` “обещает”, что мы не будем выполнять никаких действий с объектом `chw`, которые могли бы изменить его значение.

Поскольку ссылка — это еще одно имя для исходного объекта, такого понятия, как ссылка на ссылку, не существует. Определение ссылки на ссылку имеет такой же эффект, как определение ссылки на исходный объект. Например, если мы запишем следующий код

```

// hw1 и chw1 – синонимы для объекта homework.
// Ссылка chw1 предназначена только для чтения.
vector<double>& hw1 = hw;

const vector<double>& chw1 = chw;

```

то `hw1` (как и `hw`) — еще одно имя для объекта `homework`, а `chw1` (как и `chw`) — еще одно имя для объекта `homework`, не позволяющее доступ для записи.

Определяя *неконстантную* ссылку, т.е. ссылку, которая позволяет запись, мы не можем сделать ее ссылкой на `const`-объект или `const`-ссылку, поскольку это потребует разрешения на отрицание константности. Следовательно, мы не можем записать

```

vector<double>& hw2 = chw; // ошибка: требует доступа
                        // для записи в объект chw,

```

поскольку “обещали” не модифицировать объект `chw`.

Аналогично, когда мы утверждаем, что некоторый параметр имеет тип `const vector<double>&`, запрашиваем у C++-среды прямой доступ к соответствующему аргументу, отказываясь от его копирования, и при этом обещаем, что не будем изменять значение этого параметра (ведь в противном случае аргумент также был бы изменен). Поскольку параметр является ссылкой на `const`-объект, мы можем вызвать функцию `grade` от имени как `const`-, так и не-`const`-векторов. Коль параметр является ссылкой, мы избегаем расхода системных ресурсов на копирование аргумента.

Еще один аспект, на который нам бы хотелось обратить ваше внимание, — это сама функция `grade`. Эта функция (та, версия которой приведена первоначально в разделе 4.1) “спокойно” “носит” имя `grade`, несмотря на то что она вызывает другую функцию `grade`. Понятие, предусматривающее наличие сразу нескольких функций с одинаковыми именами, называется *перегрузкой* (*overloading*) и достаточно широко используется во многих C++-программах. В применении двух функций с одинаковыми именами нет никакой двусмысленности, поскольку при вызове функции `grade` мы предоставляем список аргументов, а C++-среда по типу третьего аргумента способна “догадаться”, какую именно функцию `grade` мы имеем в виду.

Наконец, заметьте, что мы проверяем, равно ли выражение `homework.size()` нулю, даже несмотря на то что знаем, что функция `median` сделает это за нас. Дело в том, что если функция `median` обнаружит, что мы хотим вычислить медиану пустого вектора, она сгенерирует исключение, которое включает сообщение *Медиана пустого вектора*. Это сообщение может оказаться не слишком информативным для того, кто с помощью нашей программы вычисляет итоговые оценки студентов. Следовательно, мы генерируем собственное исключение, которое, смеем надеяться, даст пользователю ключ к пониманию причины “неожиданного” поведения программы.

4.1.3. Считывание оценок за выполнение домашних заданий

Еще одна проблема, которую мы должны решить в нескольких контекстах, состоит в считывании в вектор оценок за домашние задания.

В разработке поведения такой функции существует одна проблема: ей нужно вернуть два значения сразу. Одно значение — это, конечно, считанные оценки за домашние задания. Второе же должно служить индикатором успешной (или нет) попытки ввода данных.

Нужно признать, что прямого способа вернуть из функции более одного значения не существует. Однако косвенным путем это все-таки можно сделать, а именно передать функции параметр-ссылку на объект, в котором можно поместить один из желаемых результатов. Такая стратегия весьма распространена для функций чтения входных данных, поэтому и мы воспользуемся ею. Тогда наша функция будет выглядеть следующим образом.

```
// Считываем оценки за домашние задания из входного потока
// в вектор типа vector<double>.
istream& read_hw(istream& in, vector<double>& hw) {
    // Эту часть нам предстоит еще заполнить.
    return in;
}
```

В разделе 4.1.2 была приведена программа с параметром `const vector<double>&`; сейчас же мы отказываемся от использования модификатора `const`. Параметр-ссылка без слова `const` обычно свидетельствует о намерении модифицировать объект, который является аргументом функции. Например, при выполнении следующих инструкций

```
vector<double> homework;
read_hw(cin, homework);
```

тот факт, что второй параметр функции `read_hw` является ссылкой, должен натолкнуть нас на мысль, что вызов функции `read_hw` может изменить значение объекта `homework`.

Поскольку мы ожидаем, что функция будет модифицировать свои аргументы, ее нельзя вызвать посредством лишь одного выражения. Вместо этого мы должны передать параметру-ссылке *l-значение* (*lvalue*) аргумента. Упомянутое *l-значение* представ-

ляет собой *сохраняемое* (в противоположность *временному*) значение объекта (т.е. выражение, которое может находиться в левой части оператора присваивания и семантически представляет собой адрес размещения переменной, массива, элемента структуры и т.п.). Например, любая переменная является *l*-значением, как и ссылка или результат вызова функции, которая возвращает ссылку. Выражение, генерирующее арифметическое значение, например `sum / count`, не является *l*-значением.

Оба параметра функции `read_hw` являются ссылками, поскольку мы ожидаем, что функция изменит состояние обоих аргументов. Даже если мы и не знаем подробности работы входного потока `cin`, нам достаточно того, что библиотека определяет его в виде некоторой структуры данных, которая сохраняет все, что нужно знать библиотеке о состоянии нашего входного файла. При считывании входных данных из стандартного входного файла изменяется состояние этого файла, поэтому и значение `cin` должно логически измениться.

Обратите внимание на то, что функция `read_hw` возвращает значение переменной `in`. Более того, она обращается с ней, как с ссылкой. В действительности, мы говорим, что нам передан объект, который мы не собираемся копировать, а возвратим тот же объект, снова-таки не копируя его. Поскольку функция возвращает поток, запись

```
if (read_hw(cin, homework)) { /* ... */ }
```

может служить сокращенным вариантом записи следующих инструкций.

```
read_hw(cin, homework);  
if (cin) { /* ... */ }
```

Теперь мы можем подумать о том, как прочитывать оценки, полученные за домашние задания. Совершенно ясно, что мы хотим прочесть столько оценок, сколько их существует (а не фиксированное их количество), поэтому может показаться, что мы могли бы просто записать следующие инструкции.

```
// Наш первый блин комом.  
double x;  
while (in >> x)  
    hw.push_back(x);
```

Эта стратегия не работает, причем по двум причинам.

Первая причина — мы не определили объект `hw`, поэтому неизвестно, какие данные могли бы туда уже попасть. Хотя мы знаем, что если наша функция используется для обработки оценок за домашние задания многих студентов, переменная `hw` могла бы содержать оценки предыдущего студента. Мы же можем решить эту проблему, вызвав перед началом нашей работы функцию `hw.clear()`.

Вторая причина провала нашей стратегии — мы не вполне представляем, где нужно остановить цикл. Мы можем продолжать чтение до тех пор, пока это возможно, но здесь у нас возникает проблема. Есть две причины, по которым мы можем не прочитать очередную оценку: обнаружен признак конца файла или прочитано данное, которое не является оценкой.

В первом случае автор вызова нашей функции может подумать, что мы прочитали признак конца файла. Эта мысль будет справедливой, но в то же время и обманчивой, поскольку признак конца файла обнаружится только после того, как мы успешно прочитаем все данные. Обычно признак конца файла означает, что попытка ввести данные потерпела неудачу.

Во втором случае (при обнаружении данного, не являющегося оценкой) библиотека отметит входной поток как находящийся в состоянии отказа. Это означает, что бу-

душие запросы на ввод данных будут обречены на неудачу, как если бы мы обнаружили конец файла. Следовательно, автор вызова нашей функции может подумать, что что-то не в порядке с вводимыми данными, в то время как единственная проблема заключалась в том, что за последней оценкой последовало данное, не являющееся оценкой за домашнее задание.

В любом случае нам бы хотелось сделать вид, что мы ничего не видели после получения последней оценки за домашнее задание. Такое притворство облегчает ситуацию: обнаружение признака конца файла означает отсутствие непрочитанных входных данных, а при попытке прочитать данное, не являющееся оценкой, библиотека оставит его непрочитанным до следующей попытки ввода данных. Следовательно, все, что мы должны сделать, — “велеть” библиотечным средствам ввода игнорировать все, что привело попытку ввести данные к провалу, будь то признак конца файла или неверное значение. Упомянутое указание библиотеке выражается в виде вызова функции `in.clear()`, позволяющей сбросить состояние ошибки объекта `in`, после чего библиотека сможет продолжать ввод данных, несмотря на сбой.

И еще. Ведь не исключено обнаружение признака конца файла или неверного значения еще до попытки прочитать *первую* оценку за домашнее задание. В этом случае мы должны оставить входной поток в абсолютном покое, чтобы по неосторожности не соблазнить автора вызова нашей функции на попытку прочитать несуществующие входные данные в какой-то момент в будущем.

Итак, настало время представить функцию `read_hw` в полном объеме.

```
// Считываем оценки за домашние задания из входного потока
// в вектор типа vector<double>.
istream& read_hw(istream& in, vector<double>& hw)
{
    if (in) {
        // Избавляемся от предыдущего содержимого.
        hw.clear();

        // Считываем оценки за домашние задания.
        double x;
        while (in >> x)
            hw.push_back(x);

        // Очищаем поток, чтобы средства ввода данных
        // были готовы принять оценки следующего студента.
        in.clear();
    }
    return in;
}
```

Обратите внимание на то, что функция-член `clear` ведет себя совершенно по-разному для `istream`- и `vector`-объектов. Для `istream`-объектов она сбрасывает любые признаки ошибок, что позволяет продолжать ввод данных, а для `vector`-объектов она отбрасывает любое содержимое, которое могло бы быть в векторе, оставляя нам пустой вектор.

4.1.4. Три вида параметров функции

Хотелось бы остановиться на следующем. Мы определили три функции: `median`, `grade` и `read_hw`, которые работают с векторами `homework`. Каждая из этих функций обрабатывает соответствующий параметр совершенно не так, как это делают остальные, и каждая обработка преследует свою цель.

Параметр функции `median` (см. раздел 4.1.1) имеет тип `vector<double>`. Следовательно, при вызове этой функции аргумент копируется, несмотря на его размеры (а ведь это может быть вектор огромного размера). Невзирая на такую неэффективность, тип `vector<double>` выбран правильно, поскольку он гарантирует, что вычисление медианы вектора не изменит сам вектор. Функция `median` сортирует значения, хранящиеся в ее параметре-векторе, с помощью функции `sort`. Если бы при вызове этой функции аргумент не копировался, то при выполнении `median(homework)` значение вектора `homework` изменилось бы.

Функция `grade`, которая принимает вектор с оценками за домашние задания (см. раздел 4.1.2), использует для этого параметра тип `const vector<double>&`. Символ `&` (в обозначении этого типа) не велит C++-среде копировать аргумент, при этом модификатор `const` “обещает”, что программа не изменит значение этого параметра. Такие параметры — важное средство повышения эффективности работы программ. Они очень полезны, когда функция не должна менять значение параметра, который имеет тип `vector` или `string` и способен принимать такие значения, что их копирование может потребовать немалых затрат времени. Обычно не стоит беспокоиться об использовании `const`-ссылок для параметров таких простых встроенных типов, как `int` или `double`. Такие маленькие объекты обычно настолько быстро копируются, что системные затраты на передачу их по значению весьма незначительны.

Параметр функции `read_hw` имеет тип `vector<double>&`, причем, заметьте, без использования модификатора `const`. И снова-таки, символ `&` велит C++-среде напрямую связать параметр с аргументом, не выполняя операции копирования аргумента. Однако в этом случае причина обойтись без копирования состоит в *намерении* функции изменить значение аргумента.

Аргументы, соответствующие параметрам, которые по типу являются неконстантными ссылками, должны представлять собой *l*-значения, т.е. они не должны быть “временно живущими” объектами. Аргументы, которые передаются по значению или “вынуждены” подчиняться ограничениям, накладываемым типом `const`-ссылки, могут содержать любые значения. Предположим, у нас есть функция, которая возвращает пустой вектор.

```
vector<double> emptyvec()
{
    vector<double> v; // Вектор не содержит элементов.
    return v;
}
```

Мы могли бы вызвать эту функцию и использовать результат вызова в качестве аргумента для нашей второй функции `grade` из раздела 4.1.2.

```
grade(midterm, final, emptyvec());
```

При выполнении функция `grade` должна немедленно сгенерировать исключение, поскольку ее аргумент пуст. Однако сам вызов функции `grade` таким способом был бы синтаксически легален. При вызове функции `read_hw` оба ее аргумента должны быть *l*-значениями, поскольку оба параметра определены как неконстантные ссылки. Если функции `read_hw`

```
read_hw(cin, emptyvec()); // Ошибка: параметр emptyvec() -
                          // не l-значение.
```

передать вектор, не являющийся *l*-значением, компилятору это будет не по нраву, поскольку неименованный вектор, который создается при обращении к функции

emptyvec, будет разрушен сразу же по выходу из функции read_hw. Если бы нам все-таки позволили сделать такое обращение к функции (предположим, закрыл бы компилятор глаза на такие “мелочи”), то в результате мы сохранили бы входные данные в объекте, к которому нельзя получить доступ!

4.1.5. Использование функций для вычисления итоговой оценки студента

Теперь применим все рассмотренные выше функции в программе вычисления итоговых оценок студентов; точнее, в реконструкции программы, приведенной в разделе 3.2.2.

```
// include-директивы и using-объявления для использования
// библиотечных средств.
// Код функции median из раздела 4.1.1.
// Код функции grade(double, double, double) из раздела 4.1.
// Код функции grade(double, double, const vector<double>&)
// из раздела 4.1.2.
// Код функции read_hw(istream&, vector<double>&)
// из раздела 4.1.3.
int main()
{
    // Запрашиваем и считываем имя студента.
    cout << "Пожалуйста, введите свое имя: ";
    string name;
    cin >> name;
    cout << "Привет, " << name << "!" << endl;

    // Запрашиваем и считываем оценки по экзаменам, проведенным
    // в середине и конце семестра.
    cout << "Пожалуйста, введите оценки по экзаменам, "
    << "проведенным в середине и конце семестра: ";
    double midterm, final;
    cin >> midterm >> final;

    // Запрашиваем оценки за выполнение домашних заданий.
    cout << "Введите все оценки за выполнение домашних заданий, "
    << "завершив ввод признаком конца файла: ";

    vector<double> homework;

    // Считываем оценки за выполнение домашних заданий.
    read_hw(cin, homework);

    // Вычисляем итоговую оценку, если это возможно.
    try {
        double final_grade = grade(midterm, final, homework);
        streamsize prec = cout.precision();
        cout << "Ваша итоговая оценка равна " << setprecision(3)
        << final_grade << setprecision(prec) << endl;
    } catch (domain_error) {
        cout << endl << "Вы должны ввести свои оценки. "
        << "пожалуйста, попробуйте снова." << endl;
        return 1;
    }
    return 0;
}
```

Изменения, внесенные в более раннюю версию программы, связаны с тем, как мы считываем оценки за выполнение домашних заданий и как вычисляем и записываем результат.

После запроса на ввод оценок за домашние задания мы вызываем нашу функцию `read_hw`, чтобы прочитать вводимые пользователем данные. Инструкция `while` внутри функции `read_hw` в цикле считывает оценки до тех пор, пока не будет обнаружен признак конца файла или данное другого типа (не `double`).

Важнейшее новшество в этом примере — инструкция `try`. Она пытается выполнить инструкции, заключенные в фигурные скобки `{}`, расположенные за ключевым словом `try`. Если в любой из этих инструкций возникнет исключение типа `domain_error`, их выполнение прекращается, а управление передается другому набору инструкций, заключенных в другие фигурные скобки, которые являются частью инструкции `catch`. Эта инструкция начинается с ключевого слова `catch` и указывает тип перехватываемого ею исключения.

Если инструкции, расположенные между `try` и `catch`, выполняются без генерирования исключения, программа полностью игнорирует выполнение инструкции `catch` и продолжает работу со следующей (после `catch`) инструкцией (в данном примере инструкции `return 0;`).

При написании `try`-инструкции необходимо всегда помнить о возможных побочных эффектах и о том, когда они могут возникнуть. Мы должны быть готовы к тому, что любая инструкция, расположенная между `try` и `catch`, может сгенерировать исключение. В этом случае любые вычисления, которые должны выполняться после “виновницы в исключении”, игнорируются. Таким образом, важно понимать, что реальная последовательность выполнения инструкций необязательно должна совпадать с последовательностью инструкций, записанной в тексте программы.

Предположим, мы бы могли кратко записать блок вывода информации следующим образом.

```
// Этот пример не работает.
try {
    streamsize prec = cout.precision();
    cout << "ваша итоговая оценка равна " << setprecision(3)
         << grade(midterm, final, homework)
         << setprecision(prec);
} ...
```

Проблема этого фрагмента кода состоит в том, что хотя операторы “<<” должны выполняться слева направо, C++-среда не обязана вычислять операнды в данном конкретном порядке. В частности, она может вызвать функцию `grade` после вывода фразы `ваша итоговая оценка равна`. Если функция `grade` сгенерирует исключение, то выходные данные будут состоять только из этой (ставшей неуместной) фразы. Более того, первое обращение к функции `setprecision` может установить точность выходного потока равной значению 3 и не предоставить возможности (посредством второго обращения к той же функции) восстановить предыдущее значение точности. В качестве альтернативного варианта C++-среда могла бы вызвать функцию `grade` до вывода каких-либо данных — точные действия (вернее, их последовательность) зависят исключительно от конкретной C++-среды.

Цель этого анализа — пояснить, почему мы разделили блок вывода данных на две инструкции: первая инструкция гарантирует, что обращение к функции `grade` будет выполнено до формирования каких бы то ни было выходных данных.

Очень хорошее правило, которое следует всегда соблюдать, гласит: избегайте нескольких побочных эффектов в одной инструкции. Генерирование исключения — это побочный эффект, поэтому инструкция, которая может сгенерировать исключение, не должна быть источником никаких других побочных эффектов, особенно включающих операции ввода-вывода.

Конечно, мы не можем выполнить нашу функцию `main` в таком виде, как она представлена выше. Необходимо включить `include`-директивы и `using`-объявления для доступа к библиотечным средствам, которые “вовсю” использует наша программа. Мы также применяем функцию `read_hw` и перегруженную функцию `grade`, которая принимает в качестве третьего аргумента ссылку `const vector<double>&`. Причем в определении этой функции, в свою очередь, используется функция `median` и функция-тезка `grade`, которая принимает три `double`-значения.

Для выполнения этой программы необходимо определить (в надлежащем порядке) перечисленные выше функции до определения нашей функции `main`. Тем самым мы завершили бы работу над созданием несколько “разбухшей” программы. Но пока рано ставить точку в этой истории, лучше повременить до раздела 4.3, в котором показано, как можно расчлнять подобные программы и размещать их части в отдельных файлах. Однако спешить не будем и рассмотрим (конечно, для пользы дела) более удачные, если таковые возможны, способы структуризации данных.

4.2. Организация данных

Вычисление итоговой оценки одного студента может быть и полезно, но подобное вычисление настолько простое, что с ним может прекрасно справиться и карманный калькулятор. Если же мы хотим сделать нашу программу полезной для преподавателя, то нужно, чтобы она могла вычислять итоговые оценки для группы студентов.

Вместо интерактивного общения программы с каждым студентом (на предмет его успеваемости), мы можем предположить, что у нас есть файл, содержащий некоторое количество имен студентов и соответствующих оценок. За каждым именем в этом файле записаны оценки, полученные по экзаменам, проведенным в середине и конце семестра, а за ними — одна или несколько оценок за выполнение домашних заданий. Такой файл может иметь следующий вид.

```
Сахно 93 91 47 90 92 73 100 87
Карпенко 75 90 87 92 93 60 0 98
...
```

Наша программа должна вычислить итоговую оценку каждого студента с помощью метода медианы, причем медианная оценка за домашние задания составляет 40%, оценка за последний экзамен — 40% и оценка за промежуточный экзамен — 20%.

Для приведенных выше данных результат работы программы должен быть следующим.

```
Карпенко      86.8
Сахно         90.4
...
```

Выходные данные организованы следующим образом: фамилии студентов упорядочены по алфавиту, а итоговые оценки выровнены по вертикали, чтобы их было легче читать. Такие требования к оформлению выходных данных означают, что нам нужно место для хранения записей всех студентов, чтобы затем мы могли расположить их в алфавитном порядке. Нам также нужно найти значение длины самой длинной фамилии, чтобы знать, сколько пробелов поместить между каждой фамилией и соответствующей ей оценкой.

Предполагая, что у нас есть место для хранения данных об одном студенте, мы можем использовать вектор для хранения данных обо всех студентах. Если вектор будет содержать данные обо всех студентах, мы сможем отсортировать хранимые в нем

значения, а затем вычислить и вывести итоговые оценки для каждого студента. Начнем с создания структуры данных и написания некоторых вспомогательных функций для чтения и обработки этих данных. После разработки описанных абстракций мы сможем использовать их для решения проблемы в целом.

4.2.1. Соберем-ка все данные о студентах в одну кучу!

Если мы знаем, что нам нужно прочитать данные о каждом студенте, а затем упорядочить их в алфавитном порядке, имеет смысл совместно хранить имена студентов и их оценки. Следовательно, нам нужен способ хранения в одном месте всей информации, которая относится к одному студенту. Роль такого места может играть некоторая структура данных, содержащая имя студента, оценки, полученные по экзаменам, проведенным в середине и конце семестра, а также все оценки за выполнение домашних заданий. В C++ такая структура определяется следующим образом.

```
struct Student_info {  
    string name;  
    double midterm, final;  
    vector<double> homework;  
}; // Обратите внимание на точку с запятой – она обязательна.
```

Это определение структуры говорит о том, что `Student_info` — это тип, который имеет четыре данных-члена. Поскольку `Student_info` — тип, мы можем определить объекты этого типа, каждый из которых будет содержать экземпляр этих четырех данных-членов.

Первый член, именуемый `name`, имеет тип `string`; второй и третий (оба типа `double`) называются `midterm` и `final`, а последний — это вектор `double`-значений по имени `homework`.

Каждый объект типа `Student_info` содержит информацию об одном студенте. Поскольку `Student_info` — это тип, для хранения произвольного числа студентов мы можем использовать объект типа `vector<Student_info>` (подобно тому, как мы применяли объект типа `vector<double>` для хранения произвольного числа оценок за выполненные домашние задания).

4.2.2. Управление записями с данными о студентах

Нашу задачу можно разбить на три управляемых компонента, которые вполне представимы в виде отдельных функций: ввод данных в объект типа `Student_info`, вычисление итоговой оценки для объекта типа `Student_info` и сортировка содержимого вектора объектов типа `Student_info`.

Функция считывания данных (одной записи) во многом подобна функции `read_hw`, описанной в разделе 4.1.3. И в самом деле, мы вполне можем использовать функцию для ввода оценок за выполнение домашних заданий. Но кроме оценок за домашние задания, нам еще нужно ввести имя студента и оценки по двум экзаменам.

```
istream& read(istream& is, Student_info& s)  
{  
    // Чтение и сохранение имени студента и оценок по двум  
    // экзаменам (проведенным в середине и конце семестра).  
    is >> s.name >> s.midterm >> s.final;  
  
    read_hw(is, s.homework); // Чтение и сохранение всех оценок  
                             // за выполнение домашних заданий.  
    return is;  
}
```

В имени функции `read` нет никакой неоднозначности, поскольку тип второго параметра точно указывает, какие данные она считывает. Несмотря на перегрузку, эту функцию нельзя спутать с любой другой одноименной функцией, которая может быть использована для ввода данных в структуру какого-нибудь другого типа. Подобно функции `read_hw`, эта функция принимает в качестве параметров две ссылки: одну — на поток `istream`, из которого вводятся данные, а другую — на объект, в котором прочитанные данные должны быть сохранены. Используя параметр `s` внутри функции, мы воздействуем на состояние переданного функции аргумента.

Эта функция вводит значения в члены объекта `s` с именами `name`, `midterm` и `final`, а затем вызывает функцию `read_hw`, чтобы ввести значения оценок за домашние задания. В любой момент этого процесса может быть обнаружен признак конца файла или может произойти сбой в операции ввода. В этом случае последующие попытки ввода данных ни к чему не приведут, поэтому после выхода из функции объект `is` будет находиться в соответствующем состоянии ошибки. Обратите внимание на то, что такое поведение обусловлено тем, что функция `read_hw` (см. раздел 4.1.3) предусмотрительно оставляет входной поток в состоянии ошибки, если он уже находился в таком состоянии на момент вызова функции `read_hw`.

Нам необходима еще одна функция, которая вычисляет итоговую оценку для объекта типа `Student_info`. Мы уже решили большую часть этой проблемы, определив функцию `grade` в разделе 4.1.2. Мы продолжим работу в этом направлении, перегрузив функцию `grade` версией, которая вычисляет итоговую оценку для объекта типа `Student_info`.

```
double grade(const Student_info& s)
{
    return grade(s.midterm, s.final, s.homework);
}
```

Эта функция обрабатывает объект типа `Student_info` и возвращает `double`-значение, представляющее итоговую оценку. Обратите внимание на то, что параметр функции имеет тип `const Student_info&`, а не просто `Student_info`, поэтому при вызове функции мы не расходует системные ресурсы на копирование всего `Student_info`-объекта.

Заметьте также, что эта функция не предохраняет от исключения, генерируемого функцией `grade`, которую она вызывает. Дело в том, что не существует никаких других средств, которые могла бы использовать наша функция `grade` для обработки исключения, кроме тех, которые уже использовала “внутренняя” функция `grade`. Поскольку наша функция `grade` не перехватывает никаких исключений, любое исключение, которое может быть сгенерировано, будет передано назад автору вызова нашей функции, а ему, вероятно, лучше знать (чем нам), что делать со студентами, которые не выполнили домашних заданий.

Итак, прежде чем написать всю программу в целом, нам осталось решить, как отсортировать содержимое вектора объектов типа `Student_info`. В функции `median` (см. раздел 4.1.1) мы сортировали содержимое параметра типа `vector<double>`, именуемого `vec`, с помощью библиотечной функции `sort`.

```
sort(vec.begin(), vec.end());
```

Тогда, предположив, что наши данные содержатся в векторе `students`, мы тем не менее не можем просто записать следующее.

```
sort(students.begin(), students.end()); // Не верно.
```

Почему? Подробнее о функции `sort` и о других библиотечных алгоритмах мы поговорим в главе 6, но пока имеет смысл абстрактно поразмышлять о том, как действует функция `sort`. В частности, откуда ей “знать”, как именно переставлять значения в заданном векторе?

Функция `sort` должна сравнивать элементы вектора в порядке их следования. Для сравнения элементов заданного типа она использует оператор “<”. Мы можем спокойно вызвать функцию `sort` для вектора типа `vector<double>`, поскольку оператор “<” корректно сравнит два `double`-значения и сгенерирует соответствующий результат. Но что произойдет, когда функция `sort` попытается сравнить значения типа `Student_info`? Ведь оператор “<” попросту “не умеет” работать с объектами типа `Student_info`. И в самом деле, при попытке функции `sort` сравнить два таких объекта компилятор не замедлит выразить свое недовольство.

К счастью, функция `sort` принимает третий необязательный аргумент, который является *предикатом* (predicate). Предикат — это функция, которая генерирует значение истинности типа `bool`. Если этот третий аргумент присутствует, функция `sort` будет использовать его для сравнения элементов вместо оператора “<”. Следовательно, нам нужно определить функцию, которая принимает два объекта типа `Student_info`, а затем сообщает результат сравнения первого объекта со вторым (меньше ли он). Поскольку мы хотим упорядочить фамилии студентов по алфавиту, запишем нашу функцию сравнения, которая будет сравнивать только фамилии.

```
bool compare(const Student_info& x, const Student_info& y)
{
    return x.name < y.name;
}
```

Эта функция просто делегирует свою работу по сравнению двух `Student_info`-объектов классу `string`, который предоставляет оператор “<” для сравнения строк (`string`-объектов). Этот оператор сравнивает строки, используя обычный лексикографический порядок (dictionary ordering), согласно которому левый операнд считается меньше правого, если он в алфавитном порядке стоит впереди правого операнда. Именно такое поведение нам и нужно было определить.

Определив функцию `compare`, мы можем отсортировать значения, хранимые в векторе, передав эту функцию библиотечной функции `sort` в качестве ее третьего аргумента.

```
sort(students.begin(), students.end(), compare);
```

Таким образом, для сравнения заданных элементов функция `sort` будет теперь вызывать нашу функцию `compare`.

4.2.3. Построение отчета

Теперь, когда у нас есть функции обработки записей с данными о студентах, мы можем сгенерировать наш отчет.

```
int main()
{
    vector<Student_info> students;
    Student_info record;
    string::size_type maxlen = 0;

    // Считываем и сохраняем все записи, затем находим
    // длину самой длинной фамилии.
    while (read(cin, record)) {
        maxlen = max(maxlen, record.name.size());
    }
```

```

        students.push_back(record);
    }

    // упорядочиваем записи по алфавиту.
    sort(students.begin(), students.end(), compare);

    for (vector<Student_info>::size_type i = 0;
         i != students.size(); ++i) {

        // выводим фамилию, дополненную справа
        // maxlen + 1 символами.
        cout << students[i].name
              << string(maxlen + 1 -
                        students[i].name.size(), ' ');

        // Вычисляем и выводим оценку.
        try {
            double final_grade = grade(students[i]);
            streamsize prec = cout.precision();
            cout << setprecision(3) << final_grade
                  << setprecision(prec);
        } catch (domain_error e) {
            cout << e.what();
        }
        cout << endl;
    }
    return 0;
}

```

Мы уже рассмотрели большую часть этого кода, но на некоторых моментах имеет смысл остановиться.

Во-первых, мы впервые использовали библиотечную функцию `max`, которая определена в заголовке `<algorithm>`. С первого взгляда поведение этой функции кажется очевидным. Однако и здесь требуются дополнительные разъяснения. Аргументы этой функции должны быть одинакового типа, но о причинах этого требования мы поговорим в разделе 8.1.3. Пока главное для нас — определить `maxlen` как переменную типа `string::size_type`, а не просто типа `int`.

Во-вторых, мы впервые использовали выражение следующего вида.

```
string(maxlen + 1 - students[i].name.size(), ' ')
```

Это выражение создает безымянный объект (см. раздел 1.1) типа `string`. Данный объект содержит `maxlen + 1 - students[i].name.size()` символов, причем все они являются пробелами. Это выражение аналогично определению переменной `space` из раздела 1.2, но здесь опущено имя подобной переменной. Это “упущение” эффективно превращает определение в выражение. Запись значения этого выражения после фамилии (`students[i].name`) обеспечивает вывод такого строкового объекта, в котором символы члена `students[i].name` дополняются справа нужным количеством пробелов, чтобы всего было выведено ровно `maxlen + 1` символов.

В `for`-инструкции индекс `i` используется для поэлементного опроса структуры данных `students`, т.е. на каждой итерации цикла индекс `i` позволяет обратиться к текущему элементу структуры типа `Student_info` и получить значение члена `name`. Затем мы выводим значение члена `name` из этого объекта, используя построенное соответствующим образом `string`-значение, которое состоит из пробелов, “доводящих” до нужной длины данный элемент выводимой информации.

Затем мы выводим итоговую оценку, рассчитанную для каждого студента. Если студент не выполнил ни одного домашнего задания, в процессе вычисления этой оценки будет сгенерировано исключение. В этом случае мы перехватываем исключе-

ние и, вместо вывода числового значения оценки, выводим сообщение, переданное как часть объекта исключения. Все исключения стандартной библиотеки (в том числе и исключение типа `domain_error`) запоминают (необязательный) аргумент, используемый для описания проблемы, послужившей причиной генерирования исключения. Каждый из упомянутых типов исключений создает копию содержимого этого аргумента, доступного посредством функции-члена с именем `what`. Инструкция `catch` в этой программе присваивает имя объекту исключения, который она получает из функции `grade` (см. раздел 4.1.2), чтобы вывести это сообщение из функции `what()`. В данном случае это сообщение уведомит пользователя, что Студент не сделал ни одного домашнего задания. При отсутствии исключений мы используем манипулятор `setprecision` для установки нужной нам точности, а именно вывода числовых значений с тремя значащими цифрами, а затем, собственно, и выводим результат вызова функции `grade`.

4.3. А теперь соберем все вместе

Итак, мы определили ряд абстракций (функций и структур данных), которые используются при решении различных проблем вычисления итоговых оценок студентов. Единственный способ использовать эти абстракции — поместить все их определения в один файл и скомпилировать этот файл. Очевидно, что сложность программирования при таком подходе очень быстро растет. В целях упрощения язык C++, подобно многим другим языкам, поддерживает *механизм раздельной компиляции* (*separate compilation*), который позволяет помещать разные части одной программы в отдельные файлы и компилировать эти файлы независимо от остальных.

Начнем с функции `median`: как с нею поступить, чтобы другие программные элементы могли ее использовать. Для начала поместим определение функции `median` в отдельный файл, чтобы его можно было скомпилировать. Этот файл должен включать объявления для всех имен, которые используются в функции `median`. Из библиотечных средств функция `median` применяет тип `vector`, функцию `sort` и исключение типа `domain_error`, поэтому мы должны включить в наш файл соответствующие заголовки.

```
// исходный файл для функции median.
#include <algorithm> // Для доступа к объявлению функции sort.
#include <stdexcept> // Для доступа к объявлению
                    // класса domain_error.
#include <vector>    // Для доступа к объявлению класса vector.

using std::domain_error; using std::sort; using std::vector;

// Вычисление медианы вектора vector<double>.
double median(vector<double> vec)
{
    // Тело функции в соответствии с определением
    // из раздела 4.1.1.
}
```

Как и в случае с любым другим файлом, мы должны присвоить нашему исходному файлу имя. Стандарт C++ не выдвигает никаких требований насчет имен исходных файлов, но, в общем, имена таких файлов должны отражать их содержимое. Однако большинство C++-сред на имена исходных файлов накладывают ограничения, обычно требуя, чтобы последние несколько символов имени имели некоторую конкретную форму. C++-среды используют эти суффиксы имен файлов для определения, является

ли данный файл исходным файлом C++. Большинство C++-сред требует, чтобы имена исходных C++-файлов имели расширения .cpp, .C или .c, поэтому мы можем поместить нашу функцию median в файл с именем median.cpp, median.C или median.c (в зависимости от конкретной C++-среды).

Теперь мы должны сделать нашу функцию median доступной для других пользователей. По аналогии со стандартной библиотекой, которая помещает определяемые ею имена в заголовки, мы можем написать собственный *заголовочный файл* (header file), который позволит пользователям получать доступ к определяемым нами именам. Например, в файле median.h мы могли бы сообщить о существовании нашей функции median, и тогда пользователи смогут использовать ее в своих программах, написав следующее.

```
// гораздо более удачный способ использования функции median.  
#include "median.h"  
#include <vector>  
  
int main() { /* ... */ }
```

Используя директиву #include с именем заголовка, заключенным в двойные кавычки (вместо угловых скобок), мы тем самым указываем компилятору, что он, вместо этой директивы #include, должен скопировать в нашу программу все содержимое соответствующего заголовочного файла. В каждой C++-среде по-своему решается вопрос, где искать указанные заголовочные файлы и какие взаимоотношения существуют между строкой, заключенной в кавычки, и именем файла. Словосочетание “заголовочный файл median.h” следует понимать как сокращенный вариант фразы “файл, который с точки зрения C++-среды соответствует имени median.h”.

Необходимо отметить, что хотя мы рассматриваем наши заголовки как заголовочные файлы, тем не менее мы относимся к заголовкам, предоставленным C++-средой, как к стандартным заголовкам, а не как к стандартным заголовочным файлам. Дело в том, что эти заголовочные файлы являются реальными файлами в каждой C++-среде, а системные заголовки необязательно реализованы как файлы. Даже несмотря на то что директива #include используется для доступа как к заголовочным, так и системным заголовкам, не существует требования их одинаковой реализации.

Теперь, когда мы знаем, что должны предоставить заголовочный файл, возникает вполне логичный вопрос о его содержимом. Ответ очень простой: в этот заголовочный файл мы должны включить *объявление* (declaration) функции median, заменив (в ее определении) тело функции точкой с запятой. Мы можем также исключить имена параметров, поскольку они нерелевантны без тела функции.

```
double median(vector<double>);
```

Наш заголовок median.h не может содержать только одно это объявление; мы должны также включить все имена, которые в нем используются. Это объявление использует тип vector, поэтому мы должны быть уверены, что это имя будет доступно “глазам” компилятора до того, как он “увидит” наше объявление.

```
// median.h  
#include <vector>  
double median(std::vector<double>);
```

Мы включаем заголовок vector, чтобы в объявлении аргумента для функции median можно было использовать имя std::vector. Более существенно то, что мы при этом явно указали тип std::vector, а не написали using-объявление.

В целом, заголовочные файлы должны объявлять только те имена, которые действительно необходимы. Накладывая ограничения на имена, содержащиеся в заголовочном файле, мы тем самым оставляем максимальную гибкость для наших пользователей. Например, мы используем составное имя `std::vector`, поскольку заранее не можем знать, как пользователь нашей функции `median` захочет указать тип `std::vector`. Пользователи нашего кода, возможно, и не захотят использовать для вектора `using`-объявление. Если бы в своем заголовке мы написали `using`-объявление, то все программы, включающие наш заголовок, получили бы объявление `using std::vector`, независимо от того, хотели они того или нет. Заголовочные файлы должны использовать только составные имена, а не `using`-объявления.

И наконец, последнее. Каждый заголовочный файл должен гарантировать безопасность своего неоднократного включения как части компиляции программы. Что касается нашего заголовка, то он в этом смысле совершенно безопасен, поскольку содержит только объявления. Однако мы считаем хорошим стилем программирования предусмотреть (и обслужить) его многократное включение в каждый заголовочный файл. Это можно реализовать добавлением в файл некоторых препроцессорных средств.

```
#ifndef GUARD_median_h
#define GUARD_median_h

// median.h — окончательная версия.
#include <vector>
double median(std::vector<double>);

#endif
```

Директива `#ifndef` проверяет, определена ли переменная `GUARD_median_h`. `GUARD_median_h` — это имя *препроцессорной переменной*, которая предоставляет один из возможных способов управления компиляцией программы. Полное рассмотрение препроцессора выходит за рамки этой книги.

В данном контексте директива `#ifndef` “просит” препроцессор обработать все, что находится между ею и следующей (соответствующей ей) директивой `#endif`, если заданное (в директиве `#ifndef`) имя *не определено*. Для проверки мы должны выбрать уникальное имя, поэтому создаем его из имени нашего файла и строки `GUARD_` в надежде, что у такого сложного имени не будет двойников.

При первом включении заголовка `median.h` в программу переменная `GUARD_median_h` будет неопределенной, поэтому препроцессор просмотрит остаток файла. Первое, что он сделает, — определит переменную `GUARD_median_h` (с помощью директивы `#define`), чтобы последующие попытки включить в программу заголовков `median.h` были обречены на неудачу.

Осталось отметить только один нюанс в этой истории: директива `#ifndef` должна быть самой первой строкой заголовочного файла, чтобы даже ни один комментарий не предшествовал ей.

```
#ifndef Переменная
...
#endif
```

Некоторые C++-среды обнаруживают файлы такой формы и, если указанная переменная определена, даже не пытаются прочитать файл второй раз.

4.4. Декомпозиция программы вычисления ИТОВЫХ ОЦЕНОК

Теперь, когда мы знаем, что надо сделать, чтобы скомпилировать функцию `median` отдельно, в качестве следующего “номера нашей программы” нужно подготовить структуру `Student_info` и соответствующие функции.

```
#ifndef GUARD_Student_info
#define GUARD_Student_info

// Заголовочный файл Student_info.h
#include <iostream>
#include <string>
#include <vector>

struct Student_info {
    std::string name;
    double midterm, final;
    std::vector<double> homework;
};

bool compare(const Student_info&, const Student_info&);
std::istream& read(std::istream&, Student_info&);
std::istream& read_hw(std::istream&, std::vector<double>&);
#endif
```

Обратите внимание на то, что при использовании имен из стандартной библиотеки мы не включаем `using`-объявления, а явно приписываем префикс `std::`; кроме того, заголовок `Student_info.h` объявляет функции `compare`, `read` и `read_hw`, которые тесно связаны со структурой `Student_info`. Мы воспользуемся этими функциями только в том случае, если также используем эту структуру, поэтому имеет смысл упаковать эти функции вместе с определением структуры.

Эти функции должны быть определены в исходном файле, который будет выглядеть примерно так.

```
// Исходный файл для функций, связанных со
// структурой Student_info.
#include "Student_info.h"

using std::istream; using std::vector;

bool compare(const Student_info& x, const Student_info& y)
{
    return x.name < y.name;
}

istream& read(istream& is, Student_info& s)
{
    // В соответствии с определением из раздела 4.2.2.
}

istream& read_hw(istream& in, vector<double>& hw)
{
    // В соответствии с определением из раздела 4.1.3.
}
```

Обратите внимание на то, что поскольку мы включили сюда (посредством директивы `#include "Student_info.h"`) файл `Student_info.h`, исходный файл будет содержать как объявления, так и определения наших функций. Эта избыточность не только безопасна, но даже и полезна. Она дает компилятору возможность проверить

соответствие между объявлениями и определениями. В большинстве C++-сред такие проверки не претендуют на полноту, поскольку для полной проверки нужно “видеть” программу целиком. Тем не менее они достаточно полезны, чтобы в исходные файлы имело смысл включать соответствующие заголовочные файлы.

Упомянутая проверка и ее неполнота проистекают из следующего: язык требует, чтобы объявления функций и их определения в точности соответствовали одни другим по типу результата, количеству и типу параметров. Это объясняет способность C++-среды проверить это соответствие — так в чем же тогда состоит неполнота проверки? Дело в том, что если объявление и определение в чем-то различны, среда может допустить, что они описывают две различные версии перегруженной функции и недостающее определение находится где-то в другом месте. Например, мы определили функцию `median`, как показано в разделе 4.1.1, а затем некорректно объявили ее следующим образом.

```
int median(std::vector<double>); // Возвращаемое значение
                               // должно иметь тип double.
```

Если компилятор встретит это объявление, то при компиляции определения функции он выразит свое “недоумение”, поскольку знает, что тип значения, возвращаемого функцией `median(vector<double>)`, не может быть одновременно и `double` и `int`. Однако предположим, что мы использовали следующее объявление.

```
double median(double); // Аргумент должен иметь
                       // тип vector<double>.
```

Теперь компилятор “не имеет права жаловаться” на нас, поскольку функция `median(double)` может быть определена где-нибудь в другой части программы. Если мы вызываем эту функцию, C++-среда должна в конце концов найти ее определение. Если это ей не удастся, она “молчать” не станет.

Обратите также внимание на то, что в исходном файле нет проблем с использованием `using`-объявлений. В отличие от заголовочного файла, исходный файл не оказывает никакого влияния на программы, которые используют эти функции. Следовательно, уверенное применение `using`-объявлений в исходном файле — решение исключительно местного значения.

Теперь нам осталось написать заголовочный файл для объявления различных перегруженных функций `grade`.

```
#ifndef GUARD_grade_h
#define GUARD_grade_h

// grade.h
#include <vector>
#include "Student_info.h"
double grade(double, double, double);
double grade(double, double, const std::vector<double>&);
double grade(const Student_info&);

#endif
```

Обратите внимание на то, как сведение объявлений этих перегруженных функций облегчает просмотр всех альтернатив. Мы определим все эти три функции в одном файле, поскольку у них “близкое родство”. И снова-таки, имя файла в зависимости от C++-среды может быть следующим: `grade.cpp`, `grade.C` или `grade.c`.

```
#include <stdexcept>
#include <vector>
#include "grade.h"
```

```
#include "median.h"
#include "Student_info.h"

using std::domain_error; using std::vector;
// Определения для функций grade из
// разделов 4.1, 4.1.2 и 4.2.2.
```

4.5. Исправленная версия программы вычисления итоговых оценок

Наконец, мы можем записать конечный вариант нашей программы.

```
#include <algorithm>
#include <iomanip>
#include <iostream>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
#include "grade.h"
#include "Student_info.h"

using std::cin;           using std::setprecision;
using std::cout;         using std::sort;
using std::domain_error; using std::streamsize;
using std::endl;         using std::string;
using std::max;           using std::vector;

int main()
{
    vector<Student_info> students;
    Student_info record;
    string::size_type maxlen = 0; // Длина самого длинного
                                // имени.
    // Считываем и сохраняем все данные об оценках студента.
    // Инвариант: вектор students содержит все записи,
    //             прочитанные до сих пор.
    //             maxlen содержит длину самой длинной фамилии
    //             name в структуре students.
    while (read(cin, record)) {
        // Находим длину самой длинной фамилии.
        maxlen = max(maxlen, record.name.size());
        students.push_back(record);
    }

    // Упорядочиваем записи с данными о студентах по алфавиту.
    sort(students.begin(), students.end(), compare);

    // Выводим фамилии и оценки.
    for (vector<Student_info>::size_type i = 0;
         i != students.size(); ++i) {
        // Выводим фамилию, дополненную справа
        // maxlen + 1 символами.
        cout << students[i].name
              << string(maxlen + 1 -
                        students[i].name.size(), ' ');

        // Вычисляем и выводим итоговую оценку.
        try {
            double final_grade = grade(students[i]);
            streamsize prec = cout.precision();
            cout << setprecision(3) << final_grade
```

```

        << setprecision(prec);
    } catch (domain_error e) {
        cout << e.what();
    }
    cout << endl;
}
return 0;
}

```

Эта программа не должна быть сложной для понимания. Как обычно, она начинается с необходимых директив `include` и `using`-объявлений. Конечно же, мы должны включить сюда только те заголовки и объявления, которые используются в этом исходном файле. В данной программе, помимо библиотечных, мы использовали и заголовки “собственноручного изготовления”. Эти заголовки делают доступным определение типа `Student_info`, а также объявления функций, которые мы используем для обработки объектов типа `Student_info` и вычисления итоговых оценок. Функция `main` осталась такой же, какой она была представлена в разделе 4.2.3.

4.6. Резюме

Структура программы

`#include` <Системный заголовок>

В угловые скобки (<>) заключаются системные заголовки, которые могут быть реализованы (а может, и нет) как файлы

`#include` "имя заголовочного файла, определенного пользователем"

Заголовочный файл, определенный пользователем, вставляется в программу с помощью директивы `#include` и посредством заключения его имени в кавычки. Обычно заголовки, определенные пользователем, имеют суффикс `.h`

Следует предотвращать многократное включение заголовочных файлов, используя директиву `#ifndef` `GUARD_ИМЯ_заголовка`. В заголовках не должны объявляться имена, которые там не используются. В частности, они должны не включать `using`-объявления, а в явном виде предвирать имена стандартной библиотеки префиксом `std::`.

Типы

`T&` Обозначает ссылку на тип `T`. Чаще всего используется для передачи параметра, который может быть изменен функцией. Аргументы, соответствующие таким параметрам, должны быть *l*-значениями

`const T&` Обозначает ссылку на тип `T`, которую нельзя использовать для изменения значения, связанного с этой ссылкой. Обычно применяется, чтобы избежать затрат на копирование параметра в функцию

Структуры. Структура — это тип, который содержит некоторое (возможно, нулевое) число членов. Каждый объект структурного типа содержит собственный экземпляр каждого члена структуры.

Каждая структура должна иметь соответствующее определение.

```

struct ИМЯ_типа
{
    Спецификатор_типа ИМЯ_члена;
}; // Обратите внимание на точку с запятой.

```

Подобно другим определениям, определение структуры может присутствовать в исходном файле только один раз, поэтому обычно оно включается в заголовочный файл, защищенный соответствующим образом.

Функции. Функция должна быть объявлена в каждом исходном файле, в котором она используется, а определена только однажды. Объявления и определения имеют подобную форму.

```
// Объявление функции:  
Тип_возврата имя_функции(Список_типов_параметров);  
  
// Определение функции:  
{ inline } Тип_возврата имя_функции(Список_типов_параметров){  
    // Здесь должно быть тело функции.  
}
```

Здесь элемент *Тип_возврата* представляет тип значения, возвращаемого функцией, а под элементом *Список_типов_параметров* подразумевается список типов параметров функции, разделенных запятыми. Функции должны быть объявлены до их вызова. Тип каждого аргумента должен быть совместим с соответствующим параметром. (Синтаксис объявления или определения функций с более сложными типами возвращаемых значений рассматриваются в разделе А.1.2.)

Имена функций могут быть перегруженными: с одним и тем же именем может быть объявлено несколько функций; при условии, что эти функции будут отличаться по числу или типам параметров. Имейте в виду, что C++-среда может причислить ссылку и const-ссылку к одному и тому же типу.

В определении функции может быть (но это необязательно) использовано ключевое слово *inline*, которое предлагает компилятору развернуть, где это возможно, обращения к этой функции "в строку", т.е. во избежание затрат, связанных с вызовом функции, заменить каждый ее вызов копией тела функции, модифицированного соответствующим образом. Чтобы реализовать такую замену, компилятор должен видеть определение функции, поэтому *inline*-функции обычно определяются в заголовочных, а не в исходных файлах.

Обработка исключений

```
try { // код
```

Инициализирует блок, который может сгенерировать исключение

```
} catch(t) { /* код */ }
```

Завершает try-блок и обрабатывает исключение, которое соответствует типу *t*. Код, следующий за инструкцией *catch*, выполняет действие, соответствующее обработке исключения, указанного аргументом *t*

```
throw e;
```

Прекращает выполнение текущей функции; передает значение *e* автору вызова

Классы исключений. Библиотека определяет ряд классов исключений, по именам которых можно догадаться о характере проблем, для сообщения о которых они используются.

<i>logic_error</i>	<i>domain_error</i>	<i>invalid_argument</i>
<i>length_error</i>	<i>out_of_range</i>	<i>runtime_error</i>
<i>range_error</i>	<i>overflow_error</i>	<i>underflow_error</i>

e.what() Возвращает значение, которое сообщает о том, что стало причиной ошибочной ситуации

Библиотечные средства

- `s1 < s2` Сравнивает `string`-объекты `s1` и `s2` на основе лексикографического порядка
- `s.width(n)` Устанавливает ширину потока `s` равной `n` для следующей операции вывода (или оставляет ее без изменения, если параметр `n` опущен). Результат дополняется слева до заданной ширины. Возвращает предыдущее значение ширины. Операторы стандартного вывода используют существующее значение ширины, а затем вызывают функцию `width(0)` для его сброса
- `setw(n)` Возвращает значение типа `streamsize` (см. раздел 3.1), что при выводе в выходной поток `s` равносильно вызову функции `s.width(n)`

Упражнения

- 4.0. Скомпилируйте, выполните и протестируйте программы, приведенные в этой главе.
- 4.1. Как было отмечено в разделе 4.2.3, важно, чтобы типы аргументов в обращении к функции `max` в точности совпадали. Будет ли тогда работать следующий код? Если здесь существует проблема, то как ее можно исправить?
- ```
int maxlen;
Student_info s;
max(s.name.size(), maxlen);
```
- 4.2. Напишите программу вычисления квадратов `int`-значений до 100. Эта программа должна вывести два столбца: в первом должно быть значение, а во втором — квадрат этого значения. Для управления выводом данных, т.е. чтобы выводимые значения были выровнены по столбцам, используйте функцию `setw` (см. описание выше).
- 4.3. Что произойдет, если переписать предыдущую программу так, чтобы она выводила числа (и их квадраты) до 1000 (но не включала значение 1000), и при этом пренебречь изменением аргументов, передаваемых функции `setw`? Перепишите программу так, чтобы она была более защищенной “перед лицом” изменений, которые позволяют переменной `i` расти, не корректируя аргументы функции `setw`.
- 4.4. А теперь измените свою программу вычисления квадратов, используя вместо `int`-значений значения типа `double`. Используйте манипуляторы для управления выводом данных так, чтобы значения были выровнены по столбцам.
- 4.5. Напишите функцию, которая считывает слова из входного потока и сохраняет их в векторе. Используйте эту функцию для написания программ, которые подсчитывают количество слов во входном потоке, а также фиксируют, сколько раз встречается в нем каждое слово.
- 4.6. Перепишите структуру `Student_info` для немедленного вычисления оценок и сохранения значения только итоговой оценки.
- 4.7. Напишите программу вычисления среднего арифметического от чисел, содержащихся в векторе типа `vector<double>`.
- 4.8. Если следующий код допустим, то какой можно сделать вывод о типе значения, возвращаемого функцией `f`?
- ```
double d = f()[n];
```

ИСПОЛЬЗОВАНИЕ ПОСЛЕДОВАТЕЛЬНЫХ КОНТЕЙНЕРОВ

Применяя спортивную терминологию, можно сказать, что мы с высокого старта взялись за освоение базового языка C++ и параллельно познакомились с классами `string` и `vector`. С помощью только этих средств мы уже можем решить множество задач.

В этой главе мы попробуем лучше понять, как можно использовать библиотечные средства, которые, как будет показано ниже, позволяют решать более сложные проблемы, чем те, с которыми мы сталкивались до сих пор.

Стандартная библиотека не только предоставляет полезные структуры данных и функции, но и отражает соответствующую архитектуру: узнав, как ведет себя контейнер одного типа, мы постепенно научимся использовать все библиотечные контейнеры.

Например, как будет показано во второй половине этой главы, `string`-объект часто можно использовать так же, как вектор. Многие полезные операции, выполняемые для одного библиотечного типа, логически совпадают с операциями, применяемыми к другому библиотечному типу. Библиотека, по сути, и построена таким образом, чтобы подобные эквивалентные операции выполнялись одинаково на объектах различных типов.

5.1. Разделение студентов на категории

Давайте вспомним нашу задачу вычисления итоговых оценок студентов из раздела 4.2. Предположим, что мы не только хотим вычислить все итоговые оценки, но и желаем знать, кто из студентов завалил экзамены. Если у нас есть вектор записей типа `Student_info`, то нам нужно выделить в нем те записи, которые соответствуют студентам, провалившим экзамены, и сохранить их в другом векторе. Мы также хотели бы удалить данные с неудовлетворительными оценками из исходного вектора, чтобы он содержал записи только о тех студентах, которые выдержали экзамены. Начнем, пожалуй, с простой функции проверки того, является ли оценка неудовлетворительной.

```
// Предикат для определения, провалил ли студент экзамен.
bool fgrade(const Student_info& s)
{
    return grade(s) < 60;
}
```

Для вычисления итоговой оценки мы используем функцию `grade` из раздела 4.2.2 и будем считать неудовлетворительной ту оценку, которая содержит меньше 60 баллов. Самый простой способ решить поставленную задачу — просмотреть запись каждого студента и поместить ее в один из двух векторов: либо в вектор удовлетворительных оценок (проходных баллов), либо в вектор недопустимо низких оценок.

```

// Отделяем записи с проходными баллами от записей с
// непроходными: первая попытка.
vector<Student_info> extract_fails(
    vector<Student_info>& students)
{
    vector<Student_info> pass, fail;
    for (vector<Student_info>::size_type i = 0;
         i != students.size(); ++i)
        if (fgrade(students[i]))
            fail.push_back(students[i]);
        else
            pass.push_back(students[i]);
    students = pass;
    return fail;
}

```

Безусловно, прежде чем компилировать этот код, необходимо для используемых здесь имен включить соответствующие `#include`-директивы и `using`-объявления. Больше мы не будем показывать эти декларативные инструкции в представляемом нами коде. Но при использовании новых заголовков мы, конечно же, включим их в явном виде.

Подобно функциям `read_hw` и `read` из главы 4, функция `extract_fails` эффективно использует возможность возврата сразу двух результатов. Первый — это объект типа `vector<Student_info>`, возвращаемый функцией обычным способом и содержащий записи с недопустимо низкими оценками, а другой создается как побочный эффект от вызова функции `extract_fails`. Параметром этой функции является ссылка, поэтому изменения, вносимые в параметр, отражаются в самом аргументе. По завершении функции вектор, переданный в качестве аргумента, будет содержать записи только тех студентов, которые имеют проходной балл.

Рассмотренная функция создает два вектора, которые содержат данные для успевающих и неуспевающих студентов соответственно. Эта функция “заглядывает” в каждую запись и присоединяет копию этой записи к концу вектора `pass` или `fail`, в зависимости от итоговой оценки студента.

После завершения инструкции `for` мы копируем записи с удовлетворительными оценками снова в вектор `students` и возвращаем вектор `fail` с неудовлетворительными оценками.

5.1.1. Удаление элементов из вектора

Наша функция `extract_fails` вполне справляется с поставленной перед ней задачей и является достаточно эффективной. Однако она имеет один существенный недостаток: эта функция требует значительного объема памяти для хранения двух копий каждой записи с оценками студентов. Дело в том, что функция наращивает векторные объекты `pass` и `fail`, оставляя исходные записи нетронутыми. И только по завершении инструкции `for` она готова скопировать рассортированные по категориям записи, в результате чего образуются две копии каждой записи студента.

Хорошо бы найти способ, позволяющий избежать хранения нескольких копий данных. Это можно сделать, совершенно исключив вектор `pass`, т.е. вместо создания двух векторов мы создадим одну локальную переменную `fail` для хранения значения, подлежащего возврату функцией. Для каждой записи в векторе `students` будет проверяться значение оценки. Если оно окажется удовлетворительным, мы оставим эту запись в покое; в противном случае присоединим копию этой записи к вектору `fail`, а саму запись удалим из вектора `students`.

Для реализации этой стратегии необходимо уметь удалять элемент из вектора. И хотя такое средство удаления уже существует, нашу радость омрачает то, что удаление

элементов из векторов происходит настолько медленно, что при большом количестве входных данных приходится отказываться от использования этого средства. Если обрабатываемые данные действительно объемны, производительность падает до невероятно низких значений.

Например, если окажется, что все наши студенты не набрали проходной балл, то время выполнения функции будет возрастать пропорционально квадрату количества студентов. Это означает, что для группы, состоящей из 100 студентов, программа будет выполняться в 10 000 раз дольше, чем для одного студента. Все дело в том, что наши входные записи хранятся в векторе, который оптимизирован для быстрого произвольного доступа. Цена этой оптимизации — дороговизна операций вставки или удаления элементов, расположенных в любом месте (а не только в конце) вектора.

Рассмотрим два способа решения проблемы производительности: можно использовать либо структуру данных, которая больше подходит к нашему алгоритму, либо более сложный алгоритм, который свободен от неоправданных расходов системных ресурсов, присущих нашему первоначальному проекту. Начиная с этого раздела (и до раздела 5.5.2), мы будем разрабатывать вариант программы, в которой используется более подходящая здесь структура данных. Алгоритмическое же решение показано в разделе 6.3.

Чтобы понять, почему намеченные нами варианты решения задачи являются шагом вперед на пути к совершенству, нам нужно от чего-то отталкиваться, т.е. необходимо иметь то, что можно совершенствовать. Поэтому начнем с медленного, но верного (т.е. прямого) решения.

// Вторая попытка: верное, но потенциально медленное решение.

```
vector<Student_info> extract_fails(
    vector<Student_info>& students)
{
    vector<Student_info> fail;
    vector<Student_info>::size_type i = 0;

    // инвариант: элементы [0, i) вектора students
    // представляют удовлетворительные оценки.
    while (i != students.size()) {
        if (fgrade(students[i])) {
            fail.push_back(students[i]);
            students.erase(students.begin() + i);
        } else
            ++i;
    }
    return fail;
}
```

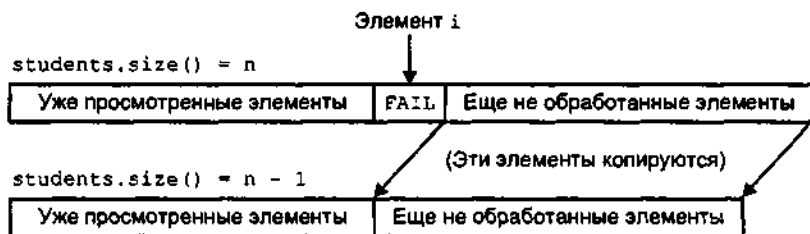
Этот вариант программы начинается с создания вектора `fail`, в который мы копируем записи студентов с неудовлетворительными оценками. Переменная `i` используется в качестве индекса для опроса элементов вектора `students`. В цикле `while` просматривается каждая запись (т.е. элемент) вектора `students` до тех пор, пока не будут проанализированы все его записи.

На каждой итерации цикла мы проверяем итоговую оценку, и если она оказывается неудовлетворительной, нам нужно скопировать запись (содержащую эту оценку) в вектор `fail` и удалить ее из вектора `students`. В вызове функции `push_back` для присоединения копии элемента `students[i]` к концу вектора `fail` нет ничего нового. Новым же является способ удаления элемента из вектора `students`.

```
students.erase(students.begin() + i);
```

Класс `vector` включает функцию-член `erase`, которая предназначена для удаления из вектора заданного элемента. Аргумент, передаваемый функции `erase`, указывает, какой элемент нужно удалить. Но оказывается, что такая версия функции `erase`, которая бы работала на основе индексов, не предусмотрена, поскольку, как показано в разделе 5.5, не все контейнеры поддерживают индексы, а для библиотеки эффективнее предложить такую форму выполнения функции `erase`, которая соответствовала бы общему характеру работы всех контейнеров. Поэтому функция `erase` принимает параметр типа, который рассматривается в разделе 5.2.1. Пока важно понять, что подлежащий удалению элемент можно обозначить путем сложения нашего индекса со значением, возвращаемым функцией `students.begin()`. Вспомните, что функция `students.begin()` возвращает значение, указывающее на начальный элемент вектора (которому соответствует индекс 0). Если сложить с этим значением целое число, выраженное индексом `i`, то результат укажет на элемент с индексом `i`. Теперь должно быть понятно, что такое обращение к функции `erase` удалит `i`-й элемент из вектора `students`.

После удаления элемента из вектора в нем будет содержаться на один элемент меньше, чем было до сих пор.



Помимо изменения размера вектора, функция `erase` удаляет элемент с индексом `i`, тем самым “присуждая” индекс `i` следующему элементу в последовательности. Каждый элемент, находящийся после позиции `i`, копируется в предыдущую позицию. Следовательно, хотя индекс `i` не меняется, функция `erase` выполняет настройку индекса на следующий элемент в векторе, а это значит, что мы не должны инкрементировать его для следующей итерации.

Если текущая запись не содержит неудовлетворительной оценки, нам нужно оставить ее в векторе `students`. В этом случае мы должны сами инкрементировать индекс `i`, чтобы на следующем проходе цикла `while` новое значение `i` указывало на следующую запись.

Конец просмотра всех записей в векторе `students` мы определяем путем сравнения индекса `i` со значением выражения `students.size()`. При удалении элемента из `vector`-объекта содержимое вектора уменьшается на один элемент. Следовательно, важно вызывать функцию `students.size` на каждом проходе через проверку этого условия. Если бы вместо очередного вызова этой функции (в проверке `while`-условия) мы ограничились ее однократным вызовом с сохранением его результата еще до входа в `while`-цикл, как показано в следующем фрагменте программы, то такая программа при выполнении потерпела бы фиаско, поскольку первый же вызов функции `erase` изменил бы количество элементов в векторе `students`.

```
// Этот код не будет работать из-за неправильной оптимизации.
vector<Student_info>::size_type size = students.size();
while (i != size) {
    if (fgrade(students[i])) {
        fail.push_back(students[i]);
```

```

        students.erase(students.begin() + i);
    } else
        ++i;
}

```

Итак, если бы мы предварительно вычислили размер вектора путем вызова функции `size`, а потом реально удалили ряд записей для неуспевающих студентов, то нам бы пришлось сделать слишком много “переходов” по вектору `students` и при этом ссылки на объекты `students[i]` оказались бы ссылками на несуществующие элементы! К счастью, обращения к функции `size()` обычно выполняются быстро, поэтому ожидаемые затраты системных ресурсов при каждом вызове функции `size` незначительны.

5.1.2. Последовательный и произвольный доступ к данным

Обе версии нашей функции `extract_fails` объединяет общее свойство, которое не видно с первого взгляда на код, но характерно для многих программ, работающих с контейнерами: каждая из этих функций получает доступ к элементам контейнера только в последовательном режиме. Это значит, что каждая версия функции просматривает все записи студентов по очереди: сначала решает, что делать с текущей записью, а затем переходит к следующей.

Причина неочевидности этого свойства состоит в том, что наша функция для доступа к каждому элементу вектора `students` использует целочисленную переменную `i`. Значение целочисленной переменной можно изменить любыми способами; для нас это означает, что при последовательном доступе к контейнеру мы должны рассмотреть каждую операцию, которая могла бы повлиять на значение переменной `i`, и установить результат ее воздействия. Еще один путь к пониманию проблемы лежит в следующем: используя для доступа к элементу вектора `students` выражение `students[i]`, мы неявным образом заявляем, что можем получить доступ к элементам вектора `students` в любом порядке, а не только в последовательном.

Причина нашего равнодушия к тому, в какой последовательности можно получить доступ к элементам контейнера, состоит в том, что различные типы контейнеров имеют различные рабочие характеристики и поддерживают различные операции. Если мы будем знать, что наша программа использует только те операции, которые контейнер определенного типа поддерживает довольно эффективно, то мы сделаем нашу программу эффективнее, используя именно этот вид контейнера.

Другими словами, поскольку нашей функции необходим только последовательный доступ, мы можем вообще обойтись без индексов, которые способны обеспечить доступ к любому элементу в произвольном порядке. Поэтому нашу функцию стоит переписать, чтобы ограничить доступ к элементам контейнера операциями, которые поддерживают исключительно последовательный доступ. Именно для такой цели библиотека C++ поддерживает ряд типов, именуемых *итераторами*, позволяющими получить доступ к структурам данных способами, которыми библиотека может управлять. Вот эти-то “рычаги управления” и позволяют библиотеке гарантировать эффективную реализацию.

5.2. Итераторы

Чтобы сделать наш разговор более конкретным, рассмотрим операции контейнера, которые в действительности использует функция `extract_fails`.

Первой такой операцией является использование индекса `i` для выборки значений из структуры `Student_info`. Например, при вычислении выражения `fgrade(students[i])` выбирается `i`-й элемент вектора `students`, и этот элемент передается функции `fgrade` в

качестве аргумента. Мы знаем, что получаем доступ к элементам вектора `students` последовательно, поскольку используем для этого только индекс `i`, и единственными выполняемыми здесь над переменной `i` операциями являются чтение значения `i` (с целью сравнения с размером вектора) и его инкрементирование.

```
while (i != students.size()) {  
    // Здесь происходит вся работа, но значение  
    // переменной i не изменяется.  
    ++i;  
}
```

Этот код убеждает нас, что мы используем переменную `i` только последовательно.

К сожалению, хотя мы и знаем об этом, у нас нет способа передать эти сведения библиотеке. Однако, используя вместо индексов итераторы, мы можем сделать эти сведения доступными и для библиотеки. *Итератор* (iterator) — это значение, которое

- идентифицирует контейнер и элемент в контейнере;
- позволяет проверить значение, хранимое в этом элементе;
- обеспечивает операции перемещения между элементами в контейнере;
- ограничивает круг операций такими, с которыми контейнер может эффективно справляться в соответствии с его возможностями.

Поскольку итераторы ведут себя аналогично индексам, зачастую мы можем переписать свои программы так, чтобы они вместо индексов использовали итераторы. В качестве примера предположим, что `students` — это вектор типа `vector<Student_info>`, который содержит записи о нескольких студентах. Рассмотрим тогда, как можно было бы вывести их имена в выходной поток `cout`. Сначала для каждой итерации будем использовать индекс.

```
for (vector<Student_info>::size_type i = 0;  
      i != students.size(); ++i)  
    cout << students[i].name << endl;
```

А теперь выполним ту же задачу с помощью итераторов.

```
for (  
    vector<Student_info>::const_iterator iter = students.begin();  
    iter != students.end(); ++iter) {  
    cout << (*iter).name << endl;  
}
```

Чтобы понять второй вариант, нужно слегка углубиться в теорию итераторов.

5.2.1. Типы итераторов

Каждый стандартный контейнер (такой, как вектор) определяет два связанных типа итераторов.

```
Тип_контейнера::const_iterator  
Тип_контейнера::iterator
```

Здесь в качестве элемента *Тип_контейнера* может быть любой тип контейнера, например `vector<Student_info>`, который в свою очередь включает тип элементов, содержащихся в данном контейнере. Если итератор предназначен для изменения значений, хранимых в контейнере, используется тип `iterator`. Если же нужен доступ только для чтения, применяется тип `const_iterator`.

Абстракция — это избирательное неведение. Детали итератора того или иного типа могут быть довольно сложны для понимания, но нам пока нет нужды вникать в эти

подробности. Сейчас нам достаточно знать, как использовать тип итератора и какие операции он позволяет выполнять. Знание типа позволит нам создавать переменные, являющиеся итераторами. Но без знаний реализации того или иного типа итератора мы можем легко обойтись. Например, следующее определение

```
vector<Student_info>::const_iterator iter = students.begin();
```

говорит о том, что переменная `iter` имеет тип `vector<Student_info>::const_iterator`. Мы не знаем *действительный* тип переменной `iter`, т.е. детали его (типа) реализации в классе `vector`; да нам и не нужно это знать. Все, что нам необходимо знать, — это то, что вектор типа `vector<Student_info>` имеет член `const_iterator`, определяющий тип, который мы можем использовать для получения доступа, разрешающего только чтение элементов вектора.

И еще нам нужно знать, что существует автоматическое преобразование объектов типа `iterator` в объекты типа `const_iterator`. Как вы вскоре узнаете, функция `students.begin()` возвращает объект типа `iterator`. Вы тут же можете выразить недоумение, указав на приведенную выше строку определения переменной `iter`, которая заявляет, что переменная `iter` имеет тип `const_iterator`. Оказывается, чтобы инициализировать переменную `iter` значением выражения `students.begin()`, C++-среда преобразует `iterator`-значение в соответствующее `const_iterator`-значение. Это преобразование должно помочь вам запомнить, что `iterator`-значение можно преобразовать в `const_iterator`-значение, но отнюдь не наоборот.

5.2.2. Операции, выполняемые итераторами

Определив переменную `iter`, мы устанавливаем ее равной значению выражения `students.begin()`. Мы уже использовали функции `begin` и `end`, поэтому вы должны иметь представление о том, что они делают: они возвращают значение, которое означает начало или конец контейнера соответственно. С определением конца контейнера дело обстоит несколько сложнее, но эти сложности мы разясним в разделе 8.2.7. Пока же полезно знать, что функции `begin` и `end` возвращают значение типа `iterator` для данного контейнера. Следовательно, функция `begin` возвращает значение типа `vector<Student_info>::iterator`, обозначающее позицию начального элемента контейнера, поэтому переменная `iter` изначально указывает на первый элемент вектора `students`.

Тогда следующее условие в инструкции `for`

```
iter != students.end()
```

проверяет, достигли ли мы конца контейнера. Вспомните, что функция `end` возвращает значение, обозначающее конец контейнера. Как и в случае функции `begin`, значение, возвращаемое функцией `end`, имеет тип `vector<Student_info>::iterator`. Мы можем сравнить два итератора, `const`-типа или `non-const`-типа, на предмет неравенства (или равенства). Если значение переменной `iter` равно значению, возвращаемому функцией `students.end()`, значит, мы дошли до конца контейнера.

Последнее выражение в заголовке `for`-инструкции, `++iter`, инкрементирует итератор, чтобы на очередном проходе в `for`-цикле он указывал на следующий элемент в векторе `students`. В выражении `++iter` используется оператор инкремента, перегруженный для типа `iterator`. Оператор инкремента обладает эффектом перемещения итератора к следующему элементу в контейнере. Мы не знаем (и нас это не должно

волновать), как работает этот оператор инкремента. Нам достаточно знать, что в результате его действия итератор будет обозначать следующий элемент в контейнере.

В теле цикла `for` переменная `iter` обозначает позицию некоторого элемента в векторе `students`, член которого (точнее, имя студента) нам нужно вывести. Доступ к члену мы получаем с помощью *оператора разыменования* (*dereference operator*), обозначаемого символом “звездочка” (*). Примененный к итератору, оператор “*” возвращает *l*-значение (см. раздел 4.1.3) члена элемента, на который ссылается данный итератор. Следовательно, следующая операция вывода

```
cout << (*iter).name
```

выводит в стандартный выходной поток член `name` (имя) текущего элемента.

Чтобы это выражение было выполнено корректно, необходимо использовать круглые скобки, которые переопределяют приоритет операторов, действующий по умолчанию. Выражение `*iter` возвращает значение, на которое указывает итератор `iter`. Приоритет оператора “точка” (.) выше приоритета оператора “звездочка” (*), а это значит, что если мы хотим, чтобы операция разыменования (*) была применена только к левому операнду оператора “точка” (.), нужно заключить выражение `*iter` в круглые скобки, получив в результате выражение `(*iter)`. Если бы мы записали выражение `*iter.name` (без круглых скобок), компилятор обработал бы его как выражение `*(iter.name)`, что было бы равносильно требованию получить член `name` из объекта `iter`, а затем применить оператор разыменования к этому объекту. Компилятор не стерпел бы такой несуразицы, поскольку объект `iter` не имеет члена `name`. Записав выражение `(*iter).name`, мы имеем в виду, что хотим обратиться к члену `name` объекта `*iter`.

5.2.3. О некоторых синтаксических нюансах

В только что рассмотренном коде мы выполнили разыменование итератора, а затем из значения, которое было возвращено в результате операции разыменования, прочитали нужный элемент (член `name`). Это сочетание операций настолько распространено, что для него в языке C++ предусмотрено специальное сокращенное обозначение. Вместо выражения

```
(*iter).name
```

можно записать следующее.

```
iter->name
```

Эту синтаксическую изюминку можно использовать в новом варианте фрагмента программы, приведенного в разделе 5.2.

```
for (
    vector<Student_info>::const_iterator iter = students.begin();
    iter != students.end(); ++iter) {
    cout << iter->name << endl;
}
```

5.2.4. Что означает выражение `students.erase(students.begin() + i)`

Теперь, когда мы больше узнали об итераторах, можно заглянуть в самую суть выражения из программы, приведенной в разделе 5.1.1.

```
students.erase(students.begin() + i);
```


Мы уже знаем, что выражение `students.begin()` возвращает итератор, который указывает на начальный элемент вектора `students`, а выражение `students.begin() + i` — итератор, который указывает на i -й элемент того же вектора. Здесь важно понимать, что последнее выражение получает свое значение на основе определения оператора “+” для типов объектов `students.begin()` и i . Другими словами, значение результата действия оператора “+” в этом выражении определяется типами итератора и индекса.

Если бы вектор `students` был контейнером, не поддерживающим индексирование с произвольной выборкой, то, по всей вероятности, тип выражения `students.begin()` не относился бы к типам, для которых определен оператор “+”, и в этом случае выражение `students.begin() + i` не скомпилировалось бы. В действительности такой контейнер был бы способен отключить произвольный доступ к своим элементам, разрешая тем не менее последовательный доступ к ним посредством итераторов.

5.3. Использование итераторов вместо индексов

Вооружившись полученными знаниями об итераторах и зная еще одного факта, мы можем так переписать нашу функцию `extract_fails`, чтобы в ней совсем не использовались индексы.

```
// версия 3: итераторы вместо индексов;  
// по-прежнему работает потенциально медленно.  
vector<Student_info> extract_fails(  
    vector<Student_info>& students)  
{  
    vector<Student_info> fail;  
    vector<Student_info>::iterator iter = students.begin();  
    while (iter != students.end()) {  
        if (fgrade(*iter)) {  
            fail.push_back(*iter);  
            iter = students.erase(iter);  
        } else  
            ++iter;  
    }  
    return fail;  
}
```

Определение функции, как и прежде, начинается с определения вектора `fail`. Затем мы определяем итератор `iter`, который будем использовать вместо индекса для просмотра всех элементов вектора `students`. Обратите внимание на то, что вместо типа `const_iterator` мы применяем здесь тип `iterator`.

```
vector<Student_info>::iterator iter = students.begin();
```

Дело в том, что мы собираемся использовать его для модификации вектора `students`, которая выражается в вызове функции `erase`. Итератор `iter` инициализируется для указания на первый элемент вектора `students`.

Определение функции продолжает инструкция `while`, которая должна просмотреть все элементы вектора `students`. Помните, что `iter` — это итератор, который указывает на элемент контейнера, поэтому выражение `*iter` возвращает значение этого элемента. Чтобы узнать, освоил ли студент курс, мы передаем это значение функции `fgrade`. Аналогично мы модифицировали код, который копирует записи с неудовлетворительными оценками в вектор `fail`, заменив инструкцию

```
fail.push_back(students[i]); // для получения элемента вектора  
                             // здесь используется индекс.
```

следующей.

```
fail.push_back(*iter); // Для получения элемента вектора здесь  
                        // применяется разыменование итератора.
```

Функция `erase` стала проще, поскольку у нас есть итератор, который непосредственно передается ей в качестве параметра.

```
iter = students.erase(iter);
```

Мы больше не должны вычислять итератор путем сложения значения индекса `i` с выражением `students.begin()`.

Новшество, которое мы здесь использовали, может легко остаться незамеченным, поэтому мы и обращаем на него ваше внимание ввиду его крайней важности: в предыдущей инструкции мы присвоили итератору `iter` значение, которое возвращает функция `erase`. Вас интересует, почему?

Следующие размышления могут убедить нас, что удаление элемента, на который указывал итератор `iter`, должно привести к тому, что этот итератор станет *недействительным* (*invalidate*). После вызова функции `students.erase(iter)` мы знаем, что итератор `iter` не может больше указывать на тот же элемент, поскольку этот элемент уже отсутствует. И в самом деле вызов функции `erase` для некоторого вектора делает недействительными все итераторы, которые ссылаются на элементы, расположенные после только что удаленного элемента. Если вернуться к рисунку, приведенному в разделе 5.1.1, то нетрудно убедиться в том, что после удаления элемента, помеченного именем `FAIL`, от него не остается и следа, а каждый из элементов, стоящих после него, сдвигается на одну позицию. Если элементы сместились, любые итераторы, ссылавшиеся на них, должны при этом потерять смысл.

К счастью, функция `erase` возвращает итератор, указывающий на элемент, который расположен сразу за только что удаленным элементом. Следовательно, выполнение следующей инструкции

```
iter = students.erase(iter);
```

заставляет `iter` указывать на элемент, расположенный после удаленного, что нам и нужно.

Если мы имеем дело с элементом, содержащим удовлетворительную оценку, то нам по-прежнему нужно инкрементировать итератор `iter` для перехода к следующему элементу, который будет обработан на следующей итерации цикла. Это реализуется с помощью оператора инкремента (`++iter;`) в `else`-ветви инструкции `if-else`.

В связи с этим, как показано в разделе 5.1.1, мы могли бы попытаться оптимизировать цикл, сохранив значение выражения `students.end()`, чтобы не вычислять его на каждой итерации `while`-цикла. Другими словами, мы могли бы попытаться заменить этот заголовок `while`-инструкции

```
while (iter != students.end())
```

следующим кодом.

```
// Этот код терпит фиаско из-за неудачной оптимизации.  
vector<Student_info>::iterator iter = students.begin(),  
                                end_iter = students.end();  
while (iter != end_iter) {  
    // ...  
}
```

Этот цикл почти гарантированно обречен на неудачу во время выполнения. Почему?

Дело в том, что даже после одного-единственного выполнения функции `students.erase` все итераторы, соответствующие позициям, расположенным за пози-

цией удаленного элемента, включая итератор `end_iter`, станут недействительными! Следовательно, важно вызывать функцию `students.end` на каждом проходе цикла, подобно тому, как в разделе 5.1.1 на каждой итерации необходимо было вызывать функцию `students.size`.

5.4. Изменение структуры данных для повышения производительности

Для небольших объемов входных данных наша программа работает вполне сносно. Однако, как было отмечено в разделе 5.1.1, по мере увеличения объема вводимой информации производительность существенно падает. Почему?

Давайте снова рассмотрим использование функции `erase` для удаления элемента из вектора. Библиотека оптимизирует векторную структуру данных для быстрого доступа к произвольно заданному элементу. Более того, как видно из раздела 3.2.3, подобные векторные структуры данных прекрасно работают при поэлементном наращивании, причем в случае, когда каждый новый элемент добавляется в *конец* вектора.

Вставка в вектор или удаление из него *произвольного* элемента — это совсем другая история, которая включает в себя (для сохранения быстрого произвольного доступа) перемещение всех элементов, расположенных после вставленного или удаленного. Перемещение элементов означает, что замедление времени выполнения нашего нового кода может быть пропорционально квадрату числа элементов в векторе. Для малых объемов обрабатываемой информации такое замедление может быть незаметным, но при каждом удвоении объема входных данных время выполнения программы учетверяется. Если мы попытаемся обработать данные обо всех студентах колледжа, а не только о студентах одной группы, то даже быстрому компьютеру понадобится для этого недопустимо большое время.

Если нас не устраивает описанное поведение программы, мы должны использовать такую структуру данных, которая позволяет эффективно вставлять элементы в любое место контейнера и удалять их оттуда. Такой контейнер вряд ли будет поддерживать произвольный доступ к своим элементам посредством индексов. Даже если бы он и поддерживал, то целочисленные индексы вряд ли были бы полезны, поскольку при вставке или удалении элементов менялись бы индексы других элементов. Теперь, когда вы знаете, как использовать итераторы, мы можем работать с такой структурой данных, которая вообще не поддерживает операций с индексами.

5.5. Тип `list`

Переписав код с использованием итераторов, мы распростились с индексами. А теперь нам нужно переориентировать нашу программу на структуру данных, которая позволит эффективно удалять элементы из контейнера.

Поскольку потребность вставлять элементы в любую структуру данных и удалять их оттуда весьма распространена, неудивительно, что в библиотеке предусмотрен тип `list` (или *список*, определенный в заголовке `<list>`), который оптимизирован для такого доступа.

Подобно тому, как векторы оптимизированы для быстрого произвольного доступа, списки оптимизированы для быстрого выполнения операций вставки элементов в любое место контейнера и удаления их оттуда. Поскольку списки должны поддерживать более сложную структуру, они работают медленнее, чем векторы, если

доступ к контейнеру осуществляется только последовательным образом. Другими словами, если контейнер увеличивается или укорачивается только (или главным образом) в своей хвостовой части, то вектор превосходит список по скоростным характеристикам. Но если программа удаляет много элементов из середины контейнера — как в случае нашей программы — то списки окажутся гораздо “проворнее”, особенно при больших объемах входных данных.

Подобно вектору, список — это контейнер, который может содержать объекты почти любого типа. Как будет показано ниже, списки и векторы имеют много общих операций. Поэтому мы часто можем переводить программы, работающие с векторами, “на рельсы” списков и наоборот. При этом приходится менять лишь типы некоторых переменных.

Одной из ключевых операций, поддерживаемых вектором, но не списком, является индексирование. Как вы только что увидели, мы легко написали версию функции `extract_fails`, которая использует векторы для удаления записей, соответствующих студентам с неудовлетворительными оценками, но вместо индексов здесь применяются итераторы. И оказывается, совсем нетрудно еще раз переделать эту версию функции `extract_fails`, чтобы она вместо векторов использовала списки. Для этого достаточно изменить соответствующие типы данных.

```
// версия 4: использование типа list вместо типа vector.
list<Student_info> extract_fails(list<Student_info>& students)
{
    list<Student_info> fail;
    list<Student_info>::iterator iter = students.begin();

    while (iter != students.end()) {
        if (fgrade(*iter)) {
            fail.push_back(*iter);
            iter = students.erase(iter);
        } else
            ++iter;
    }
    return fail;
}
```

Если сравнить этот код с версией из раздела 5.3, увидим, что единственное изменение в первых четырех строках состоит в замене типа `vector` типом `list`. Так, например, значение, возвращаемое функцией `extract_fails`, и ее параметр имеют сейчас тип `list<Student_info>`, как и локальный контейнер `fail`, в который мы помещаем записи с неудовлетворительными оценками. Аналогичным образом в классе `list` определен и тип итератора. Следовательно, мы определяем переменную `iter` типа `iterator` как член объекта класса `list<Student_info>`. Тип `list` представляет собой шаблон, поэтому мы должны сообщить, объекты какого типа будет содержать данный список, указав этот тип внутри угловых скобок, как это мы делали при определении вектора.

В логику программы никаких изменений внесено не было. Конечно же, теперь автор вызова нашей функции должен в качестве аргумента передать список и в качестве возвращаемого значения также должен получить список. Детали библиотечной реализации операций со списками совершенно другие. Тем не менее, выполняя инструкцию `++iter`, мы также перемещаем итератор к следующему элементу списка. Аналогично следующая инструкция

```
iter = students.erase(iter);
```

вызывает `list`-версию функции `erase` и присваивает переменной `iter` `list`-итератор, возвращаемый функцией `erase`. Реализации операций инкремента и удаления элемента из списка, несомненно, отличаются от их `vector`-двойников.

5.5.1. На некоторые различия стоит обратить особое внимание

Особенно ощутимо операции на списках отличаются от операций на векторах, если они связаны с итераторами. Например, при использовании функции `erase` для удаления элемента из вектора все итераторы, которые ссылаются на удаляемый или последующий элемент, становятся недействительными. Применение функции `push_back` для присоединения элемента к концу вектора также может сделать недействительными все итераторы, ссылающиеся на этот вектор. Такое поведение вытекает из того, что удаление элемента приводит к перемещению последующих элементов, а в результате присоединения элемента к концу вектора могут быть перераспределены все составляющие вектора, чтобы выделить место для нового элемента. Циклы, в которых используются эти операции, должны быть организованы особенно тщательно, чтобы быть уверенным в том, что они не сохраняют копий итераторов, которые могут стать недействительными. Необдуманное сохранение значения функции `students.end()` может оказаться богатейшим источником ошибок.

Однако при обработке списков операции `erase` и `push_back` не делают итераторы недействительными. Недействительным становится только итератор, который ссылается на удаленный элемент, поскольку этот элемент больше не существует.

Мы уже упоминали о том, что итераторы класса `list` не поддерживают свойства произвольного доступа в полном объеме. О свойствах итераторов мы подробнее поговорим в разделе 8.2.1, а пока важно знать, что из-за недостаточной поддержки мы не можем использовать библиотечную функцию `sort` для сортировки значений, сохраняемых в списке. Ввиду этого ограничения класс `list` поддерживает собственную функцию-член `sort`, которая использует алгоритм, оптимизированный для сортировки данных, хранимых в списке. Следовательно, чтобы отсортировать элементы списка, мы должны вместо глобальной функции `sort`

```
vector<Student_info> students;  
sort(students.begin(), students.end(), compare);,
```

которую мы использовали для векторов, вызвать функцию-член `sort`.

```
list<Student_info> students;  
students.sort(compare);
```

На это стоит обратить внимание, поскольку для сортировки записей типа `Student_info`, хранимых в `list`-объекте, можно использовать ту же функцию `compare`, которую мы применяли для сортировки записей такого же типа, содержащихся в `vector`-объекте.

5.5.2. Зачем так беспокоиться?

Код, который выделяет из общего набора записи о неуспевающих студентах, представляет собой хороший пример выбора структуры данных с акцентом на производительности. Этот код получает доступ к элементам контейнера последовательно, что, в общем, должно бы отдать пальму первенства вектору. Однако этот код также обеспечивает удаление элементов, расположенных в любом месте контейнера, а этот факт говорит в пользу списка.

Производительность — довольно сложная тема, обсуждение которой выходит за рамки этой книги, но необходимо отметить, что выбор структуры данных может существенно повлиять на производительность программы. Для небольших объемов входных данных списки обрабатываются медленнее, чем векторы. Но для больших объемов программа, которая применяет векторы неподходящим способом, может работать медленнее, чем при использовании списков. Даже удивительно, насколько быстро падает производительность при увеличении объемов входных данных, сохраняемых в векторах.

Чтобы протестировать производительность наших программ, мы использовали три файла записей с оценками студентов. Первый файл содержал 735 записей. Второй был в десять раз больше, а третий в десять раз больше второго, т.е. содержал 73 500 записей. В следующей таблице показано время (в секундах), которое потребовалось для выполнения сравниваемых программ при обработке файла каждого размера.

<i>Размер файла</i>	<i>Тип list</i>	<i>Тип vector</i>
735	0,1	0,1
7 350	0,8	6,7
73 500	8,8	597,1

Для файла, содержащего 73 500 записей, list-версия программы расходует меньше десяти секунд, в то время как vector-версия — почти десять минут. Это расхождение было бы еще больше, если бы в университете училось больше неуспевающих студентов.

5.6. Разберем string-объект на части

Теперь, когда стало понятно, для чего нужны контейнеры, обратим наше внимание на строки, или string-объекты. До сих пор наша работа со строками ограничивалась несколькими операциями: мы создавали их, читали, конкатенировали, выводили на экран и узнавали их размер. В каждом случае мы рассматривали string-объект как единую сущность. Зачастую, когда нас не интересует детальное содержимое строки, нам нужен именно такой вид абстрактного применения string-объектов. Но иногда необходимо проанализировать отдельные символы string-объекта.

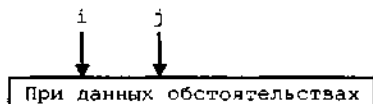
Оказывается, мы можем представить string-объект как контейнер специального типа, который содержит только символы и поддерживает некоторые (но не все) операции, свойственные другим контейнерам. В число поддерживаемых им операций входит индексирование, а тип string обеспечивает итератор, подобный векторному итератору. Следовательно, многие методы, которые мы можем применить к вектору, применимы также к string-объектам.

Например, нам может потребоваться разбить введенную строку на слова, которые отделены одно от другого пробелами или символами, подобными пробелу (символами табуляции, возврата назад на одну позицию или конца строки). Если у нас есть возможность прямого считывания входных данных, то мы сможем получить из них слова вполне тривиальным способом. Причем именно так и работает оператор ввода string-объектов: он считывает символы вплоть до пробела или символа, подобного пробелу.

Однако бывают случаи, когда нам нужно прочитать целую строку входных данных и проанализировать отдельные слова внутри этой строки. Соответствующие примеры мы рассмотрим в разделах 7.3 и 7.4.2.

Поскольку такая операция часто бывает полезной в различных программах, мы напишем специальную функцию. Функция будет принимать `string`-объект и возвращать вектор типа `vector<string>`, содержащий для каждого слова в исходной строке свой элемент. Чтобы понять реализацию этой функции, необходимо знать, что `string`-объекты поддерживают индексирование практически так же, как и `vector`-объекты. Так, например, если `s` — это `string`-объект, содержащий, по крайней мере, один символ, то первым символом объекта `s` является элемент `s[0]`, а последним — `s[s.size() - 1]`.

Наша функция определяет два индекса, `i` и `j`, которые предназначены для поочередного указания границ каждого слова. Идея состоит в том, что мы будем выделять слово, вычисляя значения индексов `i` и `j` таким образом, чтобы искомое слово имело символы в диапазоне `[i, j)`. Рассмотрим следующий пример.



Определив индексы, мы сможем использовать граничные символы для создания нового `string`-объекта, который будет скопирован в наш вектор. Автору вызова нашей функции будет возвращен вектор, содержащий соответствующие `string`-объекты.

```
vector<string> split(const string& s)
{
    vector<string> ret;
    typedef string::size_type string_size;
    string_size i = 0;

    // инвариант: мы обработали символы из
    // диапазона [исходное значение индекса i, i).
    while (i != s.size()) {
        // игнорируем ведущие пробелы.

        // инвариант: все символы в диапазоне
        // [исходное i, текущее i) являются пробелами.
        while (i != s.size() && isspace(s[i]))
            ++i;

        // Находим конец следующего слова.
        string_size j = i;
        // инвариант: ни один из символов в диапазоне
        // [исходное j, текущее j) не является пробелом.
        while (j != s.size() && !isspace(s[j]))
            ++j;

        // Если мы обнаруживаем несколько символов,
        // отличных от пробелов...
        if (i != j) {
            // копируем j - i символов из объекта s, начиная
            // с индекса i.
            ret.push_back(s.substr(i, j - i));
            i = j;
        }
    }
    return ret;
}
```

Помимо системных заголовков, с которыми мы уже встречались, для этого кода нужен заголовок `<ctype>`, который определяет функцию `isspace`. Этот заголовок определяет и другие довольно популярные функции, обрабатывающие отдельные символы. Буква “с” в начале слова “`ctype`” служит напоминанием о том, что средство `ctype` — это часть языка C++, унаследованная от C.

Функция `split` принимает один параметр, который представляет собой ссылку на `const-string`-объект с именем `s`. Поскольку мы будем копировать слова из строки `s`, функции `split` не нужно изменять этот `string`-объект. Как упоминалось в разделе 4.1.2, мы можем передать функции `const`-ссылку, чтобы избежать затрат на копирование строки, гарантируя, что функция `split` не изменит своего аргумента.

Начнем с определения переменной `get`, которая будет содержать слова из входной строки. Первые две инструкции определяют и инициализируют наш первый индекс `i`. Как было показано в разделе 2.4, `string::size_type` — это имя соответствующего типа для индексирования `string`-объекта. Поскольку нам нужно использовать этот тип не один раз, для упрощения последующих объявлений определим более короткий синоним для этого типа, как уже делалось в разделе 3.2.2. Мы будем применять переменную `i` в качестве индекса, который находит начало каждого слова, и постепенно “передвигать” `i` на одно слово “вдоль” входной строки.

Проверка во внешнем цикле `while` гарантирует, что после обработки последнего слова входной строки цикл будет остановлен.

Эта `while`-инструкция начинается с расстановки наших двух индексов. Сначала мы находим в строке `s` первый, отличный от пробела символ, находящийся в позиции символа (или за ним), на который указывает в данный момент индекс `i`. Поскольку во входной строке может быть несколько пробелов, мы инкрементируем значение индекса `i` до тех пор, пока оно не укажет на символ, не являющийся пробелом.

Рассмотрим следующую инструкцию.

```
while (i != s.size() && isspace(s[i]))  
    ++i;
```

Функция `isspace` представляет собой предикат, принимающий параметр типа `char` и возвращающий значение, которое можно интерпретировать как ответ на вопрос, является ли этот `char`-объект пробелом (или символом, подобным пробелу). Оператор “`&&`” проверяет, принимают ли оба его операнда значение `true`, и “не подпускает” к телу цикла в случае, если хотя бы один из них имеет значение `false`. В этом выражении проверка окажется успешной, если значение переменной `i` не равно размеру объекта `s` (т.е. мы еще не “дошли” до конца строки) и элемент `s[i]` представляет собой пробел или символ, подобный ему. В этом случае мы инкрементируем переменную `i` и снова выполняем ту же проверку.

Как отмечалось в разделе 2.4.2.2, при выполнении логической операции “`&&`” используется сокращенная стратегия вычисления ее операндов. В отличие от приводимых выше примеров, пример этого раздела действительно опирается на свойство сокращенного вычисления операции “`&&`”. Выполнение бинарных логических операций (“`&&`” и “`||`”) начинается с тестирования левых операндов. Если результата первого тестирования достаточно для определения общего результата, то правый операнд не вычисляется. В случае использования оператора “`&&`” второе условие тестируется только тогда, когда первое условие приняло значение `true`. Следовательно, проверка условия в `while`-инструкции начинается с проверки условия `i != s.size()`. Только в том случае, если эта проверка окажется успешной, индекс `i` будет использован для

анализа соответствующего символа в строке *s*. Безусловно, если индекс *i* равен значению *s.size()*, это говорит о том, что больше не осталось непроверенных символов и поэтому пора выходить из цикла.

При выходе из этого цикла мы знаем, что либо *i* означает символ, не являющийся пробелом, либо мы завершили просмотр введенных данных, не найдя такого символа.

Предположим, что *i* все еще остается действительным индексом, тогда следующая инструкция *while* должна найти пробел, завершающий текущее слово в строке *s*. Поиск пробела начинается с создания еще одного индекса, *j*, и инициализации его значением индекса *i*. Следующая *while*-инструкция

```
while (j != s.size() && !isspace(s[j]))
    ++j;
```

выполняется аналогично предыдущей, но на этот раз *while*-цикл останавливается, когда встречается пробел или символ, подобный ему. Как и прежде, мы начинаем в уверенности, что индекс *j* еще попадает в диапазон действительных индексов. А коль так, то мы снова вызываем функцию-предикат *isspace* для символа, индексируемого переменной *j*. На этот раз мы инвертируем значение, возвращаемое функцией *isspace*, используя *логический оператор отрицания* (!). Другими словами, нам нужно, чтобы полное условие было истинным (*true*), если значение *isspace(s[j])* не равно *true*.

Завершив выполнение двух внутренних *while*-циклов, мы знаем, что либо нашли еще одно слово, либо в процессе поиска вводимые данные исчерпались. Если верно последнее предположение, то и *i*, и *j* будут равны значению *s.size()*. В противном случае мы выделили слово, которое должны “протолкнуть” в вектор *get*.

```
// Если мы обнаруживаем несколько символов,
// отличных от пробелов...
if (i != j) {
    // копируем j - i символов из объекта s, начиная
    // с индекса i.
    ret.push_back(s.substr(i, j - i));
    i = j;
}
```

В вызове функции *push_back* используется функция-член *substr* класса *string*, которую нам еще не приходилось использовать. Она принимает индекс и значение длины, после чего создает новую строку из символов исходной строки, начиная с индекса, заданного первым аргументом, и последовательно копируя символы в количестве, заданном вторым аргументом. Выделенная нами подстрока начинается с символа, расположенного в позиции *i*, т.е. он является первым символом в только что найденном слове. Мы копируем символы из строки *s*, начиная с символа, индексированного значением *i*, и продолжаем копирование до тех пор, пока не будут скопированы символы из (полуоткрытого) диапазона [*i*, *j*). Если вспомнить (см. раздел 2.6) о том, что количество элементов в полуоткрытом диапазоне равно разности граничных значений этого диапазона, станет понятно, что мы скопируем в точности *j-i* символов.

5.7. Тестирование функции *split*

После написания функции ее следует проверить. Самый простой способ — написать программу, которая считывает входную строку и передает ее функции *split*. Затем можно вывести содержимое вектора, возвращаемого функцией *split*. Такая тестовая программа позволит легко рассмотреть результаты работы нашей функции и сравнить их с ожидаемыми.

Эта тестовая функция должна сгенерировать такие же результаты, как и программа, которая просто считывает слова из стандартного входного потока и выводит их по одному на отдельной строке. Мы можем написать такую программу и выполнить ее и нашу тестовую программу, используя одни и те же входные файлы, а затем убедиться в том, что обе программы генерируют идентичные результаты. Если все так и произойдет, мы можем быть твердо уверены в правильности своей функции `split`.

Начнем, пожалуй, с написания тестовой программы для функции `split`.

```
int main()
{
    string s;
    // Считываем и разбиваем на слова каждую вводимую строку.
    while (getline(cin, s)) {
        vector<string> v = split(s);

        // Записываем каждое слово в v.
        for (vector<string>::size_type i = 0; i != v.size(); ++i)
            cout << v[i] << endl;
    }
    return 0;
}
```

Эта программа должна считывать вводимые данные построчно. К счастью, в `string`-библиотеке предусмотрена нужная нам функция `getline`, которая читает входную информацию до тех пор, пока не достигнет конца строки. Функция `getline` принимает два аргумента. Первый — это поток типа `istream`, из которого мы выполняем чтение; второй представляет собой ссылку на `string`-объект, в котором будут сохраняться прочитанные данные. Обычно функция `getline` возвращает ссылку на входной поток `istream`, поэтому мы можем протестировать значение `istream` в условии `while`-инструкции. Если обнаружится признак конца файла или неверное данное, то значение, возвращаемое функцией `getline`, будет указывать на неудачный исход, позволив тем самым прервать выполнение `while`-инструкции.

Как только мы прочитаем входную строку, она будет сохранена в объекте `s` и передана функции `split`, результат работы которой заппомним в векторе `v`. Затем в цикле опросим вектор `v` и выведем каждый `string`-объект, хранимый в этом векторе, на отдельной строке.

Предположим, мы включили в программу соответствующие `#include`-директивы, в состав которых входит и наш собственный заголовок, содержащий объявление функции `split`. После этого мы могли бы выполнить эту программу (функцию `main`) и визуально удостовериться, что она и функция `split` работают ожидаемым образом. Можно было бы пойти еще дальше, сравнив результат этой программы с результатом программы, которая позволяет библиотеке самой сделать всю работу.

```
int main()
{
    string s;
    while (cin >> s)
        cout << s << endl;
    return 0;
}
```

Эта и предыдущая программы должны сгенерировать идентичные результаты. Здесь мы позволяем оператору ввода `string`-объекта разделить входной поток на ряд слов, которые записываются по одному на строке. Выполнив обе программы при оди-

наковых входных данных, мы можем легко убедиться в работоспособности нашей функции `split`.

5.8. Сборка string-объектов

В разделах 1.2 и 2.5.4 рассматривается программа, которая выводит имя пользователя в рамочке, построенной из символов “звездочка”. Однако в действительности мы не создавали `string`-объект для хранения результатов нашей программы. Вместо этого мы по очереди выводили различные части выходных данных, а затем в выходном файле объединяли их в одну картинку.

Теперь мы хотим по-другому взглянуть на ту же проблему, поставив перед собой новую задачу — построить единую структуру данных, которая будет представлять весь `string`-объект, заключенный в рамку. Эта программа является упрощенной версией одного из наших любимых примеров, именуемых *символьными картинками* (или *символьными изображениями*). Символьная картинка — это прямоугольный массив символов, который можно вывести на экран. Это можно считать упрощением того, что происходит в реальных приложениях — приложениях, основанных на растровых изображениях. В подобных упрощениях вместо битов используются символы, а вместо отображения на графических устройствах применяется запись в обычные файлы. Эта задача подана в виде упражнения, приведенного в первом издании книги Б. Страуструпа *Язык программирования C++*, и несколько глубже исследована в книге *Ruminations on C++* (Addison-Wesley, 1997).

5.8.1. Опять рамочка

Частный случай задачи создания символьных картинок, который мы хотим рассмотреть в этом разделе, состоит в поэлементном выводе всех данных, хранимых в векторе типа `vector<string>` (т.е. по одному элементу на строке), и заключении их в рамочку. Все строки будут выровнены по левому краю, а между рамкой, состоящей из символов “звездочка”, и выводимыми словами будет оставлен зазор шириной в один пробел.

Предположим, `p` — вектор типа `vector<string>`, который содержит `string`-объекты “Это и есть”, “пример”, “иллюстрации”, “заключения” и “в рамочку”. Теперь хотелось бы иметь функцию с именем `frame`, при вызове которой в виде выражения `frame(p)` генерируется значение типа `vector<string>` с элементами, при выводе которых образуется следующая символьная картинка.

```
*****
* Это и есть *
* пример    *
* иллюстрации *
* заключения *
* в рамочку. *
*****
```

Обратите внимание на то, что рамочка идеально прямоугольна, а не с рваными краями, несмотря на то что строки имеют разную длину. Это говорит о том, что нам нужна функция для определения длины самой длинной строки в векторе. Итак, начнем!

```
string::size_type width(const vector<string>& v)
{
    string::size_type maxlen = 0;
    for(vector<string>::size_type i = 0; i != v.size(); ++i)
        maxlen = max(maxlen, v[i].size());
}
```

```

    return maxlen;
}

```

Эта функция поочередно опрашивает все элементы вектора, устанавливая переменную `maxlen` равной самому большому размеру, выявленному до текущего момента. При выходе из цикла переменная `maxlen` будет содержать длину самого длинного `string`-элемента в векторе `v`.

Единственный сложный аспект функции `frame` — ее интерфейс. Мы знаем, что она должна обрабатывать вектор типа `vector<string>`, но как быть с типом возвращаемого ею значения? Было бы удобно, если бы эта функция создавала новое символическое изображение, а не изменяла переданную ей картинку.

```

vector<string> frame(const vector<string>& v)
{
    vector<string> ret;
    string::size_type maxlen = width(v);
    string border(maxlen + 4, '*');

    // выводим верхнюю границу рамки.
    ret.push_back(border);

    // выводим каждую строку внутри рамки, окаймленную
    // символами "звездочка" и пробел.
    for (vector<string>::size_type i = 0; i != v.size(); ++i) {
        ret.push_back("* " + v[i] +
                      string(maxlen - v[i].size(), ' ') + " *");
    }

    // выводим нижнюю границу рамки.
    ret.push_back(border);
    return ret;
}

```

Если наша функция не должна изменять переданную ей картинку, то принимаемый ею параметр должен быть объявлен как ссылка на `const`-вектор. Функция возвратит вектор типа `vector<string>`, который будет встроен в вектор `ret`. Работа функции начинается с вычисления длины выводимых строк, а затем создается `string`-объект, состоящий из символов "звездочка", который мы будем использовать для создания горизонтальных (верхней и нижней) границ рамки.

Горизонтальные границы рамки на четыре символа длиннее самого длинного `string`-объекта: два из этих символа — "звездочки" вертикальных (правой и левой) границ рамки и еще два — пробелы, отделяющие рамку от строк, заключенных в нее. Воспользовавшись синтаксической изюминкой из определения функции `spaces`, приведенного в разделе 1.2, определяем `string`-переменную `border`, которая должна содержать `maxlen + 4` символа "звездочка". Затем вызываем функцию `push_back` для присоединения копии объекта `border` к концу вектора `ret`, формируя таким образом верхнюю границу рамки.

Далее копируем картинку, заключаемую в рамку. Определяем индекс `i`, который предназначен для прохода по всему вектору `v` до тех пор, пока не будут скопированы все его элементы. В вызове функции `push_back` используется оператор "+" класса `string`, который, как мы узнали в разделе 1.2, конкатенирует ее аргументы.

Для формирования выводимой строки конкатенируем символы левой ("* ") и правой (" *") границ с подлежащим выводу `string`-объектом, который хранится в качестве элемента `v[i]`. Еще одним `string`-компонентом нашей конкатенации является значение, возвращаемое функцией `string(maxlen - v[i].size(), ' ')`, которая

создает неименованный временный `string`-объект, содержащий ряд пробелов, присоединяемых справа от основного `string`-объекта текущей строки. Этот `string`-компонент строится таким же способом, каким мы инициализировали объект `border`. Нужное количество пробелов мы получаем путем вычитания из значения переменной `maxlen` размера текущего `string`-элемента вектора.

Используя эти знания, нетрудно понять, что аргументом функции `push_back` является новый `string`-объект, который состоит из символа “звездочка”, пробела, текущего `string`-элемента вектора `v`, ряда пробелов, “доставляющих” текущий `string`-элемент до самого длинного в векторе `string`-элемента, и завершающей строку пары: пробела и символа “звездочка”.

Картину довершает присоединение к концу вектора `ret` нижней границы рамки.

5.8.2. Вертикальная конкатенация

Символьные картинки интересны тем, что с ними можно проделывать различные забавные трюки. Только что мы реализовали одну операцию — окаймление рамочкой. Теперь выполним операцию конкатенации, причем это можно сделать как в вертикальном направлении, так и в горизонтальном. В этом разделе мы ограничимся вертикальной конкатенацией, а следующий раздел посвятим горизонтальной.

Картинки обычно строятся построчно, поэтому для их представления мы используем объекты типа `vector<string>`, каждый элемент которых является отдельной строкой. Следовательно, вертикальная конкатенация двух картинок довольно проста: достаточно конкатенировать векторы, которые их представляют. В результате получим две картинки, выровненные вдоль их левых краев.

В этом случае единственная проблема состоит в том, что (хотя и существует операция конкатенации `string`-объектов) не существует операции конкатенации `vector`-объектов. Поэтому мы вынуждены искать собственный путь решения этой проблемы.

```
vector<string> vcat(const vector<string>& top,
                   const vector<string>& bottom)
{
    // Копируем верхнюю картинку.
    vector<string> ret = top;

    // Копируем целиком нижнюю картинку.
    for (vector<string>::const_iterator it = bottom.begin();
         it != bottom.end(); ++it)
        ret.push_back(*it);

    return ret;
}
```

В этой функции используются только средства, которые уже вам знакомы: вектор `ret` определяется как копия вектора `top`, к концу вектора `ret` присоединяется по очереди каждый элемент вектора `bottom`, после чего в качестве результата работы всей функции возвращается “выросший” вектор `ret`.

В цикле этой функции реализуется одна из довольно распространенных операций: вставка копии элементов из одного контейнера в другой. В этом частном случае мы выполняли присоединение элементов, которое можно рассматривать как вставку в конец контейнера.

Ввиду широкого применения этой операции, в библиотеке предусмотрен вариант ее реализации без использования цикла. Вместо кода

```
for (vector<string>::const_iterator it = bottom.begin();
      it != bottom.end(); ++it)
    ret.push_back(*it);
```

можно записать следующий вызов библиотечной функции `insert`.

```
ret.insert(ret.end(), bottom.begin(), bottom.end());
```

Результат выполнения этих двух фрагментов кода идентичен.

5.8.3. Горизонтальная конкатенация

Под горизонтальной конкатенацией двух картинок мы подразумеваем создание новой картинки, в которой одна из исходных картинок образует ее левую часть, а вторая — правую. Для начала необходимо подумать о том, что мы будем делать, если конкатенируемые картинки окажутся разного размера. Для определенности решим, что в любом случае будем выравнивать их вдоль верхнего края. Следовательно, каждая строка выходной картинки будет представлять собой результат конкатенации соответствующих строк двух исходных картинок. При этом строки левой картинки мы должны заполнять соответствующим количеством пробелов, чтобы правая часть итоговой картинки начиналась с нужной позиции.

Помимо заполнения пробелами левой картинки, мы также должны подумать о том, как поступить, если исходные картинки имеют различное число строк, например, если вектор `p` содержит нашу исходную картинку, а мы хотим конкатенировать в горизонтальном направлении исходное значение вектора `r` с результатом заключения картинки `p` в рамочку. Другими словами, мы бы хотели, чтобы функция `hcat(p, frame(p))` сгенерировала следующую картинку.

```
Это и есть      *****
пример          * Это и есть *
иллюстрации    * пример      *
заключения     * иллюстрации *
в рамочку.     * заключения  *
               * в рамочку.  *
               *****
```

Обратите внимание на то, что левая картинка содержит меньше строк, чем правая. Это означает, что в итоговом изображении мы должны соответствующим образом учесть отсутствующие строки. Если же левая картинка окажется длиннее, достаточно скопировать ее строки в новую картинку; при этом пустые строки правой картинки не нужно заполнять пробелами.

Завершив этот небольшой анализ, можно написать функцию горизонтальной конкатенации `hcat`.

```
vector<string> hcat(const vector<string>& left,
                   const vector<string>& right)
{
    vector<string> ret;

    // добавляем 1, чтобы оставить пробел между картинками.
    string::size_type width1 = width(left) + 1;

    // используем индексы для просмотра элементов из
    // векторов left и right соответственно.
    vector<string>::size_type i = 0, j = 0;

    // продолжаем обработку до тех пор, пока не увидим
    // все строки из обеих исходных картинок.
    while (i != left.size() || j != right.size()) {
```

```

// Создаем новый string-объект для хранения
// символов из обеих исходных картинок.
string s;

// копируем строку из левой картинки, если
// таковая существует.
if (i != left.size())
    s = left[i++];

// Дополняем объект s пробелами до максимальной ширины.
s += string(width1 - s.size(), ' ');

// копируем строку из правой картинки, если
// таковая существует.
if (j != right.size())
    s += right[j++];

// помещаем string-объект s в новую картинку.
ret.push_back(s);
}
return ret;
}

```

Определение функции `hcat`, как и функций `frame` и `vcap`, начинается с определения вектора итоговой картинки. Затем вычисляется ширина, до которой мы должны дополнить левую картинку. Это значение ширины будет на единицу больше ширины самой картинки, чтобы между конкатенируемыми картинками оставался зазор в один пробел. После этого выполняется циклический проход по всем элементам обеих картинок, на каждой итерации которого копируется элемент из первой картинки, дополняемый сначала необходимым количеством пробелов, а затем элементом из второй картинки.

Единственный сложный момент здесь — подумать о том, что делать в случае исчерпания элементов в одной картинке до исчерпания элементов в другой. Наша обработка продолжается до тех пор, пока не будут скопированы все элементы каждого исходного вектора. Поэтому цикл `while` продолжается до тех пор, пока оба индекса не “доберутся” до конца соответствующих векторов.

Если мы еще не исчерпали элементы левой картинки (вектора `left`), ее текущий элемент копируется в `string`-объект `s`. Независимо от того, был ли скопирован элемент из левой картинки (вектора `left`), выполняется составной оператор присваивания (`+=`), в правой части которого стоит вызов функции `string`, позволяющий дополнить пробелами выходную строку до получения соответствующей ширины. Составной оператор присваивания, определенный в библиотеке для класса `string`, действует так, как мы и ожидали: он складывает правый операнд с левым и сохраняет результат в левом операнде. Конечно же, слово “складывает” означает здесь конкатенацию `string`-объектов.

Размер “пустой” части строки определяется вычитанием значения `s.size()` из `width1`. Мы знаем, что значение `s.size()` либо является размером `string`-объекта, скопированного из вектора `left`, либо равно нулю ввиду отсутствия элемента, подлежащего копированию. В первом случае значение `s.size()` будет больше нуля, но меньше значения `width1`, поскольку для обеспечения зазора между двумя картинками мы увеличили на единицу длину самого длинного `string`-элемента. В этом случае мы присоединим к `string`-объекту `s` один или несколько пробелов. Если же значение `s.size()` равно нулю, мы дополняем `string`-объект `s` пробелами так, чтобы он соответствовал ширине левой части выходной строки.

После того как мы скопируем `string`-элемент из левой картинки и дополним его необходимым количеством пробелов, остается лишь присоединить `string`-элемент из правой картинки, предположив существование элемента из вектора `right`, подлежа-

шего копированию. Независимо от того, присоединили ли мы значение из вектора `right`, `string`-объект `s` помещается в выходной вектор, и этот процесс продолжается до тех пор, пока мы не обработаем оба исходных вектора, после чего нам остается вернуть автору вызова нашей функции созданную ею (функцией) картинку.

Важно отметить, что корректное поведение нашей программы обязано тому, что `string`-объект `s` локален по отношению к `while`-циклу. Поскольку объект `s` объявлен внутри `while`-цикла, он создается с использованием `pull`-значения и разрушается на каждой итерации этого цикла.

5.9. Резюме

Контейнеры и итераторы. Назначение стандартной библиотеки состоит в том, чтобы аналогичные операции на различных контейнерах имели один и тот же интерфейс и одну и ту же семантику. Все контейнеры, которые мы использовали до сих пор, являются *последовательными*. В главе 7 мы увидим, что библиотека также позволяет работать с ассоциативными контейнерами. Все последовательные контейнеры и класс `string` обеспечивают выполнение следующих операций.

`контейнер<T>::iterator`

`контейнер<T>::const_iterator`

Имя типа итератора, действующего на этом контейнере

`контейнер<T>::size_type`

Имя соответствующего типа для хранения размера максимально возможного экземпляра этого контейнера

`c.begin()`

`c.end()` Итераторы, указывающие на первый и последний элементы в контейнере соответственно

`c.rbegin()`

`c.rend()` Итераторы, указывающие на последний и первый элементы контейнера соответственно. Предоставляют доступ к элементам контейнера в обратном порядке

`контейнер<T> c();`

`контейнер<T> c(c2);`

Определяет `c` как пустой контейнер или как копию контейнера `c2`, если таковой задан

`контейнер<T> c(n);`

Определяет `c` как контейнер с `n` элементами, которые инициализируются значениями (см. раздел 7.2) в соответствии с типом `T`. Если `T` — тип класса, инициализация элементов определяется этим типом. Если `T` — встроенный арифметический тип, то элементы будут инициализированы значением 0

`контейнер<T> c(n, t);`

Определяет `c` как контейнер с `n` элементами, которые представляют собой копии параметра `t`

`контейнер<T> c(b, e);`

Создает контейнер, который содержит копию элементов, указанных итераторами в диапазоне `[b, e)`

<code>c = c2</code>	Заменяет содержимое контейнера <code>c</code> копией контейнера <code>c2</code>
<code>c.size()</code>	Возвращает количество элементов в <code>c</code> как значение типа <code>size_type</code>
<code>c.empty()</code>	Предикат, означающий, отсутствуют ли элементы в контейнере <code>c</code>
<code>c.insert(d, b, e)</code>	Копирует элементы, указанные итераторами в диапазоне <code>[b, e)</code> , и вставляет их в контейнер <code>c</code> непосредственно перед элементом, обозначенным итератором <code>d</code>
<code>c.erase(it)</code>	Удаляет из контейнера <code>c</code> элемент, обозначенный итератором <code>it</code> , или элементы из диапазона <code>[b, e)</code> . Эта операция выполняется быстро для списков (контейнеров типа <code>list</code>), но может потребовать много времени для контейнеров типа <code>vector</code> и <code>string</code> , поскольку для этих типов выполняется копирование всех элементов, расположенных после удаляемого. Для типа <code>list</code> недействительными становятся итераторы для удаляемого элемента (элементов). Для типа <code>vector</code> и <code>string</code> недействительными становятся все итераторы для элементов, расположенных после удаляемого
<code>c.erase(b, e)</code>	
<code>c.push_back(t)</code>	Помещает в конец контейнера <code>c</code> элемент со значением <code>t</code>
Контейнеры, которые поддерживают произвольный доступ, и объекты типа <code>string</code> также предоставляют следующее средство.	
<code>c[n]</code>	Читает из контейнера <code>c</code> символ, расположенный в позиции <code>n</code>
Операции, реализуемые с помощью итераторов	
<code>*it</code>	Разыменовывает итератор <code>it</code> для получения значения, хранимого в контейнере по указанной им (итератором) позиции. Эта операция часто сочетается с операцией “точка” (<code>.</code>) для получения члена объекта класса, как в выражении <code>(*it).x</code> , которое возвращает член <code>x</code> объекта, обозначенного итератором <code>it</code> . Оператор <code>*</code> имеет более низкий приоритет, чем оператор <code>.</code> , и такой же приоритет, как операторы <code>++</code> и <code>--</code>
<code>it->x</code>	Эквивалент выражения <code>(*it).x</code> , который возвращает член <code>x</code> объекта, полученного в результате разыменования итератора <code>it</code> . Имеет такой же приоритет, как оператор <code>.</code>
<code>++it</code>	
<code>it++</code>	Инкрементирует итератор, чтобы он указывал на следующий элемент в контейнере
<code>b == e</code>	
<code>b != e</code>	Сравнивает два итератора на равенство и неравенство соответственно

Тип `string` предоставляет итераторы, которые поддерживают те же операции, что и итераторы, работающие с векторами. В частности, тип `string` поддерживает совершенно произвольный доступ, который подробно рассматривается в главе 8. Помимо контейнерных операций, тип `string` также поддерживает следующие функции и операторы.

s.substr(i, j) Создает новый string-объект, содержащий копию символов из объекта *s* с индексами в диапазоне $[i, i + j)$

getline(is, s) Читает входную строку из потока *is* и сохраняет ее в объекте *s*

s += s2 Заменяет значение объекта *s* объектом *s + s2*

Тип **vector** предоставляет самые мощные итераторы, именуемые итераторами произвольного доступа, действующими на любых библиотечных контейнерах (более подробно они рассматриваются в главе 8).

Хотя все написанные нами функции основываются на динамическом размещении элементов вектора, существуют также механизмы предварительного размещения элементов в памяти и операция, предписывающая вектору выполнить такое размещение без использования дополнительной памяти во избежание затрат, связанных с многократным перераспределением памяти.

v.reserve(n) Резервирует пространство для содержания *n* элементов, но не инициализирует их. Эта операция не изменяет размер контейнера. Она влияет только на частоту, с которой вектор будет вынужден перераспределять память в ответ на многократные вызовы функций **insert** или **push_back**

v.resize(n) Придает вектору *v* новый размер, равный значению *n*. Если *n* меньше текущего размера вектора *v*, элементы, не “попадающие” в размер *n*, удаляются из вектора. Если *n* больше текущего размера, в вектор *v* добавляются новые элементы, которые инициализируются в соответствии с типом объектов, хранимых в векторе *v*

Тип **list** оптимизирован для эффективного выполнения операций вставки элементов в любую позицию контейнера и удаления их оттуда. Операции, которые можно выполнять применительно к спискам и **list**-итераторам, описаны в разделе 5.9. Помимо них, можно использовать и следующие.

l.sort()

l.sort(cmp) Сортирует элементы в списке *l*, используя оператор “<” для объектов, хранимых в списке, или предикат *cmp*

Заголовок <ctype> предоставляет весьма полезные функции для работы с символическими данными.

isspace(c) Возвращает значение **true**, если *c* — пробел или символ, подобный ему

isalpha(c) Возвращает значение **true**, если *c* — буквенный (алфавитный) знак

isdigit(c) Возвращает значение **true**, если *c* — цифра

isalnum(c) Возвращает значение **true**, если *c* — буква или цифра

ispunct(c) Возвращает значение **true**, если *c* — знак пунктуации

isupper(c) Возвращает значение **true**, если *c* — прописная (заглавная) буква

islower(c) Возвращает значение **true**, если *c* — строчная буква

toupper(c) Генерирует для символа *c* прописной эквивалент

tolower(c) Генерирует для символа *c* строчной эквивалент

Упражнения

- 5.0. Скомпилируйте, выполните и протестируйте программы, приведенные в этой главе.
- 5.1. Продумайте и напишите программу для получения перестановочного индекса. Перестановочный индекс — это индекс, в котором каждая фраза индексируется каждым ее словом. Например, при таких входных данных

```
The quick brown fox
jumped over the fence
```

результат должен быть следующим.

```
      the quick    brown fox
jumped over the   fence
The quick brown  fox
              jumped over the fence
              jumped over the fence
              The quick brown fox
              jumped over the fence
              The quick brown fox
```

Неплохой алгоритм предлагается в книге Ахо (Aho), Кернигана (Kernighan) и Вейнбергера (Weinberger) *The AWK Programming Language* (Addison-Wesley, 1988). Проблема делится на три этапа.

1. Читаем каждую строку входных данных и генерируем набор циклически сдвинутых фраз этой строки. При каждом циклическом сдвиге следующее слово из исходных данных помещается в первую позицию, а предыдущее первое слово перемещается в конец фразы. Вот как выглядит результат циклического сдвига фразы из первой строки наших исходных данных.

```
The quick brown fox
quick brown fox The
brown fox The quick
fox The quick brown
```

Конечно же, важно знать, где оканчивается исходная фраза и где находится начало циклически сдвинутой фразы.

2. Сортируем циклически сдвинутые фразы.
 3. Отменяем циклический сдвиг и записываем перестановочный индекс, что означает отыскать разделитель, поместить фразу на место и записать ее в отформатированном виде.
- 5.2. Напишите законченную версию программы для вычисления итоговых оценок студентов на базе векторов, которая выделяет записи о неуспевающих студентах. Напишите еще один вариант с использованием списков. Определите разницу в производительности при использовании входных файлов с 10, 1 000 и 10 000 строками.
- 5.3. Используя средство `typedef`, можно написать одну версию программы, которая реализует либо `vector`-, либо `list`-ориентированное решение. Напишите и протестируйте эту версию программы.
- 5.4. Еще раз взгляните на драйверные функции, написанные вами в качестве ответа на предыдущее упражнение. Обратите внимание на то, что можно написать драйверы, которые отличаются только в объявлении типа структуры данных, содержащей входной файл. Если ваши `vector`- и `list`-драйверы отличаются чем-то еще, перепишите их, чтобы они отличались только характером объявления.
- 5.5. Напишите функцию `center(const vector<string>&)`, возвращающую картинку, в которой все строки исходной картинки дополняются пробелами до макси-

мальной длины, а это дополнение пробелами по возможности поровну делится между левой и правой сторонами картинki. Каковы свойства картинок, для которых будет полезной такая функция? Как узнать, обладает ли данная картинка такими свойствами?

- 5.6. Перепишите функцию `extract_fails` из раздела 5.1.1 так, чтобы вместо удаления записи о каждом неуспевающем студенте из входного вектора `v` она копировала бы записи об успевающих студентах в начало вектора `v`, а затем использовала функцию `resize` для удаления “лишних” элементов с конца вектора `v`. Как сравнить производительность этой версии и варианта, приведенного в разделе 5.1.1?

- 5.7. Используя реализацию функции `frame` из раздела 5.8.1 и следующий фрагмент кода

```
vector<string> v;  
frame(v);
```

опишите, что произойдет в случае такого вызова функции. В частности, проследите, как будут действовать функции `width` и `frame`. Теперь выполните этот код. Если полученные результаты отличаются от ожидаемых, попытайтесь сначала понять причины этого различия, а затем внесите необходимые изменения, чтобы ожидаемые результаты совпали с фактическими.

- 5.8. Что произойдет в функции `hcat` из раздела 5.8.3, если определить объект `s` вне области видимости `while`-цикла? Перепишите и выполните эту программу для подтверждения своей гипотезы.

- 5.9. Напишите программу для вывода слов (из введенных данных) строчными буквами, а после них — тех же слов прописными буквами.

- 5.10. Палиндромы (перевертни) — это слова, одинаково читающиеся слева направо и справа налево. Напишите программу для отыскания всех палиндромов в словаре. Затем найдите самый длинный палиндром.

- 5.11. При обработке текстов зачастую полезно знать, имеет ли некоторое слово выступающие или свисающие элементы букв. Выступающими являются части строчных букв, которые выступают выше линии текста (в английском алфавите к числу таких букв относятся *b, d, f, h, k, l* и *t*). Аналогично свисающими являются части строчных букв, которые находятся ниже текстовой линии (в английском алфавите к числу таких букв относятся *g, j, p, q* и *y*). Напишите программу для определения, имеет ли заданное слово какие-либо выступающие или свисающие элементы букв. Расширьте возможности программы, “обучив” ее находить самое длинное слово в словаре, которое не имеет ни выступающих, ни свисающих элементов букв.

ИСПОЛЬЗОВАНИЕ БИБЛИОТЕЧНЫХ АЛГОРИТМОВ

Как упоминалось в главе 5, многие контейнерные операции применимы к контейнерам нескольких типов. Например, контейнеры типа `vector`, `string` и `list` позволяют вставлять элементы посредством вызова функции `insert` и удалять их вызовом функции `erase`. Эти операции имеют одинаковый интерфейс для всех типов контейнеров, которые их поддерживают. Многие контейнерные операции также применяются к классу `string`.

Каждый контейнер — а также класс `string` — поддерживает ряд типов итераторов, которые позволяют передвигаться по контейнеру и просматривать его элементы. И снова-таки, библиотека гарантирует, что каждый итератор, который поддерживает ту или иную операцию, делает это посредством одного и того же интерфейса. Например, для перехода итератора любого типа от одного элемента к следующему мы можем использовать оператор “++”; для получения доступа к элементу, соответствующему итератору любого типа, мы можем применить оператор “*” и т.д.

В этой главе мы узнаем, как библиотека использует эти распространенные интерфейсы для предоставления коллекциям стандартных алгоритмов. С помощью алгоритмов можно избежать многократного написания (и переделки) одного и того же кода. Более важно то, что мы можем писать программы, которые, благодаря библиотечным алгоритмам, становятся гораздо меньше и проще, чем без них.

Подобно контейнерам и итераторам, алгоритмы используют постоянное действующее соглашение об интерфейсе. Такое постоянство позволяет нам узнать принципы использования алгоритмов, а затем применять эти знания по мере необходимости. В этой главе мы попытаемся использовать некоторые библиотечные алгоритмы для решения проблем, связанных с обработкой `string`-объектов, вообще, и оценок студентов, в частности. На примере решения конкретных задач мы и освоим основные принципы работы библиотеки алгоритмов.

Если не обусловлено иное, заголовок `<algorithm>` определяет все алгоритмы, представленные в этой главе.

6.1. Анализ `string`-объектов

В разделе 5.8.2 мы использовали цикл для конкатенации двух символьных картинок.

```
for (vector<string>::const_iterator it = bottom.begin();  
     it != bottom.end(); ++it)  
    ret.push_back(*it);
```

Как уже было отмечено, этот цикл эквивалентен вставке копии элементов вектора `bottom` в конец вектора `ret`, т.е. операции, которую векторы реализуют напрямую.

```
ret.insert(ret.end(), bottom.begin(), bottom.end());
```

Эта проблема имеет даже более общее решение: мы можем отделить копирование элементов от их вставки в конец контейнера.

```
copy(bottom.begin(), bottom.end(), back_inserter(ret));
```

Здесь `copy` — это пример общего алгоритма, а `back_inserter` — пример итераторного адаптера.

Общий алгоритм (*generic algorithm*) — это алгоритм, который не является частью контейнера конкретного типа, а вбирает в себя ключевые свойства типов алгоритмов, связанные с доступом к данным. Общие алгоритмы стандартной библиотеки обычно в качестве аргументов принимают итераторы, которые используются для обработки элементов базовых контейнеров. Например, алгоритм `copy` принимает три итератора (назовем их `begin`, `end` и `out`) и копирует все элементы из диапазона `[begin, end)` в последовательность элементов, начиная с позиции, заданной итератором `out`, и занимая необходимое количество последовательных позиций. Другими словами, выполнение инструкции

```
copy(begin, end, out);
```

имеет такой же результат, как и выполнение следующей `while`-инструкции.

```
while (begin != end)
    *out++ = *begin++;
```

Единственное различие между этими инструкциями состоит в том, что тело `while`-цикла изменяет значения итераторов, а алгоритм `copy` — нет.

Прежде чем описывать итераторные адаптеры, необходимо отметить, что этот цикл использует *постфиксную* (*postfix*) версию операторов инкремента. Эта версия отличается от *префиксных* (*prefix*) версий, которые мы использовали до сих пор, тем, что выражение `begin++` возвращает копию исходного значения переменной `begin`, а в качестве побочного эффекта инкрементирует хранимое в ней значение. Другими словами, инструкция

```
it = begin++;
```

эквивалентна следующим двум:

```
it = begin;
++begin;
```

Оператор инкремента имеет такой же приоритет, как и оператор разыменования (*), и оба они правоассоциативны, а это означает, что выражение `*out++` имеет то же значение, что и выражение `*(out++)`. Следовательно, инструкция

```
*out++ = *begin++;
```

эквивалентна более «многословной»:

```
{ *out = *begin; ++out; ++begin; }
```

Теперь вернемся к *итераторным адаптерам* (*iterator adaptors*), которые представляют собой функции, генерирующие итераторы со свойствами, некоторым образом связанными с их аргументами. Итераторные адаптеры определены в заголовке `<iterator>`. Самым распространенным итераторным адаптером является `back_inserter`, принимающий в качестве аргумента контейнер и генерирующий итератор, который при использовании в качестве приемника присоединяет значения к концу заданного контейнера. Например,

`back_inserter(ret)` — это итератор, который в качестве приемника присоединяет элементы к концу контейнера `ret`. Таким образом, инструкция

```
copy(bottom.begin(), bottom.end(), back_inserter(ret));
```

копирует все элементы контейнера `bottom` и присоединяет их к концу контейнера `ret`. По завершении этой функции размер контейнера `ret` увеличится на значение выражения `bottom.size()`. Обратите внимание на то, что было бы ошибкой выполнить вызов

```
// Ошибка: ret - не итератор.  
copy(bottom.begin(), bottom.end(), ret);
```

поскольку третий аргумент алгоритма `copy` является итератором, а не контейнером. Ошибкой было бы также выполнение следующего вызова.

```
// Ошибка: в позиции ret.end() элемент отсутствует.  
copy(bottom.begin(), bottom.end(), ret.end());
```

Характер последней ошибки особенно коварный, поскольку компилятор пропустит такую инструкцию. Но последствия попытки выполнить ее — это совершенно другая история. Прежде всего, алгоритм `copy` попытается присвоить значение элементу, на который указывает итератор `ret.end()`. Однако в этой позиции элемент отсутствует, поэтому о соответствующей реакции вашей C++-среды на такой вызов можно только строить предположения.

Почему же алгоритм `copy` разработан именно таким образом? Благодаря отдельному выполнению копирования элементов и увеличения объема контейнера, программисты могут выбирать операции, которые им нужны в данный момент. Например, мы могли бы копировать элементы поверх уже существующих элементов контейнера, не изменяя размера самого контейнера. В качестве еще одного примера, который будет рассмотрен в разделе 6.2.2, мы попробуем использовать адаптер `back_inserter` для присоединения к концу контейнера элементов, которые не являются просто копиями элементов другого контейнера.

6.1.1. Еще один вариант функции `split`

С помощью стандартных алгоритмов можно более лаконично написать и функцию `split`, приведенную в разделе 5.6. Эта функция опирается на индексы, которые разграничивали каждое слово во входной строке. Мы можем заменить индексы итераторами и заставить поработать (на себя) стандартные алгоритмы.

```
// Возвращает значение true, если аргумент - пробел или  
// символ, подобный ему; в противном случае  
// возвращает значение false.  
bool space(char c)  
{  
    return isspace(c);  
}  
  
// Возвращает значение false, если аргумент - пробел или  
// символ, подобный ему; в противном случае  
// возвращает значение true.  
bool not_space(char c)  
{  
    return !isspace(c);  
}  
  
vector<string> split(const string& str)  
{  
    typedef string::const_iterator iter;
```

```

vector<string> ret;

iter i = str.begin();
while (i != str.end()) {

    // Игнорируем ведущие пробелы.
    i = find_if(i, str.end(), not_space);

    // Находим конец следующего слова.
    iter j = find_if(i, str.end(), space);

    // Копируем символы из диапазона [i, j).
    if (i != str.end())
        ret.push_back(string(i, j));
    i = j;
}
return ret;
}

```

В этом варианте функции `split` используется много новых функций, на которых имеет смысл остановиться. Ключевая идея — оставить тот же алгоритм, который был реализован в оригинальной функции, т.е. использовать переменные `i` и `j` для выделения каждого слова в объекте `str`. Обнаружив слово, мы копируем его из объекта `str` и помещаем копию в конец вектора `ret`.

Но на этот раз `i` и `j` — итераторы, а не индексы. Спецификатор объявления `typedef` позволяет более коротким именем назвать тип (в данном случае тип итератора), поэтому исключительно для удобства мы используем вместо более длинного имени `string::const_iterator` лаконичное `iter`. Хотя тип `string` не позволяет использовать все контейнерные операции, он поддерживает итераторы. Поэтому алгоритмы стандартной библиотеки мы можем применить к символам любого `string`-объекта, подобно тому, как мы делаем это по отношению к элементам любого `vector`-объекта.

В рассматриваемом здесь примере мы используем алгоритм `find_if`. Его первыми двумя аргументами являются итераторы, ссылающиеся на некоторые последовательности элементов; третий аргумент — предикат, который тестирует свой аргумент и возвращает значение `true` или `false`. Функция `find_if` вызывает заданный предикат для каждого элемента в последовательности и прекращает свою работу, когда найдет элемент, для которого этот предикат вернет значение `true`.

Функция `isspace`, предоставляемая стандартной библиотекой, проверяет, является ли заданный символ пробелом (либо символом, подобным пробелу). Однако эта функция перегружена, чтобы работать даже с таким языком, как японский, а подобный универсализм вынуждает ее использовать другие типы символов, а именно тип `wchar_t` (см. раздел 1.3). Оказывается, не так-то легко непосредственно передать шаблонной функции в качестве аргумента перегруженную функцию. Дело в том, что компилятор не знает, какую версию перегруженной функции мы имеем в виду, поскольку мы не предоставляем никаких аргументов, по которым компилятор мог бы выбрать нужную версию функции. Поэтому мы должны написать собственные предикаты, `space` и `not_space`, которые однозначно сообщат компилятору, какую версию функции `isspace` мы хотим вызвать.

При первом обращении к функции `find_if` выполняется поиск первого символа, отличного от пробела, с которого и начинается искомое слово. Вспомните: начало входной строки и разделитель смежных слов могут состоять из одного или нескольких пробелов, которые мы не должны включать в результат.

После первого обращения к функции `find_if` итератор `i` будет указывать на первый в объекте `str` символ, отличный от пробела (если таковой существует). Мы ис-

пользуем итератор `i` и при следующем обращении к функции `find_if`, которая должна найти первый пробел в диапазоне `[i, str.end())`. Если функции `find_if` не удастся найти значение, которое удовлетворяет предикату, она возвратит свой второй аргумент, которым в этом случае является значение `str.end()`. Следовательно, итератор `j` будет инициализирован значением, указывающим на пробел, который отделяет следующее слово в объекте `str` от остальной части строки, или, если мы достигли последнего слова в строке, он будет равен значению `str.end()`.

На этом этапе итераторы `i` и `j` определяют слово, найденное в объекте `str`. Теперь нам осталось использовать эти итераторы для копирования данных из объекта `str` в вектор `ret`. В предыдущей версии функции `split` (которая работала на основе индексов, а не итераторов) для создания копии мы применяли функцию `string::substr`. В новой же версии функции `split` используются итераторы, в то время как итераторной `substr`-версии не существует. Поэтому мы создаем новый `string`-объект непосредственно на основе имеющихся у нас итераторов. Для этого мы используем выражение `string(i, j)`, которое несколько напоминает определение функции `spaces` (см. раздел 1.2). В нашем нынешнем примере создается `string`-объект, который является копией символов из диапазона `[i, j)`. “Новоиспеченный” `string`-объект мы помещаем в конец вектора `ret`.

Необходимо отметить, что в этой версии программы опущено сравнение “бывшего” индекса `i` со значением `str.size()`. Дело в том, что эквивалентного сравнения итератора со значением `str.end()` не существует, да оно и не нужно, поскольку библиотечные алгоритмы написаны с целью “изящной” обработки вызовов, передающих пустой диапазон. Например, на определенном этапе первый вызов функции `find_if` установит итератор `i` равным значению, возвращаемому вызовом `str.end()`, но нет никакой необходимости проверять значение `i` до передачи его во время второго вызова функции `find_if`. А если функция `find_if` “заглянет” в пустой диапазон `[i, str.end())`, то она просто вернет значение `str.end()`, которое “красноречиво” говорит об отрицательном результате поиска.

6.1.2. Палиндромы

Еще одна задача обработки символов связана с использованием библиотеки для определения, является ли слово палиндромом. Палиндромы — это слова, одинаково читающиеся от начала к концу и от конца к началу, например “civic”, “eye”, “level”, “madam” и “rotor”.

Вот как выглядит компактное решение этой задачи с использованием библиотечных средств C++.

```
bool is_palindrome(const string& s)
{
    return equal(s.begin(), s.end(), s.rbegin());
}
```

Инструкция `return` в теле этой функции вызывает функцию `equal` и функцию-член `rbegin`, с которыми нам еще не приходилось встречаться.

Как и функция `begin`, функция-член `rbegin` возвращает итератор, но в данном случае мы имеем дело с итератором, который начинает свою работу с последнего элемента в контейнере и обходит контейнер в обратном направлении.

Функция `equal` сравнивает две последовательности с целью определения, содержат ли они равные значения. Обычно первые два итератора, передаваемые функции `equal` в качестве аргументов, задают первую последовательность. Третий аргумент означает

стартовую позицию второй последовательности. Функция `equal` работает в предположении, что вторая последовательность имеет такой же размер, как и первая, поэтому ей не нужен итератор конца последовательности. Поскольку в качестве стартовой позиции второй последовательности мы передаем выражение `s.rbegin()`, в результате этого вызова будут сравниваться значения с конца объекта `s` со значениями, взятыми с его начала. Функция `equal` будет сравнивать сначала первый символ объекта `s` с последним, затем второй со вторым от конца и т.д. Это именно то, что нам нужно для выявления палиндромов.

6.1.3. Поиск URL-адресов

В качестве последнего примера обработки символов напомним функцию для отыскания Web-адресов, именуемых URL-адресами (uniform resource locators), которые встроены в некоторый `string`-объект. Задача такой функции — просмотреть `string`-объект, в котором хранится все содержимое документа, и найти все URL-адреса, имеющиеся в нем.

URL-адрес представляет собой последовательность символов следующего формата.

имя_протокола://*имя_источника*

Здесь элемент *имя_протокола* содержит только буквы, а элемент *имя_источника* может состоять из букв, цифр и определенных знаков пунктуации. Наша функция будет принимать аргумент типа `string`, в котором она попытается найти экземпляры сочетания символов `“://”`. Каждый раз, когда функция находит такой экземпляр, она должна найти предшествующий ему элемент *имя_протокола* и следующий за ним элемент *имя_источника*.

Поскольку мы хотим, чтобы наша функция нашла все URL-адреса во входных данных, было бы логично, если бы она возвращала вектор типа `vector<string>`, элементами которого были бы найденные URL-адреса. Функция выполняется посредством перемещения итератора `b` по `string`-объекту, ища сочетание символов `“://”`, которые могут быть частью URL-адреса. Отыскав эти символы, функция “оглядывается” назад, чтобы найти элемент *имя_протокола*, а затем “заглядывает” вперед, чтобы найти элемент *имя_источника*.

```
vector<string> find_urls(const string& s)
{
    vector<string> ret;
    typedef string::const_iterator iter;
    iter b = s.begin(), e = s.end();

    // просмотр всего содержимого входных данных.
    while (b != e) {

        // поиск одной или нескольких букв, стоящих
        // за буквосочетанием “://”.
        b = url_beg(b, e);

        // Если поиск удался...
        if (b != e) {

            // ...получаем остальную часть URL-адреса.
            iter after = url_end(b, e);

            // Запоминаем URL-адрес.
            ret.push_back(string(b, after));
        }
    }
}
```

```

        // Передвигаем b для новой попытки отыскать
        // URL-адрес в этой строке.
        b = after;
    }
    return ret;
}

```

Функция начинается с объявления вектора `ret`, в который мы будем помещать URL-адреса по мере их отыскания, и итераторов, определяющих границы исходного `string`-объекта. Нам еще придется написать функции `url_beg` и `url_end`, которые будут находить во входных данных начало и конец любого URL-адреса. Функция `url_beg` должна “ответить на вопрос”, действительно ли мы имеем дело с URL-адресом, и в случае положительного ответа вернуть итератор, который ссылается на первый символ имени протокола. Если эта функция не идентифицирует URL во входных данных, она должна вернуть свой второй аргумент (`e` в данном случае), сигнализирующий о неудачном результате поиска.

Если функция `url_beg` отыщет URL-адрес, следующая наша задача — найти конец этого URL-адреса посредством вызова функции `url_end`. Эта функция начнет поиск с заданной позиции и будет продолжать его до тех пор, пока не обнаружит либо конец входных данных, либо символ, который не может быть частью URL-адреса. Функция `url_end` должна вернуть итератор, расположенный за последним символом URL-адреса.

Таким образом, после обращений к функциям `url_beg` и `url_end` итератор `b` будет указывать на начало URL-адреса, а итератор `after` — на позицию, расположенную за последним символом URL-адреса.



Мы создаем новый `string`-объект из символов, попадающих в данный диапазон, и помещаем его в конец вектора `ret`.

Нам остается лишь инкрементировать значение итератора `b` и перейти к поиску следующего URL-адреса. Поскольку URL-адреса не могут перекрывать друг друга, мы устанавливаем итератор `b` на позицию, расположенную за только что найденным URL-адресом, и продолжаем выполнение `while`-цикла до тех пор, пока не будут просмотрены все входные данные. По выходу из цикла мы возвращаем автору вызова функции `find_urls` вектор, который содержит все найденные URL-адреса.

Теперь можно вплотную заняться функциями `url_beg` и `url_end`. Функция `url_end` проще, поэтому с нее и начнем.

```

string::const_iterator
url_end(string::const_iterator b, string::const_iterator e)
{
    return find_if(b, e, not_url_char);
}

```

Эта функция просто переадресовывает свою работу библиотечной функции `find_if`, которую мы уже использовали в разделе 6.1.1. Предикат, который мы передаем функции `find_if` и который нам еще предстоит написать, называется

not_url_char. Он должен возвратить значение true, если переданный ему символ не может быть частью URL-адреса.

```
bool not_url_char(char c)
{
    // Символы, помимо буквенно-цифровых, которые
    // могут быть включены в URL-адрес.
    static const string url_ch = "~;/?:@=&$-_.+!*'(),";

    // Определяем, может ли символ оказаться частью
    // URL-адреса, и возвращаем отрицание значения ответа.
    return !(isalnum(c) ||
        find(url_ch.begin(), url_ch.end(), c) != url_ch.end());
}
```

Несмотря на свою миниатюрность, эта функция содержит немало новых для вас средств. Прежде всего, это *спецификатор статического класса памяти* (storage class specifier) `static`. Локальные переменные, которые объявляются с использованием этого спецификатора, сохраняются между вызовами функции. Таким образом, мы будем создавать и инициализировать `string`-объект `url_ch` только при первом обращении к функции `not_url_char`. Все последующие вызовы будут использовать объект, созданный при первом вызове. Поскольку `url_ch` представляет собой константный `string`-объект, его значение не будет изменяться после инициализации.

Кроме того, функция `not_url_char` использует функцию `isalnum`, которая определяется в заголовке `<cctype>`. Эта функция проверяет, является ли ее аргумент буквенно-цифровым символом (т.е. буквой или цифрой).

Наконец, `find` — это еще один алгоритм, который нам не доводилось пока использовать. Он похож на алгоритм `find_if`, за исключением того, что, вместо вызова предиката, он пытается отыскать конкретное значение, заданное в качестве третьего аргумента. Как и в случае функции `find_if`, если нужное нам значение существует, функция возвращает итератор, указывающий на первое вхождение этого значения в заданной последовательности. В противном случае алгоритм `find` возвращает свой второй аргумент.

Обладая этой информацией, нетрудно понять, как работает функция `not_url_char`. Поскольку мы инвертируем значение всего выражения до его возврата, функция `not_url_char` сгенерирует значение `false`, если аргумент `c` представляет собой букву, цифру или иной символ, включенный в `string`-объект `url_ch`. Если аргумент `c` содержит любое другое значение, функция возвратит значение `true`.

Теперь возьмемся за более трудную часть: реализацию функции `url_beg`. В этой функции должна быть предусмотрена возможность присутствия во входном потоке символов `“://”` в контексте, который не имеет отношения к действительному URL-адресу. На практике, вероятно, лучше иметь список допустимых имен протоколов, которые и следует искать. Однако для простоты ограничимся гарантией того, что разделителю `“://”` предшествует одна или несколько букв, а за ним стоит, по крайней мере, один символ.

```
string::const_iterator
url_beg(string::const_iterator b, string::const_iterator e)
{
    static const string sep = "://";

    typedef string::const_iterator iter;

    // Итератор i отмечает местонахождение разделителя.
    iter i = b;

    while ((i = search(i, e, sep.begin(), sep.end())) != e) {
```

```

// убеждаемся, что разделитель не находится
// в начале или конце строки.
if (i != b && i + sep.size() != e) {

    // итератор beg отмечает начало имени протокола.
    iter beg = i;
    while (beg != b && isalpha(beg[-1]))
        --beg;

    // Существует ли по крайней мере один подходящий
    // символ до и после разделителя?
    if (beg != i && !not_url_char(i[sep.size()]))
        return beg;
}

// найденный разделитель не является частью URL-адреса;
// перемещаем итератор i за этот разделитель.
i += sep.size();
}
return e;
}

```

Проще всего в этой функции — написать ее заголовок. Мы знаем, что функции будут переданы два итератора, обозначающие диапазон поиска, и что она должна вернуть итератор, указывающий на начало первого URL-адреса (если таковой существует) в этом диапазоне. Мы также объявляем и инициализируем локальный `string`-объект, который будет содержать символы разделителя, служащие признаком потенциального URL-адреса. Подобно `string`-объекту `url_ch` в функции `not_url_char` (см. раздел 6.1.3), этот `string`-объект — статический (`static`) и константный (`const`). Следовательно, мы не сможем изменить его значение, которое будет создано только при первом вызове функции `url_beg`.

Задача этой функции — так расположить два итератора `i` и `beg` в позициях `string`-объекта, ограниченного заданными в качестве аргументов итераторами `b` и `e`, чтобы итератор `i` указывал на начало разделителя URL-адреса, если таковой обнаружится, а итератор `beg` — на начало имени протокола.



Функция `url_beg` сначала ищет URL-разделитель посредством вызова библиотечной функции `search`, которую мы ранее еще не использовали. Эта функция принимает две пары итераторов: первая обозначает последовательность, в которой осуществляется поиск, а вторая — последовательность, которую мы хотим локализовать. Как и в случае неудачного исхода других библиотечных функций, функция `search` возвращает второй итератор. Следовательно, после обращения к функции `search` итератор `i` будет указывать либо на позицию за концом входного `string`-объекта, либо на символ `“:”`, за которым следуют два символа `“//”`.

Обнаружив разделитель, нужно выполнить следующую задачу — получить буквы (если таковые имеются), составляющие имя протокола. Первая проверка связана с местонахождением разделителя: если он находится в начале или конце входных дан-

ных, то мы знаем, что это — не URL-адрес. В противном случае необходимо попытаться разместить надлежащим образом итератор `beg`. Внутренний `while`-цикл перемещает итератор `beg` в обратном направлении вдоль входного `string`-объекта до тех пор, пока он не обнаружит либо неалфавитный символ, либо начало строки. Здесь стоит обратить ваше внимание на два новшества: первое состоит в том, что если некоторый контейнер поддерживает индексирование, то он поддерживает и итераторы. Другими словами, `beg[-1]` — это символ, расположенный в позиции, находящейся непосредственно перед той, на которую указывает итератор `beg`. Запись `beg[-1]` можно расценивать как сокращенное обозначение выражения `*(beg - 1)`. Подробнее о таких итераторах мы поговорим в разделе 8.2.6. Второе новшество — использование функции `isalpha` (определенной в заголовке `<cctype>`), которая проверяет, является ли ее аргумент буквой.

Если бы мы могли передвигать итератор на один символ, то, предположительно, могли бы найти имя протокола. Прежде чем возвратить значение итератора `beg`, мы по-прежнему должны проверить существование хотя бы одного действительного символа, следующего за разделителем. Эта проверка несколько сложнее. Мы знаем, что во входном `string`-объекте существует по крайней мере еще один символ, поскольку находимся внутри тела `if`-инструкции, которая сравнивает значение выражения `i + sep.size()` со значением итератора `e`. Мы можем получить доступ к первому такому символу с помощью выражения `i[sep.size()]`, которое можно считать сокращенным вариантом записи `*(i + sep.size())`. Мы проверяем, может ли этот символ входить в состав URL-адреса, передав его функции `not_url_char`. Эта функция возвращает значение `true`, если исследуемый символ недействителен, поэтому мы инвертируем возвращаемое значение, чтобы узнать, является ли исследуемый символ действительным.

Если разделитель не является частью URL-адреса, функция перемещает итератор `i` за этот разделитель и продолжает поиск.

В рассмотренном коде применяется *оператор декремента* (`decrement operator`), который упоминался в таблице, приведенной в разделе 2.7, но ранее не использовался. Он работает подобно оператору инкремента, но не инкрементирует, а декрементирует (т.е. уменьшает на единицу) свой операнд. Как и оператор инкремента, он существует в префиксной и постфиксной версиях. Префиксная версия, которая используется в данном случае, декрементирует операнд и возвращает новое значение.

6.2. Сравнение схем вычисления оценок

В разделе 4.2 мы представили схему для вычисления итоговых оценок студентов с помощью метода использования медианного значения оценок, полученных за выполнение домашних заданий. Некоторые предприимчивые студенты могли бы использовать эту схему, сознательно не включая всех своих оценок за домашние задания. Если они не сделали *всех* домашних заданий, гарантирующих хорошую оценку, то почему бы при расчете итоговой оценки не ограничиться реальными “достижениями”, чтобы “неуды” не портили общую картину?

В нашей практике мало кто из студентов прибегал к этой лазейке. Но у нас была возможность показать ее одной группе студентов, которые не преминули (причем с радостью) открыто ею воспользоваться. Мы удивились тому, что итоговые оценки студентов, которые не выполнили домашние задания в полном объеме, отличались в среднем от итоговых оценок тех, кто выполнил дома абсолютно все задания. Обдумывая, как объяснить такой результат, мы решили, что было бы интересно узнать, какой

был бы результат, если бы мы использовали одну из двух альтернативных схем вычисления итоговых оценок:

- вычисление среднего вместо медианного значения и использование нулевых значений в качестве оценок за невыполненные домашние задания;
- вычисление медианы только для реально предоставленных оценок.

Для каждой из этих схем вычисления итоговых оценок интересно было бы сравнить медианную оценку студентов, которые включили для подведения итогов все свои домашние задания, с медианной оценкой тех, кто опустил хотя бы одно из них. Таким образом, мы нагрузили сами себя программой, которая должна решить следующие две отдельные подзадачи.

1. Прочитать все записи студентов, отделяя от остальных те, в которых представлены оценки за *все* домашние задания.
2. Применить каждую из описанных выше схем вычисления итоговых оценок ко всем студентам каждой группы и вычислить медианную оценку каждой группы.

6.2.1. Обработка записей с оценками студентов

Наша первая подзадача — прочитать и классифицировать записи. К счастью, у нас уже есть заготовка, которую можно использовать для решения этой части задачи: мы имеем в виду тип `Student_info` из раздела 4.2.1 и соответствующую функцию `read` из раздела 4.2.2, предназначенную для чтения записей с данными об успеваемости студентов. Но у нас пока нет функции, которая проверяла бы, все ли домашние задания выполнил студент. Создание такой функции не составит большого труда.

```
bool did_all_hw(const Student_info& s)
{
    return ((find(s.homework.begin(), s.homework.end(), 0))
            == s.homework.end());
}
```

Эта функция “заглядывает” в член данных `s.homework`, чтобы узнать, равны ли нулю какие-либо из хранимых там значений. Оценка 0 означает, что соответствующее задание не было сдано студентом на проверку. Мы сравниваем значение, возвращаемое функцией `find`, со значением `homework.end()`. Если функции `find` не удастся найти искомое значение, она должна вернуть свой второй аргумент.

С помощью этих двух функций написание кода считывания и классификации записей проще пареной репы. Достаточно прочитать каждую запись, узнать, сделал ли студент все домашние задания, и в зависимости от результата добавить эту запись в конец одного из двух векторов, которые для простоты назовем `did` и `didnt`. Если окажется, что ни один из упомянутых векторов не пустует, значит, мы не зря проводим такой анализ и можем получить интересную информацию.

```
vector<Student_info> did, didnt;
Student_info student;

// читаем все записи, разделяя их по принципу выполнения
// (и невыполнения) всех домашних заданий.
while (read(cin, student)) {
    if (did_all_hw(student))
        did.push_back(student);
    else
        didnt.push_back(student);
}
```

```
// проверяем, обе ли группы содержат данные.
if (did.empty()) {
    cout << "Ни один студент не выполнил всех домашних заданий!"
    << endl;
    return 1;
}
if (didnt.empty()) {
    cout << "Все студенты выполнили все домашние задания!"
    << endl;
    return 1;
}
```

Новинкой для вас здесь можно считать только функцию-член `empty`, которая возвращает значение `true`, если контейнер пуст, и значение `false` — в противном случае. Использование этой функции — более удачный вариант проверки “пустоты” контейнера, чем сравнение размера исследуемого контейнера с нулем, поскольку для контейнеров некоторых типов более эффективной является проверка наличия в контейнере каких-либо элементов, а не подсчет точного количества элементов, содержащихся в нем.

6.2.2. Анализ оценок

Теперь мы знаем, как прочитать и разделить записи на векторы `did` и `didnt`. Следующий этап — анализ оценок.

Итак, мы должны выполнить три вида анализа; каждый вид состоит из двух частей, т.е. анализу подвергаются записи с данными об успевающих студентах (которые выполнили все домашние задания) и отдельно с данными о тех, кто не сделал какое-либо из заданных на дом заданий. Поскольку каждый вид анализа мы будем проводить на двух наборах данных, было бы целесообразно иметь для каждого вида собственную функцию. Однако существуют некоторые операции, например создание отчета в общем формате, который было бы желательно выполнять попарно, а не в отдельности для каждого набора данных. Несомненно, имело бы смысл представить вывод каждой пары результатов анализа также в виде функции.

Интересно то, что мы собираемся три раза вызывать функцию, которая выводит результаты анализа, по одному разу для каждого вида анализа. При этом мы хотим, чтобы эта функция дважды вызывала соответствующую функцию анализа, по одному разу для объектов `did` и `didnt`. Но в то же время нам нужно, чтобы в каждом обращении к функции, генерирующей отчет, имел место вызов различных функций анализа! Как же реализовать такие грандиозные планы?

Проще всего определить три функции анализа и передавать каждую из них функции генерации отчета в качестве аргумента. Вспомните: мы уже использовали такие аргументы-функции, когда передавали функцию `compare` библиотечной функции `sort` (см. раздел 4.2.2). В этом случае мы хотели бы, чтобы наша функция принимала пять аргументов:

- поток, в который выводится результат;
- `string`-объект, представляющий имя анализа;
- функция, используемая для анализа;
- два аргумента, представляющие векторы, подлежащие анализу.

Предположим, что первый анализ, который оценивает метод медиан, выполняется функцией `median_analysis`. Затем мы хотели бы вывести результаты для каждой группы студентов с помощью следующего вызова.

```
write_analysis(cout, "median", median_analysis, did, didnt);
```


Прежде чем браться за функцию `write_analysis`, займемся сначала функцией `median_analysis`. Мы хотели бы передать этой функции `vector`-объект записей с оценками студентов и получить от нее медианное значение этих оценок, вычисленное по обычной схеме. Итак, мы можем определить эту функцию следующим образом.

```
// Эта функция не вполне работоспособна.  
double median_analysis(const vector<Student_info>& students)  
{  
    vector<double> grades;  
  
    transform(students.begin(), students.end(),  
              back_inserter(grades), grade);  
    return median(grades);  
}
```

Хотя эта функция может показаться трудной на первый взгляд, она содержит только одну новинку, а именно функцию `transform`. Функция `transform` принимает три итератора и функцию преобразования. Первые два итератора определяют диапазон элементов, подлежащих преобразованию; третий итератор означает позицию приемника, в которую помещается результат выполнения заданной функции преобразования.

При вызове функции `transform` мы должны сами побеспокоиться о том, чтобы приемник имел место (память) для значений из входной последовательности. В этом случае нет никаких проблем с памятью, поскольку мы получаем приемник посредством вызова функции `back_inserter` (см. раздел 6.1), тем самым гарантируя, что результаты функции `transform` будут присоединяться к концу вектора `grades`, который при необходимости будет увеличиваться автоматически.

Четвертый аргумент — функция, которую функция `transform` применяет к каждому элементу входной последовательности для получения соответствующего элемента в выходной последовательности. Таким образом, результатом вызова функции `transform` в этом примере является применение функции `grade` к каждому элементу вектора `students` с последующим присоединением каждой вычисленной оценки к концу вектора `grades`. Вычислив все оценки студентов, мы можем вызвать функцию `median`, которую определили в разделе 4.1.1, для вычисления их медианного значения.

Осталось решить только одну проблему: заставить эту функцию заработать (как указано в комментарии, она *не вполне работоспособна*).

Первая причина ее неработоспособности состоит в том, что существует несколько перепруженных версий функции `grade`. Компилятор не знает, какую из них вызвать, поскольку мы не передали никаких аргументов. Мы-то имеем в виду вызов версии из раздела 4.2.2, но нам нужно иметь способ сообщить компилятору о своем намерении.

Вторая причина заключается в генерировании функцией `grade` исключения, если какой-нибудь студент вообще не выполнил ни одного домашнего задания. Дело в том, что функция `transform` ничего не “знает” ни о каких исключениях. При генерировании исключения функция `transform` прекратит свою работу в “точке” его возникновения, и управление будет возвращено функции `median_analysis`. Поскольку функция `median_analysis` также не обрабатывает исключений, “волна” исключений распространится дальше. В результате выполнение этой функции также будет преждевременно прекращено с передачей управления автору ее вызова и так до тех пор, пока не будет обнаружена соответствующая инструкция перехвата исключения `catch`. Если такая инструкция `catch` отсутствует, как в данном случае, прекратит работу сама программа, о чем будет оглашено (или не оглашено — все зависит от конкретной C++-среды) в соответствующем сообщении.

Обе проблемы можно решить, написав вспомогательную функцию для “заключения” функции `grade` в “объятия” `try`-блока и соответствующей обработки возможного исключения. Поскольку мы теперь будем вызывать функцию `grade` в явном виде, а не посредством передачи ее в качестве аргумента, компилятор легко сможет “понять”, какую версию мы имеем в виду.

```
double grade_aux(const Student_info& s)
{
    try {
        return grade(s);
    } catch (domain_error) {
        return grade(s.midterm, s.final, 0);
    }
}
```

Эта вспомогательная функция и станет вызывать версию функции `grade` из раздела 4.2.2. Если будет сгенерировано исключение, мы перехватим его (с помощью инструкции `catch`) и вызовем версию функции `grade` из раздела 4.1, которая принимает три `double`-аргумента, представляющих экзаменационные оценки и общую оценку за выполнение домашних заданий. Таким образом, мы предполагаем, что студент, который вообще не выполнил ни одного домашнего задания, получает 0 баллов в этой “номинации”, но его результаты на экзаменах (пусть, чего уж там) по-прежнему учитываются в общем зачете.

Теперь можно переписать функцию анализа результатов, сориентировав ее на использование функции `grade_aux`.

```
// Эта версия прекрасно работает!
double median_analysis(const vector<Student_info>& students)
{
    vector<double> grades;

    transform(students.begin(), students.end(),
               back_inserter(grades), grade_aux);
    return median(grades);
}
```

Теперь мы практически готовы определить функцию `write_analysis`, которая использует функцию `analysis` для сравнения двух подгрупп студентов.

```
void write_analysis(ostream& out, const string& name,
                   double analysis(const vector<Student_info>&),
                   const vector<Student_info>& did,
                   const vector<Student_info>& didnt)
{
    out << name << ": median(did) = " << analysis(did) <<
        ", median(didnt) = " << analysis(didnt) << endl;
}
```

И снова-таки, несмотря на миниатюрность этой функции, в ней содержится аж две новинки. Первая заключается в определении параметра, который представляет функцию. Определение параметра-функции (в данном случае параметром является функция `analysis`) напоминает определение функции, приведенное в разделе 4.3. (И в самом деле, как мы узнаем в разделе 10.1.2, здесь сокрыто несколько больше, чем видно “невооруженным” глазом.)

Вторая новинка — это `void`, тип значения, возвращаемого функцией. Встроенный тип `void` можно использовать только в некоторых случаях, одним из которых и является указание типа возвращаемого функцией значения. Если мы говорим, что функция “возвращает” `void`-значение, то на самом деле имеем в виду, что эта функция не

возвращает никакого значения. Выйти из такой функции можно с помощью инструкции `return` без указания какого бы то ни было значения.

```
return;
```

Из `void`-функции можно выйти и без инструкции `return`, как в данном случае, “не прощаясь”, просто опустив явный конец функции. В обычном случае мы не можем опустить конец функции, но язык C++ позволяет это делать функциям, которые возвращают `void`-значение.

Теперь мы можем написать остальную часть нашей программы.

```
int main()
{
    // Студенты, которые выполнили и не выполнили
    // все домашние задания.
    vector<Student_info> did, didnt;

    // читаем все записи, разделяя их по принципу выполнения
    // (и невыполнения) всех домашних заданий.
    Student_info student;
    while (read(cin, student)) {
        if (did_all_hw(student))
            did.push_back(student);
        else
            didnt.push_back(student);
    }

    // Удовверяемся, что результаты анализа существуют
    // и нам есть что показать.
    if (did.empty()) {
        cout <<
            "Ни один студент не выполнил всех домашних заданий!"
            << endl;
        return 1;
    }

    if (didn't.empty()) {
        cout << "Все студенты выполнили все домашние задания!"
            << endl;
        return 1;
    }

    // Выполнение всех видов анализа.
    write_analysis(cout, "median", median_analysis,
        did, didnt);
    write_analysis(cout, "average", average_analysis,
        did, didnt);
    write_analysis(cout, "median of homework turned in",
        optimistic_median_analysis, did, didnt);

    return 0;
}
```

Теперь нам осталось написать функции `average_analysis` и `optimistic_median_analysis`.

6.2.3. Вычисление итоговых оценок на основе среднего арифметического значения оценок за домашние задания

Здесь нам нужно, чтобы функция `average_analysis` вычисляла оценки студентов, используя не медианное, а среднее арифметическое значение оценок за домашние задания. Следовательно, сначала имеет смысл написать функцию вычисления среднего

арифметического от значений, содержащихся в векторе, которую затем мы будем использовать при вычислении итоговых оценок вместо функции `median`.

```
double average(const vector<double>& v)
{
    return accumulate(v.begin(), v.end(), 0.0) / v.size();
}
```

Эта функция использует алгоритм `accumulate`, который, в отличие от других уже используемых нами библиотечных алгоритмов, объявляется в заголовке `<numeric>`. Как следует из имени заголовка, он предлагает средства для численных расчетов. Функция `accumulate` выполняет сложение чисел из диапазона, заданного первыми двумя аргументами, начиная суммирование с исходного значения (т.е. начального значения суммы), заданного третьим аргументом.

Тип суммарного значения определяется типом третьего аргумента, поэтому для нас крайне важно использовать значение `0.0`, а не просто `0`. В противном случае результат имел бы тип `int`, и любая дробная часть была бы утеряна.

Воспользовавшись функцией `accumulate` для получения суммы всех элементов заданного диапазона, мы делим эту сумму на значение `v.size()`, которое равно количеству элементов в этом диапазоне. Результат деления, конечно же, представляет собой среднее арифметическое значение, которое возвращается автору вызова нашей функции.

Имея функцию `average`, мы можем использовать ее для реализации функции `average_grade`, которая позволяет отразить альтернативный способ вычисления итоговых оценок.

```
double average_grade(const Student_info& s)
{
    return grade(s.midterm, s.final, average(s.homework));
}
```

Эта функция использует функцию `average` для вычисления общей оценки за домашние задания, которая затем передается функции `grade` из раздела 4.1 для вычисления итоговой оценки. При такой инфраструктуре функция `average_analysis` — сама простота.

```
double average_analysis(const vector<Student_info>& students)
{
    vector<double> grades;
    transform(students.begin(), students.end(),
              back_inserter(grades), average_grade);
    return median(grades);
}
```

Единственное различие между этой функцией и функцией `median_analysis` (см. раздел 6.2.2), помимо имени, состоит в использовании функции `average_grade` вместо функции `grade_aux`.

6.2.4. Медиана оценок, полученных за выполненные домашние задания

В последней схеме анализа используется функция `optimistic_median_analysis`, которая обязана своим именем оптимистическому допущению, что оценки студентов за домашние задания, не предоставленные преподавателям на проверку, не отличаются от оценок за предоставленные домашние задания. При таком допущении было бы интересно вычислить медиану, учитывающую только оценки за предоставленные домашние задания. Начнем с написания функции вычисления этой медианы, а само

вычисление назовем *оптимистической медианой*. Безусловно, мы должны учесть вероятность того, что студент вообще не сделает ни одного домашнего задания, и тогда в качестве общей оценки за домашние задания будем использовать число 0.

```
// Возвращает медиану ненулевых элементов вектора s.homework
// или 0, если такие элементы отсутствуют.
double optimistic_median(const Student_info& s)
{
    vector<double> nonzero;
    remove_copy(s.homework.begin(), s.homework.end(),
               back_inserter(nonzero), 0);

    if (nonzero.empty())
        return grade(s.midterm, s.final, 0);
    else
        return grade(s.midterm, s.final, median(nonzero));
}
```

Эта функция извлекает ненулевые элементы из вектора `homework` и помещает их в новый вектор с именем `nonzero`. Имея ненулевые оценки за выполненные домашние задания, мы вызываем версию функции `grade`, определенную в разделе 4.1, для вычисления итоговой оценки на основе медианы оценок, полученных за реально предоставленные домашние задания.

Единственное новшество в этой функции — способ занесения значений в вектор `nonzero` с помощью алгоритма `remove_copy`. Чтобы понять обращение к алгоритму `remove_copy`, важно знать, что библиотека предоставляет “копируемые” версии многих алгоритмов. Так, например, алгоритм `remove_copy` выполняет те же действия, что и алгоритм `remove`, но при этом копирует его результаты в указанное место.

Функция `remove` находит все значения, которые совпадают с заданным, и “удаляет” их из контейнера. Все значения из входной последовательности, которые не “удалены”, будут скопированы в заданное место назначения. Чуть ниже мы раскроем, что означает взятое в кавычки слово “удалены” в этом контексте.

Функция `remove_copy` принимает в качестве параметров три итератора и значение. Как и в случае большинства алгоритмов, первые два итератора означают входную последовательность. Третий указывает на начало приемника скопированных элементов. Подобно алгоритму `copy`, алгоритм `remove_copy` предполагает наличие достаточного объема памяти приемника, чтобы принять все скопированные в него элементы. Поэтому при необходимости вызывается функция `back_inserter`, чтобы увеличить размер вектора `nonzero`.

Теперь можно понять, что результатом вызова алгоритма `remove_copy` является копирование в вектор `nonzero` всех ненулевых элементов из вектора `s.homework`. Затем мы проверяем, не пустой ли оказался вектор `nonzero`, и если нет, то выполняем вычисление с помощью функции `grade` на базе функции `median`, принимающей в качестве исходного “материала” оценки, содержащиеся в векторе `nonzero`. Если вектор `nonzero` пуст, в качестве общей оценки за домашние задания используем значение 0.

Конечно же, для выполнения всех трех видов анализа нам необходимо написать еще функцию, которая будет вызывать функцию `optimistic_median`. Мы решили оставить эту работу для вас, чтобы вы попытались сделать ее в качестве упражнения.

6.3. Новый вариант классификации студентов

В главе 5 мы рассмотрели задачу копирования записей с неудовлетворительными оценками в отдельный вектор с последующим их удалением из исходного вектора. Однако мы

столкнулись с тем, что по мере увеличения размера исходного вектора производительность нашей программы резко падает. Мы показали, что проблема производительности решается путем замены вектора списком, и пообещали вернуться к этой задаче и показать алгоритмическое решение, которое подходило бы к новой структуре данных.

Для демонстрации двух других решений можем воспользоваться библиотекой алгоритмов. Реализация первого решения не слишком радует своим быстродействием, поскольку в ней используется пара библиотечных алгоритмов и доступ к каждому элементу выполняется дважды. Улучшить результат можно с помощью более специализированного библиотечного алгоритма, который позволит нам решить проблему не в два, а только в один проход.

6.3.1. Решение в два прохода

В нашем первом решении используется стратегия, аналогичная той, которую мы применяли в разделе 6.2.4, когда учитывали только ненулевые значения оценок за домашние задания. Тогда мы не хотели изменять сам вектор `homework`, потому и воспользовались алгоритмом `remove_copy` для внесения копий ненулевых значений оценок за домашние задания в отдельный вектор. В нашей задаче нам нужно и копировать, и удалять ненулевые элементы.

```
vector<Student_info>
extract_fails(vector<Student_info>& students) {
    vector<Student_info> fail;
    remove_copy_if(students.begin(), students.end(),
                   back_inserter(fail), pgrade);
    students.erase(remove_if(students.begin(), students.end(),
                             fgrade), students.end());
    return fail;
}
```

Интерфейс этой программы идентичен интерфейсу из раздела 5.3, в котором представлено вектор-ориентированное решение, использующее итераторы вместо индексов. Как и в упомянутом решении, мы будем использовать вектор-объект, который передавали для хранения оценок успевающих студентов, и определяемый нами вектор `fail` для хранения неудовлетворительных оценок. На этом все сходства и заканчиваются.

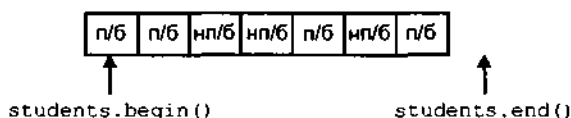
В исходной программе мы использовали итератор `iter` для прохода по контейнеру, копирования записей с неудовлетворительными оценками в вектор `fail` и для удаления последних из вектора `students` с помощью функции-члена `erase`. На этот раз для копирования записей с неудовлетворительными оценками в вектор `fail` мы используем функцию `remove_copy_if`. Эта функция действует подобно функции `remove_copy`, которую мы применили в разделе 6.2.4, за исключением того, что для проверки она использует не заданное значение, а предикат, по сути, инвертирующий всего лишь результат вызова функции `fgrade` (см. раздел 5.1).

```
bool pgrade(const Student_info& s)
{
    return !fgrade(s);
}
```

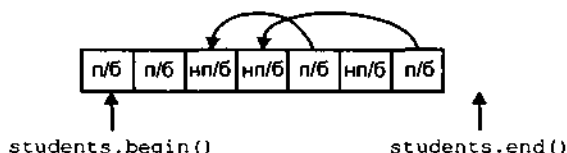
Передавая предикат функции `remove_copy_if`, мы тем самым просим ее “удалить” все элементы, которые удовлетворяют этому предикату. В данном контексте “удалить” означает *не копировать*, поэтому мы копируем только те элементы, которые не удовлетворяют предикату. Следовательно, при передаче предиката `pgrade` функции `remove_copy_if` копируются только записи с неудовлетворительными оценками.

Следующая инструкция посложнее предыдущей. Прежде всего, мы вызываем функцию `remove_if` для “удаления” элементов, которые соответствуют неудовлетворительным оценкам. И снова-таки, мы не можем обойтись без кавычек в слове “удаления”, поскольку на самом деле ничего не удаляется. Вместо удаления функция `remove_if` копирует все элементы, которые не удовлетворяют предикату (в данном случае — все записи с удовлетворительными оценками).

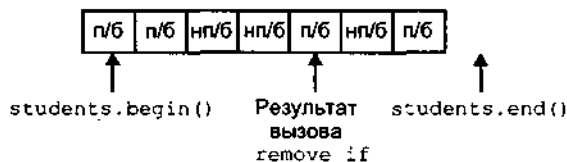
Этот вызов не прост для понимания, поскольку функция `remove_if` использует ту же последовательность, что ее аргумент-источник и аргумент-приемник. Ее реальное действие состоит в копировании в начало последовательности тех элементов, которые не отвечают условию предиката. Предположим, мы начали с семи студентов, имеющих такие оценки (*п/б* означает “проходной балл”, а *нп/б* — “непроходной балл”).



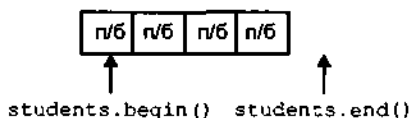
Тогда обращение к функции `remove_if` оставит первые две записи нетронутыми, поскольку они уже находятся на нужных местах. А следующие две будут “удалены”; вернее, с ними функция будет обращаться как с свободным местом, при необходимости перезаписывая их содержимое следующими записями, “заслуживающими” сохранения. Словом, когда функция доберется до пятой записи, представляющей успевающего студента, она скопирует ее на “свободное” место, которое использовалось для первой “удаленной” записи с неудовлетворительной оценкой, и т.д.



В этом случае результат выразится в копировании четырех записей, представляющих успевающих студентов, в начало последовательности, а оставшиеся три останутся “нетронутыми”. Чтобы мы могли знать, какая часть последовательности все еще релевантна, функция `remove_if` возвращает итератор, который ссылается на первый элемент, расположенный за последним “неудаленным” элементом.



Следующая инструкция функции `extract_fails` представляет собой вызов функции `erase`, которая должна удалить ненужные записи из вектора `students`. До сих пор мы не использовали эту версию функции `erase`. Она удаляет все элементы из диапазона, ограниченного итераторами, которые передаются функции в качестве аргументов. Если удалить (с помощью функции `erase`) элементы, расположенные в позициях между итератором, возвращаемым функцией `remove_if`, и итератором `students.end()`, у нас останутся только записи, содержащие проходные баллы.



6.3.2. Решение в один проход

Наше первое алгоритмическое решение выполняется довольно сносно, но у нас есть возможность улучшить его. Дело в том, что решение, приведенное в разделе 6.3.1, вычисляет итоговую оценку для каждого элемента вектора `students` дважды: первый раз — с помощью функции `remove_copy_if` и второй раз — с помощью функции `remove_if`.

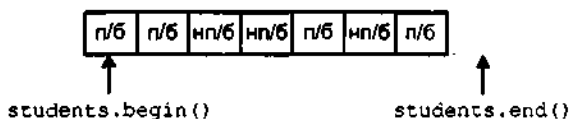
Хотя и не предусмотрен библиотечный алгоритм, который бы в точности реализовал наше желание, существует другой алгоритм, который рассматривает нашу проблему под другим углом: он принимает последовательность и переставляет в ней элементы таким образом, чтобы те, которые удовлетворяют предикату, предшествовали тем, которые не удовлетворяют ему.

В действительности существует две версии этого алгоритма: `partition` и `stable_partition`. Разница между ними в том, что алгоритм `partition` может переставлять элементы внутри каждой категории, в то время как алгоритм `stable_partition`, помимо разбиения на категории, сохраняет их в прежнем порядке. Например, если имена студентов уже были расположены в алфавитном порядке и мы захотели сохранить этот порядок внутри каждой категории, нам нужно использовать алгоритм `stable_partition`, а не `partition`.

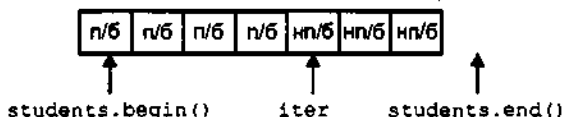
Каждый из алгоритмов возвращает итератор, который представляет первый элемент второй категории. Следовательно, мы можем извлечь записи с неудовлетворительными оценками таким способом.

```
vector<Student_info>
extract_fails(vector<Student_info>& students)
{
    vector<Student_info>::iterator iter =
        stable_partition(
            students.begin(), students.end(), pgrade);
    vector<Student_info> fail(iter, students.end());
    students.erase(iter, students.end());
    return fail;
}
```

Чтобы понять, что здесь происходит, давайте снова рассмотрим наши гипотетические данные.



После вызова функции `stable_partition` мы должны получить следующий результат.



Мы создаем вектор `fail` из копии записей с непроходными баллами, которые находятся в диапазоне `[iter, students.end())`, а затем удаляем эти элементы из вектора `students`.

Когда мы выполнили наши программы, основанные на алгоритмических решениях, оказалось, что они имеют приблизительно ту же производительность, что и `list`-ориентированное решение. Как и ожидалось, при больших объемах входных данных алгоритмические и `list`-ориентированное решения были существенно лучше, чем `vector`-решение, использующее функцию `erase`. Два алгоритмических решения вполне приемлемо обрабатывали входные файлы, содержащие до 75 000 записей. Чтобы сравнить результаты двух стратегий, используемых в функции `extract_fails`, мы отдельно проанализировали быстродействие только этой части программы. Время, необходимое для выполнения своей задачи однопроходному алгоритму, оказалось вдвое меньше времени, которое потребовалось двухпроходному алгоритму.

6.4. Алгоритмы, контейнеры и итераторы

Для понимания использования алгоритмов, итераторов и контейнеров необходимо знать следующее.

Алгоритмы обрабатывают элементы контейнера, а не сами контейнеры.

Функции `sort`, `remove_if` и `partition` перемешивают элементы на новые позиции заданного контейнера, но не изменяют свойства самого контейнера. Например, функция `remove_if` не изменяет размер контейнера, в котором она действует; она просто копирует элементы в пределах того же контейнера.

Это особенно важно для понимания принципов взаимодействия алгоритмов с контейнерами, которые используются в качестве приемников. Рассмотрим более подробно применение функции `remove_if` в разделе 6.3.1. Как вы видели, вызов

```
remove_if(students.begin(), students.end(), fgrade)
```

не изменил размер вектора `students`, а лишь скопировал все элементы, для которых заданный предикат был ложным (`false`), в начало вектора `students`, оставив в покое остальные элементы. Если вам понадобится уменьшить этот вектор, избавившись от ненужных элементов, вы должны сделать это сами.

В нашем примере мы использовали следующую инструкцию.

```
v.erase(remove_if(students.begin(), students.end(), fgrade),  
         students.end());
```

Здесь функция `erase` изменяет вектор посредством удаления последовательности, заданной ее аргументами. Это обращение к функции `erase` укорачивает вектор `students`, чтобы в нем хранились только нужные для нас элементы. Обратите внимание на то, что функция `erase` должна быть членом класса `vector`, поскольку она воздействует непосредственно на контейнер, а не просто на его элементы.

Кроме того, важно знать о взаимодействии между итераторами и алгоритмами, а также между итераторами и контейнерными операциями. Мы уже видели в разделах 5.3 и 5.5.1, что такие контейнерные операции, как `erase` и `insert`, делают итератор недействительным для удаленного элемента. Более важно то, что в случае использования `vector`- и `string`-объектов такие операции, как `erase` и `insert`, также делают недействительными любые итераторы, указывающие на элементы, расположенные после удаленного или вставленного элемента. Поэтому, используя упомянутые операции, вы должны с большой осторожностью выполнять сохранение значений итераторов.

Аналогично такие функции, как `partition` или `remove_if`, которые могут перемещать элементы в пределах контейнера, будут изменять соответствие элементов конкретным итераторам. После выполнения одной из таких функций нельзя полагаться на итератор, который продолжает ссылаться на конкретный элемент.

6.5. Резюме

Модификаторы типа

`static Тип имя_переменной;`

Для локальных объявлений объявляет переменную *имя_переменной* с использованием статического класса памяти. Значение этой переменной сохраняется в течение всего времени выполнения программы в этой области видимости, и гарантируется ее инициализация до первого использования. Когда программа выходит из этой области видимости, переменная сохраняет свое значение до тех пор, пока программа не войдет в нее в следующий раз. В разделе 13.4 мы увидим, что значение модификатора `static` меняется в соответствии с контекстом

Типы. Встроенный тип `void` можно использовать в ограниченном ряде случаев, один из которых позволяет сообщить, что функция не возвращает никакого значения. Такие функции могут завершаться как посредством инструкции `return;` (без указания значения), так и без специального признака конца функции.

Итераторные адаптеры — это функции, которые генерируют итераторы. Самыми распространенными являются адаптеры, генерирующие так называемые `insert-итераторы`, которые динамически увеличивают соответствующий контейнер. Такие итераторы можно безопасно использовать в алгоритме копирования в качестве приемника. Они определены в заголовке `<iterator>`.

`back_inserter(c)` Генерирует итератор для добавления элементов в конец контейнера `c`. Этот контейнер должен поддерживать операцию `push_back`, что имеет место для типов `list`, `vector` и `string`

`front_inserter(c)` Подобен адаптеру `back_inserter`, но вставляет элементы в начало контейнера `c`. Контейнер должен поддерживать операцию `push_front`, что имеет место для типа `list`, но не для типов `vector` и `string`

`inserter(c, it)` Подобен адаптеру `back_inserter`, но вставляет элементы перед итератором `it`

Алгоритмы. Если не оговорено иное, заголовок `<algorithm>` определяет следующие алгоритмы.

`accumulate(b, e, t)`

Создает локальную переменную (такого же типа, как у параметра `t`; это означает, что тип параметра `t` крайне важен для поведения алгоритма `accumulate`), инициализирует ее копией параметра `t`, добавляет каждый элемент из диапазона `[b, e)` в созданную переменную и возвращает в качестве результата копию этой переменной. Определен в заголовке `<numeric>`

`find(b, e, t)`
`find_if(b, e, p)`
`search(b, e, b2, e2)`

Алгоритмы, предназначенные для поиска заданного значения в последовательности `[b, e)`. Алгоритм `find` выполняет поиск значения `t`; алгоритм `find_if` проверяет каждый элемент на соответствие предикату `p`; алгоритм `search` ищет последовательность, заданную диапазоном `[b2, e2)`

`copy(b, e, d)`
`remove_copy(b, e, d, t)`
`remove_copy_if(b, e, d, p)`

Алгоритмы, используемые для копирования последовательности из диапазона `[b, e)` в приемник, заданный параметром `d`. Алгоритм `copy` копирует всю последовательность; алгоритм `remove_copy` — все элементы, не равные значению `t`; алгоритм `remove_copy_if` — все элементы, для которых предикат `p` возвращает значение `false`

`remove_if(b, e, p)`

Реорганизует контейнер, чтобы элементы в диапазоне `[b, e)`, для которых предикат `p` возвращает значение `false`, находились в начале этого диапазона. Возвращает итератор, указывающий на элемент, расположенный за диапазоном “неудаленных” элементов

`remove(b, e, t)`

Работает подобно алгоритму `remove_if`, но сохраняет элементы на основе результата сравнения их со значением `t`

`transform(b, e, d, f)`

Выполняет функцию `f` для элементов в диапазоне `[b, e)`, сохраняя результат выполнения `f` в приемнике `d`

`partition(b, e, p)`

`stable_partition(b, e, p)`

Распределяет элементы из диапазона `[b, e)` на основе предиката `p`, перемещая в начало контейнера те из них, для которых предикат возвращает значение `true`. Алгоритм возвращает итератор, указывающий на первый элемент, для которого этот предикат равен значению `false`, или параметр `e`, если предикат истинен для всех элементов. Функция `stable_partition` сохраняет исходный порядок элементов в каждой категории

Упражнения

- 6.0. Скомпилируйте, выполните и протестируйте программы, приведенные в этой главе.
- 6.1. Перепишите реализацию функций `frame` и `hcat` из разделов 5.8.1 и 5.8.3, чтобы в них использовались итераторы.
- 6.2. Напишите программу для тестирования функции `find_urls`.
- 6.3. Что делает этот фрагмент программы?

```
vector<int> u(10, 100);  
vector<int> v;  
copy(u.begin(), u.end(), v.begin());
```

Напишите программу, которая бы включала этот фрагмент, скомпилируйте и выполните ее.

- 6.4. Внесите изменения в программу, написанную вами в качестве ответа на предыдущее упражнение, чтобы в ней выполнялось копирование из вектора `u` в вектор `v`. Существует по крайней мере два возможных способа исправить эту программу. Реализуйте оба и опишите сравнительные преимущества и недостатки каждого способа.

- 6.5. Напишите функцию анализа, которая вызывала бы функцию `optimistic_median`.
- 6.6. Обратите внимание на то, что функция из предыдущего упражнения и функции из разделов 6.2.2 и 6.2.3 выполняют одну и ту же задачу. Объедините эти три функции анализа в одну.
- 6.7. Часть программы анализа методов вычисления оценок из раздела 6.2.1, которая читает и классифицирует записи в зависимости от того, сделали ли студенты все домашние задания, аналогична задаче, которую мы решаем в функции `extract_fails`. Напишите функцию для реализации этой подзадачи.
- 6.8. Напишите единую функцию, которую можно использовать для классификации записей на основе вашего собственного критерия. Протестируйте эту функцию, используя ее вместо функции `extract_fails`, и включите ее в программу анализа оценок студентов.
- 6.9. Используйте подходящий библиотечный алгоритм, чтобы конкатенировать все элементы вектора типа `vector<string>`.

ИСПОЛЬЗОВАНИЕ АССОЦИАТИВНЫХ КОНТЕЙНЕРОВ

Все контейнеры, которые мы использовали до сих пор, были последовательными, т.е. их элементы сохраняли выбранную нами последовательность. При выполнении операции `push_back` или `insert` для добавления элементов в последовательный контейнер каждый элемент остается на “своем” месте (в которое мы его поместили) до тех пор, пока мы не сделаем что-нибудь по отношению к контейнеру, в результате чего он переупорядочит свои элементы.

От некоторых программ трудно ожидать высокой эффективности, если ограничиться применением только последовательных контейнеров. Допустим, у нас есть контейнер для хранения целочисленных значений и мы хотели бы написать программу, которая определяла бы, содержит ли какой-нибудь элемент этого контейнера, скажем, значение 42. В данной ситуации мы могли бы воспользоваться двумя вполне убедительными стратегиями, но ни одна из них не является идеальной. Первая стратегия — проверять элементы контейнера до тех пор, пока не будет найдено значение 42 или не будут проверены все элементы. Это самый простой подход, однако он потенциально медленный, особенно если контейнер содержит много элементов. Вторая стратегия — содержать контейнер в соответствующем порядке и продумать эффективный алгоритм для отыскания нужного элемента. Этот подход способен обеспечить более быстрый поиск, но для подобных алгоритмов характерна определенная сложность разработки. Другими словами, мы должны либо удовлетвориться медленно работающей программой, либо подняться на более высокий уровень знаний и освоить более сложный алгоритм (возможно, предложить собственный). К счастью, как мы увидим в этой главе, библиотека предлагает еще одну альтернативу.

7.1. Контейнеры, поддерживающие эффективный поиск

Вместо последовательного контейнера мы можем использовать *ассоциативный контейнер* (associative container). Такие контейнеры автоматически меняют последовательность элементов, которая зависит от значений самих элементов, а не от порядка, в котором они вставлялись в контейнер. Более того, ассоциативные контейнеры используют этот порядок, чтобы мы могли найти заданный элемент гораздо быстрее,

чем на это способны последовательные контейнеры, причем от нас не требуется никакого участия в поддержании этого порядка.

Ассоциативные контейнеры предлагают эффективные способы отыскания элемента, который имеет конкретное значение, а также могут содержать дополнительную информацию. Та часть каждого элемента контейнера, которую мы можем использовать для организации эффективного поиска, называется *ключом* (key). Например, если бы мы отслеживали информацию о студентах, то в качестве ключа можно было бы использовать фамилию, и тогда нам обеспечен эффективный поиск студента по его фамилии.

В последовательных контейнерах ближайшим аналогом ключа можно считать целочисленный индекс, который сопровождает каждый элемент vector-объекта. Однако даже эти индексы не являются настоящими ключами, поскольку при каждой операции вставки или удаления элемента вектора мы неявным образом изменяем индекс каждого элемента, расположенного *после* вставленного или удаленного.

В ассоциативной структуре данных самого распространенного вида хранятся пары “ключ-значение”, причем значение устойчиво связано с ключом, что позволяет быстро вставлять и находить элементы по ключу. При помещении в такую структуру данных конкретной пары “ключ-значение” ключ, принадлежащий этой паре, будет оставаться связанным со “своим” значением до тех пор, пока мы не удалим целую пару. Такая структура данных называется *ассоциативной матрицей*, или *матрицей ассоциативных элементов* (associative array). В таких языках, как AWK, Perl и Snobol, ассоциативная матрица является встроенным типом. В C++ ассоциативные матрицы — это часть библиотеки. Самый распространенный вид ассоциативной матрицы в C++ называется *отображением* (map), и по аналогии с другими контейнерами он определяется в заголовке <map>.

Во многих случаях отображения ведут себя подобно векторам. Но основное отличие от вектора состоит в том, что индекс отображения необязательно должен быть целым значением; это может быть string-объект или объект любого другого типа. Главное, чтобы значения ключей можно было сравнивать, что позволяет хранить их в нужном порядке.

Существует еще одно важное отличие между ассоциативными и последовательными контейнерами. Поскольку ассоциативные контейнеры самоупорядочиваются, наши собственные программы не должны делать ничего такого, что изменяет порядок элементов. Поэтому алгоритмы, которые меняют содержимое контейнеров, часто не работают для ассоциативных контейнеров. Взамен такого ограничения ассоциативные контейнеры предлагают множество полезных операций, которые невозможно эффективно реализовать для последовательных контейнеров.

В этой главе рассматриваются примеры программ, в которых используются отображения, позволяющие организовать эффективный поиск элементов.

7.2. Подсчет слов

В качестве простого примера попытаемся подсчитать, сколько раз каждое отдельное слово встречается во входных данных. С использованием ассоциативных матриц решение получается почти тривиальным.

```
int main()
{
    string s;
    map<string, int> counters; // Храним каждое слово
                             // и соответствующий счетчик.
```

```

// Считываем входные данные, отслеживая количество
// вхождений каждого слова.
while (cin >> s)
    ++counters[s];

// Выводим слова и соответствующие счетчики.
for (map<string, int>::const_iterator it = counters.begin();
     it != counters.end(); ++it) {
    cout << it->first << "\t" << it->second << endl;
}
return 0;
}

```

Как и в случае с другими контейнерами, мы должны указать тип объектов, которые будут содержаться в этом отображении. Поскольку `map`-объект содержит пары “ключ-значение”, нам необходимо указать не только тип значений, но и тип ключей. Поэтому инструкция

```
map<string, int> counters;
```

определяет объект `counters` как отображение, которое содержит значения типа `int`, связанные с ключами типа `string`. Мы часто представляем себе такой контейнер как преобразование из `string`-объекта в `int`-объект (или, можно сказать, “отображение” `string`-объекта на `int`-объект), поскольку объект типа `map` после передачи ему `string`-объекта (в качестве ключа) можно использовать для получения соответствующего `int`-значения.

Способ определения объекта `counters` говорит о нашем намерении связать каждое считанное нами слово с целочисленным счетчиком, который подсчитывает, сколько раз мы встретили это слово. В цикле ввода данных в объект `s` пословно считываются данные из стандартного входного потока. При этом заслуживает внимания следующая инструкция.

```
++counters[s];
```

Под ее лаконизмом скрывается следующее: мы “заглядываем” в объект `counters`, используя только что считанное слово в качестве ключа. Результат выражения `counters[s]` представляет собой целое значение, которое связано с некоторым `string`-значением, хранящимся в объекте `s`. Затем мы используем оператор “++”, чтобы инкрементировать это целое значение; это говорит о том, что мы *встретили данное слово еще раз*.

А что же происходит, когда мы встречаем некоторое слово в первый раз? В этом случае объект `counters` еще не будет содержать элемента с таким ключом. Когда мы обращаемся к `map`-объекту, используя ключ, который до сих пор не встречался в считанных данных, отображение автоматически создает новый элемент с этим ключом. Этот элемент инициализируется определенным значением (т.е. имеет место так называемая *инициализация значения*), что для таких простых типов, как `int`, эквивалентно установке нулевого значения. Следовательно, при первоначальном считывании нового слова и выполнении инструкции `++counters[s]` с использованием этого нового слова мы можем быть уверены в том, что значение выражения `counters[s]` будет равно нулю (еще до инкрементирования). Инкрементирование значения `counters[s]` и в этом случае будет справедливо означать, что “мы встретили данное слово еще раз”, т.е. в первый раз.

Завершив считывание входных данных, мы должны вывести на экран значения счетчиков и соответствующие им слова. Вывод полученных результатов реализуется здесь практически так же, как мы обычно выводим содержимое списка или вектора. Мы опрашиваем в цикле все элементы контейнера, используя переменную итераторного типа, определен-

ного в классе `map`. И только одно реальное отличие от использования списков и векторов состоит в том, как именно выводятся данные в теле инструкции `for`.

```
cout << it->first << "\t" << it->second << endl;
```

Вспомните, что любая ассоциативная матрица предназначена для хранения коллекции пар “ключ-значение”. Использование квадратных скобок (`[]`) для доступа к элементу отображения несколько маскирует этот факт, поскольку мы помещаем ключ внутри квадратных скобок и получаем соответствующее значение. Так, например, выражение `counters[s]` имеет тип `int`. Однако при опросе элементов `map`-объекта мы должны иметь возможность получить доступ как к ключу, так и к значению, соответствующему этому ключу. Контейнер типа `map` позволяет реализовать такой доступ с помощью библиотечного типа `pair`.

Тип `pair` представляет собой простую структуру данных, которая содержит два элемента, именуемые `first` и `second`. Каждый элемент в отображении на самом деле является `pair`-объектом, в котором член `first` содержит ключ, а член `second` — связанное с ключом значение. При разыменовании итератора `map`-объекта мы получаем значение, которое имеет тип `pair`, соответствующий данному `map`-объекту.

Класс `pair` может хранить значения различных типов, поэтому при создании `pair`-объекта мы указываем, какого типа должны быть данные-члены `first` и `second`. Для `map`-объекта, в котором ключ имеет тип `K`, а значение — тип `V`, соответствующий тип `pair` можно выразить в виде `pair<const K, V>`.

Обратите внимание на то, что тип `pair`, соответствующий `map`-контейнеру, включает тип ключа с модификатором `const`. Поскольку ключ определен как константный, мы защищены от изменений значения ключа в любом элементе отображения. Если бы ключ не был константным, мы могли бы неявно изменить позицию элемента в пределах `map`-объекта. Следовательно, `const`-ключ всегда остается постоянным, чтобы, если нам понадобится разыменовать итератор отображения `map<string, int>`, мы получили объект типа `pair<const string, int>`. Таким образом, выражение `it->first` представляет собой ключ текущего элемента, а `it->second` — соответствующее этому ключу значение. Поскольку `it` — итератор, то `*it` — `l`-значение (см. раздел 4.1.3), и тогда `it->first` и `it->second` — также `l`-значения. Однако тип `it->first` включает модификатор `const`, который предотвращает внесение каких бы то ни было изменений в объект `it`.

Теперь вы должны понимать, что рассматриваемая выше инструкция вывода данных выводит каждый ключ (т.е. каждое отдельное слово из входных данных), а за ним — символ табуляции и соответствующее значение счетчика слов.

7.3. Генерирование таблицы перекрестных ссылок

Если мы уже знаем, как подсчитать частоту вхождения отдельных слов в некотором тексте, то теперь вполне естественным будет стремление написать программу генерирования таблицы перекрестных ссылок, которая указывает, где именно в тексте встречается каждое слово. Неудивительно, что новая задача требует внести изменения в нашу основную программу.

Прежде всего, вместо пословного считывания данных, нам придется выполнять построчное считывание, чтобы мы могли связывать номера строк со словами. Если вместо слов мы будем считывать строки, нам потребуется способ разделения каждой строки на составляющие ее слова. К счастью, мы уже написали такую функцию, именуемую `split`, в разделе 6.1.1. Теперь можно использовать эту функцию для преобразования каждой входной строки в вектор типа `vector<string>`, из которого мы будем извлекать каждое слово.

Вместо прямого вызова функции `split`, воспользуемся ею в качестве параметра при обращении к функции генерирования перекрестных ссылок. Тем самым мы оставляем открытой возможность изменения способа отыскания слов в строке. Например, вместо функции `split`, мы могли бы передать функцию `find_urls` из раздела 6.1.3 и использовать функцию генерирования перекрестных ссылок, чтобы узнать, где в исходном тексте встречаются URL-адреса.

Как и прежде, мы будем использовать контейнер типа `map` с ключами, которые представляют собой отдельные слова из входного текста. Однако на этот раз нам придется связывать с каждым ключом более сложное значение. Вместо отслеживания частоты вхождения слов, мы хотим знать номера всех строк, в которых расположено каждое слово. Поскольку любое слово может встречаться во многих строках, нам придется хранить номера строк в каком-нибудь контейнере.

Получив номер новой строки, нам останется лишь присоединить его к номерам, которые уже имеются для данного слова. Вполне достаточно обеспечить последовательный доступ к элементам такого контейнера, поэтому для отслеживания номеров строк мы можем использовать вектор. Таким образом, для решения данной задачи нам подойдет такой вид ассоциативной матрицы, как отображение `string`-объекта на вектор типа `vector<int>`.

Итак, можно “подвести черту” под предварительными замечаниями и приступить к программному коду.

```
// Находим все строки, в которых есть каждое слово
// из исходного текста.
map<string, vector<int> >
    xref(istream& in,
        vector<string> find_words(const string&) = split)
{
    string line;
    int line_number = 0;
    map<string, vector<int> > ret;

    // Считываем следующую строку.
    while (getline(in, line)) {
        ++line_number;

        // Разбиваем строку на слова.
        vector<string> words = find_words(line);

        // Помним, что каждое слово встречается
        // на текущей строке.
        for (vector<string>::const_iterator it = words.begin();
            it != words.end(); ++it)
            ret[*it].push_back(line_number);
    }
    return ret;
}
```

Как тип значения, возвращаемого функцией `xref`, так и список ее аргументов заслуживают особого внимания. После того как вы посмотрите на объявление типа возвращаемого значения и объявление локальной переменной `ret`, хотим обратить ваше внимание на то, что мы написали символы “> >”, а не “>>”. Пробел нужен только компилятору, поскольку если он “увидит” символы “>>” (без промежуточного пробела), предположит, что это оператор ввода “>>”, а не два отдельных символа “>”.

Теперь рассмотрим список аргументов. Обратите внимание на то, что функция `find_words` определяет параметр, который реализует наше намерение передать функции `xref` функцию, предназначенную для разбиения входной строки на слова. Инте-

ресно также то, что уточнение `= split`, стоящее после определения параметра `find_words`, означает, что этот параметр имеет *аргумент по умолчанию* (default argument). Присваивая параметру аргумент по умолчанию, мы тем самым сообщаем, что авторы вызова нашей функции при желании могут опустить этот аргумент. Если они предоставят его, функция послушно будет его использовать, а если опустят, компилятор подставит аргумент, заданный по умолчанию. Таким образом, пользователи могут вызвать эту функцию любым из следующих двух способов.

```
xref(cin); // Использует для отыскания слов
           // во входном потоке функцию split.
xref(cin, find_urls); // Использует для отыскания слов
                      // функцию с именем find_urls.
```

Тело этой функции начинается с определения `string`-переменной `line`, которая будет содержать каждую входную строку (после ее считывания), и определения `int`-переменной `line_number`, предназначенной для хранения номера строки, обрабатываемой в данный момент. В цикле построчного ввода данных вызывается функция `getline` (см. раздел 5.7) для считывания одной строки в переменную `line`. До тех пор, пока продолжается ввод данных, мы инкрементируем счетчик строк и обрабатываем каждое слово в строке.

Обработка слов начинается с объявления локальной переменной `words`, которая будет содержать все слова текущей строки (переменной `line`). Поэтому мы инициализируем переменную `words` путем вызова функции, заданной параметром `find_words`. Вызываемой функцией будет либо (наша старая знакомая) функция `split` (см. раздел 6.1.1), которая разбивает строку на составляющие ее слова, либо некоторая другая функция, которая принимает аргумент типа `string` и возвращает результат типа `vector<string>`. Эстафету обработки подхватывает инструкция `for`, которая опрашивает каждый элемент в векторе `words`, обновляя `map`-объект `ret` при каждом перемещении по вектору `words`.

В заголовке `for`-инструкции не должно быть ничего для вас нового: он определяет итератор и проводит его последовательно по всему вектору `words`. Однако инструкция, составляющая тело `for`-цикла, с первого взгляда может показаться непонятной.

```
ret[*it].push_back(line_number);
```

Поэтому уделим ей немного внимания. Итератор `it` указывает на элемент вектора `words`, и поэтому выражение `*it` представляет собой одно из слов входной строки. Это слово мы используем в качестве индекса для нашего `map`-объекта. Выражение `ret[*it]` возвращает значение, хранимое в позиции отображения, обозначенной индексом `*it`. Это значение представляет собой вектор типа `vector<int>`, содержащий номера строк, в которых до сих пор было замечено данное слово. Чтобы присоединить к концу этого вектора номер текущей строки, мы вызываем функцию-член `push_back` класса `vector`.

Как мы видели в разделе 7.2, если некоторое слово встречается впервые, то будет инициализирован связанный с ним вектор типа `vector<int>`. Как будет показано в разделе 9.5, инициализация объектов типа класса несколько сложнее инициализации объектов примитивного типа. Пока же нам нужно знать, что векторы инициализируются точно так же, как переменные типа `vector` в момент их создания без присваивания им в явном виде какого бы то ни было значения. В обоих случаях создаваемый вектор не будет содержать элементы. Следовательно, когда мы вставляем новый `string`-ключ в отображение, он сразу же будет связан с пустым объектом типа `vector<int>`. При обращении к функции `push_back` текущий номер строки будет присоединен к этому изначально пустому вектору.

Написав функцию `xref`, мы можем использовать ее для генерации таблицы перекрестных ссылок.

```

int main()
{
    // Вызываем функцию xref, используя функцию split
    // по умолчанию.
    map<string, vector<int> > ret = xref(cin);

    // Выводим результаты.
    for (map<string, vector<int> >::const_iterator it =
        ret.begin(); it != ret.end(); ++it) {
        // Выводим слово,...
        cout << it->first << " встречается на строке (строках): ";

        // ...а за ним – один или несколько номеров строк.
        vector<int>::const_iterator line_it = it->second.begin();
        cout << *line_it; // Выводим первый номер строки.

        ++line_it;
        // Выводим остальные номера строк, если таковые имеются.
        while (line_it != it->second.end()) {
            cout << ", " << *line_it;
            ++line_it;
        }
        // Выводим символ новой строки, чтобы отделить
        // каждое новое слово от следующего.
        cout << endl;
    }
    return 0;
}

```

Несмотря на то что в этом коде мы впервые имеем дело с обновлением map-объекта, здесь используются только знакомые вам операции.

Функция main начинается с вызова функции xref, предназначенной для построения структуры данных, содержащей номера строк, в которых встречается каждое слово из входного потока. Поскольку для параметра-функции предусмотрено значение, действующее по умолчанию, в данном обращении к функции xref используется функция split, разбивающая входные строки на слова. Остальная часть программы посвящена выводу содержимого структуры данных, возвращаемой функцией split.

Большая часть программы сосредоточена в инструкции for, с формой которой вы ознакомились в разделе 7.2. Она последовательно просматривает все элементы отображения ret, начиная с первого.

Рассматривая тело этого for-цикла, помните, что при разыменовании итератора map-объекта мы получаем значение типа pair. Член first объекта типа pair содержит const-ключ, а член second представляет значение, связанное с этим ключом.

Цикл for начинается с вывода обрабатываемого нами слова и сообщения.

```
cout << it->first << " встречается на строке (строках): ";
```

Это слово является ключом, расположенным в позиции отображения (map-объекта), которая соответствует итератору it. Мы получаем доступ к ключу посредством разыменования итератора it и считывания члена first из парного pair-элемента.

Мы вправе выводить приведенное выше сообщение уже в этой инструкции программы, поскольку знаем, что информация заносится в очередной элемент отображения ret только в том случае, если она представляет собой слово, имеющее одну или несколько ссылок (номеров строк). Таким образом, выводя значение ключа (слово), мы полностью уверены в том, что за выводимым нами сообщением будет указан по крайней мере один номер строки, на которой встречается это слово. Но так как в данный момент мы не знаем точное количество ссылок на это слово, предусмотрительно указываем в скобках форму множественного числа.

Если значение выражения `it->first` — ключ, то выражение `it->second` содержит соответствующее этому ключу значение, которое в данном случае представляет собой вектор типа `vector<int>`, содержащий номера строк, на которых встречается текущее слово. Мы определяем переменную `line_it` как итератор, который будет использован для получения доступа к элементам вектора `it->second`.

Номера строк при выводе мы хотим отделять запятыми, но при этом нельзя оставлять “бесхозной” запятую в конце. Поэтому необходимо выполнить специальную проверку, является ли выводимый элемент первым или последним. Первый элемент вообще выводится отдельно от остальных (если таковые присутствуют). В этом нет никакой опасности, поскольку каждый элемент отображения `get` представляет слово хотя бы с одной ссылкой на него. После вывода первого элемента мы инкрементируем итератор `line_it`, что означает переход к обработке остальных номеров строк, которые выводятся в цикле `while`. Этот цикл обеспечивает доступ к каждому из оставшихся элементов вектора типа `vector<int>`, выводя сначала запятую, а затем значение элемента.

7.4. Генерирование предложений

“На десерт” мы подготовили более сложный пример: попробуем применить тип `map` в программе, которая использует описание структуры предложения — грамматику, т.е. основные правила — и генерирует случайным образом предложения, отвечающие этому описанию. Например, мы могли бы описать самое простое предложение, состоящее из существительного и глагола, или более сложное — из существительного, глагола, дополнения и т.д.

Создаваемые нами предложения будут интереснее, если мы сможем обрабатывать более сложные правила. Например, вместо того чтобы утверждать, что предложение состоит из существительного и следующего за ним глагола, мы можем позволить использование именных групп, где именная группа представляет собой либо просто существительное, либо имя прилагательное, предшествующее некоторой именной группе. В качестве конкретного примера рассмотрим следующие исходные данные.

Категории	Правила
<существительное>	кот
<существительное>	собака
<существительное>	стол
<именная группа>	<существительное>
<именная группа>	<прилагательное> <именная группа>
<прилагательное>	большой
<прилагательное>	рыжий
<прилагательное>	забавный
<глагол>	прыгает
<глагол>	сидит
<определение места>	на ступеньках лестницы
<определение места>	под открытым небом
<определение места>	куда ему хочется
<предложение>	А <именная группа> <глагол> <определение места>

В результате наша программа может сгенерировать следующее предложение.

А стол прыгает куда ему хочется

Программа должна всегда начинать свою работу с отыскания правила, описывающего, как нужно построить предложение. В наших исходных данных есть только одно такое правило, которое занимает последнюю строку в нашей таблице.

<предложение> А <именная группа> <глагол> <определение места>

Это правило утверждает, что для построения предложения мы должны вывести слово “А”, именную группу, глагол и, наконец, определение места. После вывода слова “А” программа начнет случайным образом искать правило, соответствующее элементу <именная группа>. В конце концов, она выберет следующее правило.

<именная группа> <существительное>

Затем для элемента <существительное> может быть выбран такой вариант.

<существительное> стол

Программа должна еще принять решение в отношении элементов <глагол> и <определение места>. Что касается глагола, возможен следующий выбор.

<глагол> прыгает

В отношении определения места программе “понравился” такой вариант.

<определение места> куда ему хочется

Обратите внимание на то, что последнее правило отображает категорию, которой соответствует несколько слов, призванных достойно завершить построение предложения.

7.4.1. Представление правил

Наша таблица содержит два столбца: категории, которые заключены в угловые скобки, и обычные слова. Каждая категория имеет одно или несколько правил; каждое обычное слово означает само себя. Когда программа встречает string-объект, заключенный в угловые скобки, мы знаем, что он представляет собой категорию, поэтому необходимо заставить программу найти правило, которое соответствует этой категории, чтобы раскрыть правую часть этого правила. Если программа встречает слова, которые лишены угловых скобок, мы знаем, что она сумеет вставить эти слова в генерируемое предложение без изменений.

Думая о том, как будет действовать наша программа, кажется, что она должна считать описание способа построения предложений, а затем случайным образом генерировать “свой вариант”. Отсюда возникает первый вопрос: как нам следует хранить описание? Когда мы создаем предложения, нам нужно уметь сопоставлять каждую категорию с правилом, которое раскроет (проще говоря, заменит) эту категорию. Например, сначала нам нужно найти правило создания элемента <предложение>. Исходя из “трактовки” этого правила, нам придется отыскать правила для элементов <именная группа>, <глагол>, <определение места> и т.д. По-видимому, для реализации таких действий нам подойдет отображение (тип map), которое преобразует категории в соответствующие правила.

Но отображение какого типа? С категориями все просто: мы можем хранить их как string-объекты, т.е. ключ каждого элемента отображения будет иметь тип string.

С типом значения дело обстоит гораздо сложнее. Если мы снова взглянем на приведенную выше таблицу, то можно заметить, что любое заданное в ней правило можно

представить как коллекцию `string`-объектов. Например, категории <предложение> соответствует правило, состоящее из четырех компонентов: слова `A` и трех других `string`-объектов, которые являются категориями. Мы знаем, как представлять значения этого вида: для хранения этого правила можно использовать вектор типа `vector<string>`. Проблема в другом: каждая категория может встречаться во входных данных более одного раза. Например, в нашем варианте исходных данных категория <существительное> встречается три раза, как и категории <прилагательное> и <определение места>. Поскольку эти категории встречаются три раза, каждая будет иметь три соответствующих правила.

Проще всего справиться с несколькими экземплярами одного и того же ключа посредством хранения каждой коллекции правил в собственном векторе. Тогда выходит, что грамматика будет храниться в отображении, преобразующем `string`-объекты в `vector`-объекты, которые в свою очередь содержат векторы типа `vector<string>`.

Да, у нас вырисовывается довольно-таки грандиозное сооружение. Возможно, наша программа станет понятнее, если ввести синонимы для промежуточных типов. Итак, мы сказали, что каждое правило представляет собой вектор типа `vector<string>` и каждая категория преобразуется в вектор этих правил. Это означает, что нам нужно определить три типа: один — для правила, второй — для коллекции правил и третий — для отображения.

```
typedef vector<string> Rule;
typedef vector<Rule> Rule_collection;
typedef map<string, Rule_collection> Grammar;
```

7.4.2. Чтение грамматических правил

Определившись с представлением грамматики, можно написать функцию для ее считывания.

```
// Считываем грамматику из данного входного потока.
Grammar read_grammar(istream& in)
{
    Grammar ret;
    string line;

    // Считываем входные данные.
    while (getline(in, line)) {

        // С помощью функции split разбиваем входные данные
        // на отдельные слова.
        vector<string> entry = split(line);

        if (!entry.empty())
            // используем категорию, чтобы сохранить
            // соответствующее правило.
            ret[entry[0]].push_back(
                Rule(entry.begin() + 1, entry.end()));
    }
    return ret;
}
```

Эта функция будет считывать данные из входного потока и в качестве результата генерировать объект типа `Grammar`. Цикл `while` внешне не отличается от рассмотренных нами до сих пор: из входного потока `in` построчно считываются данные, которые сохраняются в переменной `line` типа `string`. Цикл `while` оканчивается, когда исчерпываются входные данные или встречается некорректное данные.

Тело цикла `while` поражает своей лаконичностью. Мы используем функцию `split` из раздела 6.1.1, чтобы разбить входную строку на отдельные слова, а полученный

вектор сохраняем в переменной `entry` типа `vector<string>`. Если вектор `entry` пуст, т.е. мы приняли пустую входную строку, мы игнорируем его. В противном случае мы знаем, что первым элементом в объекте `entry` будет категория (в соответствии с нашим определением).

Этот элемент мы используем в качестве индекса для доступа к содержимому отображения `ret` типа `Grammar`. Выражение `ret[entry[0]]` генерирует объект типа `rule_collection`, который связан с категорией, хранимой в элементе `entry[0]`. Вы должны помнить, что объект типа `rule_collection` представляет собой вектор, каждый элемент которого содержит объект типа `rule` (или, что эквивалентно, вектор типа `vector<string>`). Таким образом, `ret[entry[0]]` — это вектор, в конец которого мы помещаем только что считанное правило. Это правило содержится в векторе `entry`, начиная со второго элемента; первым же элементом в векторе `entry` является категория. Итак, мы создаем новый неименованный объект типа `rule`, копируя элементы из `entry` (за исключением первого элемента) и помещая этот новоиспеченный объект типа `rule` в конец вектора типа `rule_collection`, индексируемого значением выражения `ret[entry[0]]`.

7.4.3. Генерирование предложения

Прочитав все входные данные, мы должны случайным образом сгенерировать предложение. Мы знаем, что в считанных данных содержатся грамматические правила, которые позволяют построить какое-нибудь предложение. Результатом выполнения нашей программы будет вектор типа `vector<string>`, содержащий компоненты, составляющие предложение.

Эта часть работы несложна. Более интересная задача — организация нашей функции. Мы знаем, что сначала нам придется найти правило, которое соответствует категории <предложение>. Более того, мы собираемся построить результат по частям, которые будут собраны из различных правил и их частей.

В принципе, мы могли бы конкатенировать эти части, чтобы сформировать результат целиком. Но поскольку для векторов не существует встроенной операции конкатенации, мы начнем с пустого вектора и необходимое число раз вызовем для него функцию `push_back`.

Эти два ограничения — старт с категории <предложение> и неоднократный вызов функции `push_back` для изначально пустого вектора — предполагают, что мы собираемся определить наш генератор предложений с помощью некоторой вспомогательной функции `gen_aux`, которую будем вызывать следующим образом.

```
vector<string> gen_sentence(const Grammar& g)
{
    vector<string> ret;
    gen_aux(g, "<предложение>", ret);
    return ret;
}
```

В действительности обращение к функции `gen_aux` — это запрос на использование грамматики `g` для генерирования предложения в соответствии с правилом, относящимся к категории <предложение>, с последующим присоединением сформированного предложения к концу вектора `ret`.

Теперь нам осталось определить функцию `gen_aux`. Необходимо отметить, что функция `gen_aux` должна сначала выяснить, представляет ли анализируемое слово категорию, признаком которой служат угловые скобки, в которые заключено это слово. Для этого можно определить следующий предикат.

```
bool bracketed(const string& s)
{
    return s.size() > 1 && s[0] == '<'
        && s[s.size() - 1] == '>';
}
```

Цель функции `gen_aux` — заменить входной `string`-объект, заданный в качестве второго аргумента, найдя его (вместе со значением замены) в грамматике, заданной первым аргументом функции, и поместить результат (значение замены) в ее третий аргумент. Под “заменой” мы подразумеваем процесс, описанный в разделе 7.4. Если наш `string`-объект заключен в угловые скобки, мы должны найти соответствующее правило, которым нужно заменить заключенную в угловые скобки категорию. Если входной `string`-объект не заключен в угловые скобки, тогда он сам является частью результата, который можно поместить в выходной вектор без дальнейшей обработки.

```
void gen_aux(const Grammar& g,
            const string& word, vector<string>& ret)
{
    if (!bracketed(word)) {
        ret.push_back(word);
    } else {
        // Ищем правило, соответствующее значению word.
        Grammar::const_iterator it = g.find(word);
        if (it == g.end())
            throw logic_error("пустое правило");

        // Считываем набор возможных правил, ...
        const Rule_collection& c = it->second;

        //...из которых случайным образом выбираем только одно.
        const Rule& r = c[nrand(c.size())];

        // Рекурсивно заменяем выбранное правило.
        for (Rule::const_iterator i = r.begin(); i != r.end(); ++i)
            gen_aux(g, *i, ret);
    }
}
```

Наша первая задача тривиальна: если принятое слово не заключено в угловые скобки, значит, оно представляет само себя, и поэтому мы можем сразу поместить его в конец вектора `ret` и дело с концом. Теперь нам предстоит более интересная часть функции: найти в грамматике `g` правило, соответствующее переданному функции слову. Если просто обратиться к элементу `g[word]`, это может дать нам неверный результат. Вспомните раздел 7.2! При попытке индексировать отображение несуществующим ключом автоматически создается новый элемент с этим ключом. Нас это совсем не устраивает, поскольку мы не собираемся расширять грамматику фиктивными правилами. Более того, `map`-объект `g` является константным отображением, поэтому, даже если бы мы и захотели создать новый элемент, нам бы это не удалось.

По-видимому, мы должны использовать другое средство: функция-член `find` класса `map` ищет элемент по заданному ключу и возвращает итератор, который ссылается на искомый элемент, если его удалось найти. Если такой элемент не существует в отображении `g`, алгоритм `find` возвращает значение выражения `g.end()`. Следовательно, сравнение итератора `it` с выражением `g.end()` необходимо для подтверждения существования искомого правила. Если оно не существует, это значит, что входные данные алогичны: в них используется заключенное в угловые скобки слово без соответствующего правила; такая ситуация заслуживает того, чтобы ее назвать исключительной, поэтому мы и генерируем исключение.

На данном этапе `it` — это итератор, который указывает на элемент отображения `g`. Разыменование этого итератора дает объект типа `pair`, второй член которого представляет собой значение элемента отображения (`map`). Следовательно, выражение `it->second` означает коллекцию правил, которые соответствуют этой категории. Ради удобства мы определяем ссылку `c` в качестве синонима для этого объекта.

Наша следующая задача — выбрать случайный элемент из этой коллекции, что мы и делаем при инициализации переменной `r`. Код

```
const Rule& r = c[nrand(c.size())];
```

выглядит для вас незнакомым, и поэтому ему следует уделить внимание. Прежде всего, вспомним, что мы определили ссылку `c` как синоним для коллекции правил типа `Rule_collection`, собранных в виде вектора. Мы вызываем функцию `nrand`, которую нам еще предстоит определить в разделе 7.4.4, чтобы выбрать случайный элемент из этого вектора. Функция `nrand`, приняв аргумент `n`, возвращает случайно выбранное целое число в диапазоне `[0, n)`. Наконец, мы определяем `r` как синоним для этого элемента.

В качестве заключительного “аккорда” функции `gen_aux` мы должны проанализировать каждый элемент объекта `r`. Если этот элемент заключен в угловые скобки, мы должны заменить его некоторой последовательностью слов; в противном случае мы присоединяем его к концу вектора `ret`. Этот процесс (с первого взгляда может показаться, что в нем есть нечто магическое) в точности соответствует тому, что мы делаем в самой функции `gen_aux`, а значит, как это ни странно, мы можем вызвать для его (процесса) реализации функцию `gen_aux`!

Такой вызов называется *рекурсивным* (*recursive*). Рекурсия относится к числу приемов, которые кажутся неработоспособными до тех пор, пока их не опробовать несколько раз. Поэтому, чтобы убедиться в работоспособности этой функции, сначала проверьте, успешно ли она выполняется, если переданное ей слово (аргумент `word`) не заключено в угловые скобки.

Затем предположите, что аргумент `word` представляет собой слово, заключенное в угловые скобки, а правая часть его правила не содержит слов, заключенных в угловые скобки. Тогда совсем нетрудно понять, что сделает программа в этом случае, поскольку при очередном рекурсивном вызове функции `gen_aux` будет немедленно обнаружено, что исследуемое слово не заключено в угловые скобки. А посему функция `gen_aux` присоединит его к концу вектора `ret` и возвратит результат.

Теперь нам следует предположить, что объект `word` ссылается на более сложное правило, в правой части которого присутствуют слова, заключенные в угловые скобки. Когда вы встречаете рекурсивный вызов функции `gen_aux`, не старайтесь понять, что она делает в этом случае, а просто вспомните, что однажды вы уже убедились в ее работоспособности; ведь вы знаете, что рано или поздно ее аргументом будет категория, которая не приведет больше ни к каким словам, заключенным в угловые скобки. В конечном счете вы увидите, что функция работает во всех случаях, поскольку очередной рекурсивный вызов функции упрощает ее аргумент.

Нам неведом ни один гарантированный способ разъяснения рекурсии. Но по опыту мы знаем, что сначала студенты долго приглядываются к рекурсивным программам, не понимая, как они работают. Но в один прекрасный день наступает прозрение, и им становится вдруг непонятно, почему они считали рекурсию трудной темой. Очевидно, главное для понимания рекурсии — начать ее понимать. Остальное — уже просто.

Написав функции `gen_sentence`, `read_grammar` и ряд вспомогательных функций, хочется использовать их в одном программном механизме.

```
int main()
{
    // Генерируем предложение.
    vector<string> sentence = gen_sentence(read_grammar(cin));

    // Выводим первое слово, если таковое имеется.
    vector<string>::const_iterator it = sentence.begin();
    if (!sentence.empty()) {
        cout << *it;
        ++it;
    }

    // Выводим остальные слова, предварив каждое пробелом.
    while (it != sentence.end()) {
        cout << " " << *it;
        ++it;
    }

    cout << endl;
    return 0;
}
```

Здесь мы считываем грамматику, генерируем на ее основе предложение, а затем пословно его выводим. Единственное (причем незначительное) затруднение связано с выводом пробела перед вторым и последующими словами предложения.

7.4.4. Выбор случайного элемента

Пришло время написать функцию `rand`. Начнем с того, что стандартная библиотека включает функцию `rand` (определенную в заголовке `<cstdlib>`). Эта функция не принимает аргументов и возвращает случайное целое число в диапазоне $[0, \text{RAND_MAX}]$, где `RAND_MAX` — некоторое очень большое целое число, определенное в заголовке `<cstdlib>`. Наша задача — сократить диапазон $[0, \text{RAND_MAX}]$, который включает как 0, так и значение `RAND_MAX`, до диапазона $[0, n)$, который включает число 0, но не n , причем $n < \text{RAND_MAX}$.

Может показаться, что вполне достаточно вычислить выражение `rand() % n`, которое представляет собой остаток от деления случайного целого на число n . На практике этот вариант терпит фиаско по двум причинам.

Наиболее важная причина связана с тем, что функция `rand()` в действительности возвращает только псевдослучайные числа. Многие генераторы псевдослучайных чисел в C++-средах предлагают остатки от деления, которые не являются случайными, когда частные представляют собой малые числа. Например, нет ничего необычного в результатах работы функции `rand()`, поочередно генерирующей четные и нечетные числа. Если n равно 2, то последовательные результаты выражения `rand() % n` будут представлять собой чередование значений 0 и 1.

Существует еще одна причина, которая заставляет избегать использования выражения `rand() % n`: если значение n велико, а значение `RAND_MAX` не делится без остатка на n , то некоторые значения остатков будут встречаться чаще других. Предположим, `RAND_MAX` равно 32767 (наименьшее допустимое значение `RAND_MAX` для любой C++-среды), а n — 20000. В этом случае могут существовать два отдельных значения функции `rand()`, которые приведут к тому, что результат выражения `rand() % n` будет равен 10000 (а именно: 10000 и 30000), но есть только одно значение функции

rand(), при котором значение выражения `rand() % n` будет равно 15000 (а именно: 15000). Следовательно, простая реализация функции `rand` генерирует число 10000 в результате вызова `rand(20000)` вдвое чаще, чем число 15000.

Чтобы избежать подобных “перекосов”, мы будем использовать стратегию, заключающуюся в разделении диапазона допустимых случайных чисел на области абсолютно равного размера. Тогда мы сможем вычислить случайное число и вернуть номер соответствующей области. Поскольку области имеют одинаковый размер, некоторые случайные числа могут вообще не попадать ни в одну из них. В этом случае мы продолжаем запрашивать случайные числа до тех пор, пока не получим число, попавшее в область.

Эта функция относится к разряду таких, которые проще написать, чем объяснить.

```
// функция возвращает случайное целое число в диапазоне [0, n).
int nrand(int n)
{
    if (n <= 0 || n > RAND_MAX)
        throw domain_error(
            "Аргумент функции nrand вне диапазона");

    const int bucket_size = RAND_MAX / n;
    int r;

    do r = rand() / bucket_size;
    while (r >= n);

    return r;
}
```

Определение переменной `bucket_size` основывается на том, что при целочисленном делении результат усекается. Это свойство подразумевает, что выражение `RAND_MAX/n` представляет собой самое большое целое значение, которое меньше точного частного или равно ему. Следовательно, переменная `bucket_size` равна наибольшему целому, удовлетворяющему условию $n * bucket_size \leq RAND_MAX$.

Рассмотрим инструкцию `do-while`. Она подобна `while`-инструкции, за исключением того, что всегда выполняет тело цикла хотя бы один раз и проверяет условие в конце. Если это условие истинно, выполнение цикла повторяется до тех пор, пока `while`-условие не станет ложным. В этом случае тело цикла установит переменную `r` равной номеру области. Область 0 будет соответствовать значениям функции `rand()` из диапазона `[0, bucket_size)`, область 1 — значениям из диапазона `[bucket_size, bucket_size * 2)` и т.д. Если значение функции `rand()` настолько велико, что $r \leq n$, программа продолжит поиск случайных чисел до тех пор, пока не найдет “подходящее”, после чего вернет соответствующее значение переменной `r`.

Предположим, что значение `RAND_MAX` равно 32767, а `n` — 20000. Тогда значение `bucket_size` будет равно 1, а функция `nrand` будет работать, отбраковывая случайные числа до тех пор, пока не найдет число меньше 20000. И еще один пример. Предположим, `n` равно 3. Тогда значение `bucket_size` будет равно 10922. В этом случае значения функции `rand()` будут следующими: 0 — в диапазоне `[0, 10922)`, 1 — в диапазоне `[10922, 21844)`, 2 — в диапазоне `[21844, 32766)`, а значения 32766 или 32767 будут просто отброшены.

7.5. Вспомним о производительности

Если вам приходилось использовать ассоциативные матрицы в других языках программирования, то там они, вероятно, были реализованы с помощью структуры дан-

ных, именуемой *хэш-таблицей* (hash table). Хэш-таблицы могут работать очень быстро, но они имеют следующие недостатки.

- Для каждого типа ключа необходимо предоставить хэш-функцию, которая вычисляет соответствующее целое значение от значения ключа.
- Быстродействие хэш-таблицы очень зависит от деталей реализации хэш-функции.
- Обычно не существует простого способа считывания элементов из хэш-таблицы. Ассоциативные контейнеры C++ трудны для реализации с использованием хэш-таблиц.
 - Типу ключа требуется только оператор “<” или эквивалентная функция сравнения.
 - Время доступа к элементу ассоциативного контейнера с данным ключом находится в логарифмической зависимости от общего числа элементов в этом контейнере, безотносительно к значениям ключа.
 - Элементы ассоциативного контейнера всегда отсортированы по ключу.

Другими словами, хотя ассоциативные контейнеры C++ обычно немного медленнее самых лучших структур данных, построенных на основе хэш-таблиц, все же они работают гораздо лучше обычных структур данных. Их производительность не требует от пользователей разработки “подходящих” хэш-функций; благодаря их автоматическому упорядочению, они более удобны, чем хэш-таблицы. Если вы знакомы в общих чертах с ассоциативными структурами данных, то вам будет интересно узнать, что C++-библиотеки для реализации ассоциативных контейнеров обычно используют пропорциональную самонастраивающуюся древовидную структуру.

Если вы хотите воспользоваться хэш-таблицами, они доступны как составные части многих C++-сред. Но поскольку они не являются частью стандарта C++, эта тема выходит за рамки данной книги. Хотя ни один стандарт не может быть идеальным для каждой конкретной задачи, для большинства приложений стандартные ассоциативные контейнеры более чем достаточны.

7.6. Резюме

Инструкция **do while** подобна инструкции **while** (см. раздел 2.3.1), за исключением того, что условие выполнения цикла проверяется в конце. Общая форма этой инструкции имеет следующий вид.

```
do инструкция
while (Условие);
```

Сначала выполняется элемент *инструкция*, после чего элементы *Условие* и *инструкция* выполняются поочередно до тех пор, пока *Условие* не примет значение false.

Инициализация значением. При попытке получения доступа к несуществующему элементу отображения создается элемент со значением V(), где v — это тип значений, хранимых в отображении. Этот процесс называется инициализацией значением (подробнее — в разделе 9.5). Важно отметить, что объекты встроенных типов инициализируются в подобных случаях нулевыми значениями.

rand() — это функция, которая генерирует случайное целое число в диапазоне [0, RAND_MAX]. Функция rand и значение RAND_MAX определены в заголовке <cstdlib>.

pair<K, V> — это простой тип, объект которого содержит пару значений. Доступ к значениям таких данных предоставляется посредством их имен first и second соответственно.

map<K, V> — это ассоциативная матрица с ключом типа **K** и значением типа **V**. Элементы отображения представляют собой пары “ключ-значение”, которые упорядочены по ключу, чтобы позволить эффективный доступ к элементам по ключу. Итераторы, определенные для **map**-контейнеров, являются двунаправленными (см. раздел 8.2.5). При разыменовании **map**-итератора считывается соответствующее значение типа **pair<const K, V>**. В классе **map** определены следующие операции.

map<K, V> m; Создает новое пустое отображение с ключами типа **const K** и значениями типа **V**

map<K, V> m(cmp);

Создает новое пустое отображение с ключами типа **const K** и значениями типа **V**, а для определения порядка расположения элементов использует предикат **cmp**

m[k] Индексирует отображение, используя ключ **k** типа **K**, и возвращает *l*-значение типа **V**. Если для заданного ключа элемента не существует, создается новый (инициализируемый значением) элемент, который вставляется в отображение в паре с этим ключом. Поскольку использование квадратных скобок (**[]**) для доступа к отображению может привести к созданию нового элемента, квадратные скобки не разрешены для ассоциативных матриц типа **const map**

m.begin()

m.end()

Возвращают итераторы, которые можно использовать для доступа к элементам отображения (контейнера типа **map**). Обратите внимание на то, что разыменование одного из этих итераторов дает пару “ключ-значение”, а не просто значение

m.find(k)

Возвращает итератор, который указывает на элемент, содержащий ключ **k**, или значение **m.end()** — при отсутствии такого элемента

Для отображения типа **map<K, V>** и связанного с ним итератора **p** можно использовать следующие выражения.

p->first

Считывает *l*-значение типа **const K**, являющееся ключом для элемента, на который указывает итератор **p**

p->second

Считывает *l*-значение типа **V**, являющееся значимой частью элемента, на который указывает итератор **p**

Упражнения

- 7.0. Скомпилируйте, выполните и протестируйте программы, приведенные в этой главе.
- 7.1. Дополните приведенную в разделе 7.2 программу так, чтобы она выводила результаты, отсортированные по частоте вхождений слов. Это значит, что сначала должны быть выведены слова, встречающиеся один раз, за ними — слова, встречающиеся два раза, и т.д.
- 7.2. Дополните программу, приведенную в разделе 4.2.3, чтобы она присваивала буквенные обозначения различным уровням успеваемости студентов по следующим диапазонам.

A	90-100
B	80-89.99 ...
C	70-79.99 ...

D 60–69.99 ...
F < 60

В результатах должно быть указано количество студентов каждой категории.

- 7.3. Программу создания таблицы перекрестных ссылок из раздела 7.3 можно немного улучшить. В предыдущем ее варианте предусмотрен многократный вывод одного и того же номера строки, если на этой входной строке некоторое слово встречается больше одного раза. Измените код так, чтобы при обнаружении нескольких вхождений одного и того же номера строки программа выводила этот номер только один раз.
- 7.4. Результаты, выводимые программой создания таблицы перекрестных ссылок, будут выглядеть неуклюже, если входной файл имеет большие размеры. Перепишите программу так, чтобы при образовании слишком длинных выходных строк результаты были представлены в виде нескольких строк.
- 7.5. Переделайте программу построения предложения на основе заданных грамматических правил, используя в качестве структуры данных для создания предложения тип `list`.
- 7.6. Переделайте функцию `gen_sentence`, используя два вектора: один будет содержать полностью сгенерированное предложение, а второй — правила, и кроме того, при этом он будет использован в качестве стека. Не используйте никаких рекурсивных вызовов.
- 7.7. Измените драйверную часть программы создания таблицы перекрестных ссылок так, чтобы она выводила фразу встречается на строке, если в этой таблице указывается только одна строка, и фразу встречается на строках — в противном случае.
- 7.8. Измените программу создания таблицы перекрестных ссылок, чтобы она находила в файле все URL-адреса и выводила все строки, на которых встречается каждый отдельный URL-адрес.
- 7.9. (Задание повышенной сложности.) Реализация функции `nrand`, приведенная в разделе 7.4.4, не будет работать для аргументов, превышающих значение `RAND_MAX`. Обычно это ограничение — не проблема, поскольку `RAND_MAX` часто является максимально возможным целым числом. Тем не менее существуют C++-среды, в которых `RAND_MAX` гораздо меньше максимально возможного целого числа. Например, довольно редко значение `RAND_MAX` оказывается равным числу 32767 ($2^{15} - 1$), в то время как максимально возможное целое число равно 2147483647 ($2^{31} - 1$). Переделайте функцию `nrand` так, чтобы она нормально работала для всех значений `n`.

СОЗДАНИЕ ОБОБЩЕННЫХ ФУНКЦИЙ

В первой части этой книги внимание читателя акцентируется на написании программ, в которых для решения конкретных задач используются основные средства языка C++ и абстракции, предоставляемые стандартной библиотекой. Начиная с этой главы, мы переключим наше внимание на создание собственных абстракций.

Абстракции принимают различные формы. В этой главе рассматриваются обобщенные функции, т.е. функции, типы параметров которых мы не узнаем до тех пор, пока не вызовем эти функции. Главы 9–12 посвящены реализации абстрактных типов данных. Наконец, начиная с главы 13, мы займемся объектно-ориентированным программированием (object-oriented programming — OOP).

8.1. Что такое обобщенная функция

Если до сих пор мы писали какую-нибудь функцию, то нам были известны типы ее параметров и тип возвращаемого ею значения. С первого взгляда может показаться, что эти знания являются неотъемлемой частью описания любой функции. Однако при более близком рассмотрении оказывается, что мы уже использовали (но не писали сами) функции с аргументами и возвращаемыми ими значениями, типов которых не знали до тех пор, пока не использовали эти функции.

Например, в разделе 6.1.3 мы прибегали к услугам библиотечной функции `find`, которая принимает в качестве аргументов два итератора и значение. Мы можем использовать ту же функцию `find` для отыскания значений любого подходящего типа в контейнере произвольного типа. Подобное использование функций предполагает, что мы заранее не знаем типы аргументов и тип результата функции `find`. Такая функция и называется *обобщенной* (generic function), а возможность использовать и создавать обобщенные функции является ключевой особенностью языка C++.

Нетрудно принять к сведению поддержку обобщенных функций средствами языка C++. Труднее понять, что в точности подразумевают, когда говорят, что функция `find` может принимать аргументы “любого подходящего типа”. Например, как можно описать поведение функции `find`, чтобы тот, кто хочет ее использовать, мог узнать, будет ли она работать с конкретными, интересующими его аргументами? Ответ на этот вопрос частично лежит в рамках языка C++ и частично вне его.

Внутриязыковая часть ответа сводится к идее, согласно которой средства, позволяющие функции использовать параметр неизвестного типа, накладывают ограничения на тип этого параметра. Например, если функция принимает параметры `x` и `y`, вычисляя значение

выражения $x + y$, то простое существование этого вычисления неявно требует, чтобы параметры x и y имели типы, для которых определена операция сложения ($x + y$). При вызове такой функции C++-среда проверит, чтобы типы аргументов отвечали требованиям, вытекающим из условий, в которых эта функция использует свои параметры.

Часть ответа, лежащая вне языка C++, подразумевает условия, в которых стандартная библиотека формирует ограничения, накладываемые на параметры таких функций. Мы уже показали вам один пример такого формирования, а именно понятие итератора. Одни типы являются итераторами, а другие — нет. Функция `find` принимает три аргумента, первые два из которых должны быть итераторами.

Когда мы говорим, что конкретный тип должен быть итератором, то в действительности подразумеваем операции, поддерживаемые этим типом: тип является итератором тогда и только тогда, когда он поддерживает специфическую коллекцию операций. Если бы мы вознамерились сами написать функцию `find`, сделали бы это способом, предполагающим только операции, которые должен поддерживать каждый итератор. Если бы мы собрались создать собственный контейнер (достаточно добраться до главы 11 — и это произойдет наяву), то нам пришлось бы предусмотреть в нем итераторы, которые поддерживают все необходимые операции.

Понятие итератора не является частью ядра языка C++. Однако это основная часть организации стандартной библиотеки, и именно эта часть делает обобщенные функции настолько полезными. В этой главе приведены примеры того, как библиотека может реализовать обобщенные функции. А по ходу дела мы поговорим о том, что представляет собой итератор или, точнее, что представляют собой итераторы, поскольку они могут встречаться в пяти различных «ипостасях».

Эта глава более абстрактна, чем предыдущие, поскольку абстракция заложена в самой природе обобщенных функций. Если бы мы писали функции для решения конкретных проблем, такие функции не могли бы быть обобщенными. Тем не менее вы вскоре согласитесь с тем, что большинство описываемых нами функций вам уже знакомо, поскольку мы использовали их в предыдущих примерах. И потом, трудно представить, как можно было использовать незнакомые функции.

8.1.1. Медианы неизвестного типа

Языковое средство, которое реализует обобщенные функции, называется *шаблонными функциями* (template functions). Шаблоны позволяют написать одно определение для целого семейства функций (или типов), которые ведут себя подобным образом, за исключением того, что мы можем задавать типы их *шаблонных параметров* (template parameters). О шаблонных функциях мы поговорим в этой главе, а о шаблонных классах — в главе 11.

Ключевая идея шаблонов состоит в том, что объекты различных типов могут демонстрировать сходное поведение. Шаблонные параметры позволяют писать программы в расчете на такое сходное поведение, несмотря на то что при определении шаблона мы еще не знаем, какие конкретно типы будут соответствовать шаблонным параметрам. Эти типы нам станут известны при использовании шаблона, а наши знания станут достоянием C++-среды при компиляции и компоновке программ. Что касается обобщенных параметров, то C++-среда во время выполнения программы нет нужды беспокоиться о том, что делать с объектами, типы которых могут меняться, поскольку все «беспокойства» ограничиваются только процессом компиляции.

Хотя шаблоны — это краеугольный камень стандартной библиотеки, мы можем использовать их также для собственных программ. Например, в разделе 4.1.1 мы написали функцию для вычисления медианы вектора типа `vector<double>`. Эта функция, опираясь

на возможность сортировки вектора (с помощью библиотечной функции `sort`), считывать нужный элемент, заданный индексом, поэтому нелегко заставить ее обрабатывать произвольные последовательности значений. Но даже при этих условиях не существует фатальной причины ограничивать функцию лишь типом `vector<double>`, поскольку мы можем с таким же успехом вычислить медиану векторов и других типов. Шабононные функции позволяют нам реализовать следующие потенциалные действия.

```
template<class T>
T median(vector<T> v)
{
    typedef typename vector<T>::size_type vec_sz;
    vec_sz size = v.size();
    if (size == 0)
        throw domain_error("медиана пустого вектора");
    sort(v.begin(), v.end());
    vec_sz mid = size/2;
    return size % 2 == 0 ? (v[mid] + v[mid+1]) / 2 : v[mid];
}
```

В этом фрагменте программы новым для вас является заголовок шаблона

```
template<class T>
```

и обозначение `T`, используемое в списке параметров и в качестве типа значения, возвращаемого функцией. Заголовок шаблона сообщает C++-среде о том, что определяется шабононная функция, которая будет принимать *параметр-тип*. Параметр-тип во многом напоминает параметр-функцию: он определяет имя, которое можно использовать в пределах области видимости функции. Однако параметр-тип относится к типу, а не к переменной. Таким образом, если в функции присутствует обозначение `T`, C++-среда предположит, что именем `T` называется некоторый тип. В функции `median` мы используем параметр-тип, чтобы в явном виде указать тип объектов, содержащихся в векторе `v`, и тип значения, возвращаемого этой функцией.

При вызове функции `median` C++-среда свяжет обозначение `T` с типом, определяемым им в данный момент компиляции. Например, посредством вызова `median(vi)` мы могли бы передать функции `median` в качестве аргумента векторный объект `vi`, имеющий тип `vector<int>`. Начиная с этого вызова, C++-среда может сделать вывод, что под обозначением `T` подразумевается тип `int`. В любом месте этой функции, где используется символ `T`, C++-среда сгенерирует такой код, как если бы мы вместо `T` записали `int`. В действительности C++-среда так обрабатывает наш код, как если бы мы написали конкретную версию функции `median`, которая принимает параметр типа `vector<int>` и возвращает `int`-значение.

Помимо заголовка шаблона, новым для вас, вероятно, является использование зарезервированного слова `typename` в определении типа `vec_sz`. Это — способ сообщить C++-среде о том, что `vector<T>::size_type` представляет собой имя некоторого типа, несмотря на то что C++-среда пока не знает, какой тип представляет обозначение `T`. Всякий раз когда указываете такой тип, как `vector<T>`, зависящий от шабононного параметра, и хотите использовать член этого типа, например `size_type`, который сам является типом, вы должны предварить полное имя словом `typename`, чтобы C++-среда знала, что ей следует обрабатывать это имя как тип. Хотя стандартная библиотека гарантирует, что `vector<T>::size_type` — это имя типа, используемого для любого типа `T`, C++-среда, не

обладая специальными знаниями о типах стандартной библиотеки, не имеет возможности узнать об этом.

Глядя на шаблон, обычно понятно, какой параметр-тип используется в его определении, даже если многие зависимости, связанные с типом, выражены неявным образом. В нашей функции `median` мы используем параметры-типы в явном виде только при указании типа значения, возвращаемого функцией, в списке параметров и определении `vec_sz`. Но поскольку параметр `v` имеет тип `vector<T>`, любая операция, включающая параметр `v`, неявно использует этот тип. Например, в выражении

$$(v[mid] + v[mid1]) / 2$$

мы должны знать тип элементов вектора `v`, чтобы знать типы операндов `v[mid]` и `v[mid1]`. Эти типы, в свою очередь, определяют типы результатов действия операторов `+` и `/`. Если мы вызовем функцию `median`, передав ей аргумент типа `vector<int>`, то нетрудно понять, что операторы `+` и `/` принимают `int`-операнды и возвращают `int`-результаты. При вызове функции `median` для вектора типа `vector<double>` арифметические операции выполняются с `double`-значениями. Мы не можем вызвать функцию `median` с аргументом типа `vector<string>`, поскольку функция `median` использует операцию деления, а тип `string` не имеет оператора деления. Такое поведение нас вполне устраивает. И потом, подумайте сами, что значит найти медиану вектора типа `vector<string>`?

8.1.2. Реализация шаблонов

При вызове функции `median` для вектора типа `vector<int>` C++-среда эффективно создаст и скомпилирует экземпляр этой функции, в котором каждое вхождение символа `T` будет заменено символами `int`. Если мы также вызовем функцию `median` для вектора типа `vector<double>`, то C++-среда снова “подставит” нужный тип из вызова. В этом случае обозначение `T` будет связано с типом `double`, и C++-среда сгенерирует другую версию функции `median`, используя вместо обозначения типа `T` тип `double`.

В стандарте C++ ничего не сказано о том, как C++-среды должны управлять реализацией шаблонов, поэтому каждая C++-среда делает это по-своему. Несмотря на то что мы не можем точно сказать, как ваш компилятор будет обрабатывать реализацию, важно помнить следующее. Во-первых, в C++-средах, которые используют традиционную модель редактирование—компиляция—компоновка, процесс реализации часто происходит не во время компиляции, а во время компоновки. Он не начинается до тех пор, пока шаблоны не будут конкретизированы, чтобы C++-среда могла убедиться в том, что код шаблона действительно можно использовать с заданными типами. Поэтому в процессе компоновки можно получить ошибки, которые, казалось бы, являются ошибками времени компиляции.

Во-вторых, при создании собственных шаблонов чаще всего требуется, чтобы для реализации шаблона его определение (а не просто объявление) было доступно C++-среде. Как правило, такое требование подразумевает доступ к исходным файлам, которые определяют шаблон, а также к файлу заголовка. Различные C++-среды по-разному находят исходные файлы. Многие C++-среды предполагают, что заголовочный файл для шаблона включен в исходный файл либо напрямую, либо посредством директивы `#include`. Для того чтобы узнать, как ведет себя в этом случае ваша C++-среда, существует проверенный способ — обратиться к документации.

8.1.3. Обобщенные функции и типы

Как было отмечено в разделе 8.1, самая трудная часть разработки и использования шаблонов заключается в точном взаимодействии между шаблоном и “подходящими типами”, которые можно использовать с шаблоном. В нашем определении шаблонной версии функции `median` мы увидели одну очевидную зависимость типов: типы значений, хранимых в векторах, которые передаются функции `median`, должны поддерживать операции сложения и деления в обычном “арифметическом” смысле. К счастью, большинство типов, в которых определена операция деления, являются арифметическими типами, поэтому маловероятно, что такие зависимости будут создавать какие-то проблемы на практике.

Более существенные проблемы при использовании шаблонов связаны с преобразованием типов. Например, чтобы узнать, все ли домашние задания выполнили студенты, мы использовали функцию `find`.

```
find(s.homework.begin(), s.homework.end(), 0);
```

В данном случае `homework` — это вектор типа `vector<double>`, но с помощью этой функции мы хотим найти заданное `int`-значение. Это несоответствие типов не имеет никаких негативных последствий: мы спокойно можем сравнивать `int`-значение с `double`-значением без каких-либо потерь.

Но, вероятно, вы заметили, что при вызове функции `accumulate`

```
accumulate(v.begin(), v.end(), 0.0)
```

корректность работы нашей программы зависела от использования нами именно `double`-формы нуля (`0.0`), а не `int`-формы (`0`). Дело в том, что функция `accumulate` применяет тип третьего аргумента в качестве типа накапливающего сумматора. Если бы для третьего аргумента мы использовали тип `int`, то даже при сложении последовательности `double`-значений результат усекался бы до целой части. В этом случае C++-среда позволила бы нам передать функции `int`-аргумент, но сумма, которую мы получили бы в качестве значения, возвращаемого этой функцией, пострадала бы от потери точности.

Наконец, при вызове функции `max`

```
string::size_type maxlen = 0;  
maxlen = max(maxlen, name.size());
```

мы отметили, что важно, чтобы тип аргумента `maxlen` в точности совпадал с типом значения, возвращаемого функцией `name.size()`. Если типы не совпадают, вызов функции `max` не будет скомпилирован. Теперь, когда мы знаем, что типы параметров шаблона зависят от типов аргументов, нетрудно понять причину такого поведения. Рассмотрим следующую реализацию функции `max`.

```
template<class T>  
T max(const T& left, const T& right)  
{  
    return left > right ? left : right;  
}
```

Если в качестве аргументов передать этой функции `int`- и `double`-значения, C++-среда не сможет решить, какой из аргументов следует преобразовать в значение типа другого аргумента: то ли реализовать этот вызов как сравнение двух `int`-значений и, следовательно, вернуть `int`-результат, то ли сравнить `double`-значения и вернуть `double`-результат. Поскольку C++-среда не в состоянии сама принять такое решение, этот вызов будет “забракован” во время компиляции.

8.2. Независимость структур данных

В функции `median`, о которой только что шла речь, шаблоны используются для обобщения типов значений, которые может содержать вектор. Мы можем вызвать эту функцию для вычисления медианы вектора значений любого арифметического типа.

В более общем случае мы хотели бы написать простую функцию, которая обрабатывала бы значения, хранимые в структуре данных любого типа, например типа `list`, `vector` или `string`. Более того, мы хотели бы иметь возможность обрабатывать не весь контейнер, а только его некоторую часть.

Например, стандартная библиотека использует итераторы, чтобы позволить нам вызывать функцию `find` для любой непрерывной части какого-либо контейнера. Если `c` — это контейнер, а `val` — значение типа, которое хранится в этом контейнере, мы можем использовать функцию `find`, записав следующее выражение.

```
find(c.begin(), c.end(), val)
```

Почему мы должны дважды указывать контейнер `c`? Почему библиотека не позволяет для вызова функции `find` использовать выражение

```
c.find(val)
```

по аналогии с выражением `c.size()`? Или даже выражение

```
find(c, val)
```

с непосредственной передачей функции `find` контейнера в качестве аргумента? Оказывается, что оба вопроса имеют один и тот же ответ: используя итераторы и, следовательно, требуя от нас дважды указать контейнер `c`, библиотека делает возможным создание единой функции `find`, которая может находить значение в любой непрерывной части какого-либо контейнера. Ни один из других способов не позволяет нам достичь такого универсализма.

Рассмотрим сначала выражение `c.find(val)`. Если бы библиотека позволила нам использовать вызов `c.find(val)`, мы вызывали бы функцию `find` как член типа контейнера, т.е. типа, который имеет контейнер `c`, а это значит, что тот, кто определял тип `c`, обязан был бы определить функцию `find` в качестве члена класса `c`. Кроме того, если бы библиотека использовала для алгоритмов стиль `c.find(val)`, то мы бы не смогли применять их для встроженных массивов, о которых поговорим в главе 10.

Почему библиотека требует, чтобы мы в качестве аргументов функции `find` передавали выражения `c.begin()` и `c.end()`, вместо того чтобы позволить нам напрямую передавать тип `c`? Дело в том, что, передавая два значения, мы тем самым ограничиваем диапазон поиска; это позволяет искать нужное значение не в целом контейнере, а только в заданной его части. Подумайте, например, о том, как вы написали бы функцию `split` (см. раздел 6.1.1), если бы функция `find_if` была ограничена необходимостью выполнять поиск обязательно в целом контейнере.

Существует и более важная причина передачи обобщенным функциям аргументов-итераторов вместо аргументов-контейнеров: итераторы позволяют получать более гибкий доступ к элементам контейнера. Например, в разделе 6.1.2 мы использовали функцию `rbegin`, которая возвращает итератор, предоставляющий доступ к элементам контейнера в обратном направлении. Передавая такой итератор в качестве аргумента функции `find` или `find_if`, мы можем просматривать элементы контейнера в обратном порядке, что было бы невозможно, если бы эти функции “настраивали” на непосредственном принятии контейнера в качестве аргумента.

Конечно, можно перегрузить библиотечные функции, чтобы их можно было вызывать, передавая в качестве аргумента либо контейнер, либо пару итераторов. Однако вряд ли это удобство стоит дополнительного усложнения библиотечных средств.

8.2.1. Алгоритмы и итераторы

Самый простой способ понять, как шаблоны позволяют писать программы, независимые от структур данных, — изучить реализацию некоторых популярных функций стандартной библиотеки. Все эти функции включают в качестве аргументов итераторы, идентифицирующие элементы контейнера, с которыми будут взаимодействовать эти функции. Все контейнеры стандартной библиотеки, а также некоторые другие типы, например `string`, поддерживают итераторы, которые позволяют этим функциям обрабатывать элементы контейнеров.

Некоторые контейнеры поддерживают операции, которые другие контейнеры не поддерживают. Эта поддержка или ее отсутствие означает, что существуют операции, которые одними итераторами поддерживаются, а другими — нет. Например, к элементу вектора с заданным индексом можно получить прямой доступ, а к элементу списка — нет. Следовательно, если у нас есть итератор, который указывает на некоторый элемент вектора, то природа такого итератора позволяет получить итератор, который указывает на другой элемент того же вектора, прибавив к первому итератору разность индексов их элементов. Итераторы, которые ссылаются на элементы списка, не предусматривают подобной возможности.

Поскольку разные виды итераторов поддерживают различные виды операций, важно понимать требования, предъявляемые разными алгоритмами к используемым в них итераторам и к операциям, которые поддерживаются различными видами итераторов. Всякий раз, когда два итератора поддерживают одну и ту же операцию, они используют для нее одно и то же имя. Например, все итераторы используют оператор “++”, после применения которого данный итератор должен ссылаться уже на следующий элемент контейнера.

Не всем алгоритмам необходимы все итераторные операции. Такие алгоритмы, как `find`, используют только несколько итераторных операций. Поэтому с помощью алгоритма `find` мы можем находить значения, используя итераторы любых контейнеров. Другие же алгоритмы, например `sort`, выполняют с итераторами множество операций, включая арифметические. Из знакомых нам библиотечных типов только `vector`- и `string`-объекты могут позволить себе такую “роскошь”, как алгоритм `sort`. (При сортировке `string`-объектов их отдельные символы перестраиваются в неубывающем порядке.)

Библиотека определяет пять *категорий итераторов*, каждая из которых соответствует определенной коллекции итераторных операций. Эти категории определяют вид итератора, предоставляемый каждым библиотечным контейнером. В каждом алгоритме стандартной библиотеки указывается, какая категория итераторов подходит для каждого аргумента-итератора. Таким образом, категории итераторов позволяют понять, какие контейнеры можно использовать в тех или иных алгоритмах.

Каждая категория итераторов соответствует стратегии доступа к элементам контейнера. А поскольку категории итераторов соотносятся со стратегиями доступа, они также соответствуют определенным видам алгоритмов. Например, некоторые алгоритмы делают только один проход по входным данным, и поэтому им не нужны итераторы, которые способны на многократные проходы. Другим же необходимо иметь возможность получе-

ния эффективного доступа к произвольному элементу, заданному только индексом, и поэтому им необходима способность складывать индексы и целые значения.

Теперь нам предстоит рассмотреть каждую стратегию доступа и алгоритм, который ее использует, а также описать соответствующую категорию итераторов.

8.2.2. Последовательный доступ только для чтения

Один из самых простых способов получения доступа к последовательности — последовательное считывание ее элементов. К библиотечным функциям, которые именно так и поступают, относится функция `find`, которую мы можем реализовать следующим образом.

```
template <class In, class X> In find(In begin,
                                     In end, const X& x)
{
    while (begin != end && *begin != x)
        ++begin;
    return begin;
}
```

Если мы вызовем функцию `find(begin, end, x)`, результат будет равен либо первому итератору `iter` в диапазоне `[begin, end)`, для которого справедливо выражение `*iter == x`, либо второму аргументу `end`, если такой итератор не существует.

Мы знаем, что эта функция последовательно получает доступ к элементам в диапазоне `[begin, end)`, поскольку единственной операцией, используемой здесь для изменения значения аргумента `begin`, является инкрементирование (`++`). Помимо оператора `++`, в этой функции используется оператор `!=` для сравнения значений `begin` и `end` и оператор `*` для доступа к элементу контейнера, на который ссылается итератор `begin`. Этих операций вполне достаточно для последовательного считывания элементов, на которые ссылаются итераторы из заданного диапазона.

Но несмотря на их достаточность, они не являются единственными операциями, которые мы бы хотели использовать. Например, функцию `find` можно было бы реализовать следующим образом.

```
template <class In, class X> In find(In begin,
                                     In end, const X& x)
{
    if (begin == end || *begin == x)
        return begin;
    begin++;
    return find(begin, end, x);
}
```

Хотя большинство C++-программистов посчитали бы этот рекурсивный стиль программирования необычным, программисты, знакомые с такими языками, как Lisp или ML, чувствовали бы себя как рыба в воде. В этой версии функции `find` вместо выражения `++begin` используется выражение `begin++`, а вместо оператора `!=` оператор `==`. На основании этих двух примеров можно заключить, что итератор, обеспечивающий только последовательное считывание элементов некоторой последовательности (доступ только для чтения), должен поддерживать оператор инкремента `++` (как в префиксной, так и в постфиксной форме), операторы сравнения `==` и `!=`, а также унарный оператор разыменования `*`.

Такой итератор должен поддерживать еще один оператор, обеспечивающий эквивалентность между выражением `(*iter).member` и выражением `iter->member`, кото-

рое мы использовали в разделе 7.2 в качестве сокращенной записи первого выражения, и мы хотели бы поступать так и впредь.

Если некоторый тип обеспечивает все эти операции, мы называем его *входным итератором* (input iterator). Все (уже знакомые нам) контейнерные итераторы поддерживают все перечисленные выше операции, поэтому они являются входными итераторами. Безусловно, они поддерживают и другие операции, но эта дополнительная поддержка никак не влияет на факт их причисления к входным итераторам.

Когда мы говорим, что функция `find` требует, чтобы в качестве ее первого и второго аргументов использовались входные итераторы, то это означает, что мы можем передать функции `find` аргументы любого типа, которые отвечают требованиям входных итераторов, включая итераторы, поддерживающие дополнительные операции.

8.2.3. Последовательный доступ только для записи

Входные итераторы можно использовать только для чтения элементов некоторой последовательности. Очевидно, в некоторых ситуациях нам бы хотелось иметь возможность применять итераторы для записи элементов последовательности. Например, рассмотрим функцию `copy`.

```
template<class In, class Out>
Out copy(In begin, In end, Out dest)
{
    while (begin != end)
        *dest++ = *begin++;
    return dest;
}
```

Эта функция принимает три итератора: первые два означают последовательность, из которой нужно копировать, а третий указывает на начало последовательности-приемника. Мы уже видели этот `while`-цикл в разделе 6.1: функция обрабатывает элементы, продвигая вперед аргумент `begin` по контейнеру до тех пор, пока не достигнет “отметки” `end`, и копируя при этом каждый элемент в приемный контейнер, а именно в позицию, обозначенную текущим значением аргумента `dest`.

Нетрудно догадаться по имени `In`, что аргументы `begin` и `end` — входные итераторы. Мы используем их только для считывания элементов, подобно тому, как мы делали это в функции `find`. Что же можно сказать об `Out`, типе параметра `dest`? Судя по операциям, в которых участвует параметр `dest`, мы видим, что по крайней мере в этой функции нам нужно иметь возможность вычислять выражения `*dest = Значение` и `dest++`. Как и в случае с функцией `find`, из соображений логической полноты необходимо, чтобы мы могли также вычислять выражение `++dest`.

Однако существует и еще одно требование, которое менее очевидно. Предположим, что `it` — это итератор, который мы хотим использовать только для вывода данных. Например, попробуем это сделать следующим образом.

```
*it = x;
++it;
++it;
*it = y;
```

Инкрементируя значение итератора `it` дважды между присваиваниями некоторых значений элементам `*it`, мы тем самым оставляем промежуток в нашей выходной последовательности. Следовательно, если мы хотим использовать некоторый итератор исключительно для вывода, необходимо соблюдать одно неявное требование: не выполнять выражение `++it` более одного раза между присваиваниями элементу `*it` или

не присваивать некоторое значение элементу `*it` более одного раза без инкрементирования итератора `it`.

Если некоторая функция использует какой-либо тип с выполнением этих требований, мы можем назвать этот тип *выходным итератором* (output iterator). Примером такой функции может служить итераторный адаптер `back_inserter`. Все стандартные контейнеры предоставляют итераторы, отвечающие этим требованиям. Необходимо отметить, что свойство “однократной записи” — это требование, предъявляемое к программам, использующим итераторы, а не к самим итераторам. Другими словами, итераторы, которые удовлетворяют только требованиям выходных итераторов, необходимы лишь для поддержки программ, которым присуще это свойство. Итератор, генерируемый функцией `back_inserter`, является выходным итератором, поэтому программы, которые используют его, должны подчиняться требованию “однократной записи”. Все контейнерные итераторы способны выполнять и другие операции, поэтому программы, которые их используют, не ограничены в этом плане.

8.2.4. Последовательный доступ для чтения и записи

Предположим, мы хотим иметь возможность считывать и записывать элементы некоторой последовательности, но только последовательно, т.е. перемещая итераторы только вперед (но не обратно). Примером библиотечной функции, которая именно так и действует, может служить функция `replace` из заголовка `<algorithm>`.

```
template<class For, class X>
void replace(For beg, For end, const X& x, const X& y)
{
    while (beg != end) {
        if (*beg == x)
            *beg = y;
        ++beg;
    }
}
```

Эта функция проверяет элементы в диапазоне `[beg, end)` и заменяет каждый элемент, который равен значению `x`, значением `y`. Вероятно, нетрудно согласиться с тем, что тип `For` должен поддерживать все операции, поддерживаемые любыми входными итераторами, а также все операции, поддерживаемые любыми выходными итераторами. Более того, он не обязан отвечать требованию однократного присваивания выходных итераторов, поскольку имеет смысл считывание значения элемента (после его присваивания) с последующим (возможным) его изменением. Такой тип называется *прямым*, или *однонаправленным итератором* (forward iterator), и “право” носить это имя присваивается при поддержке следующих операций:

```
*it (как для чтения, так и для записи);
++it и it++ (но не --it или it--);
it == j и it != j (где j имеет тот же тип, что и it);
it->member (в качестве синонима для выражения (*it).member).
```

Все контейнеры стандартной библиотеки отвечают требованиям прямых итераторов.

8.2.5. Двухнаправленный доступ

Некоторым функциям необходимо получать доступ к элементам контейнера в обратном порядке. Простейшим примером такой функции является `reverse`, которую стандартная библиотека определяет в заголовке `<algorithm>`.


```
template<class Bi> void reverse(Bi begin, Bi end)
{
    while (begin != end) {
        --end;
        if (begin != end)
            swap(*begin++, *end);
    }
}
```

В этом алгоритме мы продвигаем итератор `end` в обратном направлении с конца вектора, а итератор `beg` — с начала вектора, “по дороге” меняя местами элементы, на которые ссылаются эти итераторы.

Эта функция использует итераторы `begin` и `end` так, как если бы они были прямыми, за исключением того, что здесь также применяется оператор “--”, который, очевидно, является ключевым при прохождении последовательности в обратном направлении. Если некоторый тип отвечает всем требованиям прямого итератора, а также поддерживает оператор “--” (как префиксную, так и постфиксную его форму), мы называем его *двунаправленным итератором* (bidirectional iterator).

Двунаправленные итераторы поддерживаются всеми контейнерными классами стандартной библиотеки.

8.2.6. Произвольный доступ

Некоторым функциям необходимо иметь возможность “перескакивать” с одного элемента контейнера на другой. Подходящим примером такой функции может служить классический алгоритм двоичного поиска. Стандартная библиотека реализует этот алгоритм в нескольких формах, и самый простой из них называется `binary_search`. На самом деле стандартная библиотека использует ряд довольно сложных методов (их рассмотрение выходит за рамки этой книги), которые позволяют выполнять двоичный поиск в последовательностях, определяемых однонаправленными итераторами. Более простая версия, для реализации которой требуются итераторы произвольного доступа, выглядит следующим образом.

```
template<class Ran, class X>
bool binary_search(Ran begin, Ran end, const X& x)
{
    while (begin < end) {
        // Находим среднюю точку диапазона.
        Ran mid = begin + (end - begin) / 2;

        // Выясняем, какая часть диапазона содержит значение x;
        // продолжаем искать только в этой части.
        if (x < *mid)
            end = mid;
        else if (*mid < x)
            begin = mid + 1;
        // Если мы дошли сюда, значит, справедливо *mid == x,
        // т.е. мы нашли искомое значение.
        else return true;
    }
    return false;
}
```

Помимо использования прочих известных свойств итераторов, эта функция опирается на возможность выполнения с итераторами арифметических действий. Например, она вычитает один итератор из другого, чтобы получить некоторое целочисленное значение, и выполняет сложение итератора и `int`-значения, чтобы получить дру-

гой итератор. Снова-таки, логическая полнота расширяет требования, предъявляемые к итераторам произвольного доступа. Если p и q — такие итераторы, а n — целочисленное значение, то полный список дополнительных требований (помимо тех, которые предъявляются к двунаправленным итераторам) имеет следующий вид:

```
p + n, p - n и n + p;  
p - q;  
p[n] (эквивалент записи *(p + n));  
p < q, p > q, p <= q и p >= q.
```

Результатом вычитания двух итераторов является промежуток между итераторами, представленный в виде некоторого значения целочисленного типа, о котором пойдет речь в разделе 10.1.4. В перечисленные выше требования мы не включили операторы “==” и “!=”, поскольку итераторы произвольного доступа также поддерживают требования, предъявляемые к двунаправленным итераторам.

Единственным (из знакомых нам) алгоритмом, который требует итераторы произвольного доступа, является функция `sort`. Итераторы, предоставляемые классами `vector` и `string`, — это итераторы произвольного доступа, но итератор, предоставляемый классом `list`, таковым не является: списки поддерживают только двунаправленные итераторы. Почему?

Основная причина заключается в том, что списки оптимизированы для быстрого выполнения операций вставки и удаления элементов. Другими словами, специализация списков не предусматривает быстрого перехода к произвольно заданному элементу. Единственный способ перемещения по списку — последовательный просмотр каждого элемента.

8.2.7. Диапазоны итераторов и оконечные значения

Как видим, соглашение, в соответствии с которым алгоритмы принимают два аргумента для задания диапазонов, практически универсально в библиотеке. Первый аргумент ссылается на первый элемент диапазона; второй указывает на элемент, расположенный за последним элементом диапазона. Почему мы задаем элемент, уже не входящий в диапазон?

В разделе 2.6 мы уже обсудили одну причину использования верхней границы диапазона, которая представляет начало области, расположенной за последним значением диапазона. Если бы мы задавали конец диапазона значением, равным последнему элементу, то нам бы пришлось неявно оговаривать, что последний элемент должен обрабатываться специальным образом. Но в таком случае при написании программ очень легко совершать ошибки, связанные с остановкой обработки на одну итерацию раньше нужного окончания цикла. В силу самой природы итераторов существуют по крайней мере еще три причины отмечать конец диапазона итератором, который указывает не на последний элемент, а на элемент, расположенный за последним элементом диапазона.

Первая причина заключается в следующем. Если бы диапазон вообще не содержал элементов, не существовало бы и последнего элемента для указания его (диапазона) конца. Тогда бы нам пришлось изобрести способ для обозначения пустого диапазона посредством итератора, который бы указывал, где должен находиться элемент до начала диапазона. При таком подходе нам бы пришлось обрабатывать пустые диапазоны не так, как непустые, что сделало бы наши программы более трудными для понимания и менее надежными. В разделе 6.1.1 отмечалось, что, обрабатывая пустые диапазоны так же, как любые другие, мы значительно упрощаем свои программы.

Вторая причина состоит в следующем. Обозначение конца диапазона итератором, который указывает на элемент, расположенный за последним элементом диапазона, позволяет “отделаться” только проверкой равенства и неравенства итераторов и делает излишним определение, разъясняющее, что означает для одного итератора быть меньше другого. В этом случае мы можем сразу выяснить, является ли диапазон пустым, сравнив всего лишь два итератора: диапазон является пустым тогда и только тогда, когда равны итераторы, обозначающие его границы. Если они не равны, мы знаем, что итератор начала диапазона указывает на некоторый (существующий!) элемент, поэтому мы можем выполнять некоторые действия, а затем инкрементировать этот итератор, чтобы уменьшить размер обрабатываемого диапазона. Другими словами, отмечая диапазоны начальным элементом и элементом, расположенным за последним элементом диапазона, мы можем использовать циклы следующего вида (и должны при этом иметь возможность выполнять операции сравнения итераторов на равенство и неравенство).

```
// инвариант: мы должны по-прежнему обрабатывать элементы
// в диапазоне [begin, end).
while (begin != end) {
    // выполнение действий с элементом, на который
    // указывает итератор begin.
    ++begin;
}
```

Третья причина состоит в том, что определение диапазона его начальным элементом и элементом, расположенным за последним элементом диапазона, предоставляет нам естественный способ обозначить ситуацию “вне диапазона”. Многие алгоритмы стандартной библиотеки — и алгоритмы, которые мы пишем сами — используют преимущество значения, расположенного “вне диапазона”, возвращая второй итератор диапазона для обозначения неудачного выполнения. Например, наша функция `url_beg`, приведенная в разделе 6.1.3, применяла это соглашение, чтобы сообщить о неспособности найти URL-адрес. Если бы у алгоритмов не было в распоряжении этого значения, им бы пришлось изобрести его, что значительно усложнило бы как сами алгоритмы, так и программы, которые их используют.

Итак, хотя и кажется странным обозначать конец диапазона итератором, который указывает на элемент, расположенный за последним элементом диапазона, все же соблюдение этого соглашения делает большинство программ проще и надежнее, чем они были бы в противном случае. С этой целью необходимо, чтобы каждый тип контейнера поддерживал для своих итераторов “запредельное” (off-the-end) значение. Такое значение возвращает член `end` каждого контейнера и может быть также результатом других операций, выполняемых с итераторами. Например, если `c` — некоторый контейнер, то копирование итератора `c.begin()` и инкрементирование этой копии некоторое число раз, равное значению `c.size()`, сгенерирует итератор, равный итератору `c.end()`. Результат разыменования “запредельного” итератора не определен, поскольку он эквивалентен результату получения (посредством итератора) значения, расположенного до начала контейнера, или значения, находящегося несколькими позициями дальше от его конца.

8.3. Входные и выходные итераторы

Почему входные и выходные итераторы отделяются от односторонних, если ни один стандартный контейнер не требует такого разграничения? Прежде всего, не все итераторы связываются с контейнерами. Приведем пример: если `c` — контейнер, ко-

торый поддерживает операцию `push_back`, тогда выражение `back_inserter(c)` возвращает выходной итератор, который не отвечает никаким другим требованиям, предъявляемым к итераторам.

Рассмотрим еще один пример. Стандартная библиотека предоставляет итераторы, которые могут быть связаны с входным и выходным потоками. Неудивительно, что итераторы для потока `istream` отвечают требованиям, предъявляемым входным итераторам, а итераторы для потока `ostream` — требованиям, предъявляемым выходным итераторам. Используя соответствующий потоковый итератор, мы можем с помощью обычных итераторных операций обрабатывать поток `istream` или `ostream`. Например, оператор “++” переместит итератор к следующему значению в потоке. Для входных потоков оператор “*” прочитает значение, расположенное в текущей позиции входного потока, а для выходных потоков он позволит записать значение в выходной поток. Потоковые итераторы определяются в заголовке `<iterator>`.

Итератор входного потока имеет тип `istream_iterator`:

```
vector<int> v;
```

```
// Считываем int-значения из стандартного входного потока и
// присоединяем их к концу вектора v.
copy(istream_iterator<int>(cin), istream_iterator<int>(),
      back_inserter(v));
```

Как обычно, первые два аргумента, передаваемые функции `copy`, задают диапазон, из которого нужно копировать элементы. Первый аргумент создает новый итератор типа `istream_iterator`, связанный с входным потоком `cin`, для считывания значений типа `int`. Вспомните, что ввод и вывод данных в C++ являются типизированными операциями: при считывании из потока мы всегда указываем тип значения, которое собираемся прочитать, несмотря на то что эти типы часто неявным образом задаются в операциях, реализующих считывание данных. Например:

```
getline(cin, s);           // Считываем данные в string-объект.
cin >> s.name >> s.midterm >> s.final; // Считываем string-значение и
                                         // два double-значения.
```

Аналогично при определении итератора потока мы должны иметь способ сообщить ему, значение какого типа он должен прочитать или записать в поток. Следовательно, итераторы потока представляют собой шаблоны.

Второй аргумент, передаваемый функции `copy`, создаст по умолчанию (пустой) итератор типа `istream_iterator<int>`, который не связан ни с каким файлом. Тип `istream_iterator` имеет значение по умолчанию, обладающее следующим свойством: любой итератор типа `istream_iterator`, который достиг признака конца файла (end-of-file) или находится в состоянии ошибки, становится равным некоторому стандартному значению, заданному по умолчанию. Таким образом, мы можем использовать это стандартное значение для индикации достижения конца диапазона (one-past-the-end) в соответствии с соглашением для функции `copy`.

Теперь становится понятно, что при обращении к функции `copy` из стандартного входного потока будут считываться значения до тех пор, пока не обнаружится признак конца файла или значение, не соответствующее заданному типу `int`.

Мы не можем использовать итератор типа `istream_iterator` для вывода данных. Для этого необходим итератор типа `ostream_iterator`.

```
// Выводим элементы вектора v, отделяя один от другого пробелами.
copy(v.begin(), v.end(), ostream_iterator<int>(cout, " "));
```

В данном случае мы копируем весь вектор в стандартный выходной поток. Третий аргумент создает новый итератор, связанный с выходным потоком `cout`, для вывода значений типа `int`.

Второй аргумент, используемый для создания объекта типа `ostream_iterator<int>`, задает значение, которое должно быть выведено после каждого элемента вектора. Обычно это значение представляет собой строковый литерал. Если не задать это значение, `ostream_iterator` выведет элементы вектора без разделителей.

Таким образом, если опустить разделитель, то при обращении к функции `copy` все выводимые значения будут нечитабельны.

```
// Без разделителя между выводимыми элементами!  
copy(v.begin(), v.end(), ostream_iterator<int>(cout));
```

8.4. Использование итераторов для повышения гибкости программирования

Теперь мы можем немного усовершенствовать функцию `split`, приведенную в разделе 6.1.1. Как вы помните, функция `split` возвращает вектор типа `vector<string>`, что “связывает руки” нашим пользователям: вместо вектора они хотели бы работать, например, со списком типа `list<string>` или другим типом контейнера. Пожелания пользователей удовлетворить совсем нетрудно, поскольку в алгоритме `split` нет ничего такого, чтобы заставляло нас использовать именно тип `vector`.

Мы можем переписать функцию `split`, обеспечив определенную гибкость в ее использовании, но теперь вместо возврата жестко заданного значения функция будет принимать выходной итератор. В новой версии функции мы будем использовать этот итератор для вывода обнаруженных слов. Автор вызова должен связать выходной итератор с некоторым удобным для него приемником, в котором будут размещены выводимые значения.

```
template <class Out> // Изменено.  
void split(const string& str, Out os) { // Изменено.  
  
    typedef string::const_iterator iter;  
  
    iter i = str.begin();  
    while (i != str.end()) {  
        // Игнорируем ведущие пробелы.  
        i = find_if(i, str.end(), not_space);  
  
        // Находим конец следующего слова.  
        iter j = find_if(i, str.end(), space);  
  
        // Копируем символы из диапазона [i, j).  
        if (i != str.end())  
            *os++ = string(i, j); // Изменено.  
        i = j;  
    }  
}
```

Подобно функции `write_analysis`, которую мы написали в разделе 6.2.2, наша новая версия функции `split` не возвращает никакого значения, поэтому в качестве типа возвращаемого значения мы указываем `void`. Мы превратили функцию `split` в шаблонную функцию, которая принимает параметр-тип `Out`, в имени которого угадывается выходной итератор. Вспомните, что поскольку однонаправленные и двунаправленные итераторы, а также итераторы произвольного доступа удовлетворяют всем требованиям, предъявляемым

к выходным итераторам, мы можем использовать нашу функцию `split` с любым типом итератора, за исключением входного итератора типа `istream_iterator`.

Параметр `os` имеет тип `out`. Мы будем использовать его для записи значений слов по мере их обнаружения. Запись найденного слова в приемный контейнер реализуется одной инструкцией практически в конце тела функции.

```
*os++ = string(i, j);    // Изменено.
```

Подвыражение `*os` означает текущую позицию в контейнере, с которым связан итератор `os`, поэтому мы присваиваем значение `string(i, j)` элементу, расположенному в этой позиции. Выполнив операцию присваивания, мы инкрементируем значение итератора `os` (что целиком отвечает требованиям, предъявляемым к выходным итераторам), чтобы на следующей итерации цикла очередное найденное слово было присвоено следующему элементу контейнера.

Программистам, которые хотели бы использовать обновленную функцию `split`, придется внести некоторые изменения в свои программы, но зато теперь они могут записывать слова практически в любой контейнер. Например, если `s` — это `string`-объект, отдельные слова из которого мы хотим поместить в конец списка `word_list`, то нам нужно вызвать шаблонную функцию `split` следующим образом.

```
split(s, back_inserter(word_list));
```

Можно написать простую программу для тестирования нашей новой функции `split`.

```
int main()
{
    string s;
    while (getline(cin, s))
        split(s, ostream_iterator<string>(cout, "\n"));
    return 0;
}
```

Подобно драйверной функции из раздела 5.7, эта функция вызывает функцию `split`, чтобы разбить входную строку на отдельные слова и записать эти слова в стандартный выходной поток. Запись в выходной поток `cout` реализуется путем передачи функции `split` выходного итератора типа `ostream_iterator<string>`, который мы связываем с объектом `cout`. Таким образом, при присваивании элементу `*os` найденного значения функция `split` тем самым записывает это значение в объект выходного потока `cout`.

8.5. Резюме

Шаблонные функции, которые возвращают значения простых типов, имеют следующую форму.

```
template<class параметр-тип [, class параметр-тип] ... >
Тип_возврата Имя_функции(Список_параметров)
```

Каждый элемент *параметр-тип* представляет собой имя, которое может применяться внутри определения функции в любом месте, где требуется тип. Каждое такое имя должно быть использовано в качестве составляющего звена элемента *Список_параметров* функции, чтобы указать тип одного или нескольких параметров.

Если при вызове шаблонной функции не все типы указываются в списке аргументов, то автор вызова такой функции должен квалифицировать элемент *Имя_функции* с указанием реальных типов, которые не могут быть заданы в списке аргументов. Например, заголовок

```
template<class T> T zero() { return 0; }
```

определяет `zero` как шаблонную функцию с одним параметром-типом, который используется для указания типа значения, возвращаемого этой функцией. При вызове этой функции необходимо явным образом указать тип возвращаемого значения.

```
double x = zero<double>();
```

Ключевое слово `typename` необходимо использовать для квалификации объявлений с помощью типов, которые определяются шаблонными параметрами-типами. Например, инструкция

```
typename T::size_type ИМЯ;
```

объявляет элемент *ИМЯ* в качестве имени для типа `size_type`, который должен быть определен как тип внутри типа `T`.

C++-среда автоматически создает отдельный экземпляр данной шаблонной функции для каждого набора типов, используемых в обращении к этой функции.

Итераторы. Основной вклад стандартной библиотеки C++ — это воплощение идеи, согласно которой алгоритмы можно реализовать вне зависимости от структур данных. Это стало возможным благодаря использованию итераторов в качестве связующего звена между алгоритмами и контейнерами. Кроме того, тот факт, что алгоритмы могут быть разделены на основе операций, которые необходимо поддерживать итераторам, означает, что нетрудно найти соответствие между контейнером и алгоритмами, которые могут быть с ним использованы.

Выделяют пять категорий итераторов. Для них действует общее правило: каждой следующей категории присущи операции, относящиеся к предыдущим категориям.

Входной итератор	Последовательный доступ в одном направлении, только для ввода данных
Выходной итератор	Последовательный доступ в одном направлении, только для вывода данных
Однонаправленный итератор	Последовательный доступ в одном направлении, для ввода и вывода данных
Двунаправленный итератор	Последовательный доступ в обоих направлениях, для ввода и вывода данных
Итератор произвольного доступа	Эффективный доступ к любому элементу, для ввода и вывода данных

Упражнения

- 8.0. Скомпилируйте, выполните и протестируйте программы, приведенные в этой главе.
- 8.1. Обратите внимание на то, что функции с именем `analysis`, которые мы написали в разделе 6.2, ведут себя одинаково; они отличаются только названиями функций, вызываемых ими для вычисления итоговой оценки. Напишите шаблонную функцию, параметризованную по типу функции вычисления итоговой оценки, и используйте ее для определения качества оценки успеваемости студентов с помощью различных схем.

- 8.2. Реализуйте следующие библиотечные алгоритмы, которые мы использовали в главе 6 и описали в разделе 6.5. Укажите, итераторы каких типов им необходимо применять. Попробуйте минимизировать количество отдельных итераторных операций, которые требуются каждой функции. Завершив свой вариант реализации, обратитесь к разделу Б.3, чтобы узнать, насколько вам удалось справиться с этим заданием.

<code>equal(b, e, d)</code>	<code>search(b, e, b2, e2)</code>
<code>find(b, e, t)</code>	<code>find_if(b, e, p)</code>
<code>copy(b, e, d)</code>	<code>remove_copy(b, e, d, t)</code>
<code>remove_copy_if(b, e, d, p)</code>	<code>remove(b, e, t)</code>
<code>transform(b, e, d, f)</code>	<code>partition(b, e, p)</code>
<code>accumulate(b, e, t)</code>	

- 8.3. Как отмечалось в разделе 4.1.4, было бы слишком дорогим “удовольствием” возвращать (или передавать функции) контейнер по значению. Тем не менее функция `median`, которую мы написали в разделе 8.1.1, передает `vector`-объект по значению. Можно ли переписать функцию `median`, чтобы она вместо передачи вектора манипулировала итераторами? Если вам удалось это сделать, то как ваша переделка отразилась на производительности?
- 8.4. Реализуйте функцию `swap`, которую мы использовали в разделе 8.2.5. Почему мы вызываем функцию `swap`, а не напрямую меняем местами значения `*beg` и `*end`? *Подсказка:* попробуйте это сделать и все поймете сами.
- 8.5. Напишите новые версии функций `gen_sentence` и `xref` из главы 7, чтобы они использовали выходные итераторы вместо вывода результатов прямо в вектор типа `vector<string>`. Протестируйте эти новые версии, написав программы, которые связывают выходной итератор непосредственно со стандартным выходным потоком, и сохранив результаты в списке типа `list<string>` и отображении типа `map<string, vector<string> >` соответственно.
- 8.6. Предположим, что `m` имеет тип `map<int, string>` и мы встречаем в некоторой программе обращение к функции `copy(m.begin(), m.end(), back_inserter(x))`. Что можно сказать о типе объекта `x`? Что случится, если вместо этого обращения использовать вызов `copy(x.begin() x.end(), back_inserter(m))`?
- 8.7. Почему функция `max` не использует два шаблонных параметра, по одному для каждого типа аргумента?
- 8.8. Почему в функции `binary_search`, приведенной в разделе 8.2.6, мы не написали выражение `(begin + end)/2` вместо более сложного выражения `begin + (end - begin)/2`?

ОПРЕДЕЛЕНИЕ НОВЫХ ТИПОВ

В языке C++ прекрасно “уживаются” два вида типов: встроенные и типы классов. Встроенные типы (своим названием они обязаны тому, что определены как часть базового языка) включают `char`, `int` и `double`. Типы, которые мы “взяли” из библиотеки, например `string`, `vector` и `istream`, относятся к типам классов. За исключением некоторых низкоуровневых специализированных системных программ, “прописанных” в библиотеке ввода-вывода, библиотечные классы опираются только на те библиотечные средства, которые может использовать любой программист для определения собственных типов, связанных с конкретным применением.

Существенная часть C++-программирования основывается на идее предоставления программистам возможности создавать типы, которые будут так же просты в применении, как и встроенные типы. Как мы вскоре увидим, эта возможность (создавать типы с простым интуитивным интерфейсом) требует значительной языковой поддержки, подобно тому, как в проектировании классов необходимы вкус и проницательность. Начнем, пожалуй, с уже знакомой нам задачи вычисления итоговых оценок из главы 4; возможно, так нам будет проще рассмотреть основные принципы определения классов. Начиная с главы 11, мы будем постепенно “надстраивать” эти базовые концепции, чтобы понять, как создавать типы, которые можно сравнить с библиотечными по полноте и совершенству.

9.1. Вернемся к структуре `Student_info`

В разделе 4.2.1 мы написали простую структуру данных `Student_info` и ряд функций, которые значительно облегчили написание программ учета успеваемости студентов. Однако, как оказалось, написанные нами функции (вместе со структурой данных) не вполне устраивали других программистов.

Программисты, которые хотели бы использовать наши функции, вынуждены были соблюдать определенные соглашения. Например, мы предполагали, что пользователь только что созданного объекта типа `Student_info` должен сначала считать в него данные. Не сделав (или не сумев сделать) этого, пользователь получил бы объект с пустым вектором `homework` и неопределенными значениями (см. раздел 3.1) переменных `midterm` и `final`. Любое использование этих значений привело бы к непредсказуемому поведению: либо к некорректным результатам, либо к незамедлительному отказу. Более того, если пользователь захотел бы проверить, содержит ли объект типа `Student_info` достоверные данные, то единственный способ сделать это — обратиться к реальным членам данных — потребовал бы подробных знаний того, как мы реализовали класс `Student_info`.

Кроме того, тот, кто воспользовался нашими программами, мог предположить, что если запись с оценками студента была считана из файла, то это означает, что данные не могут измениться в будущем. К сожалению, наш код не дает оснований для подобных предположений.

Еще одна проблема состоит в “разбросанности” интерфейса для работы с нашей исходной структурой `Student_info`. По определенному соглашению мы могли бы поместить такие функции, как `read`, которая изменяет состояние объекта типа `Student_info`, в некоторый файл заголовка. В этом случае потенциальным пользователям было бы проще использовать наш код (конечно, при условии существования такого соглашения), но для подобного группирования функций не предусмотрено никаких требований.

Как будет показано в этой главе, мы можем несколько расширить структуру `Student_info`, что позволит решить все очерченные выше проблемы.

9.2. Типы классов

На фундаментальном уровне тип класса — это механизм объединения значений связанных данных в некоторую структуру, позволяющий обрабатывать эту структуру данных как единое целое. Например, структура `Student_info` (которую мы построили в разделе 4.2.1)

```
struct Student_info {  
    std::string name;  
    double midterm, final;  
    std::vector<double> homework;  
};
```

позволяет определять и обрабатывать объекты типа `Student_info`. Каждый объект этого типа имеет четыре элемента данных: член `name` типа `std::string`, член `homework` типа `std::vector<double>` и два `double`-члена с именами `midterm` и `final`.

Программисты, которые используют тип `Student_info`, могут (и должны) непосредственно обрабатывать эти элементы данных. Они могут это делать, поскольку определение структуры `Student_info` никак не ограничивает доступ к этим членам. Они должны это делать, поскольку никакие другие операции в структуре `Student_info` недоступны.

Вместо того чтобы разрешать пользователям прямой доступ к данным, мы хотели бы скрыть детали реализации, т.е. информацию о том, как хранятся в памяти объекты типа `Student_info`. В частности, мы хотели бы потребовать от пользователей этого типа данных, чтобы они получали доступ к объектам только посредством функций. Для этого, прежде всего, нам необходимо обеспечить пользователей удобными операциями, предназначенными для обработки объектов типа `Student_info`. Вот эти операции и образуют *интерфейс* для нашего класса.

Прежде чем рассматривать эти функции, имеет смысл вспомнить, почему мы используем полностью специфицированные (составные) имена для типов `std::string` и `std::vector`, а не предполагаем, что будет сделано `using`-объявление, позволяющее получать к именам прямой доступ. Код, в котором предполагается применение нашей структуры `Student_info`, должен иметь доступ к определению класса, поэтому мы поместим это определение в файл заголовка. Как отмечалось в разделе 4.3, код, предназначенный для использования другими программистами, должен содержать минимальное количество необходимых объявлений. Очевидно, мы должны определить имя `Student_info`, поскольку это как раз то имя, которое, по нашему разумению, и нуж-

но знать пользователям. Тот факт, что в структуре `Student_info` используются типы `string` и `vector`, относится к подробностям реализации. Поэтому нет причины пользователям структуры `Student_info` навязывать `using`-объявления только потому, что мы решили использовать эти типы в реализации нашей структуры.

В наших примерах программ (в качестве образца хорошего стиля программирования) мы используем составные имена в коде, который будет составлять содержимое файлов заголовков, но мы будем все также предполагать, что соответствующие исходные файлы содержат надлежащие `using`-объявления. Таким образом, при написании текста программ, который не входит в файл заголовка, в большинстве случаев мы не будем использовать полностью специфицированные имена.

9.2.1. Функции-члены

Для управления доступом к объектам типа `Student_info` нам необходимо определить интерфейс, который могут использовать любые программисты. Начнем с определения операций, необходимых для считывания очередной записи (функция `read`) и вычисления итоговой оценки (функция `grade`).

```
struct Student_info {
    std::string name;
    double midterm, final;
    std::vector<double> homework;

    std::istream& read(std::istream&); // Добавлено.
    double grade() const;             // Добавлено.
};
```

Этим определением мы по-прежнему сообщаем, что каждый объект типа `Student_info` содержит четыре элемента (члена) данных; кроме того, мы расширили двумя функциями-членами тип `Student_info`. Эти функции-члены позволят нам считывать запись из входного потока и вычислять итоговую оценку для каждого объекта типа `Student_info`. Модификатор `const` в объявлении функции `grade` — это своего рода обещание, что вызов функции `grade` не изменит ни одного из членов данных объекта типа `Student_info`.

Впервые мы столкнулись с функциями-членами еще в разделе 1.2, когда говорили об использовании функции `size`, члена класса `string`. По сути, *функция-член* (member function) — это функция, которая является членом объекта какого-нибудь класса. Чтобы вызвать функцию-член, пользователи должны указать объект, членом которого является вызываемая функция. Так, по аналогии с вызовом функции `greeting.size()` для `string`-объекта с именем `greeting` наши пользователи должны вызывать функции `s.read(cin)` или `s.grade()` “от лица” объекта типа `Student_info` с именем `s`. При вызове функции `s.read(cin)` из стандартного входного потока будут считаны введенные значения и установлено соответствующее состояние объекта `s`. При вызове функции `s.grade()` будет вычислено и возвращено значение итоговой оценки для объекта `s`.

Определение первой из наших функций-членов выглядит во многом как исходная версия функции `read`, приведенная в разделе 4.2.2.

```
istream& Student_info::read(istream& in)
{
    in >> name >> midterm >> final;
    read_hw(in, homework);
    return in;
}
```

Как и прежде, мы должны поместить эти функции в обычный исходный файл с именем `Student_info.cpp`, `Student_info.C` или `Student_info.c`. Самое главное здесь то, что объявления этих функций являются ныне частью нашей структуры `Student_info`, поэтому они должны быть доступны всем пользователям класса `Student_info`.

Между этим кодом и исходным вариантом есть три важных различия.

1. Функция-член имеет имя `Student_info::read`, а не просто `read`.
2. Поскольку эта функция является членом объекта типа `Student_info`, нам не нужно передавать `Student_info`-объект в качестве аргумента или вообще определять этот объект.
3. Мы получаем прямой доступ к элементам данных нашего объекта. Например, в разделе 4.2.2 при обращении к элементу структуры мы использовали выражение `s.midterm`; здесь же мы просто указываем имя `midterm`.

Теперь рассмотрим эти различия подробнее.

Символ “`::`” в имени функции — это тот же оператор разрешения области видимости, который мы уже использовали в “далеком” разделе 0.7 для доступа к именам, определяемым стандартной библиотекой. Например, для уточнения имени `size_type` мы указали составное имя `string::size_type`, поскольку `size_type` является членом класса `string`. Аналогично, написав `Student_info::read`, мы определяем функцию, именуемую `read`, которая является членом типа `Student_info`.

Эта функция-член требует при вызове (и передаче параметра) только указания типа `istream&`, поскольку параметр `Student_info&` будет неявно присутствовать в любом вызове. Вспомните, что когда мы вызываем функцию, которая является членом какого-нибудь `vector`- или `string`-объекта, мы должны указать, какой именно `vector`- или `string`-объект нас интересует. Например, если `s` — это `string`-объект, то для вызова члена `size` объекта `s` мы используем выражение `s.size()`. Без указания `string`-объекта невозможно вызвать функцию `size` из класса `string`. И точно так же, когда мы вызываем функцию `read`, то должны явно сообщить, в какой объект типа `Student_info` мы будем считывать данные. Этот объект неявно используется в функции `read`.

Ссылки на члены внутри функции `read` являются неспецифицированными (т.е. используются без уточняющего префикса), поскольку они представляют собой ссылки на члены объекта, на который мы воздействуем. Другими словами, если мы вызываем функцию `s.read(cin)` для `Student_info`-объекта с именем `s`, то воздействуем на объект `s`. Если в теле функции `read` используются члены `midterm`, `final` и `homework`, значит, на самом деле она будет применять члены `s.midterm`, `s.final` и `s.homework` соответственно.

А теперь рассмотрим функцию-член `grade`.

```
double Student_info::grade() const
{
    return ::grade(midterm, final, homework);
}
```

В таком виде она напоминает версию, приведенную в разделе 4.2.2; список различий между ними аналогичен списку, приведенному выше для функции-члена `read`: мы определяем функцию `grade` как член типа `Student_info`, функция принимает неявную (а не явную) ссылку на объект типа `Student_info` и получает доступ к членам этого объекта без какого бы то ни было уточнения.

В отношении этого кода можно указать еще два важных отличия от исходного варианта. Во-первых, обратите внимание на обращение `::grade`. Символ “`::`” перед именем

“настаивает” на использовании версии того имени, которое не является членом чего-либо. В данном случае нам нужен символ “:”, чтобы этот вызов заставил работать версию функции `grade` (определенную в разделе 4.1.2), которая принимает два `double`-значения и вектор типа `vector<double>`. Без этого префикса компилятор решил бы, что мы ссылаемся на функцию `Student_info::grade`, и выразил бы свое “недовольство”, поскольку мы попытались вызвать ее и передать слишком много аргументов.

Во-вторых, еще одно важное отличие состоит в применении модификатора `const` сразу за списком параметров функции `grade`. Такое его использование можно понять, сравнив объявление нашей новой функции с исходным (см. раздел 4.2.2).

```
double Student_info::grade() const { . . . } // версия
                                           // функции-члена.
double grade(
    const Student_info&) { . . . } // исходная версия
                                // из раздела 4.2.2.
```

В исходной версии мы передавали функции в качестве параметра `Student_info`-объект в виде ссылки на `const`-значение. Тем самым мы обеспечили вызов функции `grade` для `const`-объекта типа `Student_info` и гарантировали, что компилятор не позволит функции `grade` изменить свой параметр.

При вызове функции-члена объект, членом которого является эта функция, не используется в качестве аргумента. Следовательно, в списке параметров не существует элемента, для которого мы могли бы заявить о константности объекта. Поэтому мы и квалифицируем модификатором `const` саму функцию, тем самым делая ее *константной функцией-членом*. Константные функции-члены (`const`-функции-члены) не могут изменить внутреннее состояние объекта, для которого они выполняются: если мы вызываем функцию `s.grade()` “от имени” `Student_info`-объекта с именем `s`, то гарантируем, что этот вызов не изменит значения членов данных объекта `s`.

Поскольку эта функция гарантирует, что она не изменит значения данных в объекте, мы можем спокойно вызывать ее для `const`-объектов. Но неконстантные функции мы не можем вызывать для `const`-объектов; например, мы не можем вызвать функцию-член `read` “от имени” `const`-объекта типа `Student_info`. И потом, в самой природе такой функции, как `read`, заложено, что она может изменить состояние объекта. Ее вызов для `const`-объекта нарушил бы наше “`const`-обещание”.

Важно отметить, что даже если программа никогда напрямую не создает никаких `const`-объектов, она может посредством вызовов функций создавать множество ссылок на `const`-объекты. Когда мы передаем функции, которая должна принимать `const`-ссылку, неконстантный объект, то функция обрабатывает его так, как если бы он был константным, а компилятор позволяет вызывать только `const`-члены таких объектов.

Обратите внимание на то, что мы включили модификатор `const` как в объявление функции внутри определения класса, так и в определение функции. Как всегда, типы аргументов должны быть идентичны в объявлении и определении функции (см. раздел 4.4).

9.2.2. Функции-не-члены

В нашем новом проекте функции `read` и `grade` превратились в функции-члены. А как насчет функции `compare`? Следует ли и ее сделать функцией-членом класса?

Как будет отмечено в разделах 9.5, 11.2.4, 11.3.2, 12.5 и 13.2.1, язык C++ содержит требование, согласно которому только определенные виды функций могут быть определены как члены классов. Оказывается, что функция `compare` не относится к таковым. Ну что ж, не беда! У нас есть другая возможность выполнить ее. Существует общее правило, которое помогает нам решать, как поступать в таких случаях: если

функция изменяет состояние объекта, она должна быть членом этого объекта. К сожалению, даже это правило умалчивает о функциях, которые не изменяют состояние объекта, поэтому мы сами должны принять решение относительно функции `compare`.

Для этого нам необходимо проанализировать действия функции и то, как пользователи хотели бы ее вызывать. Функция `compare` определяет, какой из ее двух аргументов типа `Student_info` “меньше” другого, анализируя члены `name` своих аргументов. Как мы увидим в разделе 12.2, иногда можно извлечь некоторую пользу, если определить такие операции, как сравнение (`compare`) вне тела класса. Следовательно, оставим функцию `compare` как глобальную, а к ее реализации перейдем чуть позже.

9.3. Средства защиты

Определив `grade` и `read` как функции-члены, мы решили только половину задачи: пользователи объектов типа `Student_info` больше не *обязаны* напрямую управлять внутренним состоянием объекта. Но они по-прежнему *могут* это сделать. Мы же хотели бы скрыть данные и позволить пользователям получать доступ к данным только посредством наших функций-членов.

Язык C++ поддерживает сокрытие данных, разрешая авторам типов указывать, какие члены этих типов являются *открытыми* (`public`) и, следовательно, доступными для всех пользователей этого типа, а какие — закрытыми (`private`) и, соответственно, недоступными для пользователей этого типа.

```
class Student_info {
public:
    // Место для интерфейса.
    double grade() const;
    std::istream& read(std::istream&);

private:
    // Место для реализации.
    std::string name;
    double midterm, final;
    std::vector<double> homework;
};
```

В наше определение типа `Student_info` мы внесли два изменения: вместо ключевого слова `struct` мы употребили слово `class` и добавили две *метки защиты* (protection labels). Каждая метка защиты определяет доступность всех членов, расположенных за ней. Метки могут находиться в любом порядке внутри класса и могут встречаться по несколько раз.

Поместив члены `name`, `homework`, `midterm` и `final` после метки `private`, мы сделали эти элементы данных недоступными для пользователей типа `Student_info`. Ссылки на эти члены из функций, не являющихся членами, теперь недопустимы, и при наличии таковых компилятор сгенерирует диагностическое сообщение о том, что данный член является закрытым или недоступным. Члены в `public`-разделе абсолютно доступны, и любой пользователь может вызывать функцию `read` или `grade`.

Теперь поговорим о замене слова `struct` словом `class`. Вообще говоря, определяя новый тип, можно использовать любое из этих ключевых слов. Единственное различие между ними состоит в действующих по умолчанию средствах защиты, которые применяются к каждому члену, определенному до первой защитной метки. Коль мы написали `class Student_info`, то каждый член между первой фигурной скобкой (`{`) и первой меткой защиты является закрытым (`private`). Если бы мы написали `struct`

Student_info, то каждый член между первой фигурной скобкой и первой меткой защиты был бы открытым (public). Например, определение

```
class Student_info {
public:
    double grade() const;
    // И так далее.
};
```

эквивалентно следующему.

```
struct Student_info {
    double grade() const; // Действует public по умолчанию.
    // И так далее.
};
```

А это определение

```
class Student_info {
    std::string name; // Действует private по умолчанию.
    // Другие private-члены.
public:
    double grade() const;
    // Другие public-члены.
};
```

эквивалентно следующему.

```
struct Student_info {
private:
    std::string name;
    // Другие private-члены.
public:
    double grade() const;
    // Другие public-члены.
};
```

В каждом из этих определений мы заявляем, что пользователям разрешается получать доступ к функциям-членам объектов типа Student_info, но не позволяем им получать доступ к членам данных.

Нет никакой разницы между тем, что мы можем делать со структурой (struct) или классом (class). На самом деле для наших пользователей просто не существует способа (если, конечно, не видеть сам код), позволяющего выяснить, какое ключевое слово мы использовали для определения нашего типа класса: struct или class. Наш выбор, пожалуй, может иметь значение для документации. Обычно мы придерживаемся следующего стиля программирования: использовать struct для обозначения простых типов, когда у нас нет намерения скрывать свои структуры данных. Поэтому мы и применили слово struct для определения нашего исходного типа данных Student_info в главе 4. Теперь же, когда мы решили построить тип, который должен управлять доступом к своим членам извне, используем слово class.

9.3.1. Функции доступа

Теперь, когда мы скрыли наши члены данных, пользователи не смогут больше модифицировать данные в объекте типа Student_info. Вместо этого они должны использовать операцию чтения (с помощью функции-члена read) для установки значений членов данных и функцию grade, вычисляющую итоговую оценку для данного Student_info-объекта. Есть еще одна операция, которую мы по-прежнему должны предоставить в распоряжение наших пользователей: мы должны дать возможность узнать имя студента. На-

пример, вспомните программу из раздела 4.5, в которой мы выводили форматированный отчет об успеваемости студентов. Этой программе, чтобы сгенерировать отчет, необходимо иметь доступ к каждому имени студента. При этом мы хотим разрешить доступ для чтения, но не для записи. Реализация этих пожеланий достаточно проста.

```
class Student_info {
public:
    double grade() const;
    std::istream& read(std::istream&); // Необходимо
                                       // изменить определение.
    std::string name() const { return n; } // Добавлено.

private:
    std::string n; // Изменено.
    double midterm, final;
    std::vector<double> homework;
};
```

Однако вместо предоставления нашим пользователям доступа к члену данных `name`, мы добавили в определение типа `Student_info` еще одну функцию-член, также именуемую `name`, которая обеспечит доступ (только для чтения) к соответствующему значению данных. Конечно, теперь мы должны изменить имя этого члена данных, чтобы избежать путаницы с именем функции.

Функция `name` — это константная функция-член, которая не принимает никаких аргументов и возвращает `string`-объект, представляющий собой копию члена данных `n`. Благодаря тому, что мы копируем значение члена `n`, а не возвращаем ссылку на него, мы гарантируем, что пользователи могут лишь считывать значение `n` (но не изменять). Поскольку нам нужно обеспечить доступ только для чтения этого члена данных, мы объявляем функцию-член `name` с модификатором `const`.

Функции `grade` и `read` мы определили вне определения класса. Определяя же функцию-член как часть определения класса (именно так мы поступили с функцией `name`), мы тем самым подсказываем компилятору, что ему следует избежать затрат на вызовы этой функции и развернуть, где это возможно, обращения к ней “в строку” (см. раздел 4.6), заменив вызов функции ее кодом.

Такие функции, как `name`, часто называются *аксессорными* (accessor functions), или *функциями доступа*. Этот термин потенциально вводит в заблуждение, поскольку он подразумевает, что мы предоставляем доступ к некоторой части нашей структуры данных. И в самом деле, исторически сложилось так, что такие функции часто вводились для обеспечения простого доступа к скрытым данным, нарушая, таким образом, инкапсуляцию, которой мы так упорно пытались добиться. Аксессоры должны предоставляться только в случае, когда они являются частью абстрактного интерфейса класса. Что касается типа `Student_info`, то наша абстракция включает некоторые данные о студенте и его итоговой оценке. Доступ к имени студента является частью нашей абстракции, и эту часть вполне можно “рассекретить” посредством функции `name`. Но мы отнюдь не обеспечиваем средствами доступа другие данные: оценки `midterm`, `final` или вектор оценок `homework`, полученных за домашние задания. Эти оценки составляют существенную часть нашей реализации, но они не являются частью нашего интерфейса.

Добавив функцию-член `name`, мы можем теперь написать функцию `compare`.

```
bool compare(const Student_info& x, const Student_info& y)
{
    return x.name() < y.name();
}
```


Эта функция во многом напоминает версию из раздела 4.2.2. Единственное различие состоит в способе получения имени студента. В исходной версии мы могли получить прямой доступ к нужному члену данных; здесь же мы должны вызвать для этого функцию `name`, которая возвратит имя студента. Поскольку функция `compare` является частью нашего интерфейса, мы должны включить ее объявление в тот же заголовок, который определяет тип `Student_info`, а это определение функций — в соответствующий исходный файл, содержащий определения функций-членов.

9.3.2. Тестирование на пустоту

Скрыв наши члены данных и предоставив соответствующую функцию доступа, нам осталось решить только одну проблему: все-таки еще существует причина, которая может разбудить у нашего пользователя желание взглянуть на данные объекта. Например, посмотрим, что случится, если мы вызовем функцию `grade` для объекта, для которого еще не вызывалась функция `read`.

```
student_info s;  
cout << s.grade() << endl; // Исключение: объект s  
                           // не имеет данных.
```

Поскольку мы еще не вызывали функцию `read`, которая могла бы наделить объект `s` хоть какими-то значениями, член `homework` объекта `s` будет пустым и обращение к функции `grade` сгенерирует исключение. Хотя наши пользователи могут перехватить это исключение, они не в состоянии обнаружить проблему заранее, чтобы можно было вообще избежать опасного вызова функции `grade`.

Используя исходную структуру `Student_info` из главы 4, пользователи могли бы протестировать член `homework` и узнать, будет ли обращение к функции `grade` успешным. Если вектор `homework` окажется пустым, они не станут вызывать функцию `grade`. Да, этот подход работал, но какой ценой — пользователи должны были знать все детали структуры объекта, поскольку только это позволяло выполнить тестирование. Мы можем улучшить ситуацию, предложив тот же тест в более абстрактной форме.

```
class Student_info {  
public:  
    bool valid() const { return !homework.empty(); }  
    // Все остальное остается в силе.  
};
```

Функция `valid` сообщит пользователю, содержит ли объект достоверные данные: значение `true` будет означать, что студент выполнил хотя бы одно домашнее задание и, следовательно, имеет право на вычисление итоговой оценки. Наши пользователи могут вызвать функцию `valid`, чтобы узнать, будут ли успешными последующие операции. Например, прежде чем вызывать функцию `grade`, пользователь может проверить, с достоверным ли объектом он имеет дело, избегая таким образом потенциально возможной исключительной ситуации.

9.4. Класс `Student_info`

На данный момент мы удовлетворили большую часть требований, которые можно было бы предъявить к исходной структуре `Student_info`, поэтому имеет смысл представить полную картину наших нововведений.

```
class Student_info {  
public:
```

```

std::string name() const { return n; }
bool valid() const { return !homework.empty(); }
// как определено в разделе 9.2.1, вносим изменения для
// считывания имени в член n вместо бывшего члена name.
std::istream& read(std::istream&);
double grade() const; // как определено в разделе 9.2.1.
private:
    std::string n;
    double midterm, final;
    std::vector<double> homework;
};
bool compare(const Student_info&, const Student_info&);

```

Пользователи могут изменить состояние `Student_info`-объекта только посредством вызова функции-члена `read`. Они не могут проникнуть внутрь объекта и напрямую изменить какие-либо члены данных. Мы предоставили операции, позволяющие работать с объектом без знаний деталей нашей реализации. И потом, наконец-то, все операции, которые можно выполнять с объектом типа `Student_info`, логически собраны воедино.

9.5. Конструкторы

Несмотря на то что наш класс вполне завершен и применим в таком виде, существует еще одна деталь, о которой следовало бы позаботиться: ведь мы ничего не сказали о том, что происходит в момент создания объектов.

Мы знаем, что библиотека гарантирует, что при создании объект библиотечного класса начинает свою “жизнь” с определенного значения. Например, при определении `string`- или `vector`-объекта без начального значения мы автоматически получаем пустой `string`- или `vector`-объект. При этом классы `string` и `vector` также позволяют нам придавать новому объекту некоторое начальное значение, задающее, например, некоторый заполняющий символ или размер.

Конструкторы (constructors) — это специальные функции-члены, которые определяют, как инициализируются объекты. Не существует способа вызвать конструктор в явном виде. Вместо этого при создании объекта типа класса автоматически (в качестве побочного эффекта) вызывается соответствующий конструктор.

Если мы сами не определим ни одного конструктора, компилятор синтезирует его за нас. Мы еще поговорим о синтезируемых операциях в разделе 11.3.5. А пока нам необходимо знать, что все-таки произойдет, если мы не определим ни одного конструктора. В этом случае наши пользователи смогут определять `Student_info`-объекты, но не смогут инициализировать их явным образом, если не считать возможность копирования других `Student_info`-объектов.

Синтезированный конструктор будет инициализировать члены данных с использованием значений, которые зависят от того, как создается объект. Если объект представляет собой локальную переменную, то члены данных будут инициализированы значениями по умолчанию (см. раздел 3.1). Если объект используется для инициализации элемента контейнера как побочный эффект операции добавления нового элемента к отображению либо как элемент контейнера с заданным размером, то члены данных этого объекта будут инициализированы определенными значениями (см. раздел 7.2). Эти правила несколько сложны, но суть их в следующем.

- Если объект имеет тип класса, который определяет один или несколько конструкторов, то соответствующий конструктор полностью управляет инициализацией объектов этого класса.

- Если объект имеет встроенный тип, то при инициализации значением он будет установлен равным нулю, а при инициализации по умолчанию будет иметь неопределенное значение.
- Во всех других случаях объект может иметь только тип класса, в котором не определен ни один конструктор. И тогда каждый член данных этого объекта инициализируется либо определенным значением, либо по умолчанию. Этот процесс инициализации будет рекурсивным, если какой-либо член данных имеет тип класса с собственным конструктором.

В соответствии с этими правилами наш класс `Student_info` попадает в третью категорию: мы использовали тип класса, но не указали в явном виде, как создавать `Student_info`-объекты. Поэтому, если мы определяем локальную переменную типа `Student_info`, то члены `n` и `homework` будут автоматически инициализированы пустыми `string`- и `vector`-объектами, поскольку они являются объектами классов с заданными конструкторами. В отличие от них, члены `midterm` и `final` будут инициализированы по умолчанию неопределенными значениями, т.е. они будут содержать “мусор”, который оказался в памяти компьютера в момент создания этого объекта.

При заданных нами довольно простых операциях такое поведение может не иметь опасных последствий: ни одна из наших операций не использует значений членов `midterm` или `final` без предварительной инициализации объекта посредством вызова функции-члена `read`, которая присваивает этим членам значения, считанные из входного потока. Но все-таки хорошим стилем программирования считается предоставление гарантии, что каждый член данных имеет некоторое разумное значение в любой момент использования данного объекта. Например, не исключено, что позже мы сами (или другой программист, которому выпадет “честь” поддерживать и развивать наш код) решим добавить операции, опрашивающие эти члены данных. И если мы не обеспечим инициализацию их в конструкторе, эти новые операции могут ожидать печальное будущее. Более того, как мы увидим в разделе 11.3.5, несмотря на то что мы не используем в явном виде члены `midterm` или `final`, этот “пробел” могут восполнить синтезированные операции, выполняемые с классами. А поскольку любое использование неопределенных значений, за исключением перезаписи какими-нибудь определенными значениями, недопустимо (см. раздел 3.1), то, строго говоря, мы должны эти члены все-таки инициализировать сами.

На практике стоит определить два конструктора: первый без аргументов будет создавать пустой `Student_info`-объект, а второй, принимая ссылку на входной поток, должен инициализировать объект посредством считывания из этого потока записи с данными о студенте. Такая стратегия позволит нашим пользователям написать код, подобный следующему.

```
Student_info s;           // Пустой Student_info-объект.
Student_info s2(cin);     // Инициализируем объект s2 путем
                          // считывания данных из потока cin.
```

Конструкторы отличаются от других функций-членов следующим: их имена совпадают с именем самого класса и для них не нужно указывать тип возвращаемого значения. Конструкторы подобны иным функциям в том, что мы можем определять несколько версий конструкторов, которые отличаются один от другого количеством или типом своих аргументов. Итак, мы просто обязаны обновить наш класс путем добавления в него двух конструкторов.

```

class Student_info {
public:
    Student_info();           // конструктор, создающий пустой
                             // Student_info-объект.
    Student_info(std::istream&); // конструктор, создающий
                                // объект посредством
                                // считывания данных из потока.
    // Все остальное остается в силе.
};

```

9.5.1. Конструктор по умолчанию

Конструктор, который не принимает аргументов, называют *конструктором по умолчанию* (default constructor). Его вызов обычно гарантирует, что члены данных объекта будут инициализированы надлежащим образом. В случае с объектами класса `Student_info` инициализация их данных должна означать, что мы еще не прочитали ни одной записи, т.е. мы хотим, чтобы член `homework` был пустым вектором, член `n` представлял собой пустой `string`-объект, а члены `midterm` и `final` были равны нулю.

```

Student_info::Student_info(): midterm(0), final(0) { }

```

В определении конструктора используется новый для вас синтаксис. Между символом ":" и открывающей фигурной скобкой ({} находится ряд *инициализаторов конструктора* (constructor initializers), которые велят компилятору инициализировать заданные члены значениями, указанными в соответствующих круглых скобках. В частности, действия этого конструктора по умолчанию заключаются в явной установке членов `midterm` и `final` равными 0. Больше никаких других "очевидных" действий он не совершает: тело этой функции попросту отсутствует, поскольку между фигурными скобками зияет пустота. Как мы вскоре увидим, члены `n` и `homework` инициализируются неявным образом.

Чтобы понять, как создаются и инициализируются объекты, очень важно уяснить работу инициализаторов конструктора. При создании нового объекта класса последовательно выполняются следующие действия.

1. C++-среда выделяет память для хранения объекта.
2. C++-среда инициализирует объект в соответствии со списком инициализаторов конструктора.
3. C++-среда выполняет тело конструктора.

C++-среда инициализирует каждый член данных каждого объекта, независимо от того, упоминаются ли в списке инициализаторов конструктора эти члены. Впоследствии тело конструктора может изменить эти начальные значения, но инициализация всегда выполняется до начала выполнения тела конструктора. Обычно лучше явно наделить каждый член некоторым начальным значением, вместо того чтобы присваивать ему значение в теле конструктора. Инициализируя член (вместо присваивания), мы тем самым избегаем повторного выполнения одних и тех же действий.

Стоит повторить, что смысл существования конструкторов состоит в гарантии того, что в результате создания объектов их члены данных перейдут в некоторое имеющее определенный смысл состояние. В общем случае сказанное выше означает, что каждый конструктор должен инициализировать каждый член данных. Необходимость придавать членам значения особенно важна для членов встроенного типа. Если конструктор не инициализирует такие члены, объекты, объявленные в локальной области видимости, будут инициализированы случайным "мусором".

Теперь, вероятно, стало понятно, почему мы заявили, что конструктор по умолчанию класса `Student_info` не выполнил никаких других “очевидных” действий. Хотя мы явно инициализировали только члены `midterm` и `final`, другие члены данных инициализируются неявным образом. А именно член `n` инициализируется конструктором по умолчанию класса `string`, а член `homework` — конструктором по умолчанию класса `vector`.

9.5.2. Конструкторы с аргументами

С нашим вторым конструктором класса `Student_info` несколько легче разобраться.

```
Student_info::Student_info(istream& is) { read(is); }
```

Этот конструктор всю работу перекладывает “на плечи” функции `read`. Здесь нет ни одного явного инициализатора, поэтому члены `homework` и `n` будут инициализированы конструкторами по умолчанию классов `vector` и `string` соответственно. Члены `midterm` и `final` будут иметь явно обозначенные начальные значения только в том случае, если объект инициализируется значением. Но отсутствие инициализации как таковой здесь не имеет никаких последствий, поскольку функция `read` немедленно присвоит этим переменным новые (считанные из потока) значения.

9.6. Использование класса `Student_info`

Наш обновленный класс `Student_info` теперь существенно отличается от исходной структуры `Student_info` из главы 4. Неудивительно, что и использование этого класса будет отличаться от использования исходной структуры. Прежде всего, наша цель — запретить пользователям изменять значения наших данных, чего мы добились, сделав их закрытыми с помощью спецификатора `private`. Взамен мы предоставили пользователям возможность писать программы, опираясь на интерфейс, предложенный нашим классом. В качестве примера мы можем так переписать нашу исходную функцию `main` из раздела 4.5, которая выводила итоговые оценки студентов в виде отформатированного отчета, чтобы новая функция `main` использовала новую версию класса `Student_info`.

```
int main()
{
    vector<Student_info> students;
    Student_info record;
    string::size_type maxlen = 0;

    // Считываем и сохраняем данные.
    while (record.read(cin)) { // Изменено.
        maxlen = max(maxlen, record.name().size()); // Изменено.
        students.push_back(record);
    }

    // Выстраиваем записи в алфавитном порядке.
    sort(students.begin(), students.end(), compare);

    // Выводим фамилии и оценки.
    for (vector<Student_info>::size_type i = 0;
         i != students.size(); ++i) {
        cout << students[i].name() // Изменено.
              << string(maxlen + 1 - students[i].name.size(), ' ');
        try {
            double final_grade = students[i].grade(); // Изменено.
            streamsize prec = cout.precision();
            cout << setprecision(3) << final_grade
```

```

        << setprecision(prec) << endl;
    } catch (domain_error e) {
        cout << e.what() << endl;
    }
}
return 0; *
}

```

В этой версии изменения связаны с вызовами функций `name`, `read` и `grade`. Так, например, первый `while`-цикл вместо заголовка

```
while (read(cin, record)) {
```

имеет следующий заголовок.

```
while (record.read(cin)) {
```

Наша новая версия сейчас вызывает функцию-член `read` объекта `record`. А прежняя версия вызывала глобальную функцию `read`, передавая ей объект `record` в качестве явно заданного параметра. Оба вызова имеют одинаковый результат: объекту `record` будут присвоены значения, считанные из входного потока `cin`.

9.7. Резюме

Типы, определенные пользователем. Пользователь может определить собственный тип либо как `struct`, либо как `class`. Единственное различие между ними состоит в действующей по умолчанию защите, которая применяется к членам, определенным до первой метки защиты: члены, определенные после имени типа `struct`, являются открытыми (`public`), а после имени типа `class` — закрытыми (`private`).

Метки защиты управляют доступом к членам типа класса: `public`-члены — общедоступны, а к `private`-членам доступ могут получить только члены того же класса. Метки защиты внутри класса могут располагаться в любом порядке и в любом количестве.

Функции-члены. Типы, наряду с членами данных, могут определять и функции-члены. Функции-члены явно вызываются “от имени” заданного объекта. Ссылки на члены данных или функции внутри функции-члена неявно связаны с этим объектом.

Функции-члены могут быть определены внутри определения класса или вне его. Определение функции-члена внутри класса “подталкивает” C++-среду к расширению обращений к этой функции, т.е. к замене всех ее вызовов ее телом прямо “по месту” вызова, устраняя тем самым расходы системных ресурсов на обработку вызовов. При определении вне класса имя функции должно быть составным, отражая область видимости класса: `имя_класса::имя_функции-члена`. Такое составное имя означает, что функция с именем `имя_функции-члена` принадлежит классу с именем `имя_класса`.

Функции-члены можно определить как константные, вставив ключевое слово `const` после списка параметров. Такие функции-члены не могут изменить состояние объекта, для которого они вызываются. Для `const`-объектов можно вызывать только константные функции-члены.

Конструкторы — это специальные функции-члены, которые определяют, как инициализируются объекты данного типа. Конструкторы имеют имя, совпадающее с именем класса, и не возвращают никакого значения. Класс может определить несколько конструкторов, отличающихся количеством или типами их аргументов. Хорошим стилем программирования считается для каждого конструктора гарантировать, что на выходе из конструктора каждый член данных будет иметь некоторое разумное значение.

Список инициализаторов конструктора — это список разделенных запятыми парных элементов *Имя_члена(Значение)*. Каждый член с именем *Имя_члена* инициализируется заданным значением элемента *Значение*. Члены данных, которые не инициализируются в явном виде, инициализируются неявно.

Порядок инициализации членов данных определяется порядком их объявления в классе, поэтому нужно внимательно отнестись к тому, когда один член класса используется для инициализации другого. Безопаснее, конечно, было бы избегать таких взаимозависимостей, присваивая значения таким членам внутри тела конструктора, а не инициализируя их в инициализаторе конструктора.

Упражнения

- 9.0. Скомпилируйте, выполните и протестируйте программы, приведенные в этой главе.
- 9.1. Предложите новое определение класса `Student_info`, в котором бы итоговая оценка вычислялась при считывании записи с данными о студенте, а результат хранился в объекте. Переделайте функцию `grade`, чтобы она использовала это вычисленное заранее значение.
- 9.2. Если мы определим функцию `name` как обычную `non-const`-функцию-член, то какие еще функции в нашей системе должны измениться и почему?
- 9.3. Наша функция `grade` была написана с учетом возможности генерирования исключения, если пользователь попытается вычислить итоговую оценку для объекта типа `Student_info`, содержащего еще не прочитанные значения. Предполагается, что это исключение должны перехватывать пользователи. Напишите программу, которая генерирует исключение, но не перехватывает его. Напишите также программу с перехватом исключения.
- 9.4. Переделайте программу из предыдущего упражнения так, чтобы она использовала функцию `valid`, позволяющую совершенно избежать генерирования исключения.
- 9.5. Напишите класс и соответствующие функции для генерирования оценок для студентов, которые получают за прослушивание курса зачет (оценка типа `зачет/незачет`). Предположите, что учитываются только оценки, полученные в середине и конце семестра, и что студентам зачитывается курс, если их средний балл выше 60. В отчете фамилии студентов расположите в алфавитном порядке, а в качестве оценки используйте буквы `З` (зачет) или `Н` (незачет).
- 9.6. Переделайте программу из предыдущего упражнения, чтобы в отчете сначала указывались фамилии студентов, получивших за прослушанный курс “зачет”, а за ними — фамилии студентов, проваливших курс.
- 9.7. Функция `read_hw` из раздела 4.1.3 решает общую задачу (считывает последовательность значений в вектор), несмотря на то что она (если судить по ее имени), казалось бы, должна быть частью реализации типа `Student_info`. Конечно, мы могли бы изменить ее имя, а не объединять с остальным кодом типа `Student_info`, чтобы было ясно, что она не предназначена для открытого доступа. Как вы поступили бы здесь?

УПРАВЛЕНИЕ ПАМЯТЬЮ И ИСПОЛЬЗОВАНИЕ СТРУКТУР ДАННЫХ НИЗКОГО УРОВНЯ

До сих пор мы сохраняли данные либо в переменных, либо в таких контейнерах, как вектор (vector), которые “родом” из стандартной библиотеки. Средства стандартной библиотеки обычно отличаются большей гибкостью и простотой применения по сравнению со средствами, которые являются частью базового языка.

Узнав, как использовать библиотеку, вам, вероятно, будет интересно понять, как она работает. Ключ к пониманию лежит в использовании некоторых инструментов и методов программирования базового языка, которые оказываются весьма полезными и в других контекстах. В основу работы стандартной библиотеки положены идеи, которые связаны с использованием термина *низкоуровневые средства* (low-level) и описанием работы аппаратуры типичного компьютера. Поэтому эти идеи труднее и опаснее использовать, чем аналогичные идеи в стандартной библиотеке, но при этом они могут быть более эффективными (естественно, при условии их глубокого понимания). Поскольку ни одна библиотека не в состоянии решить всех проблем, во многих C++-программах время от времени используются низкоуровневые методы.

Стиль изложения материала в этой главе несколько отличается от обычного стиля представления проблем с последующим их решением, поскольку инструменты, о которых пойдет речь, работают на достаточно низком уровне, и поэтому какой-либо один инструмент трудно использовать в одиночку для решения задач, представляющих интерес для читателя. Мы начнем с представления двух связанных понятий: массивы и указатели, а затем покажем, как они сочетаются с new- и delete-выражениями, благодаря которым существует такое понятие, как *распределение динамической памяти*. Используя операции new и delete, программисты могут управлять памятью более прямым способом, чем если бы они полагались на автоматические средства управления памятью, предлагаемые такими библиотечными классами, как vector и list.

Разобравшись в работе массивов и указателей, в следующей главе (главе II) мы рассмотрим, как библиотека использует эти средства для реализации своих контейнеров.

10.1. Указатели и массивы

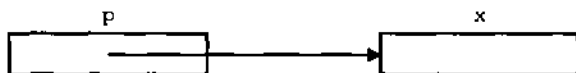
Массив — это разновидность контейнера, похожая на вектор, но менее мощная. Указатель — это итератор произвольного доступа, который используется для доступа к элементам массива, а также имеет другое применение. Указатели и массивы относятся к самым примитивным структурам данных в C и C++. В сущности, они неотделимы друг от друга в

том смысле, что невозможно сделать что-либо полезное с массивом, не используя указатели, а роль указателей существенно возрастает при наличии массивов.

Поскольку эти два понятия тесно взаимосвязаны, то, когда придет время, вы сможете по достоинству оценить их совместную работу. Однако проще разъяснить указатели, не рассматривая массивы, чем попытаться описать массивы без рассмотрения указателей, поэтому сначала приступим к указателям.

10.1.1. Указатели

Указатель (pointer) — это значение, которое представляет адрес объекта. Каждый отдельный объект имеет уникальный адрес, который обозначает область памяти компьютера, в которой содержится этот объект. Если вы можете получить доступ к объекту, значит, вы можете получить его адрес, и наоборот. Например, если *x* — это объект, то *&x* — адрес этого объекта, и если *p* — адрес некоторого объекта, то **p* — это сам объект. Символ “&” в выражении *&x* представляет собой *оператор взятия адреса* (address operator), который не следует путать с использованием символа “&” для определения ссылочных типов (см. раздел 4.1.2). Символ “*” — *оператор разыменования* (dereference operator), работа которого аналогична его применению к любому другому итератору (см. раздел 5.2.2). Если *p* содержит адрес объекта *x*, мы также говорим, что *p* — указатель, который указывает на *x*. Обычно для такого представления используется следующее схематическое изображение.



Как и при других встроенных типах, локальная переменная-указатель не имеет никакого смыслового значения до тех пор, пока мы не позаботимся об этом. Программисты часто для инициализации указателей используют значение 0, поскольку в результате преобразования числа 0 в указатель генерируется значение, гарантированно отличное от указателя на какой бы то ни было реальный объект. Более того, константа 0 — это единственное целое значение, которое можно преобразовать в тип указателя, а результат этого преобразования, именуемый *пустым, или нулевым указателем* (null pointer), часто используется в операциях сравнения.

Как и все C++-значения, указатели имеют типы. Адрес объекта типа *T* имеет тип “указатель на тип *T*”, в определениях и подобных контекстах обозначаемый как *T**.

Предположим, что *x* — это объект типа *int*, определенный следующим образом.

```
int x;
```

В то же время мы хотим определить переменную *p*, тип которой позволит ей содержать адрес объекта *x*. Это реализуется посредством заявления о том, что типом переменной *p* является “указатель на *int*”, причем это заявление неявно выражается в виде утверждения, что объект **p* имеет тип *int*.

```
int *p;           // Объект *p имеет тип int.
```

Здесь **p* — *описатель* (declarator), который является частью определения единой переменной. Несмотря на то что символы “*” и “p” — это части одного описателя, большинство C++-программистов записывают это определение следующим образом.

```
int* p;           // Объект p имеет тип int*.
```

Тем самым обращается внимание на то, что объект `p` имеет особый тип (т.е. `int*`). Эти две формы объявления эквивалентны, поскольку пробелы возле символа `**` не имеют значения. Тем не менее вторая запись таит в себе ловушку, которая столь коварна, что заслуживает особого внимания.

```
int* p, q;           // что означает это определение?
```

Эта запись определяет `p` как объект типа “указатель на `int`”, а `q` — как объект типа `int`. Этот пример проще понять, если посмотреть на него под таким углом зрения.

```
int *p, q;           // Объекты *p и q имеют тип int.
```

Или, что то же самое, под таким углом зрения.

```
int (*p), q;         // объекты (*p) и q имеют тип int.
```

Конечно, лучше всего сделать свои намерения совершенно прозрачными. Для этого достаточно написать следующие отдельные объявления.

```
int* p;               // объект *p имеет тип int.  
int q;                // объект q имеет тип int.
```

Теперь приобретенные знания позволяют написать нам простую программу с использованием указателей.

```
int main()  
{  
    int x = 5;  
  
    // объект p указывает на объект x.  
    int* p = &x;  
    cout << "x = " << x << endl;  
  
    // Изменяем значение x, используя объект p.  
    *p = 6;  
    cout << "x = " << x << endl;  
    return 0;  
}
```

В результате выполнения эта программа выведет следующее.

```
x = 5  
x = 6
```

Сразу после того, как мы определили указатель `p`, состояние наших переменных стало следующим.



Вас ведь не удивляет, что после выполнения первой инструкции вывода значение переменной `x` равно 5. Следующая инструкция (`*p = 6`) изменяет значение `x`, устанавливая его равным 6. Вспомните, если указатель `p` содержит адрес переменной `x`, то `*p` и `x` — два различных способа ссылки на один и тот же объект. Следовательно, после выполнения второй инструкции вывода значение `x` совершенно справедливо будет равно 6.

Вполне уместно представлять указатель на некоторый объект как итератор, который ссылается на единственный элемент “контейнера”, содержащего этот объект (и ничего больше!).

10.1.2. Указатели на функции

В разделе 6.2.2 мы рассматривали программу, в которой некоторая функция передавалась в качестве аргумента другой функции, причем в, казалось бы, безобидной записи вызова функции с передачей ей аргумента-функции таилось значительно больше, чем видели наши глаза. Дело в том, что функции не являются объектами и не существует способа скопировать их или присвоить им значение или напрямую передать их как аргументы. В частности, программа не в состоянии создать или модифицировать функцию — это “по силам” лишь компилятору. Все, что программа может сделать с функцией, — это вызвать ее или принять ее адрес.

Тем не менее мы можем вызвать функцию, используя другую функцию в качестве аргумента, что мы и делали, передавая функцию `median_analysis` как аргумент функции `write_analysis` в разделе 6.2.2. Происходившие при этом события можно описать следующим образом: компилятор так воспринимает подобные вызовы, как будто бы вместо самих функций мы использовали указатели на функции. Указатели на функции ведут себя абсолютно так же, как любые другие указатели. Однако после разыменования такого указателя (и получения, естественно, функции) можно лишь вызвать эту функцию или снова получить ее адрес.

Описатели, используемые для указателей на функции, внешне похожи на другие описатели. Например, если объявление

```
int *p;
```

говорит о том, что объект `*p` имеет тип `int`, то, предполагая, что `fp` — указатель на функцию, мы можем записать следующее.

```
int (*fp)(int);
```

Эта запись подразумевает, что если мы разыменуем указатель `fp` и вызовем результат разыменования с некоторым `int`-аргументом, то результат этого вызова будет иметь тип `int`. Итак, смысл этой записи таков: `fp` — это указатель на функцию, которая принимает `int`-аргумент и возвращает `int`-результат.

Поскольку все, что мы можем сделать с функцией, ограничивается взятием ее адреса или ее вызовом, предполагается, что любое использование любой функции, отличное от ее вызова, представляет собой взятие ее адреса, даже без явного использования оператора взятия адреса (`&`). Аналогично можно вызвать указатель на функцию без явного разыменования этого указателя. Например, у нас есть функция, определение которой выглядит следующим образом.

```
int next(int n)
{
    return n + 1;
}
```

Мы можем сделать так, чтобы объект `fp` указывал на функцию `next`, записав любую из следующих двух инструкций.

```
// Эти две инструкции эквивалентны.
fp = &next;
fp = next;
```

Аналогично, если у нас есть `int`-переменная с именем `i`, мы можем использовать указатель `fp`, чтобы вызвать функцию `next`, и, таким образом, инкрементировать переменную `i`, написав любую из следующих двух инструкций.

```
// Эти две инструкции эквивалентны.
i = (*fp)(i);
i = fp(i);
```

Наконец, если написать функцию, по внешнему виду которой можно сказать, что в качестве параметра она принимает другую функцию, то компилятор интерпретирует этот параметр как указатель на функцию. Так, например, в функции `write_analysis` (из раздела 6.2.2) параметр, записанный как

```
double analysis(const vector<Student_info>&),
```

можно без потери эквивалентности записать следующим образом.

```
double (*analysis)(const vector<Student_info>&)
```

Однако такой автоматический перевод неприменим к значениям, возвращаемым из функций. Если бы мы хотели написать функцию, которая возвращает указатель на функцию, того же типа, что и параметр, передаваемый функции `write_analysis`, то нам бы пришлось явно указать, что функция возвращает указатель. Это можно сделать, например, с помощью спецификатора `typedef`, определив, скажем, `analysis_fp` как имя типа соответствующего указателя.

```
typedef double (*analysis_fp)(const vector<Student_info>&);
```

Затем мы можем использовать этот тип для объявления нашей функции.

```
// функция get_analysis_ptr возвращает указатель
// на функцию analysis_fp.
analysis_fp get_analysis_ptr();
```

Альтернативный вариант

```
double (*get_analysis_ptr())(const vector<Student_info>&);
```

менее понятен. По сути, мы утверждаем, что если вызвать функцию `get_analysis_ptr()` и разыменовать результат, то вы получите функцию, которая принимает `const`-вектор типа `vector<Student_info>&` и возвращает `double`-значение. К счастью, функции, которые возвращают указатели на функции, используются довольно редко! Мы же больше не используем подобный синтаксис в этой книге, если не считать более подробных разъяснений в разделе A.1. Указатели на функции чаще всего используются как аргументы, передаваемые другим функциям. В качестве примера приведем простую реализацию библиотечной функции `find_if`.

```
template<class In, class Pred>
In find_if(In begin, In end, Pred f)
{
    while (begin != end && !f(*begin))
        ++begin;
    return begin;
}
```

В этом примере тип `Pred` может потенциально быть любым типом, если выражение `f(*begin)` имеет какое-то разумное значение. Предположим, у нас есть предикативная функция

```
bool is_negative(int n)
{
    return n < 0;
}
```

и мы используем функцию `find_if` для отыскания первого отрицательного элемента в векторе типа `vector<int>` с именем `v`.

```
vector<int>::iterator i = find_if(v.begin(),  
                                v.end(), is_negative);
```

Мы можем написать `is_negative` вместо `&is_negative` только потому, что имя функции автоматически преобразуется в указатель на эту функцию. Аналогично реализация функции `find_if` позволяет вызвать `f(*beg)` вместо `(*f)(*beg)` только потому, что при вызове указателя на функцию автоматически вызывается функция, на которую он указывает.

10.1.3. Массивы

Массив — это разновидность контейнера, которая является частью базового языка, а не стандартной библиотеки. Каждый массив содержит последовательность, состоящую из одного или нескольких объектов одинакового типа. Количество элементов в массиве должно быть известно во время компиляции, а это означает, что массивы не могут увеличиваться или уменьшаться динамически подобно тому, как это происходит с библиотечными контейнерами.

Поскольку массивы не относятся к типу класса, они не имеют членов. В частности, у них нет члена `size_type`, позволяющего применять соответствующий тип для размера массива. Вместо этого можно использовать более общий тип `size_t`, который определен в заголовке `<cstddef>`. Каждая C++-среда определяет `size_t` как подходящий тип `unsigned`, который в состоянии обеспечить хранение достаточно больших значений, соответствующих размерам любого объекта. Таким образом, мы можем (и должны) использовать тип `size_t` для описания размера любого массива, подобно тому, как мы применяем тип `size_type` для описания размера какого-либо библиотечного контейнера.

Например, программа, в которой используется трехмерная геометрия, может представить точку следующим образом.

```
double coords[3];
```

Конечно, эта жесткость обусловлена тем, что существует слишком малая вероятность того, что количество измерений в физическом пространстве изменится в скором времени. Более опытный программист мог бы представить точку несколько иначе.

```
const size_t NDim = 3;  
double coords[NDim];
```

Здесь используется преимущество того факта, что значение переменной `NDim` станет известно во время компиляции (поскольку мы имеем дело с `const`-значением типа `size_t`, которое инициализируется константой). При использовании переменной `NDim` (вместо числа 3) число 3, которое представляет количество измерений, отличается от числа 3, представляющего, скажем, количество сторон в треугольнике.

Независимо от того, как мы определим массив, всегда существует следующее фундаментальное соотношение между массивами и указателями: *всякий раз когда мы используем имя массива как значение, это имя представляет собой указатель на начальный элемент массива (первое фундаментальное свойство массивов)*. Выше мы определили `coords` как массив, поэтому при использовании `coords` в качестве значения получаем адрес начального элемента массива. Как и в случае с любым другим указателем, мы

можем разыменовать его значение с помощью оператора “*”, чтобы получить объект, на который он указывает. Таким образом, при выполнении инструкции

```
*coords = 1.5;
```

начальный элемент массива `coords` будет установлен равным значению 1.5.

10.1.4. Арифметические операции с указателями

Теперь мы знаем, как определить массивы и как получить адрес начального элемента массива. А как насчет других элементов? Вспомните, что в разделе 10.1 упоминалось о том, что указатель — это разновидность итератора. Точнее, указатель — это итератор произвольного доступа. Из этого мы извлекаем *второе фундаментальное свойство массивов*: если p указывает на m -й элемент массива, то выражение $p+n$ указывает на $(m+n)$ -й элемент массива, а выражение $(p-n)$ — на $(m-n)$ -й элемент (конечно, в предположении, что эти элементы существуют).

Отметим, что начальный элемент любого массива (а значит, и массива `coords`) в C++ имеет номер 0, поэтому, продолжая рассматривать наш пример, выражение `coords+1` представляет собой адрес элемента массива `coords` с номером 1 (т.е. адрес элемента, следующего за начальным), а выражение `coords+2` дает адрес элемента с номером 2, который является последним элементом, поскольку мы определили `coords` как массив, состоящий из трех элементов.

А что можно сказать о выражении `coords+3`? Это значение представляет адрес в массиве `coords`, где мог бы находиться элемент с номером 3, если бы такой элемент существовал, но он не существует.

Тем не менее, выражение `coords+3` представляет собой действительный указатель, даже несмотря на то что он не указывает ни на какой элемент. По аналогии с итераторами классов `vector` и `string`, сложение значения n с адресом начального элемента n -элементного массива генерирует адрес, о котором можно сказать, что нет никакой гарантии, что он будет адресом какого-то объекта, но этот адрес все же можно использовать для операций сравнения. Более того, правила, которые устанавливают связь между выражениями p , $p+n$ и $p-n$, действительны даже в том случае, если одно или несколько из этих выражений генерируют значение, которое относится к первой позиции (именно к первой), расположенной за концом массива.

Так, например, выполнив следующие инструкции, мы можем скопировать содержимое массива `coords` в вектор.

```
vector<double> v;  
copy(coords, coords + NDim, back_inserter(v));
```

Здесь, как и выше, `NDim` — это просто “необычный” способ “произношения” числа 3. В этом примере выражение `coords + NDim` не указывает на какой бы то ни было элемент, но является действительным итератором “конца” (off-the-end iterator), и в передаче его функции `copy` в качестве второго аргумента нет никакого противоречия.

Поскольку у нас есть возможность построить вектор на основе двух итераторов, в качестве второго примера рассмотрим создание вектора `v` посредством прямого копирования элементов из массива `coords`. Для этого достаточно написать следующую инструкцию.

```
vector<double> v(coords, coords + NDim);
```

Другими словами, предположим, что `a` — это n -элементный массив, `v` — вектор и мы хотим применить алгоритмы стандартной библиотеки к элементам массива `a`. Тогда везде, где мы можем применить значения `v.begin()` и `v.end()`, чтобы предоста-

вить алгоритмам стандартной библиотеки доступ к элементам вектора v , нужно использовать в качестве аргументов выражения a и $a + n$ в случаях, когда мы хотим применить эти алгоритмы к элементам массива a .

Если p и q — указатели на элементы одного и того же массива, то выражение $p - q$ возвращает целочисленное значение, которое представляет расстояние (“измеряемое” в элементах) между p и q . Точнее, выражение $p - q$ определяется таким образом, чтобы значение $(p - q) + q$ было равно p . Поскольку значение $p - q$ может быть отрицательным, оно имеет целочисленный тип со знаком. Каким именно будет этот тип (`int` или `long`) — зависит от конкретной C++-среды, поэтому библиотека для представления соответствующего типа предоставляет синоним, именуемый `ptrdiff_t`. Как и `size_t`, тип `ptrdiff_t` определен в заголовке `<cstdlib>`.

Как отмечалось в разделе 8.2.7, не существует гарантированного способа вычислить итератор, который указывает на позицию перед началом контейнера. Аналогично недопустимо вычислять адрес “элемента”, расположенного перед началом любого массива. Другими словами, если a — n -элементный массив, то $a+i$ является допустимым указателем тогда и только тогда, когда $0 \leq i \leq n$, а указатель $a+i$ указывает на элемент массива a тогда и только тогда, когда $0 \leq i < n$ (исключая случай, когда i и n равны).

10.1.5. Индексирование

В разделе 10.1 утверждалось, что указатели — это итераторы произвольного доступа для массивов. Следовательно, подобно всем итераторам произвольного доступа, они поддерживают индексирование. Так, если p указывает на m -й элемент массива, то $p[n]$ — это $(m+n)$ -й элемент массива (причем не адрес этого элемента, а сам элемент).

Как упоминалось в разделе 10.1.3, имя массива является адресом начального элемента этого массива. Этот факт (а также определение элемента $p[n]$) говорит о том, что если a — массив, то $a[n]$ — n -й элемент этого массива. Говоря более формально, если p — указатель, а n — целочисленное значение, то выражение $p[n]$ эквивалентно выражению $*(p + n)$.

В отличие от большинства языков программирования, в C++ индексирование является не прямым свойством массивов, а дополнением к свойствам имен массивов и указателей и тому факту, что указатели поддерживают операции, определенные для итераторов произвольного доступа.

10.1.6. Инициализация массивов

В отличие от библиотечных контейнеров, у массивов есть одно важное свойство: для присваивания начального значения каждому элементу массива в языке C++ предусмотрен удобный синтаксис. Более того, использование этого синтаксиса часто позволяет избежать явного описания размера массива.

Например, если бы мы писали программу для обработки дат, то нам нужно было бы знать, сколько дней в каждом месяце. Такую информацию можно представить в программе следующим образом.

```
const int month_lengths[] = {  
    31, 28, 31, 30, 31, 30, // пока не будем учитывать  
    31, 31, 30, 31, 30, 31 // високосные годы.  
};
```

Здесь каждому элементу присваивается начальное значение, соответствующее количеству дней в месяце, причем январю соответствует элемент с номером 0, а декабрю — элемент с номером 11, т.е. январь обозначается как месяц с номером 0, а де-

кабрь — как месяц с номером 11. Теперь мы можем использовать элемент `month_lengths[i]` для учета продолжительности месяца с номером `i`.

Обратите внимание на то, что мы не указали в явном виде количество элементов в массиве `month_lengths`. Но благодаря его явной инициализации, компилятор подсчитывает элементы массива за нас. (Эта задача, пожалуй, больше подходит компилятору, чем нам, не правда ли? Мы же в это время займемся более серьезными вещами.)

10.2. Снова о строковых литералах

Наконец, мы узнали достаточно, чтобы понять истинное значение строковых литералов: строковый литерал — это в действительности массив типа `const char`, в котором на один элемент больше, чем символов в литерале. Этим дополнительным символом является нуль-символ (т.е. `'\0'`), который компилятор автоматически присоединяет к остальным символам. Другими словами, при определении массива

```
const char hello[] = { 'h', 'e', 'l', 'l', 'o', '\0' };
```

переменная `hello` будет иметь в точности такое же значение, как и строковый литерал `"hello"`. Однако переменная и строковый литерал — это два отдельных объекта, которые, следовательно, имеют различные адреса.

Компилятор вставляет нуль-символ, чтобы позволить программисту обнаружить конец литерала, заданного лишь адресом его начального символа. Нуль-символ действует как маркер конца, чтобы программист мог понять, где строка оканчивается. В заголовке `<cstring>` определена библиотечная функция `strlen`, которая может подсчитать количество символов в строковом литерале или любом другом массиве символов, оканчивающемся символом конца строки, причем не учитывая этот конечный нуль-символ. Функция `strlen` может быть реализована следующим образом.

```
// пример реализации функции стандартной библиотеки.
size_t strlen(const char* p)
{
    size_t size = 0;
    while (*p++ != '\0')
        ++size;
    return size;
}
```

В разделе 10.1.3 отмечалось, что `size_t` — это целочисленный тип без знака, который подходит для хранения размера любого массива. Функция `strlen` подсчитывает символы в массиве, заданном указателем `p`, не включая в это количество нуль-символ.

Поскольку переменная `hello` имеет то же значение, что и строковый литерал `"hello"`, с помощью следующей инструкции

```
string s(hello);
```

будет определена `string`-переменная с именем `s`, которая содержит копию символов, хранимых в переменной `hello`, подобно тому, как инструкция

```
string s("hello");
```

определяет `string`-переменную `s`, которая содержит копию символов литерала `"hello"`. Более того, поскольку можно построить строку с помощью двух итераторов, мы вправе записать следующее.

```
string s(hello, hello + strlen(hello));
```


Здесь использование имени массива `hello` генерирует указатель на первый начальный символ массива `hello`, а выражение `hello + strlen(hello)` дает нам указатель на символ `'\0'`, который находится в конце массива и является одновременно символом, непосредственно следующим за буквой `"o"`, т.е. первым символом массива `hello`, расположенным за буквой `"o"`. Поскольку указатели являются итераторами, мы можем создать `string`-объект на базе двух указателей, подобно тому, как мы поступили в разделе 6.1.1, создавая новый `string`-объект с помощью двух итераторов. В обоих случаях первый итератор указывает на начальный символ последовательности, которую мы хотим использовать для инициализации создаваемого `string`-объекта, а второй — на символ, непосредственно следующий за последним символом копируемой части объекта.

10.3. Инициализация массивов указателей на символы

В разделе 10.2 отмечалось, что строковый литерал — это просто удобный способ записи адреса начального символа некоторой последовательности, завершающейся нуль-символом. В разделе 10.1.6 мы показали, что можно инициализировать элементы массива (подобно инициализатору конструктора) посредством последовательности соответствующих значений, заключенной в фигурные скобки. Если объединить эти два факта, то окажется, что мы можем инициализировать массив указателей на символы, задав последовательность строковых литералов.

С первого взгляда это утверждение может показаться неясным. Чтобы сделать его более конкретным, предположим, что мы хотим преобразовать баллы, выраженные в числах, в буквенные обозначения в соответствии со следующим правилом.

Если оценка не меньше: 97 94 90 87 84 80 77 74 70 60 0,
то ей соответствует буква: A+ A A- B+ B B- C+ C C- D F.

Вот как выглядит программа, выполняющая это преобразование.

```
string letter_grade(double grade)
{
    // шкала числовых оценок.
    static const double numbers[] = {
        97, 94, 90, 87, 84, 80, 77, 74, 70, 60, 0
    };

    // имена для "буквенных" оценок.
    static const char* const letters[] = {
        "A+", "A", "A-",
        "B+", "B", "B-",
        "C+", "C", "C-",
        "D", "F"
    };

    // Вычисляем количество оценок на основе размера массива
    // и размера одного элемента этого массива.
    static const size_t ngrades = sizeof(numbers)/
                                   sizeof(*numbers);

    // По заданной числовой оценке grade находим и возвращаем
    // соответствующую "буквенную" оценку.
    for (size_t i = 0; i < ngrades; ++i) {
        if (grade >= numbers[i])
```

```

        return letters[i];
    }
    return "?\?\?";
}

```

В определении массива `numbers` используется ключевое слово `static`, с которым мы уже встречались в разделе 6.1.3. В данном контексте оно уведомляет компилятор о необходимости инициализировать массивы `letters` и `numbers` только однажды, т.е. при первом их использовании. Без слова `static` компилятор должен был бы инициализировать массив при каждом вызове этой функции, что замедлило бы выполнение всей программы. Мы уже упоминали о том, что элементы этих массивов являются константными, поскольку у нас нет намерения изменять их, и этот факт позволяет нам организовать их инициализацию только один-единственный раз.

Массив `letters` — это массив константных указателей на тип `const char`. В данном случае каждый элемент этого массива указывает на начальный элемент строкового литерала, соответствующего определенной “буквенной” оценке.

В определении переменной `ngrades` используется новое ключевое слово `sizeof`, благодаря которому можно вычислить количество элементов в массиве `numbers`, не подсчитывая сами элементы. Если `e` — некоторое выражение, то `sizeof(e)` возвращает `size_t`-значение, соответствующее объему памяти, занимаемому объектом `e`. Результат при этом получается без реального вычисления выражения, благодаря тому, что для определения типа выражения не нужно его вычислять, а также потому, что все объекты данного типа занимают одинаковый объем памяти.

Оператор `sizeof` сообщает результат в байтах, которые представляют собой блоки памяти, точный размер которых изменяется при переходе от одной C++-среды к другой. Относительно байтов можно гарантировать лишь то, что один байт содержит по крайней мере восемь битов (разрядов), при этом каждый объект занимает по крайней мере один байт, а объект типа `char` — ровно один байт.

Конечно же, мы хотим определить, сколько элементов хранится в массиве `numbers`, а не сколько байтов он занимает в памяти компьютера. Для этого мы делим размер всего массива на размер одного элемента. Вспомните из раздела 10.1.3, что поскольку `numbers` — массив, то `*numbers` — это элемент этого массива, причем его начальный элемент. Но в данном контексте неважно, какой именно элемент, поскольку все элементы имеют одинаковый размер. Важно то, что выражение `sizeof(*numbers)` возвращает размер одного элемента массива `numbers`, поэтому частное от деления `sizeof(numbers)/sizeof(*numbers)` равно количеству элементов, содержащихся в этом массиве.

Теперь осталось самое простое — определить оценку в буквенном выражении. Для этого последовательно просматриваем элементы массива `numbers` до тех пор, пока не обнаружим, что значение `grade` больше одного из них или равно ему. При отыскании релевантного элемента массива `numbers` мы возвращаем соответствующий элемент массива `letters`. Этот элемент представляет собой указатель, но в разделе 10.2 мы уже видели, что указатель на символ можно преобразовать в `string`-объект.

Если мы не можем отыскать соответствующую “буквенную” оценку, это значит, что наш пользователь предоставил отрицательную оценку в числовом выражении, и тогда мы возвращаем “признак недоумения”. Наличие символов “\” объясняется тем (подробнее можно узнать в разделе A.2.1.4), что C++-программы не должны содержать двух или более последовательно расположенных вопросительных знаков. Поэто-

му для представления в программе трех вопросительных знаков (???) мы должны использовать выражение "?\?\?".

10.4. Аргументы для функции main

Освоив указатели и символьные массивы, можно понять, как передаются аргументы в функцию main. В большинстве операционных систем предусмотрен способ передачи функции main последовательности символьных строк в качестве аргумента, если, конечно, функция main готова принять эту “передачку”. Автор функции main сигнализирует о такой готовности посредством двух параметров: int-параметра и указателя на указатель на char-значение. Подобно любым другим параметрам, эти параметры могут иметь произвольные имена, но программисты часто называют их argc и argv. Значение второго параметра argv представляет собой указатель на начальный элемент массива указателей. Значение первого параметра argc равно количеству указателей в массиве, на начальный элемент которого указывает параметр argv. Начальный элемент этого массива всегда представляет имя, под которым вызывается данная программа, поэтому значение argc всегда не меньше 1. Аргументы, если таковые присутствуют, занимают последовательные элементы этого массива.

В качестве примера эта программа выводит свои аргументы (если они предоставлены), разделенные пробелами.

```
int main(int argc, char** argv)
{
    // Если существуют аргументы, выводим их.
    if (argc > 1) {
        int i; // Объявляем переменную i вне for-цикла,
               // поскольку нам нужно использовать ее
               // значение после окончания цикла.
        for (i = 1; i < argc; ++i) // Выводим все элементы,
                                   // кроме последнего, и пробелы.
            cout << argv[i] << " "; // argv[i] - это объект
                                   // типа char*.
        cout << argv[i] << endl; // Выводим последний элемент,
                                   // но без пробела.
    }
    return 0;
}
```

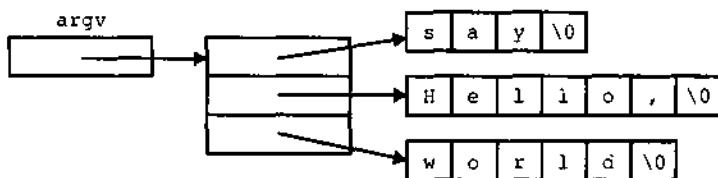
Если скомпилировать эту программу и поместить выполняемый код в файл с именем say, то можно предложить системе выполнить ее следующим образом.

```
say hello, world
```

Тем самым мы заставим эту программу вывести следующее.

```
hello, world
```

В этом случае значение argc будет равно 3, а тремя элементами интересующего нас массива будут указатели на начальные символы массивов, инициализированных значениями say, hello и world соответственно. Мы можем визуализировать значение аргумента argv следующим образом.



10.5. Чтение и запись файлов

Программы, которые приводятся в этой книге, используют для входных и выходных потоков только имена `cin` и `cout`. Однако в более масштабных приложениях зачастую необходимо работать с несколькими файлами, используемыми как для ввода, так и вывода данных. Для этого в C++ предусмотрен широкий набор средств, но мы рассмотрим только некоторые из них.

10.5.1. Стандартный поток ошибок

Зачастую имеет смысл позаботиться о том, чтобы программа могла комментировать свои действия, причем способом, отличным от обычного вывода данных. Благодаря возможности генерировать такие комментарии, пользователя можно уведомить о возникновении сбойной ситуации либо можно вести журнал учета событий, которые программа считает заслуживающими внимания.

Чтобы отличить такие комментарии от обычных выходных данных, в библиотеке C++, помимо стандартных входных и выходных потоков, определен **стандартный поток ошибок** (`standard error stream`). Этот поток часто объединяется со стандартным выходным потоком, но в большинстве систем предоставляется возможность их раздельного использования.

Чтобы вывести данные в стандартный поток ошибок, C++-программы могут использовать выходные потоки `cerr` или `clog`. Эти выходные потоки связаны с одним и тем же приемником информации. Различие между ними состоит в способе обработки буферизации (см. раздел 1.1).

Поток `clog` предназначен для регистрации событий. Поэтому он обладает такими же свойствами буферизации, как и поток `cout`: он сохраняет символы в буфере и выводит их лишь тогда, когда операционная система решит, что для этого наступил подходящий момент. Поток `cerr`, напротив, всегда немедленно выводит данные. Эта стратегия гарантирует, что подготовленные для вывода данные станут видимыми при “первой” же возможности, но за “срочность” придется расплачиваться существенными затратами системных ресурсов. Таким образом, для вывода неотложных сообщений следует использовать поток `cerr`, а для генерирования текущих комментариев о работе программы — поток `clog`.

10.5.2. Использование нескольких входных и выходных файлов

Стандартные потоки ввода, вывода и ошибок могут быть (а могут и не быть!) связаны с файлами. Например, оконные системы могут выполнять C++-программы с потоками, связанными с некоторым окном, которое соответствует данной программе, и могут использовать, помимо получения доступа к дисковым файлам, совершенно другие возможности.

По этой причине объекты, которые использует стандартная библиотека C++ для файлового ввода и вывода, имеют другие типы по сравнению с объектами, используемыми библиотекой для идентификации стандартных входных и выходных потоков. Если вы хотите работать с входным или выходным файлом, необходимо создать объект типа `ifstream` или `ofstream` соответственно. Может показаться, что это требование создает излишние трудности. Как мы вскоре увидим, все библиотечные средства ввода-вывода определены с использованием типов `istream` и `ostream`. Так неужели в библиотеке есть еще один набор определений для типов `ifstream` и `ofstream`?

К счастью, нет. Как мы увидим в главе 13, у нас есть все основания утверждать, что типы настолько подобны, что один тип может замещать другой. По определению стандартной библиотеки, тип `ifstream` — это разновидность типа `istream`, а тип `ofstream` — разновидность типа `ostream`. В результате тип `ifstream` можно использовать везде, где библиотека “ожидает увидеть” тип `istream`, а тип `ofstream` — везде, где библиотека “ожидает увидеть” тип `ostream`. Определения обоих классов “вложены” в заголовок `<fstream>`.

Когда мы определяем объект типа `ifstream` или `ofstream`, предполагается, что необходимо поддерживать имя файла (в виде `string`-объекта), который мы хотим использовать. В действительности нужно поддерживать не `string`-объект, а указатель на начальный элемент символьного массива с завершающим нуль-символом. Одна из причин такого требования — предоставить программам возможность использовать библиотеку ввода-вывода без применения `string`-средств. Другая причина имеет исторический оттенок: библиотека ввода-вывода “старше” библиотечного класса `string` на несколько лет. И еще одно: это требование упрощает интерфейс со средствами ввода-вывода операционной системы, которые обычно используют такие указатели для передачи системных сообщений. Но какие бы ни были причины, факт остается фактом: программы, которые работают с файлами, должны безоговорочно выражать имена этих файлов в виде указателей на символьные массивы с завершающим нуль-символом.

В качестве примера рассмотрим программу, которая копирует файл с именем `in` в файл с именем `out`.

```
int main()
{
    ifstream infile("in");
    ofstream outfile("out");

    string s;
    while (getline(infile, s))
        outfile << s << endl;
    return 0;
}
```

Эта программа использует преимущества того, что строковый литерал, по сути, представляет собой указатель на начальный символ массива с завершающим нуль-символом. Если мы не хотим передавать имя файла в виде литерала, то лучше всего сохранить это имя в некотором `string`-объекте, а затем использовать функцию-член `c_str`, о которой пойдет речь в разделе 12.6. Например, если `file` — это `string`-переменная, содержащая имя файла, который мы хотим прочитать, то можем создать для чтения этого файла объект типа `ifstream`, определив его следующим образом.

```
ifstream infile(file.c_str());
```

А теперь приведем пример программы, которая, используя стандартный выходной поток, генерирует копию содержимого всех файлов, имена которых передаются в качестве аргументов функции `main`.

```
int main(int argc, char **argv)
{
    int fail_count = 0;
    // Для каждого файла в списке:
    for (int i = 1; i < argc; ++i) {
        ifstream in(argv[i]);

        // Если файл существует, выводим его содержимое,
        // в противном случае генерируем сообщение об ошибке.
        if (in) {
            string s;
            while (getline(in, s))
                cout << s << endl;
        } else {
            cerr << "не удается открыть файл " << argv[i] << endl;
            ++fail_count;
        }
    }
    return fail_count;
}
```

Для каждого аргумента, переданного функции `main` (см. раздел 10.4), программа создает объект типа `ifstream` для чтения файла с именем, заданным аргументом. Если результат тестирования этого объекта окажется равным значению `false`, это значит, что файл с таким именем не существует или не может быть прочитан по какой-то причине. В этом случае программа для выражения “недовольства” использует поток ошибок `cerr` и подсчитывает количество подобных “проколов”. Если программа успешно создает `ifstream`-объект, она построчно считывает содержимое файла в объект `s` и выводит содержимое каждой строки в стандартный выходной поток.

При возврате управления операционной системе программа “в придачу” возвращает и количество файлов, которые ей не удалось прочитать. Как правило, если значение, возвращаемое программой, равно нулю, это говорит о ее успешном выполнении. В данном случае это означает, что программа смогла прочитать содержимое всех заданных файлов.

10.6. Три вида управления памятью

До сих пор мы имели дело с двумя отдельными видами управления памятью, хотя и не акцентировали на этом внимание. Первый вид обычно называется *автоматическим управлением памятью* и имеет отношение к локальным переменным: локальная переменная занимает область памяти, которая выделяется системой во время выполнения программы при обнаружении определения переменной. Система автоматически освобождает эту память в конце блока, содержавшего это определение.

Если переменная была освобождена, то любые указатели на нее становятся недействительными. Вся ответственность за использование недействительных указателей лежит на программисте. Рассмотрим, например, следующую функцию.

```
// Эта функция специально генерирует недействительный
// указатель. Цель функции - показать, как нельзя поступать!
int* invalid_pointer()
{
    int x;
```

```

    return &x; // Просто катастрофа!
}

```

Эта функция возвращает адрес локальной переменной *x*. К сожалению, по завершении функции завершается и выполнение блока, который содержит определение переменной *x*, в результате чего выделенная для этой переменной память освобождается. Указатель, созданный выражением *&x*, становится недействительным, но функция так или иначе пытается вернуть его. О происходящем в этот момент можно лишь догадываться. В частности, C++-среды не считают нужным диагностировать эту ошибку — вы, как программист, просто получаете по заслугам.

Если вы все-таки хотите вернуть адрес такой переменной, как *x*, это можно сделать, используя другой вид управления памятью, а именно запросив *статически выделяемую* область памяти для переменной *x*.

```

// Эта функция совершенно легитимна.
int* pointer_to_static()
{
    static int x;
    return &x;
}

```

Объявляя переменную *x* с использованием слова *static*, мы тем самым сообщаем о своем намерении выделить ей память только однажды, причем еще до первого вызова функции *pointer_to_static*. В результате эта память не будет освобождена до тех пор, пока не завершится выполнение программы. В возврате адреса статической переменной нет ничего некорректного, поскольку этот указатель будет действительным все время, пока программа работает, а нерелевантным он станет только после ее завершения.

Однако статическое выделение имеет потенциальный недостаток, поскольку каждый вызов функции *pointer_to_static* будет возвращать указатель на один и тот же объект! Предположим, мы хотим определить функцию, при каждом обращении к которой будем получать указатель на совершенно новый объект, причем этот объект должен оставаться действительным до тех пор, пока мы не решим, что больше не нуждаемся в его услугах. Для этого необходимо воспользоваться *динамическим распределением памяти* (dynamic allocation), которое возможно при использовании ключевых слов *new* и *delete*.

10.6.1. Размещение объекта в памяти и освобождение этой памяти

Если *T* — тип некоторого объекта, то *new T* представляет собой выражение, которое размещает инициализируемый по умолчанию объект типа *T* и генерирует указатель на этот только что размещенный (причем неименованный) объект. При инициализации этого объекта можно присвоить ему конкретное значение, выполнив инструкцию *new T(args)*. Этот объект будет существовать до тех пор, пока программа либо не завершится, либо не выполнит инструкцию *delete p* (смотря что случится раньше), где *p* — это копия указателя, возвращаемого оператором *new*. Чтобы можно было использовать операцию *delete*, указатель *p* должен указывать на объект, размещенный в памяти посредством операции *new*, или должен быть равным нулю. При удалении нулевого указателя никаких действий не выполняется.

Например, следующая инструкция

```
int* p = new int(42);
```

разместит в памяти новый неименованный объект типа `int`, инициализированный значением 42, а указатель `p` будет указывать на этот объект. Мы можем изменить значение этого объекта, например, с помощью следующей инструкции, после выполнения которой объект будет иметь значение 43.

```
++*p; // Значение *p сейчас равно 43.
```

Когда объект нам не будет нужен, мы сможем выполнить следующую инструкцию.

```
delete p;
```

В результате область, занимаемая объектом `*p`, будет освобождена, а указатель `p` станет недействительным, т.е. его нельзя использовать до тех пор, пока мы не присвоим ему новое значение.

Рассмотрим пример функции, которая размещает в памяти некоторый `int`-объект, инициализирует его и возвращает указатель на этот объект.

```
int* pointer_to_dynamic()
{
    return new int(0);
}
```

Ответственность за освобождение объекта эта функция перекладывает на автора вызова.

10.6.2. Размещение в памяти массива

Если `T` — это некоторый тип, а `n` — неотрицательное целое значение, то выражение `new T[n]` размещает в памяти массив, состоящий из `n` объектов типа `T`, и возвращает указатель (который имеет тип `T*`) на начальный элемент этого массива. Каждый объект инициализируется по умолчанию, а это означает, что если `T` — встроенный тип и массив создается в локальной области видимости, то его объекты деинициализированы. Если `T` — тип класса, то каждый элемент инициализируется посредством выполнения конструктора по умолчанию.

Если `T` — тип класса, то процесс инициализации может иметь два важных последствия. Во-первых, если класс не допускает инициализацию по умолчанию, то компилятор откажется от выполнения этой программы. Во-вторых, инициализация каждого из `n` элементов массива (не говоря уже обо всех) может вызвать существенные расходы системных ресурсов. В главе 11 будет показано, что стандартная библиотека предоставляет более гибкий механизм динамического размещения массивов в памяти, и чаще всего следует отдавать предпочтение именно этому механизму, а не оператору `new`.

Хотя каждый обычный массив должен иметь хотя бы один элемент, тем не менее, выполнив инструкцию `new T[n]` при `n` равном нулю, можно разместить в памяти “массив” без элементов. В этом случае у оператора `new` возникают небольшие проблемы с возвратом указателя на начальный элемент (ввиду его отсутствия). Тогда этому оператору ничего не остается, как вернуть действительный указатель “конца массива” (off-the-end pointer), который позже мы можем использовать в качестве аргумента для операции `delete[]`. Кроме того, мы можем представить себе это возвращаемое значение как указатель, который бы указывал на начальный элемент, если бы таковой существовал.

Цель такого необычного поведения — разрешить программам, подобным следующей

```
T* p = new T[n];
vector<T> v(p, p + n);

delete[] p;
```


работать даже в том случае, если n равно нулю. Тот факт, что p не указывает ни на один элемент, когда n равно нулю, не так важен; важно то, что p и $p+n$ — указатели, значения которых можно сравнить, и при этом можно узнать, равны ли они. Во всех случаях вектор v будет иметь n элементов. Для программ же очень удобно обрабатывать указатели даже тогда, когда n равно нулю.

Обратите внимание на использование оператора `delete[]` в этом примере. Квадратные скобки необходимы, чтобы сообщить системе об освобождении целого массива, а не одного элемента. Массив, размещенный в памяти посредством оператора `new[]`, существует до тех пор, пока не завершится программа или не будет выполнена инструкция `delete[] p`, где p — это копия указателя, сгенерированного оператором `new[]`. Прежде чем будет освобожден весь массив, система разрушит каждый элемент, причем в обратном порядке.

В качестве примера рассмотрим функцию, которая принимает указатель на начальный элемент символьного массива с завершающим нуль-символом (например, строкового литерала), копирует все символы массива (включая нуль-символ в конце) в “новоиспеченный” массив, динамически размещенный в памяти, и возвращает указатель на начальный элемент этого нового массива.

```
char* duplicate_chars(const char* p)
{
    // выделяем достаточно большую область памяти,
    // не забыв учесть завершающий нуль-символ.
    size_t length = strlen(p) + 1;
    char* result = new char[length];

    // копируем в наш новый динамически размещенный в памяти
    // массив и возвращаем указатель на его первый элемент.
    copy(p, p + length, result);
    return result;
}
```

В разделе 10.2 говорилось о том, что функция `strlen` возвращает количество символов в массиве с завершающим нуль-символом, не считая этот нуль-символ. Поэтому к результату вызова функции `strlen` мы прибавляем 1, чтобы учесть завершающий символ, и создаем запрос на выделение суммарного числа символов. Поскольку указатели являются итераторами, для копирования символов из массива, заданного указателем p , в массив, заданный указателем `result`, мы можем использовать алгоритм `copy`. А так как значение `length` включает нуль-символ в конце массива, при обращении к функции `copy` этот символ копируется так же, как и все предыдущие.

Подобно рассматриваемой выше функции `pointer_to_dynamic`, функция `duplicate_chars` перекладывает ответственность за освобождение памяти, выделенной ею для объекта, на автора вызова. В общем, найти подходящее время для освобождения динамически выделяемой памяти не так-то легко. Методы автоматизированного решения этой проблемы рассматриваются в разделе 11.3.4.

10.7. Резюме

Указатели — это итераторы произвольного доступа, которые содержат адреса объектов.

$p = \&s$;

Указатель p указывает на объект s

$*p = s2$;

Разыменовывает указатель p и присваивает новое значение объекту, на который p указывает

`vector<string> (*sp)(const string&) = split;`

	Определяет <code>sp</code> как указатель на функцию <code>split</code>
<code>int nums[100];</code>	Определяет <code>nums</code> как массив, содержащий 100 элементов типа <code>int</code>
<code>int* bn = nums;</code>	Определяет <code>bn</code> как указатель на первый элемент массива <code>nums</code>
<code>int* en = nums + 100;</code>	Определяет <code>en</code> как указатель на "элемент", расположенный непосредственно за последним элементом массива <code>nums</code>

Указатели могут указывать на одиночные объекты, массивы объектов или функции. Когда указатель ссылается на функцию, его значение может быть использовано только для вызова этой функции.

Массивы — это встроенные контейнеры фиксированного размера, итераторы которых являются указателями. Имя массива автоматически преобразуется в указатель на начальный элемент этого массива. Строковый литерал — это символьный массив с завершающим нуль-символом. Индексирование массива определяется посредством выполнения операций над указателями: для каждого массива `a` и индекса `n` выражение `a[n]` эквивалентно выражению `*(a + n)`. Если `a` — массив, содержащий `n` элементов, то диапазон `[a, a + n)` представляет все элементы массива `a`. Массивы можно инициализировать при их определении.

```
string days[] = { "Mon", "Tues", "Wed", "Thu", "Fri", "Sat", "Sun" };
```

C++-среда "вычислит" размер массива `days`, исходя из количества инициализаторов.

Функция `main` может (необязательно) принять два аргумента. Первый аргумент (типа `int`) означает, сколько символьных массивов задает второй аргумент, который имеет тип `char**`. Второй аргумент функции `main` иногда записывается следующим образом.

```
char* argv[]
```

Эта запись эквивалентна выражению `char**`, причем такой синтаксис допустим только в списке параметров.

Потоки ввода-вывода

<code>cerr</code>	Стандартный поток ошибок. Выходные данные не буферизируются
<code>clog</code>	Стандартный поток ошибок, предназначенный для их регистрации. Выходные данные буферизируются
<code>ifstream(cp)</code>	Входной поток, связанный с файлом, именуемым выражением <code>char* cp</code> . Поддерживает потоковые операции с объектами типа <code>istream</code>
<code>ofstream(cp)</code>	Выходной поток, связанный с файлом, именуемым выражением <code>char* cp</code> . Поддерживает потоковые операции с объектами типа <code>ostream</code>

Потоки, предназначенные для работы с входными и выходными файлами, определены в заголовке `<ifstream>`.

Управление памятью

<code>new T</code>	Размещает в памяти новый объект типа <code>T</code> , инициализирует его по умолчанию и возвращает указатель на этот объект
<code>new T(args)</code>	Размещает в памяти новый объект типа <code>T</code> , инициализирует его, используя значение <code>args</code> . Возвращает указатель на этот объект
<code>delete p</code>	Разрушает объект, на который указывает указатель <code>p</code> , и освобождает память, которую занимал объект <code>*p</code> . Указатель должен указывать на объект, который был динамически размещен в памяти

<code>new T[n]</code>	Размещает в памяти массив, состоящий из <code>n</code> новых объектов типа <code>T</code> , и инициализирует его по умолчанию. Возвращает указатель на начальный элемент этого массива
<code>delete[] p</code>	Разрушает объекты массива, на который указывает <code>p</code> , и освобождает память, занимаемую этим массивом. Указатель должен указывать на начальный элемент массива, который был динамически размещен в памяти

Упражнения

- 10.0. Скомпилируйте, выполните и протестируйте программы, приведенные в этой главе.
- 10.1. Переделайте программу вычисления итоговых оценок студентов из раздела 9.6, чтобы она генерировала оценки, выраженные в буквах A–F.
- 10.2. Перепишите функцию `median` из раздела 8.1.1, чтобы ее можно было вызывать либо с вектором, либо со встроенным массивом. Эта функция должна быть способна обрабатывать контейнеры любого арифметического типа.
- 10.3. Напишите тестовую программу, позволяющую удостовериться в корректной работе функции `median`. При этом вы должны гарантировать, что вызов функции `median` не изменит порядок элементов в контейнере.
- 10.4. Напишите класс, который реализует список, содержащий `string`-объекты.
- 10.5. Напишите двунаправленный итератор для вашего класса `String_list`.
- 10.6. Протестируйте этот класс, переделав функцию `split` так, чтобы она помещала свои выходные данные в контейнер типа `String_list`.

ОПРЕДЕЛЕНИЕ АБСТРАКТНЫХ ТИПОВ ДАННЫХ

В главе 9 мы познакомились с некоторыми средствами базового языка, необходимыми для определения новых типов. Однако из определения типа `Student_info` (последняя его версия также создана в главе 9) неясно, что происходит при копировании объектов типа `Student_info`, а также присваивании или освобождении занимаемой ими памяти. Как будет показано в данной главе, автор класса может управлять и этими аспектами поведения объекта. Удивительно здесь то, что корректное определение этих операций имеет крайне важные последствия для простого и интуитивного использования объектов этого типа.

Освоив в определенной степени тип `vector`, попробуем создать подобный ему класс, чтобы лучше понять, как вообще проектируются и реализуются классы. Сама реализация будет в значительной степени упрощена в пользу операций, на которые мы хотим обратить ваше внимание. Поскольку мы будем создавать упрощенную версию библиотечного класса `vector`, назовем ее `Vec`, чтобы не допустить путаницы с библиотечным классом. Хотя в первую очередь нас интересует, как копировать, присваивать и разрушать объекты класса `Vec`, все же имеет смысл начать с реализации более простых функций-членов. Справившись с ними, мы вернемся к тому, что нас беспокоит больше всего, а именно к возможности управлять копированием, присваиванием и разрушением объектов класса.

11.1. Класс `Vec`

Разрабатывая класс, мы обычно начинаем с определения интерфейса, который хотим предоставить пользователям этого класса. Чтобы научиться определять интерфейс, стоит просмотреть уже написанные и работающие программы соответствующего типа. Поскольку мы хотим реализовать некоторое подмножество стандартного класса `vector`, имеет смысл начать с рассмотрения характера использования векторов.

```
// Создаем vector-объекты.
vector<Student_info> vs;           // Пустой вектор.
vector<double> v(100);             // Вектор, содержащий 100 элементов.

// Определяем переменные типов, используемых классом vector.
vector<Student_info>::const_iterator b, e;
vector<Student_info>::size_type i = 0;

// Для просмотра каждого элемента вектора
// используем функцию size и операцию доступа по индексу.
```

```
for (i = 0; i != vs.size(); ++i)
    cout << vs[i].name();
```

```
// Возвращаем итераторы, которые указывают на первый элемент и
// "элемент", расположенный за последним элементом вектора.
b = vs.begin(); e = vs.end();
```

Конечно же, этот список операций — только некоторое подмножество возможностей, предоставляемых стандартным классом `vector`. Однако реализуя даже это подмножество, мы можем понять те языковые средства, которые необходимы для поддержки большей части интерфейса класса `vector`.

11.2. Реализация класса `Vec`

Определив операции, можно подумать о том, как представить класс `Vec`.

Самое простое решение состоит в необходимости определить *шаблонный класс* (template class). Мы хотим позволить программистам использовать объекты класса `Vec` с целью хранения в них объектов различных типов. Такое средство, как шаблоны (описанное в разделе 8.1.1 для функций), применимо и к классам. Это средство позволяет написать одно-единственное определение шаблонной функции, а затем использовать его для создания версий, которые могут выполняться с различными типами данных. Аналогично мы можем определить шаблонный класс, а затем использовать его для создания семейства типов, которые различаются лишь соответствием типам, указанным в списке параметров шаблона. Выше мы уже использовали такие типы, включая `vector`, `list` и `map`.

Как и в случае с шаблонными функциями, при определении шаблонного класса мы должны сообщить о том, что данный класс является шаблоном, и перечислить параметры-типы, которые будут использоваться в определении класса.

```
template <class T> class Vec {
public:
    // Раздел интерфейса
private:
    // Раздел реализации
};
```

Этот код означает, что `Vec` — шаблонный класс, у которого только один параметр-тип с именем `T`. Подобно другим типам классов, мы можем предположить, что в нем будут существовать `public`- и `private`-части, которые определяют разделы интерфейса и реализации соответственно.

Теперь необходимо решить, какие данные мы будем хранить. По-видимому, нам понадобится некоторая область памяти для хранения элементов, помещаемых в объект класса `Vec`, при этом мы должны иметь возможность отслеживать количество элементов, содержащихся в этом `Vec`-объекте. Исходя из сказанного, можно заключить, что элементы лучше всего хранить в массиве, динамически размещаемом в памяти.

А какую информацию об этом массиве мы должны хранить? Было бы весьма полезно реализовать функции `begin`, `end` и `size`. Из этого вытекает, что мы хотели бы хранить адрес начального элемента, адрес элемента, расположенного за последним элементом массива, и общее количество элементов. Однако нет необходимости хранить все три вида информации, поскольку любой из них можно вычислить на основании остальных двух. Следовательно, мы принимаем решение хранить лишь указатели на начальный элемент и элемент, расположенный за последним элементом массива.

ва, и при необходимости будем вычислять размер массива. Вероятно, структура данных будет выглядеть примерно так.



Приняв такое решение по реализации, мы можем обновить наш класс Vec.

```
template <class T> class Vec {
public:
    // интерфейс
private:
    T* data;           // первый элемент в объекте типа Vec.
    T* limit;          // "Элемент", расположенный за последним
                      // элементом контейнера Vec.
};
```

Об этом определении класса можно сказать, что Vec принимает один параметр-тип. В теле определения класса мы будем называть этот тип именем T. Везде, где используется обозначение T, компилятор заменит его типом, который укажет пользователь при создании объекта класса Vec. Так, например, определение

```
Vec<int> v;
```

заставит компилятор реализовать (см. раздел 8.1.2) версию класса Vec, в которой каждое вхождение обозначения T будет заменено обозначением типа int. Код, генерируемый компилятором для этого класса, будет преобразовывать выражения, в которых участвуют обозначения T, в выражения с использованием обозначения int. Таким образом, поскольку мы применили параметр-тип T в объявлении переменных data и limit, тип этих указателей зависит от типа объектов, которые будут содержаться в объекте класса Vec.

Этот тип остается неизвестным до тех пор, пока не будет реализовано определение класса Vec. Заявив о своем намерении использовать объект типа Vec<int>, мы вносим ясность в типы переменных data и limit: для этого экземпляра класса Vec они будут иметь тип int*. Аналогично, если мы захотим создать объект типа Vec<string>, компилятор сгенерирует второй экземпляр класса Vec, в котором обозначению T будет соответствовать обозначение типа string, и тогда переменные data и limit в этом экземпляре будут иметь тип string*.

11.2.1. Распределение памяти

Поскольку наш класс будет размещать свой массив динамически (см. раздел 10.6.2), то, предположительно, мы должны выделить память для нашего класса Vec, используя выражение new T[n], где n — количество элементов, которое мы хотим поместить в массив. Однако вспомните, что выражение new T[n] не только выделяет память, но и инициализирует элементы посредством выполнения конструктора по умолчанию для типа T. Если бы мы использовали выражение new T[n], то были бы ограничены требованием, предъявляемым к типу T: пользователи могли бы создать объект класса Vec<T> только в том случае, если бы тип T имел конструктор по умолчанию. Стандартный класс vector не накладывает подобного ограничения. А так как мы хотим имитировать стандартные векторы, то также не должны накладывать это ограничение.

Оказывается, в библиотеке предусмотрен класс распределения памяти, который предлагает более детализированные средства управления выделением памяти. Этот класс — как раз то, что нам нужно, если мы откажемся от использования операторов `new` и `delete`. Данный класс позволяет заблаговременно выделить память, а затем строить объекты в этой памяти. Вместо собственноручного распределения памяти согласно “потребностям” этого класса, мы допускаем, что в конце концов нам придется написать ряд служебных функций (утилит), которые каким-то образом распределят память за нас. А пока предположим, что такие функции уже существуют, и мы будем использовать их в дополнение к классу `Vec`. По мере их использования мы все яснее будем представлять панораму действий, которые должны выполнять эти функции, чтобы к моменту, когда придет время реализовать их, мы точно знали, что именно нам предстоит реализовать.

Эти новые утилиты станут частью `private`-реализации нашего класса. Они будут отвечать за выделение и освобождение нужной нам памяти, а также за инициализацию и разрушение элементов, содержащихся в объекте класса `Vec`. Таким образом, эти функции будут управлять нашими указателями `data` и `limit`. Только благодаря функциям распределения памяти эти члены данных смогут получать новые значения. А `public`-члены класса `Vec` должны лишь считывать значения `data` и `limit`, не изменяя их при этом.

Когда конкретным `public`-членам потребуется выполнить некоторое действие, например создать новый объект класса `Vec`, в котором придется изменить значение члена `data` или `limit`, они вызовут для этого соответствующую функцию управления памятью. Такая стратегия позволит нам раздробить работу класса: один набор членов будет обеспечивать интерфейс для пользователя нашего класса, а другой — отвечать за детали реализации.

К деталям реализации упомянутых утилит мы вернемся в разделе 11.5.

11.2.2. Конструкторы

Мы уже знаем, что должны определить два конструктора.

```
vec<Student_info> vs; // используется конструктор по умолчанию.  
vec<double> vs(100); // используется конструктор, принимающий  
                     // размер.
```

Стандартный класс `vector` предоставляет и третий конструктор, который принимает размер и начальное значение, предназначенное для инициализации элементов вектора, а затем инициализирует все элементы копиями этого значения. Этот конструктор подобен своему “коллеге”, принимающему только размер, поэтому мы можем реализовать и этот третий конструктор.

Роль любого конструктора — гарантировать корректную инициализацию объекта. Для объектов класса `Vec` нам необходимо инициализировать члены данных `data` и `limit`, что означает выделение памяти для хранения элементов `Vec`-объекта и инициализацию этих элементов соответствующими значениями. При использовании конструктора по умолчанию мы хотели бы создать пустой `Vec`-объект, поэтому не будем выделять какую бы то ни было память. Для конструкторов, которые принимают размер, сформируем запрос на выделение заданного объема памяти. Если пользователь, помимо размера, предоставит нам и некоторое начальное значение, мы будем использовать это значение для инициализации всех элементов, размещаемых в памяти. Если пользователь предоставит только размер, то для получения значения инициализации элементов будем использовать конструктор по умолчанию для типа `T`. Пока же инициализацию членов данных `data` и `limit`, а также размещение в памяти элементов и их инициализацию мы передадим нашим (еще ненаписанным) функциям управления памятью.

```

template <class T> class Vec {
public:
    Vec() { create(); }
    explicit Vec(size_type n,
                  const T& val = T()) { create(n, val); }
    // Остальная часть интерфейса.

private:
    T* data;
    T* limit;
};

```

Конструктор по умолчанию, который не принимает аргументов, должен каким-то образом обозначить, что вектор пустой (т.е. не содержит элементов). Это реализуется посредством вызова функции-члена с именем `create`, которую нам еще предстоит написать. По возвращении из функции `create` оба указателя `data` и `limit` должны быть равны нулю.

Наш второй конструктор использует ключевое слово `explicit`, с которым нам еще не приходилось встречаться. Прежде всего, попробуем понять, что делает этот конструктор. Обратите внимание на то, что для своего второго параметра он использует аргумент по умолчанию (см. раздел 7.3). Таким образом, посредством определения этого конструктора успешно определяются сразу два конструктора: один принимает единственный аргумент типа `size_t`, а другой — два аргумента (типа `size_t` и `const T&`). В обоих случаях мы вызываем версию функции `create`, которая принимает размер и значение. Допустим, что эта функция, которую мы напомним в разделе 11.5, выделит объем памяти, достаточный для хранения `n` объектов типа `T`, и присвоит этим элементам начальное значение, заданное аргументом `val`. Нашим пользователям придется предоставить это значение в явном виде. В противном случае конструктор по умолчанию для типа `T` сгенерирует его на основании правил, перечисленных в разделе 9.5 для инициализации значением.

Теперь мы подошли к использованию слова `explicit`. Это ключевое слово имеет смысл только в определении конструктора, который принимает один аргумент. Определяя конструктор со словом `explicit`, мы тем самым заявляем, что компилятор будет использовать его только в контекстах, связанных с четко обозначенным (т.е. явным) вызовом конструктора (и ни в каких других).

```

Vec<int> vi(100); // Явно создаем объект класса Vec
                  // для хранения int-объектов.
Vec<int> vi = 100; // Ошибка: неявно создаем объект класса Vec
                  // (см. раздел §11.3.3) и копируем его в vi.

```

В других контекстах использования конструктора наличие слова `explicit` может быть даже опасным, поэтому мы вернемся к его обсуждению в разделе 12.2. Для соответствия стандартному классу `vector` мы сделали этот конструктор явным (`explicit`), несмотря на то что ни один из последующих примеров этой главы не опирается на этот факт.

11.2.3. Определение типов

Следуя соглашению, используемому для стандартных шаблонных классов, мы должны предоставить имена типов, которые могут применять пользователи, а также позаботиться о сокрытии деталей реализации нашего класса. В частности, посредством `typedef` нам нужно обеспечить синонимы для `const`- и `non-const`-типов итераторов, а также для типа, используемого при указании размера объекта типа `Vec`.

Оказывается, библиотечные контейнеры также определяют тип, именуемый `value_type`, который является синонимом для типа объектов, предназначенных для хранения в этом контейнере. В класс `Vec` обязательно нужно поместить функцию-член `push_back`, чтобы пользователи могли динамически наращивать свои `Vec`-объекты. Если мы также определим тип `value_type`, то другие программисты смогут использовать функцию `back_inserter` (которая зависит от `push_back` и `value_type`), чтобы сгенерировать выходной итератор, позволяющий наращивание `Vec`-объекта.

Самое трудное в определении этих типов — выбрать сами типы. Как уже отмечалось, итераторы — это объекты, которые перемещаются по объектам контейнера и позволяют опрашивать их значения. Часто итераторы сами имеют тип класса. Например, рассмотрим класс, который реализует связанный список. Логическую стратегию для такого класса можно было бы представить так: моделируем список как набор узлов, в котором каждый узел содержит значение и указатель на следующий узел в списке. Итератор для такого класса должен содержать указатель на один из узлов и поддерживать оператор “++” для формирования указателя на следующий узел в списке. Такой итератор должен быть реализован как тип класса.

Поскольку для хранения элементов `Vec`-объекта мы использовали массив, в качестве типа итератора для доступа к его элементам можно применять простые указатели. Каждый такой указатель будет указывать на базовый массив `data`. Как мы узнали в разделе 10.1, указатели поддерживают все операции итераторов произвольного доступа. Используя в качестве нашего базового итераторного типа указатель, мы тем самым предоставим полный спектр свойств итератора произвольного доступа, который согласуется со стандартным классом `vector`.

А как насчет других типов? Тип `value_type` очевиден: им должен быть тип `T`. Но у нас еще есть тип, представляющий размер контейнера. Как быть с ним? Мы знаем, что `size_t` — это тип, который позволяет хранить достаточно большие значения объема массива, т.е. количество его элементов. Поскольку мы храним элементы `Vec`-объекта в массиве, то в качестве базового типа для типа `Vec::size_type` можно использовать `size_t`. Определившись с типами, получаем следующее определение класса.

```
template <class T> class Vec {
public:
    typedef T* iterator;           // Добавлено.
    typedef const T* const_iterator; // Добавлено.
    typedef size_t size_type;      // Добавлено.
    typedef T value_type;          // Добавлено.
    typedef T& reference;          // Добавлено.
    typedef const T& const_reference; // Добавлено.

    Vec() { create(); }
    explicit Vec(size_type n,
                 const T& val = T()) { create(n, val); }
    // Остальная часть интерфейса.

private:
    iterator data;                 // Изменено.
    iterator limit;                // Изменено.
};
```

Помимо добавления в определение класса соответствующих `typedef`-объявлений, мы также слегка обновили класс за счет наших новых типов. Используя внутри класса те же имена, которые определили с помощью наших `typedef`-объявлений, мы делаем наш код более читабельным, а главное, гарантируем, что он не обнаружит “несинхронность”, если мы впоследствии изменим один из этих типов.

11.2.4. Индексирование и определение размера

Как было отмечено, наши пользователи должны иметь возможность вызывать функцию `size`, чтобы узнать, сколько элементов содержится в `Vec`-объекте, и использовать *индекс-оператор* (index operator) для доступа к элементам контейнера `Vec`.

```
for (i = 0; i != vs.size(); ++i)
    cout << vs[i].name();
```

Из этого кода видно, что функция `size` должна быть членом класса `Vec` и нам необходимо определить средство, позволяющее использовать *оператор, возвращающий элемент* `Vec`-объекта *по индексу* (subscript operator), `[]`. С функцией `size` проще всего: она не принимает аргументов и должна возвращать количество элементов в `Vec`-объекте в виде значения типа `Vec::size_type`. Прежде чем определять операцию доступа по индексу, нам необходимо ближе познакомиться с работой перегруженных операторов.

Перегруженный оператор определяется практически так же, как любая другая функция: он (оператор) имеет имя, принимает аргументы и возвращает значение заданного типа.

Имя перегруженного оператора формируется присоединением символа оператора к слову `operator`. В данном случае функция, которую мы должны определить, будет называться `operator[]`.

Вид оператора — унарный или бинарный — определяется количеством параметров, принимаемых соответствующей функцией. Если оператор представлен функцией, которая не является членом класса, эта *операторная функция* будет иметь столько аргументов, сколько оператор имеет операндов. Первый аргумент соответствует левому операнду, а второй — правому. Если же оператор определяется как функция-член, то его левый операнд неявно соответствует объекту, для которого вызывается этот оператор. Следовательно, функции-члены, служащие для определения оператора, принимают на один аргумент меньше, чем обозначено этим оператором.

В общем случае операторные функции могут быть членами или не-членами класса. Однако оператор доступа по индексу *обязательно* должен быть представлен функцией-членом. При выполнении такого выражения, как `vs[i]`, для объекта `vs` будет вызвана функция-член с именем `operator[]` с передачей ей в качестве аргумента значения переменной `i`.

Мы знаем, что операнд должен иметь любой целый тип, позволяющий выражать довольно большие значения, достаточные для указания последнего элемента в `Vec`-объекте максимально возможного размера, и что этот тип называется `Vec::size_type`. Теперь нам осталось лишь решить, значение какого типа должен возвращать оператор доступа по индексу.

Если хорошо подумать, то нетрудно прийти к заключению, что оператор доступа по индексу должен возвращать ссылку на элемент, хранимый в объекте типа `Vec`. Это позволит пользователям записывать и считывать значения по индексу. И хотя следующая простая программа использует индекс только для считывания значения из объекта `vs`, нетрудно предположить, что пользователи захотят получить доступ к элементам и для записи. На основе проведенного анализа мы можем обновить определение нашего класса следующим образом.

```
template <class T> class Vec {
public:
    // typedef-определения из раздела 11.2.3:
    typedef T* iterator;
    typedef const T* const_iterator;
```

```

typedef size_t size_type;
typedef T value_type;
typedef T& reference;
typedef const T& const_reference;

Vec() { create(); }
explicit Vec(size_type n,
             const T& val = T()) { create(n, val); }

// Новые операции: size и доступ по индексу.
size_type size() const { return limit - data; }

T& operator[](size_type i) { return data[i]; }
const T& operator[](size_type i) const { return data[i]; }

private:
    iterator data;
    iterator limit;
};

```

Функция `size` вычисляет количество элементов в объекте класса `Vec` посредством получения разности указателей, ограничивающих массив, в котором хранятся наши значения. Вспомните из раздела 10.1.4, что вычитание двух указателей дает значение типа `ptrdiff_t`, обозначающее “расстояние” между ними, которое “измеряется” в элементах. Прежде чем возвращать это значение, функция `size` преобразует его в значение типа `size_type`, который указан в качестве типа значения, возвращаемого этой функцией, причем имя этого типа является синонимом для имени типа `size_t` (см. раздел 10.1.3). Процесс получения размера `Vec`-объекта не изменяет сам объект, поэтому мы объявляем функцию-член `size` как `const`-функцию, что позволяет нам с помощью функции `size` определять размер `const`-объекта типа `Vec`.

Оператор доступа по индексу находит соответствующую позицию массива, положенного в основу класса `Vec`, и возвращает ссылку на элемент, расположенный в этой позиции. Возвращая ссылку, мы разрешаем пользователю с помощью операции доступа по индексу изменять значения, хранимые в `Vec`-объекте. Возможность перезаписи элемента подразумевает, что нам нужно иметь две версии индекс-оператора: одну — для `const`-объектов типа `Vec`, а другую — для не-`const`-объектов типа `Vec`. Обратите внимание на то, что `const`-версия возвращает ссылку на `const`-значение, тем самым гарантируя, что индекс можно использовать только для чтения `Vec`-объекта, а не для записи в него. Стоит отметить, что мы возвращаем ссылку, а не значение, и все это — ради соответствия библиотечному классу `vector`. Причина возврата ссылки (а не значения) весьма убедительна: если объекты, хранимые в контейнере, имеют большой размер, то в целях эффективности имеет смысл избежать их копирования.

Возможно, кого-то удивит сама возможность перегрузки оператора доступа по индексу, поскольку в обоих определениях списки аргументов одинаковы: каждая функция принимает один параметр типа `size_type`. Однако вспомните, что каждая функция-член, в том числе и каждый из этих операторов, принимает один неявный параметр, а именно объект, для которого и выполняется данная функция. Поскольку операции, реализуемые этими операторами, отличаются константностью объекта, это дает нам право перегрузить рассматриваемую операцию.

11.2.5. Операции, возвращающие итераторы

Теперь рассмотрим функции, которые возвращают итераторы. Речь идет о реализации операций `begin` и `end`, которые возвращают итератор, располагаемый в начале и сразу за последним элементом `Vec`-объекта соответственно.

```

template <class T> class Vec {
public:
    // typedef-определения из раздела 11.2.3.
    Vec() { create(); }
    explicit Vec(size_type n,
                const T& val = T()) { create(n, val); }

    T& operator[](size_type i) { return data[i]; }
    const T& operator[](size_type i) const { return data[i]; }

    size_type size() const { return limit - data; }

    // новые функции, возвращающие итераторы:
    iterator begin() { return data; } // Добавлено.
    const_iterator begin() const { return data; } // Добавлено.
    iterator end() { return limit; } // Добавлено.
    const_iterator end() const { return limit; } // Добавлено.

private:
    iterator data;
    iterator limit;
};

```

Мы предоставили две версии операций `begin` и `end`, которые перегружены на основе возможной константности и неконстантности `Vec`-объекта; `const`-версии возвращают итераторы типа `const_iterator`, чтобы наши пользователи с помощью этих итераторов могли читать, но не изменять элементы `Vec`-объекта.

Хотя “строительство” нашего класса `Vec` все еще находится на уровне закладки фундамента, тем не менее “предметы первой необходимости” уже на месте. И в самом деле, если добавить еще несколько операций, например `push_back` и `clear`, то для всех примеров, приведенных в этой книге, мы могли бы использовать этот класс вместо стандартного библиотечного класса `vector`. Хотя, к сожалению, наш класс `Vec` еще уступает спецификации класса `vector` в некоторых крайне важных аспектах, к рассмотрению которых мы перейдем в следующем разделе.

11.3. Управление копированием

В начале этой главы было отмечено, что именно автор класса управляет действиями, совершаемыми при создании, копировании, разрушении объектов, а также при присвоении им некоторых значений. Пока мы рассмотрели, как создавать объекты, и нам предстоит еще освоить управление остальными событиями в жизни “новорожденных” объектов. Как будет показано ниже, если нам не удастся определить перечисленные выше операции, компилятор синтезирует необходимые (с его точки зрения) определения за нас. Иногда эти синтезированные операции в точности совпадают с нашим представлением о них. Но в большинстве случаев такие операции способны привести к алогичному, трудному для понимания поведению и даже к отказу в работе программы.

C++ — это единственный язык широкого профиля, который предоставляет программисту подобный уровень управления поведением объектов. Поэтому неудивительно, что корректное определение этих операций — крайне важный этап разработки новых типов данных.

11.3.1. Конструктор копирования

При передаче функции объекта по значению или при возврате функцией объекта выполняется *неявно* заданная операция копирования этого объекта. Рассмотрим следующий код.

```
vector<int> vi;  
double d;  
d = median(vi); // Копируем vi в параметр функции median.  
string line;  
vector<string> words = split(line); // копируем в вектор words  
// значение, возвращаемое функцией split.
```

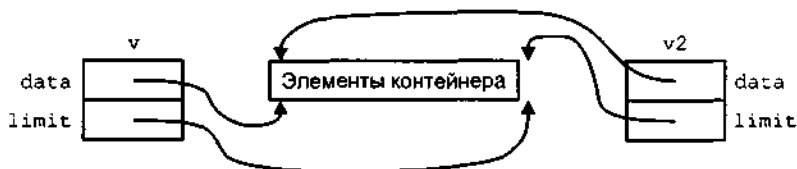
Кроме того, мы можем *явно* скопировать объект, используя его для инициализации другого объекта.

```
vector<Student_info> vs;  
vector<Student_info> v2 = vs; // Копируем vs в v2.
```

Как явное, так и неявное создание копий управляется посредством специального конструктора, именуемого *конструктором копирования* (copy constructor). Подобно остальным конструкторам, конструктор копирования является функцией-членом, имя которой совпадает с именем класса. Поскольку этот конструктор предназначен для инициализации нового объекта в виде копии существующего объекта того же типа, отсюда следует, что конструктор копирования принимает единственный аргумент, тип которого совпадает с типом самого класса. Поскольку автор класса сам определяет, что означает сделать копию объекта этого класса, включая создание копий аргументов-функций, именно в этом случае крайне важно указать, что параметр должен иметь ссылочный тип! Более того, процесс копирования не должен модифицировать копируемый объект, поэтому рассматриваемый конструктор должен принимать const-ссылку на объект, который копируется.

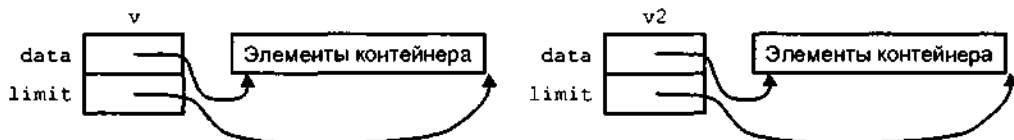
```
template <class T> class Vec {  
public:  
    Vec(const Vec& v); // конструктор копирования.  
    // Все остальное остается в силе.  
};
```

Объявив конструктор копирования, мы должны определить его действия. В общем случае конструкторы копирования “копируют” каждый элемент данных из существующего объекта в новый. Слово “копируют” мы взяли в кавычки, поскольку иногда в понятие копирования вкладывается более широкий смысл, чем просто копирование содержимого членов данных. Например, в нашем классе Vec есть два члена данных, и оба они — указатели. Если мы будем копировать значения этих указателей, то как исходный член, так и его копия будут указывать на одно и то же значение. Предположим, что объект v имеет тип Vec и мы хотим скопировать объект v в объект v2. Если бы мы скопировали указатели, то результат можно было бы отобразить следующим образом.



Очевидно, что любое изменение, внесенное в элемент одной “копии”, отразилось бы на значении элемента другой “копии”. Другими словами, если бы мы присвоили некоторое значение элементу `v[0]`, одновременно изменился бы элемент `v2[0]`. Разве нам нужно именно такое поведение?

Подобно тому, как мы поступали с другими операциями, можно ответить на этот вопрос, “подсмотрев”, как решается эта проблема в стандартном библиотечном классе `vector`. Вспомните, в разделе 4.1.1 отмечалось, что для копирования вектора нам нужно было передать функции `median` вектор по значению. Создание копии гарантировало, что изменения, внесенные внутри функции `median`, не распространятся за пределы этой функции. Этот анализ и поведение функции `median` в действии показывают, что стандартный класс `vector` при создании копии не использует одну и ту же область памяти. Его автор позаботился о том, чтобы каждая копия `vector`-объекта была самостоятельна (независима от других), и тогда изменения, внесенные в одну копию, никак не отразятся на других.



Очевидно, когда мы будем копировать некоторый `vec`-объект, нам придется выделить для объекта-копии новую область памяти и скопировать в нее содержимое исходного `vec`-объекта. Как и прежде, предположим, что одна из наших служебных функций должным образом позаботится о выделении области памяти и реализации процесса копирования, чтобы конструктор копирования мог “спокойно” переложить свою работу “на плечи” этой функции.

```
template <class T> class Vec {
public:
    Vec(const Vec& v) { create(v.begin(), v.end()); }
    // Все остальное остается в силе.
};
```

Придет время — и мы вплотную займемся функцией `create`, а пока ее гипотетическая версия должна принимать два итератора (т.е. указателя) и инициализировать элементы нового `Vec`-объекта элементами из диапазона, ограниченного заданными указателями.

11.3.2. Присваивание

Подобно тому, как определение класса “описывает”, что происходит при копировании объектов этого класса, оно точно так же управляет поведением *оператора присваивания* (assignment operator). Несмотря на то что класс может определить несколько экземпляров оператора присваивания (перегружая его, как обычно, различием в типах аргументов), версия, которая принимает `const`-ссылку на сам класс, отличается особой важностью: она определяет, что, собственно, означает присвоить одно значение типа класса другому. Эта версия обычно называется “оператором присваивания”, даже если класс определяет несколько других версий функции `operator=`. Оператор присваивания, подобно оператору доступа по индексу, должен быть членом класса. Как и любой другой оператор, оператор присваивания возвращает некоторое значение, а потому он обязан определить тип этого значения. В целях согласования со

встроенными операторами присваивания мы возвращаем ссылку на значение, расположенное с левой стороны от знака “=”.

```
template <class T> class Vec {
public:
    Vec& operator=(const Vec&);
    // Все остальное остается в силе.
};
```

Функция, реализующая присваивание, отличается от конструктора копирования тем, что присваивание всегда включает уничтожение существующего значения (расположенного слева) и его замену новым значением (расположенным справа). При создании копии мы создаем новый объект, т.е. нам не нужно заниматься освобождением ранее созданного объекта. Подобно конструктору копирования, процесс присваивания обычно включает присваивание значения каждого члена данных. Члены данных, являющиеся указателями, несут с собой те же проблемы, которые возникают и при копировании. Наша задача — так организовать процесс присваивания, чтобы каждый объект гарантированно имел собственную копию данных объекта, заданного с правой стороны от знака операции присваивания.

Прежде чем переходить к рассмотрению кода, стоит обратить ваше внимание еще на одну деталь, а именно самоприсваивание. Вполне возможно, что пользователь решит выполнить присваивание объекта самому себе. Как будет показано ниже, очень важно, чтобы операторы присваивания корректно обрабатывали процесс самоприсваивания.

```
template <class T>
Vec<T>& Vec<T>::operator=(const Vec& rhs)
{
    // проверка на самоприсваивание.
    if (&rhs != this) {
        // Освобождаем массив, расположенный в левой стороне.
        uncreate();

        // Копируем элементы с правой стороны в левую.
        create(rhs.begin(), rhs.end());
    }
    return *this;
}
```

Эта функция использует два новых для вас средства, на которых следует остановиться.

Во-первых, это синтаксис, который мы используем для определения шаблонной функции-члена вне заголовка класса. Подобно тому, как обычно поступают с любым шаблоном, мы начинаем с уведомления компилятора о том, что определяем именно шаблон, в котором используются шаблонные параметры. Затем указываем тип значения, возвращаемого этой шаблонной функцией: в данном случае — тип `Vec<T>&`. Если сравнить это определение с соответствующим объявлением в заголовочном файле, нетрудно заметить, что, согласно объявлению, эта функция должна возвращать значение типа `Vec&`. Выходит, что в типе значения, возвращаемого функцией, мы явно не указали имя параметра-типа. Оказывается, язык C++ (в качестве своего рода синтаксической “изюминки”) позволяет опустить параметры-типы, когда мы находимся в пределах области видимости шаблона. Таким образом, внутри заголовочного файла нам не нужно повторять обозначение `<T>`, поскольку шаблонный параметр задается неявно. Когда же мы определяем тип значения, возвращаемого функцией, то находимся вне области видимости класса и поэтому должны явно указать шаблонные параметры, если таковые имеются. Аналогично здесь и имя функции указано полностью: `Vec<T>::operator=`, а не про-

сто `Vec::operator=`. Но если мы указали, что данная функция является членом класса `Vec<T>`, который мы определяем, нам не нужно больше повторять спецификаторы шаблона. Следовательно, в качестве типа аргумента просто указывается `const Vec&`, хотя мы могли бы использовать избыточный вариант `const Vec<T>&`.

Во-вторых, еще один новый аспект этой функции — использование нового ключевого слова `this`. Ключевое слово `this` допустимо только внутри функции-члена, где оно обозначает указатель на объект, для которого выполняется эта функция-член. Например, внутри функции `Vec::operator=` указатель `this` имеет тип `Vec*`, поскольку `this` — это указатель на объект класса `Vec`, в котором функция `operator=` является членом. Для любого бинарного оператора (например, оператора присваивания) указатель `this` связан с левым операндом. Обычно слово `this` используется, когда нужно сослаться на сам объект, что мы и делаем здесь как в начальной `if`-проверке, так и в инструкции `return`.

Мы используем слово `this`, чтобы определить, указывают ли правые и левые части присваивания на один и тот же объект. Если так оно и есть, обе части будут иметь один и тот же адрес. Как было показано в разделе 10.1.1, выражение `&rhs` возвращает указатель, который представляет собой адрес объекта `rhs`. Для проверки на самоприсваивание мы сравниваем указатель `&rhs` и указатель `this`, который указывает на левый операнд. Если объекты одинаковы, то в операторе присваивания нам больше делать нечего, и мы сразу же переходим к инструкции `return`. Если в операторе присваивания участвуют различные объекты, нам нужно освободить старую область памяти и присвоить всем элементам данных новые значения, копируя содержимое правого операнда в заново размещенный в памяти массив. Очевидно, нам придется написать еще одну служебную функцию `uncreate`, предназначенную для разрушения элементов старого `Vec`-объекта и освобождения занимаемой им памяти. После вызова функции `uncreate`, разрушившей старые значения, мы можем использовать ту версию служебной функции `create`, которая выделит новую память для левого объекта и скопирует в нее значения правого `Vec`-объекта.

Крайне важно, чтобы операторы присваивания корректно обрабатывали возможное самоприсваивание. Наша обработка заключается в явной проверке, не являются ли левый и правый операнды одним и тем же объектом. Чтобы понять, насколько это важно, посмотрим, что произойдет, если удалить эту проверку из нашего оператора присваивания. В этом случае мы всегда будем выполнять функцию `uncreate`, разрушая элементы существующего массива из левого операнда и возвращая память, которую они занимали. Но если два операнда представляют собой один и тот же объект, то правый операнд будет ссылаться на ту же память. И когда мы перейдем к копированию элементов из правого операнда и созданию нового массива для левого операнда, результат окажется просто катастрофическим: освобождая память, занимаемую левым операндом, мы тем самым освободим память, занимаемую правым операндом. А когда вспомогательная функция `create` попытается скопировать элементы из объекта `rhs`, они уже будут разрушены, а занимаемая ими память будет успешно возвращена системе.

Хотя самая распространенная обработка самоприсваивания состоит в прямом сравнении адресов операндов, которое продемонстрировано в нашем определении оператора присваивания, тем не менее такой вариант не является универсальным и даже не всегда предпочтителен. Главное, чтобы обработка была корректной, а ее детали — это вопрос тактики.

И последнее. Обратите внимание на инструкцию `return`, которая разыменовывает указатель `this`, чтобы получить объект, на который этот `this` указывает. Мы же возвращаем ссылку на этот объект. Обычно при возврате ссылки важно позаботиться о том, чтобы объ-

ект, на который она ссылается, продолжал существовать после завершения функции. При возврате ссылки на локальный объект катастрофа гарантирована: объект, на который ссылается возвращаемая ссылка, исчезнет с возвратом из функции, оставив нас “с носом” — результат будет ссылаться на “мусор” в памяти компьютера. В реализации оператора присваивания мы возвращаем ссылку на объект, который расположен в левой части выражения. Этот объект существует вне области видимости оператора присваивания и поэтому гарантированно продолжает существовать даже после выхода из функции.

11.3.3. Присваивание — это не инициализация

Опыт заставляет нас поверить, что различие между присваиванием и инициализацией — один из сложных аспектов освоения C++. Многие языки программирования, и язык C в том числе, не обнаруживают этого различия, поэтому программисты часто и не подозревают о его существовании. То, что символ “=” можно использовать как при инициализации, так и в операторе присваивания, лишь мешает понять разницу. Используя символ “=” для присвоения переменной некоторого начального значения, мы вызываем конструктор копирования, а когда применяем его в выражении присваивания, вызываем функцию `operator=`. Авторы классов, чтобы правильно реализовать семантику класса, должны четко понимать это различие.

Ключевое различие состоит в следующем: в процессе присваивания (`operator=`) всегда уничтожается предыдущее значение, а при инициализации этого не происходит. Однако инициализация подразумевает создание нового объекта с одновременным присвоением ему некоторого значения. Инициализация выполняется

- в объявлениях переменных;
- при передаче функции параметра-функции;
- при выходе из функции (для значения, возвращаемого функцией);
- в инициализаторах конструкторов.

Присваивание выполняется только в случае, когда в выражении используется оператор “=”.

```
string url_ch = "~;/?:@=&$-_.+!*'(),"; // инициализация.
string spaces(url_ch.size(), ' ');      // инициализация.
string y;                               // инициализация.
y = url_ch;                             // присваивание.
```

В первом объявлении создается новый объект. Таким образом, мы знаем, что инициализируем этот объект, и, следовательно, будем вызывать конструктор. Синтаксис

```
string url_ch = "~;/?:@=&$-_.+!*'(),";
```

ведит компилятору создать `string`-объект на базе указателя `const char*`, который представляет строковый литерал `"~;/?:@=&$-_.+!*'(),"`. Для этого компилятор вызовет конструктор класса `string`, который принимает параметр типа `const char*`. Этот конструктор может создать объект `url_ch` прямо из заданного строкового литерала или же может построить, опираясь на строковый литерал, неименованную временную переменную, а затем вызвать конструктор копирования для создания объекта `url_ch` в виде копии этой временной переменной.

Второе объявление демонстрирует иную форму инициализации, а именно осуществляемую посредством прямой передачи конструктору одного или нескольких аргументов. В этом случае компилятор вызовет конструктор, который наиболее близко соответствует объявлению по количеству аргументов и их типам. В данном примере бу-

дет использован конструктор класса `string`, который принимает два аргумента. Первый аргумент сообщает, сколько символов должна иметь переменная `spaces`, а второй определяет, какое значение нужно присвоить каждому из этих символов. В результате компилятор определит объект `spaces` с таким же количеством символов, как у объекта `url_ch`, но все эти символы будут пробелами.

С третьим объявлением гораздо проще: мы вызываем конструктор по умолчанию, чтобы создать пустой `string`-объект.

Последняя инструкция — это вообще не объявление. Здесь используется оператор “=” как часть выражения; следовательно, это и есть присваивание. Оно будет реализовано посредством выполнения оператора присваивания, определенного в классе `string`.

Возьмем пример чуть посложнее, а именно рассмотрим аргументы-функции и значения, возвращаемые функциями. Предположим, объект `line` содержит некоторую входную строку, для которой мы вызываем функцию `split` из раздела 6.1.1.

```
vector<string> split(const string&); // Объявление функции.
vector<string> v;                  // инициализация.

v = split(line); // На входе выполняется инициализация
                // параметра функции split на базе
                // содержимого объекта line.
                // На выходе выполняется как инициализация
                // значения, возвращаемого функцией split, так
                // и присваивание этого значения переменной v.
```

Объявление функции `split` интересно тем, что тип возвращаемого ею значения является типом класса. Присваивание такого значения, возвращаемого функцией, — это двухступенчатый процесс. Сначала выполняется конструктор копирования, чтобы в точке вызова функции скопировать возвращенное функцией значение во временную переменную. Затем выполняется оператор присваивания, чтобы присвоить значение этой временной переменной левому операнду.

Различие между инициализацией и присваиванием имеет важное значение, поскольку в каждом из этих случаев выполняются различные операции.

- Инициализацией всегда управляют конструкторы
- Присваиванием всегда управляет функция-член `operator=`

11.3.4. Деструктор

Мы должны предусмотреть в нашем классе еще одну операцию, которая определяет события, происходящие при разрушении `vec`-объекта. Объект, который создается в локальной области видимости, разрушается сразу на выходе из этой области; динамически размещенный в памяти объект разрушается, когда мы удаляем указатель на этот объект. Например, рассмотрим функцию `split` из раздела 6.1.1.

```
vector<string> split(const string& str)
{
    vector<string> ret;
    // Разбиваем str на слова и сохраняем их в векторе ret.
    return ret;
}
```

При выходе из функции `split` локальная переменная `ret` выходит из области видимости этой функции и разрушается.

Подобно копированию и присваиванию, только автор класса определяет, что должно происходить при разрушении объектов этого класса. По аналогии с конструкторами, кото-

рые управляют созданием объектов, существует специальная функция, именуемая *деструктором* (destructor), которая управляет событиями, происходящими при разрушении объектов этого типа. Имя деструктора совпадает с именем класса, но оно предваряется тильдой (~). Деструкторы не принимают аргументов и не возвращают никакого значения.

Назначение деструктора — выполнить все, что необходимо при разрушении объекта. Обычно под этим подразумевается освобождение ресурсов, а именно памяти, выделенной ранее конструктором для этого объекта.

```
template <class T> class Vec {  
public:  
    ~Vec() { uncreate(); }  
    // Все остальное остается в силе.  
};
```

Для Vec-объектов мы выделяем память в конструкторах и поэтому имеем полное право освободить ее в деструкторе. Эта работа подобна той, которую выполняет оператор присваивания для уничтожения старого объекта, расположенного в левой части выражения. Неудивительно, что из деструктора мы можем вызвать ту же служебную функцию (пока ненаписанную), чтобы разрушить элементы Vec-объекта и освободить занимаемую ими память.

11.3.5. Операции по умолчанию

В некоторых классах (например, класс Student_info, который мы определяли в главах 4 и 9) не определен явным образом конструктор копирования, оператор присваивания и деструктор. Естественно, возникает вопрос: что происходит, когда создаются, копируются и разрушаются объекты таких типов или когда им присваиваются некоторые значения? Ответ простой: если автор класса сам не определит эти операции, компилятор синтезирует за него их стандартные версии (операции по умолчанию).

Стандартные версии определяются в расчете на рекурсивное выполнение копирования, присваивания или разрушения каждого элемента данных по правилам, соответствующим типу данного элемента. Члены, имеющие тип класса, копируются, присваиваются или разрушаются посредством вызова конструктора копирования, оператора присваивания или деструктора, которые определены для данного элемента данных. Члены встроенных типов копируются или присваиваются посредством копирования или присваивания их значений. Деструктору встроенных типов вообще нечего делать, даже если этим типом является указатель. В частности, при разрушении указателя с помощью деструктора по умолчанию память, на которую он указывает, не освобождается.

Теперь становится понятно, как выполняются операции по умолчанию для объектов класса Student_info. Например, конструктор копирования копирует четыре члена данных. Для этого он вызывает конструкторы копирования классов string и vector, чтобы скопировать члены name и homework, и напрямую копирует два double-значения midterm и final.

Наконец, как было показано в разделе 9.5, существует вариант, действующий по умолчанию для конструктора по умолчанию. Если в классе вообще не определен ни один конструктор, то компилятор синтезирует конструктор по умолчанию, который представляет собой конструктор, не принимающий ни одного параметра. Синтезированный по умолчанию конструктор рекурсивно инициализирует все члены данных таким же способом, каким инициализируется сам объект: если контекст требует инициализации по умолчанию, конструктор инициализирует эти члены данных по умолчанию; если контекст требует инициализации значениями, эти члены данных будут инициализированы значениями.

Необходимо отметить, что если класс явно определит какой-нибудь конструктор, даже конструктор копирования, то компилятор не станет синтезировать по умолчанию конструктор для этого класса. Конструкторы по умолчанию имеют важное значение в ряде контекстов. Один из них заключен в самом конструкторе, синтезированном по умолчанию. Чтобы некоторый тип данных можно было использовать в качестве члена данных класса, который полагается на синтез конструктора по умолчанию, этот тип данных сам должен предоставить конструктор по умолчанию. Поэтому хороший стиль программирования подразумевает следующее: всегда снабжать класс конструктором по умолчанию, либо в явном виде, как в главе 9, либо в неявном, как в главе 4.

11.3.6. Тройное правило

Классы, которые управляют такими ресурсами, как память, требуют серьезного отношения к управлению копированием. В общем случае операций по умолчанию недостаточно для таких классов. Неспособность управлять всеми копиями может сбить с толку пользователей класса и привести к возникновению ошибок времени выполнения.

Рассмотрим наш класс `Vec` и предположим, что мы не определили конструктор копирования, оператор присваивания или деструктор. Как мы видели в разделе 11.3.1, в лучшем случае мы удивим наших пользователей. Пользователи класса `Vec` будут почти уверены в том, что если они скопировали один `Vec`-объект в другой, эти два объекта будут совершенно отдельными объектами. Они вправе ожидать, что операции, выполняемые с одним `Vec`-объектом, не окажут никакого влияния на данные, содержащиеся в другом.

Гораздо хуже то, что если мы не определим деструктор, то будет использован деструктор по умолчанию. Этот деструктор разрушит указатель, но само разрушение указателя не освободит память, на которую он указывает. Результат выразится в потере ресурсов памяти, или "утечке памяти": память, занимаемая такими `Vec`-объектами, никогда не будет восстановлена.

Если не допустить утечки памяти, предоставив деструктор, но при этом не позаботиться о конструкторе копирования и операторе присваивания, то можно существенно повысить вероятность аварийного завершения программы. При такой некорректной реализации возможно возникновение ситуации, когда два `Vec`-объекта будут занимать одну и ту же область памяти, как было показано на первом рисунке в разделе 11.3.1. При разрушении одного из этих объектов деструктор разрушит их "общую" область памяти, и любая последующая ссылка на "неразрушенную" копию приведет к катастрофе.

Для классов, которые распределяют ресурсы памяти в своих конструкторах, необходимо, чтобы каждая копия корректно обращалась с этими ресурсами. Таким классам почти всегда нужен деструктор для корректного освобождения ресурсов. Если классу нужен деструктор, ему практически не обойтись без конструктора копирования, а также оператора присваивания. При копировании или присваивании объектов классов, которые распределяют ресурсы памяти, размещение объектов в памяти обычно выполняется таким же способом, как и при создании объекта с "нуля". Для корректного управления каждой копией объектов класса `T` в класс необходимо включить следующие члены:

<code>T::T()</code>	один или несколько конструкторов, возможно, с аргументами;
<code>T::~~T()</code>	деструктор;
<code>T::T(const T&)</code>	конструктор копирования;
<code>T::operator=(const T&)</code>	оператор присваивания.

Если эти операции определены в классе, компилятор будет вызывать их, когда объект этого класса создается, копируется, присваивается или разрушается. Не забывайте, что

объекты могут создаваться, копироваться или разрушаться неявно. Однако в любом случае — явном или неявном — компилятор выполнит соответствующую операцию.

Поскольку конструктор копирования, деструктор и оператор присваивания так тесно связаны, отношения между ними известны как тройное правило: если вашему классу нужен деструктор, то, скорее всего, ему нужны также конструктор копирования и оператор присваивания.

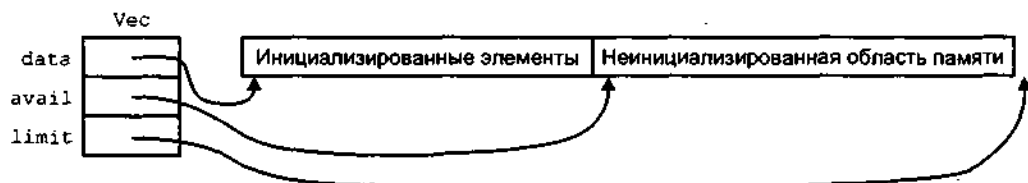
11.4. Динамические Vec-объекты

Прежде чем браться за реализацию наших функций управления памятью, необходимо хорошо понимать, что `Vec`-объекты не такие “могучие”, как стандартные `vector`-объекты. И вот по какой причине: мы не обеспечили `push_back`-операцию, и поэтому наши `Vec`-объекты имеют фиксированный размер. Вспомните, функция `push_back` помещает свой аргумент в конец вектора и при этом увеличивает его размер на один элемент.

Мы могли бы добавить функцию `push_back`, которая выделяет новую область памяти, чтобы поместить в нее на один элемент больше, чем содержится в текущем `Vec`-объекте. В этом случае нам пришлось бы скопировать существующие элементы в эту новую область памяти, присоединив к ним новый последний элемент из аргумента, переданного функции `push_back`. Нетрудно понять, что такая стратегия оказалась бы слишком дорогой, если бы наши пользователи часто обращались к функции `push_back`.

Для решения такой проблемы, как эта, существует классический подход: выделять больше памяти, чем нужно в данный момент. И только тогда, когда заранее выделенная память будет исчерпана, вам придется запрашивать дополнительный объем. Проще говоря, когда функции `push_back` понадобится дополнительная память, мы выделим область, вдвое большую, чем используется в данный момент. Так, если мы создадим `Vec`-объект со 100 элементами, а затем вызовем функцию `push_back` в первый раз, она выделит память, которой будет достаточно для размещения 200 элементов. Эта функция копирует существующие 100 элементов в первую половину вновь выделенной памяти и построит новый последний элемент в конце этой последовательности. Следующие 99 вызовов функции `push_back` будут удовлетворяться без необходимости обращения за новой порцией памяти.

Такая стратегия подразумевает, что нам придется изменить способ ведения массива, содержащего наши элементы. Нам по-прежнему нужно отслеживать первый элемент, но теперь необходимо иметь два “конечных” указателя. Один из них будет указывать на элемент, расположенный за последним элементом действующего массива, т.е. на первый доступный (при необходимости наращивания объекта) элемент. Второй указатель должен указывать на “элемент”, расположенный за последним элементом, для которого была выделена память. Итак, наши `Vec`-объекты будут выглядеть таким образом.



Наши функции `size` и `end` должны быть переписаны в расчете на использование нового члена данных `avail`. Этот новый член должна использовать как функция

push_back, так и еще ненаписанные функции управления памятью. Более того, сама по себе функция push_back очень проста: она переадресовывает всю тяжелую работу двум функциям управления памятью grow и unchecked_append, которые в конце концов нам придется написать.

```
template <class T> class vec {
public:
    size_type size() const {
        return avail - data; } // изменено.
    iterator end() { return avail; } // изменено.
    const_iterator end() const { return avail; } // изменено.
    void push_back(const T& val) {
        if (avail == limit) // При необходимости
            grow(); // получаем память.
        unchecked_append(val); // присоединяем новый элемент.
    }
private:
    iterator data; // Как и прежде, указатель на первый
                  // элемент в vec-объекте.
    iterator avail; // указатель на следующий элемент после
                  // элемента, созданного последним.
    iterator limit; // указатель на следующий элемент после
                  // последнего доступного элемента.
    // Остальная часть разделов интерфейса и реализации
    // класса остается в силе.
};
```

11.5. Гибкое управление памятью

В отношении нашего vec-класса мы уже отмечали, что не хотели бы для управления памятью использовать встроенные операции new и delete. Дело в том, что если бы мы полагались на эти операции, наш класс vec был бы более ограничен, чем стандартный класс vector. Оператор new слишком много “берет на себя”: он выделяет память и инициализирует ее. При его использовании для размещения в памяти массива типа T потребовался бы конструктор по умолчанию класса T. Данный подход не позволяет предложить пользователям такую степень гибкости, какую мы хотели бы обеспечить.

Кроме того, использование оператора new было бы чрезмерно дорогим “удовольствием”, поскольку он всегда инициализирует каждый элемент T-массива посредством выражения T::T(). Если бы мы захотели инициализировать элементы vec-объекта самостоятельно, нам пришлось бы инициализировать каждый элемент дважды: сначала посредством оператора new, а затем еще раз, устанавливая значение, предложенное пользователем. Положение дел окажется еще хуже, если рассмотреть стратегию распределения памяти, которую мы предполагаем использовать для функции push_back. Эта стратегия подразумевает, что мы должны удваивать размер vec-объекта каждый раз, когда нам потребуется дополнительная память. При этом нет никакого смысла в инициализации пока неиспользуемых элементов, т.е. “заготовленных” впрок или “на вырост” объекта. Они будут обрабатываться лишь функцией push_back, которая “вспомнит” о зарезервированной памяти только в случае, когда нам понадобится разместить в ней новый элемент vec-объекта. Если бы для размещения в памяти базового массива мы использовали оператор new, эти дополнительные элементы были бы инициализированы в любом случае, независимо от того, используем мы их или нет.

Вместо встроенных операторов new и delete, мы можем использовать средства стандартной библиотеки, разработанные для поддержки гибкого управления памятью.

Сам по себе базовый язык не имеет ни малейшего представления о памяти и ее распределении, поскольку свойства памяти слишком непостоянны, чтобы можно было жестко связать их с самим языком.

Например, современные компьютеры имеют несколько видов памяти. Различные запоминающие устройства могут различаться скоростью работы. Существуют устройства памяти, обладающие специальными свойствами, например графические буферы или совместно используемые запоминающие устройства. Есть и такие, на работу которых не влияют возможные нарушения энергоснабжения. Не исключено, что пользователи, возможно, захотят использовать один из таких видов памяти (или какой-то другой), поэтому вопросы распределения памяти и управления ею лучше всего предоставить библиотеке. Стандартная библиотека не поддерживает все эти виды памяти, но она позволяет управлять памятью с помощью интерфейса, единого для различных распределителей памяти. Решение сделать управление памятью частью библиотеки (вместе с решением сделать ввод-вывод данных библиотечным, а не встроенным средством) предоставляет нам большую гибкость в использовании различных видов памяти.

Заголовок `<memory>` позволяет использовать класс, именуемый `allocator<T>`, который выделяет блок неинициализированной памяти для размещения в ней объектов типа `T` и возвращает указатель на начальный элемент этой памяти. Такие указатели опасны, поскольку их тип предполагает, что они указывают на объекты, но память на самом деле их еще не содержит. Библиотека также предоставляет возможность создавать объекты в этой памяти с последующим их разрушением, не отказываясь при этом от самой памяти. Используя класс `allocator`, программист должен сам отслеживать, в какой части памяти содержатся созданные им объекты, а какая еще не инициализирована.

В классе `allocator` есть весьма интересная для нас часть, которая включает четыре функции-члена и две связанные с ними функции, не являющиеся членами. Рассмотрим список функций-членов этого класса.

```
template<class T> class allocator {
public:
    T* allocate(size_t);
    void deallocate(T*, size_t);
    void construct(T*, T);
    void destroy(T*);
    // ...
};
```

Функция-член `allocate` выделяет типизированную, но неинициализированную память, достаточную для размещения в ней заданного количества элементов. Типизированной мы назвали эту память потому, что в конце концов мы будем использовать ее для хранения значений типа `T` и обращаться к ней с помощью указателей типа `T*`. В то же время мы назвали ее неинициализированной, поскольку в ней еще не создано ни одного объекта.

Функция-член `deallocate` освобождает неинициализированную память. Она принимает указатель на область памяти, которая была выделена функцией-членом `allocate`, и размер, означающий, сколько элементов было размещено тогда в памяти.

Функция-член `construct` создает в этой неинициализированной памяти один объект. В качестве аргументов функции `construct` передаются указатель на область памяти, выделенную функцией `allocate`, и значение, подлежащее копированию в эту область. Функция `destroy` разрушает объект типа `T`, на который указывает передаваемый ей аргумент. Для этого объекта она вызывает деструктор класса `T`, в результате чего освобождаемая им память снова становится неинициализированной.

А вот список функций-не-членов, имеющих отношение к классу `allocator`.

```
template<class In, class For>
    For uninitialized_copy(In, In, For);
template<class For, class T>
    void uninitialized_fill(For, For, const T&);
```

Эти функции создают и инициализируют новые объекты в памяти, которая была предварительно выделена функцией `allocate`. Параметр-тип `In` — это тип входного итератора (см. раздел 8.2.2), а `For` — это тип однонаправленного итератора (см. раздел 8.2.4), которым обычно является указатель. Создание нового объекта — это нечто большее, чем просто присваивание ему значения, поэтому `For` должен быть типом однонаправленного итератора, а не просто типом выходного итератора.

Функция `uninitialized_copy` сходна с библиотечной функцией `copy`: она копирует значения из последовательности, заданной первыми двумя аргументами, в последовательность, заданную ее третьим аргументом. Функция `uninitialized_fill` создает столько копий своего третьего аргумента, сколько потребуется для заполнения области памяти, заданной первыми двумя аргументами.

При использовании типа `allocator<T>` компилятор, как и в случае любого шаблона, сгенерирует за вас соответствующий класс `allocator`. Чтобы получить класс `allocator`, который можно использовать для размещения и освобождения объектов типа `T`, в наш класс `vec<T>` необходимо добавить член класса `allocator<T>` с именем `аллос`. Добавив этот член и используя соответствующие библиотечные функции, мы сможем обеспечить эффективное и гибкое (до определенной степени) управление памятью, подобное тому, которое предоставляет стандартный класс `vector`.

11.5.1. Конечный вариант класса *Vec*

Наш законченный класс `vec`, включающий объявления, но не определения функций управления памятью, выглядит следующим образом.

```
template <class T> class vec {
public:
    typedef T* iterator;
    typedef const T* const_iterator;
    typedef size_t size_type;
    typedef T value_type;
    typedef T& reference;
    typedef const T& const_reference;

    vec() { create(); }
    explicit vec(size_type n, const T& t = T()) { create(n, t); }
    vec(const vec& v) { create(v.begin(), v.end()); }
    vec& operator=(const vec&); //Как определено в разделе 11.3.2.
    ~vec() { uncreate(); }

    T& operator[](size_type i) { return data[i]; }
    const T& operator[](size_type i) const { return data[i]; }

    void push_back(const T& t) {
        if (avail == limit)
            grow();
        unchecked_append(t);
    }

    size_type size() const { return avail - data; } // Изменено.

    iterator begin() { return data; }
    const_iterator begin() const { return data; }
```



```

    iterator end() { return avail; }           // Изменено.
    const_iterator end() const { return avail; } // Изменено.

private:
    iterator data; // первый элемент в объекте vec.
    iterator avail; // Элемент, следующий за последним
                   // реальным элементом в объекте vec.
    iterator limit; // Элемент, следующий за последним
                   // элементом выделенной памяти.

    // Средства выделения памяти.
    allocator<T> alloc; // объект, управляющий выделением
                       // памяти.

    // Средства размещения в памяти базового массива и его
    // инициализации.
    void create();
    void create(size_type, const T&);
    void create(const_iterator, const_iterator);

    // Средства разрушения элементов массива и освобождения
    // памяти.
    void uncreate();

    // Поддержка функций для реализации функции push_back.
    void grow();
    void unchecked_append(const T&);
};

```

Теперь нам осталось реализовать private-члены, управляющие выделением памяти. Как только мы напишем эти члены класса, наша программа станет понятнее, если не забывать о том, что всякий раз, когда мы будем располагать действительным объектом класса `vec`, следующие четыре утверждения должны оставаться справедливыми.

1. Член данных `data` указывает на начальный элемент `vec`-объекта, если таковой существует, и равен нулю в противном случае.
2. $data \leq avail \leq limit$.
3. Элементы `vec`-объекта, которые уже созданы, находятся в диапазоне `[data, avail)`.
4. Элементы, которые отсутствуют (еще не созданы), находятся в диапазоне `[avail, limit)`.

Эти утверждения (условия) мы будем называть *инвариантом класса* (class invariant). Много из того, что касалось инвариантов циклов в разделе 2.3.2, мы должны воплотить в инварианте класса, коль уж мы беремся создавать объект этого класса. Если нам это удастся и мы будем уверены, что ни одна из наших функций-членов не опровергнет инвариант класса, можно гарантировать, что инвариант будет всегда истинным.

Обратите внимание на то, что ни один из public-членов не в состоянии сделать ложным инвариант класса, поскольку этого можно добиться только одним способом — путем изменения значений членов данных `data`, `avail` или `limit`, но ни одна из функций-членов этого не делает.

Теперь рассмотрим различные версии функции `create`, которые отвечают за выделение памяти, инициализацию элементов в этой памяти и надлежащую установку указателей. В каждом случае мы инициализируем выделяемую для объекта память, и поэтому после выполнения функции `create` указатели `limit` и `avail` будут всегда равны, т.е. элемент, созданный последним, и элемент, размещенный в памяти последним, — это один и

тот же элемент. Вы должны убедиться в том, что инвариант класса остается истинным после того, как мы выполним любую из следующих функций.

```
template <class T> void vec<T>::create()
{
    data = avail = limit = 0;
}

template <class T> void vec<T>::create(size_type n,
                                     const T& val)
{
    data = alloc.allocate(n);
    limit = avail = data + n;
    uninitialized_fill(data, limit, val);
}

template <class T>
void vec<T>::create(const_iterator i, const_iterator j)
{
    data = alloc.allocate(j - i);
    limit = avail = uninitialized_copy(i, j, data);
}
```

Версия функции `create`, которая не принимает аргументов, создает пустой `vec`-объект, поэтому ее работа состоит в том, чтобы все указатели начинали свою “жизнь” с нулевых значений.

Версия, которая принимает размер и значение, использует размер для выделения соответствующего объема памяти. Член `allocate` класса `allocator<T>` выделяет область памяти, достаточную для хранения заданного количества объектов типа `T`. Следовательно, при вызове функции `alloc.allocate(n)` выделяется область памяти, достаточная для хранения `n` объектов. Функция `allocate` возвращает указатель (сохраняемый в члене данных `data`) на начальный элемент. Память, выделенная функцией `allocate`, не инициализирована, поэтому мы осуществляем ее инициализацию посредством вызова функции `uninitialized_fill`, которая копирует свой третий аргумент в последовательность неинициализированных элементов, заданную ее первыми двумя аргументами. По завершении этой функции новые элементы будут размещены в области, выделенной функцией `allocate`, и все эти элементы будут инициализированы значением `val`.

Последняя версия функции `create` действует аналогично двум предыдущим, за исключением того, что для инициализации памяти, выделенной функцией `allocate`, она вызывает функцию `uninitialized_copy`. Эта функция копирует элементы последовательности, обозначенной ее первыми двумя аргументами, в приемную последовательность, заданную ее третьим аргументом. Она возвращает указатель на следующий элемент, расположенный за последним инициализированным элементом, и в точности представляет собой значение для членов данных `limit` и `avail`.

Функция-член `uncreate` предназначена для аннулирования последствий работы функций-членов `create`: она должна вызвать деструкторы, чтобы разрушить элементы `vec`-объекта, и вернуть занимаемую ими память.

```
template <class T> void vec<T>::uncreate()
{
    if (data) {
        // Разрушаем (в обратном порядке) созданные ранее
        // элементы.
        iterator it = avail;
        while (it != data)
            alloc.destroy(--it);
    }
}
```

```

        // Возвращаем всю выделенную ранее память.
        alloc.deallocate(data, limit - data);
    }
    // Восстанавливаем указатели, означающие,
    // что vec-объект снова пустой.
    data = limit = avail = 0;
}

```

Если значение указателя `data` равно нулю, никаких действий не выполняется. Если бы мы использовали оператор `delete`, то нам не нужно было бы беспокоиться о сравнении `data` с нулем, поскольку выполнение `delete` для нулевого указателя совершенно безопасно. Но в отличие от `delete`, для выполнения функции `alloc.deallocate` требуется ненулевой указатель, даже если память при этом не освобождается. Поэтому мы и должны проверить значение `data` на равенство нулю.

Если нам все же предстоит работа, мы перемещаем итератор `it` по всем элементам `vec`-объекта, вызывая функцию `destroy` для разрушения каждого элемента по очереди. Мы проходим `vec`-объект в обратном направлении, что напоминает поведение оператора `delete[]`, который также разрушает элементы в обратном порядке. Разрушив элементы, с помощью функции `deallocate` мы освобождаем всю занимаемую ими память. Эта функция принимает указатель на первый элемент освобождаемой памяти и целое значение, сообщаящее, сколько элементов типа `T` должно быть освобождено. Поскольку мы хотим освободить всю ранее выделенную для `vec`-объекта память, отдаем функции `deallocate` “на утилизацию” память, “отмеченную” итераторами `data` и `limit`.

Наконец, мы должны реализовать члены, используемые функцией `push_back`.

```

template <class T> void vec<T>::grow()
{
    // При увеличении объекта выделяем область памяти, вдвое
    // большую используемой в данный момент.
    size_type new_size = max(2 * (limit - data), ptrdiff_t(1));

    // Выделяем новую область памяти и копируем в нее
    // существующие элементы.
    iterator new_data = alloc.allocate(new_size);
    iterator new_avail = uninitialized_copy(data,
                                           avail, new_data);

    // Освобождаем старую память.
    uncreate();

    // Переводим указатели “на рельсы” новой области памяти.
    data = new_data;
    avail = new_avail;
    limit = data + new_size;
}

// Предполагаем, что указатель avail указывает на выделенную,
// но еще неинициализированную область памяти.
template <class T> void vec<T>::unchecked_append(const T& val)
{
    alloc.construct(avail++, val);
}

```

Назначение функции `grow` — выделить область памяти, достаточную для хранения по крайней мере еще одного элемента. Но она выделяет больше, чем необходимо в данный момент, чтобы последующие вызовы функции `push_back` могли использовать этот избыток памяти, позволяя тем самым избежать расходов системных ресурсов, связанных с частыми запросами на выделение памяти. В разделе 11.4 отмечалось, что наша стратегия — удваивать объем памяти при переходе к каждой заново выделяемой области памяти. Ко-

нечно же, `Vec`-объект может в данный момент быть пустым, и как раз для такого случая предусматривается возможность (с помощью функции `max`) выделения максимального значения из двух вариантов. Этими вариантами являются всего лишь один элемент и удвоенный объем существующей памяти. Вспомните из раздела 8.1.3, что два аргумента, передаваемые функции `max`, должны иметь абсолютно одинаковый тип, поэтому мы явно создаем одноэлементный объект типа `ptrdiff_t`, а это как раз тот самый тип (см. раздел 10.1.4), который имеет выражение `limit - data`.

Вспомните, что переменная `new_size` содержит количество элементов, в расчете на которое мы и будем запрашивать новую область памяти. Выделив память соответствующего объема, мы вызываем функцию `uninitialized_copy` для копирования элементов из текущей области памяти в заново выделенную. Затем с помощью функции `uncreate` освобождаем старую область памяти и разрушаем занимавшие ее элементы. Наконец, мы переустанавливаем указатели: значение `data` должно теперь указывать на первый элемент массива, заново размещенного в памяти, `avail` — на элемент, расположенный за последним реально созданным элементом в `Vec`-объекте, а `limit` — на элемент, расположенный за последним размещенным в памяти, но еще не инициализированным элементом.

Обратите внимание на то, что мы запоминаем значения, возвращаемые функциями `allocate` и `uninitialized_copy`. Если бы мы использовали эти значения для немедленной переустановки указателей `data` и `limit`, то последующий вызов функции `uncreate`, вместо того чтобы избавить нас от старой области памяти, разрушил бы данные и освободил бы только что выделенную область памяти!

Функция `unchecked_append` создает элемент в первой ячейке памяти, расположенной за уже созданными элементами. Предполагается, что `avail` указывает на область памяти, которая была выделена, но еще не использована для хранения элементов. Поскольку мы вызываем функцию `unchecked_append` лишь сразу после обращения к функции `grow`, знаем, что этот вызов безопасен.

11.6. Резюме

Шаблонные классы можно сформировать с помощью спецификатора `template`, описанного в разделе 8.1.1.

```
template <class Параметр-тип [, class Параметр-тип] ... >
class Имя_класса { ... };
```

Этот синтаксис создает класс `Имя_класса`, точная реализация которого зависит от заданных элементов `Параметр-тип`. Имена элементов `Параметр-тип` могут быть использованы внутри шаблона везде, где нужно обозначение типа. Внутри области видимости класса к шаблонному классу можно обращаться без уточняющей спецификации, а вне ее элемент `Имя_класса` должен быть специфицирован с указанием параметров-типов.

```
template <class T>
Vec<T>& Vec<T>::operator=(const Vec&) { ... }
```

Реальные типы пользователи указывают при создании объектов шаблонных типов; например, при создании объекта типа `Vec<int>` C++-среда реализует версию `Vec`-шаблона, в которой элемент `Параметр-тип` соответствует типу `int`.

Управление копированием. В общем случае классы управляют событиями, происходящими при создании, копировании, присваивании или разрушении объектов. Кон-

структуры вызываются в качестве побочного эффекта при создании или копировании объектов; оператор присваивания выполняется в выражениях, включающих операцию присваивания; наконец, деструктор выполняется автоматически, когда объекты разрушаются или выходят за область видимости.

Классы, которые распределяют ресурсы в конструкторе, практически без исключений должны определить конструктор копирования, оператор присваивания и деструктор. При написании оператора присваивания важно выявить случай самоприсваивания. Считается хорошим стилем программирования, когда определяемый вами оператор присваивания, подобно встроенным, возвращает ссылку на левый операнд.

Синтезированные операции. Если в классе не определен ни один конструктор, компилятор синтезирует конструктор по умолчанию. Если класс не определяет их явным образом, компилятор синтезирует конструктор копирования, оператор присваивания и/или деструктор. Синтезированные операции определяются рекурсивно: каждый синтезированный оператор рекурсивно применяет соответствующую операцию к членам данного класса.

Перегруженные операторы определяются посредством использования функции с именем *operator Op*, где элемент *Op* — определяемый оператор. По крайней мере один параметр, принимаемый операторной функцией, должен иметь тип класса. Когда операторная функция является членом класса, ее левый операнд (если это бинарный оператор) или ее единственный операнд (если это унарный оператор) связан с объектом, для которого он вызывается. Оператор доступа по индексу и оператор присваивания должны быть членами класса.

Упражнения

- 11.0. Скомпилируйте, выполните и протестируйте программы, приведенные в этой главе.
- 11.1. В структуре `Student_info`, которую мы определили в главе 9, не определен конструктор копирования, оператор присваивания и деструктор. Почему?
- 11.2. В той же структуре не определен конструктор по умолчанию. Почему?
- 11.3. Что делает синтезированный оператор присваивания для объектов типа `Student_info`?
- 11.4. Сколько членов разрушает синтезированный деструктор объектов типа `Student_info`?
- 11.5. Оснастите класс `Student_info` средством подсчета частоты создания объектов, их копирования, присваивания и разрушения. Используйте этот модернизированный класс для выполнения программы вычисления итоговых оценок студентов из главы 6. Это позволит вам узнать, сколько копий создается библиотечными алгоритмами. Сравнение числа копий позволит оценить различия в издержках, связанных с использованием каждого библиотечного класса. Выполните это задание и проанализируйте полученные результаты.
- 11.6. Добавьте операцию удаления элемента из объекта типа `vec` и операцию очистки всего `vec`-объекта. Эти операции должны быть аналогичны векторным операциям `erase` и `clear`.
- 11.7. После внесения в класс `vec` операций `erase` и `clear` его можно использовать вместо класса `vector` в большинстве приведенных выше программ этой книги. Перепишите `Student_info`-программы из главы 9 и программы, которые рабо-

тают с символьными изображениями из главы 5, с ориентацией на использование класса `Vec` вместо стандартного класса `vector`.

- 11.8. Напишите упрощенную версию стандартного класса `list` и соответствующего ему итератора.
- 11.9. Функция `grow` из раздела 11.5.1 удваивает объем памяти каждый раз, когда запрашивается дополнительная память. Оцените эффективность, которая достигается благодаря использованию этой стратегии. Если вы можете предсказать, насколько “ошибается” эта стратегия, измените функцию `grow` соответствующим образом и оцените полученные результаты.

СОЗДАНИЕ ОБЪЕКТОВ КЛАССОВ, ИСПОЛЬЗУЕМЫХ КАК ЗНАЧЕНИЯ

Поведение объектов встроенных типов в общем случае подобно поведению значений: при копировании исходный объект такого типа и его копия имеют одно и то же значение, но во всем остальном они независимы. Последующие изменения, вносимые в один объект, не отражаются на другом. Мы можем создавать объекты таких типов, передавать их функциям и получать в качестве возвращаемого значения, а также копировать их или присваивать другим объектам.

Для большинства встроенных типов в языке C++ определен богатый набор операторов и предусмотрена возможность автоматического преобразования логически подобных типов. Например, если мы сложим `int`- и `double`-значения, компилятор автоматически преобразует `int`-значение в `double`.

При определении наших собственных классов мы управляем условиями, при которых результирующие объекты ведут себя как значения. В главах 9 и 11 было показано, что именно автор класса управляет событиями, которые происходят, когда объекты создаются, копируются, присваиваются и разрушаются. Определяя соответствующим образом конструктор копирования и оператор присваивания, автор класса может так организовать объекты класса, что они будут использоваться как значения. Другими словами, автор класса может позаботиться о том, чтобы каждый объект имел состояние, независимое от любого другого объекта. Наши классы `Vec` и `Student_info` — это примеры типов, объекты которых действуют как значения.

В этой главе мы покажем, что автор класса может также управлять преобразованиями и операциями, выполняемыми над объектами класса, создавая таким образом классы, объекты которых ведут себя подобно объектам встроенных типов. Класс стандартной библиотеки `string` — хороший пример такого типа (благодаря его богатому набору операторов и поддержке автоматического преобразования типов). И поэтому в данной главе мы определим упрощенную версию библиотечного класса `string` (с именем `Str`) во многом аналогично тому, как мы определяли упрощенную версию библиотечного класса `vector` в главе 11. На этот раз мы сфокусируем наше внимание на операторах и преобразованиях, которые позволяют писать выражения, включающие `string`-объекты. В этой главе мы не будем утруждать себя вопросами эффективности. А когда подойдем к главе 14, то вновь займемся классом `Str`, чтобы освоить методы более эффективного управления хранением информации, связанной с каждым объектом класса `Str`.

Нам не придется слишком беспокоиться о деталях реализации нашего класса `Str`, поскольку большую часть этой работы мы уже проделали при реализации класса `Vec`.

Поэтому основное внимание в этой главе будет уделено разработке соответствующего интерфейса для нашего класса.

12.1. Простой класс string

Начнем с определения класса Str, которое позволит нам создавать объекты нужного нам поведения.

```
class Str {
public:
    typedef Vec<char>::size_type size_type;

    // конструктор по умолчанию создает пустой Str-объект.
    Str() { }

    // Создаем Str-объект, содержащий n копий символа c.
    Str(size_type n, char c): data(n, c) { }

    // Создаем Str-объект из массива char-элементов
    // с завершающим нуль-символом.
    Str(const char* cp) {
        std::copy(cp, cp + std::strlen(cp),
                  std::back_inserter(data));
    }

    // Создаем Str-объект из диапазона, обозначенного
    // итераторами b и e.
    template<class In> Str(In b, In e) {
        std::copy(b, e, std::back_inserter(data));
    }

private:
    Vec<char> data;
};
```

Наш класс делегирует работу по управлению данными классу Vec, который мы написали в главе 11. Этот класс почти всегда подходит для поддержки объектов нашего Str-класса, но ему недостает только функции clear, которую в главе 11 было предложено написать читателям в качестве упражнения.

Класс Str содержит четыре конструктора, каждый из которых предназначен для создания члена данных data посредством соответствующей инициализации Vec-объекта.

Конструктор по умолчанию класса Str неявно вызывает конструктор по умолчанию класса Vec, чтобы создать пустой Str-объект. Обратите внимание на то, что поскольку наш класс содержит другие конструкторы, мы должны явно определить конструктор по умолчанию, несмотря на то что он делает в точности то же самое, что и синтезированный конструктор по умолчанию. Другие три конструктора принимают значения, которые мы используем для создания и инициализации данных.

Конструктор, который принимает размер и символ, для создания члена данных data использует соответствующий Vec-конструктор. Этому конструктору больше, собственно, и нечего делать, поэтому его тело пустое.

Последние два конструктора подобны. Их инициализаторы пусты; это значит, что член data неявно инициализируется как пустой Vec-объект. Каждый конструктор вызывает функцию sru для присоединения заданных символов к изначально пустому члену data. Например, конструктор, который принимает аргумент типа const char*, использует функцию strlen, чтобы определить длину строки. “Узнав” длину строки, он вычисляет два итератора, которые указывают на входные символы, и использует

функции `copy` и `back_inserter` для присоединения этих символов к `data`. Таким образом, в результате выполнения этого конструктора член данных `data` будет содержать копии символов, хранящихся в массиве, обозначенном `sr`.

Самый интересный конструктор — последний. Он принимает два итератора и создает новый `Str`-объект, который содержит копию символов в заданной последовательности. Подобно предыдущему конструктору, он использует функции `copy` и `back_inserter` для присоединения к члену `data` значений, указанных в диапазоне `[b, e)`. Этот конструктор, чем он и интересен, представляет собой шаблонную функцию. Являясь шаблоном, он эффективно определяет семейство конструкторов, которые могут быть реализованы для различных типов итераторов. Например, этот конструктор можно было бы использовать для создания `Str`-объекта из массива символов или объекта типа `vec<char>`.

Важно отметить, что в этом классе не определяется конструктор копирования, оператор присваивания и деструктор. Почему?

Ответ прост: здесь используются средства по умолчанию. Сам класс `Str` не занимается выделением памяти. Он может оставить детали управления памятью синтезированным операциям, которые вызывают соответствующие `vec`-операции. Чтобы убедиться в том, что здесь применяются средства по умолчанию, достаточно заметить, что классу `Str` не нужен деструктор. И в самом деле, если бы он был, ему нечего было бы делать. И вообще, классу, которому не нужен деструктор, не нужен ни явно заданный конструктор копирования, ни оператор присваивания (см. раздел 11.3.6).

12.2. Автоматические преобразования

На данный момент мы явно определили некоторый набор конструкторов и неявно — операции копирования, присваивания и деструкции. Эти операции “навязывают” `Str`-объектам поведение, характерное для значений: при копировании `Str`-объекта оригинал и копия будут независимы один от другого. Теперь необходимо подумать о преобразованиях. Значения встроенного типа часто могут быть автоматически преобразованы из одного типа в другой. Например, мы можем инициализировать `double`-значение посредством `int`-значения, а также `int`-значение присвоить `double`-значению.

```
double d = 10; // преобразуем число 10 в double-значение и
               // используем преобразованное значение для
               // инициализации переменной d.
double d2;
d2 = 10;       // преобразуем число 10 в double-значение и
               // присвоим преобразованное значение
               // переменной d2.
```

В случае нашего `Str`-класса мы определили, как построить `Str`-объект из значения типа `const char*`, и поэтому можем записать следующее.

```
Str s("hello"); // Создаем объект s.
```

Это определение создает объект `s` посредством явного вызова конструктора, который принимает аргумент типа `const char*`. Мы хотели бы также иметь возможность выполнять такие инструкции.

```
Str t = "hello"; // инициализируем объект t.
s = "hello";     // Присваиваем объекту s новое значение.
```

Вспомните из раздела 11.3.3, что в последнем примере символ “=” имеет два различных значения. В первой инструкции определяется переменная `t`, поэтому символ “=” означает инициализацию. Для этой формы инициализации всегда требуется конструктор копирования, который принимает в качестве аргумента значение типа `const`

Str&. Вторая инструкция — это инструкция-выражение, а не объявление, поэтому символ “=” означает оператор присваивания. Единственный оператор присваивания, который подходит для Str-объектов, — это такой оператор, который компилятор определил за нас и который также принимает в качестве аргумента значение типа const Str&. Другими словами, каждая инструкция в этом втором примере использует строковый литерал, который имеет тип const char*, но при этом ожидает, что будет передан аргумент типа const Str&.

Отсюда, казалось бы, следует, что нам нужно усилить класс Str дополнительным оператором присваивания с параметром типа const char* и поразмыслить над тем, как перегрузить конструктор копирования. К счастью, оказывается, что нам не нужно этого делать, поскольку уже существует конструктор, который принимает параметр типа const char*, и этот конструктор также действует в качестве *определенного пользователем преобразования* (user-defined conversion). Определенные пользователем преобразования указывают, как преобразовывать объекты типа класса. Как и в случае встроенных преобразований, компилятор будет применять определенные пользователем преобразования, чтобы преобразовать значение заданного типа в значение требуемого типа.

Преобразования в классе можно определять двумя способами: из других типов в “свой” или из “своего” типа в другие. Вторую форму преобразования мы рассмотрим в разделе 12.5. Более общее преобразование определяет, как преобразовать другие типы в тип, который мы определяем. Это реализуется посредством определения конструктора с одним аргументом.

Наш класс Str уже имеет такой конструктор, а именно конструктор, который принимает значение типа const char*. Следовательно, компилятор будет использовать этот конструктор, когда в его распоряжение предоставляется объект типа const char*, а требуется объект типа Str. В присваивании объекта типа const char* объекту типа Str просматривается точно такая же ситуация. Когда компилятор встречает инструкцию s = “hello”;, сначала он использует конструктор Str(const char*) для создания неименованного локального временного объекта типа Str из строкового литерала, а затем вызывает (синтезированный) оператор присваивания класса Str, чтобы присвоить значение этого временно-го объекта объекту s.

12.3. Операции над Str-объектами

Рассмотрим следующий код, в котором используются операторы, выполняющие действия над string-объектами.

```
cin >> s    // Используем оператор ввода для считывания
             // данных в string-объект.
cout << s    // Используем оператор вывода для вывода
             // значения string-объекта.
s[i]        // Используем индекс-оператор для доступа к символу.
s1 + s2     // Используем оператор сложения для конкатенации
             // двух string-объектов.
```

Все представленные выше операторы являются бинарными. Если мы будем определять их как функции, каждая функция должна иметь два параметра, один из которых может задаваться неявно, если данная функция будет определена как член класса. Как было показано в разделе 11.2.4, имена перегруженных операторов формируются посредством присоединения символа оператора к слову operator. Следовательно, operator>> — это имя функции, которое перегружает оператор ввода, operator[] — это имя оператора доступа по индексу и т.д.

Начнем, пожалуй, с оператора доступа по индексу, поскольку в разделе 11.2.4 уже было показано, как реализуется эта операция, и мы знаем, что соответствующая операторная функция должна быть членом класса.

```
class Str {
public:
    // Конструкторы, показанные выше.
    char& operator[](size_type i) { return data[i]; }
    const char& operator[](size_type i) const { return data[i]; }

private:
    vec<char> data;
};
```

Операторы доступа по индексу просто переключают свою работу “на плечи” соответствующих `Vec`-операций. Важно отметить, что, как и для класса `Vec`, мы определяем две версии индекс-оператора: одна будет работать с `const`-, а другая с не-`const`-объектами. Возвращая ссылку на символ, не-`const`-версия предоставляет доступ (для записи) к символу, который она возвращает. А `const`-версия возвращает ссылку на значение типа `const char`, тем самым предотвращая случайную перезапись существующего символа. В целях согласования со стандартным классом `string` наша `const`-версия возвращает значение типа `const char&`, а не просто типа `char`.

А как насчет других операторов? Самое интересное в определении этих функций — решить, должны ли эти операции быть членами класса `Str`. Оказывается, что ответ на этот вопрос охватывает различные аспекты для каждого из рассматриваемых операторов. Сначала разберемся с операторами ввода-вывода, а затем (в разделе 12.3.3) перейдем к оператору конкатенации.

12.3.1. Операторы ввода-вывода

В разделе 9.2.2 мы должны были решить, следует ли делать функцию `compare` членом класса `Student_info`. Мы предположили, что один из способов решения этой задачи — узнать, влияет ли рассматриваемая операция на состояние объекта. Без сомнения, оператор ввода изменяет состояние объекта, ведь мы используем его, прежде всего, для считывания нового значения в уже существующий объект. Поэтому у нас есть основания считать, что оператор ввода должен быть членом класса `Str`. Однако при реализации наших намерений оказалось, что результат не оправдал ожиданий.

Чтобы понять причину неудачи, необходимо вспомнить (см. раздел 11.2.4), как операнды выражения связаны с параметрами перегруженной операторной функции. Для любой бинарной операции левый операнд всегда связан с первым параметром, а правый — со вторым. В случае операторных функций-членов первый параметр (левый операнд) всегда передается функции-члену неявно. Таким образом, инструкция

```
cin >> s;
```

эквивалентна инструкции

```
cin.operator>>(s);
```

которая вызывает перегруженный оператор “>>”, определенный для объекта `cin`. Такое поведение подразумевает, что оператор “>>” должен быть членом класса `istream`.

Конечно же, у нас нет собственного определения класса `istream`, поэтому мы не можем добавить в него эту операцию. Если же сделать функцию `operator>>` членом класса `Str`, то нашим пользователям придется вызывать эту операцию от имени `Str`-объекта. Тогда выходит, что инструкция

```
s.operator>>(cin);
```

которая эквивалентна инструкции

```
s >> cin;
```

попросту копирует соглашения, используемые библиотекой. Отсюда можно заключить, что оператор ввода (и, по аналогии, оператор вывода) должен быть не-членом-функцией.

Теперь мы можем соответствующим образом обновить наш класс Str, добавив в заголовок Str.h объявления операторов ввода-вывода.

```
std::istream& operator>>(std::istream&, Str&); // Добавлено.  
std::ostream& operator<<(std::ostream&, const Str&); // Добавлено.
```

Оператор вывода написать несложно: он итеративно проходит по Str-объекту, выводя данные посимвольно.

```
ostream& operator<<(ostream& os, const Str& s)  
{  
    for (Str::size_type i = 0; i != s.size(); ++i)  
        os << s[i];  
    return os;  
}
```

Единственная загвоздка состоит в том, что при такой реализации мы должны “осчастливить” класс Str функцией size.

```
class Str {  
public:  
    size_type size() const { return data.size(); }  
    // Все остальное как прежде.  
};
```

Несмотря на простую форму записи оператора вывода, все же здесь есть на чем остановиться. На каждой итерации цикла мы вызываем функцию Str::operator[], чтобы прочесть символ, подлежащий выводу. Этот оператор, в свою очередь, вызывает функцию Vec::operator[], чтобы получить реальное значение из базового вектора. Аналогично на каждой итерации цикла мы определяем размер нашего Str-объекта, вызывая функцию s.size(), которая вызывает функцию-член size базового Vec-объекта для определения его размера.

12.3.2. “Друзья”

Оператор ввода написать ненамного сложнее, чем оператор вывода. Его задача — считывать символы из входного потока и запоминать их. Каждый раз, когда мы вызываем оператор ввода, он должен считать и отбросить любой ведущий пробел (или символ, подобный пробелу), а затем считывать и сохранять символы до тех пор, пока не встретится хвостовой пробел или признак конца файла. Наш оператор ввода несколько упрощен: он игнорирует некоторые тонкости библиотеки ввода-вывода, рассмотрение которых выходит за рамки этой книги, но при этом делает все, что нам нужно.

```
// Этот код пока не скомпилируется.  
istream& operator>>(istream& is, Str& s)  
{  
    // Удаляем существующее значение (значения).  
    s.data.clear();  
  
    // Считываем и отбрасываем ведущий пробел или подобный  
    // ему символ.
```

```

char c;
while (is.get(c) && isspace(c))
    ; // ничего не выполняется, за исключением
      // проверки условия.

// Если есть что считывать, делаем это до тех пор, пока не
// встретится следующий пробел или подобный ему символ.
if (is) {
    do s.data.push_back(c); // Ошибка компиляции!
                           // (data - это private-член)
    while (is.get(c) && !isspace(c));

    // Если прочитаем пробел или подобный ему символ,
    // вернем его в поток.
    if (is)
        is.unget();
}
return is;
}

```

Сначала рассмотрим эту функцию, а затем разьясим, почему она не компилируется.

Оператор ввода начинает свою работу с отмены любого предыдущего значения, которое мог бы иметь член `data`, поскольку в процессе считывания данных в `Str`-объект должны быть стерты любые хранимые там “старые” данные. Затем нам нужно из данного потока выполнять посимвольное считывание информации до тех пор, пока мы не встретим символ, который не является пробелом или подобным ему символом. Поскольку нам нужно уметь обнаруживать, был ли прочитан пробел или подобный ему символ, мы используем во входном потоке функцию `get`. В отличие от перегруженных операторов “>>”, которые игнорируют пробелы, функция `get` считывает и возвращает любой следующий символ в потоке, включая пробел. Следовательно, цикл `while` считывает символы до тех пор, пока не будет обнаружен символ, отличный от пробела или подобного ему символа, либо не будут исчерпаны входные данные. Если прочитанный нами символ окажется пробелом, нам ничего не остается другого, как прочитать следующий символ, поэтому тело `while`-цикла пустое.

Инструкция `if` позволяет узнать, по какой причине мы вышли из `while`-цикла: из-за считывания отличного от пробела символа или просто потому, что исчерпались входные данные. В первом случае мы должны считывать символы (присоединяя каждый из них к уже прочитанным) до тех пор, пока снова не попадется пробел или подобный ему символ. Именно это и делается в следующей инструкции, которая представляет собой `do-while`-цикл (см. раздел 7.4.4). Этот цикл присоединяет к данным символ, который был считан в предыдущем `while`-цикле, а затем продолжает считывание символов до тех пор, пока данные не исчерпаются или не встретится пробел (либо подобный ему символ). При каждом считывании любого отличного от пробела символа вызывается функция `push_back`, присоединяющая его к считанным ранее данным.

Итак, из `do-while`-цикла мы могли “выпасть” либо потому, что не можем больше ничего прочитать из потока `is`, либо потому, что прочитали пробел. Если имел место второй вариант (прочитанный пробел), значит, мы прочитали на один символ больше, чем нужно, и поэтому должны вернуть его обратно во входной поток, вызвав функцию `is.unget()`. Функция `unget` отменяет результат, достигнутый самым последним выполнением функции `get`, путем обратного перемещения на один символ входного потока. После вызова функции `unget` поток находится в таком состоянии, как будто предыдущее выполнение функции `get` никогда не имело места.

Как видно из комментария, этот код не “выдержит” компиляции. Дело в том, что функция `operator>>` не является членом класса `Str`, поэтому она не может получить доступ к члену данных объекта `s`. С подобной проблемой мы столкнулись в разделе 9.3.1, когда функции `compare` нужен был доступ к члену `name` объектов `Student_info`. Эту проблему мы решили тогда введением функции доступа. В данном случае недостаточно предоставить доступ для чтения данных: оператору ввода нужно иметь возможность записывать данные, а не просто читать их. Оператор ввода — это часть нашей общей абстракции `Str`, поэтому было бы прекрасно предоставить ей доступ для записи данных. С другой стороны, мы не намерены предоставлять такой доступ (для записи данных) всем пользователям, а значит, не можем решить нашу проблему добавлением `public`-члена, который позволил бы функции `operator>>` (а значит, и любому пользователю) записывать данные.

Вместо того чтобы добавлять `public`-функцию доступа, мы можем объявить, что оператор ввода — это “друг” (`friend`) класса `Str`. “Друг” класса (`friend`) имеет такие же права доступа, как и сам член класса. Возведя оператор ввода в ранг “друга”, мы можем позволить ему, наряду с другими нашими функциями-членами, получить доступ к `private`-членам класса `Str`.

```
class Str {  
    friend std::istream& operator>>(std::istream&, Str&);  
    // Все остальное остается в силе.  
};
```

Как видите, мы добавили в класс `Str` объявление `friend`. Согласно этому объявлению, версия функции `operator>>`, которая принимает аргументы типа `istream&` и `Str&`, может беспрепятственно получать доступ к `private`-членам `Str`-объекта. После внесения этого объявления в класс `Str` наш оператор ввода нормально скомпилируется.

`friend`-объявление может находиться в любом месте определения класса, независимо даже от расположения меток `private` или `public`. Поскольку `friend`-функция имеет специальные привилегии доступа, она является частью интерфейса класса. Следовательно, имеет смысл располагать `friend`-объявления в начале определения класса, рядом с его `public`-интерфейсом.

12.3.3. Другие бинарные операторы

Теперь нам осталось реализовать в классе `Str` оператор “+”. Прежде чем мы сможем сделать это, нам необходимо принять ряд решений. Должен ли этот оператор быть членом? Какие типы будут иметь его операнды? Значение какого типа должен возвращать этот операнд? Как будет показано ниже, эти вопросы (вернее, ответы на них) могут иметь важные последствия.

Пока попробуем высказать некоторые предположения относительно возможных вариантов ответов. Во-первых, мы знаем, что нам нужно иметь возможность конкатенировать значения типа `Str`. Во-вторых, мы можем отметить, что конкатенация не изменяет значения ни одного операнда. Стало быть, и нет веской причины делать этот оператор функцией-членом. Наконец, мы знаем, что должны быть способны объединять несколько конкатенаций в единое выражение, чтобы имели право на существование выражения, подобные следующему.

```
s1 + s2 + s3
```

Здесь все операнды `s1`, `s2` и `s3` имеют тип `Str`. Такая форма предполагает, что оператор “+” должен возвращать значение типа `Str`.

Эти решения подразумевают, что мы должны реализовать конкатенацию не как член класса.

```
Str operator+(const Str&, const Str&);
```

Прежде чем заняться реализацией, подумаем немного о том, что если уж предлагать пользователям функцию `operator+`, то нужно так же поступить и с функцией `operator+=`. Другими словами, мы хотели бы предоставить нашим пользователям возможность присваивания объекту `s` значения, полученного в результате конкатенации объектов `s` и `s1`, в любой из следующих форм.

```
s = s + s1;  
s += s1;
```

Оказывается, что самый удобный способ реализовать операторную функцию `operator+` — сначала реализовать `operator+=`. В отличие от простого оператора конкатенации (`operator+`), его составная версия (`operator+=`) изменяет свой левый операнд, поэтому мы делаем ее членом класса `Str`. После добавления определений для новых операций конкатенации, наш класс `Str` принимает следующий вид.

```
class Str {  
    // Оператор ввода из раздела 12.3.2.  
    friend std::istream& operator>>(std::istream&, Str&);  
public:  
    Str& operator+=(const Str& s) {  
        std::copy(s.data.begin(), s.data.end(),  
                  std::back_inserter(data));  
        return *this;  
    }  
  
    // Остальное остается в силе.  
    typedef Vec<char>::size_type size_type;  
  
    Str() { }  
    Str(size_type n, char c): data(n, c) { }  
    Str(const char* cp) {  
        std::copy(cp, cp + std::strlen(cp),  
                  std::back_inserter(data));  
    }  
    template<class In> Str(In i, In j) {  
        std::copy(i, j, std::back_inserter(data));  
    }  
  
    char& operator[](size_type i) { return data[i]; }  
    const char& operator[](size_type i) const { return data[i]; }  
    size_type size() const { return data.size(); }  
  
private:  
    Vec<char> data;  
};  
  
// Оператор вывода из раздела 12.3.2.  
std::ostream& operator<<(std::ostream&, const Str&);  
  
Str operator+(const Str&, const Str&);
```

Поскольку в качестве нашего базового типа мы используем тип `Vec`, реализация функции `operator+=` тривиальна: мы вызываем функцию `copy`, чтобы присоединить копию правого операнда к `Vec`-объекту, который представляет собой левый операнд. Как и в случае присваивания, в качестве результата мы возвращаем ссылку на левый объект.

Вот теперь, опираясь на функцию `operator+=`, ничего не стоит реализовать функцию `operator+`.

```
Str operator+(const Str& s, const Str& t)
{
    Str r = s;
    r += t;
    return r;
}
```

Вспомните, что конкатенация выражается функцией-не-членом, которая должна создать новый объект типа `Str`. Этот новый `Str`-объект мы создаем с помощью локальной переменной с именем `r`, инициализируемой копией объекта `s`. (При инициализации используется конструктор копирования класса `Str`.) После этого мы вызываем оператор `“+=”` для конкатенации объекта `r` с объектом `t`, а затем в качестве результата возвращаем значение `r` (и снова-таки посредством неявного вызова конструктора копирования).

12.3.4. Выражения смешанного типа

Мы определили оператор конкатенации, принимающий операнды типа `const Str&`. А как быть с выражениями, которые включают указатели на символы, например, если мы захотим использовать наш класс `Str` для реализации программы из раздела 1.2? Та программа содержала код следующего вида.

```
const std::string greeting = "Привет, " + name + "!";
```

Здесь `name` представляет собой `string`-объект. Аналогично мы хотели бы иметь возможность записать такую инструкцию.

```
const Str greeting = "Привет, " + name + "!";
```

А здесь `name` — объект класса `Str`.

Мы знаем, что оператор `“+”` левоассоциативный, а это значит, что вычисление предыдущего выражения эквивалентно вычислению выражения

```
"Привет, " + name
```

и применению оператора `“+”` к результату и операнду `“!”`. Другими словами, это выражение эквивалентно следующему выражению.

```
("Привет, " + name) + "!"
```

После разбивки этого выражения на компоненты становится понятно, что мы имеем две различные формы оператора `“+”`. В одном случае мы передаем строковый литерал в качестве первого операнда и `Str`-объект в качестве второго. В другом случае левым операндом является `Str`-объект, полученный в результате конкатенации, а правым — строковый литерал. Таким образом, в каждом случае мы вызываем оператор `“+”` для операндов типа `const char*` и `Str`, переданных в разном порядке.

В разделе 12.3.3 мы определили оператор `“+”` для аргументов типа `Str`, а не `const char*`. Но из раздела 12.2 мы знаем, что, определяя конструктор, который принимает аргумент типа `const char*`, мы также определяем оператор преобразования значения типа `const char*` в значение типа `Str`. Значит, наш класс `Str` справится и с этими выражениями. В каждом случае компилятор будет преобразовывать аргумент типа `const char*` в значение типа `Str`, а затем вызывать функцию `operator+`.

Важно понимать последствия операций преобразования. Например, инструкция

```
Str greeting = "Привет, " + name + "!";
```


наделит переменную `greeting` таким же значением, как и в случае выполнения следующего кода.

```
Str temp1("Привет, "); // Str::Str(const char*)
Str temp2 = temp1 + name; // operator+(const Str&,
                          // const Str&)
Str temp3("!") // Str::Str(const char*)
Str greeting = temp2 + temp3; // operator+(const Str&,
                              // const Str&)
```

Глядя на все эти временные объекты, нетрудно предположить, какие немалые затраты может повлечь за собой вариант решения рассматриваемой задачи с использованием этих объектов. Поэтому на практике, в коммерческих реализациях библиотечного класса `string`, из-за довольно ощутимых затрат на создание временных переменных, вместо автоматического преобразования, часто выбирается (хотя и более утомительный) вариант определения различных версий оператора конкатенации для каждой комбинации операндов.

12.3.5. Разработка бинарных операторов

Важно оценить роль преобразований в разработке бинарных операторов. Если некоторый класс поддерживает преобразования, то считается хорошим стилем программирования определить бинарные операторы как функции-не-члены. Тем самым мы сохраняем симметрию между операндами.

Если оператор является членом класса, то его левый операнд не может быть результатом автоматического преобразования. Причина такого ограничения состоит в том, чтобы не нагружать компилятор необходимостью проверять каждый тип во всей программе, если программист использует такое выражение, как `x + y`, для того, чтобы узнать, можно ли преобразовать `x` в значение такого типа, в котором определен член с именем `operator+`. Благодаря такому ограничению компилятор (и программист) должен искать нужную функцию `operator+` только среди функций-не-членов и функций-членов класса, объектом которого является `x`.

Левый операнд любого оператора-не-члена и правый операнд любого оператора следуют тем же правилам, что и обычный аргумент функции: операнд может иметь любой тип, который может быть преобразован в тип параметра. Если бинарный оператор сделать функцией-членом, то мы внесем асимметрию в его операнды: правый операнд может быть результатом автоматического преобразования, а левый — нет. Такая асимметрия прекрасно подходит для таких асимметричных по своей сути операторов, как `“+=”`, но в контексте симметричных операндов это может сбивать с толку и способствует проникновению в код ошибок. Практически всегда желательно обрабатывать оба операнда таких операторов эквивалентно, а это можно реализовать только в случае, если сделать этот оператор функцией-не-членом класса.

При определении составных версий бинарных операторов присваивания мы хотим ограничить левый операнд, обязав его иметь тип класса. А что произошло бы в противном случае? Если разрешить преобразования для левого операнда, то мы могли бы преобразовать этот операнд в объект типа класса и присвоить новое значение полученному временному объекту. Поскольку это значение принадлежало бы временному объекту, то, завершив выполнение присваивания, мы не имели бы возможности получить доступ к объекту, которому только что присвоили значение! Следовательно, подобно самому оператору присваивания, все составные операторы присваивания должны быть членами класса.

12.4. Некоторые преобразования просто опасны

Вспомните, что в разделе 11.2.2 мы определили конструктор, который в качестве параметра принимает размер объекта, с ключевым словом `explicit`. Теперь, когда мы знаем, что конструкторы, которые принимают единственный аргумент, определяют преобразования, становится понятно, что происходит, когда мы делаем некоторый конструктор `explicit`-конструктором: тем самым мы велит компилятору использовать этот конструктор только для создания явно заданных объектов. И тогда компилятор не станет применять `explicit`-конструктор для создания объектов, неявно преобразуя операнды в выражениях или вызовах функций.

Чтобы понять, чем `explicit`-конструкторы могут оказаться полезными, предположим, что мы не объявляли конструктор класса `Vec` как `explicit`-конструктор. Теперь у нас была бы возможность неявно построить `Vec`-объект заданного размера. Мы могли бы использовать это неявное преобразование при вызове такой функции, как `frame` из раздела 5.8.1. Вспомните, что эта функция принимает единственный параметр типа `const vector<string>&` и создает символьное изображение в виде рамки, в которую помещается входной вектор слов. Предположим, что функция `frame` вместо входного `vector`-объекта использовала `Vec`-объект, в котором не определен `explicit`-конструктор, и при этом мы должны выполнить следующую инструкцию.

```
vec<string> p = frame(42);
```

Что в этом случае произошло бы? А что, с нашей точки зрения, должно было бы произойти? Более важно то, как пользователь мог бы предвидеть ход событий.

В данном случае произошло бы следующее: пользователь получил бы “пустую” рамочку, “окаймляющую” 42 пустые строки. А разве такой результат он хотел получить? Вероятно, пользователь предполагал, что программа “нарисует” рамочку вокруг числа 42. Подобный вызов, скорее всего, ошибочен, и поэтому наш класс `Vec` (и, конечно же, стандартный класс `vector`) делает конструктор, который принимает целое значение, `explicit`-конструктором.

В общем случае весьма полезно иметь `explicit`-конструкторы, которые определяют структуру создаваемого объекта, а не его содержимое. А конструкторы, аргументы которых становятся частью объекта, обычно не должны объявляться с использованием ключевого слова `explicit`.

Например, классы `string` и `Str` имеют конструкторы, которые принимают единственный аргумент типа `const char*` и при этом не являются `explicit`-конструкторами. Каждый конструктор использует свой `const char*`-аргумент для инициализации значения своего объекта. Если аргумент определяет значение результирующего объекта, имеет смысл в выражениях или вызовах функций разрешить автоматические преобразования значений типа `const char*`.

Однако конструкторы классов `vector` и `Vec`, которые принимают единственный аргумент типа `Vec::size_type`, являются `explicit`-конструкторами. Они используют значение своего аргумента для выяснения, сколько элементов им следует разместить в памяти компьютера. Такой аргумент конструктора определяет структуру объекта, а не его значение.

12.5. Операторы преобразования

В разделе 12.2 мы видели, что некоторые конструкторы могут определять и преобразования. Авторы класса также могут определить явно заданные *операторы преоб-*

зования (conversion operators), которые показывают, как преобразовать объект из одного типа в заданный. Операторы преобразования должны быть определены как члены класса. Имя оператора преобразования состоит из слова `operator`, за которым следует имя заданного типа. Следовательно, если некоторый класс имеет член с именем `operator double`, то этот член определяет, как создать значение типа `double` из значения типа этого класса. Например, следующий код

```
class Student_info {
public:
    operator double() const;
    // ...
};
```

должен показать, как создать объект типа `double` из объекта типа `Student_info`. Значение этого преобразования зависит от определения оператора, который должен преобразовать объект в соответствующее значение итоговой оценки. Компилятор будет использовать этот оператор преобразования везде, где у нас есть объект типа `Student_info`, но нужен объект типа `double`. Так, например, если бы объект `vs` имел тип `vector<Student_info>`, мы могли бы вычислить среднюю оценку для всех студентов следующим образом.

```
vector<Student_info> vs;
// Заполняем vs.

double d = 0;
for (int i = 0; i != vs.size(); ++i)
    d += vs[i]; // vs[i] автоматически преобразуется
                // в double-значение.

cout << "Средняя оценка: " << d / vs.size() << endl;
```

Операторы преобразования чаще всего используются при преобразовании из типа класса в какой-нибудь встроенный тип, но они также могут быть полезны при преобразовании в тип другого класса, кодом которого мы не располагаем. В любом случае мы не можем добавить конструктор в тип результата преобразования, поэтому оператор преобразования можно определить только как часть класса, владельцем которого мы являемся.

По сути, мы используем этот вид оператора преобразования каждый раз, когда пишем цикл, в котором неявно тестируется `istream`-значение. Как отмечалось в разделе 3.1.1, мы можем применять объект типа `istream` там, где ожидается какое-нибудь условие.

```
if (cin >> x) { /* ... */ }
```

Этот код, как мы видели, эквивалентен следующему.

```
cin >> x;
if (cin) { /* ... */ }
```

Теперь разберемся, что происходит при вычислении этого выражения.

Как мы знаем, инструкция `if` тестирует условие, которое представляет собой выражение, генерирующее значение истинности. Это выражение имеет тип `bool`. Используемое в качестве значения истинности, значение любого арифметического типа или типа указателя автоматически преобразуется в значение типа `bool`, поэтому мы можем использовать значения этих типов в качестве выражения в любом условии. Конечно, тип `istream` — это и не указатель, и не арифметический тип. Однако стандартная библиотека определяет преобразование значения типа `istream` в значение типа `void*`, которое представляет собой указатель на тип `void`. Это реализуется

посредством определения функции `istream::operator void*`, которая тестирует различные флажки состояния, позволяющие узнать, действительно ли данное `istream`-значение, и возвращает либо 0, либо определяемое C++-средой ненулевое `void*`-значение, по которому можно судить о состоянии потока.

Ранее мы не использовали тип `void*`. В разделе 6.2.2 отмечалось, что тип `void` можно применять лишь в нескольких случаях; основа для указателя — это один из них. Указатель на тип `void` иногда называется универсальным, поскольку он может указывать на объект любого типа. Конечно, такой указатель нельзя разыменовывать, поскольку неизвестно, какой тип объекта нужно при этом получить. Но с типом `void*` все же что-то можно сделать, а именно преобразовать в тип `bool`; вот эта возможность здесь и используется.

Причина того, что класс `istream` определяет функцию `operator void*`, а не `operator bool`, заключается в том, чтобы позволить компилятору обнаружить следующее ошибочное использование его объекта.

```
int x;  
cin << x; // мы должны были написать cin >> x;.
```

Если бы в классе `istream` был определен оператор `operator bool`, это выражение использовало бы функцию `istream::operator bool` для преобразования объекта `cin` в объект типа `bool`, преобразовало бы полученное `bool`-значение в значение типа `int`, сдвинуло бы это значение влево на количество разрядов, равное значению `x`, и в итоге отбросило бы результат! Определяя преобразование в значение типа `void*`, а не в значение какого-то арифметического типа, стандартная библиотека по-прежнему позволяет использовать `istream`-объект в качестве условия, но предотвращает его использование в качестве арифметического значения.

12.6. Преобразования и управление памятью

Многие C++-программы взаимодействуют с системами, написанными на C или языке ассемблера, которые для хранения строковых данных используют массивы символов с завершающим нуль-символом. Как было показано в разделе 10.5.2, стандартная библиотека C++ сама использует это преобразование для получения имен входных и выходных файлов. Вследствие этого преобразования мы могли бы заключить, что наш класс `Str` должен обеспечить преобразование `Str`-объекта в массив символов с завершающим нуль-символом. Если бы мы сделали это, наши пользователи могли бы (автоматически) передавать `Str`-объекты функциям, которые обрабатывают массивы символов с завершающим нуль-символом. К сожалению, как будет показано ниже, это приводит к ошибкам управления памятью.

Предполагая преобразование `Str`-объекта в `char*`-объект, мы, возможно, хотели бы предоставить как `const`-, так и `non-const`-версии.

```
class Str {  
public:  
    // Приемлемые, но проблематичные операции преобразования.  
    operator char*(); // Добавлено.  
    operator const char*() const; // Добавлено.  
  
    // Остальное остается в силе.  
private:  
    Vec<char> data;  
};
```

После такой переделки пользователи класса `Str` могли бы написать код, подобный следующему.

```
Str s;  
// ...  
ifstream in(s); // желаемое действие: преобразует s, а затем  
                // открывает stream-объект с именем s.
```

Единственная проблема состоит в том, что эти преобразования практически невозможно удачно реализовать. Мы не можем просто вернуть член `data`, в основном, из-за неверного типа: член данных `data` имеет тип `vec<char>`, а нам нужен массив `char`-элементов. Более важно то, что, даже если бы типы совпадали, возврат члена `data` нарушил бы инкапсуляцию класса `Str`: пользователь, который получил бы указатель на член `data`, мог бы использовать этот указатель, чтобы изменить значение этого `string`-объекта. Теперь рассмотрим, что происходит, когда объект типа `Str` разрушается. Если пользователь попытается применить этот указатель после того, как объект `Str`-класса прекратит свое существование, то указатель будет ссылаться на область памяти, которая уже была возвращена системе и больше не является действительной.

Мы можем решить проблему инкапсуляции, обеспечив лишь преобразование в тип `const char*`, но это не предотвратит разрушения пользователем `Str`-объекта с последующим использованием недействительного указателя. Вторую проблему мы можем решить, выделив новую область памяти для копии символов из члена данных `data` и вернув указатель на эту новую область памяти. Затем пользователь должен сам распорядиться этой областью, освободив ее, когда она перестанет быть нужной.

Как оказалось, и этот проект работать не будет. Преобразования могут выполняться неявно, и в этом случае пользователь не будет иметь указателя, чтобы освободить ненужные объекты! Рассмотрим снова следующий код.

```
Str s;  
ifstream is(s); // Неявное преобразование: как освободить массив?
```

Если бы класс `Str` имел предложенное преобразование, то при передаче аргумента `s` конструктору класса `ifstream` мы бы неявно преобразовывали `Str`-объект в объект типа `const char*`, получая тип, ожидаемый конструктором. Это преобразование потребовало бы выделения новой области памяти для хранения копии значения объекта `s`. Однако явно заданного указателя на эту область памяти не существует, и поэтому пользователь не может освободить ее. Отсюда вывод: стратегия, приводящая к утечке памяти, не может быть приемлемой.

При разработке класса мы стремимся не предоставлять пользователю возможности “споткаться” при написании внешне безопасного кода, который может стать источником неприятностей. В процессе “становления” стандарта C++ многие разработчики библиотеки предлагали различные способы построения `string`-объектов. Некоторые предоставляли неявные преобразования в символьные массивы, но реализовывали это способом, который не мешает пользователям сделать одну или даже обе описанные выше потенциальные ошибки.

Стандартная библиотека `string` использует другой подход: она разрешает пользователям получать копию `string`-объекта в символьном массиве, но заставляет делать это явным образом. Стандартный класс `string` предоставляет три функции-члена для получения `char`-массива из `string`-объекта. Первая, `c_str()`, копирует содержимое `string`-объекта в `char`-массив с завершающим нуль-символом. Класс `string` является владельцем этого массива, и предполагается, что пользователь не будет удалять указатель с помощью `delete`. Данные в этом массиве недолговечны и будут действительны только до следующего вызова

функции-члена, который может изменить `string`-объект. Предполагается, что пользователи должны либо сразу же использовать этот указатель, либо скопировать данные в область памяти, которой они смогут сами управлять. Вторая функция, `data()`, подобна функции `c_str()`, за исключением того, что она возвращает массив, который не завершается нулевым символом. Наконец, функция `copy()` принимает в качестве аргументов значения типа `char*` и `int`, а затем копирует столько символов, сколько задано целочисленным аргументом, в область, заданную аргументом типа `char*` (эту область пользователь должен сначала выделить, а впоследствии освободить). Реализацией этих функций читателю предлагается заняться в качестве упражнения.

Обратите внимание на то, что функции `c_str` и `data` имеют общие “ловушки”, связанные с неявным преобразованием в значение типа `const char*`. Но поскольку пользователи должны обеспечить явное преобразование, им, по всей вероятности, кое-что известно о вызванных ими функциях. Кроме того, они должны знать о ловушках, связанных с хранением копии указателя. Если бы библиотека разрешила неявные преобразования, пользователям было бы легче споткнуться на этих проблемах. Более того, они могли бы даже и не подозревать о выполнении преобразования и поэтому могли не понимать причины неудачного выполнения программы.

12.7. Резюме

Преобразования определяются посредством не-`explicit`-конструкторов, которые принимают единственный аргумент, или посредством оператора преобразования в форме `operator Имя_типа()`, где элемент *Имя_типа* называет тип, в объект которого может быть преобразован объект данного класса. Операторы преобразования должны быть членами класса. Если один класс определяет преобразование в значения типа другого класса, а другой класс — в значения типа первого, результат может быть неопределенным.

friend-объявления могут встречаться в любом месте определения класса и позволяют `friend`-компоненту получать доступ к `private`-членам этого класса.

```
template<class T>
class Thing {
    friend std::istream& operator>>(std::istream&, Thing&);
    // ...
};
```

Как будет показано в разделе 13.4.2, друзьями (`friend`) могут также называться “целые” классы.

Шаблонные функции как члены. Класс в качестве своего члена может иметь шаблонную функцию. Сам класс при этом необязательно должен быть шаблонным. Класс, который содержит шаблонную функцию-член, имеет неограниченное семейство функций-членов с одинаковым именем. Шаблонные функции-члены объявляются и определяются как любые другие шаблонные функции.

Операции с `string`-объектами

`s.c_str()` Генерирует значение типа `const char*`, которое указывает на массив с завершающим нуль-символом. Данные в этом массиве действительны только до тех пор, пока не будет выполнена следующая `string`-операция, которая может модифицировать объект `s`. Пользователь не может удалить указатель с помощью оператора `delete` и не должен хранить его копию, поскольку содержимое, на которое он указывает, имеет ограниченное время существования.

- `s.data()` Аналогично функции `s.c_str()`, но генерируемый массив не имеет завершающего нуля-символа
- `s.copу(p, n)` Копирует `n` символов из объекта `s` в область памяти, на которую указывает символьный указатель `p`. Пользователь несет ответственность за то, чтобы указатель `p` указывал на область, которая достаточна для хранения `n` символов (`n` — целое число)

Упражнения

- 12.0. Скомпилируйте, выполните и протестируйте программы, приведенные в этой главе.
- 12.1. Модифицируйте класс `Str`, но выберите стратегию, которая требует, чтобы этот класс сам управлял распределением памяти. Например, вы можете хранить массив `char`-элементов и длину массива. Подумайте, какие последствия имеет это изменение для управления копированием. Кроме того, оцените затраты на использование класса `Vec` (например, по части расходов ресурсов памяти).
- 12.2. Реализуйте функции `c_str`, `data` и `copy`.
- 12.3. Определите операторы отношений для класса `Str`. Для этого вам необходимо знать, что заголовок `<cstring>` определяет функцию `strcmp`, которая сравнивает два символьных указателя. Эта функция возвращает отрицательное целое значение, если символьный массив с завершающим нуль-символом, заданный первым указателем, меньше второго; нуль — если две строки равны; или положительное значение — если первая строка больше второй.
- 12.4. Определите операторы равенства и неравенства для класса `Str`.
- 12.5. Реализуйте конкатенацию для `Str`-объектов, не полагаясь на преобразования из значений типа `const char*`.
- 12.6. Введите в класс `Str` операцию, которая позволит неявно использовать `Str`-объект в качестве условия. Условие должно считаться невыполненным, если проверяемый `Str`-объект представляет собой пустую строку, и выполненным в противном случае.
- 12.7. Стандартный класс `string` предоставляет итераторы произвольного доступа для обработки символов `string`-объекта. Дополните свой класс `Str` итераторами и итераторными операциями `begin` и `end`.
- 12.8. Добавьте в класс `Str` функцию `getline`.
- 12.9. Используйте класс `ostream_iterator` для модификации оператора вывода `Str`-класса. Почему мы не предлагаем вам модифицировать оператор ввода, используя класс `istream_iterator`?
- 12.10. Глядя на то, как в разделе 12.1 класс `Str` определяет конструктор, который принимает два итератора, мы можем предположить, что такой конструктор был бы полезен и в классе `Vec`. Добавьте этот конструктор в класс `Vec` и переделайте класс `Str` на предмет использования `Vec`-конструктора вместо вызова функции `copy`.
- 12.11. Если вы добавите операции, перечисленные в этих упражнениях, то можете использовать класс `Str` во всех примерах этой книги. Попробуйте это реализовать для символьных изображений из главы 5 и версий функции `split` из разделов 5.6 и 6.1.1.

- 12.12. Определите функцию `insert`, которая принимает два итератора для классов `Vec` и `Str`.
- 12.13. Предоставьте функцию `assign`, которую можно было бы использовать для присваивания значений массива объекту типа `Vec`.
- 12.14. Напишите программу инициализации `Vec`-объекта с помощью `string`-объекта.
- 12.15. Функция `read_hw` из раздела 4.1.3 проверяла поток, из которого она считывала информацию, на предмет обнаружения признака конца файла или недействительного значения. Оператор ввода, определенный в классе `Str`, не выполняет такой проверки. Почему? Оставит ли он поток в этом случае в работоспособном состоянии?

НАСЛЕДОВАНИЕ И ДИНАМИЧЕСКОЕ СВЯЗЫВАНИЕ

Последние несколько глав были посвящены возможности построения собственных типов данных. Эта возможность — одна из основ объектно-ориентированного программирования (object-oriented programming), или ООП. Данная глава — первый шаг к рассмотрению двух иных ключевых компонентов ООП: наследования и динамического связывания.

В главе 9 мы описали небольшой класс, предназначенный для инкапсуляции операций, используемых при решении проблемы вычисления итоговой оценки из главы 4. В этой главе мы возвращаемся к той задаче. На этот раз предположим, что в спецификацию внесено одно изменение: здесь могут быть учтены как студенты, набирающие баллы за базовый университетский курс, так и выпускники. Для категории студентов-выпускников требуется, чтобы они выполнили некоторую дополнительную работу. Предполагается, что помимо домашних заданий и экзаменов, которые должны сдать все студенты, выпускники также должны написать диссертацию. Как будет показано ниже, это изменение в спецификации задачи само собою приводит к объектно-ориентированному решению, которое мы будем использовать для демонстрации языковых средств, предусмотренных в C++ для поддержки ООП.

Наша цель — написать новые классы, которые будут отражать эти новые требования. Мы попытаемся развить наше предыдущее решение задачи вычисления итоговых оценок из раздела 9.6. Другими словами, мы хотели бы создать классы, которые позволяют сгенерировать отчет об итоговых оценках студентов посредством чтения файла, содержащего оценки, и вывода форматированного отчета на базе исходного кода.

13.1. Наследование

В нашей задаче вычисления итоговых оценок мы знаем, что запись с данными для выпускника такая же, как и для студента, изучающего базовый университетский курс, за исключением одной особенности, связанной с диссертацией. Такой контекст — самое место для применения *наследования* (inheritance). Наследование — один из краеугольных камней ООП. Основная его идея состоит в том, что об одном классе, за исключением, мы можем часто думать как о другом. Согласно условиям этой задачи, все студенты должны сдать домашние задания и экзамены, но некоторые из них должны также написать и диссертацию. Поэтому мы хотели бы определить два класса: один — для представления основных требований, а другой — для требований, предъявляемых к выпускнику.

В основном, мы уже знаем, как написать первый из этих классов: он аналогичен нашему предыдущему классу `Student_info`, который мы переименуем в класс `Core` (по причинам, которые станут совершенно очевидными в разделе 13.4). А пока напомним, что класс `Core` больше не представляет всех студентов, а только тех, которые отвечают базовым требованиям (в переводе с англ. “core” — сердцевина, ядро, основа. — *Прим. перев.*). Мы бы хотели оставить имя `Student_info` для класса, который подходит для любого студента. Помимо изменения имени, мы добавим в наш класс `Core` вспомогательную `private`-функцию, предназначенную для чтения той части данных о студенте, которая одинакова для всех студентов.

```
class Core {
public:
    Core();
    Core(std::istream&);
    std::string name() const;
    std::istream& read(std::istream&);
    double grade() const;
private:
    std::istream& read_common(std::istream&);
    std::string n;
    double midterm, final;
    std::vector<double> homework;
};
```

Класс `Grad` (сокращение от англ. “graduate” — выпускник учебного заведения) будет включать все дополнительные требования, предъявляемые студентам для получения диплома об окончании университета.

```
class Grad: public Core {
public:
    Grad();
    Grad(std::istream&);
    double grade() const;
    std::istream& read(std::istream&);
private:
    double thesis;
};
```

Это определение говорит о том, что мы создаем новый тип с именем `Grad`, который *выведен из* (derived from) класса `Core` или *унаследован от* (inherits from) него, или, что эквивалентно, `Core` — это *базовый класс* (base class) класса `Grad`. Поскольку класс `Grad` является потомком класса `Core`, каждый член класса `Core` также есть членом класса `Grad` (исключение составляют конструкторы, оператор присваивания и деструктор). Класс `Grad` может иметь собственные (дополнительные) члены, а в нашем примере таковыми являются член данных `thesis` (диссертация) и конструкторы класса `Grad`. Класс-потомок может переопределить члены базового класса, что мы и делаем с функциями `grade` и `read`. Однако производный класс не может удалить ни одного члена базового класса.

Использование слова `public` в словосочетании `public Core` (в заголовке определения класса `Grad`) свидетельствует о том, что класс `Grad` выведен из класса `Core` в части его интерфейса, а не в части его реализации. Другими словами, класс `Grad` наследует `public`-интерфейс класса `Core`, который становится частью `public`-интерфейса класса `Grad`. Открытые (т.е. `public`) члены класса `Core` являются `public`-членами класса `Grad`. Например, если бы у нас был объект класса `Grad`, мы могли бы вызвать его функцию-член `name`, чтобы получить имя студента, даже хотя в классе `Grad` собственная функция `name` не определена.

Класс `Grad` отличается от класса `Core` тем, что он отслеживает результаты защиты диссертации и использует другой алгоритм вычисления итоговой оценки. Таким образом, объекты класса `Grad` будут иметь пять элементов данных. Четыре из них унаследованы от класса `Core`, а пятый представляет собой `double`-значение с именем `thesis`. Класс-потомок будет иметь два конструктора и четыре функции-члена, две из которых переопределяют соответствующие члены класса `Core`, а остальные две (функции `name` и `read_common`) просто наследуются от класса `Core`.

13.1.1. Снова о защите

Согласно определению класса `Core`, все четыре элемента данных и функция `read_common` недоступны для функций-членов класса `Grad`. Это ведь было нашим желанием сделать эти члены класса `Core` закрытыми (`private`). К `private`-членам может получить доступ только сам класс и его друзья (`friend`-компоненты). К сожалению, чтобы написать `Grad`-версии функций `grade` и `read`, нам нужен доступ к некоторым из этих `private`-членов. Обозначенную проблему можно решить, переписав класс `Core` с использованием метки защиты, с которой мы до сих пор еще не встречались.

```
class Core {
public:
    Core();
    Core(std::istream&);
    std::string name() const;
    double grade() const;
    std::istream& read(std::istream&);
protected:
    std::istream& read_common(std::istream&);
    double midterm, final;
    std::vector<double> homework;
private:
    std::string n;
};
```

Как видно из нового определения класса `Core`, мы по-прежнему хотим, чтобы член `n` оставался `private`-членом, но теперь функция `read_common` и члены-данных `midterm`, `final` и `homework` объявлены с использованием спецификатора `protected`. Метка защиты `protected` предоставляет таким производным классам, как `Grad`, доступ к `protected`-членам объектов базового класса, но сохраняет эти элементы недоступными для пользователей этих классов.

Поскольку `n` — это `private`-член данных класса `Core`, то только члены и друзья класса `Core` могут получить к нему доступ. Класс `Grad` не имеет специального доступа к члену `n` и может обратиться к нему только посредством `public`-функций-членов класса `Core`.

Функции `read`, `name` и `grade` — это `public`-члены класса `Core`, и поэтому они доступны для всех пользователей класса `Core`, включая классы, выведенные из класса `Core`.

13.1.2. Операции

Чтобы закончить определения наших классов, нужно реализовать четыре конструктора: конструктор по умолчанию и конструктор, который принимает аргумент типа `istream` (по одному для каждого класса). Мы должны также реализовать шесть функций: функции `name` и `read_common` в классе `Core` и функции `read` и `grade` для обоих классов. О том, как написать конструкторы, речь пойдет в разделе 13.1.3.

Прежде чем писать код, необходимо подумать о том, как будут структурированы данные о студентах. Как и прежде, мы хотим иметь возможность учитывать перемен-

ное количество выполненных домашних заданий, поэтому эти оценки должны располагаться в конце каждой записи студента. Тогда предположим, что наши записи будут состоять из имени студента, за которым следуют оценки, полученные за экзамены в середине и в конце семестра. Если запись относится к студенту базового университетского курса, то за экзаменационными оценками сразу же располагаются оценки за домашние задания. Если запись принадлежит выпускнику, то после оценки, полученной на последнем экзамене (но впереди оценок за домашние задания), будет следовать оценка за диссертацию.

Договорившись о структуре обрабатываемой информации, можно приступить к написанию функций класса `Core`.

```
string Core::name() const { return n; }

double Core::grade() const
{
    return ::grade(midterm, final, homework);
}

istream& Core::read_common(istream& in)
{
    // Считываем и сохраняем имя студента
    // и экзаменационные оценки.
    in >> n >> midterm >> final;
    return in;
}

istream& Core::read(istream& in)
{
    read_common(in);
    read_hw(in, homework);
    return in;
}
```

Функция `Grad::read` аналогична функции `Core::read`, но перед вызовом функции `read_hw` считывает значение `thesis`.

```
istream& Grad::read(istream& in)
{
    read_common(in);
    in >> thesis;
    read_hw(in, homework);
    return in;
}
```

Обратите внимание на то, что в определении функции `Grad::read` мы можем обращаться к элементам базового класса без какого-либо специального обозначения, поскольку эти элементы также являются членами класса `Grad`. Чтобы явно подчеркнуть факт унаследования членов от класса `Core`, можно использовать оператор разрешения области видимости (`::`).

```
istream& Grad::read(istream& in)
{
    Core::read_common(in);
    in >> thesis;
    read_hw(in, Core::homework);
    return in;
}
```

Безусловно, член `thesis` не имеет префикса, он является частью класса `Grad`, а не частью класса `Core`. (Мы могли записать `Grad::thesis`, но не `Core::thesis`.)

Функция `grade` изменена для учета члена `thesis`. Ее новая версия чрезвычайно проста: студент получает меньшее из двух значений (значения оценки за диссертацию и значения оценки, учитывающей только результаты сдачи экзаменов и выполнения домашних заданий).

```
double Grad::grade() const
{
    return min(Core::grade(), thesis);
}
```

Здесь мы вызываем функцию `grade` базового класса, чтобы вычислить итоговую оценку без учета результатов защиты диссертации. В этом случае использование оператора разрешения области видимости просто необходимо. Если бы мы записали инструкцию

```
return min(grade(), thesis);,
```

то тем самым бы организовали (рекурсивный) вызов `Grad`-версии функции `grade`, который неминуемо ведет к катастрофе местного (для данной программы) масштаба. Чтобы узнать, какую оценку считать итоговой, мы используем функцию `min` из заголовка `<algorithm>`. Функция `min` действует подобно функции `max`, за исключением того, что она возвращает не большее, а меньшее из значений двух своих аргументов. Как и в случае функции `max`, аргументы функции `min` должны иметь абсолютно одинаковый тип.

13.1.3. Наследование и конструкторы

Прежде чем мы напишем конструкторы для классов `Core` и `Grad`, нам нужно понять, как C++-среда создаст объекты производного типа. Как и в случае любого типа класса, деятельность C++-среды начинается с выделения области памяти для нового объекта. Затем запускается соответствующий конструктор для инициализации этого объекта. Тот факт, что объект имеет производный тип, добавляет в процесс создания объекта еще одно действие. Это дополнительное действие состоит в построении той части объекта, которая относится к части базового класса. Производные объекты создаются посредством выполнения следующих действий.

- Выделяется память для целого объекта (для членов базового и производного классов).
- Вызывается конструктор базового класса для инициализации части (частей) объекта, относящейся к базовому классу.
- Инициализируются члены производного класса согласно указаниям инициализатора соответствующего конструктора.
- Выполняется тело конструктора производного класса, если оно не пустое.

Новизна состоит лишь в том, как мы определяем, какой именно конструктор базового класса должен выполняться. Неудивительно, что для задания желаемого конструктора базового класса используется инициализатор конструктора производного класса. Инициализатор конструктора производного класса указывает свой базовый класс, за которым следует (возможно, пустой) список аргументов. Эти аргументы представляют собой начальные значения, используемые при построении части базового класса; они предназначены для выбора конструктора базового класса, который во время выполнения должен инициализировать члены базового класса. Если инициализатор не задает, какой именно конструктор

базового класса следует выполнить, то для построения базовой части объекта используется конструктор по умолчанию базового класса.

```
class Core {
public:
    // Конструктор по умолчанию для класса Core.
    Core(): midterm(0), final(0) { }

    // Конструктор, создающий объект класса Core из объекта
    // потока istream.
    Core(std::istream& is) { read(is); }
    // . . .
};

class Grad: public Core {
public:
    // Оба конструктора для инициализации базовой части объекта
    // неявно используют конструктор Core::Core().
    Grad(): thesis(0) { }

    Grad(std::istream& is) { read(is); }
    // . . .
};
```

Конструкторы класса Core идентичны конструкторам, приведенным в разделах 9.5.1 и 9.5.2: они определяют, как создать Core-объект “из ничего” или из istream-объекта. Конструкторы класса Grad “показывают”, как создать Grad-объект из тех же значений, т.е. либо при отсутствии аргументов, либо при использовании аргумента типа istream&. Необходимо отметить, что не существует никаких требований относительно того, чтобы конструкторы производного класса принимали те же аргументы, что и конструкторы базового класса.

Конструктор по умолчанию класса Grad “утверждает”, что для создания Grad-объекта “из ничего” C++-среда должна создать Core-часть объекта и установить член thesis равным 0. Большая часть этой работы выполняется неявным образом: прежде всего, поскольку инициализатор конструктора пустой, мы неявно вызываем конструктор по умолчанию для класса Core, чтобы инициализировать члены midterm, final, homework и name. И точно так же конструктор по умолчанию класса Core неявно инициализирует члены name и homework посредством конструкторов по умолчанию их типов, и только члены midterm и final инициализируются явным образом. Единственное явное действие, которое предпринимает конструктор по умолчанию класса Grad, связано с инициализацией члена thesis. Кроме инициализации thesis, этому конструктору больше нечего делать, поэтому тело этой функции пустое.

Создание объекта класса Grad из потокового объекта (класса istream) во многом аналогично тому, как создается объект класса Core из istream-объекта, а именно посредством вызова функции-члена read. Однако сначала мы (неявно) вызываем конструктор по умолчанию базового класса, чтобы инициализировать базовую часть этого объекта. Затем вызывается функция read, а поскольку этот конструктор — член класса Grad, то ее “полным” именем является Grad::read. Нам не нужно беспокоиться об инициализации члена thesis, поскольку функция read должна записать в thesis значение, считанное из объекта is.

Важно понимать, как создаются объекты производного класса. При выполнении инструкции

```
Grad g;
```

система выделяет область памяти, достаточную для хранения пяти членов данных класса Grad, после чего выполняет конструктор по умолчанию класса Core для инициализации членов данных в Core-части Grad-объекта g, а затем конструктор по умолчанию класса Grad. Аналогично при выполнении инструкции

```
Grad g(cin);
```

после выделения соответствующего объема памяти C++-среда запускает конструктор по умолчанию класса Core, а после него — конструктор Grad:Grad(istream&), чтобы прочитать из входного потока значения и сохранить их в членах данных name, midterm, final, thesis и homework.

13.2. Полиморфизм и виртуальные функции

Мы еще не полностью реорганизовали исходную абстракцию Student_info. Эта абстракция опиралась на функцию-не-член, которая поддерживала часть ее интерфейса: она использовала функцию compare для сравнения двух записей с данными о студентах. Эта функция использовалась в алгоритме sort для упорядочения записей по алфавиту.

Наша новая функция сравнения идентична той, которую мы написали в разделе 9.3.1, за исключением одного изменения в имени типа.

```
bool compare(const Core& c1, const Core& c2)
{
    return c1.name() < c2.name();
}
```

Сравнение двух записей с данными о студентах заключается в сравнении их имен. По сути, мы делегируем реальную работу оператору "<" библиотечного класса string. В этом коде интересно то, что мы можем использовать его для сравнения как Core-, так и Grad-объектов или даже Core-объекта с Grad-объектом.

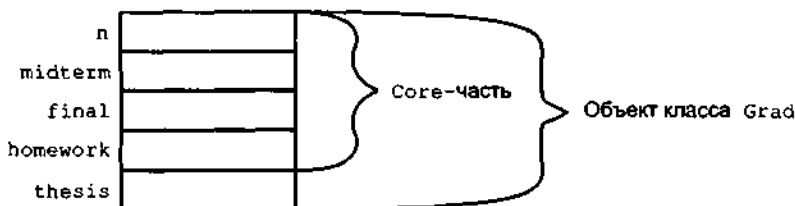
```
Grad g(cin);           // считываем Grad-запись.
Grad g2(cin);          // считываем Grad-запись.

Core c(cin);           // считываем Core-запись.
Core c2(cin);          // считываем Core-запись.

compare(g, g2);        // сравниваем две Grad-записи.
compare(c, c2);        // сравниваем две Core-записи.
compare(g, c);         // сравниваем Grad-запись с Core-записью.
```

В каждом из этих обращений к функции compare будет выполняться функция-член name() класса Core, чтобы определить значение, возвращаемое функцией compare. Очевидно, этот член вполне правомерно вызывать для класса Core, а как насчет класса Grad? Определяя класс Grad (а вам известно, что он выведен из класса Core), мы не переопределили функцию name. Следовательно, вызывая функцию g.name() для Grad-объекта g, мы в действительности вызываем функцию-член name, которая унаследована от класса Core. Поведение этой функции для класса Grad такое же, как и для класса Core: из Core-части объекта она считывает содержимое поля n.

Передать Grad-объект функции, которая ожидает аргумент типа Core&, можно по той простой причине, что класс Grad выведен из класса Core, поэтому каждый объект класса Grad имеет Core-часть.



Поскольку объект класса `Grad` имеет `Core`-часть, мы можем связывать ссылочные параметры функции `compare` с `Core`-частями `Grad`-объектов точно так же, как мы можем связывать их с простыми `Core`-объектами. Аналогично мы могли бы определить функцию `compare`, которая принимала бы указатели на класс `Core` или на объекты типа `Core` (в противоположность ссылке на тип `Core`). В любом случае мы могли бы по-прежнему вызывать эту функцию от имени объекта класса `Grad`. Если бы она принимала указатели, мы могли бы вызывать ее, передав указатель на объект класса `Grad`. Компилятор преобразовал бы тогда значение аргумента типа `Grad*` в значение аргумента типа `Core*` и связал бы этот указатель с `Core`-частью `Grad`-объекта. Если бы эта функция принимала объект класса `Core`, то переданный нами аргумент был бы просто `Core`-частью этого объекта. Как будет показано ниже, поведение функции может в значительной степени зависеть от того, передаем ли мы ей сам объект, ссылку или указатель на объект.

13.2.1. Получение значения при неизвестном типе объекта

Наша функция `compare` правильно срабатывает, когда мы вызываем ее с `Grad`-объектом в качестве аргумента, поскольку функция `name` совместно используется как `Grad`-, так и `Core`-объектами. А что если бы мы захотели сравнивать информацию о студентах не по именам, а по их итоговым оценкам? Например, вместо создания списка итоговых оценок, отсортированных по именам, нам понадобилось бы создать список, отсортированный по итоговым оценкам.

В качестве первого приближения к решению этой задачи попробуем написать функцию, подобную функции `compare`.

```
bool compare_grades(const Core& c1, const Core& c2)
{
    return c1.grade() < c2.grade();
}
```

Единственное отличие функции `compare_grades` от `compare` состоит в том, что здесь мы вызываем функцию `grade`, а не `name`. И это различие оказывается весьма существенным!

Дело в том, что класс `Grad` переопределяет функцию `grade`, а мы ничего не сделали, чтобы как-то различать эти две версии функции `grade`. При выполнении функция `compare_grades` вызывает функцию-член `Core::grade` точно так же, как функция `compare` вызывает функцию-член `Core::name`. В этом случае для объекта класса `Grad` версия из класса `Core` даст неверный ответ, поскольку функции `grade` в классах `Core` и `Grad` ведут себя по-разному. Для объектов класса `Grad` мы должны выполнить функцию-член `Grad::grade`, чтобы учесть результаты защиты диссертации.

Нам необходимо найти способ, чтобы функция `compare_grades` вызывала нужную функцию `grade`, зависящую от реального типа передаваемого объекта: если аргументы `c1` или `c2` будут ссылаться на `Grad`-объект, то мы хотели бы, чтобы использовалась `Grad`-версия функции `grade`; если сравниваемые объекты будут иметь тип `Core`, то

должна использоваться функция `grade` из класса `Core`. При этом необходимо, чтобы правильное решение принималось во время выполнения программы, т.е. мы хотим, чтобы система вызывала нужную функцию, основываясь на реальном типе объектов, передаваемых функции `compare_grades`, а он (этот тип) станет известен только во время выполнения программы.

Для поддержки возможности такого оперативного выбора в C++ предусмотрен такой инструмент, как *виртуальная функция* (virtual function).

```
class Core {
public:
    virtual double grade() const; // Добавлено слово virtual.
    // ...
};
```

Теперь мы можем сказать, что `grade` — виртуальная функция. При вызове функции `compare_grades` C++-среда определит подлежащую выполнению версию функции `grade`, рассмотрев реальные типы объектов, с которыми связаны ссылки `c1` и `c2`. Другими словами, она определит, какую функцию вызывать, изучив каждый объект, который мы передали в качестве аргумента функции `compare_grades`. Если аргумент представляет собой `Grad`-объект, она выполнит функцию `Grad::grade`, а если аргументом окажется `Core`-объект, то функцию `Core::grade`.

Ключевое слово `virtual` можно использовать только внутри определения класса. Если функции определяются отдельно от своих объявлений, то в определениях слово `virtual` не повторяется. Таким образом, определение функции `Core::grade()` в изменениях не нуждается. Кроме того, факт виртуальности функции передается “по наследству”, поэтому нам не нужно повторять обозначение `virtual` в объявлении функции `grade` внутри класса `Grad`. Но теперь мы должны перекомпилировать наш код с новым определением класса `Core`. После перекомпиляции, благодаря виртуальности версии этой функции из базового класса, мы получаем желаемое поведение.

13.2.2. Динамическое связывание

Выбор виртуальной функции во время работы программы релевантен только в случае, когда эта функция вызывается посредством ссылки или указателя. Если вызывать виртуальную функцию “от имени” объекта (в противоположность ссылке или указателю), то мы будем знать точный тип объекта во время компиляции. Тип объекта фиксирован: он такой, какой есть, и не меняется во время работы программы. И совсем другая картина, если мы имеем дело со ссылкой или указателем на объект базового класса. В этом случае ссылка (или указатель) может ссылаться (или указывать) как на объект базового класса, так и на объект типа, выведенного из базового класса, а это значит, что тип ссылки или указателя и тип объекта, с которым связана эта ссылка или указатель, могут варьироваться во время выполнения программы. На этом различии, собственно, и основан механизм поддержки виртуальных функций.

Предположим, что мы переписали функцию `compare_grades` следующим образом.

```
// Некорректная реализация функции!
bool compare_grades(Core c1, Core c2)
{
    return c1.grade() < c2.grade();
}
```

В этой версии мы утверждаем, что параметрами являются объекты, а не ссылки на объекты. И поэтому мы всегда будем знать тип объектов, представленных параметра-

ми `c1` и `c2`: они являются объектами класса `Core`. Тем не менее мы можем вызвать эту функцию “от имени” объекта класса `Grad`, но тот факт, что аргумент имеет тип `Grad`, оказывается несущественным, поскольку то, что мы передаем функции, является базовой частью объекта. Объект `Grad` будет “урезан” до своей `Core`-части, и функции `compare_grades` будет передана копия этой части `Grad`-объекта. Поскольку мы сказали, что параметрами являются `Core`-объекты, обращения к функции `grade` *статически связываются* (причем во время компиляции) с функцией `Core::grade`.

Это различие между *динамическим связыванием* (dynamic binding) и *статическим связыванием* (static binding) существенно для понимания того, как C++ поддерживает объектно-ориентированное программирование. Словосочетание *динамическое связывание* означает, что связывание функций может происходить во время работы программы, а статическое связывание осуществляется во время компиляции. Если вызвать виртуальную функцию “от имени” объекта, этот вызов будет связан статически (т.е. во время компиляции), поскольку невозможно, чтобы тип объекта во время компиляции отличался от его же типа во время работы программы. И совсем другое дело, если некоторая виртуальная функция вызывается посредством указателя или ссылки: в этом случае имеет место динамическое связывание функции, т.е. связывание во время выполнения программы, когда реально используемая версия `virtual`-функции зависит от типа объекта, с которым связывается ссылка или указатель.

```
Core c;  
Grad g;  
Core* p;  
Core& r = g;  
  
c.grade(); // Объект c статически связан с Core::grade().  
g.grade(); // Объект g статически связан с Grad::grade().  
p->grade(); // динамическое связывание, зависящее от типа  
             // объекта, на который указывает указатель p.  
r.grade();  // динамическое связывание, зависящее от типа  
             // объекта, на который ссылается ссылка r.
```

Первые два вызова могут быть связаны статически: мы знаем, что `c` — это объект класса `Core` и во время выполнения программы `c` по-прежнему останется объектом класса `Core`. Следовательно, компилятор жестко увяжет всех “участников” этого вызова, несмотря на то что `grade` — `virtual`-функция. Но в третьем и четвертом вызовах мы не будем знать тип объекта (на который ссылается ссылка `r` или указывает указатель `p`) до тех пор, пока программа не станет выполняться: это могут быть `Core`-или `Grad`-объекты. Следовательно, принятие решения о том, какую функцию выполнить в этих случаях, должно быть отложено до времени выполнения. C++-среда принимает это решение на базе типа объекта, на который указывает указатель `p` или на который ссылается ссылка `r`.

Тот факт, что мы можем использовать производный тип там, где ожидается указатель или ссылка на базовый тип, является примером ключевого понятия в ООП, именуемого *полиморфизм* (polymorphism). Это слово, которое произошло от греческого слова *polymorphos* (πολυμορφος — “много форм”), уже употреблялось в английском языке в середине девятнадцатого века. В контексте программирования оно определяет возможность одного типа замещать другие типы. C++ поддерживает полиморфизм посредством свойств динамического связывания, которыми обладают виртуальные функции. Вызывая `virtual`-функцию посредством указателя или ссылки, мы создаем полиморфический вызов. Тип ссылки (или указателя) фиксирован, но тип объекта, на который она ссылается (или он указывает), может быть либо заданным типом (ссылки

или указателя), либо любым производным от него типом. Следовательно, указав один тип, потенциально мы можем вызвать одну из многих функций.

Еще одно замечание по поводу использования виртуальных функций: эти функции должны быть определены независимо от того, вызывает их программа или нет. Невиртуальные функции могут быть объявлены, но не определены, если программа их не вызывает. Многие компиляторы генерируют загадочные сообщения об ошибках для классов, в которых не определена одна или несколько виртуальных функций. Если ваша программа получит сообщение от компилятора, смысл которого вам непонятен, и в этом сообщении утверждается, что в вашей программе что-то не определено, вы должны удостовериться в том, что определили все свои виртуальные функции. Чаще всего, в этом и кроется причина ошибки (и сообщения о ней).

13.2.3. Подведем некоторые итоги

Если путь не близок, всегда стоит остановиться и отдохнуть, а заодно оглядеться и попытаться понять, сколько уже пройдено (сделано) и сколько еще осталось пройти (сделать). Подытожив сделанное, мы понимаем, что желательно внести одно небольшое изменение: хорошо бы функцию `read` также сделать виртуальной. Было бы неплохо иметь возможность управления запуском нужной версии функции `read` в зависимости от типа объекта, для которого она вызывается. После внесения этого последнего изменения наши классы выглядят следующим образом.

```
class Core {
public:
    Core(): midterm(0), final(0) { }
    Core(std::istream& is) { read(is); }

    std::string name() const;

    // Как определено в разделе 13.1.2.
    virtual std::istream& read(std::istream&);
    virtual double grade() const;

protected:
    // Члены, доступные для производных классов.
    std::istream& read_common(std::istream&);
    double midterm, final;
    std::vector<double> homework;

private:
    // Член, доступный только для класса Core.
    std::string n;
};

class Grad: public Core {
public:
    Grad(): thesis(0) { }
    Grad(std::istream& is) { read(is); }

    // Замечание: функции grade и read виртуальны
    // влечение наследования.
    double grade() const;
    std::istream& read(std::istream&);

private:
    double thesis;
};

bool compare(const Core&, const Core&);
```

Итак, мы определили два класса, инкапсулирующие два типа студентов. Первый класс, `Core`, представляет студентов, изучающих базовый университетский курс. Наш второй класс, `Grad`, производный от класса `Core`, учитывает дополнительные требования к студентам, связанные с защитой диссертации. Мы можем создавать `Core`- или `Grad`-объекты двумя способами. Конструктор по умолчанию создает пустой объект, инициализированный соответствующим образом; второй конструктор принимает аргумент типа `istream&` и считывает начальные значения из заданного потока. Определенные в классах операции позволяют считать в объект новые данные и узнать имя студента или его итоговую оценку. Обратите внимание на то, что в этой версии обе функции `grade` и `read` объявлены виртуальными. Наконец, наш интерфейс включает глобальную функцию-член `compare`, которая сравнивает два объекта посредством сравнения имен студентов.

13.3. Использование наследования для решения нашей “старой” задачи

Теперь, когда у нас есть классы, которые моделируют различные категории студентов, было бы заманчиво использовать эти классы для решения задачи вычисления итоговых оценок из раздела 9.6. Та программа считывала файл, который содержит записи с оценками студентов, вычисляла итоговую оценку для каждого студента и выводила форматированный отчет в алфавитном порядке (по имени студента). Мы бы хотели решить ту же проблему, но с использованием файла, который содержит записи для обеих категорий студентов.

Прежде чем решать всю задачу целиком, попытаемся решить две задачки попроще: напишем программы, которые могут считывать файлы, состоящие исключительно из записей одного и того же вида. Обе программы должны быть похожи на исходный вариант, за исключением объявлений типа.

```
int main()
{
    vector<Core> students; // Считываем и обрабатываем
                        // Core-записи.
    Core record;
    string::size_type maxlen = 0;

    // Считываем и сохраняем данные.
    while (record.read(cin)) {
        maxlen = max(maxlen, record.name().size());
        students.push_back(record);
    }

    // Располагаем записи в алфавитном порядке.
    sort(students.begin(), students.end(), compare);

    // Выводим имена и итоговые оценки.
    for (vector<Core>::size_type i = 0;
         i != students.size(); ++i) {
        cout << students[i].name()
              << string(maxlen + 1 - students[i].name.size(),
                        ' ');

        try {
            double final_grade = students[i].grade(); //
                                                // Core::grade
            streamsize prec = cout.precision();
        }
```

```

        cout << setprecision(3) << final_grade
              << setprecision(prec) << endl;
    } catch (domain_error e) {
        cout << e.what() << endl;
    }
}
return 0;
}

```

Мы можем написать аналогичную программу для обработки Grad-записей, изменив соответствующие определения типов.

```

int main()
{
    vector<Grad> students; // В векторе элементы другого типа.
    Grad record;          // Считывание данных происходит
                          // в объекты другого типа.
    string::size_type maxlen = 0;

    // Считываем и сохраняем данные.
    while (record.read(cin)) { // Считываем в Grad-,
                              // а не в Core-объект.
        maxlen = max(maxlen, record.name().size());
        students.push_back(record);
    }

    // Располагаем записи в алфавитном порядке.
    sort(students.begin(), students.end(), compare);

    // Выводим имена и итоговые оценки.
    for (vector<Grad>::size_type i = 0;
         i != students.size(); ++i) {
        cout << students[i].name()
              << string(maxlen + 1 - students[i].name.size(),
                          ' ');
        try {
            double final_grade = students[i].grade(); //
                                                    // Grad::grade
            streamsize prec = cout.precision();
            cout << setprecision(3) << final_grade
                  << setprecision(prec) << endl;
        } catch (domain_error e) {
            cout << e.what() << endl;
        }
    }
    return 0;
}

```

Конечно, функции, которые выполняются в каждом варианте программы, зависят от типа объекта `record` и от типа объектов, содержащихся в векторе. Например, выражение

```
record.read(cin)
```

вызывает функцию `Core::read` или `Grad::read`, в зависимости от типа объекта `record`. Этот вызов статически связан с участниками вызова: жесткая зависимость от типа объекта `record` не позволяет “проявиться” виртуальной природе функции `read`. Мы вызываем эту функцию “от имени” объекта, а не с использованием указателя или ссылки на объект. Следовательно, объект `record` — это объект либо класса `Core`, либо класса `Grad`, — все зависит от выполняемой версии программы. После определения объекта `record` его тип остается неизменным, и поэтому обращение к функции `record.read(cin)` связывается во время компиляции. Аналогично функция `grade`

```
students[i].grade(),
```

которую мы вызываем при генерации отчета, будет статически связана с ее реализацией из класса `Core` при выполнении первой программы и с соответствующей реализацией из класса `Grad` при выполнении второй программы.

В обоих вариантах этой программы вызовы функции `name()` относятся к ее (не-виртуальной) версии, определенной в классе `Core`. Эта функция наследуется классом `Grad`, поэтому при ее вызове для объекта класса `Grad` мы по-прежнему выполняем версию, определенную в классе `Core`. Функция `compare`, которую мы передаем функции `sort`, использует ссылки на класс `Core`. При выполнении варианта программы, который обрабатывает `Grad`-объекты, будут сравниваться `Core`-части этих объектов.

Ясно, что весьма утомительно писать отдельные версии нашей программы. Поэтому мы хотим написать единую версию, которая могла бы обрабатывать либо `Core`-, либо `Grad`-объекты.

Чтобы написать единую функцию, которая могла бы читать файл либо `Core`-, либо `Grad`-записей, нам необходимо тщательно рассмотреть код программ и идентифицировать те места, где важен тип записи. Чтобы написать единую версию программы, придется устранить эти зависимости от типа.

- Определение вектора, в котором мы храним элементы данных, считанные из файла.
- Определение локального временного объекта, в который мы считываем записи с данными о студентах.
- Функция `read`.
- Функция `grade`.

Остальной код либо не зависит от типа (например, код сортировки вектора или опроса его элементов), либо одинаков для `Grad`- и `Core`-версий (например, в функциях `name` и `compare`). Определив функции `grade` и `read` как виртуальные, мы тем самым решили последние две части нашей задачи.

Оказывается, что с первыми двумя подзадачами (какой тип использовать для локального временного объекта и значения какого типа хранить в контейнере) можно справиться путем применения той же стратегии. И оказывается, что к решению этих подзадач существует два различных подхода. Первый довольно прост, поэтому мы рассмотрим его в следующем разделе. Второй представляет собой популярную идиому C++-программирования, обсуждению которой посвящен раздел 13.4.

13.3.1. Контейнеры (фактически) неизвестного типа

Нам предстоит справиться с зависимостью от типа в следующем коде.

```
vector<Core> students;    // В векторе должны храниться
                          // Core-объекты, а не объекты
                          // полиморфического типа.
Core record;              // Core-объект, а не объект типа,
                          // производного от Core.
```

Зависимость от типа в этом коде довольно прозрачна. Определяя объект `record`, мы можем точно сказать, объектом какого типа является `record`: это объект класса `Core`, поскольку таковым мы его определили. Аналогично определяя объект `students`, мы утверждаем, что это вектор, который содержит объекты типа `Core`. Подробнее о таких контейнерах мы поговорим в разделе 13.6.1, а пока важно понять, что определяя вектор типа `vector<Core>`, мы утверждаем, что каждый объект в этом векторе будет объектом класса `Core`, а не объектом типа, выведенного из класса `Core`.

Говоря о зависимости от типа в наших двух программах, мы отмечали, что уже решили половину этой проблемы, объявив функции `read` и `grade` виртуальными. Другая половина проблемы состоит в том, что наши программы создают статически связанные вызовы этих `virtual`-функций. Чтобы добиться нужного нам поведения (присущего динамическому связыванию), необходимо вызывать функции `read` и `grade` посредством указателя или ссылки на объект класса `Core`. В этом случае тип связываемого объекта может отличаться от типа указателя или ссылки. Подобные рассуждения приводят нас к решению всех четырех подзадач: мы можем написать программу, которая будет управлять не объектами, а указателями. Ведь мы можем иметь вектор типа `vector<Core*>`, а также определить объект `record` как указатель. В этом случае мы можем получить желаемое динамическое поведение, избавившись от зависимости от типа, которая имеет место в определениях этого вектора и локального временного объекта. К сожалению, как будет показано ниже, это решение значительно усложняет жизнь наших пользователей. Например, попытка напрямую использовать это решение не работает вообще.

```
int main()
{
    vector<Core*> students;

    Core* record;
    while (record->read(cin)) { // Аварийный отказ!
        // . . .
    }
}
```

Фатальный недостаток этой программы состоит в том, что нам никогда бы не удалось заставить переменную `record` указывать на объект!

Мы можем решить эту проблему, потребовав, чтобы наши пользователи сами управляли памятью, занимаемой объектами, в которых должны храниться данные, считанные из файла. Пользователи должны также уметь распознавать, какого типа запись считывает программа. Предположим, что каждая запись будет содержать индикатор, позволяющий распознать вид записи: записи выпускников будут начинаться с буквы "G", а записи остальных студентов — с буквы "U".

Прежде чем переориентировать (значит, переписывать) программу на использование указателей, необходимо решить еще одну проблему: как отсортировать вектор указателей? Самый простой выход — написать новую функцию сравнения, которая будет принимать два указателя на объекты класса `Core`. Интересно то, что мы не можем назвать эту функцию именем `compare`. Вспомните, что в разделе 8.1.3 мы рассматривали различные аспекты получения верных типов для значений, которые передаются в качестве шаблонных аргументов. По аналогичным причинам мы не можем передать имя перегруженной функции как шаблонный аргумент. Если бы мы сделали это, компилятор не смог бы определить, какую версию функции мы имеем в виду. Очевидно, для передачи функции `sort` нам придется написать функцию сравнения с неперегруженным именем.

```
bool compare_Core_ptrs(const Core* cp1, const Core* cp2)
{
    return compare(*cp1, *cp2);
}
```

Написав специализированную функцию сравнения, мы можем теперь переписать нашу программу.

```

// Этот код почти работоспособен (см. раздел 13.3.2).
int main()
{
    vector<Core*> students; // храним указатели, а не объекты.
    Core* record;          // Временный объект также должен
                           // быть указателем.
    char ch;
    string::size_type maxlen = 0;

    // Считываем и сохраняем данные.
    while (cin >> ch) {
        if (ch == 'u')
            record = new Core; // Размещаем в памяти
                               // Core-объект.
        else
            record = new Grad; // Размещаем в памяти
                               // Grad-объект.
        record->read(cin);      // virtual-вызов
        maxlen = max(maxlen,
                     record->name().size()); // Разыменование
        students.push_back(record);
    }

    // Передаем версию функции compare, которая работает
    // с указателями.
    sort(students.begin(), students.end(), compare_Core_ptrs);

    // Выводим имена и оценки.
    for (vector<Core*>::size_type i = 0;
         i != students.size(); ++i) {
        // Элемент students[i] - это указатель, который мы
        // разыменовываем для вызова функций.
        cout << students[i]->name()
              << string(maxlen + 1 - students[i]->name.size(),
                        ' ');
        try {
            double final_grade = students[i]->grade();
            streamsize prec = cout.precision();
            cout << setprecision(3) << final_grade
                  << setprecision(prec) << endl;
        } catch (domain_error e) {
            cout << e.what() << endl;
        }
        delete students[i]; // Освобождаем объект, размещенный
                           // в памяти при считывании данных.
    }
    return 0;
}

```

В комментариях отмечены многочисленные отличия этого кода от исходного. Все внесенные изменения свидетельствуют о том, что мы должны манипулировать указателями, а не объектами.

Цикл `while` изменен, чтобы считать из входного потока первый символ, который впоследствии мы тестируем, и выяснить, какого вида запись нужно считать. Выяснив, к какой категории студента относятся считанные данные, мы будем знать, какого типа объект нам нужно создать, поэтому и размещаем в памяти объект соответствующего типа, чтобы использовать его для считывания данных из стандартного входного потока. Функция `read` — `virtual`-функция, поэтому будет вызвана нужная ее версия, которая зависит от того, на объект какого типа (`Grad` или `Core`) указывает текущий указатель `record`. В обоих случаях функция `read` считывает в объект значения оставшейся части записи. Обратите внимание на

то, что для получения доступа к функции `read` мы должны разыменовывать значение указателя `record`. Код вычисления длины самого длинного имени также изменен из-за необходимости разыменования указателя.

Приступая к рассмотрению цикла, обеспечивающего вывод обработанных данных, мы должны помнить, что элемент вектора `students[i]` представляет собой указатель. Поэтому для получения доступа к объекту, на который указывает значение элемента `students[i]`, необходимо выполнить операцию разыменования. Как и в случае функции `read`, вызов функции `grade` является виртуальным, поэтому для вычисления итоговой оценки студента автоматически вызывается нужная версия функции `grade`, учитывающая результат защиты диссертации, если объект имеет тип `Grad`, и не учитывающая его в противном случае. Заключительное изменение связано с необходимостью возвратить C++-среде память, занимаемую объектом, что мы реализуем посредством вызова `delete` для указателя, который содержится в элементе `students[i]`.

13.3.2. Виртуальные деструкторы

Наша программа почти работоспособна. Возникает только одна проблема, когда мы удаляем объекты с помощью оператора `delete`, поскольку это осуществляется внутри цикла вывода обработанной информации. При размещении этих объектов в памяти мы выделяли пространство как для `Grad`-, так и `Core`-объектов, но указатели на эти объекты сохранялись как `Core*`, а не `Grad*`-указатели. Следовательно, используя оператор `delete` для их удаления, мы удаляем указатель на `Core`-, а не `Grad`-объект, даже если этот конкретный указатель реально указывает на объект класса `Grad`. К счастью, эта проблема решается довольно легко.

При удалении с помощью оператора `delete` по указателю выполняются два действия: вызывается деструктор для данного объекта и освобождается область памяти, занимаемая этим объектом. Когда программа удаляет указатель, содержащийся в элементе `students[i]`, то удаляется указатель либо на `Core`-, либо на `Grad`-объект. Ни в классе `Core`, ни в классе `Grad` явно деструктор не определен, а это означает, что при выполнении оператора `delete` вызывается синтезированный деструктор, а затем системе возвращается память, которую занимал этот объект. Синтезированный деструктор должен запустить деструктор для каждого члена данных в классе. Но в данном случае непонятно, какой деструктор при выполнении оператора `delete` должна запустить система? Другими словами, члены объекта какого класса (`Core` или `Grad`) должны быть разрушены деструктором? Да и какой объем памяти необходимо вернуть системе — достаточный для хранения `Core`- или `Grad`-объекта?

На эти вопросы, похоже, может дать ответ уже знакомый нам виртуальный механизм. Чтобы обладать виртуальным деструктором, класс по крайней мере должен иметь деструктор, который мы могли бы сделать виртуальным.

```
class Core {
public:
    virtual ~Core() { }
    // Остальное остается в силе.
};
```

Теперь при выполнении инструкции `delete students[i]`; выполняемый деструктор будет зависеть от типа объекта, на который в действительности указывает элемент `students[i]`. Аналогично тип объекта при возвращении системе занимаемой им памяти будет определяться типом, на который реально указывает элемент `students[i]`.

Обратите внимание на то, что тело этого деструктора пустое. Единственное, что от него требуется для разрушения объекта класса `Core` — разрушить его члены, и система выполняет эту работу автоматически. В пустых `virtual`-деструкторах нет ничего необычного. Виртуальный деструктор потребуется, когда, возможно, будет разрушаться объект производного типа посредством указателя на базовый тип. Если не существует другой причины для определения деструктора, этому деструктору нечего будет делать, и его тело останется пустым.

Интересно, что в обновлении класса `Grad`, чтобы внести в него деструктор, нет никакой необходимости. Как и в случае всех виртуальных функций, виртуальность деструктора “передается по наследству”. Класс явно не “заставляет” делать какую бы то ни было работу для разрушения объектов, поэтому нет смысла переопределять деструктор в производном классе. Поскольку производный класс наследует свойство `virtual` от деструктора своего базового класса, все, что нам осталось сделать — перекомпилировать программу.

13.4. Простой дескрипторный класс

Хотя только что рассмотренный нами подход довольно прост, все же он не лишен проблем: наша программа “обросла” дополнительными сложностями, связанными с управлением указателями, а также рядом “ловушек”, которые могут легко привести к ошибкам. Пользователи не должны забывать о выделении памяти для записей с данными о студентах во время их считывания, а затем точно так же не должны забывать об освобождении этой памяти, когда исчезнет необходимость в использовании этих данных. Кроме того, для получения доступа к реальным объектам без конца выполняются операции разыменования указателей. Тем не менее мы решили проблему написания программы, которая может читать файл, содержащий смешанные записи двух видов.

Теперь нам хотелось бы найти способ сохранить лучшие качества наших предыдущих (более простых) программ, которые обрабатывали либо `Core`-, либо `Grad`-объекты, и одновременно устранить проблемы нашего нового решения, позволяющего обрабатывать записи обоих типов. Оказывается, существует распространенный прием программирования, известный как *класс дескриптора* (*handle class*), который позволит осуществить наши намерения.

Наш код стал более громоздким, когда мы поняли, что должны уметь обрабатывать объекты, типы которых остаются неизвестными до тех пор, пока программа не заработает. Мы ведь знали, что каждый объект будет иметь либо тип `Core`, либо некоторый другой тип, производный от класса `Core`. В нашем решении использовались указатели, поскольку мы вполне могли разместить в памяти указатель на объект класса `Core`, а затем заставить этот указатель указывать либо на `Core`-, либо на `Grad`-объект. Проблема данного решения состоит в том, что в этом случае пользователи должны выполнять подверженный ошибкам учет использования системных ресурсов. Да, мы не можем устранить эту “бухгалтерию”, но можем скрыть ее от наших пользователей, написав новый класс, который будет инкапсулировать указатель на объект класса `Core`.

```
class Student_info {
public:
    // конструкторы и управление копированием.
    Student_info(): cp(0) { }
    Student_info(std::istream& is): cp(0) { read(is); }
    Student_info(const Student_info&);
    Student_info& operator=(const Student_info&);
};
```

```

~Student_info() { delete cp; }

// Операции.
std::istream& read(std::istream&);

std::string name() const {
    if (cp) return cp->name();
    else throw std::runtime_error(
        "Неинициализированные Student-данные");
}
double grade() const {
    if (cp) return cp->grade();
    else throw std::runtime_error(
        "Неинициализированные Student-данные");
}
static bool compare(const Student_info& s1,
                    const Student_info& s2) {
    return s1.name() < s2.name();
}

private:
    Core* cp;
};

```

Идея состоит в том, что объект класса `Student_info` может представлять или `Core`-, или `Grad`-объект. В этом смысле он будет действовать как указатель. Но пользователям класса `Student_info` при этом не придется беспокоиться о размещении в памяти объекта, с которым связывается данный `Student_info`-объект. Таким образом, этот класс берет на себя заботу об утомительных и подверженных ошибкам аспектах наших программ.

Каждый объект класса `Student_info` будет хранить указатель с именем `cp`, который указывает на объект либо типа `Core`, либо типа, производного от `Core`. Как будет показано в разделе 13.4.1, в функции `read` мы разместим в памяти объект, на который и будет указывать указатель `cp`. Следовательно, оба конструктора инициализируют указатель `cp` значением 0, а это значит, что объект класса `Student_info` пока ни с чем не связан. В конструкторе, который принимает аргумент типа `istream`, мы вызываем функцию `Student_info::read`. Эта функция разместит в памяти новый объект соответствующего типа и сохранит в нем значение, считанное из заданного `istream`-объекта.

Исходя из тройного правила (см. раздел 11.3.6), мы знаем, что для управления указателем нам нужен конструктор копирования, оператор присваивания и деструктор. Работа, которую должен выполнять деструктор, довольно проста: просто он должен разрушить объект, размещенный в памяти конструктором. Поскольку (в разделе 13.3.2) мы наделили класс `Core` виртуальным деструктором, деструктор класса `Student_info` будет действовать корректно в любом случае: когда разрушаемый объект имеет тип `Grad` или `Core`. Определением конструктора копирования и оператора присваивания мы займемся в разделе 13.4.2.

Поскольку пользователи будут писать программы, ориентируясь на объекты класса `Student_info`, а не на объекты классов `Core` или `Grad`, класс `Student_info` должен предоставить такой же интерфейс, как и класс `Core`. Что же касается функций `name` и `grade`, то у них нет никакого “спецзадания” в отношении класса `Student_info`, поэтому эти функции переадресовывают свою работу опорному `Core`- или `Grad`-объекту, на который указывает указатель `cp`.

Однако значение указателя `cp` может быть равным 0. Это имеет место, когда пользователь создает объект класса `Student_info` с помощью конструктора по умолчанию, а затем не считывает в него данные. Если указатель `cp` равен 0, мы не можем просто

переадресовать вызовы функций опорному объекту. Поэтому нам не остается ничего другого, как сгенерировать исключительную ситуацию типа `runtime_error`, чтобы оповестить систему о возникшей проблеме.

Важно не забывать о виртуальности функции `Core::grade`. Это значит, что когда мы вызываем ее посредством указателя `cp`, то версия, реально вызываемая во время работы программы, будет зависеть от типа объекта, на который указывает `cp`. Например, если `cp` указывает на `Grad`-объект, то мы выполним функцию `Grad::grade`.

Стоит остановиться на функции `compare`, которая отличается некоторыми интересными свойствами. Вспомните, что для класса `Core` функция `compare` была глобальной и не являлась его членом, в то время как здесь она реализована как статическая функция-член. Статические функции-члены отличаются от обычных функций-членов тем, что они не оказывают воздействия на объект данного типа класса. В отличие от других функций-членов, они связаны с самим классом, а не с конкретным объектом этого класса. По сути, они не могут получить доступ к нестатическим членам данных объектов этого класса: ведь если нет объекта, связанного с этой функцией, то и нет членов, которые можно было бы использовать.

Что касается нашей конкретной задачи, то весьма кстати, что `static`-функции-члены обладают одним существенным преимуществом: их имена находятся в пределах области видимости их класса. Поэтому, когда мы говорим, что функция `compare` — `static`-член, мы тем самым определяем функцию с именем `Student_info::compare`. Поскольку эта функция имеет составное имя, она не перегружает функцию `compare`, которую мы использовали для сравнения `Core`-объектов. Следовательно, наши пользователи смогут “спокойно” вызывать функцию `sort`, передавая ей в качестве аргумента функцию `Student_info::compare`, а компилятор “поймет”, какую функцию мы имеем в виду.

Не менее интересной является и реализация этой функции. Она использует функцию `Student_info::name`, чтобы заполучить имена студентов, хранимые в записях. Переведем дух и подумаем, что здесь происходит. При обращении к функции `Student_info::name` будет вызвана функция `Core::name`, если указатель `cp` установлен (не равен нулю). Если же `cp` равен 0, то функция `name` сгенерирует исключение, которое доходит до автора вызова функции `compare`. Поскольку функция `compare` использует `public`-интерфейс с классом `Student_info`, эта функция не нуждается в прямой проверке указателя `cp`. Как и в случае использования другого кода пользовательского уровня, она передает эту проблему классу `Student_info`.

13.4.1. Считывание дескриптора

Функция `read` должна справиться с тремя обязанностями: освободить предыдущий объект (если таковой существует), с которым связан текущий дескриптор; выяснить тип объекта, подлежащий считыванию; и расположить в памяти объект нужного типа, инициализировав его значением, считанным из потока.

```
istream& Student_info::read(istream& is)
{
    delete cp;    // Удаляем предыдущий объект, если он есть.

    char ch;
    is >> ch;      // Считываем тип записи.

    if (ch == 'u') {
        cp = new Core(is);
    } else {
        cp = new Grad(is);
    }
}
```

```
    return is;
}
```

Функция `read` начинает свою работу с освобождения существующего объекта (если таковой имеется), с которым ранее был связан дескрипторный объект. Перед удалением объекта нам не нужно проверять, равен ли указатель `sr` нулю, поскольку язык C++ гарантирует безопасность удаления нулевого указателя. Освободив старое значение, мы вполне готовы прочитать новое. Но, принимая “кота в мешке”, мы осторожно считываем и “пробуем на вкус” лишь первый символ строки. Протестировав его, создаем объект соответствующего типа, инициализируя этот объект посредством выполнения подходящего конструктора, который принимает в качестве аргумента `istream`-объект. Эти конструкторы вызывают собственные функции `read`, чтобы прочитать в создаваемый объект значения из входного потока. Построив объект, мы сохраняем указатель на него в переменной `sr`. Рассматриваемая функция завершается возвратом объекта потока, который был ей передан в качестве аргумента.

13.4.2. Копирование дескрипторных объектов

При управлении `Core`-указателем не обойтись без конструктора копирования и оператора присваивания. Конструктор размещает в памяти этот указатель в качестве побочного эффекта вызова функции `read`. При копировании объекта класса `Student_info` мы должны разместить в памяти новый объект и инициализировать его значениями, “взятыми” из копируемого объекта. Однако возникает вопрос: объект какого типа мы копируем? Оказывается, нет очевидного способа узнать, на какой объект (класса `Core` или типа, выведенного из класса `Core`) указывает объект класса `Student_info`, который мы копируем.

Эту проблему можно решить, наделив класс `Core` и производные от него классы новой виртуальной функцией. Эта функция должна создать новый объект, который содержит копии значений исходного объекта.

```
class Core {
    friend class Student_info;
protected:
    virtual Core* clone() const { return new Core(*this); }
    // Все остальное как прежде.
};
```

Функция `clone` вполне оправдывает наши ожидания, причем с удивительным лаконизмом. Мы размещаем в памяти новый `Core`-объект и используем конструктор копирования класса `Core`, чтобы обеспечить этот новый объект соответствующими значениями. Вспомните: в классе `Core` конструктор копирования определен неявно. Тем не менее из раздела 11.3.5 мы знаем, что он все же существует: компилятор по умолчанию синтезировал конструктор копирования, который должен скопировать в новый объект каждый член из существующего объекта класса `Core`.

Поскольку функция `clone` создана как артефакт нашей реализации, мы не добавили ее в `public`-интерфейс класса `Core`. Тот факт, что `clone` — защищенная (`protected`) функция, означает, что мы должны возвести класс `Student_info` “в ранг” друга (`friend`) класса `Core`, чтобы `Student_info`-объекты могли получить доступ к функции `clone`. “Дружба” классов аналогична отношениям между `friend`-функцией и классом (см. раздел 12.3.2). Как мы знаем, `friend`-функции имеют доступ к `private`- и `protected`-членам класса. “Звание” `friend`-класса имеет такой же эффект, как если бы все члены этого класса стали `friend`-членами. Другими словами, добавляя инструкцию

```
friend class student_info;
```

в определение класса Core, мы заявляем, что все функции-члены класса Student_info могут получить доступ ко всем private- и protected-членам класса Core.

Добавив virtual-функцию clone в базовый класс, мы должны не забыть переопределить эту функцию в производном классе, чтобы при клонировании объекта производного класса мы размещали в памяти новый объект класса Grad.

```
class Grad {  
protected:  
    Grad* clone() const { return new Grad(*this); }  
    // Все остальное как прежде.  
};
```

Как и в случае функции Core::clone, мы размещаем в памяти новый объект в виде копии объекта *this, но на этот раз мы возвращаем не Core*, а Grad*-указатель. Обычно, когда производный класс переопределяет некоторую функцию базового класса, список параметров и тип значения, возвращаемого этой функцией, должны быть идентичны соответствующим атрибутам функции базового класса. Но если функция базового класса возвращает указатель (или ссылку) на базовый класс, то функция производного класса может возвращать указатель (или ссылку) на соответствующий производный класс.

Нам не нужно делать класс Student_info другом класса Grad, даже несмотря на то что "дружба" не наследуется, поскольку наш класс Student_info никогда прямо не обращается к функции Grad::clone; он делает это только посредством виртуальных обращений к функции Core::clone, к которой он может получить доступ на основании "дружбы" с классом Core.

После внесения рассмотренных выше изменений мы можем реализовать конструктор копирования и оператор присваивания.

```
student_info::student_info(const Student_info& s): cp(0)  
{  
    if (s.cp) cp = s.cp->clone();  
}  
Student_info& Student_info::operator=(const Student_info& s)  
{  
    if (&s != this) {  
        delete cp;  
        if (s.cp)  
            cp = s.cp->clone();  
        else  
            cp = 0;  
    }  
    return *this;  
}
```

В конструкторе копирования мы инициализируем указатель cp значением 0 и вызываем функцию clone, если есть что клонировать. В противном случае (когда нечего клонировать) указатель cp останется равным 0; это говорит о том, что дескриптор не связан ни с каким объектом. Оператор присваивания также вызывает функцию clone при выполнении определенного условия. Конечно, здесь и условие другое, и действия, совершаемые перед вызовом функции clone, тоже другие. Прежде всего, мы должны не допустить самоприсваивания, проверив равенство адресов двух операндов. Если в операции присваивания участвуют различные объекты, мы должны освободить объект, на который указывает cp в данный момент, а затем уже заставить указатель cp указывать на новый объект, создаваемый функцией clone.

Если указатель `sr` равен 0, ни конструктор копирования, ни оператор присваивания не выполняют практически никаких действий, поскольку совершенно допустимо копировать или присваивать ни с чем не связанный дескриптор.

13.5. Использование дескрипторного класса

Завершив построение дескрипторного класса, мы можем “подключить” его, чтобы наша исходная программа из раздела 9.6 могла успешно работать, но при условии внесения еще одного изменения.

```
int main()
{
    vector<Student_info> students;
    Student_info record;
    string::size_type maxlen = 0;

    // Считываем и сохраняем данные.
    while (record.read(cin)) {
        maxlen = max(maxlen, record.name().size());
        students.push_back(record);
    }

    // Располагаем записи в алфавитном порядке.
    sort(students.begin(),
        students.end(), Student_info::compare);

    // Выводим имена и итоговые оценки.
    for (vector<Student_info>::size_type i = 0;
        i != students.size(); ++i) {
        cout << students[i].name()
            << string(maxlen + 1 - students[i].name.size(),
                ' ');
        try {
            double final_grade = students[i].grade();
            streamsize prec = cout.precision();
            cout << setprecision(3) << final_grade
                << setprecision(prec) << endl;
        } catch (domain_error e) {
            cout << e.what() << endl;
        }
    }
    return 0;
}
```

Теперь в цикле ввода данных считываются и обрабатываются два вида записей. Записи одного вида представляют студентов, к которым предъявляются требования освоить только базовый университетский курс, а записи другого вида — студентов-выпускников. Успешная работа этого цикла обеспечивается возможностью функции `read` класса `Student_info` считывать записи любого вида. Эта функция сначала считывает символ, который является признаком того или иного вида записи, подлежащей считыванию, а затем размещает объект соответствующего типа, инициализируя его данными из входного потока. Она создает опорный `Core`- или `Grad`-объект непосредственно во время считывания данных и сохраняет указатель на новый объект в переменной `record`. Объект класса `Student_info` копируется в вектор, причем копирование этого объекта является побочным эффектом выполнения конструктора копирования класса `Student_info`.

Следующий этап — сортировка данных в алфавитном порядке с помощью функции `sort`, которой в качестве аргумента передается функция `Student_info::compare`.

Эта функция вызывает функцию `name`, определенную в базовом классе для сравнения имен двух объектов.

Цикл вывода данных остался неизменным. На каждой итерации этого цикла элемент `students[i]` представляет очередной объект класса `Student_info`. Этот объект содержит указатель на объект либо класса `Core`, либо класса `Grad`. При вызове функции `grade` для `Student_info`-объекта используется указатель, чтобы вызвать `virtual`-функцию `grade` для опорного объекта. Тип объекта, на который указывает дескриптор, только во время работы программы определит, какая версия функции `grade` будет вызвана.

Наконец, объекты, которые были размещены в памяти при выполнении функции `read` класса `Student_info`, будут автоматически освобождены при выходе из функции `main`. По завершении функции `main` будет разрушен созданный в ней вектор. Деструктор для вектора `students` разрушит каждый его элемент, что в свою очередь вызовет деструктор для каждого объекта класса `Student_info`. В процессе своей работы этот деструктор удалит (с помощью оператора `delete`) каждый объект, размещенный в памяти функцией `read`.

13.6. Вникнем в некоторые подробности

Несмотря на могущество идей наследования и динамического связывания, они могут показаться непостижимыми, по крайней мере на первых порах их освоения. Теперь, когда мы рассмотрели пример использования этих идей, давайте разберемся в некоторых тонкостях, которые часто вызывают проблемы.

13.6.1. Наследование и контейнеры

В разделе 13.3.1 мы отметили, что, говоря о желании хранить объекты класса `Core` в каком-нибудь контейнере, мы имеем в виду, что некоторый контейнер будет хранить `Core`-объекты и ничего кроме `Core`-объектов. Это утверждение может кого-то удивить: казалось бы, мы должны иметь возможность хранить в контейнере `Core`-объекты или объекты класса, выведенного из класса `Core`. Но если вспомнить нашу собственную реализацию класса `Vec` из главы 11, то мы знаем, что на некотором этапе объект класса `Vec` должен выделить область памяти для элементов, которые он содержит. Выделяя память, мы точно указываем тип объектов, подлежащих размещению в памяти компьютера. Не существует никакого `virtual`-подобного механизма, который бы определял, какой тип объекта нам понадобится, чтобы выделить достаточный объем памяти для хранения этого объекта.

Возможно, еще более удивительными окажутся последствия нашей настойчивости в определении вектора типа `vector<Core>`, в котором мы собирались разместить либо `Core`-, либо `Grad`-объекты. Да, мы могли бы так поступить, но результаты, скорее всего, удивили бы нас. Например, рассмотрим следующий код.

```
vector<Core> students;
Grad g(cin);           // считываем объект класса Grad.
students.push_back(g);  // Сохраняем Core-часть (!) объекта g
                        // в векторе students.
```

Нам разрешено хранить объект класса `Grad` в векторе `students`, поскольку мы можем использовать `Grad`-объект везде, где требуется ссылка на `Core`-объект. Функция `push_back` принимает ссылку на тип элемента вектора, поэтому мы можем передать объект `g` функции `push_back`. Но когда мы поместим этот объект в вектор `students`, будет выполнено копирование только `Core`-части объекта `g`! Как отмеча-

лось в разделе 13.2.2, именно такое поведение нам и нужно, хотя это может кого-то обескуражить, особенно на первых порах. Оказывается, функция `push_back`, ожидая получить `Core`-объект, создаст элемент для объекта класса `Core`, копируя только `Core`-части этого объекта и игнорируя все, что относится к классу `Grad`.

13.6.2. Какая функция вам нужна

Важно понимать, что при наличии одинаковых имен функций базового и производного классов, но при несовпадении количества и типов параметров эти функции будут вести себя как совершенно разные функции. Например, мы могли бы добавить в нашу иерархию функцию доступа, которую можно было бы использовать для изменения оценки студента по последнему экзамену. Для `Core`-студентов эта функция должна установить только итоговую оценку, а для `Grad`-студентов она должна принимать два параметра (второй параметр предназначен для установки оценки за диссертацию).

```
void Core::regrade(double d) { final = d; }  
void Grad::regrade(double d1, double d2) { final = d1;  
                                         thesis = d2; }
```

Если `r` — это ссылка на объект класса `Core`, то рассмотрим следующие вызовы.

```
r.regrade(100);    // Нет проблем, вызывается  
                  // функция Core::regrade.  
r.regrade(100, 100); // Ошибка компиляции, поскольку функция  
                  // Core::regrade принимает один аргумент.
```

Второй вызов ошибочен, даже если ссылка `r` ссылается на объект типа `Grad`. Тип переменной `r` — это ссылка на объект класса `Core`, а версия функции `regrade` в классе `Core` принимает одно значение типа `double`.

Более удивителен результат подобных вызовов, если окажется, что `r` — ссылка на объект класса `Grad`.

```
r.regrade(100);    // Ошибка компиляции, поскольку функция  
                  // Grad::regrade принимает два аргумента.  
r.regrade(100, 100); // Нет проблем, вызывается  
                  // функция Grad::regrade.
```

На этот раз `r` — ссылка на объект класса `Grad`. Функция `regrade`, которая применяется к `Grad`-объектам, принимает два аргумента. Несмотря на то что существует версия базового класса, которая принимает один аргумент, она эффективно сокрыта самим лишь существованием функции `regrade` в производном классе. Если мы захотим выполнить версию из базового класса, должны вызвать ее явным образом.

```
r.Core::regrade(100); // Нет проблем, вызывается  
                     // функция Core::regrade.
```

Если мы захотим использовать функцию `regrade` как виртуальную, должны наделять ее одинаковым интерфейсом в базовом и производном классах, а это можно сделать, усилив `Core`-версию дополнительным неиспользуемым параметром с аргументом, действующим по умолчанию.

```
virtual void Core::regrade(double d, double = 0) { final = d; }
```

13.7. Резюме

Наследование позволяет моделировать классы, которые (за некоторыми исключениями) подобны один другому.

```

class base {
public:
    // Общедоступный интерфейс.
protected:
    // Члены реализации, доступные производным классам.
private:
    // Члены реализации, доступные только классу base.
};
// Общедоступный интерфейс класса base является частью
// интерфейса класса derived.
class derived: public base { ... };

```

Производные классы могут переопределять операции базового класса и добавлять собственные члены. Классы также могут наследовать private-часть базового класса.

```

class priv_derived: private base { ... };

```

Но это встречается крайне редко и обычно используется только ради удобства C++-среды.

Объект public-производного типа можно использовать везде, где ожидается объект, ссылка или указатель на объект базового класса.

Цепочки выведения классов могут быть многоуровневыми.

```

class derived2: public derived { ... };

```

Объекты типа derived2 имеют derived-часть, которая в свою очередь имеет base-часть. Следовательно, derived2-объекты обладают свойствами derived- и base-объектов.

Объекты производного класса создаются посредством выделения объема памяти, достаточного для целого объекта, построения части (частей) базового класса и, наконец, построения части, относящейся только к производному классу. Выбор выполняемого производного конструктора зависит, как обычно, от аргументов, используемых при создании объекта производного класса. Этот конструктор, посредством своего списка инициализаторов, может передавать аргументы, используемые при построении его непосредственного базового класса. Если инициализатор конструктора явно не инициализирует свою базовую часть, выполняется конструктор по умолчанию базового класса.

Динамическое связывание означает возможность выбора (во время работы программы) выполняемой функции на основе реального типа объекта, для которого вызывается данная функция. По сути, динамическое связывание — это инструмент вызова виртуальных функций посредством указателя или ссылки на объект. Виртуальность функции передается по наследству производным классам, поэтому спецификатор `virtual` в их определениях повторять не нужно.

Производные классы не нуждаются в переопределении своих `virtual`-функций. Если в классе некоторая `virtual`-функция не переопределяется, значит, этот класс наследует ближайшее определение этой функции. Однако любая виртуальная функция, которую содержит класс, должна быть определена. Часто причиной загадочных сообщений компилятора об ошибках служит наличие объявления `virtual`-функции без ее определения.

Переопределение функций. Функция-член производного класса переопределяет функцию базового класса с таким же именем, если эти две функции имеют одинаковое количество параметров при соответствующем совпадении их типов, причем обе должны быть (или обе не должны быть) `const`-функциями. При этом типы значений, возвращаемых этими функциями, также должны совпадать, за исключением случая, описанного в разделе 13.4.2: если функция базового класса возвращает указатель (или ссылку) на объект этого класса, то функция производного класса может возвращать указатель (или ссылку) на объ-

ект производного класса. Если список аргументов не совпадает, функции базового и производного классов совершенно не связаны между собой.

Виртуальные деструкторы. Если указатель на базовый класс используется для удаления (с помощью оператора `delete`) объекта, который в действительности может оказаться объектом производного класса, то базовому классу необходимо иметь `virtual`-деструктор. Если класс не имеет иной причины для определения деструктора, `virtual`-деструктор все равно должен быть определен, но с пустым телом.

```
class base {
public:
    virtual ~base() { }
};
```

Как и в случае любой другой функции, виртуальная природа деструктора наследуется производными классами, и поэтому нет необходимости в переопределении деструктора в производных классах.

Конструкторы и `virtual`-функции. При построении объекта некоторого класса его типом становится тип этого класса, даже если объект является частью объекта производного класса. Следовательно, обращения к виртуальным функциям из конструктора статически связаны с версией типа создаваемого объекта.

“Дружба” классов. Один класс может назвать другой своим другом (`friend`-классом). В этом случае “дружеские отношения” распространяются на все функции-члены этого другого класса. Однако такая дружба ни наследуется, ни передается каким-то другим способом: друзья друзей и классы, производные от друзей, не имеют никаких специальных привилегий.

Статические члены существуют как члены класса, а не как экземпляры в каждом объекте этого класса. Следовательно, для использования в `static`-функции-члене ключевое слово `this` недопустимо. Такие функции могут получать доступ только к статическим членам данных. Для целого класса существует единственный экземпляр каждого статического члена данных, который должен быть инициализирован, и обычно он размещается в исходном файле, реализующем функции-члены класса. Поскольку член класса реализуется вне определения класса, при инициализации необходимо полностью специфицировать его имя.

Значение_типа имя_класса::имя_статического_члена = Значение;

Эта синтаксическая запись означает, что `static`-член с именем *имя_статического_члена* из класса *имя_класса* имеет тип *Значение_типа* и ему присваивается начальное значение *Значение*.

Упражнения

13.0. Скомпилируйте, выполните и протестируйте программы, приведенные в этой главе.

13.1. Усовершенствуйте конструкторы классов `Cone` и `Grad` с помощью средств идентификации, чтобы при выполнении конструктора выводилось его имя и список аргументов. Для этого, например, в конструктор класса `Grad`, принимающий параметр типа `istream&`, можно добавить следующую инструкцию.

```
cerr << "Grad::Grad(istream&)" << endl;
```

Затем напишите небольшую программу, которая заставляет работать каждый конструктор. Спрогнозируйте, какой результат вы получите. Переделывайте

программу до тех пор, пока ваши прогнозы не совпадут с результатом работы программы.

- 13.2. Используя классы `Core` и `Grad`, определенные в этой главе, укажите, какая функция вызывается в каждом из следующих вызовов.

```
Core* p1 = new Core;  
Core* p2 = new Grad;  
Core s1;  
Grad s2;
```

```
p1->grade();  
p1->name();
```

```
p2->grade();  
p2->name();
```

```
s1.grade();  
s1.name();
```

```
s2.name();  
s2.grade();
```

Проверьте, насколько вы правы, добавив в функции `name` и `grade` инструкции вывода, чтобы было понятно, какая функция выполняется.

- 13.3. Класс, построенный в главе 9, включал функцию-член `valid`, которая позволяла пользователям проверить, содержит ли данный объект значения, соответствующие данным о студенте. Добавьте эту функцию в иерархическую систему классов.
- 13.4. Добавьте в эти классы функцию, которая преобразует числовую оценку студента в буквенную в соответствии с классификацией, описанной в разделе 10.3.
- 13.5. Напишите предикат для проверки, удовлетворяет ли конкретный студент всем предъявляемым ему требованиям. Другими словами, проверьте, выполнил ли студент все домашние задания, а если тестируемый студент — выпускник, то написал ли он диссертацию.
- 13.6. Добавьте в нашу систему классов еще один класс для представления студентов, успеваемость которых оценивается на основе зачетов. Предположим, что такие студенты не обязаны выполнять домашние задания, но при желании могут это делать. Если они выполняют домашние задания, эти оценки должны учитываться по обычной формуле. Если же они их не выполнили, итоговая оценка вычисляется как среднее арифметическое от оценок, полученных на экзаменах в середине и конце семестра. Проходным считается результат, равный 60 баллам и выше.
- 13.7. Добавьте в нашу систему классов еще один класс для представления студентов, сдающих тесты.
- 13.8. Напишите программу генерирования отчета об успеваемости студентов, учитывающего все четыре категории студентов.
- 13.9. Опишите, что произойдет, если оператор присваивания в разделе 13.4.2 не будет проверять возможность самоприсваивания.

ПОЧТИ АВТОМАТИЧЕСКОЕ УПРАВЛЕНИЕ ПАМЯТЬЮ

В главе 13 при построении дескрипторного класса `Student_info` мы объединили две отдельные абстракции. Построенный нами класс не только служил интерфейсом для выполнения операций с записями студентов, но и позволял управлять указателем на объект реализации. Объединение двух независимых абстракций в одном классе часто является признаком слабого проекта.

Хотелось бы определить класс, подобный классу `Student_info`, но это должен быть абсолютно интерфейсный класс. Такие интерфейсные классы весьма распространены в C++, особенно при взаимодействии с иерархией наследования. Мы подготовимся к созданию интерфейсного класса, попытавшись передать детали реализации другому классу, который будет вести себя как указатель и при этом управлять распределением памяти. Отделив интерфейсный класс от класса, подобного указателю (класса-указателя), мы сможем использовать один класс-указатель с несколькими интерфейсными.

Как будет показано ниже, мы можем также использовать подобные классы для повышения производительности программ, которые часто выполняют операции по управлению памятью. Организовав несколько объектов, подобных классу-указателю, так, чтобы они ссылались на единый опорный объект (когда это будет нужно), мы можем избежать излишнего копирования объектов.

Большая часть этой главы посвящена ответу на один-единственный вопрос: что означает копирование объекта? На первый взгляд может показаться, что этот вопрос имеет очевидный ответ: копия — это отдельный объект, который имеет все свойства исходного объекта. Однако в тот момент, когда для одного объекта реализуется возможность указывать на другой объект, этот вопрос резко усложняется: если объект `x` указывает на объект `y`, нужно ли, чтобы копирование `x` приводило также к копированию `y`?

Иногда ответ на последний вопрос вполне ясен: если `y` — член объекта `x`, ответом должен быть вариант “да”, а если `x` — не более чем указатель, который указывает на `y`, ответом будет “нет”. В этой главе мы определим три различные версии нашего класса-указателя, и каждая из них будет отличаться от других тем, как в ней определено копирование.

Эти вопросы о копировании (да и сама идея создания класса-указателя) представляют собой довольно абстрактные понятия. Поскольку мы таки возьмемся за реализацию этих абстракций, неудивительно, что эту главу, безусловно, можно считать “самой абстрактной” во всей книге. Поэтому от вас потребуются больше внимания к рассматриваемому здесь материалу.

14.1. Дескрипторы, которые копируют свои объекты

Давайте снова мысленно вернемся к задаче вычисления итоговой оценки, которую мы решали в главе 13. В решении этой задачи нам нужно было сохранить и обработать коллекцию объектов, представляющих различные типы студентов. Эти объекты имели один из двух возможных типов, связанных наследованием, причем возможность расширения “диапазона типов” была добавлена позже. В нашем первом решении (см. раздел 13.3.1) использовались указатели, которые позволяли хранить смешанную коллекцию объектов. Каждый из этих указателей мог указывать на `Core`-объект или на объект класса, выведенного из класса `Core`. Пользовательский код должен был нести ответственность за динамическое размещение объектов в памяти и последующее освобождение этой памяти. Программа была загромождена деталями, связанными с управлением указателями, и потому в целом сложна.

Дело в том, что указатель — это примитивная низкоуровневая структура данных. Общеизвестно, что программирование с использованием указателей — это “распахнутые двери” для различных ошибок. Многие проблемы возникают именно потому, что указатели не зависят от объектов, на которые они указывают, что приводит в конце концов к различного рода “ловушкам”.

- При копировании указателя соответствующий объект не копируется; это ведет к неожиданному результату, если оказывается, что два указателя непреднамеренно указывают на один и тот же объект.
- При разрушении указателя объект, на который он указывает, не разрушается, что приводит к утечке памяти.
- При удалении объекта без разрушения указателя на него образуется висячий указатель, который может стать причиной неопределенного поведения программы, если она продолжает его использовать.
- При создании указателя без его инициализации образуется ни с чем не связанный указатель, который может стать причиной неопределенного поведения программы, если она продолжает его использовать.

В разделе 13.5 мы снова вернулись к решению задачи вычисления итоговых оценок, но на этот раз используя дескрипторный класс `Student_info`. Поскольку этот класс управляет указателями, код наших пользователей должен работать с объектами класса `Student_info`, а не с самими указателями. Однако класс `Student_info` (в том его виде) был тесно связан с `Core`-иерархией: он содержал операции, которые отражали эту взаимосвязь в `public`-интерфейсе `Core`-класса.

Теперь нам хотелось бы отделить эти абстракции одну от другой. Мы по-прежнему будем использовать класс `Student_info` для предоставления интерфейса, но управление “дескриптором” будет возложено на другой класс. Иными словами, этот другой класс будет управлять указателями на объекты реализации. Поведение этого нового типа может быть и будет независимым от типа объектов, с которыми этот дескриптор может быть связан.

14.1.1. Обобщенный дескрипторный класс

Мы хотим, чтобы наш класс был независимым от типа объекта, которым он управляет, поэтому, как мы уже знаем, он должен быть шаблонным. А поскольку мы также хотим, чтобы наш класс инкапсулировал поведение дескриптора, назовем его `handle`. Ниже перечислены свойства, которыми должен обладать наш класс.

- Объект класса `handle` — это значение, которое связано с определенным объектом.
- Мы должны иметь возможность скопировать объект класса `handle`.
- Мы должны иметь возможность протестировать объект класса `handle`, чтобы определить, связан ли он с другим объектом.
- Мы должны иметь возможность так использовать объект класса `handle`, чтобы в случае, когда он будет указывать на объект класса, который принадлежит некоторой иерархии наследования, “срабатывало” его полиморфное поведение. Другими словами, мы хотели бы, чтобы при вызове некоторой виртуальной функции с помощью нашего класса C++-среда для выполнения выбирала нужную функцию динамически, как если бы мы вызывали эту функцию посредством реального указателя.

Наш класс `handle` будет иметь ограниченный интерфейс: если `handle`-объект связан с некоторым другим объектом, то класс `handle` должен взять на себя управление памятью для этого объекта. После того как пользователи свяжут один `handle`-объект с любым другим, они не должны получать доступ к этому объекту с помощью указателя, т.е. любой доступ к нему должен обеспечивать `handle`-объект. Эти ограничения позволяют `handle`-объектам избежать проблем, присущих встроенным указателям. При копировании `handle`-объекта мы создаем новую копию объекта, чтобы каждый `handle`-объект указывал на собственную копию. При разрушении `handle`-объекта им будет разрушен и связанный с ним объект, причем это самый простой способ освободить объект. Пользователи могут создавать несвязанные `handle`-объекты, но если они попытаются получить доступ к объекту, на который указывает (вернее, не указывает) ни с чем не связанный `handle`-объект, будет сгенерировано исключение. Чтобы избежать исключения, можно проверить `handle`-объект на наличие связи.

Эти свойства подобны тем, которые мы реализовали в классе `Student_info`. Конструктор копирования и оператор присваивания этого класса вызывали функцию `clone`, чтобы скопировать соответствующий объект класса `Core`. Деструктор класса `Student_info` разрушал и объект класса `Core`. Операции, в которых задействован опорный объект, включали предварительную проверку, позволяющую убедиться в том, что объект класса `Student_info` связан с некоторым реальным объектом. Поэтому наша задача — создать класс, который инкапсулирует такое поведение, но при этом позволяет управлять объектом любого типа.

```
template <class T> class handle {
public:
    handle(): p(0) { }
    handle(const handle& s): p(0) { if (s.p) p = s.p->clone(); }
    handle& operator=(const handle&);
    ~handle() { delete p; }

    handle(T* t): p(t) { }

    operator bool() const { return p; }
    T& operator*() const;
    T* operator->() const;

private:
    T* p;
};
```

Класс `handle` — это шаблонный класс, поэтому мы можем создать `handle`-объекты, которые будут указывать на любой тип. Каждый объект типа `handle<T>` содержит указатель на некоторый объект. Этим указателем можно управлять посред-

вом операций, определенных в классе `handle<T>`. Если не считать изменений, связанных с именем переменной, первые четыре функции идентичны соответствующим версиям, определенным в классе `Student_info`. Конструктор по умолчанию устанавливает указатель равным нулю, чтобы обозначить, что данный объект класса `handle` ни с чем не связан. Конструктор копирования (при выполнении заданного условия) вызывает функцию `clone` для соответствующего объекта, чтобы создать новую копию объекта. Деструктор класса `handle` освобождает этот объект. Оператор присваивания, подобно конструктору копирования (при выполнении заданного условия), вызывает функцию `clone`, чтобы создать новую копию этого объекта.

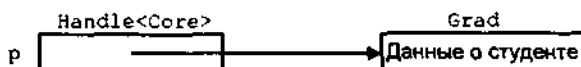
```
template<class T>
handle<T>& handle<T>::operator=(const handle& rhs)
{
    if (&rhs != this) {
        delete p;
        p = rhs.p ? rhs.p->clone() : 0;
    }
    return *this;
}
```

Оператор присваивания, как обычно, сначала проверяет возможность самоприсваивания и, если оно имеет место, не выполняет никаких действий. Если тестирование не выявило самоприсваивания, управляемый нами объект освобождается, а затем создается копия объекта, заданного в правой части оператора присваивания. Чтобы узнать, не опасно ли вызывать функцию `clone`, инструкция, которая создает копию объекта, использует условный оператор (см. раздел 3.2.2). Если указатель `rhs.p` установлен, мы вызываем функцию `rhs.p->clone` и присваиваем полученное (в результате этого вызова) значение указателя члену `p`. В противном случае мы устанавливаем член `p` равным 0.

Поскольку класс `handle` моделирует поведение указателя, нам нужен способ, позволяющий связать указатель с реальным объектом, и эта связь устанавливается в конструкторе, который принимает аргумент типа `T*`. Этот конструктор запоминает заданный указатель, тем самым связывая `handle`-объект с объектом, на который указывает аргумент `t`. Например, используя следующее определение

```
handle<Core> student(new Grad);,
```

мы создаем `handle`-объект с именем `student`, содержащий указатель `Core*`, который инициализируем таким образом, чтобы он указывал на только что созданный объект типа `Grad`.



Наконец, мы определяем три операторные функции. Первая из них, `operator bool()`, позволяет пользователям тестировать значение `handle`-объекта в любом условии. Если `handle`-объект связан с каким-нибудь объектом, эта функция возвращает значение `true`, в противном случае — `false`. Две другие функции (`operator*` и `operator->`) позволяют получить доступ к объекту, связанному с `handle`-объектом.

```
template <class T>
T& handle<T>::operator*() const
{
    if (p)
        return *p;
    throw runtime_error("несвязанный handle-объект.");
}
```



```

template <class T>
T* Handle<T>::operator->() const
{
    if (p)
        return p;
    throw runtime_error("Несвязанный handle-объект.");
}

```

В результате применения к указателю встроенного унарного оператора “*” генерируется объект, на который указывает этот указатель. Здесь мы определяем собственный оператор “*”, чтобы он в применении к handle-объекту генерировал значение, получаемое в результате применения встроенного оператора “*” к указателю, который является членом данных handle-объекта. Итак, при заданном объекте student выражение *student сгенерирует результат применения оператора “*” к выражению student.p (в предположении, что мы можем получить доступ к члену p объекта student). Другими словами, результатом выражения *student будет ссылка на Grad-объект, который мы создали при инициализации объекта student.

Оператор “->” несколько сложнее. Внешне он выглядит как бинарный оператор, но на самом деле ведет себя совсем не так, как “подобало бы” обычному бинарному оператору. Подобно оператору разрешения области видимости или оператору точки, данный оператор (“->”) используется для получения доступа к члену, имя которого указывается в качестве правого операнда по отношению к объекту, заданному его левым операндом. Поскольку имена не являются выражениями, у нас нет прямого доступа к имени, затребованному пользователем. При этом язык C++ требует, чтобы мы определили оператор “->” для возврата значения, с которым можно обращаться, как с указателем. Определяя функцию operator->, мы говорим, что если x — это значение типа, который определяет функцию operator->, то выражение

x->y

эквивалентно следующему.

(x.operator->())->y

В этом случае функция operator-> возвращает указатель, который содержится в его же объекте. Итак, для объекта student выражение

student->y

эквивалентно выражению

(student.operator->())->y,

которое в соответствии со способом определения функции operator-> эквивалентно следующему (если игнорировать то, что защита обычно не позволяет напрямую получить доступ к члену student.p).

student.p->y

Таким образом, применение оператора “->” эквивалентно переадресации вызовов, сделанных посредством handle-объекта, опорному указателю, который является членом этого handle-объекта.

Одним из наших намерений в отношении класса Handle было сохранить полиморфное поведение, связанное со встроенными указателями. Разобравшись с определениями операторных функций operator* и operator->, становится понятно, что мы добились поставленной цели. Эти операции генерируют либо ссылку, либо указатель, посредством которых

мы получаем динамическое связывание. Например, выполняя функцию `student->grade()`, мы тем самым вызываем функцию `grade`, используя указатель `p` внутри объекта `student`. Конкретная версия функции `grade`, которая будет реально выполнена, зависит от типа объекта, на который указывает член `p`. Если объект `student` указывает на объект класса `Grad` (с помощью которого объект `student` был инициализирован), то в действительности будет вызвана функция `Grad::grade`. Аналогично функция `operator*` генерирует ссылку, поэтому при вычислении выражения `(*student).grade()` функция `grade` будет вызвана посредством ссылки, а какой именно будет эта функция `grade`, C++-среда решит во время работы программы.

14.1.2. Использование обобщенного дескриптора

Теперь мы могли бы переделать ориентированную на указатели программу вычисления итоговых оценок из раздела 13.3.1, используя `handle`-объекты.

```
int main()
{
    vector< handle<Core> > students;           // Измененный тип.
    handle<Core> record;                       // Измененный тип.
    char ch;
    string::size_type maxlen = 0;

    // Считываем и сохраняем данные.
    while (cin >> ch) {
        if (ch == 'u')
            record = new Core; // Размещаем в памяти
                               // объект класса Core.
        else
            record = new Grad; // Размещаем в памяти
                               // объект класса Grad.
        Record->read(cin);      // handle<T>::->, т.е.
                               // виртуальный вызов read.
        maxlen = max(maxlen,
                      record->name().size()); // handle<T>::->
        students.push_back(record);
    }

    // Функцию compare нужно переписать для обработки
    // аргументов типа const handle<Core>&.
    sort(students.begin(),
         students.end(), compare_Core_handles);

    // Выводим имена и оценки.
    for (vector< handle<Core> >::size_type i = 0;
         i != students.size(); ++i) {
        // Элемент students[i] - это handle-объект, который
        // необходимо разыменовать для вызова нужных функций.
        cout << students[i]->name()
              << string(maxlen + 1 - students[i]->name.size(),
                        ' ');

        try {
            double final_grade = students[i]->grade();
            streamsize prec = cout.precision();
            cout << setprecision(3) << final_grade
                  << setprecision(prec) << endl;
        } catch (domain_error e) {
            cout << e.what() << endl;
        }

        // Инструкция delete отсутствует.
    }
    return 0;
}
```

Эта программа вместо Core*-объектов хранит объекты типа handle<Core>, и поэтому, как и в разделе 13.3.1, нам придется позаботиться о неперегруженной операции сравнения для операндов типа const handle<Core>&, которую мы можем передать функции sort. Ее реализацию мы оставим читателю в качестве упражнения, но предположим, что она имеет имя compare_Core_handles.

Поскольку наш класс handle использует член clone объекта, с которым он связан, нам также нужно изменить класс Core (см. раздел 13.4.2), чтобы разрешить объекту типа handle<Core> использовать этот член, либо сделав класс handle<Core> другом (friend), либо сделав функцию clone открытой (public).

Все остальные различия связаны с циклом вывода данных. При разыменовании элемента students[i] генерируется handle-объект, содержащий операторную функцию operator->, которую мы используем для доступа к функциям name и grade посредством опорного объекта Core*. Например, в выражении students[i]->grade() используется перегруженный оператор "->", поэтому здесь успешно вызывается функция students[i].p->grade(). Поскольку функция grade виртуальна, в действительности будет вызвана версия, которая соответствует типу объекта, на который указывает значение students[i].p. Более того, поскольку заботу об управлении памятью берет на себя класс handle, нам больше не нужно применять инструкцию delete для удаления объектов, на которые указывают элементы вектора students.

При этом важно то, что мы можем также изменить реализацию класса Student_info, который стал теперь "чистокровным" интерфейсным классом, передав работу по управлению указателями классу handle.

```
class Student_info {
public:
    Student_info() { }
    Student_info(std::istream& is) { read(is); }
    // конструктор копирования, оператор присваивания
    // и деструктор больше не нужны.

    std::istream& read(std::istream&);

    std::string name() const {
        if (cp) return cp->name();
        else throw runtime_error(
            "Неинициализированный объект Student");
    }

    double grade() const {
        if (cp) return cp->grade();
        else throw runtime_error(
            "Неинициализированный объект Student");
    }

    static bool compare(const Student_info& s1,
                        const Student_info& s2) {
        return s1.name() < s2.name();
    }
private:
    handle<Core> cp;
};
```

В этой версии класса Student_info член cp имеет тип handle<Core>, а не Core*. Следовательно, нам больше не нужно заниматься реализацией функций управления копированием, поскольку опорным объектом теперь управляет handle-объект. Другие конструкторы действуют как прежде. Функции name и grade выглядят, как близнецы, но их выполнение зависит от преобразования члена cp в значение типа bool. Это

преобразование происходит при тестировании члена `sr` и при вызове перегруженного оператора `operator->` из класса `handle`, который используется для доступа к функциям, соответствующим опорным объектам.

Для того чтобы завершить наш новый вариант реализации класса `Student_info`, необходимо написать функцию `read`.

```
istream& Student_info::read(istream& is)
{
    char ch;
    is >> ch; // Получаем тип записи.

    // Размещаем в памяти новый объект соответствующего типа.
    // Используем handle<T>(T*) для построения объекта
    типа handle<Core> из указателя на этот объект.
    // Вызываем функцию handle<T>::operator=, чтобы присвоить
    объект типа handle<Core> левому операнду.
    if (ch == 'U')
        cp = new Core(is);
    else
        cp = new Grad(is);

    return is;
}
```

Этот код напоминает представленную выше функцию `Student_info::read`, но выполняется он совсем по-другому. Инструкции `delete`, как видите, больше нет, поскольку в процессе присваивания члену `sr` соответствующий объект, если потребуется, будет освобожден. При выполнении выражения `new Core(is)` мы получаем доступ к объекту `Core*`, который неявно преобразуем в объект типа `handle<Core>` с помощью конструктора `handle(T*)`. Это `handle`-значение затем присваивается члену `sr` с помощью оператора присваивания класса `handle`, который автоматически удаляет (посредством инструкции `delete`) объект, на который, возможно, указывал предыдущий `handle`-объект. В результате этого присваивания создается и разрушается созданная нами дополнительная копия `Core`-объекта (чего мы попытаемся избежать).

14.2. Дескрипторы с подсчетом количества ссылок

На данном этапе нам удалось отделить действия, связанные с управлением указателями, от интерфейса класса. Теперь мы можем использовать `handle`-объекты для реализации большого множества интерфейсных классов, ни один из которых не обязан заниматься распределением памяти. Однако в нашем классе `handle` еще не решена проблема, связанная с тем, что при копировании или присваивании объектов опорные данные копируются даже в случае, когда в этом нет необходимости. Все дело в том, что объект класса `handle` всегда копирует объект, с которым он связан.

Конечно, мы хотели бы сами управлять созданием таких копий. Например, было бы неплохо иметь объекты, которые представляли бы собой копии один другого для совместного использования их опорных данных. Для таких классов не нужно обеспечивать поведение, характерное для значений. Ведь классы могут не иметь ни единой возможности изменить свое состояние после создания объекта. В таких случаях не имеет смысла копировать опорный объект. Копирование таких объектов означало бы, что время и память были потрачены напрасно. Для поддержки подобных классов мы бы хотели создать такой класс `handle`, который бы не копировал опорный объект при копировании самого `handle`-объекта. Конечно, если мы позволим нескольким `handle`-объектам связываться с одним и тем же опорным объектом, то рано или

поздно нам все-таки придется освободить этот объект. Это придется сделать, когда будет освобожден последний handle-объект, который на него указывал.

С этой целью мы воспользуемся объектом, именуемым *счетчиком ссылок* (reference count), который будет отслеживать количество объектов, ссылающихся на некоторый другой (целевой) объект. Каждый целевой объект будет иметь связанный с ним счетчик ссылок. Наша задача — обеспечить инкрементирование счетчика ссылок при каждом создании нового дескриптора, который указывает на наш целевой объект, а также его декрементирование при освобождении очередного объекта ссылки. Когда будет освобожден последний объект ссылки, содержимое счетчика ссылок станет равным нулю. В этот момент мы будем точно знать, что целевой объект также можно разрушить — это действие уже не будет содержать никакой опасности.

Такой подход может сэкономить немалые системные ресурсы, затрачиваемые на ненужное управление памятью и копирование данных. Сначала мы построим новый класс (назовем его `Ref_handle`), который будет “знать”, как добавлять счетчики ссылок в наш класс `handle`. Затем (в следующих двух разделах) мы рассмотрим, как подсчет ссылок может оказаться полезным в определении классов, которые ведут себя подобно значениям при совместном использовании данных.

Чтобы добавить в класс механизм подсчета ссылок, мы должны разместить в памяти счетчик и изменить операции создания, копирования и разрушения объектов, чтобы они соответствующим образом обновляли содержимое этого счетчика. Каждый объект, для которого мы предоставим `ref_handle`-объект, будет иметь связанный с ним счетчик ссылок. Осталось найти ответ только на один вопрос: где хранить этот счетчик. В общем случае мы не имеем доступа к исходному коду для типов, из которых хотим получить `ref_handle`-объекты, поэтому не можем просто добавить счетчик в сам тип класса. Вместо этого для управления работой счетчика мы добавим в наш класс `Ref_handle` еще один указатель. Каждый объект, к которому мы присоединили `ref_handle`-объект, также должен иметь соответствующий счетчик ссылок, который будет отслеживать, сколько копий этого объекта мы сделали.

```
template <class T> class Ref_handle {
public:
    // Управляем счетчиком ссылок и указателем.
    Ref_handle(): refptr(new size_t(1)), p(0) { }
    Ref_handle(T* t): refptr(new size_t(1)), p(t) { }
    Ref_handle(const Ref_handle& h): refptr(h.refptr), p(h.p) {
        ++*refptr;
    }

    Ref_handle& operator=(const Ref_handle&);
    ~Ref_handle();

    // Как прежде.
    operator bool() const { return p; }
    T& operator*() const {
        if (p)
            return *p;
        throw std::runtime_error(
            "Несвязанный Ref_handle-объект.");
    }
    T* operator>() const {
        if (p)
            return p;
        throw std::runtime_error(
            "Несвязанный Ref_handle-объект.");
    }
}
```

```
private:
    T* p;
    size_t* refptr;    // Добавлено.
};
```

Мы добавили в наш класс `Ref_handle` второй член данных и обновили конструкторы для инициализации этого нового члена. Конструктор по умолчанию и конструктор, который принимает аргумент типа `T*`, создают новые `Ref_handle`-объекты, поэтому они размещают новый счетчик ссылок (типа `size_t`) и устанавливают значение этого счетчика равным 1. Конструктор копирования не создает новый объект. Вместо этого он копирует указатели из объекта типа `Ref_handle<T>`, который был передан, и инкрементирует счетчик ссылок, чтобы показать, что теперь количество указателей на объект типа `T` увеличилось на единицу. Таким образом, новый объект типа `Ref_handle<T>` указывает на тот же объект типа `T` и на тот же счетчик ссылок, что и объект типа `Ref_handle<T>`, с которого мы “лепили” копию. Так, например, если `X` — это объект типа `Ref_handle<T>` и мы создаем `Y` как копию объекта `X`, то ситуация выглядит следующим образом.



Оператор присваивания также манипулирует счетчиком ссылок, а не копирует опорный объект.

```
template<class T>
Ref_handle<T>& Ref_handle<T>::operator=(const Ref_handle& rhs)
{
    ++*rhs.refptr;
    // Освобождаем левый операнд, разрушая,
    // если нужно, указатели.
    if (--*refptr == 0) {
        delete refptr;
        delete p;
    }
    // Копируем значения из правого операнда.
    refptr = rhs.refptr;
    p = rhs.p;
    return *this;
}
```

Как всегда, важно предусмотреть возможность самоприсваивания и принять соответствующие меры, которые заключаются в инкрементировании счетчика ссылок правого операнда до декрементирования счетчика ссылок левого операнда. Если оба операнда указывают на один и тот же объект, то в результате наших манипуляций счетчик ссылок останется неизменным и не будет случайно обнулен.

Если содержимое счетчика ссылок “падает” до нуля, когда мы декрементируем его, то это означает, что левый операнд является последним `Ref_handle`-объектом, связанным с опорным объектом. Поскольку мы в этом случае должны ликвидировать значение левого операнда, нам придется также удалить последнюю ссылку на этот объект. Следовательно, перед тем как перезаписывать значения в члены `refptr` и `p`, мы обязаны удалить сам объект (с помощью инструкции `delete`) и соответствующий счетчик ссылок. Мы удаляем как член `p`, так и член `refptr`, поскольку оба эти объекта были динамически размещены в памяти; следовательно, чтобы избежать утечки памяти, мы должны освободить их.

Удалив, если нужно, указатели, мы затем копируем значения правого операнда в члены данных левого операнда и, как обычно, возвращаем ссылку на левый операнд.

Деструктор, как и оператор присваивания, проверяет, является ли разрушаемый `ref_handle`-объект последним объектом, который был связан с объектом типа `T`. Если да, деструктор удаляет объекты, на которые указывают указатели разрушаемого объекта.

```
template<class T> Ref_handle<T>::~~Ref_handle()
{
    if (--*refptr == 0) {
        delete refptr;
        delete p;
    }
}
```

Эта версия класса `Ref_handle` нормально работает для классов, которые могут разделять свое состояние между копиями различных объектов, но как быть с такими классами, как `Student_info`, которые стремятся обеспечить поведение, характерное для значений? Например, если мы использовали класс `Ref_handle` для реализации класса `Student_info`, то после выполнения, скажем, следующих инструкций

```
Student_info s1(cin); // инициализируем s1 из
                      // стандартного потока.
Student_info s2 = s1; // "Копируем" это значение в объект s2.
```

два объекта `s1` и `s2` будут ссылаться на один и тот же опорный объект, даже если окажется, что `s2` является копией объекта `s1`. Если мы предпримем меры для изменения значения одного из этих объектов, тем самым изменим и значение другого объекта.

Наш исходный класс `handle`, определенный в разделе 14.1.1, обеспечивал поведение, характерное для значений, поскольку он всегда копировал соответствующий объект посредством вызова функции `clone`. Нетрудно заметить, что наш новый класс `Ref_handle` вообще никогда не вызывает функцию `clone`. А поскольку класс `Ref_handle` никогда не вызывает функцию `clone`, дескрипторы этого типа никогда не копируют объекты, с которыми они связаны. Однако данная версия класса `Ref_handle`, несомненно, имеет преимущество, заключающееся в избежании излишнего копирования данных. Но вся беда в том, что мы избегаем *всевозможного* копирования, не разбираясь, излишнее оно или нет. Надо с этим что-то делать, но что?

14.3. Дескрипторы для решения проблемы совместного использования данных

Выше мы рассмотрели два возможных определения обобщенного дескрипторного класса. Первая версия всегда копирует опорный объект, а вторая — никогда. Нетрудно предположить, что более приемлемым является дескриптор, который позволяет программе, где он используется, самой решать, когда ей нужно копировать целевой объект, а когда — нет. Такой дескрипторный класс должен сохранить эффективность класса `Ref_handle` и позволить автору класса обеспечить поведение, характерное для значений, т.е. как у объектов класса `handle`. Этот дескрипторный класс, сохраняя полезные свойства встроенных указателей, позволит избежать многих "ловушек". Итак, назовем последнюю версию дескрипторного класса именем `Ptr`, которое будет напоминать нам о том, что этим классом можно эффективно заменить встроенные указатели. В общем случае наш класс `Ptr` будет копировать объект, если мы собираемся изменить его содержимое, но при условии, что существует другой дескриптор, связанный с тем же объектом. К счастью, подсчитывая количество

ссылки, мы имеем возможность узнать, действительно ли только наш дескриптор связан со своим объектом.

Принципы создания класса `Ptr` такие же, как у класса `ref_handle`, который мы спроектировали в разделе 14.2. Нам нужно лишь добавить в этот класс еще одну функцию-член, чтобы передать “бразды правления” в руки пользователя.

```
template<class T> class Ptr {
public:
    // Новый член для копирования объекта при необходимости.
    void make_unique() {
        if (*refptr != 1) {
            --*refptr;
            refptr = new size_t(1);
            p = p? p->clone(): 0;
        }
    }

    // Остальная часть класса совпадает с классом Ref_handle,
    // за исключением его имени.
    Ptr(): refptr(new size_t(1)), p(0) { }
    Ptr(T* t): refptr(new size_t(1)), p(t) { }
    Ptr(const Ptr& h): refptr(h.refptr), p(h.p) { ++*refptr; }

    Ptr& operator=(const Ptr&); // Реализован, как в разделе 14.2.
    ~Ptr(); // Реализован, как в разделе 14.2.
    operator bool() const { return p; }
    T& operator*() const; // Реализован, как в разделе 14.2.
    T* operator->() const; // Реализован, как в разделе 14.2.

private:
    T* p;
    size_t* refptr;
};
```

Новая функция-член `make_unique` выполняет как раз то, что нам нужно: если содержимое счетчика ссылок равно 1, она ничего не делает; в противном случае она использует функцию-член `clone` объекта, с которым связан этот дескриптор, чтобы сделать копию объекта, и устанавливает член `p`, чтобы он указывал на эту копию. Если содержимое счетчика ссылок не равно 1, должен существовать по крайней мере еще один `Ptr`-объект, который ссылается на исходный объект. Следовательно, мы декрементируем счетчик ссылок, связанный с исходным объектом (возможно, доведя его содержимое до 1, но не до 0). Затем создаем новый счетчик ссылок для нашего дескриптора, а также для других, которые могут быть созданы в будущем в качестве его копий. Поскольку пока существует только один `Ptr`-объект, связанный с копией, которую мы создали, инициализируем счетчик значением 1. Еще до вызова функции `clone` мы проверяем, связан ли с реальным объектом указатель на объект, с которого мы “лепили” копию. Если да, вызываем функцию `clone` для копирования этого объекта. По завершении описанных действий мы будем знать, что этот `Ptr`-объект — единственный объект, который связан с объектом, на который указывает указатель `p`. Этот объект является либо прежним объектом (если исходное содержимое счетчика ссылок было равно единице), либо его копией (если содержимое счетчика ссылок превышало единицу).

Мы можем использовать последнюю версию класса `Ptr` в реализации класса `Student_info` (см. раздел 14.1.2), ориентированного на дескриптор. В этом случае оказывается, что в изменении реализации класса `Student_info` вообще нет никакой необходимости, поскольку ни одна из наших операций не изменяет значения объекта

без его замены. Единственной операцией класса `Student_info`, изменяющей значение объекта, является функция `read`, но эта функция всегда присваивает абсолютно новое значение своему `Ptr`-члену. В результате оператор присваивания класса `Ptr` либо освободит старое значение, либо сохранит его, в зависимости от того, существуют ли другие объекты, которые ссылаются на старое значение. В любом случае объект, в который мы считываем данные, будет иметь новый `Ptr`-объект и, следовательно, будет единственным пользователем этого объекта. Рассмотрим следующий код.

```
Student_info s1;
read(cin, s1);           // объект s1 получает некоторое значение.
Student_info s2 = s1;    // копируем это значение в объект s2.
read(cin, s2);           // считываем данные в объект s2; при
                        // этом изменяется только объект s2,
                        // а не s1.
```

Здесь в результате вызова функции `read` устанавливается значение объекта `s2`, но значение объекта `s1` остается прежним.

Но если бы мы добавили в `Core`-иерархию виртуальную версию функции `regrade`, описанную в разделе 13.6.2, и усилили класс `Student_Info` соответствующей интерфейсной функцией, то эту функцию пришлось бы изменить, добавив в нее вызов функции `make_unique`.

```
void Student_info::regrade(double final, double thesis)
{
    // Получаем собственную копию до изменения объекта.
    cp.make_unique();

    if (cp)
        cp->regrade(final, thesis);
    else throw run_time_error(
        "пересчет оценки для неизвестного студента.");
}
```

14.4. Усовершенствование управляемых дескрипторов

Даже будучи таким гибким, созданный выше дескриптор все же удовлетворяет нас не полностью. Предположим, мы хотим использовать его для обновления реализации класса `Str` из главы 12. Как показано в разделе 12.3.4, мы неявно копируем множество символов, чтобы сформировать новые `Str`-объекты, которые образуются в результате конкатенации двух существующих `Str`-объектов. Используя подсчет ссылок для класса `Str`, мы могли бы предположить, что можем избежать создания по крайней мере некоторых таких копий.

```
// Работает ли эта версия?
class Str {
    friend std::istream& operator>>(std::istream&, Str&);
public:
    Str& operator+=(const Str& s) {
        data.make_unique();
        std::copy(s.data->begin(), s.data->end(),
            std::back_inserter(*data));
        return *this;
    }

    // интерфейс остался прежним.
    typedef vec<char>::size_type size_type;
```

```

// Новая реализация конструкторов для создания Ptr-объектов.
Str(): data(new Vec<char>) { }
Str(const char* cp): data(new Vec<char>) {
    std::copy(cp, cp + std::strlen(cp),
              std::back_inserter(*data));
}

Str(size_type n, char c): data(new Vec<char>(n, c)) { }
template<class In> Str(In i, In j): data(new Vec<char>) {
    std::copy(i, j, std::back_inserter(*data));
}

// Вызываем функцию make_unique при необходимости.
char& operator[](size_type i) {
    data.make_unique();
    return (*data)[i];
}
const char& operator[](size_type i) const {
    return (*data)[i];
}
size_type size() const { return data->size(); }
private:
    // Сохраняем Ptr-объект в векторе.
    Ptr< Vec<char> > data;
};
// Как реализовано в разделах 12.3.2 и 12.3.3.
std::ostream& operator<<(std::ostream&, const Str&);
Str operator+(const Str&, const Str&);

```

Мы сохранили интерфейсную часть класса Str, но фундаментально изменили его реализацию. Вместо хранения вектора непосредственно в каждом Str-объекте, мы храним ptr-объект, связанный с этим вектором. Такой подход позволяет нескольким Str-объектам совместно использовать одни и те же символьные данные. Конструкторы инициализируют этот Ptr-объект посредством размещения в памяти нового вектора, инициализированного соответствующими значениями. Код реализации операций, которые считывают данные, но не изменяют член data, не отличается от нашей предыдущей версии. Безусловно, эти операции сейчас производятся с использованием Ptr-объекта, поэтому имеет место не прямой доступ (через указатель, хранимый в Ptr-объекте) к опорным символам, которые составляют Str-объект. Особый интерес представляют операции, которые изменяют Str-объект, например оператор ввода данных, составной оператор конкатенации и не-const-версия оператора, возвращающего элемент контейнера по индексу.

Например, рассмотрим реализацию операторной функции Str::operator+=. Ее задача — присоединить данные к концу опорного вектора, чтобы обеспечить вызов функции data.make_unique(). В этом случае Str-объект будет иметь собственную копию опорных данных, которую он может свободно модифицировать.

14.4.1. Копирование типов, которыми мы не можем управлять

К сожалению, определение функции make_unique имеет серьезную проблему.

```

template<class T>
void Ptr<T>::make_unique()
{
    if (*refptr != 1) {
        --*refptr;
        refptr = new size_t(1);
        p = p? p->clone(): 0; // Здесь есть проблема.
    }
}

```

Рассмотрим вызов функции `p->clone`. Поскольку мы используем тип `Ptr<vector<char>>`, этот вызов может попытаться вызвать функцию `clone`, которая должна быть членом класса `vector<char>`. К сожалению, такой функции не существует!

Кроме того, функция `clone` должна быть членом класса, к которому мы присоединяем `Ptr`-объект, поскольку только таким способом эта функция может быть виртуальной. Другими словами, членство функции `clone` — чрезвычайно важное свойство, позволяющее ей беспрепятственно работать для всех членов иерархии наследования; но пока это невозможно, поскольку мы не можем изменить определение класса `vec`. Этот класс разработан с целью реализации сокращенной интерфейсной части (интерфейсного подмножества) стандартного класса `vector`. Если в определение класса `vec` добавить функцию-член `clone`, мы не будем иметь этого подмножества, поскольку класс `vector` не имеет члена, который будет у нас. Как же теперь быть?

В решении таких трудных проблем часто используется то, что программисты шутливо называют *основной теоремой разработки программного обеспечения*: все проблемы можно решить путем введения дополнительного уровня косвенности. Итак, наша проблема состоит в желании вызвать функцию-член, которой не существует, и у нас нет возможности “заставить” эту функцию-член существовать. Тогда решение заключается не в прямом вызове этой функции-члена, а в определении промежуточной глобальной функции, которую мы можем и вызвать, и создать. Мы по-прежнему будем называть эту функцию именем `clone`

```
template<class T> T* clone(const T* tp)
{
    return tp->clone();
}
```

и изменим нашу функцию-член `make_unique`, чтобы она вызывала именно новую версию функции `clone`.

```
template<class T>
void Ptr<T>::make_unique()
{
    if (*refptr != 1) {
        --*refptr;
        refptr = new size_t(1);
        p = p? clone(p): 0; // вызываем глобальную версию
                           // функции clone, а не функцию-член.
    }
}
```

Очевидно, что введение этой промежуточной функции не изменит поведения функции `make_unique`. Она по-прежнему вызывает функцию `clone`, которая вызывает функцию-член `clone` копируемого объекта. Однако теперь функция `make_unique` работает, преодолевая определенный уровень косвенности: она вызывает функцию-член `clone`, которая в свою очередь вызывает функцию-член `clone` объекта, на который указывает член `p`. Для классов (например, `Student_info`), которые определяют “свою” функцию `clone`, эта косвенность нам не мешает. Но для классов (например, `Str`), содержащих `Ptr`-указатели на типы, которые не предоставляют функцию `clone`, эта косвенность — как раз то, что нам нужно, чтобы весь механизм заработал. Для таких типов мы можем определить еще одну промежуточную функцию.

```
// “ключ зажигания”, который заставит работать
// тип Ptr<vec<char>>.
template<>
vec<char>* clone(const vec<char>* vp)
{
```

```

    return new Vec<char>(*vp);
}

```

Использование обозначения `template<>` в начале этой функции означает, что она (функция) представляет собой *шаблонную специализацию* (template specialization). Такие специализации определяют конкретную версию шаблонной функции для конкретного типа аргумента. Определяя эту специализацию, мы тем самым заявляем, что поведение функции `clone`, если передать ей указатель на объект типа `Vec<char>`, будет отличаться от ее поведения, если передать ей указатель любого другого типа. При передаче функции `clone` аргумента типа `Vec<char>*` компилятор будет использовать эту специализированную версию функции `clone`. При передаче же указателей других типов компилятор реализует обобщенную шаблонную форму функции `clone`, которая вызывает функцию-член `clone` для указателя, переданного в качестве аргумента. Наша специализированная версия функции использует конструктор копирования типа `Vec<char>`, чтобы построить новый объект типа `Vec<char>` из объекта, который был ей передан. Справедливости ради нужно отметить, что эта специализация функции `clone` не обеспечивает виртуального поведения, но в данном случае мы в таковом и не нуждаемся, поскольку не существует классов, выведенных из класса `Vec`.

Описанные выше действия были направлены на то, чтобы смягчить зависимость от функции-члена `clone`, если окажется, что такой член попросту не существует. В результате дополнительного уровня косвенности мы сделали возможным *так* определить шаблонную функцию `clone`, чтобы она выполняла действия, адекватные конкретной ситуации: или копировала объект заданного класса, используя соответствующую функцию-член `clone`, или вызывала конструктор копирования, или делала что-нибудь еще. При отсутствии описанной специализации класс `Ptr` использовал бы функцию `clone`, но это имело бы место только при условии существования вызова функции `make_unique`. Другими словами, возможны следующие варианты.

- Если вы используете класс `Ptr<T>`, но не используете функцию `Ptr<T>::make_unique`, то не имеет значения, определена ли функция `T::clone`.
- Если вы используете функцию `Ptr<T>::make_unique` и при этом определена функция `T::clone`, функция `make_unique` будет использовать функцию `T::clone`.
- Если вы применяете функцию `Ptr<T>::make_unique`, но не собираетесь использовать функцию `T::clone` (возможно, вследствие того, что она не существует), можете специализировать шаблон `clone<T>`, “заказав” нужное вам поведение.

Благодаря дополнительному уровню косвенности мы получили возможность более детально управлять поведением объектов класса `Ptr`. Теперь осталась одна мелочь — решить, что следует сделать в первую очередь.

14.4.2. Когда в копии есть необходимость

Последняя часть нашего примера стоит того, чтобы углубиться в детали. Рассмотрим снова определения двух версий функции `operator[]`. Одна из них вызывает функцию `data.make_unique`, а другая — нет. Почему в их поведении существует такое различие?

Это различие связано с использованием (и соответственно неиспользованием) модификатора `const`. Вторая версия операторной функции `operator[]` является `const`-функцией-членом, которая “обещает” не изменять содержимое объекта. Ей удастся сдержать свое “обещание”, возвращая автору ее вызова значение типа `const char&`. Следовательно, в разделении опорного `Vec<char>`-объекта с другими `Str`-объектами

никакого вреда не существует. Главное то, что пользователь не сможет использовать полученное значение, чтобы изменить значение `Str`-объекта.

И наоборот, первая версия функции `operator[]` возвращает значение типа `char&`, которое подразумевает, что пользователь мог бы использовать это возвращаемое значение для изменения содержимого `Str`-объекта. Если пользователь и в самом деле воспользуется такой возможностью, то здесь важно ограничиться изменением данного `Str`-объекта и не распространять это изменение на любые другие `Str`-объекты, которые, возможно, будут использовать тот же опорный `Vec`-объект. Наша цель — защититься от возможности изменения значений любых других `Str`-объектов посредством вызова функции `make_unique` для `Ptr`-объекта еще до возврата ссылки на какой бы то ни был символ `Vec`-объекта.

14.5. Резюме

Шаблонные специализации выглядят как определения шаблонов, но в них опущен один или несколько параметров-типов, вместо которых указываются конкретные типы. Обширное использование шаблонных специализаций выходит за рамки этой книги, но вы должны знать, что они существуют и используются для принятия решений в отношении типов во время компиляции.

Упражнения

- 14.0. Скомпилируйте, выполните и протестируйте программы, приведенные в этой главе.
- 14.1. Реализуйте операцию сравнения для объектов типа `Ptr<Core>`.
- 14.2. Реализуйте и протестируйте программу вычисления итоговых оценок студентов, используя `Ptr<Core>`-объекты.
- 14.3. Реализуйте класс `Student_info` с использованием последней версии класса `Ptr` и примените эту версию для реализации программы вычисления итоговых оценок из раздела 13.5.
- 14.4. Реализуйте класс `Str` с использованием последней версии класса `Ptr`.
- 14.5. Протестируйте новый вариант реализации класса `Str`, перекомпилировав и выполнив программы, которые используют класс `Str` (например, версию функции `split` и операции, которые используют класс `Vec<Str>`).
- 14.6. Класс `Ptr` в действительности решает две проблемы: управление подсчетом ссылок, а также размещение в памяти объектов с последующим ее освобождением. Определите класс, который выполняет подсчет ссылок и ничего больше, а затем используйте этот класс для нового варианта реализации класса `Ptr`.

ВОЗВРАЩАЯСЬ К СИМВОЛЬНЫМ ИЗОБРАЖЕНИЯМ

Наследование — наиболее полезное свойство классов в моделировании больших и сложных систем, которые вряд ли рассматриваются в книгах подобного уровня. Одной из причин обращения к примеру символьных изображений, который описан в разделе 5.8, является то, что создание таких картинок требует объектно-ориентированного решения, которое займет не более нескольких сотен строк кода. Мы использовали этот пример в течение многих лет, шлифуя без конца текст программы и упрощая изложение материала. Возвращаясь к прежнему примеру, мы могли бы теперь сократить почти половину представленного выше кода, используя стандартную библиотеку и наш обобщенный класс дескриптора из главы 14.

В разделе 5.8 мы написали несколько функций, которые представляли символьное изображение в виде вектора типа `vector<string>`, используя стратегию копирования символов при создании новой картинки. На копирование всех этих символов тратится время и память компьютера. Например, если бы мы собрались конкатенировать две копии изображения, то нам бы пришлось тогда хранить три копии каждого символа: один — для исходной картинки и еще по одному — для каждой части конкатенированной картинки.

Более важно то, что решение, представленное в разделе 5.8, вообще не учитывает структурную информацию о картинках. Мы не имеем понятия о том, *как* данное изображение было сформировано. Другими словами, мы ничего не знаем о его происхождении: то ли оно было создано в результате получения входных данных от пользователя, то ли в результате применения одной или нескольких операций к более простым изображениям. Некоторые потенциально полезные операции требуют сохранения структуры изображения. Например, если бы мы хотели иметь возможность в нашей картинке менять символы рамки, могли бы это сделать только в том случае, если бы знали, какие компоненты изображения заключены в рамку, а какие — нет. Мы не можем тривиальным поиском находить экземпляры символов рамки, поскольку эти символы могут случайно оказаться частью исходного изображения.

Как будет показано в этой главе, используя наследование и наш обобщенный класс дескриптора, мы сможем сохранить структурную информацию, присущую изображению, и при этом резко сократить затраты памяти на нашу систему.

15.1. Проект системы

Мы попытаемся здесь решить две отдельные задачи. Первая — задача проектирования, т.е. мы бы хотели сохранить структурную информацию о том, как изображение было соз-

дано. Вторая — задача реализации; мы бы хотели хранить как можно меньше копий одних и тех же данных. Обе задачи вытекают из нашего решения хранить изображение в виде вектора типа `vector<string>`, поэтому мы должны вспомнить это решение.

Задачу реализации мы можем решить, управляя нашими данными с помощью класса `Ptr`, который разработали в главе 14. Этот класс позволит нам хранить реальные символьные данные в одном объекте, чтобы несколько изображений могли использовать один и тот же объект. Например, если мы захотим заключить в рамку данное изображение, нам больше не придется копировать его символы. Вместо этого класс `Ptr` будет управлять счетчиком ссылок (связанным с этими данными), который покажет, сколько изображений используют эти данные.

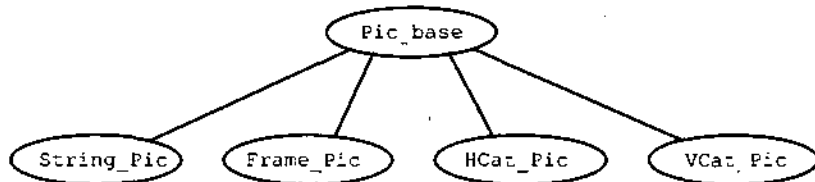
Задача проектирования несколько сложнее. Каждое создаваемое нами изображение имеет структуру, которую необходимо сохранить. Мы создаем изображение либо из некоторой начальной коллекции символов, либо посредством одной из трех операций: `frame` (для создания изображения, заключенного в рамочку), `hcat` или `vcat` (для создания изображений, конкатенированных горизонтально или вертикально соответственно).

Итак, у нас есть четыре вида изображений. Несмотря на их сходство, мы создаем их по-разному, а посему на этих различиях стоит остановиться.

15.1.1. Использование наследования при моделировании структуры

Наша задача прекрасно подходит для наследования классов: у нас есть несколько видов похожих структур данных, которые тем не менее различаются некоторыми важными особенностями. Каждая из наших структур данных — это вид изображения, создаваемый в предположении, что наследование — это удобный способ представления подобных структур данных. Мы можем определить общий базовый класс, который моделирует общие свойства каждого вида изображения, а затем вывести из этого базового класса отдельный класс для каждого конкретного вида изображения, который мы хотим поддерживать.

Назовем производные классы следующим образом: `String_Pic` (для изображений, созданных из строк, заданных нашими пользователями); `Frame_Pic` (для изображений, созданных посредством заключения в рамочку другого изображения), а также `hCat_Pic` и `VCat_Pic` (для изображений, которые являются результатом горизонтальной или вертикальной конкатенации двух других изображений соответственно). Связав эти классы наследованием, мы можем использовать виртуальные функции для написания кода, которому даже не требуется точная информация о виде обрабатываемого им изображения. Таким образом, пользователи могут применить любую из наших операций, не зная, к какому виду изображения она применяется. Каждый из этих классов мы выведем из общего базового класса, который назовем `Pic_base`, в результате чего получим следующую иерархию наследования.



Следующий вопрос — стоит ли делать иерархию наследования видимой для наших пользователей. Казалось бы, для этого нет причины. Ни одна из наших операций не

связана с конкретными видами изображений: все они имеют дело с абстрактным “понятием” изображения. Поэтому в раскрытии иерархии и в самом деле нет никакой необходимости. Более того, поскольку мы собираемся использовать стратегию подсчета количества ссылок, наши пользователи сочтут более удобным для себя, если мы скроем это наследование и связанный с ним подсчет ссылок, тем самым избавив их от таких утомительных подробностей.

Вместо того чтобы нагружать пользователей проблемами прямого общения с классом `Pic_base` и соответствующими производными классами, мы определим специальный интерфейсный класс. Получив доступ к этому классу, наши пользователи будут освобождены от необходимости вникать в какие бы то ни было детали реализации. В частности, использование интерфейсного класса скроет иерархию наследования, а вместе с ней и тот факт, что наш класс зависит от класса `Ptr`. Очевидно, в этом случае нам придется определить шесть классов: класс интерфейса, базовый класс для нашей иерархии наследования и четыре производных класса. Назовем интерфейсный класс `Picture`. Класс `Picture` для управления своими данными будет использовать класс `Ptr`.

Так объектами какого же типа будет управлять класс `Ptr`? Он будет управлять нашим классом реализации, `Pic_base`. Следовательно, классу `Picture` достаточно иметь только один член данных типа `Ptr<Pic_base>`. Но объекты, связанные с классом `Ptr`, будут всегда объектами одного из упомянутых производных классов. Таким образом, создавая (или разрушая) `Pic_base`-объекты, мы будем делать это посредством `Pic_base`-указателя, но эти объекты будут иметь тип одного из производных классов. Как мы видели в разделе 13.3.2, такой проект означает, что нам понадобится определить в классе `Pic_base` виртуальный деструктор. А чтобы класс `Ptr` мог использовать этот деструктор, нужно сделать его открытым (`public`).

Мы сказали, что хотим скрыть использование класса `Pic_base` и связанной с ним иерархии, чтобы пользователи могли манипулировать этими объектами только косвенно, посредством класса `Picture`, и не получали прямой доступ ни к одному из этих классов. Оказывается, что самый прямой способ скрыть эти классы состоит в использовании обычных механизмов защиты. Предоставив этим классам пустой `public`-интерфейс, мы тем самым великодушно позволяем компилятору осуществить наше решение, заключающееся в том, что все контакты с нашими изображениями должны происходить только через интерфейсный класс `Picture`.

Чтобы придать нашему решению конкретность, напомним код, в котором попытаемся выразить все, что нам известно на данный момент.

```
// private-классы для использования только в разделе реализации.
class Pic_base { };

class String_Pic: public Pic_base { };
class Frame_Pic: public Pic_base { };
class VCat_Pic: public Pic_base { };
class HCat_Pic: public Pic_base { };

// интерфейсный public-класс и операции.
class Picture {
public:
    Picture(const std::vector<std::string>& =
            std::vector<std::string>());
private:
    Ptr<Pic_base> p;
};
```


Каждый объект класса `Picture` будет содержать `private`-объект типа `Ptr<Pic_base>`. Класс `Pic_base` — это общий базовый класс для четырех классов, представляющих наши четыре вида изображений. Класс `Ptr` будет управлять счетчиками ссылок, чтобы позволить нам совместно использовать опорные `Pic_base`-объекты. Каждую операцию, выполняемую с `Picture`-объектом, мы реализуем посредством ее переадресации через `Ptr`-объект к опорному объекту производного класса. Мы еще не определили точно, какими будут эти операции, поэтому пока мы оставили пустыми тела класса `Pic_base` и классов, выведенных из него.

Пока мы видим, что класс `Picture` довольно прост: в нем определена единственная операция создания `Picture`-объекта из вектора `string`-объектов. Чтобы сделать этот вектор оптимальным, мы используем аргумент по умолчанию (см. раздел 7.3). Если пользователь создает `Picture`-объект без аргументов, компилятор автоматически предоставит в качестве аргумента объект типа `vector<string>()`, который генерирует вектор типа `vector<string>` без элементов. Следовательно, задавая аргумент по умолчанию для создания `Picture`-объекта без строк, мы можем использовать определение следующего вида.

```
Picture p; // Пустой Picture-объект.
```

Теперь можно подумать о том, как представить другие операции над `Picture`-объектами. Мы знаем, что хотим реализовать операции `frame`, `hcat` и `vcat`. Нам нужно решить, как это сделать и должны ли быть эти операции членами класса `Picture`. Эти операции не меняют состояние `Picture`-объекта, к которому они применяются, поэтому не существует веской причины делать их членами. Наоборот, существует веская причина этого не делать: как было показано в разделе 12.3.5, сделав операции не членами, мы тем самым позволяем выполнение преобразований.

Например, поскольку конструктор класса `Picture` (который мы уже написали) не является `explicit`-конструктором, пользователи смогут написать следующее.

```
vector<string> vs;  
Picture p = vs;
```

При этом C++-среда преобразует объект `vs` в `Picture`-объект за нас. Если мы хотим легализовать такое поведение (а мы этого хотим), то необходимо также позволить пользователям писать такие выражения, как `frame(vs)`. Если функция `frame` будет членом класса, то пользователи не смогут писать такое кажущееся эквивалентным выражение, как `vs.frame()`. Вспомните, что преобразования не применяются к левому операнду оператора “точка” (`.`), поэтому этот вызов будет интерпретирован как вызов (несуществующей) функции-члена `frame` объекта `vs`.

Более того, мы надеемся, что наши пользователи сочтут более удобным использовать для построения более сложных изображений синтаксис, характерный для выражений. По нашему мнению, запись

```
hcat(frame(p), p)
```

выглядит яснее, чем запись

```
p.frame().hcat(p),
```

поскольку первый пример отражает симметрию аргументов функции `hcat`, а второй скрывает ее.

Помимо функций, позволяющих строить `Picture`-объекты, мы считаем нужным определить оператор вывода, который способен вывести содержимое `Picture`-объекта. Итак, теперь мы можем конкретизировать остальную часть проекта интерфейса.

```
Picture frame(const Picture&);
Picture hcat(const Picture&, const Picture&);
Picture vcat(const Picture&, const Picture&);
std::ostream& operator<<(std::ostream&, const Picture&);
```

15.1.2. Класс *Pic_base*

Следующий этап в нашем проекте — уточнение деталей иерархии класса *Pic_base*. Если оглянуться на нашу исходную реализацию, то нетрудно заметить, что для определения количества *string*-объектов в данном изображении мы использовали функцию `vector<string>::size`. При этом вспомните, что для форматирования выходных данных мы написали отдельную функцию `width` (см. раздел 5.8). Обдумывая, как отобразить символьную картинку, мы поняли, что нам, вероятно, придется выполнять те же операции и над объектами наших классов, которые выведены из класса *Pic_base*. Эти операции должны быть виртуальными, чтобы мы могли узнать о любом *Pic_base*-объекте, сколько у него строк и какова ширина самой широкой строки. Более того, поскольку наши пользователи будут использовать оператор вывода для отображения содержимого конкретного *Pic_base*-объекта, мы можем сделать вывод о том, что нам понадобится еще одна виртуальная функция для вывода заданного *Pic_base*-объекта в заданный *ostream*-объект.

Особого внимания заслуживает функция `display`. Нетрудно догадаться, что одним из параметров функции `display` должен быть поток, в который выводятся данные, а относительно других параметров еще нужно хорошо подумать, чтобы правильно определить поведение этой функции. Проектируя класс *Picture*, мы предполагали, что он будет включать один или несколько компонентов, каждый из которых представляет собой объект класса, выведенного из класса *Pic_base*. Если подумать о создании горизонтально конкатенированного изображения, то, по-видимому, вывод каждой итоговой строки должен включать в этом случае вывод соответствующей строки от каждого из составляющих изображений. В частности, мы не можем вывести все содержимое одного составляющего изображения, а затем — содержимое другого составляющего изображения. Вместо этого мы должны поочередно выводить по одной строке от содержимого каждого составляющего изображения. Таким образом, функция `display` должна иметь параметр, который указывает, какую строку изображения следует выводить в данный момент.

При отображении с помощью функции `display` левой части горизонтально конкатенированного изображения нам нужно сообщить о том, нужно ли дополнять (с помощью функции `width()`) каждую строку соответствующего изображения до полной ширины. Но если мы выводим *Picture*-объект, который содержит только объект класса *String_pic*, или вертикально конкатенированный *Picture*-объект, составленный только из *String_pic*-объектов, то дополнение выводимых строк заключается лишь в выводе ненужных хвостовых пробелов. Поэтому, в качестве средства оптимизации, мы передадим функции `display` третий аргумент, который означает, нужно ли вообще дополнять пробелами выводимые строки.

Эти размышления приводят нас к решению передать функции `display` три аргумента: поток, в котором будут сгенерированы выходные данные, номер выводимой строки и *bool*-значение, которое позволит понять, нужно ли дополнять изображение до полной ширины. Приняв такое решение, можно конкретизировать поведение *Pic_base*-семейства классов.

```
class Pic_base {
    // public-интерфейс отсутствует (за исключением
```

```
// виртуального деструктора).
typedef std::vector<std::string>::size_type ht_sz;
typedef std::string::size_type wd_sz;

virtual wd_sz width() const = 0;
virtual ht_sz height() const = 0;
virtual void display(std::ostream&, ht_sz, bool) const = 0;
```

```
public:
    virtual ~Pic_base() ( )
};
```

Определение класса `Pic_base` начинается с присваивания сокращенных имен для `size_type`-типов, которые понадобятся нам в реализации. Поскольку мы знаем, что в качестве опорных данных по-прежнему используется объект типа `vector<string>`, член `size_type` вектора типа `vector<string>` будет подходящим типом для представления высоты и ширины изображения (роль ширины будет “играть” самый широкий `string`-объект вектора). Обозначим эти типы как `ht_sz` и `wd_sz` соответственно.

Еще одна задача — определение виртуальных функций для базового класса, которые, как вы заметили, имеют новую форму: в каждом случае вместо ожидаемого тела функции стоит `= 0`. Этот синтаксис означает, что никаких других определений этой виртуальной функции не предусматривается. Зачем же определять эти функции подобным образом?

Чтобы ответить на этот вопрос, поразмышляем о том, как бы выглядели эти определения, если бы мы попытались их написать. В нашем проекте класс `Pic_base` нужен только как общий базовый класс для последующего построения конкретных классов изображений. Мы будем создавать объекты этих конкретных типов как результат выполнения одной из `Picture`-операций или в ответ на создание пользователем `Picture`-объекта из объекта типа `vector<string>`. Ни одна из этих операций прямо не создает `Pic_base`-объекты и не манипулирует ими. А коль `Pic_base`-объекты никогда не заявляют о своем существовании, то что тогда означает высота (`height`) или ширина (`width`) такого объекта (в сравнении с высотой и шириной объекта типа, выведенного из класса `Pic_base`)? Эти операции нужны только для производных классов, в которых всегда будет иметь место конкретное изображение. Для самого же класса `Pic_base` понятие высоты или ширины не имеет смысла.

Вместо того чтобы принуждать нас к “изобретению” бессмысленных определений этих операций, язык C++ позволяет нам сообщить, что для данной виртуальной функции определение попросту не предусмотрено. В качестве побочного эффекта отказа в реализации `virtual`-функции мы также обещаем, что объекты соответствующего типа никогда не будут существовать. Это заявление отнюдь не мешает существованию объектов производных типов (выведенных из данного), но речь ни в коем случае не идет об объектах именно этого типа.

Итак, всего лишь с помощью двух символов (`= 0`) мы можем сообщить о своем намерении не реализовывать виртуальную функцию, что мы и сделали для таких функций, как `height`, `width` и `display`. Таким способом создается “чистая” виртуальная функция (pure virtual function), т.е. функция с “чистым спецификатором” (`= 0`), или функция без реализации, а ее реализации (возможно, различные) могут содержаться в производных классах. Определяя некоторый класс хотя бы с одной “чистой” виртуальной функцией, мы также неявно заявляем о том, что никогда не будем создавать объекты этого класса. Такие классы называются *абстрактными базовыми классами* (abstract base classes), поскольку существуют только с целью фиксации абстрактного интерфейса для некоторой иерархии наследования. Такие классы являются чисто аб-

страктными: существование их объектов невозможно. После того как мы “наградили” некоторый класс чистыми *virtual*-функциями, компилятор не позволит нам создать ни один объект этого абстрактного класса.

15.1.3. Производные классы

Как виртуальность сама по себе, так и “чистая” виртуальность функции “передаются по наследству”. Если производный класс определяет *все* унаследованные чистые виртуальные функции, он становится конкретным классом, что позволяет нам создавать объекты этого класса. Но если производный класс не определит *хотя бы одну* чистую виртуальную функцию, которую он унаследовал, значит, он унаследовал абстрактную “природу” своего базового класса. В этом случае производный класс сам является абстрактным классом, и мы не сможем создавать объекты этого производного класса. Поскольку в каждом из наших производных классов мы стремимся смоделировать конкретный класс, *все* эти производные классы должны переопределять *все* *virtual*-функции.

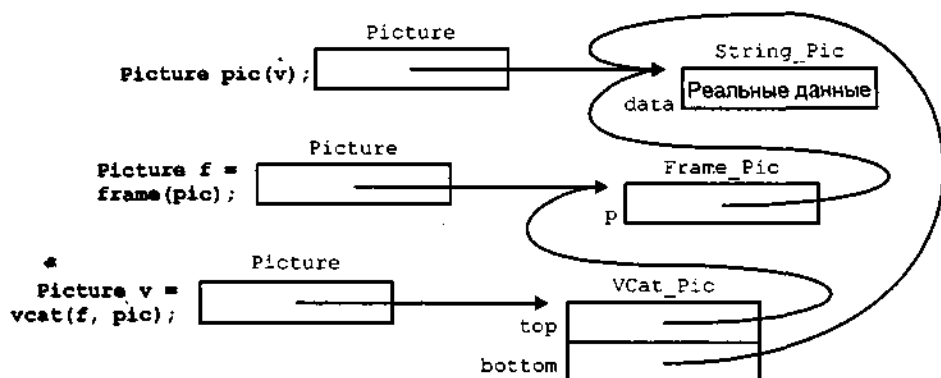
Теперь необходимо подумать о том, какие данные будет содержать каждый из наших производных классов и как мы будем создавать объекты каждого типа. Цель проектирования этих классов — смоделировать способ формирования изображения. О том, как объект изображения был создан, можно будет судить по его типу: *String_Pic*-объект создается из символьных данных, предоставленных пользователем; *Frame_Pic*-объект — это результат выполнения функции *frame* применительно к некоторому другому *Picture*-объекту и т.д. Помимо информации о том, как объект был создан, мы также должны сохранить объект (объекты), из которого он был создан. Для *String_Pic*-объекта нам придется запомнить символы, предоставленные пользователем, и это можно сделать в векторе типа *vector<string>*. Создавая *Frame_Pic*-объект посредством заключения в рамочку другого *Picture*-объекта, нам также придется сохранить исходный *Picture*-объект. Аналогично мы создаем *nCat_Pic*- и *vCat_Pic*-объекты, объединяя два других *Picture*-объекта. Эти классы должны сохранять *Picture*-объекты, используемые в создании нового (объединенного) объекта.

Прежде чем остановиться на проекте, который должен сохранять *Picture*-объекты в классах, выведенных из класса *Pic_base*, нам следует более серьезно подумать о последствиях функционирования этого проекта. Класс *Picture* — интерфейсный класс, предназначенный для использования нашими конечными пользователями. А раз так, то он представляет собой интерфейс с нашей предметной областью, а не реализацией. Другими словами, он не имеет операций *height*, *width* или *display*. Если подумать о том, как эти функции могут быть реализованы, то станет ясно, что нам понадобится доступ к соответствующим операциям над *Picture*-объектом (объектами), сохраняемым в каждом из производных типов. Например, чтобы вычислить высоту *vCat_Pic*-объекта, нам придется сложить высоты двух *Picture*-объектов, из которых он был сформирован. Аналогично мы получим ширину (*width*), найдя максимальное из *width*-значений двух составляющих *Picture*-объектов.

Из-за необходимости сохранять *Picture*-объект в каждом из производных классов мы должны наделить класс *Picture* функциями, которые дублируют операции, определенные в классе *Pic_base*. Такой подход практически зачеркнет наше начальное намерение сделать класс *Picture* “ответственным” за интерфейс, а не за реализацию. Мы можем оставить в силе наше намерение, осмыслив следующее: то, что нам нужно иметь в производных классах, является объектом не интерфейса, а реализации. Осмысление этого факта означает, что вместо сохранения *Picture*-объекта мы должны

сохранять объект класса `Ptr<Pic_base>`. Этот подход сохранит четкое разделение между интерфейсом и реализацией, при этом останется в силе наше намерение подсчитывать ссылки на объекты реализации во избежание ненужного дублирования данных.

Несмотря на ясность нашего проекта, некоторая степень косвенности в нем все же присутствует, поэтому следующий рисунок должен помочь понять смысл происходящего.



Здесь мы предполагаем, что генерируется три `Picture`-объекта. Первый `Picture`-объект представляет `String_Pic`-объект, который содержит данные, полученные от пользователя. Второй представляет `Frame_Pic`-объект, построенный с помощью вызова функции `frame` для первого `Picture`-объекта. Наконец, мы создаем `Picture`-объект, который представляет результат вызова функции `vcat` для двух предыдущих `Picture`-объектов. Каждый `Picture`-объект имеет один член данных типа `Ptr<Pic_base>`, который указывает на объект соответствующего типа, выведенного из класса `pic_base`. Каждый такой объект, в свою очередь, содержит либо вектор, в котором хранится копия данных, полученных от пользователя, либо один или два `Ptr`-члена, которые указывают на `pic_base`-объекты, используемые для создания `Picture`-объекта. На этом рисунке не показаны счетчики ссылок, связанные с `Ptr`-объектами, поскольку мы предполагаем, что класс `Ptr` исправно делает свою работу и можно не вникать в ее детали.

От проекта, представленного в главе 5, этот проект отличается тем, что только `String_Pic`-объект содержит символы. Другие объекты содержат один или два `Ptr`-члена. Следовательно, при создании объекта `f` или `v` мы не будем копировать ни одного символа. Просто класс `Ptr` обеспечит еще одну ссылку на `Ptr`-члены (в `Picture`-объектах), которые используются при создании нового `Picture`-объекта, и тот же класс `Ptr` позаботится о подсчете ссылок за нас. Итак, результатом вызова функции `frame(pic)` является создание нового `Frame_Pic`-объекта, причем его `Ptr`-член будет указывать на тот же `String_Pic`-объект, который сохранен в объекте `pic`. Аналогично `VCat_Pic`-объект будет содержать два `Ptr`-члена, которые указывают на `Frame_Pic`- и `String_Pic`-объекты соответственно. Мы не будем разрушать ни один из этих `pic_base`-объектов, поскольку это — обязанность класса `Ptr`. Он реализует разрушение каждого `pic_base`-объекта при удалении последнего `Ptr`-объекта, который указывает на этот `pic_base`-объект.

На данном этапе мы должны воплотить эти проектные решения в коде. Мы знаем, какие данные будет содержать каждый объект и какими должны быть наши операции.

```
class Pic_base {
    // public-интерфейс отсутствует (за исключением
    // виртуального деструктора).
    typedef std::vector<std::string>::size_type ht_sz;
```

```

typedef std::string::size_type wd_sz;

// Этот класс - абстрактный базовый класс.
virtual wd_sz width() const = 0;
virtual ht_sz height() const = 0;
virtual void display(std::ostream&, ht_sz, bool) const = 0;

public:
    virtual ~Pic_base() { }
};

class Frame_Pic: public Pic_base {
    // public-интерфейс отсутствует.
    Ptr<Pic_base> p;
    Frame_Pic(const Ptr<Pic_base>& pic): p(pic) { }

    wd_sz width() const;
    ht_sz height() const;
    void display(std::ostream&, ht_sz, bool) const;
};

```

Из этого кода видно, что класс `Frame_Pic` выведен из класса `Pic_base`. Кроме того, мы заявляем о своем намерении определить конкретные версии каждой из трех `virtual`-функций, унаследованных от базового класса. Следовательно, класс `Frame_Pic` — не абстрактный класс, и мы сможем создавать `Frame_Pic`-объекты.

Обратите внимание на то, что мы объявили эти `virtual`-функции в `private`-разделе класса. Это позволяет компилятору реализовать наше проектное решение о том, что только класс `Picture` и операции над `Picture`-объектами могут получать доступ к `Pic_base`-иерархии. Конечно, поскольку эти `virtual`-функции закрыты (`private`), нам, возможно, придется возвратиться к определению этого класса, чтобы включить при необходимости `friend`-объявления для класса `Picture` или соответствующие операции.

Конструктор класса `Frame_Pic` необходим только для копирования `Ptr`-члена из объекта, который заключается в рамочку, причем это копирование выполняется в инициализаторе конструктора. Тело конструктора пустое, поскольку ему больше нечего делать.

Теперь перейдем к другим производным классам. Классы конкатенации будут действовать подобно классу `Frame_Pic`: каждому из них придется запоминать два составляющих изображения. Информация о способе конкатенации (вертикальном или горизонтальном) будет неявным образом содержаться в самом типе класса.

```

class VCat_Pic: public Pic_base {
    Ptr<Pic_base> top, bottom;
    VCat_Pic(const Ptr<Pic_base>& t, const Ptr<Pic_base>& b):
        top(t), bottom(b) { }

    wd_sz width() const;
    ht_sz height() const;
    void display(std::ostream&, ht_sz, bool) const;
};

class HCat_Pic: public Pic_base {
    Ptr<Pic_base> left, right;
    HCat_Pic(const Ptr<Pic_base>& l, const Ptr<Pic_base>& r):
        left(l), right(r) { }

    wd_sz width() const;
    ht_sz height() const;
    void display(std::ostream&, ht_sz, bool) const;
};

```

Класс `String_Pic` несколько отличается от других тем, что он сохраняет копию вектора типа `vector<string>`, который содержит данные, относящиеся к изображению.

```
class String_Pic: public Pic_base {
    std::vector<std::string> data;
    String_Pic(const std::vector<std::string>& v): data(v) { }

    wd_sz width() const;
    ht_sz height() const;
    void display(std::ostream&, ht_sz, bool) const;
};
```

Мы по-прежнему копируем символы из параметра `v` векторного типа, заданного пользователем, в собственный член класса `data`. И это единственное место во всей программе, где происходит копирование символов. Во всех остальных случаях копируются только объекты типа `Ptr<Pic_base>`, т.е. указатели, и выполняется обработка содержимого счетчиков ссылок.

15.1.4. Управление копированием

Возможно, самым интересным аспектом нашего проекта является *то*, чего в нем нет. В нем нет конструкторов копирования, операторов присваивания и деструкторов. Почему?

Дело в том, что здесь работают функции, синтезированные по умолчанию. Класс `vector` заботится о выделении памяти для начальной копии символов, предоставляемых пользователем при создании нового `Picture`-объекта. Если мы копируем (или присваиваем) два `Picture`-объекта, которые ссылаются на `String_Pic`-объекты, либо разрушаем `Picture`-объект, то `Ptr`-операции выполняют корректные действия по управлению `Picture`-объектами и вовремя удалят опорный `String_Pic`-объект. В более общем случае класс `Ptr` заботится о копировании, присваивании и разрушении `Ptr`-членов в других `Pic_base`-классах (и в самом классе `Picture`).

15.2. Реализация

На данном этапе у нас готов неплохой проект как интерфейса, так и реализации. Класс `Picture` и соответствующие операции над `Picture`-объектами будут управлять пользовательским интерфейсом. Посредством конструктора класса `Picture` и операций будут создаваться объекты одного из типов, выведенных из класса `Pic_base`. Для управления памятью, занимаемой опорными объектами, мы будем использовать класс `Ptr<Pic_base>`, тем самым избегая создания излишних копий данных. А теперь пора приступить к реализации операций интерфейса и каждого из производных классов.

15.2.1. Реализация пользовательского интерфейса

Начнем, пожалуй, с реализации интерфейсного класса и операций. Итак, все, что нам известно на данный момент, можно выразить следующим кодом.

```
class Picture {
public:
    Picture(const std::vector<std::string>& =
            std::vector<std::string>());
private:
    Ptr<Pic_base> p;
};

Picture frame(const Picture&);
```

```
Picture hcat(const Picture&, const Picture&);
Picture vcat(const Picture&, const Picture&);
std::ostream& operator<<(std::ostream&, const Picture&);
```

Рассмотрим сначала операции, которые создают новые Picture-объекты. Каждая из этих операций создает объект соответствующего класса, выведенного из класса Pic_base. Этот объект скопирует Ptr-член из Picture-объекта (объектов), для которого выполнялась данная операция. Мы свяжем Picture-объект с заново созданным объектом производного от Pic_base класса и возвратим этот Picture-объект. Например, если pic — Picture-объект, то в результате операции frame(pic) должен быть создан новый Frame_Pic-объект, который будет связан с Pic_base-частью объекта pic. Затем должен быть сгенерирован новый Picture-объект, который будет связан с новым Frame_Pic-объектом. Вот как такое начало выглядит в коде.

```
Picture frame(const Picture& pic)
{
    Pic_base* ret = new Frame_Pic(pic.p);
    // что мы должны вернуть?
}
```

Работа функции frame начинается с определения локального указателя на класс Pic_base, инициализируемого созданием нового Frame_Pic-объекта, который копирует опорный Ptr-член внутри объекта pic. Теперь перед нами четко обозначились две проблемы. Первая (более простая) состоит в том, что конструктор класса Frame_Pic закрыт (т.е. является private-конструктором). Как было показано в разделе 15.1.3, каждый из классов Pic_base-иерархии — скрытый класс. Мы не хотим, чтобы пользователи знали об этих классах, и поэтому определили только private-операции, благодаря чему компилятор сам реализует это проектное решение. Мы же можем решить эту проблему, сделав операцию frame другом (friend) класса Frame_Pic.

Вторая проблема посложнее: мы создали новый объект типа Frame_Pic, но нам все же нужно получить объект типа Picture. В более общем случае мы можем представить себе, что hcat, vcat и другие функции, которые мы могли бы написать впоследствии, будут генерировать объекты других типов, выведенных из класса Pic_base, и что они будут действовать так в контекстах, в которых на самом деле мы хотели бы получить объекты типа Picture. Дело в том, что функция frame и другие операции возвращают Picture-объекты, а не объекты Pic_base-иерархии. К счастью, из раздела 12.2 мы знаем, что объект одного типа можно преобразовать в объект другого типа, если обеспечить соответствующий конструктор. В этом случае *соответствующим* конструктором является тот, который создает Picture-объект из объекта типа Pic_base*.

```
class Picture {
    Ptr<Pic_base> p;
    Picture(Pic_base* ptr): p(ptr) { }
    // Остальное как прежде.
};
```

Наш конструктор инициализирует член p заданным указателем на класс Pic_base. Вспомните, что класс Ptr имеет конструктор, принимающий аргумент типа T*, которым в данном случае является объект типа Pic_base*. Инициализатор p(ptr) вызывает упомянутый конструктор Ptr::Ptr(T*), передавая ему аргумент ptr. Определив этот конструктор класса Picture, мы можем спокойно завершить определение операции frame.

```
Picture frame(const Picture& pic)
{
    return new Frame_Pic(pic.p);
}
```


Как видите, мы избавились от локального объекта (указателя на класс `Pic_base`), поскольку он оказался ненужным. Вместо него мы создаем новый `Frame_Pic`-объект, адрес которого автоматически преобразуется в `Picture`-объект, который мы возвращаем из этой функции. Чтобы ясно представить себе, что делает эта маленькая функция, необходимо понимать тонкости использования автоматических преобразований и конструкторов копирования. Единственная инструкция в этой функции имеет результат, эквивалентный следующему коду.

```
// Создаем новый объект типа Frame_Pic.  
Pic_base* temp1 = new Frame_Pic(pic.p);  
  
// Создаем Picture-объект из объекта типа Pic_base*.  
Picture temp2(temp1);  
  
// Возвращаем этот Picture-объект посредством вызова  
// конструктора копирования класса Picture.  
return temp2;
```

Подобно функции `frame`, функции конкатенации также полагаются на наш новый конструктор класса `Picture`.

```
Picture hcat(const Picture& l, const Picture& r)  
{  
    return new HCat_Pic(l.p, r.p);  
}  
  
Picture vcat(const Picture& t, const Picture& b)  
{  
    return new VCat_Pic(t.p, b.p);  
}
```

В каждом случае мы создаем объект соответствующего типа, связываем с ним указатель типа `Ptr<Pic_base>`, затем на основе `Ptr<Pic_base>`-указателя создаем `Picture`-объект и возвращаем копию этого `Picture`-объекта. Конечно, чтобы эти функции были работоспособны, нам необходимо добавить в классы `HCat_Pic` и `VCat_Pic` соответствующие `friend`-объявления.

Чтобы построить `Picture`-объект из вектора типа `vector<string>`, мы используем ту же стратегию, которую применяли для других видов изображений.

```
Picture::Picture(const vector<string>& v):  
    p(new String_Pic(v)) { }
```

И снова-таки, мы создаем новый `String_Pic`-объект, но на сей раз, вместо возврата этого объекта, используем его напрямую для инициализации указателя `p`. Конечно, мы не должны забыть о том, чтобы сделать класс `Picture` другом класса `String_Pic`, и тогда он сможет получить доступ к конструктору класса `String_Pic`.

Важно понимать, чем этот конструктор отличается от функций `frame`, `hcat` и `vcat`. Каждая из этих функций определена для возврата `Picture`-объекта, и в каждой из них (в инструкции `return`) мы используем указатель на класс, выведенный из класса `Pic_base`. Следовательно, для создания `Picture`-объекта, подлежащего возврату из функции (`frame`, `hcat` или `vcat`), мы неявно использовали конструктор `Picture(Pic_base*)`. А в только что написанном конструкторе класса `Picture` мы по-прежнему создаем указатель на класс, выведенный из класса `Pic_base` (в данном случае это класс `String_Pic`), но теперь мы используем этот указатель для инициализации члена `p`, который имеет тип `Ptr<Pic_base>`. Тем самым мы используем конструктор `Ptr(T*)` в классе `Pic_base`, а не конструктор `Picture(Pic_base*)`, поскольку создаем объект типа `Ptr<Pic_base>`, а не объект типа `Picture`.

Чтобы завершить реализацию наших интерфейсных функций, мы должны определить оператор вывода. Эта операция довольно проста: нам нужно “пройти” по всему опорному объекту `Pic_base`-иерархии и вызвать функцию `display` для вывода каждой строки выходных данных.

```
ostream& operator<<(ostream& os, const Picture& picture)
{
    const Pic_base::ht_sz ht = picture.p->height();
    for (Pic_base::ht_sz i = 0; i != ht; ++i) {
        picture.p->display(os, i, false);
        os << endl;
    }
    return os;
};
```

Переменную `ht` мы инициализируем вызовом `virtual`-функции `height` для опорного объекта `Pic_base`-иерархии, чтобы не вычислять высоту изображения на каждой итерации цикла. Вспомните, что член `p` — это на самом деле объект типа `Ptr<Pic_base>` и что класс `Ptr` имеет перегруженный оператор “`->`” для реализации ссылок посредством `Ptr`-объекта как ссылки посредством указателя, содержащегося в этом `Ptr`-объекте. Мы выполняем `ht` итераций, проходя по опорному объекту `Pic_base`-иерархии, и на каждой итерации вызываем `virtual`-функцию `display` для вывода текущей строки. Напомним, что ее третий аргумент (`false`) означает, что функция `display` не должна дополнять выводимые данные до полной ширины изображения. Если нам понадобится дополнить данные при выводе внутреннего `Picture`-объекта, внутренние функции `display` позаботятся об этом. На данном этапе мы пока не можем сказать, нужно ли обеспечивать дополнение до полной ширины изображения. По окончании вывода каждой строки, задаваемой вторым аргументом `i`, мы выводим манипулятор `endl`, а по завершении всего цикла возвращаем потоковый объект `os` (первый аргумент функции `display`).

Как и в случае других уже реализованных нами операций, мы должны добавить `friend`-объявление в класс `Pic_base`, чтобы позволить операторной функции `operator<<` получить доступ к его членам `display` и `height`.

15.2.2. Класс *String_Pic*

Завершив определение интерфейсного класса и операций, мы можем переключить наше внимание на производные классы. Начнем с класса `String_Pic`.

```
class String_Pic: public Pic_base {
    friend class Picture;
    std::vector<std::string> data;
    String_Pic(const std::vector<std::string>& v): data(v) { }

    ht_sz height() const { return data.size(); }
    wd_sz width() const;
    void display(std::ostream&, ht_sz, bool) const;
};
```

Мы реализовали функцию `height`, но уже не так, как в определении класса `String_Pic`, представленном в разделе 15.1.3. Функция `height` тривиальна: она переадресует запрос значения высоты члену `size` векторного объекта `data`.

Чтобы определить ширину изображения типа `String_Pic`, функция `width` должна просмотреть каждый элемент вектора `data` и узнать длину самого длинного из них.

```
Pic_base::wd_sz String_Pic::width() const
{
```

```

    Pic_base::wd_sz n = 0;
    for (Pic_base::ht_sz i = 0; i != data.size(); ++i)
        n = max(n, data[i].size());
    return n;
}

```

За исключением имен типов, эта функция выглядит, как исходная функция `width` из раздела 5.8. Поскольку `String_Pic`-объект содержит вектор типа `vector<string>`, такому сходству удивляться не стоит.

Функция `display` несколько сложнее. Она должна, обходя опорный вектор, выводить `string`-объекты, соответствующие заданному номеру строки.

А как же насчет дополнения до полной ширины изображения? Обратите внимание на то, что эта функция может быть вызвана непосредственно из оператора вывода, как в случае, если бы выводимый `Picture`-объект указывал на `String_Pic`-объект, или она может быть вызвана косвенно, как часть вывода большего `Picture`-объекта, в котором этот `String_Pic`-объект является лишь его частью. В последнем случае функции `display` передается запрос дополнить каждую выводимую строку пробелами, чтобы сделать их одинаково широкими. Количество дополняющих пробелов для каждой строки разное и составляет число символов, недостающее до достижения длины самого длинного `string`-объекта. Другими словами, нам придется дополнять данный `string`-объект, начиная с позиции, равной его длине, и до значения `width()` данного `String_Pic`-объекта. Чуть-чуть предусмотрительности — и мы уже понимаем, что нам придется дополнять пробелами и другие изображения. А пока предположим, что у нас есть функция `pad`, которая принимает объект выходного потока и номера крайних позиций символов, нуждающихся в заполнении пробелами. Чуть ниже мы займемся вплотную реализацией этой функции.

Еще одна сложность вытекает из того, что количество строк, передаваемое функции `display`, может превышать значение `height` данного `String_Pic`-объекта. Такая ситуация возможна, если этот `String_Pic`-объект является частью горизонтально конкатенированного `Picture`-объекта, в котором одна сторона короче другой. Наши `Picture`-объекты, составляющие итоговое изображение, выравниваются по верхнему краю, но могут иметь различную высоту. Таким образом, нам придется проверять, попадает ли строка, заданная для вывода, в диапазон. Только после такого анализа можно написать следующий код.

```

void String_Pic::display(ostream& os, ht_sz row,
                        bool do_pad) const
{
    wd_sz start = 0;

    // Выводим строку, если она еще в пределах диапазона.
    if (row < height()) {
        os << data[row];
        start = data[row].size();
    }

    // Дополняем при необходимости выведенную строку.
    if (do_pad)
        pad(os, start, width());
}

```

Сначала проверяем, находится ли строка, заданная для вывода, в пределах диапазона, т.е. меньше ли заданное значение `row` значения `height()` этого `String_Pic`-объекта. Если да, мы выводим указанную строку и устанавливаем переменную `start` равной количеству выведенных символов. Независимо от того, вывели мы строку или нет, проверяем, нужно ли дополнять пробелами выходные данные. Если нужно, вы-

ываем функцию `pad`, передавая ей значения `start` и `width()` для данного `String_Pic`-объекта. Если заданная строка находится вне диапазона, значение переменной `start` остается равным 0, и в этом случае мы выводим целую строку (во всю ширину изображения) пробелов.

15.2.3. Дополнение выходных данных пробелами

Теперь мы можем вплотную заняться функцией дополнения выводимых строк пробелами. Поскольку нам нужно иметь доступ к этой функции из каждого производного класса, имеет смысл определить операцию `pad` как статическую (`static`) и защищенную (`protected`) функцию-член класса `Pic_base`.

```
class Pic_base {
    // Все остается в силе.
protected:
    static void pad(std::ostream& os, wd_sz beg, wd_sz end) {
        while (beg != end) {
            os << " ";
            ++beg;
        }
    };
};
```

Эта функция принимает `ostream`-объект, в который выводятся пробелы, и два значения, которые управляют их количеством. Когда функции `display` нужно вызвать функцию `pad`, она передает ей текущий номер позиции и номер, следующий за последней позицией изображения. Эти номера обозначают диапазон позиций, подлежащих заполнению пробелами в процессе выполнения текущей операции `display`.

Обратите внимание на использование ключевого слова `static` в объявлении функции `pad`. Как было указано в разделе 13.4, модификатор `static` означает, что `pad` — статическая функция-член. Такие функции отличаются от обычных функций-членов тем, что они не связаны с объектом данного типа класса.

Вас может удивить тот факт, что мы вправе определить функцию-член для абстрактного базового класса. Мол, если не могут существовать объекты базового класса, то почему должны существовать функции-члены? Однако вспомните, что каждый объект производного класса содержит часть базового класса. Каждый производный класс наследует любые функции-члены, определенные в базовом классе. Следовательно, функция базового класса будет выполняться на той “территории” объекта производного класса, которая “закреплена” за базовым классом. В этом конкретном случае определяемая нами функция является статическим членом, поэтому вопрос доступа к членам базового класса является спорным. Но важно понимать, что абстрактные классы могут определять члены данных и функции-члены (причем как обычные, так и статические). Эти функции должны получать доступ к тем частям (производных объектов), которые относятся к базовому классу.

Статические члены (как функции, так и данные, которые мы также можем определять) полезны тем, что они позволяют минимизировать имена, которые определены глобально. Наша функция `pad` — хороший тому пример. Мы можем представить много абстракций, которые включают понятие дополнения пробелами. В этой книге мы рассматривали дополнение пробелами в контексте написания форматированного отчета об успеваемости студентов, а также в контексте вывода символьных изображений (`Picture`-объектов). Если бы в классе `Picture` функция `pad` была определена как глобальная, то мы бы не смогли определить функцию `pad` для класса `Student_info` и

наоборот. Сделав функцию `pad` статическим членом, мы предусматриваем, что и другие абстракции в нашей программе могут иметь “свою” функцию `pad`. Если каждый класс определит, что функция `pad` (в данном случае) имеет смысл только в контексте данного класса, эти взаимно независимые функции смогут прекрасно сосуществовать внутри одной программы.

15.2.4. Класс *VCat_Pic*

Реализовать классы конкатенации совсем нетрудно. Начнем с класса `VCat_Pic`.

```
class VCat_Pic: public Pic_base
{
    friend Picture vcat(const Picture&, const Picture&);
    Ptr<Pic_base> top, bottom;
    VCat_Pic(const Ptr<Pic_base>& t, const Ptr<Pic_base>& b):
        top(t), bottom(b) { }

    wd_sz width() const
    { return max(top->width(), bottom->width()); }
    ht_sz height() const
    { return top->height() + bottom->height(); }
    void display(std::ostream&, ht_sz, bool) const;
};
```

Мы добавили `friend`-объявление для операции `vcat` и реализовали функции `height` и `width` как встраиваемые, или подставляемые (`inline`-функции). Если изображение конкатенируется вертикально, его высота (`height`) представляет собой сумму высот двух конкатенируемых компонентов, а ширина (`width`) определяется как большее из значений их ширины.

Функция `display` ненамного труднее.

```
void VCat_Pic::display(ostream& os, ht_sz row,
                      bool do_pad) const
{
    wd_sz w = 0;
    if (row < top->height()) {
        // Верхняя составляющая изображения.
        top->display(os, row, do_pad);
        w = top->width();
    } else if (row < height()) {
        // Нижняя составляющая изображения.
        bottom->display(os, row - top->height(), do_pad);
        w = bottom->width();
    }
    if (do_pad)
        pad(os, w, width());
}
```

Сначала мы определяем переменную `w`, которая будет содержать ширину текущей строки, на тот случай, если нам придется дополнять ее пробелами. Затем (посредством сравнения значения `row` с значением `height()` верхнего изображения) проверяем, в верхней ли составляющей изображения мы находимся. Если это так и есть, вызываем функцию `display` для вывода `top`-компонента, передавая ей `bool`-значение, которое означает необходимость дополнения выводимой строки пробелами. Вспомните, что функция `display` виртуальна, поэтому при ее выполнении будет вызвана функция `display`, которая соответствует тому виду объекта `Pic_base`-иерархии, на который в действительности указывает объект `top`. После вывода текущей строки мы запоминаем ее ширину в переменной `w`.

Если мы выводим не верхнюю составляющую изображения (top), значит, обрабатываем ее нижнюю часть (bottom). Если попали в else-ветвь, мы знаем, что номер данной строки (row) больше значения top->height(), поэтому при проверке попадания в диапазон теперь используется общая высота всего изображения. Если проверка дала истинный результат, значит, мы выводим bottom-составляющую изображения. Как и для ее верхней части, мы вызываем функцию display для объекта bottom, чтобы вывести нижнее изображение, отрегулировав номер строки с учетом уже выведенных строк верхнего изображения. После вывода очередной строки нижней составляющей мы сохраняем ее ширину. Если мы находимся вне диапазона, значение переменной w остается равным 0.

После вывода строки проверяется необходимость дополнения ее пробелами. В случае вызова функции pad пробелы выводятся, начиная с позиции, сохраненной в переменной w, и заканчивая полной шириной всего изображения.

15.2.5. Класс HCat_Pic

Не удивительно, что класс HCat_Pic во многом напоминает класс VCat_Pic.

```
class HCat_Pic: public Pic_base {
    friend Picture hcat(const Picture&, const Picture&);
    Ptr<Pic_base> left, right;
    HCat_Pic(const Ptr<Pic_base>& l, const Ptr<Pic_base>& r):
        left(l), right(r) { }

    wd_sz width() const
    { return left->width() + right->width(); }
    ht_sz height() const
    { return max(left->height(), right->height()); }
    void display(std::ostream&, ht_sz, bool) const;
};
```

Поскольку мы конкатенируем два изображения “бок о бок”, на этот раз ширина (width) итогового изображения равна сумме ширин его компонентов, а высота (height) — большей из двух его высот. Здесь функция display проще, чем ее “коллега” из класса VCat_Pic, поскольку мы делегируем управление выводом строк (и дополнением пробелами) составляющим изображениям.

```
void HCat_Pic::display(
    ostream& os, ht_sz row, bool do_pad) const
{
    left->display(os, row, do_pad || row < right->height());
    right->display(os, row, do_pad);
}
```

Сначала мы выводим заданную строку левого изображения. Эта строка будет дополнена в том случае, если на это будет указывать значение аргумента do_pad или номер этой строки будет находиться в пределах диапазона правого изображения (в этом случае мы просто обязаны дополнить каждую строку левого изображения, чтобы соответствующая строка правого изображения начиналась в нужной позиции выводимой строки). Если строка вышла за пределы диапазона левого изображения, эту проблему будет решать функция display, выполняемая для left-объекта. Аналогично мы делегируем вывод заданной строки правого изображения функции display, выполняемой для right-объекта. На этот раз мы передаем полученное в качестве аргумента значение do_pad без изменений, поскольку у нас нет причин заботиться о дополнении пробелами строк для правого изображения.

15.2.6. Класс `Frame_Pic`

Нам осталось реализовать только один производный класс, `Frame_Pic`.

```
class Frame_Pic: public Pic_base {
    friend Picture frame(const Picture&);
    Ptr<Pic_base> p;
    Frame_Pic(const Ptr<Pic_base>& pic): p(pic) { }

    wd_sz width() const { return p->width() + 4; }
    ht_sz height() const { return p->height() + 4; }
    void display(std::ostream&, ht_sz, bool) const;
};
```

Операции `height` и `width` передают свои “полномочия” изображению, которое заключается в рамочку. К значениям ширины и высоты прибавляется число 4 для учета “рамочных” символов и пробелов, которые отделяют рамку от “интерьера”, т.е. самого изображения, заключаемого в рамку.

Функция `display` громоздка, но не сложна.

```
void Frame_Pic::display(
    ostream& os, ht_sz row, bool do_pad) const
{
    if (row >= height()) {
        // вне диапазона.
        if (do_pad)
            pad(os, 0, width());
    } else {
        if (row == 0 || row == height() - 1) {
            // Верхняя или нижняя строка.
            os << string(width(), '*');
        } else if (row == 1 || row == height() - 2) {
            // Строка, примыкающая к верхней или нижней.
            os << " * ";
            pad(os, 1, width() - 1);
            os << " * ";
        } else {
            // Строка самого изображения.
            os << " * ";
            p->display(os, row - 2, true);
            os << " * ";
        }
    }
}
```

Сначала мы проверяем, попадает ли в диапазон заданный номер строки; если нет и при этом функции `display` велено (посредством значения аргумента `do_pad`) выполнить дополнение пробелами, она послушно заполняет ими всю строку. Если же заданный номер строки попадает в диапазон, мы рассматриваем три случая: вывод верхней или нижней границы рамочки, вывод строки пробелов, которая отделяет рамочку от внутреннего изображения, и вывод строки, принадлежащей изображению.

Мы знаем, что нам нужно выводить верхнюю или нижнюю границу рамочки, если номер строки равен 0 или значению выражения `height() - 1`. В этом случае мы выводим строку, которая полностью состоит из символов “звездочка”, и тем самым “рисует” одну из двух горизонтальных сторон рамочки. Если номер строки, подлежащей выводу, отличается от граничной на единицу, значит, мы должны вывести один (крайний левый) символ

“звездочка”, за ним — соответствующее количество пробелов, а потом — еще один (крайний правый) символ “звездочка”. Наконец, если мы выводим внутреннюю строку изображения, сначала необходимо вывести левую пару символов рамки (“звездочка” и пробел), затем строку, принадлежащую, собственно, самому изображению, и правую пару символов рамки (пробел и “звездочка”). Внутреннее изображение выводится посредством вызова функции `display`, но при передаче номера строки (`row - 2`) учитываются уже выведенные символы верхней границы рамки. При обращении к функции `display` мы указываем (посредством третьего аргумента `true`), что внутреннее изображение необходимо дополнять пробелами, чтобы правая сторона рамки получилась в виде вертикальной “линии” символов “звездочка”.

15.2.7. Не забывайте о друзьях

Итак, нам осталось добавить соответствующие `friend`-объявления в классы `Picture` и `Pic_base`. Мы уже отмечали, что нам нужно добавить `friend`-объявление в класс `Picture` для каждой `Picture`-операции, поскольку все эти операции используют `Ptr`-член внутри класса `Picture` и им необходимо разрешение для доступа к этому члену. Менее очевидной является коллекция `friend`-объявлений, которую нам нужно добавить в класс `Pic_base`.

```
// Пережающее объявление, описанное в разделе 15.3.
class Picture;

class Pic_base {
    friend std::ostream& operator<<(std::ostream&,
                                    const Picture&);

    friend class Frame_Pic;
    friend class HCat_Pic;
    friend class VCat_Pic;
    friend class String_Pic;

    // public-интерфейс отсутствует (за исключением деструктора).
    typedef std::vector<std::string>::size_type ht_sz;
    typedef std::string::size_type wd_sz;

    // Этот класс - абстрактный базовый класс.
    virtual wd_sz width() const = 0;
    virtual ht_sz height() const = 0;
    virtual void display(std::ostream&, ht_sz, bool) const = 0;

public:
    virtual ~Pic_base() { }

protected:
    static void pad(std::ostream& os, wd_sz, wd_sz);
};

class Picture {
    friend std::ostream& operator<<(std::ostream&,
                                    const Picture&);

    friend Picture frame(const Picture&);
    friend Picture hcat(const Picture&, const Picture&);
    friend Picture vcat(const Picture&, const Picture&);

public:
    Picture(const std::vector<std::string>& =
            std::vector<std::string>());

private:
    Picture(Pic_base* ptr): p(ptr) { }
    Ptr<Pic_base> p;
```



```
// Операции над Picture-объектами.
Picture frame(const Picture&);
Picture hcat(const Picture&, const Picture&);
Picture vcat(const Picture&, const Picture&);
std::ostream& operator<<(std::ostream&, const Picture&);
```

Первое friend-объявление в классе `Pic_base` понять нетрудно. Оператор вывода вызывает как функцию `height()`, так и функцию `display()`, поэтому он должен иметь доступ к этим членам. Удивить вас могут friend-объявления для классов, которые выведены из класса `Pic_base`. Разве они не получают “законный” доступ к членам класса `Pic_base`, так сказать, “по наследству”? Получать-то они получают этот самый доступ, но только к `protected`-членам конкретного объекта, а за исключением `protected`-функции `pad`, все члены класса `Pic_base` являются `private`-членами. Почему бы нам тогда не сделать и остальные члены защищенными? Все дело в том, что это не решило бы нашу проблему.

Член производного класса (такого, как `Frame_Pic`) может получить доступ к `protected`-членам “базовых” частей объектов собственного класса (такого, как `Frame_Pic`) или других классов, выведенных из него, но он не может получить доступ к `protected`-членам не связанных ни с чем объектов базового класса, которые не являются частью объекта производного класса. Следовательно, функции-члены класса `Frame_Pic`, который выведен из класса `Pic_base`, могут получить доступ к `protected`-членам `Pic_base`-частей `Frame_Pic`-объектов или объектов классов, выведенных из класса `Frame_Pic`, но они не могут получить прямой доступ к `protected`-членам не связанных ни с чем объектов класса `Pic_base`.

Может показаться, что это ограничение не относится к нашей программе. Как-никак, класс `Pic_base` — абстрактный класс, поэтому отдельные объекты этого класса существовать никак не могут. Однако правила доступа применяются к любой попытке получить доступ к члену, казалось бы, отдельного `Pic_base`-объекта, даже если во время выполнения этот объект является объектом производного класса.

Рассмотрим, к примеру, функцию `height` класса `Frame_Pic`.

```
ht_sz Frame_Pic::height() const { return p->height() + 4; }
```

Эта функция использует выражение `p->height()`, которое для получения указателя неявно вызывает член `operator->` класса `Ptr` (см. раздел 14.3). Этот указатель имеет тип `Pic_base*`, и для получения доступа к члену `height` соответствующего объекта мы должны выполнить его разыменование. А поскольку, повторимся, типом этого указателя является `Pic_base*`, компилятор проверит защиту, как будто бы мы попытались получить доступ к некоторому члену `Pic_base`-объекта, хотя реальный объект будет иметь тип, выведенный из класса `Pic_base`. Таким образом, даже если мы и сделали член `height` защищенным, для разрешения этого доступа мы по-прежнему должны включить friend-объявления. По аналогичным причинам соответствующее friend-объявление требуется каждому из производных классов в нашей иерархии.

Это правило может показаться удивительным, но его логика довольно проста: если бы язык предоставил производным объектам доступ к защищенным членам объекта базового класса, то было бы несложно ниспровергнуть существующие механизмы защиты. Если бы нам понадобился доступ к некоторому `protected`-члену класса, мы могли бы определить новый класс, выведенный из класса, к которому хотим получить доступ. Затем определили бы операцию (в качестве члена этого нового производного класса), которой требовался бы доступ к `protected`-члену класса. Тем самым мы, по

сути, переопределили бы стратегию защиты дизайнера исходного класса. По этой причине protected-доступ ограничивается для членов “базовой” части объекта производного класса, а прямой доступ к членам объектов базового класса не разрешается.

15.3. Резюме

Абстрактные базовые классы имеют одну или несколько чистых виртуальных (virtual) функций.

```
class Node {  
    virtual Node* clone() const = 0;  
};
```

Это определение означает, что clone — чистая виртуальная функция, а Node — абстрактный базовый класс. Создание объектов абстрактного класса невозможно. Свойство абстрактности классов может “передаваться по наследству”: если некоторый класс не переопределяет хотя бы одну унаследованную чистую виртуальную функцию, значит, этот производный класс также является абстрактным.

Опережающие объявления (forward declarations). Требование определять имена до их использования (см. раздел 0.8) создает проблемы при написании семейств классов, которые обращаются один к другому. Чтобы избежать этой проблемы, можно объявить только имя класса.

```
class имя_класса;
```

В этом случае элемент *имя_класса* лишь называет класс, но не описывает его.

Мы использовали одно такое опережающее объявление для класса Picture в разделе 15.2.7. Класс Picture содержит член типа Ptr<Pic_base>, а в классе Pic_base имеется friend-объявление для функции operator<<, которая использует тип const Picture&. Следовательно, эти два класса обращаются один к другому.

Подобные взаимные зависимости типов могут привести к созданию программ, которые невозможно выполнить. Рассмотрим пример.

```
class Yang; // опережающее объявление.
```

```
class Yin {  
    Yang y;  
};
```

```
class yang {  
    Yin y;  
};
```

Этот код означает, что каждый объект класса Yin (*инь*, т.е. женское начало в китайской философии. — Прим. перев.) содержит объект класса Yang (*ян*, т.е. мужское начало), который содержит объект класса Yin, и т.д. Реализация подобных типов потребовала бы бесконечной памяти.

Взаимная зависимость в наших классах символьных изображений не вызывает таких проблем, поскольку класс Picture не содержит члена типа Pic_base напрямую. Он включает член типа Ptr<Pic_base>, который содержит объект типа Pic_base*. При таком использовании указателей удастся избежать бесконечно вложенных объектов.

Более того, при использовании указателя (или ссылки) компилятору на самом деле не нужно знать подробности типа объекта до тех пор, пока не будут вызваны операции, выполняемые посредством этого указателя (или ссылки). Поскольку в объявлении функции operator<< тип const Picture& используется только для объявления

типа параметра, компилятору необходимо знать только то, что имя `Picture` обозначает некоторый тип. Детали же реализации этого типа не потребуются до тех пор, пока мы не определим оператор `operator<<`.

Упражнения

15.0. Скомпилируйте, выполните и протестируйте программы, приведенные в этой главе.

15.1. Протестируйте свою систему, написав программу, которая выполняет следующий код.

```
Picture p = // Некоторое исходное стартовое изображение.
Picture q = frame(p);
Picture r = hcat(p, q);
Picture s = vcat(q, r);
cout << frame(hcat(s, vcat(r, q))) << endl;
```

15.2. Переделайте класс `Frame_Pic`, чтобы для обрамления (рамки) использовались три различных символа: один — для углов, второй — для верхней и нижней границ, а третий — для боковых сторон.

15.3. Предоставьте пользователям возможность самим задавать символы, используемые для создания рамки.

15.4. Добавьте в класс `Picture` операцию `reframe`, при выполнении которой должны меняться “рамочные” символы. Эта операция должна изменить все рамки внутри изображения.

15.5. Переделайте класс `hCat_Pic` так, чтобы при конкатенации изображений различных размеров более короткое было отцентрировано относительно области, занимаемой более длинным. Другими словами, если горизонтально конкатенировать два изображения, одно из которых занимает четыре строки, а другое — две, то в части, занимаемой более коротким изображением, первая и последняя строки итогового изображения должны остаться пустыми.

15.6. Классы `Vec` и `Str`, которые мы разработали в главах 11 и 12, можно успешно использовать для реализации `Picture`-классов. Переделайте кодовый “материал”, представленный в этой главе, для использования контейнера `Vec<Str>` вместо `vector<string>` и протестируйте свой вариант реализации.

КУДА ТЕПЕРЬ ДЕРЖАТЬ НАМ ПУТЬ

Мы подошли к концу основной части нашей книги. Такое “резкое” окончание может показаться преждевременным: ведь и в самом языке C++, и в библиотеке есть еще разделы, не описанные нами, но весьма достойные внимания читателя. Все же мы решили поставить точку и сделали это по двум важным причинам.

Первая состоит в том, что мы уже рассмотрели инструментальные средства, которые можно использовать для решения довольно широкого круга задач программирования. Мы считаем, что наилучшая стратегия — на практике освоить эти средства для решения собственных задач, а потом уже браться за изучение новых возможностей языка. Пожалуй, неплохо было бы с самого начала перечитать эту книгу, выполняя все предложенные в ней упражнения, пропущенные при первоначальном знакомстве с материалом.

Если вас интересуют идеи, связанные со стилем или технологией программирования, мы рекомендуем обратиться к нашей предыдущей книге, *Ruminations on C++* (Addison-Wesley, 1997), которая содержит смесь стилистических опусов и примеров программирования.

Вторая причина касается тех, кто уже написал достаточно много программ с использованием рассмотренного здесь материала. Такие читатели вряд ли долго выдержат несколько наставнический стиль, принятый в этой книге. Возможно, дальнейший опыт C++-программирования им стоит приобретать с помощью книг, которые содержат больше детализированной информации и меньше пояснений, чем в данной книге.

16.1. Используйте уже освоенные абстракции

Существует очень старая история о приезде, который заблудился в Нью-Йорке. Однажды с билетом в руках на фортепианный концерт он останавливает прохожего и спрашивает: “Простите, как мне попасть в Карнеги-Холл?”. Ответ был таков: “Практикой!”.

Важно хорошо понимать, как применять уже знакомые вам абстракции, и только потом пытаться освоить новые. При использовании абстракций часто достаточно библиотечных средств, но иногда приходится создавать и собственные. Сочетая идеи, почерпнутые из стандартной библиотеки (которые можно применить к широкому диапазону задач), с идеями решения проблем в конкретной предметной области, мы можем писать эффективные программы, затрачивая на это удивительно небольшие усилия. В частности, хорошо разработанные абстракции “собственноручного”, так сказать, изготовления часто можно использовать для решения задач, о которых в момент разработки никто даже и не помышлял.

Примерами таких абстракций могут служить классы, написанные в главе 13 для учета успеваемости студентов, и классы из главы 15, позволяющие создавать символьные изображения. Если классы, генерирующие символьные изображения, мы использовали в различных вариациях в течение ряда лет, то классы по учету успеваемости были написаны специально для этой книги. Теперь мы поняли, что могли бы очень удачно объединить эти абстракции.

Идея совместного использования этих абстракций состоит в построении гистограммы оценок студентов с помощью символьных изображений. Ни для кого не секрет, что визуальная индикация позволяет увидеть аномалии гораздо быстрее, чем информация, представленная в форме обычных числовых таблиц. Итак, основная идея — преобразовать каждую итоговую оценку в строку символов "=", длина которой должна быть пропорциональна значению этой оценки. Например, при соответствующих входных данных мы могли бы сгенерировать отчет в следующем виде.

```
*****
*                                     *
* James      =====               *
* Kevin      =====               *
* Lynn       =====               *
* Marykate   =====               *
* Pat        =====               *
* Paul       =====               *
* Rhia       =====               *
* Sarah      =====               *
*                                     *
*****
```

Лишь взглянув на эту гистограмму, можем сразу сказать, что у студента Pat с учебой неладит.

Наш пример удачен по своей лаконичности. Кроме того, он показывает, насколько прямо решение отображает проблему.

```
picture histogram(const vector<Student_info>& students)
{
    picture names;
    picture grades;

    // для каждого студента ...
    for (vector<Student_info>::const_iterator it = students.begin();
         it != students.end(); ++it) {

        // ...создаем вертикально конкатенированные изображения
        // имен и оценок отдельно.
        names = vcat(names, vector<string>(1, it->name()));
        grades = vcat(grades,
                      vector<string>(1,
                                     " " + string(it->grade() / 5,
                                     '=')));
    }

    // Горизонтально конкатенируем изображения имен и оценок,
    // объединяя их в одно единое изображение.
    return hcat(names, grades);
}
```

Наша функция `histogram` принимает `const`-ссылку на вектор объектов типа `Student_info`, каждый из которых представляет данные об одном студенте. Из этих данных мы создаем (в цикле `for`) два `picture`-объекта: один (`names`) будет содержать имена

всех студентов, а другой (grades) — строку, соответствующую их итоговым оценкам. По завершении цикла `for` выполняем конкатенацию этих двух `Picture`-объектов в горизонтальном направлении, в результате которой на каждой строке выводится имя студента и соответствующая ему итоговая оценка. Поскольку каждое изображение программным путем “принимает форму” прямоугольника, можно сказать, что горизонтальная конкатенация автоматически настраивается для различных длин имен студентов.

В функции `main` этой программы выполняется построение вектора посредством обычного чтения файла, содержащего данные о студентах. Затем вызывается функция `histogram`, генерирующая объект класса `Picture`, который заключается в рамочку (с помощью функции `frame`). Результат вызова функции `frame` отображается на экране за счет использования оператора вывода.

```
int main()
{
    vector<Student_info> students;
    Student_info s;

    // Считываем имена и оценки.
    while (s.read(cin))
        students.push_back(s);

    // Размещаем данные в алфавитном порядке.
    sort(students.begin(), students.end(), Student_info::compare);

    // Выводим имена и гистограммы.
    cout << frame(histogram(students)) << endl;
    return 0;
}
```

Самая важная идея этого примера — отсутствие новых идей! Вы видите, насколько прост приведенный программный код; эта простота — результат знакомства с идеями, рассмотренными выше в этой книге. Здесь мы их всего лишь удачно объединили, причем способы объединения мы никак не могли предугадать, когда только приступали к реализации этих идей. Такой уровень осознания возможностей программирования приходит, конечно же, только с практикой.

16.2. Стремитесь узнать больше

Рано или поздно, но вы обязательно почувствуете необходимость поближе познакомиться с языком C++ и его библиотекой. Идеал освоения неведомого ранее (не братья за новое, не разобравшись в “старом”), как и большинство идеалов, редко достижим на практике. На определенном этапе у вас возникнут вопросы, на которые в этой книге вы не найдете ответов, и вам придется обратиться к другим источникам.

Идя вам навстречу, мы можем отметить две книги. Первая, *Язык программирования C++ Бьярни Страуструпа*, третье издание (Addison-Wesley, 1998), представляет собой наиболее полный источник информации о C++. В ней описаны все аспекты как языка, так и библиотеки и собрано все (и даже больше), что необходимо знать программисту. Вторая книга, *Обобщенное программирование и STL* Мэтью Остерна (*Generic Programming and the STL* by Matthew Austern) (Addison-Wesley, 1999), содержит алгоритмы стандартной библиотеки, описанные более подробно. Важно то, что этой книге можно абсолютно доверять, поскольку Остерн, хотя он и не был первым автором этой части библиотеки — сия честь выпала на долю Александра Степанова (Alex Stepanov), тесно сотрудничавшего со Степановым в течение последних семи лет и являющегося одной из основных “движущих сил” дальнейшей эволюции этой библиотеки.

Однако объемы перечисленных книг довольно внушительны, поэтому вы вольны обращаться к любой дополнительной литературе. На этот случай мы сообщаем вам следующий Web-адрес: <http://www.acsu.org>. Там вы найдете обзор более чем 2 000 книг, многие из которых связаны с языком C++.

Пребывая в поисках нужной, с вашей точки зрения, книги, всегда помните о том, что книги на полке не сделают вас более квалифицированным программистом. В конечном счете, единственный способ совершенствования в программировании — писать программы. Удачи вам!

Упражнения

- 16.0. Скомпилируйте, выполните и протестируйте программы, приведенные в этой главе.
- 16.1. Напишите программу самовоспроизведения. Такая программа не принимает никаких входных данных, а при выполнении выводит в стандартный выходной поток копию собственного исходного текста.

Приложение А

Язык C++ (подробно)

Это приложение преследует две цели: предоставить некоторый дополнительный материал, связанный с синтаксическими подробностями низкого уровня, а также собрать в одном разделе описание выражений и инструкций языка, включая те, о которых еще не было упомянуто в этой книге. Материал “низкоуровневого” характера относится, в основном, к сложностям синтаксиса C++ и к деталям встроенных арифметических типов, которые унаследованы от языка программирования C. Эти детали не образуют необходимое условие для понимания программ, приведенных в этой книге, и вообще для написания эффективных C++-программ. Тем не менее существует множество программ, которые требуют знания этих деталей, поэтому в рассмотрении данной темы существует определенная польза.

В этом приложении синтаксис описывается следующим образом: символы, набранные равноширинным шрифтом, используются как есть; слова, выделенные *курсивом*, означают синтаксические категории; многоточие (...) подразумевает некоторое (возможно, нулевое) количество повторений предшествующего элемента; а конструкции, заключенные в “курсивные” квадратные скобки (*[]*), попросту необязательны. Кроме того, “курсивные” фигурные скобки (*{ }*) используются для группирования, а символ “|” — для обозначения альтернативных вариантов. Например, запись

*инструкция_объявления: Спецификаторы [Описатель [инициализатор]]
[, Описатель [инициализатор]]... ;*

означает, что *инструкция_объявления* состоит из элемента-последовательности *Спецификаторы*, за которым следует некоторое (возможно, нулевое) количество элементов *Описатель*, необязательно сопровождаемых соответствующими элементами *инициализатор*, но вся цепочка непременно должна завершаться точкой с запятой.

А.1. Объявления

Объявления могут быть трудны для понимания, особенно если они используются для объявления нескольких имен с различными типами или относятся к функциям, которые возвращают указатели на функции. Например, в разделе 10.1.1 мы видели, что запись

```
int* p, q;
```

определяет *p* как объект типа “указатель на int-значение”, а *q* — как объект типа *int*. В разделе 10.1.2 мы видели, что запись

```
double (*get_analysis_ptr())(const vector<Student_info>&);
```

объявляет *get_analysis_ptr* как функцию без аргументов, которая возвращает указатель на функцию с аргументом типа *const vector<Student_info>&*, которая возвра-

шает double-значение. Подобные объявления можно сделать яснее, переписав их, например, следующим образом.

```
int* p;  
int q;
```

и

```
// Определяем analysis_fp как имя для типа функции, которая  
// принимает аргумент типа const vector<Student_info>& и  
// возвращает double-значение.  
typedef double (*analysis_fp)(const vector<Student_info>&);  
analysis_fp get_analysis_ptr();
```

К сожалению, эта стратегия не в состоянии уберечь вас от необходимости разбираться в подобных, сбивающих с толку объявлениях, которые могут встречаться в программах других программистов.

В общем случае объявление имеет следующий вид.

*инструкция_объявления: Спецификаторы [Описатель [инициализатор]]
[, Описатель [инициализатор]]... ;*

Оно объявляет имя для каждого из указанных элементов *Описатель*. Эти имена существуют с момента их объявления и до конца области видимости данного объявления. Некоторые объявления являются также определениями. Имена могут объявляться несколько раз, но должны быть определены только однажды. Объявление является одновременно определением, если требует выделения некоторой области памяти или содержит определение тела класса или функции.

C++ унаследовал синтаксис объявления от языка C. Ключ к пониманию объявлений лежит в осознании того, что каждое объявление начинается с последовательности *Спецификаторы*, которая коллективно определяет тип и другие атрибуты объявляемого элемента. За последовательностью *Спецификаторы* следует некоторое (возможно, нулевое) количество элементов *Описатель*, каждый из которых может иметь соответствующий ему элемент *инициализатор*. Каждый элемент *Описатель* присваивает (вернее, приписывает) имени тип, который зависит от спецификаторов и формы данного описателя.

Первый шаг к пониманию любого объявления — найти границу между спецификаторами и описателями. Сделать это чрезвычайно просто: все спецификаторы представляют собой ключевые слова или имена типов, поэтому они (спецификаторы) заканчиваются сразу перед первым символом, который не относится ни к одному из них. Например, в объявлении

```
const char * const * const * cp;
```

первым символом, который не является ни ключевым словом, ни именем типа, есть символ *, поэтому в качестве спецификаторов здесь используются слова const char, а единственным описателем служит * const * const * cp.

Рассмотрим для примера еще одно объявление из раздела 10.1.2.

```
double (*get_analysis_ptr()(const vector<Student_info>&);
```

Границу между спецификаторами и первым описателем найти здесь несложно: double — это имя типа, а следующая за ним открывающая скобка не является ни ключевым словом, ни именем типа. Следовательно, в качестве элемента-последовательности *Спецификаторы* здесь используется просто слово double, а описателем является вся остальная часть объявления, не включая точку с запятой.

А.1.1. Спецификаторы

Элемент-последовательность *Спецификаторы* можно разделить на три большие категории: спецификаторы типов, спецификаторы классов памяти и смешанные спецификаторы.

Спецификаторы: { *Спецификатор_типа* | *Спецификатор_класса_памяти* | *иной_спецификатор_объявления* } ...

Однако такое разделение на категории служит только для облегчения понимания, поскольку в самих объявлениях такого разделения нет: спецификаторы могут располагаться в любом порядке.

Спецификаторы типа определяют тип, который лежит в основе любого объявления. (Встроенные типы мы рассмотрим в разделе А.2.)

Спецификатор_типа: char | wchar_t | bool | short | int |
long | signed | unsigned | float |
double | void | *имя_типа* | const | volatile
имя_типа: *имя_класса* | *имя_перечисления* | typedef-*имя*

Спецификатор `const` означает, что объекты данного типа не могут быть модифицированы. Спецификатор `volatile` сообщает компилятору, что переменная может быть изменена вне определения языка и что агрессивные оптимизации не должны проводиться.

Обратите внимание на то, что ключевое слово `const` может выступать как часть элемента *Спецификаторы*, модифицируя таким образом тип, и как часть элемента *Описатель*, задавая `const`-указатель. Однако какая бы то ни было неопределенность исключена, поскольку в качестве части элемента *Описатель* ключевое слово `const` всегда сопровождается символом “звездочка” (*).

Спецификаторы класса памяти определяют местонахождение и время существования переменной.

Спецификатор_класса_памяти: register | static | extern | mutable

Спецификатор `register` означает, что компилятор должен попытаться оптимизировать код с точки зрения производительности, поместив по возможности объект в регистр.

Обычно локальные переменные разрушаются при выходе из блока, в котором они были объявлены; статические (`static`) переменные сохраняют свое значение при входе в область видимости и выходе из нее.

Спецификатор `extern` означает, что текущее объявление не является определением, тем самым подразумевая, что соответствующее определение существует в каком-то другом месте программы.

Класс памяти `mutable` используется только для членов данных класса и позволяет этим членам данных быть модифицированными даже в случае, если они являются членами `const`-объектов.

Иные спецификаторы объявлений определяют свойства, которые не связаны с типами.

иной_спецификатор_объявления: friend | inline | virtual | typedef

Спецификатор `friend` (см. разделы 12.3.2 и 13.4.2) переопределяет степень защиты.

Спецификатор `inline` предназначен для определений функций и служит указанием компилятору встроить по возможности заданный код “в строку” вызова функции. При разворачивании такой функции ее определение должно находиться в области видимости, поэтому тело встраиваемой (`inline`-) функции обычно помещают в тот же заголовок, в котором эта функция объявляется.

Спецификатор `virtual` (см. раздел 13.2.1) может быть использован только с функциями-членами и означает функцию, вызовы которой могут быть связаны динамически.

Спецификатор `typedef` (см. раздел 3.2.2) означает синоним для заданного типа.

А.1.2. Описатели

В любом объявлении для каждого описателя объявляется только один элемент, которому присваивается некоторое имя, (неявно) класс памяти, тип и другие атрибуты в соответствии с заданными спецификаторами. Спецификаторы и описатель вместе определяют, что указанное имя относится к объекту, массиву, указателю, ссылке или функции. Например, инструкция

```
int *x, f();
```

объявляет, что `x` — это указатель на `int`-значение, а `f` — функция, которая возвращает `int`-значение. Описатели `*x` и `f()` имеют такой вид, который позволяет понять различие между типами элементов `x` и `f`.

```
Описатель: [ * [ const ] | & ] ... Непосредственный_описатель
Непосредственный_описатель:      Описатель_идентификатор |
                                   (Описатель) |
                                   Непосредственный_описатель(Список_объявляемых_параметров) |
                                   Непосредственный_описатель[ Константное_выражение ]
```

Элемент *Описатель_идентификатор* представляет собой идентификатор, возможно составной.

```
Описатель_идентификатор:
    [ Спецификатор_вложенного_имени ] Идентификатор
Спецификатор_вложенного_имени:
    { Имя_класса_или_пространства_имен :: } ...
```

Если описатель представляет собой элемент *Непосредственный_описатель*, который состоит только из элемента *Описатель_идентификатор*, то он означает, что этот идентификатор имеет свойства, определяемые элементом-последовательностью *Спецификаторы*, т.е. без каких-либо последующих модификаций. Например, в объявлении

```
int n;
```

`n` является описателем, который представляет собой элемент *Непосредственный_описатель*, состоящий только из элемента *Описатель_идентификатор*, это подразумевает, что переменная `n` имеет тип `int`.

Если описатель имеет одну из иных возможных форм, то тип идентификатора можно определить следующим образом. Сначала допустите, что `T` — тип, обозначаемый элементом *Спецификаторы*, игнорируя не относящиеся к типу свойства (такие, как `friend` или `static`), а `D` — описатель. Затем повторяйте следующие действия до тех пор, пока элемент `D` не сократится до элемента *Описатель_идентификатор*, и на этом этапе элемент `T` будет обозначать искомый тип.

1. Если элемент `D` имеет форму `D1`, замените его элементом `D1`.
2. Если `D` имеет форму `* D1` или `* const D1`, замените элемент `T` обозначением “указатель на `T`-значение” или “константный указатель на `T`-значение” (а не “указатель на константное `T`-значение”), в зависимости от того, имеется ли ключевое слово `const`. Затем замените элемент `D` элементом `D1`.
3. Если элемент `D` имеет форму `D1(Список_объявляемых_параметров)`, замените элемент `T` обозначением “функция, возвращающая `T`-значение” с аргументами,

определенными элементом *Список_объявляемых_параметров*, и замените элемент *D* элементом *D1*.

4. Если элемент *D* имеет форму *D1[Константное_выражение]*, замените элемент *T* обозначением “массив *T*-значений”, количество элементов в котором задается элементом *Константное_выражение*, и замените элемент *D* элементом *D1*.
5. Наконец, если описатель имеет форму *& D1*, замените *T* обозначением “ссылка на *T*-значение”, а элемент *D* элементом *D1*.

В качестве примера рассмотрим следующее объявление.

```
int *f();
```

Начнем с того, что элементам *T* и *D* соответствуют обозначения *int* и **f()*, соответственно, поэтому элемент *D* имеет форму **D1*, где *D1* “равно” *f()*.

Можно было бы подумать, что элементу *D* соответствует одна из двух возможных (в данном случае) форм: *D1()* или **D1*. Но если бы элемент *D* имел форму *D1()*, то элементу *D1* “досталось” бы обозначение **f* и тому же элементу *D1* пришлось бы также “играть роль” элемента *Непосредственный_описатель* (поскольку, согласно приведенным в начале этого раздела правилам грамматики, перед круглыми скобками разрешается стоять только элементу *Непосредственный_описатель*). Но определение элемента *Непосредственный_описатель* говорит о том, что он не может содержать символ “звездочка” (*). Следовательно, элемент *D*, которому соответствует выражение **f()*, может иметь лишь форму **D1*, где элементом *D1* является обозначение *f()*.

Выяснив, что *D1* — это *f()*, мы должны заменить элемент *T* обозначением “указатель на *T*-значение”, что в данном случае соответствует обозначению “указатель на *int*-значение”, и заменить элемент *D* обозначением *f()*.

Пока еще мы не сократили элемент *D* до формы, допустимой для элемента *Описатель_идентификатор*, поэтому должны повторить этот процесс. На этот раз элементом *D1* может служить только обозначение *f*, поэтому мы заменяем элемент *T* обозначением “функция, возвращающая *T*-значение” (в данном случае эта формула звучит как “функция без аргументов, возвращающая указатель на *int*-значение”), и заменяем элемент *D* обозначением *f*.

На этом этапе мы все же сократили элемент *D* до формы, допустимой для элемента *Описатель_идентификатор*, поэтому процесс окончен. Мы определили, что инструкция

```
int *f();
```

объявляет, что *f* имеет тип “функции без аргументов, возвращающей указатель на *int*-значение”.

Рассмотрим еще один пример. Объявление

```
int* p, q;
```

имеет два описателя, **p* и *q*. Для каждого описателя элементом *T* является тип *int*. Для первого описателя элементом *D* служит обозначение **p*, поэтому мы “переводим” *T* как “указатель на *int*-значение”, а *D* — как *p*. Следовательно, это объявление надевает переменную *p* типом “указателя на *int*-значение”.

Второй описатель анализируется независимо от первого, т.е. элементом *T* опять-таки является тип *int*, а элементом *D* — имя *q*. Теперь очевидно, что переменная *q* имеет просто тип *int*.

Наконец, проанализируем “загадочный” пример из раздела 10.1.2.

```
double (*get_analysis_ptr())(const vector<Student_info>&);
```

Анализ будет включать следующие пять этапов.

1. T: double

D: (*get_analysis_ptr())(const vector<Student_info>&)

2. T: функция, возвращающая double-значение с аргументом типа const vector<Student_info>&

D: (*get_analysis_ptr())

3. T: функция, возвращающая double-значение... (как записано выше)

D: *get_analysis_ptr()

4. T: указатель на функцию, возвращающую double-значение...

D: get_analysis_ptr()

5. T: функция, возвращающая указатель на функцию, возвращающую double-значение...

D: get_analysis_ptr

Итак, мы выяснили, что `get_analysis_ptr` — это функция, которая возвращает указатель на функцию, возвращающую double-результат и принимающую аргумент типа `const vector<Student_info>&`. Оставляем читателю возможность развернуть `const vector<Student_info>&` в качестве упражнения.

К счастью, немногие объявления функций имеют такой “ужасный” вид; большинство из них выглядит следующим образом.

Описатель: Описатель_идентификатор(Список_объявляемых_параметров)

Безусловно, самые большие трудности вызывают объявления функций, которые возвращают указатель на функцию.

A.2. Типы

Типами “пропитаны” все C++-программы. Каждый объект, выражение и функция имеют тип, и именно тип объекта (выражения или функции) определяет его (ее) поведение. Тип каждого объекта (выражения или функции), кроме типа объекта внутри иерархии наследования, к которому реализуется доступ посредством указателя или ссылки, известен во время компиляции.

В C++ типы можно воспринимать как средства структурирования и доступа к памяти, а также как способы определения операций, которые могут быть выполнены над объектами данного типа. Другими словами, типы определяют как свойства данных, так и операции, которые можно выполнять над этими данными.

Несмотря на то что в этой книге ставится акцент на использование и создание высокоуровневых структур данных, не менее важно понимать примитивные типы, используемые для их построения. Эти примитивные типы представляют абстракции, очень близкие к таким низкоуровневым, “железным” формам представления информации, как числа (целые и с плавающей точкой), символы (включая “широкие”, или двухбайтовые, символы, позволяющие представлять алфавиты всех существующих в мире языков), значения истинности и машинные адреса (указатели, ссылки и массивы). Литералы, также именуемые константами, выражают целочисленные, вещественные, булевы (логические), символьные или строковые значения. Этот раздел содержит обзор средств, связанных с использованием встроенных типов.

А.2.1. Целые типы

Язык C++ унаследовал от C множество целых типов, включая целочисленные, булевы и символьные. Поскольку по своему замыслу C++ ориентирован на эффективную работу в самых разнообразных аппаратных системах, многие детали реализации фундаментальных типов не определяются точно синтаксисом языка, а оставлены (для уточнения) конкретным C++-средам.

А.2.1.1. Целочисленные типы

Существует три различных целочисленных типа со знаком и столько же без него.

short int	int	long int
unsigned short int	unsigned int	unsigned long int

Типы short и long можно употреблять в “сокращенном” виде, опустив ключевое слово int. Ключевые слова, если их больше одного, могут следовать в любом порядке.

Каждый из этих типов может представлять любое целое число в пределах диапазона, определяемого конкретной C++-средой. Каждый тип (кроме первого) должен предлагать диапазон, не меньший того, который предлагается предыдущим типом. Диапазоны для типов short int и int должны быть не меньше $\pm 32767 (\pm(2^{15} - 1))$, а диапазон для типа long int — не меньше $\pm 2147483647 (\pm(2^{31} - 1))$.

Каждому целочисленному типу со знаком соответствует тип без знака. Каждый тип без знака представляет целые числа по модулю 2^n , где n зависит от типа и C++-среды. Аналогично типам со знаком число n , которое соотносится с каждым типом без знака (за исключением типа unsigned char), должно быть не меньше числа n для предыдущего типа. Более того, каждый тип без знака должен быть в состоянии сохранить каждое неотрицательное число в диапазоне соответствующего типа со знаком, а каждый тип со знаком должен иметь такое же внутреннее представление, как и соответствующий тип без знака (для общих значений). Из этих требований следует, что четыре типа без знака должны иметь дополнительный бит, который соответствует знаковым битам типов со знаком, т.е. типы без знака должны соответствовать значениям числа n , которые по меньшей мере равны 8, 16, 16 и 32 соответственно.

Компиляторам для типов со знаком разрешается использовать представление в виде дополнения до единицы либо до двух.

Стандартная библиотека определяет тип size_t, который служит синонимом для одного из типов без знака. Он гарантирует способность хранить размер максимально большого объекта, включая массивы. Этот тип определяется в системном заголовке <stddef>.

Целочисленные литералы. Любой целочисленный литерал представляет собой последовательность цифр (необязательно упрежденную индикатором основания системы счисления), за которой необязательно следует индикатор размера. Строго говоря, целочисленные литералы не имеют знаков, поэтому -3 является выражением, а не литералом.

Если литерал начинается с обозначения 0x или 0X, значит, данное целое число представлено в шестнадцатеричной системе, в которой, помимо обычных десятичных цифр, используются и такие “цифры”: A a B b C c D d E e F f. Если литерал начинается с нуля (0), за которым не стоит буква x или X, значит, данное целое число представлено в восьмеричной системе, “строительными кирпичиками” которой являются только восемь цифр: 0 1 2 3 4 5 6 7.

В качестве индикатора размера используются буквы u, l, ul или lu (прописные или строчные). Если литерал включает индикатор ul или lu, то он (литерал) имеет тип unsigned long. Если же использован индикатор u, то этот литерал имеет тип

unsigned при условии, если значение литерала подходит к этому типу; в противном случае используется тип unsigned long. Индикатор l означает, что данный литерал имеет тип long при условии, если значение литерала подходит к этому типу; в противном случае используется тип unsigned long.

Если литерал содержит индикатор основания системы счисления, но не имеет индикатора размера, то данному литералу присваивается тип, который первым подойдет по значению из следующей цепочки типов: int, unsigned, long и unsigned long. Если отсутствует как индикатор основания системы счисления, так и индикатор размера, то данный литерал имеет тип int при условии, если значение литерала подходит к этому типу, и long в противном случае.

Эти правила подразумевают, что тип целочисленного литерала часто зависит от C++-среды. К счастью, в хорошо написанных программах целочисленные литералы большого размера используются редко, поэтому эти детали не имеют такого большого значения. Тем не менее мы сочли нужным привести их на случай, если вам потребуется такая информация.

A.2.1.2. Булев тип

Выражения, которые обрабатываются как условия, имеют тип bool. Возможными значениями типа bool являются true и false. В качестве значения истинности можно использовать число или указатель. В таких контекстах ноль считается значением false, а любое другое число — значением true.

При использовании bool-значения в качестве числа значение false воспринимается как 0, а true — как 1.

Булевы литералы. Булевыми литералами являются только обозначения true и false, каждое из которых имеет тип bool с явно выраженным значением.

A.2.1.3. Символьный тип

В C++ символы представляются как очень маленькие целые числа. В частности, их можно использовать в арифметических выражениях таким же образом, как обычные целые.

Подобно целым числам, символы могут быть со знаками или без них, поэтому предусмотрены отдельные типы signed char и unsigned char. В любой C++-среде диапазон для значений типа signed char должен обеспечивать хранение каждого символа из базового набора и быть не меньше ± 127 ($\pm (2^7 - 1)$).

Кроме того, существует обычный тип char, который, хотя и является отдельным типом, должен иметь такое же представление, как один из двух упомянутых выше. Право выбора предоставляется C++-среде.

Существует также тип “широких символов”, wchar_t, значения которого должны содержать по меньшей мере 16 разрядов. Этот тип предназначен для представления алфавитов с большим количеством символов (например, японского). Значения типа wchar_t должны обрабатываться так же, как значения одного из иных целых типов. Выбор этого “другого” целого типа зависит от C++-среды и обычно имеет целью получить максимально эффективное представление.

Символьные литералы. Символьный литерал (например, 'a') обычно представляет собой единственный символ, заключенный в одинарные кавычки. Он имеет тип char, который, как мы знаем из раздела A.2.1.3, в действительности является разновидностью целого типа. Каждая C++-среда определяет соответствие между символами, например a, и их целыми значениями. Большинство программ не зависит от этого соот-

ветствия, поскольку программисты могут писать такие литералы, как 'a', имея в виду "целое значение, которое соответствует символу a". Поскольку это соответствие может изменяться при переходе от одной C++-среды к другой, программисты не должны опираться на арифметические свойства символов. Например, нет никакой гарантии, что 'a'+1 будет равно 'b'. Однако цифры должны гарантированно иметь последовательные значения. Например, '3' + 1 всегда должно быть равно '4' (но необязательно 4).

Строковые литералы. Строковый литерал (например, "привет, мир!") обычно представляет собой последовательность, состоящую из некоторого количества символов (которое может быть нулевым) и заключенную в двойные кавычки. Строковый литерал имеет тип `const char*`. Компилятор вставляет в конец каждого строкового литерала нуль-символ.

Два строковых литерала, разделенных только пробелом (или символом, подобным пробелу), автоматически конкатенируются, образуя строковый литерал большего размера. Это позволяет более удобно записывать строковые литералы, которые не помещаются на одной строке.

A.2.1.4. Представления символов

Выше было отмечено, что символьный литерал обычно представляет собой единственный символ, заключенный в одинарные кавычки, а строковый литерал — последовательность, заключенную в двойные кавычки. Здесь нельзя обойтись без слова "обычно", поскольку это общее правило имеет ряд исключений, которые применяются в равной степени как к символьным, так и строковым литералам.

- Для представления кавычки (двойной или одинарной), которая является признаком начала литерала, необходимо предварить ее символом "обратная косая черта" (например, `'\''` или `"the \"quotes\""`), чтобы было ясно, что данная кавычка не завершает литерал. Для удобства можно использовать в этом случае кавычки другого вида, например `'\"'`.
- Для представления обратной косой черты необходимо предварить ее другой обратной косой чертой (например, `'\\'`), чтобы компилятор не "подумал", что она придает некоторое специальное значение следующему за ней символу.
- Существуют правила, касающиеся международных наборов символов, которые выходят за рамки этой книги, но могут оказаться существенно важными для программ, в тексте которых имеется два или больше последовательных вопросительных знаков. Во избежание нескольких последовательных вопросительных знаков, C++ позволяет для представления знака вопроса использовать сочетание символов `\?`, чтобы можно было писать такие литералы, как `"что?\?"`, избегая употребления последовательных вопросительных знаков.
- Ряд управляющих символов, которые оказывают влияние на способ вывода данных, имеет печатаемое представление внутри литералов: переход на новую строку (`\n`), горизонтальная табуляция (`\t`), вертикальная табуляция (`\v`), возврат на одну позицию (`\b`), возврат каретки в исходное положение (`\r`), подача страницы (`\f`) и предупреждение (`\a`). Реальное влияние этих управляющих символов при выводе данных на выходное устройство зависит от конкретной C++-среды.
- Если вам действительно нужно использовать символ в специальном внутреннем представлении, можно выразить его в виде обозначения `\x` с последующими шестнадцатеричными цифрами (строчными или прописными) или в виде обозначения

\ с последующими восьмеричными цифрами (не больше трех). Так, например, оба обозначения '\x20' и '\40' представляют символ, внутреннее представление которого выражается десятичным числом 32 (20 — в шестнадцатеричной системе счисления и 40 — в восьмеричной). В C++-средах, основанных на символьных наборах ASCII, числу 32 соответствует пробел (' '). Популярность использования этого представления состоит в том, что '\0' определяет символ, значение которого равно нулю.

A.2.2. Тип значений с плавающей точкой

В C++ используются три типа значений с плавающей точкой: `float`, `double` и `long double` (перечислены в порядке неубывающей точности). C++-среда может реализовать тип `float` с такой же точностью, как и тип `double`, или тип `double` с такой же точностью, как и тип `long double`. Каждая C++-среда обязана предоставить по крайней мере шесть значащих (десятичных) цифр в значениях типа `float` и десять — в значениях типа `double` и `long double`. Большинство C++-сред предлагает только шесть значащих цифр для значений типа `float` и пятнадцать — для значений типа `double`.

Литералы в форме с плавающей точкой. Литерал в форме с плавающей точкой представляет собой непустую последовательность цифр, возможно, с имеющимся показателем степени числа (в конце) или десятичной точкой (в любом месте) либо и тем и другим одновременно. Подобно целочисленным литералам, литералы в форме с плавающей точкой не могут начинаться со знака (-3.1 — это выражение, а не литерал). Десятичная точка может стоять в начале, середине или в конце последовательности цифр; она может быть даже опущена (при наличии показателя степени). Показатель степени обозначается символом `e` или `E`, за которым следует необязательный знак и одна или несколько цифр. Показатель степени всегда интерпретируется в десятичной системе счисления.

Например, обозначения `312E5` и `31200000.` представляют одно и то же число, но `31200000` — целочисленный литерал, а не литерал в форме с плавающей точкой. Еще пример: запись `1.2E-3` представляет то же число, что и `.0012`, или, что то же самое, `0.000012e+2`.

Литералы в форме с плавающей точкой обычно имеют тип `double`. Если вам нужно, чтобы литерал имел тип `float`, припишите к концу последовательности символ `f` или `F`; а чтобы литерал имел тип `long double`, припишите к концу символ `l` или `L`.

A.2.3. Константные выражения

Константное выражение — это выражение целого типа, значение которого известно во время компиляции. Операнды в константном выражении могут содержать только литералы, нумераторы (перечислители), `const`-переменные или `static`-члены данных целого типа, которые инициализируются константными выражениями или `sizeof`-выражениями. Такие выражения не могут включать функций, объектов классов, указателей или ссылок, и им не разрешается использовать операторы присваивания, инкремента и декремента, а также вызовы функций или оператор "запятая".

Константное выражение может стоять везде, где ожидается наличие константы. В качестве примера можно привести указание размерности в объявлениях массивов (см. раздел 10.1.3), метки в `switch`-операторах (см. раздел A.4) и инициализаторы для перечислителей (см. раздел A.2.5).

А.2.4. Преобразования

Преобразования необходимы для приведения операндов каждого оператора к общему типу. Если есть выбор, то предпочтение отдается тем преобразованиям, которые сохраняют информацию, а не тем, которые ее теряют. Более предпочтительны преобразования в значения типов без знака, а не в значения типов со знаком. Более того, все арифметические операции над целочисленными значениями `short`-типа или символами подразумевают преобразования в значения типа `int` или более “длинные”, а арифметические операции с плавающей точкой подразумевают преобразования в значения типа `double` или более “длинные”.

Самыми простыми преобразованиями являются продвижения по “типовой лестнице”. Продвижения позволяют значениям “меньшего” типа (например, типа `char`) получить “большой”, но “родственный” тип (например, тип `int`); они сохраняют знак исходного значения. При продвижениях целых типов выполняется преобразование значений типа `char`, `signed char`, `unsigned char`, `short` и `unsigned short` в значения типа `int`, если эти значения уместаются в `int`-области памяти, и в значения `unsigned int` — в противном случае. “Широкие” (двухбайтовые) символы и перечислимые типы (см. раздел А.2.5) преобразуются в значения самого маленького `int`-типа, который способен представить все значения данного типа. Для этого по порядку апробируются типы `int`, `unsigned int`, `long` и `unsigned long`. Значения типа `bool` преобразуются в значения `int`, а типа `float` — в значения типа `double`.

При преобразовании значений целого типа в значения типа с плавающей точкой сохраняется максимальная точность, которая может быть определена оборудованием компьютера.

Преобразование большего значения со знаком (например, типа `long`) в меньшее (например, типа `short`) определяется C++-средой. Преобразование большего значения без знака в меньшее выполняется по модулю 2^n , где n — количество разрядов в представлении более короткого типа. При преобразовании значения с плавающей точкой в значение целого типа происходит усечение посредством отбрасывания дробной части. Результат преобразования не определен, если усеченное значение не уместится в предназначенной для него области памяти.

Указатели, значения целых типов и типов с плавающей точкой можно преобразовать в значения типа `bool`. Если преобразуемое значение равно 0, то `bool`-результат будет равен значению `false`, в противном случае — значению `true`. Значения типа `bool` также могут быть преобразованы в значения других типов: `true` преобразуется в значение 1, а `false` — в значение 0.

Константное выражение (см. раздел А.2.3), которое равно 0, может быть преобразовано в указатель.

Любой указатель может быть преобразован в значение типа `void*`. Кроме того, указатель на `non-const`-значение может быть преобразован в указатель на `const`-значение (то же справедливо и для `non-const`-ссылки). Указатель или ссылка на объект типа, который был открыто выведен из некоторого класса, могут быть преобразованы в указатель или ссылку на любой из его базовых классов.

Арифметические преобразования. Поскольку операнды могут быть целыми или с плавающей точкой, со знаком или без знака, иногда сложно определить тип результата арифметической операции. Поэтому предусмотрены некоторые правила, именуемые *обычными арифметическими преобразованиями*, которые заключаются в следующем.

- Если хотя бы один операнд имеет тип значения с плавающей точкой, результат будет иметь тип значения с плавающей точкой с точностью самого точного операнда.
- Если хотя бы один операнд имеет тип `unsigned long`, то и результат будет иметь этот тип.
- Если один операнд имеет тип `long int`, а другой — `unsigned`-тип (т.е. любой тип без знака, но не тип `unsigned long`, который, согласно предыдущему правилу, заставил бы результат иметь тип `unsigned long`), то результат будет зависеть от конкретной C++-среды: если диапазон значений типа `long int` включает диапазон значений типа `unsigned int`, то результат будет иметь тип `long int`; в противном случае — тип `unsigned int`.
- Если хотя бы один операнд имеет тип `long int`, то результат будет иметь тип `long int`.
- В противном случае операнды должны быть целочисленными значениями со знаком (типа `int` или более короткого), а результат будет иметь тип `int`.

Одно следствие из этого правила — результат любой арифметической операции никогда не будет иметь тип `short` или `char` (со знаком или без). Все арифметические операции выполняются только над `int`-значениями или значениями более «широких» типов.

А.2.5. Перечислимые типы

Перечислимый тип задает набор целых значений. Объекты этого типа могут принимать значения, заданные только этим типом.

```
enum имя_перечисления{
    перечислитель [ , перечислитель ] ...
};
```

Здесь элемент *имя_перечисления* представляет собой имя типа, которое можно использовать везде, где ожидается имя типа.

Переменные типа *имя_перечисления* могут иметь значения только из списка элементов *Перечислитель*.

перечислитель: идентификатор [= константное_выражение]

Если не задано иначе, значения перечислимых типов соответствуют последовательным целым числам, начиная с нуля. Можно также для перечислителей указать явные значения. Инициализаторы должны иметь целый тип (см. раздел А.2.1), а их значения компилятор должен иметь возможность определить во время компиляции (см. раздел А.2.3). Если значение перечислимого типа используется в контексте, требующем целочисленный тип, оно будет преобразовано автоматически.

А.2.6. Перегрузка

Несколько функций могут иметь одинаковые имена, но при этом функции-тезки должны отличаться типом или количеством параметров.

При вызове перегруженной функции предполагается, что во время компиляции выполняется проверка на возможность определения, какая именно из перегруженных функций должна быть вызвана. Вызываемая функция определяется посредством сравнения реальных аргументов вызова с типами формальных параметров. Выбирается функция с *наилучшим соответствием* реальным аргументам. Под наилучшим соответствием подразумевается, что функция должна обнаружить лучшее (по сравнению с другими) совпадение по одному или нескольким аргументам и не иметь ни одного худшего (по сравнению с другими) совпадения ни по одному из них.

Наилучшее совпадение для любого аргумента определяется следующим образом.

- Наилучшим является точное совпадение (типы реальных аргументов функции идентичны формальным параметрам).
- Соответствие с использованием продвижения по “типовой лестнице” (см. раздел А.2.4) считается лучшим вариантом, чем соответствие с использованием встроенных преобразований, которое, в свою очередь, лучше соответствия с использованием преобразований, определенных типом класса (см. разделы 12.2 и 12.5).

Ситуация, когда несколько функций имеют наилучшее совпадение аргументов с параметрами при вызове, является ошибочной.

А.3. Выражения

C++ унаследовал от языка C богатый синтаксис выражений, к которому была добавлена перегрузка операторов (см. раздел А.3.1). Перегрузка операторов позволяет программистам определять типы аргументов и возвращаемых значений, а также значение самих операторов, но не их приоритет, валентность (количество операндов) или ассоциативность и не действие встроенных операторов над операндами встроенных типов. В этом разделе описываются операторы применительно к операндам встроенных типов.

Каждый оператор генерирует некоторое значение, тип которого зависит от типов его операндов. В общем случае, если операнды имеют одинаковый тип, то и результат будет иметь тот же тип. В противном случае выполняются стандартные преобразования, позволяющие привести все операнды к общему типу (см. раздел А.2.4).

Значение, которое обозначено некоторым сохраняемым программой объектом (следовательно, имеющим конкретный адрес), называется *l*-значением. Определенные операции действительны только для *l*-значений, а некоторые операции генерируют *l*-значения. Каждое выражение генерирует некоторое значение, а некоторые выражения — *l*-значения.

Только четыре оператора гарантируют порядок вычисления для своих операндов.

- && Правый операнд вычисляется только в случае, если левый операнд имеет значение `true`
- || Правый операнд вычисляется, если левый операнд имеет значение `false`
- ? : Вычисляется только одно выражение после условия. Выражение после символа “?” вычисляется в случае, если условие оказывается истинным; в противном случае вычисляется выражение, стоящее после символа “:”. Результат равен результату вычисленного выражения; он является *l*-значением, если оба выражения — *l*-значения одинакового типа
- , Левый операнд вычисляется первым и отбрасывается; результатом является правый операнд

Для других операторов, за исключением правил предшествования, порядок вычисления не гарантируется. Другими словами, компилятору разрешается вычислять операнды в любом порядке. Для переопределения приоритетов, установленных по умолчанию, можно использовать круглые скобки, но для полного управления порядком вычислений требуются временные объекты.

Каждый оператор имеет определенный приоритет и свойство ассоциативности. В следующей таблице сведены все операторы в порядке убывания приоритета. Операторы разбиты на группы. Операторы, принадлежащие одной группе, имеют общий приоритет и ассоциативность. Каждое группирование вводит новый уровень предшествования. Эта таблица включает все операторы и является расширением таблицы, впервые приведенной в главе 2.

Обозначение **Идентификатор или литеральная константа (идентификаторы являются l-значениями, а литералы — нет)**

<code>C::m</code>	Член <i>m</i> класса <i>C</i>
<code>N::m</code>	Член <i>m</i> из пространства имен <i>N</i>
<code>::m</code>	Имя <i>m</i> в глобальной области видимости
<code>x[y]</code>	Элемент с индексом <i>y</i> в объекте <i>x</i> . Генерирует <i>l</i> -значение
<code>x->y</code>	Член <i>y</i> объекта, на который указывает <i>x</i> . Генерирует <i>l</i> -значение
<code>x.y</code>	Член <i>y</i> объекта <i>x</i> . Генерирует <i>l</i> -значение, если <i>x</i> является <i>l</i> -значением
<code>f(Аргументы)</code>	Вызов функции <i>f</i> с передачей элемента <i>Аргументы</i> в качестве аргумента (аргументов)
<code>x++</code>	Инкрементирует <i>l</i> -значение <i>x</i> . Возвращает исходное значение <i>x</i>
<code>x--</code>	Декрементирует <i>l</i> -значение <i>x</i> . Возвращает исходное значение <i>x</i>
<code>*x</code>	Выполняет разыменование указателя <i>x</i> . Генерирует объект как <i>l</i> -значение
<code>&x</code>	Адрес объекта <i>x</i> . Генерирует указатель
<code>~x</code>	Унарный минус. Может быть применен к выражениям только числового типа
<code>!x</code>	Логическое отрицание. Если <i>x</i> равен нулю, то <code>!x</code> равен значению <code>true</code> , в противном случае — значению <code>false</code>
<code>~x</code>	Поразрядное дополнение значения <i>x</i> до единицы. Значение <i>x</i> должно иметь целый тип
<code>++x</code>	Инкрементирует <i>l</i> -значение <i>x</i> . Возвращает инкрементированное значение как <i>l</i> -значение
<code>--x</code>	Декрементирует <i>l</i> -значение <i>x</i> . Возвращает декрементированное значение как <i>l</i> -значение
<code>sizeof(e)</code>	Количество байтов (как значение типа <code>size_t</code>), занимаемое выражением <i>e</i>
<code>sizeof(T)</code>	Количество байтов (как значение типа <code>size_t</code>), занимаемое объектом <i>T</i>
<code>T(Аргументы)</code>	Создает объект типа <i>T</i> из элемента <i>Аргументы</i>
<code>new T</code>	Размещает в памяти новый объект типа <i>T</i> , инициализируемый по умолчанию
<code>new T(Аргументы)</code>	Размещает в памяти новый объект типа <i>T</i> , инициализируемый элементом <i>Аргументы</i>
<code>new T[n]</code>	Размещает в памяти массив, содержащий <i>n</i> объектов типа <i>T</i> , инициализируемых по умолчанию
<code>delete p</code>	Освобождает объект, на который указывает <i>p</i>
<code>delete [] p</code>	Освобождает массив объектов, на который указывает <i>p</i>
<code>x * y</code>	Произведение <i>x</i> и <i>y</i>
<code>x / y</code>	Частное от деления <i>x</i> на <i>y</i> . Если оба операнда целочисленные, C++-среда выбирает, в какую сторону округлять: до нуля или до $-\infty$
<code>x % y</code>	Остаток от деления <i>x</i> на <i>y</i> , эквивалент значению выражения $x - ((x / y) * y)$

$x + y$	Сумма x и y , если оба операнда — числа. Если один операнд — указатель, а другой — целое значение, генерирует указатель на позицию, удаленную на y элементов после x
$x - y$	Результат вычитания y из x , если оба операнда — числа. Если x и y — указатели, генерирует значение расстояния (в элементах) между ними
$x \gg y$	Для целых значений x и y значение x сдвигается вправо на y разрядов; значение y при этом должно быть неотрицательным. Если x имеет тип <i>istream</i> , выполняется чтение из x в y и возвращается <i>l</i> -значение x
$x \ll y$	Для целых значений x и y значение x сдвигается влево на y разрядов; значение y при этом должно быть неотрицательным. Если x имеет тип <i>ostream</i> , выполняется запись y в x и возвращается <i>l</i> -значение x
$x \text{ Op } y$	Операторы отношений (<i>Op</i>) генерируют значение типа <i>bool</i> , означающее истинность данного отношения. Операторы отношений ($<$, $>$, $<=$ и $>=$) имеют свои очевидные значения. Если x и y — указатели, они должны указывать на один и тот же объект или массив
$x == y$	Генерирует значение типа <i>bool</i> , означающее, равно ли значение x значению y
$x != y$	Генерирует значение типа <i>bool</i> , означающее, не равно ли значение x значению y
$x \& y$	Поразрядное логическое умножение (“И”). Значения x и y должны быть целого типа
$x \wedge y$	Поразрядное исключающее “ИЛИ”. Значения x и y должны быть целого типа
$x y$	Поразрядное логическое сложение (“ИЛИ”). Значения x и y должны быть целого типа
$x \&\& y$	Генерирует значение типа <i>bool</i> , означающее, истинны ли значения обеих переменных x и y . Вычисляет y только в том случае, если x имеет значение <i>true</i>
$x y$	Генерирует значение типа <i>bool</i> , означающее, истинно ли значение хотя бы одной из переменных x и y . Вычисляет y только в том случае, если x имеет значение <i>false</i>
$x = y$	Присваивает переменной x значение переменной y , генерируя <i>l</i> -значение x в качестве результата
$x \text{ op} = x$	Составные операторы присваивания; эквивалент инструкции $x = x \text{ op } y$, где <i>op</i> — арифметический, поразрядный оператор или оператор сдвига
$x ? y1 : y2$	Результатом является значение $y1$, если x равно <i>true</i> ; в противном случае — $y2$. Вычисляется только одно из выражений: $y1$ или $y2$. Выражения $y1$ или $y2$ должны иметь одинаковый тип. Если $y1$ и $y2$ — <i>l</i> -значения, результат также представляет собой <i>l</i> -значение. Оператор является правоассоциативным

throw x	Сигнализирует об ошибке путем “выбрасывания” значения x. Тип значения x определяет, какой обработчик перехватит эту ошибку
x, y	Вычисляет x, отбрасывает результат, затем вычисляет y. Возвращает значение y

А.3.1. Операторы

Большинство встроенных операторов могут быть перегружены. Операторы throw, разрешения области видимости, “точка” и условный оператор (оператор ? :) не могут быть перегружены, а все остальные могут. Как определить перегруженный оператор — описано в разделе 11.2.4.

Постфиксный оператор инкремента/декремента отличается от его префиксной версии определением фиктивного, неиспользуемого параметра. Другими словами, чтобы перегрузить постфиксные операторы, запишем следующий код.

```
class Number {
public:
    Number operator++(int) { /* Тело функции */ }
    Number operator--(int) { /* Тело функции */ }
};
```

К наиболее часто перегружаемым операторам относятся операторы присваивания и доступа по индексу, операторы сдвига, используемые для операций ввода-вывода с помощью потоковых ostream- и istream-объектов, и операторы, используемые для работы с итераторами (см. раздел Б.2.5).

А.4. Инструкции

Подобно большинству языков программирования, C++ выделяет объявления, выражения и инструкции. В соответствующих контекстах объявления и инструкции могут быть вложены в другие объявления и инструкции, но не в выражения. Каждая инструкция, в конечном счете, находится внутри определения некоторой функции, формируя часть действий, выполняемых после вызова функции.

Если не указан иной способ действий, инструкции, составляющие функцию, выполняются в порядке их следования. Исключениями являются циклы, обращения к функциям, инструкции goto, break и continue, а также инструкции try и throw, связанные с обработкой исключительных ситуаций.

Инструкции записываются в свободном формате. Начало новой строки посреди инструкции не оказывает влияния на значение этой инструкции. Большинство инструкций завершается точкой с запятой, но основным исключением из этого правила является блок (который начинается с символа “{”, а оканчивается символом “}”).

```
;      Пустая инструкция; при выполнении не имеет никакого эффекта
e;     Инструкция-выражение; вычисляет выражение e для получения его побочных эффектов
{ }    Блок инструкций; выполняет инструкции в блоке одну за другой. Переменные, определенные в блоке, разрушаются по окончании блока
if (условие) инструкция1
      Вычисляет элемент условие и выполняет элемент инструкция1, если Условие имеет значение true
if (условие) инструкция1 else инструкция2
```

Вычисляет элемент *Условие* и выполняет элемент *Инструкция1*, если *Условие* имеет значение true; в противном случае выполняет элемент *Инструкция2*. Каждое ключевое слово else связывается с ближайшим “непарным” словом if

while (*Условие*) *Инструкция*

Тестирует элемент *Условие* и выполняет элемент *Инструкция* до тех пор, пока *Условие* сохраняет значение true

do *Инструкция* while (*Условие*);

Выполняет элемент *Инструкция*, а затем тестирует элемент *Условие*. Продолжает выполнять элемент *Инструкция* до тех пор, пока *Условие* не примет значение false

for (*инструкция_инициализации Условие; Выражение*)
инструкция

Выполняет элемент *инструкция_инициализации* один раз при входе в цикл, а затем тестирует элемент *Условие*. Если *Условие* принимает значение true, выполняет элемент *Инструкция*, а затем вычисляет элемент *Выражение*. Продолжает последовательно тестировать *Условие*, выполнять элементы *Инструкция* и *Выражение* до тех пор, пока *Условие* не примет значение false.

Если элемент *инструкция_инициализации* является объявлением, то областью видимости для объявляемой в нем переменной является сама инструкция for

switch (*Выражение*) *инструкция*

На практике элемент *Инструкция* почти всегда представляет собой блок, который включает инструкции с метками следующего вида.

case *Значение*:

Здесь каждый элемент *Значение* должен представлять собой отдельное константное выражение целого типа (см. раздел А.2.3). Кроме того, switch-инструкция может включать следующую метку.

default:

Но эту метку нельзя использовать более одного раза.

При выполнении switch-инструкции вычисляется *Выражение* и управление передается той case-метке, значение которой совпадает с результатом вычисления элемента *Выражение*. Если совпадения не обнаружено, управление передается метке default: (если таковая предусмотрена) или инструкции, непосредственно следующей за инструкцией switch.

Поскольку case-метки — всего лишь метки, управление будет передаваться от одной к последующей, если программист не предпримет явных действий для предотвращения этого. Обычно с такой целью используется инструкция break, размещаемая перед каждой case-меткой (но после первой)

break; Передает управление инструкции, непосредственно следующей за окончанием ближайшей инструкции while, for, do или switch, которая включает инструкцию break;

continue;

Передает управление назад к началу следующей итерации (включающей тестирование) в ближайшей инструкции for, while или do, которая включает инструкцию continue;. Если ближайшей включающей инструкцией является for-инструкция, то следующая итерация включает также и элемент *Выражение* этой for-инструкции. Например, следующий for-цикл


```

for (int i = 0; i < 10; ++i) {
    if (i % 3 == 0)
        continue;
    cout << i << endl;
}

```

выводит каждое из значений 1, 2, 4, 5, 7 и 8 на отдельной строке

goto Метка;

Поведение этой инструкции аналогично подобным инструкциям в других языках программирования. Целевым объектом инструкции *goto* является метка, которая представляет собой идентификатор с двоеточием. Имена меток могут совпадать с именами других объектов программы, не вызывая никакой неоднозначности. Область видимости метки — это полная функция, в которой она находится, т.е. возможна передача управления на нее извне блока. Однако такой переход не может игнорировать инициализацию переменной

```

try { инструкции } catch (параметр_1) { инструкции_1 }
    [catch (параметр_2) { инструкции_2 } ]...

```

Выполняет код (представленный элементом *инструкции*), который может сгенерировать (посредством инструкции *throw*) исключение. Это исключение должно быть обработано одной или несколькими последующими инструкциями *catch*.

Инструкция *catch* обрабатывает исключения (генерируемые значения которых должны иметь *подобный* тип, как и тип элемента *Параметр_п*) посредством выполнения кода, представленного элементом *инструкции_п*. Термин “подобный” здесь означает, что генерируемое значение имеет такой же тип, как и параметр, или тип, выведенный из типа этого параметра. Если инструкция *catch* имеет форму *catch (...)*, значит, она обрабатывает любые иные неперехваченные исключения.

Если соответствующая *catch*-инструкция, которая совпала бы по типу сгенерированного исключения, отсутствует, то такое исключение распространяется за пределы функции к ближайшей инструкции *try*. Если соответствующая *try*-инструкция отсутствует, выполнение программы прекращается

throw Выражение;

Прекращает выполнение программы или передает управление *catch*-ветви текущей инструкции *try*. Передает *выражение*, тип которого определяет, в какой именно *catch*-ветви будет обработано это исключение. Если в данный момент соответствующая *try*-инструкция не выполняется, программа прекращается.

Исключения зачастую представляют собой объекты классов и обычно генерируются в одной функции, а обрабатываются в другой

Приложение Б

СТАНДАРТНАЯ БИБЛИОТЕКА (КРАТКИЙ ОБЗОР)

Стандартная библиотека — существенный вклад в стандартизацию языка C++. На протяжении всей книги мы постоянно опирались на библиотеку, которая позволила нам написать лаконичные и эффективные C++-программы. В этом приложении дан обзор библиотечных средств, которые мы использовали, а также описаны некоторые другие весьма полезные средства, которые нам не довелось продемонстрировать в этой книге. В каждом разделе представлен один библиотечный класс или семейство родственных классов и разъясняется, как использовать предоставляемые ими возможности.

В общем случае имена стандартной библиотеки принадлежат пространству имен `std`. Поэтому программы, которые используют средства стандартной библиотеки, должны либо предварять такие имена префиксом `std::`, либо делать их общедоступными с помощью `using`-объявлений. В данном приложении мы не будем явно упоминать о необходимости сделать это. Так, например, мы будем использовать имя `cout`, а не `std::cout`.

В наших примерах предполагается, что следующие обозначения несут описанную ниже смысловую нагрузку.

<code>n</code>	Переменная или выражение, которое генерирует значение любого целого типа
<code>t</code>	Значение типа <code>T</code>
<code>s</code>	Значение типа <code>string</code>
<code>cp</code>	Указатель на начальный элемент символьного массива с завершающим нуль-символом
<code>c</code>	Значение типа <code>char</code>
<code>p</code>	Предикат, который представляет собой функцию, возвращающую <code>bool</code> -значение или значение, приводимое к типу <code>bool</code>
<code>os</code>	Выходной поток
<code>is</code>	Входной поток
<code>strm</code>	Входной или выходной поток
<code>b</code>	Итератор, который обозначает начало последовательности
<code>e</code>	Итератор, который обозначает конец последовательности (точнее, начало смежной, но не принадлежащей ей области памяти)
<code>d</code>	Итератор, который обозначает приемник информации (пункт назначения)
<code>it</code>	Итератор, который обозначает некоторый элемент

Б.1. Ввод-вывод информации

Объекты классов `istream`, `ostream`, `ifstream` и `ofstream` означают последовательные потоки, причем в каждый момент времени объект связывается с одним потоком. Объекты этих типов нельзя копировать или присваивать; следовательно, поток можно передать функции или получить от нее только посредством указателя или ссылки.

Основные положения

`#include <iostream>`

Объявляет классы ввода-вывода и соответствующие операции

`cout`

`cerr`

`clog` Объекты типа `ostream`. Связываются со стандартным потоком вывода данных (`cout`) и потоками ошибок (`cerr`, `clog`). Вывод данных в объекты `cout` и `clog` буферизируется, а вывод в объект `cerr` не буферизируется по умолчанию

`cin` Объект типа `istream`, связанный со стандартным входным потоком

Считывание и запись данных

`is >> t` Считывает значение из объекта входного потока `is` в объект `t`, опустив пробелы. Входные данные должны быть в форме, пригодной для преобразования в значение типа объекта `t`, т.е. должны представлять собой не-const-/значение. Непригодные входные данные формируют запрос на отказ, оставляя объект `is` в неисправном состоянии до тех пор, пока не будет вызвана функция `is.clear()`. Библиотека определяет оператор ввода для встроенных типов и типа `string`

`os << t` Посылает значение объекта `t` выходному объекту `os` в формате, соответствующем типу объекта `t`. Библиотека определяет оператор вывода для встроенных типов и типа `string`

`is.get(c)` Считывает следующий символ (даже если он оказывается пробелом или символом, подобным пробелу) из потокового объекта `is` в объект `c`

`is.unget()` Восстанавливает предшествующее состояние потока `is`, возвращаясь на один символ. Эту функцию полезно использовать, когда нужно считывать данные до тех пор, пока не встретится конкретный символ, причем этот символ должен остаться в потоке для последующей обработки. Гарантируется запоминание только одного символа для восстановления потока

Итераторы

`#include <iterator>`

Объявляет итераторы входных и выходных потоков

`istream_iterator<T> in(is);`

Определяет объект `in` как итератор входного потока для считывания значений типа `T` из потокового объекта `is`

`ostream_iterator<T> out(os, const char* sep = "");`

Определяет объект `out` как итератор выходного потока для записи значений типа `T` в поток `os`, используя `sep` в качестве значения разделителя после каждого выводимого элемента. По умолчанию разделитель представляет собой пустую строку, но это может быть и строковый литерал (см. раздел 10.2) или указатель на символьный массив с завершающим нулевым символом

Файловые потоки

`#include <fstream>`

Объявляет потоковые средства ввода-вывода информации, связанные с файлами

`ifstream is(cp);`

Определяет объект `is` и связывает его с файлом, заданным параметром `cp` (способ задания зависит от конкретной C++-среды). Класс `ifstream` выведен из класса `istream`

`ofstream os(cp);`

Определяет объект `os` и связывает его с файлом, заданным параметром `cp`. Класс `ofstream` выведен из класса `ostream`

Управление форматом вывода информации

`#include <ios>`

Определяет тип `streamsize`, который представляет собой целый тип со знаком, пригодный для представления размеров буферов ввода-вывода

`os.width()`

`os.width(n)`

Возвращает значение ширины (типа `streamsize`), ранее связанное с потоковым объектом `os`. Устанавливает ширину потока равной значению `n`, если таковое задано. Следующий элемент, выводимый в этот поток, будет дополняться пробелами слева до текущей ширины потока, после чего ширина будет установлена равной 0

`os.precision()`

`os.precision(n)`

Возвращает значение точности (типа `streamsize`), ранее связанное с потоковым объектом `os`. Устанавливает точность потока равной значению `n`, если таковое задано. Последующие значения с плавающей точкой, выводимые в этот поток, будут отображаться с заданным количеством значащих цифр

Манипуляторы

`#include <iomanip>`

Объявляет манипуляторы, отличные от манипулятора `endl`, который объявляется в заголовке `<iostream>`

`os << endl`

Завершает текущую выходную строку и сбрасывает на диск данные потока, связанного с объектом `os`

`os << flush`

Сбрасывает на диск данные потока, связанного с объектом `os`

`os << setprecision(n)`

`os << setw(n)`

Эквиваленты функций `os.precision(n)` и `os.width(n)` соответственно

Признаки ошибки и конца файла

`strm.bad()`

Возвращает значение типа `bool`, означающее, произошел ли сбой при выполнении объектом `strm` последней операции в результате обработки неверных данных

`strm.clear()`

Делает попытку восстановить потоковый объект `strm`, чтобы его можно было использовать после выполнения некорректной операции. Генерирует исключение `ios::failure`, если объект `strm` нельзя восстановить

`strm.eof()`

Возвращает значение типа `bool`, означающее, обнаружен ли объектом `strm` признак конца файла

`strm.fail()`

Возвращает значение типа `bool`, означающее, произошел ли сбой при выполнении объектом `strm` последней операции в результате аппаратных или других проблем системного уровня

`strm.good()`

Возвращает значение типа `bool`, означающее, была ли успешно выполнена объектом `strm` последняя операция

Б.2. Контейнеры и итераторы

В этой книге рассматриваются последовательные контейнеры `vector` и `list`, ассоциативный контейнер `map` и класс `string`, у которого много общих с контейнерами свойств. Все контейнеры при выполнении конкретной операции используют сходные интерфейсы. Сначала мы рассмотрим общие контейнерные операции, а затем — операции, присущие отдельным контейнерам.

Программистам, которым нужны последовательные контейнеры, следует использовать класс `vector`, если не существует причины поступить иначе. Самой распространенной причиной такого рода является необходимость вставлять (или удалять) множество элементов в произвольное место (а не только в конец) контейнера. В этом случае стоит воспользоваться классом `list`, который поддерживает эту операцию гораздо эффективнее, чем класс `vector`.

Б.2.1. Общие контейнерные операции

Все контейнеры и класс `string` предлагают использовать следующий интерфейс.

Контейнер`<T>::iterator`

Контейнер`<T>::const_iterator`

Типы итераторов, связанных с элементом *Контейнер*`<T>`. Объекты любого из этих типов можно использовать для чтения значений элементов контейнера; объекты типа *Контейнер*`<T>::iterator` можно также применять для модификации элементов контейнера

Контейнер`<T>::reference`

Контейнер`<T>::const_reference`

Синонимы для `T&` и `const T&` соответственно

Контейнер`<T>::reverse_iterator`

Контейнер`<T>::const_reverse_iterator`

Типы итераторов, которые получают доступ к элементам контейнера в обратном порядке

<i>Контейнер</i> <T>::size_type	Целый тип без знака, позволяющий содержать размер самого большого контейнера
<i>Контейнер</i> <T>::value_type	Тип элементов контейнера (синоним для T)
c.begin() c.end()	Итераторы, которые указывают на первый элемент, если таковой имеется, и на следующий за последним элементом контейнера соответственно. Обе эти функции генерируют значения типа const_iterator или iterator, в зависимости от того, является ли контейнер с const-объектом
c.rbegin() c.rend()	Итераторы (типа const_reverse_iterator или reverse_iterator, в зависимости от того, является ли контейнер с const-объектом), которые получают доступ к элементам контейнера с в обратном порядке
<i>Контейнер</i> <T> c;	Определяет c как пустой контейнер, для которого c.size() == 0
<i>Контейнер</i> <T> c2(c);	Определяет c2 как контейнер, для которого c2.size() == c.size(). Каждый элемент контейнера c2 является копией соответствующего элемента контейнера c
c = c2	Заменяет элементы контейнера c копиями элементов контейнера c2. Возвращает c как l-значение
c.size()	Количество элементов в контейнере c
c.empty()	Возвращает значение true, если контейнер c пуст, в противном случае — значение false
c.clear()	Очищает контейнер c. Эта операция эквивалентна функции c.erase(c.begin(), c.end()). После выполнения этой операции c.size() == 0. Возвращает void-значение

Б.2.2. Последовательные контейнеры

В дополнение к общим контейнерным операциям, класс string и последовательные контейнеры (vector и list) поддерживают также следующие операции.

<i>Контейнер</i> <T> c(n, t);	Определяет контейнер c, содержащий n элементов, каждый из которых является копией объекта t
<i>Контейнер</i> <T> c(b, e);	Определяет контейнер c и инициализирует его копией элементов, содержащихся в последовательности, обозначенной входными итераторами b и e
c.insert(it, t)	
c.insert(it, n, t)	
c.insert(it, b, e)	

Вставляет элементы в контейнер *c* непосредственно перед позицией, обозначенной итератором *it*. Если *c* — контейнер типа *vector* или *string*, эта операция делает недействительными все итераторы, которые указывают на заданную позицию вставки или позиции, расположенные за ней, и может привести к перевыделению памяти, в результате чего все итераторы в контейнере *c* станут недействительными. Обратите внимание на то, что для *vector*- и *string*-объектов эта операция может выполняться достаточно медленно, если место вставки находится далеко от конца контейнера.

Первая форма этой операции предназначена для вставки копии объекта *t* и возвращает итератор, который указывает на только что вставленный элемент. Вторая форма позволяет вставить *n* копий объекта *t* и возвращает *void*-значение. С помощью третьей формы этой операции, которая также возвращает *void*-значение, можно вставить копии элементов последовательности, обозначенной входными итераторами *b* и *e*. Итераторы *b* и *e* не должны быть элементами контейнера *c*

c.erase(it)
c.erase(b, e)

Удаляет из контейнера *c* элемент, обозначенный итератором *it*, или элементы в диапазоне [*b*, *e*), делая недействительными все итераторы, указывающие на удаленные элементы. Если *c* — *vector*- или *string*-объект, то все итераторы, которые указывают на элементы, расположенные за удаленным (удаленными), также становятся недействительными. Возвращает итератор, который указывает на позицию, непосредственно примыкающую к области удаления. Обратите внимание на то, что операция *erase* для *vector*- или *string*-объекта может выполняться достаточно медленно, если место удаления находится далеко от конца контейнера

c.assign(b, e)

Заменяет элементы контейнера *c* элементами последовательности, обозначенной входными итераторами *b* и *e*

c.front()

Возвращает ссылку на первый элемент контейнера *c*. Результат не определен, если контейнер *c* пуст

c.back()

Возвращает ссылку на последний элемент контейнера *c*. Результат не определен, если контейнер *c* пуст

c.push_back(t)

Присоединяет к концу контейнера *c* копию объекта *t*, увеличивая размер контейнера *c* на единицу. Возвращает *void*-значение

c.pop_back()

Удаляет последний элемент из контейнера *c*. Возвращает *void*-значение. Результат не определен, если контейнер *c* пуст

inserter(c, it)

Возвращает выходной итератор, который вставляет значения в контейнер *c*, начиная с позиции, непосредственно следующей за позицией, обозначенной итератором *it*. Объявление содержится в заголовке `<iterator>`

back_inserter(c)

Возвращает выходной итератор, который позволяет присоединять новые значения к концу контейнера *c* посредством вызова функции *c.push_back*. Объявление содержится в заголовке `<iterator>`

Б.2.3. Дополнительные последовательные операции

Некоторые операции поддерживаются только теми контейнерами, для которых они могут выполняться эффективно.

c[n] Ссылка на *n*-й элемент контейнера *c*, в котором начальный элемент расположен в позиции 0. Эта ссылка является константной, если объект *c* — *const*-объект, и неконстантной в противном случае. Ссылка не определена, если *n* — вне диапазона. Действительна только для векторов и *string*-объектов

c.push_front(t) Вставляет копию объекта *t* в начало контейнера *c*, увеличивая размер контейнера *c* на единицу. Возвращает *void*-значение. Операция не действительна для векторов и *string*-объектов

c.pop_front() Удаляет первый элемент из контейнера *c*. Возвращает *void*-значение. Результат не определен, если контейнер *c* пуст. Операция действительна только для списков

front_inserter(c) Возвращает выходной итератор, который позволяет вставлять новые значения в начало контейнера *c* посредством вызова функции *c.push_front*. Объявление содержится в заголовке *<iterator>*

Б.2.4. Ассоциативные контейнеры

Ассоциативные контейнеры оптимизированы для быстрого доступа на основе ключа. В дополнение к общим контейнерным операциям, перечисленным в разделе Б.2.1, ассоциативные контейнеры также поддерживают следующие операции.

Контейнер<T>::key_type

Тип ключа контейнера. Ассоциативный контейнер с ключами типа *K* и элементами типа *V* содержит значение типа *value_type* (*a* не *V*), которое принадлежит объекту класса *pair<const K, V>*

Контейнер<T> c(cmp);

Определяет контейнер *c* как пустой ассоциативный контейнер, который для упорядочения элементов использует предикат *cmp*

Контейнер c(b, e, cmp);

Определяет контейнер *c* как ассоциативный контейнер (инициализированный копией значений последовательности, обозначенной входными итераторами *b* и *e*), который для упорядочения элементов использует предикат *cmp*

c.insert(b, e)

Вставляет элементы в контейнер *c* из последовательности, обозначенной входными итераторами *b* и *e*. Контейнер *map* вставляет только те элементы, ключи которых еще отсутствуют в контейнере *c*

c.erase(it)

Удаляет из контейнера *c* элемент, обозначенный итератором *it*. Возвращает *void*-значение

c.erase(b, e)

Удаляет элементы из диапазона *[b, e)* контейнера *c*. Возвращает *void*-значение

`c.erase(k)`

Удаляет из контейнера все элементы с ключом `k`. Возвращает количество удаленных элементов

`c.find(k)`

Возвращает итератор, который указывает на элемент с ключом, равным значению `k`. Если такой элемент не существует, возвращает значение `c.end()`

Б.2.5. Итераторы

Стандартная библиотека, опираясь на итераторы, делает свои алгоритмы независимыми от структур данных. Итераторы представляют собой некоторую абстракцию указателей, обеспечивая операции, которые предоставляют доступ к элементам контейнера аналогично тому, как указатели предоставляют доступ к элементам массива.

Стандартные алгоритмы написаны в предположении, что итераторы отвечают требованиям, которые библиотека разбивает на так называемые *итераторные категории*. Каждый библиотечный алгоритм, использующий итераторы определенной категории, может работать с каждым библиотечным или определенным пользователем классом, который предоставляет итераторы, попадающие в данную категорию.

- **Выходной (output) итератор.** Этот итератор можно использовать для поэлементного перемещения по контейнеру и для однократного вывода каждого элемента, обрабатываемого только один раз. Пример: класс `ostream_iterator` — это выходной итератор, и алгоритм `copy` требует, чтобы его третий аргумент обладал только свойствами выходных итераторов.
- **Входной (input) итератор.** Данный итератор можно использовать для поэлементного перемещения по контейнеру и для многократного (при необходимости) считывания каждого элемента перед переходом к следующему элементу. Пример: класс `istream_iterator` — это входной итератор, и алгоритм `copy` требует, чтобы его первые два аргумента обладали только свойствами входных итераторов.
- **Однонаправленный (forward) итератор.** Этот итератор можно использовать для поэлементного перемещения по контейнеру, для возврата к элементам, на которые указывают ранее сохраненные итераторы, и для считывания либо вывода каждого элемента с нужной частотой. Пример: алгоритм `replace` требует, чтобы первые его два аргумента обладали свойствами однонаправленных итераторов.
- **Двунаправленный (bidirectional) итератор.** Данный итератор можно использовать для поэлементного перемещения по контейнеру либо в прямом, либо в обратном направлении. Пример: контейнеры типа `list` и `map` предоставляют двунаправленные итераторы, а алгоритм `reverse` требует, чтобы его оба аргумента обладали свойствами двунаправленных итераторов.
- **Итератор произвольного доступа (random access).** Этот итератор можно применять для перемещения по контейнеру, используя все операции, поддерживаемые указателями. Пример: классы `vector`, `string` и встроенные массивы поддерживают итераторы произвольного доступа. Алгоритм `sort` требует использования итераторов произвольного доступа.

Все категории итераторов поддерживают тестирование на равенство (неравенство). Итераторы произвольного доступа поддерживают все операции отношений.

Итераторные категории можно рассматривать кумулятивно, в том смысле что каждый однонаправленный итератор одновременно является входным, каждый двунаправленный — однонаправленным, а каждый итератор произвольного доступа — дву-

направленным. Таким образом, любой алгоритм, который принимает аргумент любого итераторного типа, с таким же успехом примет итератор произвольного доступа. Класс `ostream_iterator` и итераторные адаптеры вставки предоставляют выходные итераторы, и, следовательно, они могут применяться лишь в алгоритмах, которые используют операции, поддерживаемые только входными итераторами.

Все итераторы поддерживают следующие операции.

- `++r`
`r++` Перемещает `r` на следующую позицию контейнера. Операция `++r` возвращает `r` как *l*-значение после перемещения, а `r++` возвращает копию предыдущего значения `r`
- `*r` Элемент, на который указывает `r`. Для выходных итераторов операцию `*r` можно использовать только в качестве левого операнда операции присваивания (`=`), и каждое отдельное значение `r` можно применять таким способом только однажды. Для входных итераторов операцию `*r` можно использовать только для считывания данных; а действие инкрементирования `r` делает недействительными все копии, которые могли быть сделаны при использовании предыдущего значения `r`. Для всех других итераторных типов операция `*r` генерирует ссылку на значение, хранимое в элементе контейнера, на который ссылается `r`, и значение `r` остается действительным до тех пор, пока этот элемент существует
- `r == r2` Генерирует значение `true`, если `r` равно `r2`, в противном случае — значение `false`
- `r != r2` Генерирует значение `true`, если `r` не равно `r2`, в противном случае — значение `false`

Все итераторы, отличные от выходных, поддерживают следующую операцию.

- `r->x` Эквивалент операции `(*r).x`
Двунаправленные итераторы и итераторы произвольного доступа также поддерживают операции декремента.

- `--r`
`r--` Перемещает `r` назад к предыдущему элементу. Операция `--r` возвращает `r` как *l*-значение после перемещения, а `r--` возвращает копию предыдущего значения `r`

Итераторы произвольного доступа поддерживают все “указательные” операции, включая перечисленные ниже.

- `r + n` Если `n >= 0`, то результат представляет собой итератор, который указывает на позицию, расположенную через `n` позиций после элемента, на который указывает `r`. Операция не определена, если за элементом, обозначенным итератором `r`, находится меньше `n - 1` элементов. Если `n < 0`, то результат представляет собой итератор, который указывает на позицию, расположенную за `n` позиций до элемента, на который указывает `r`. Операция не определена, если элемент, расположенный на результирующей позиции, не попадает в диапазон контейнера
- `n + r` Эквивалент операции `r + n`
- `r - n` Эквивалент операции `r + (-n)`
- `r2 - r` Операция определена только в случае, если `r` и `r2` указывают на позиции одного и того же контейнера. Если `r2 ≥ r`, результатом является количество элементов в диапазоне `[r, r2)`. В противном случае генерируется отрицание количества элементов в диапазоне `[r2, r)`. Результат имеет тип `ptrdiff_t` (см. раздел 10.1.4)

<code>p[n]</code>	Эквивалент операции <code>*(p + n)</code>
<code>p < p2</code>	Генерирует значение <code>true</code> , если <code>p</code> указывает на позицию, расположенную ближе к началу контейнера, чем позиция, на которую указывает <code>p2</code> . Операция не определена, если <code>p</code> и <code>p2</code> не указывают на позиции в одном и том же контейнере
<code>p <= p2</code>	Эквивалент операции <code>(p < p2) (p == p2)</code>
<code>p > p2</code>	Эквивалент операции <code>p2 < p</code>
<code>p >= p2</code>	Эквивалент операции <code>p2 <= p</code>

Б.2.6. Класс *vector*

Класс *vector* предоставляет динамически размещаемые в памяти массивы и поддерживает итераторы произвольного доступа. В дополнение к операциям, общим для всех контейнеров с последовательным доступом (см. разделы Б.2.1 и Б.2.2), класс *vector* также поддерживает следующие операции.

```
#include <vector>
```

Объявляет класс *vector* и соответствующие операции

```
v.reserve(n)
```

Размещает в памяти объект *v* так, чтобы он мог увеличиться и уместить по крайней мере *n* элементов без последующего перераспределения памяти

```
v.resize(n)
```

Размещает в памяти объект *v* так, чтобы он мог содержать *n* элементов. Делает недействительными все итераторы, ссылающиеся на элементы объекта *v*. Сохраняет первые *n* элементов. Если новый размер меньше старого, “лишние” элементы разрушаются. Если новый размер больше старого, новые элементы инициализируются значением (см. раздел 9.5)

Б.2.7. Класс *list*

Класс *list* предоставляет динамически размещаемые в памяти, независимые от типа и дважды связанные списки, а также поддерживает двунаправленные итераторы (в отличие от класса *vector*, который поддерживает итераторы произвольного доступа). В дополнение к операциям, общим для всех контейнеров с последовательным доступом (см. разделы Б.2.1 и Б.2.2), класс *list* также поддерживает следующие операции.

```
#include <list>
```

Объявляет класс *list* и соответствующие операции

```
l.splice(it, l2)
```

Вставляет все элементы списка *l2* в список *l* сразу перед позицией, обозначенной итератором *it*, и удаляет эти элементы из списка *l2*. Делает недействительными все итераторы и ссылки в списке *l2*. После выполнения операции размер списка *l* (`l.size()`) равен сумме исходных размеров списков *l* и *l2*, а размер списка *l2* становится равным нулю (`l2.size() == 0`). Возвращает `void`-значение

```
l.splice(it, l2, it2)
```

```
l.splice(it, l2, b, e)
```

Вставляет элемент, обозначенный итератором *it2*, или элементы последовательности, заданные диапазоном [*b*, *e*), в список *l* непосредственно перед позицией, обозначенной итератором *it*, и удаляет эти элементы из списка *l2*. Элемент, обозначенный итератором *it2*, или элементы последовательности, заданные диапазоном [*b*, *e*), должны полностью принадлежать списку *l2*. Делает недействительными все итераторы и ссылки на вставленные элементы. Возвращает `void`-значение

`l.remove(t)`
`l.remove_if(p)`
 Удаляет из списка `l` все элементы, значения которых равны значению `t` или для которых истинен предикат `p`. Возвращает `void`-значение

`l.sort(cmp)`
`l.sort()`
 Сортирует список `l`, используя для сравнения элементов оператор “<” или предикат `cmp` (если таковой задан)

Б.2.8. Класс *string*

Класс *string* предоставляет символьные строки переменной длины и итераторы произвольного доступа для доступа к этим символам. Хотя *string*-объекты — не настоящие контейнеры, они поддерживают контейнерные операции, перечисленные выше (см. разделы Б.2.1 и Б.2.2). Кроме того, *string*-объекты можно использовать с алгоритмами (см. раздел Б.3). Класс *string* также поддерживает следующие операции.

`#include <string>`

Объявляет класс *string* и соответствующие операции

`string s(cp);`
 Определяет `s` как *string*-объект, инициализированный копией символов, заданных параметром `cp`

`os << s`
 Выводит символы объекта `s` в поток `os`. Возвращает ссылку на потоковый объект `os`

`is >> s`
 Считывает слово из потока `is` в объект `s`, удаляя предыдущее содержимое объекта `s`. Возвращает ссылку на потоковый объект `is`. Слова отделяются пробелами или символами, им подобными (символ табуляции, новой строки)

`getline(is, s)`
 Считывает данные из входного потока `is` вплоть до следующего символа новой строки (и включая его) и сохраняет считанные символы (исключая символ новой строки) в объекте `s`, удаляя предыдущее содержимое объекта `s`. Возвращает ссылку на потоковый объект `is`

`s += s2`
`s + s2`
`s op s2`
 Присоединяет объект `s2` к объекту `s` и возвращает ссылку на объект `s`
 Возвращает результат конкатенации объектов `s` и `s2`
 Возвращает значение типа `bool`, означающее истинность выполнения заданной операции отношения `op`. Библиотека *string* определяет все операторы отношений: `<`, `<=`, `>`, `>=`, `==` и `!=`. Два *string*-объекта равны, если равны их соответствующие элементы. Если один *string*-объект является префиксом другого, то более короткий считается меньше более длинного. В противном случае результат определяется сравнением первой пары соответствующих символов, в которой строки отличаются одна от другой

`s.substr(n, n2)`
 Возвращает новую строку, которая содержит `n2` символов, скопированных из объекта `s`, начиная с позиции `n`. Результат не определен, если `n > s.size()`. Копирует символы в диапазоне от `n` до конца объекта `s`, если значение выражения `n + n2` больше `s.size()`

`s.c_str()`
 Генерирует `const`-указатель на символьный массив (с завершающим нуль-символом), который содержит копию символов объекта `s`. Массив сохраняется только до тех пор, пока для объекта `s` не будет вызвана очередная не-`const`-функция-член

`s.data()`

Аналогично функции `c_str`, но массив не завершается нуль-символом

`s.copy(cp, n)`

Копирует первые `n` символов (без завершения нулевым символом) из объекта `s` в определенный пользователем символьный массив, обозначенный параметром `cp`. Ответственность за нехватку памяти, по крайней мере для `n` символов, лежит на авторе вызова этой функции

Б.2.9. Класс *pair*

Класс `pair<K, V>` предоставляет абстракцию для пары значений типа `K` и `V` соответственно. Над объектами типа `pair<K, V>` можно выполнять следующие операции.

`#include <utility>`

Объявляет класс `pair` и соответствующие операции

`x.first` Первый элемент `pair`-объекта с именем `x`

`x.second`

Второй элемент `pair`-объекта с именем `x`

`pair<K, V> x(k, v);`

Определяет `x` как новый `pair`-объект, состоящий из элементов, которые имеют типы `K` и `V` и значения `k` и `v`, т.е. член `pair`-объекта `x.first` имеет тип `K`, а член `x.second` — тип `V`. Обратите внимание на то, что для явного объявления `pair`-объекта необходимо знать типы его членов

`make_pair(k, v)`

Генерирует новый объект типа `pair<K, V>` со значениями элементов `k` и `v`. Обратите внимание на то, что для использования этой формы создания объекта необязательно знать типы значений элементов `k` и `v`

Б.2.10. Класс *map*

Класс `map` (отображение) предоставляет динамически размещаемые в памяти, независимые от типа ассоциативные массивы. Для хранения пар значений (имя, значение), которые являются элементами `map`-объекта, используется вспомогательный класс `pair`. Класс `map` поддерживает двунаправленные итераторы. Каждый `map`-объект хранит значения типа `V`, связанные с ключами типа `const K`. Таким образом, значение, хранимое в элементе `map`-объекта, может быть изменено, а ключ — нет. В дополнение к общим контейнерным операциям (см. раздел Б.2.1) и операциям, присущим ассоциативным контейнерам (см. раздел Б.2.4), класс `map` также поддерживает следующие операции.

`#include <map>`

Объявляет класс `map` и соответствующие операции

`map<K, V, P> m(cmp);`

Определяет `m` как новый пустой `map`-объект, который содержит значения типа `V`, связанные с ключами типа `const K`, и использует предикат `cmp` типа `P` для сравнения элементов при вставке их в данное отображение

`m[k]` Генерирует ссылку на значение в объекте `m` в позиции, индексируемой значением `k`. Если такой элемент не существует, то в отображение вставляется элемент типа `V`, инициализируемый значением (см. раздел 9.5). Поскольку вычисление `m[k]` может потенциально изменить содержимое объекта `m`, этот объект не должен быть константным

`m.insert(make_pair(k, v))`

Вставляет значение *v* в объект *m* в позицию, обозначенную ключом *k*. Если уже существует некоторое значение, соответствующее ключу *k*, то оно не изменяется. Возвращает объект типа `pair<map<K, V>::iterator, bool>`, первый компонент которого указывает на элемент с заданным ключом, а второй означает, был ли вставлен новый элемент. Обратите внимание на то, что параметр `make_pair` генерирует объект типа `pair<K, V>`, который преобразуется посредством функции `insert` в объект типа `pair<const K, V>`

`m.find(k)`

Возвращает итератор, который указывает на элемент (если такой существует), связанный с ключом *k*. Если такой элемент не существует, возвращает значение `m.end()`

`*it`

Генерирует объект типа `pair<const K, V>`, который содержит ключ и значение, с которым связан этот ключ, в позиции, обозначенной итератором *it*. Таким образом, элемент `it->first` имеет тип `const K` и представляет ключ, а элемент `it->second` имеет тип `V` и представляет значение, соответствующее этому ключу

Б.3. Алгоритмы

Стандартная библиотека включает множество обобщенных алгоритмов, которые написаны в расчете на использование итераторов; таким образом алгоритмы обретают независимость от конкретных структур данных, с которыми они работают, и типов их членов. Обратите внимание на то, что ассоциативные контейнеры обладают итераторами, которые ссылаются на такие составные типы, как класс `pair<const K, V>`. Следовательно, использование этих алгоритмов с ассоциативными контейнерами требует тщательной разработки.

Большинство алгоритмов ориентировано на последовательности, ограниченные парой итераторов, причем первый итератор указывает на первый элемент, а второй — на область памяти, расположенную за последним элементом этой последовательности. За исключением отдельно оговоренных случаев, все алгоритмы определены в заголовке `<algorithm>`.

`#include <algorithm>`

Включает объявления для общих алгоритмов

`accumulate(b, e, t)`

`accumulate(b, e, t, f)`

Определен в заголовке `<numeric>`. Создает временный объект *obj* такого же типа и с таким же значением, как у объекта *t*. Для каждого входного итератора *it* в диапазоне `[b, e)` вычисляет `obj = obj + *it` или `obj = f(obj, *it)`, в зависимости от формы функции `accumulate`, которая была вызвана. Результат представляет собой копию объекта *obj*. Обратите внимание на то, что поскольку оператор “+” может быть перегруженным, даже первая форма функции `accumulate` может работать с типами, отличными от встроенных арифметических типов. Например, мы можем использовать алгоритм `accumulate` для конкатенации всех строк в контейнере

<code>binary_search(b, e, t)</code>	Возвращает значение типа <code>bool</code> , означающее, принадлежит ли значение <code>t</code> (отсортированной) последовательности, ограниченной однонаправленными итераторами <code>b</code> и <code>e</code>
<code>copy(b, e, d)</code>	Копирует значения последовательности, заданной входными итераторами <code>b</code> и <code>e</code> , в приемную область, заданную выходным итератором <code>d</code> . При выполнении этой функции предполагается, что приемник обладает пространством памяти, достаточным для хранения копируемых значений. Возвращает значение, которое указывает позицию за последним элементом приемника
<code>equal(b, e, b2)</code> <code>equal(b, e, b2, p)</code>	Возвращает значение типа <code>bool</code> , означающее, равны ли элементы последовательности, заданной входными итераторами <code>b</code> и <code>e</code> , элементам последовательности того же размера, начало которой задано входным итератором <code>b2</code> . Для проверки использует предикат <code>p</code> или оператор <code>"=="</code> , если предикат <code>p</code> не задан
<code>fill(b, e, t)</code>	Устанавливает элементы последовательности, заданной входными итераторами <code>b</code> и <code>e</code> , равными значению <code>t</code> . Возвращает <code>void</code> -значение
<code>find(b, e, t)</code> <code>find_if(b, e, p)</code>	Возвращает итератор, обозначающий первое вхождение значения <code>t</code> , или итератор, указывающий на элемент, для которого истинен предикат <code>p</code> (если таковой задан), в последовательности, заданной входными итераторами <code>b</code> и <code>e</code> . Возвращает значение <code>e</code> , если такой элемент не существует
<code>lexicographical_compare(b, e, b2, e2)</code> <code>lexicographical_compare(b, e, b2, e2, p)</code>	Возвращает значение типа <code>bool</code> , означающее, меньше ли последовательность элементов в диапазоне <code>[b, e)</code> последовательности элементов в диапазоне <code>[b2, e2)</code> . Для сравнения элементов используется предикат <code>p</code> или оператор <code>"<"</code> , если предикат <code>p</code> не задан. Если одна последовательность является префиксом другой, то меньшей считается более короткая последовательность. В противном случае результат определяется сравнением первой пары соответствующих элементов, в которой обнаружено различие между последовательностями. Итераторы <code>b</code> , <code>e</code> , <code>b2</code> и <code>e2</code> должны быть только входными
<code>max(t1, t2)</code> <code>min(t1, t2)</code>	Возвращает большее (для функции <code>max</code>) или меньшее (для функции <code>min</code>) из значений, заданных аргументами <code>t1</code> и <code>t2</code> , которые должны иметь одинаковый тип
<code>max_element(b, e)</code> <code>min_element(b, e)</code>	Возвращает итератор, указывающий на наибольший (наименьший) элемент в последовательности, заданной однонаправленными итераторами <code>b</code> и <code>e</code>

`partition(b, e, p)`
`stable_partition(b, e, p)`

Делит последовательность, заданную двунаправленными итераторами `b` и `e`, так, чтобы элементы, для которых предикат `p` был истинен, размещались в начале контейнера. Возвращает итератор, указывающий на первый элемент, для которого этот предикат дает значение `false`, или значение `e`, если этот предикат истинен для всех элементов. Функция `stable_partition` сохраняет исходный порядок элементов в каждой части последовательности

`remove(b, e, t)`
`remove_if(b, e, p)`

Переставляет элементы в последовательности, заданной однонаправленными итераторами `b` и `e`, чтобы элементы, значения которых не совпадают со значением `t` или для которых предикат `p` возвращает значение `false` (если предикат `p` задан), группировались в начале указанной последовательности. Возвращает итератор, который указывает на элемент, стоящий за удаленными элементами

`remove_copy(b, e, d, t)`
`remove_copy_if(b, e, d, p)`

Аналогична функции `remove`, но помещает копию элементов, значения которых не совпадают со значением `t` или для которых предикат `p` возвращает значение `false` (если предикат `p` задан), в приемную область, заданную выходным итератором `d`. Возвращает значение итератора, указывающего на элемент, расположенный за последним элементом приемника. Предполагается, что приемная область достаточно велика, чтобы в ней поместились скопированные значения. Элементы последовательности, заданной итераторами `b` и `e`, не перемещаются. Таким образом, `b` и `e` должны быть только входными итераторами

`replace(b, e, t1, t2)`
`replace_copy(b, e, d, t1, t2)`

Заменяет каждый элемент, значение которого равно `t1`, значением `t2` в последовательности, заданной однонаправленными итераторами `b` и `e`. Возвращает `void`-значение. Вторая форма функции предназначена для копирования элементов с заменой значений `t1` значениями `t2` в последовательности, заданную выходным итератором `d`, и возвращает значение итератора, указывающего на элемент, расположенный за последним элементом приемника. Для `copy`-версии `b` и `e` должны быть только входными итераторами

`reverse(b, e)`
`reverse_copy(b, e, d)`

Первая форма функции располагает в обратном порядке элементы последовательности, заданной двунаправленными итераторами `b` и `e`, переставляя элементы попарно, и возвращает `void`-значение. Вторая форма сохраняет обращенную последовательность в приемной области, начало которой задается выходным итератором `d`, и возвращает значение итератора, указывающего на элемент, расположенный за последним элементом приемника. Приемник должен быть достаточно большим, чтобы в нем поместились значения обращенной последовательности

search(b, e, b2, e2)
search(b, e, b2, e2, p)

Возвращает однонаправленный итератор, указывающий на первое вхождение подпоследовательности, заданной однонаправленными итераторами b2 и e2, в последовательности, заданной однонаправленными итераторами b и e. Для проверки использует предикат p или оператор "==" , если предикат p не задан

transform(b, e, d, f)
transform(b, e, b2, d, f)

Если итератор b2 не задан, функция f должна принимать один аргумент; функция transform вызывает функцию f для элементов последовательности, заданной входными итераторами b и e. Если итератор b2 задан, функция f должна принимать два аргумента, которые принимаются попарно из последовательности, заданной итераторами b и e, и последовательности такой же длины, начало которой задается входным итератором b2. В любом случае функция transform помещает результирующую последовательность в приемную область, заданную выходным итератором d, и возвращает значение итератора, указывающего на элемент, расположенный за последним элементом приемника. Предполагается, что приемник имеет достаточно большой размер, чтобы вместить все сгенерированные элементы. Обратите внимание на то, что итератор d может быть равным итератору b или b2 (если таковой задан), и в этом случае результат заменит исходную последовательность

sort(b, e)
sort(b, e, p)
stable_sort(b, e)
stable_sort(b, e, p)

Сортирует "по месту" последовательность, определенную итераторами произвольного доступа b и e. Для тестирования использует предикат p или оператор "<", если предикат p не задан. Функции stable_sort сохраняют исходный порядок среди равных элементов

unique(b, e)
unique(b, e, p)

Трансформирует последовательность, ограниченную однонаправленными итераторами b и e, таким образом, чтобы первый экземпляр каждой подпоследовательности следующих друг за другом равных элементов перемещался в начало контейнера. Возвращает итератор, указывающий на первый элемент, который не является частью результата (или значение e, если все следующие друг за другом пары исходных элементов состоят из неравных элементов). Для тестирования использует предикат p или оператор "==", если предикат p не задан

unique_copy(b, e, d, p)

Копирует последовательность, ограниченную входными итераторами b и e, в последовательность, начало которой задано выходным итератором d, удаляя по ходу дела любые смежные дубликаты. Возвращает значение d после увеличения его на количество скопированных элементов. Предполагается, что приемная последовательность, заданная итератором d, достаточно велика, чтобы принять копируемые элементы. Для тестирования использует предикат p или оператор "==", если предикат p не задан

Предметный указатель

Символы

!, 117
!=, 42, 53
", 26
#include, 22
%, 69
&&, 49
*, 108
?::, 69
[, 47
, 26
+, 33
++ , 42
<<, 23, 24
<=, 53
==, 46
->, 108

A

Abstract base classes, 322
accumulate, алгоритм, 144, 150, 373
Algorithm, generic, 130
allocate, функция, 244
assign, функция, 366
Assignment operator, 235
Associative array, 154
Associative container, 153
AWK, 154

B

b, 26
back, функция, 366
back_inserter(c), итераторный адаптер, 150
back_inserter, итераторный адаптер, 130
back_inserter, функция, 366
bad, функция, 364
Base class, 272
begin(), функция, 73
begin, функция, 107, 124, 365
binary_search, алгоритм, 181, 374
bool, тип, 42, 56, 350
break, 360

C

c_str(), функция, 268
c_str, функция, 268, 371
case, 359
catch, 360
catch (...), 360
catch, инструкция, 85
cerr, 362
cerr, выходной поток, 217
cerr, поток, 223

char, 35
char, тип, 350
cin, 362
class, 194
Class invariant, 246
clear, функция, 364, 365
clog, 362
clog, выходной поток, 217
clog, поток, 223
Concatenation, 33
const, 33, 36, 345
const, модификатор, 191
const_iterator, тип, 106, 364
const_reference, тип, 364
const_reverse_iterator, тип, 364
construct, функция, 244
Constructor, 198
Container, 67
continue, 360
Copy constructor, 234
copy(p, n), функция, 269
copy, алгоритм, 130, 151, 374
copy, функция, 179, 268, 372
cout, 362

D

data(), функция, 268, 269, 372
deallocate, функция, 244
Declarator, 206
Decrement operator, 138
default, 359
Default argument, 158
Default constructor, 200
Default-initialization, 62
Definition, 30
delete, 220, 224, 268
Dereference operator, 108
destroy, функция, 244
Destructor, 240
do while, инструкция, 168
domain_error, тип исключения, 78
double, тип, 61, 352
do-while, 167, 359
do-while-цикл, 259

E

else, 46
empty, функция, 140, 365
empty, функция-предикат, 125
end(), функция, 73, 107, 124, 365
endl, манипулятор, 363
enum, 354
eof, функция, 364

equal, алгоритм, 374
equal, функция, 133
erase, функция, 104, 125, 366, 367
Exception, 78
Exception object, 78
explicit, 229, 264
Expression, 24
Expression statement, 27
extern, спецификатор, 345

F

fail, функция, 364
false, 42, 350
fill, алгоритм, 374
find, алгоритм, 136, 151, 374
find, функция, 178, 368, 373
find_if, алгоритм, 132, 151, 374
float, тип, 61, 352
flush, манипулятор, 363
for, 359
for, инструкция, 57
Forward declaration, 337
for-заголовок, 50
friend, объявление, 260, 268, 345
front, функция, 366
front_inserter, итераторный адаптер, 150, 365
front_inserter, функция, 367
Function, 22

G

Generic algorithm, 130
Generic function, 171
get, функция, 259, 362
getline, функция, 118, 126, 371
good, функция, 364
goto, 360
grow, функция, 248

H

Handle class, 288
Hash table, 168

I

if, 359
if, инструкция, 46, 57
if-else, инструкция, 57
ifstream, класс, 363
ifstream, тип, 218
Increment, 42
Index, 70
Index operator, 231
Inheritance, 271
Initialization, 30
inline, спецификатор, 345
Inline-функции, 332
insert, функция, 122, 125, 365, 367, 373
insert(c, it), итераторный адаптер, 150, 366
Interface, 30
ios
 failure, тип исключения, 364

iostream, 22, 23
isalnum, функция, 127, 136
isalpha, функция, 127, 138
isdigit, функция, 127
islower, функция, 127
ispunct, функция, 127
isspace, библиотечная функция, 132
isspace, функция, 116, 127
istream, класс, 362
istream_iterator, тип, 184
isupper, функция, 127
Iterator, 106
 bidirectional, 181
 forward, 180
 input, 179
 output, 180
iterator, тип, 106, 364

K

key_type, тип, 367

L

Left-associate, 24
lexicographical_compare, алгоритм, 374
Lisp, 178
Local variable, 30
Logical-and operator, 49
Logical-or operator, 47
long double, тип, 352
long, тип, 56
Lvalue, 80
l-значение, 80, 355

M

main, функция, 22, 223
make_pair, функция, 372
Manipulator, 25
map, 154
max, алгоритм, 374
max, функция, 74, 90
max_element, алгоритм, 374
Member function, 191
min, алгоритм, 374
min, функция, 275
min_element, алгоритм, 374
ML, 178
mutable, спецификатор, 345

N

n, 26
Namespace, 23
new, 220, 224
Null pointer, 206
Null statement, 27
Null string, 31

O

Object, 30
Object-oriented programming, 171

ofstream, класс, 363
 ofstream, тип, 218
 OOP, 171
 operator, 231
 operator void*, 266
 operator{}, 231, 257
 operator+, 261
 operator+==, 261
 operator=, 235
 operator>>, 257
 ostream, класс, 362
 ostream_iterator, тип, 184
 Output operator, 23
 Overloading, 80

P

pair, тип, 156
 pair<K, V>, тип, 169
 partition, алгоритм, 148, 151, 375
 Perl, 154
 Pointer, 206
 Polymorphism, 280
 pop_back, функция, 366
 pop_front, функция, 367
 Postfix, 130
 Precedence, 47
 precision, функция, 63, 74, 363
 Predicate, 89
 private, 194
 protected, 273
 ptrdiff_t, тип, 212
 public, 194
 Pure virtual function, 322
 push_back, функция, 67, 73, 125, 366
 push_front, функция, 367

Q

Qualified name, 25

R

rand, функция, 166
 rbegin, функция, 124, 133, 365
 Reference, 79
 Reference count, 307
 reference, тип, 364
 register, спецификатор, 345
 remove, алгоритм, 151, 375
 remove, функция, 371
 remove_copy, алгоритм, 145, 151, 375
 remove_copy_if, алгоритм, 151
 remove_copy_if, функция, 146
 remove_if, алгоритм, 151, 375
 remove_if, функция, 371
 rend, функция, 124, 365
 replace, алгоритм, 375
 replace, функция, 180
 replace_copy, алгоритм, 375
 reserve, функция, 126, 370
 resize, функция, 126, 370
 return, инструкция, 23, 57

reverse, алгоритм, 375
 reverse, функция, 180
 reverse_copy, алгоритм, 375
 reverse_iterator, тип, 364

S

Scope, 25
 Scope operator, 25
 search, алгоритм, 151, 376
 search, функция, 137
 setprecision, манипулятор, 60, 363
 setprecision, функция, 74
 setw, манипулятор, 363
 setw, функция, 99
 short, тип, 56
 Short-circuit evaluation, 47
 signed char, тип, 350
 size(), функция, 68, 125, 365
 size_t, тип, 56, 210, 349
 size_type, тип, 68, 365
 sizeof, оператор, 215
 Snobol, 154
 sort, алгоритм, 376
 sort, функция, 69, 73, 126, 371
 splice, функция, 370
 split, функция, 185
 stable_partition, алгоритм, 148, 375
 Stable_partition, алгоритм, 151
 stable_sort, алгоритм, 376
 Standard error stream, 217
 static, 215, 220, 297, 331
 static, модификатор, 136, 150, 345
 std, префикс, 23
 std, пространство имен библиотеки, 26, 361
 streamsize, тип, 60, 74
 string, 35
 size_type, тип, 56
 strlen, функция, 213
 struct, 194
 substr, функция, 117, 126, 371
 switch, 359

T

t, 26
 Template class, 226
 Template classes, 67
 Template functions, 172
 Template parameters, 172
 Template specialization, 314
 template, спецификатор, 249
 template<>, 314
 this, 237
 throw, 361
 tolower, функция, 127
 toupper, функция, 127
 transform, алгоритм, 151, 376
 transform, функция, 141
 true, 42, 350
 try, инструкция, 85, 360
 Type, 24
 typedef, 68, 132, 209, 346

typename, 173, 187

U

uncreate, функция, 247
unset, функция, 259, 362
Uniform resource locators, 134
uninitialized_copy, функция, 245
unique, алгоритм, 376
unique_copy, алгоритм, 376
unsigned char, тип, 350
Unsigned type, 45
unsigned, тип, 56
URL-адрес, 134
using-объявление, 50, 57

V

value_type, тип, 230, 365
Variable, 30
virtual, 279, 296
Virtual function, 279
virtual, спецификатор, 346
void*, тип, 266, 353
void, тип, 150
volatile, 345

W

wchar_t, тип, 35, 350
what(), функция, 99
while, инструкция, 41, 57, 359
width, функция, 99, 363

A

Абстрактный базовый класс, 322, 337
Адаптеры, итераторные, 130
Алгоритм
 accumulate, 144, 150
 copy, 130
 copy, 151
 find, 136, 151
 find_if, 132, 151
 partition, 148, 151
 remove, 151
 remove_copy, 145, 151
 remove_copy_if, 151
 remove_if, 151
 search, 151
 stable_partition, 148, 151
 transform, 151
Алгоритм, общий, 130
Алгоритмы, 373
Аргумент по умолчанию, 158
Аргументы функции, 76
Ассоциативная матрица, 154
Ассоциативность, 33
Ассоциативный контейнер, 153, 367

Б

Базовый класс, 272
Базовый язык, 22

Байт, 215
Библиотека
 стандартная, 22
Бит, 215
Блок, 27
Булев тип, 350
Буфер, 31

В

Ввод данных, 29, 36
Вектор, 67
 удаление элементов, 102
Виртуальная функция, 279
 чистая, 322
Виртуальный деструктор, 297
Вывод данных, 27
Выражение, 24
 константное, 352
Выражения, 54, 355

Д

Деструктор, 239
 виртуальный, 287, 297
Диапазон
 полуоткрытый, 50
Динамическое связывание, 279, 296
Директива
 #define, 93
 #endif, 93
 #ifdef, 93
 #include, 22, 92
Доступ
 двунаправленный, 180
 последовательный
 для чтения и записи, 180
 только для записи, 179
 только для чтения, 178
 произвольный, 181
Дружба классов, 297
Друзья, 258

З

Заголовок
 <algorithm>, 69, 90, 129, 150, 275, 373
 <cctype>, 116, 127, 136
 <cstdlib>, 56, 210, 212, 349
 <cstdlib>, 166
 <fstream>, 218, 363
 <iomanip>, 60, 74, 363
 <iostream>, 60, 74, 363
 <iostream>, 26, 362
 <iterator>, 130, 184, 362, 366
 <list>, 111, 370
 <map>, 154, 372
 <memory>, 244
 <numeric>, 144, 373
 <stdexcept>, 78
 <string>, 371
 <utility>, 372
 <vector>, 73, 370

стандартный, 22
шаблона, 173
Заголовочный файл, 92

И

Имя
 составное, 25
Инвариант класса, 246
 цикла, 43, 65
Индекс, 70
Индексирование, 212
Инициализатор конструктора, 200, 203
Инициализация
 string-переменной, 30
 значением, 168
 массивов, 212
 по умолчанию, 62
 объекта, 238
Инкрементирование объекта, 42
Инструкции, 359
Инструкция
 catch, 85
 do while, 168
 do-while, 167
 for, 50
 if, 46
 return, 23, 27
 try, 85
 while, 41
 пустая, 27
Инструкция-выражение, 27, 256
Интерфейс, 30, 225
 класса, 190
Исключение, 78
 типа domain_error, 85
Исключительная ситуация, 78
Итератор, 105, 187, 362, 368
 входной, 179, 368
 выходной, 180, 368
 двунаправленный, 181, 368
 однонаправленный, 180, 368
 произвольного доступа, 368
Итераторные адаптеры, 130
Итерация цикла, 51

К

Класс
 allocator<T>, 244
 ifstream, 363
 list, 364, 370
 map, 372
 ofstream, 363
 pair, 156, 372
 string, 371
 vector, 364, 370
 функция-член erase, 104
 абстрактный базовый, 322
 базовый, 272
 дескриптора, 288
 потомок, 272
 шаблонный, 67, 226, 249

Ключ, 154
Комментарии, 21
Конкатенация, 33
Константные выражения, 352
Конструктор, 198, 202
 explicit-конструктор, 264
 инициализаторы, 200
 копирования, 234
 по умолчанию, 200
 с аргументами, 201
Контейнер, 67, 364
 ассоциативный, 153, 367
 последовательный, 365

Л

Левосторонний оператор, 24
Лексикографический порядок, 89
Литерал, 348
 булев, 350
 в форме с плавающей точкой, 352
 символьный, 34, 350
 строковый, 26, 213, 351
 целочисленный, 349
Логические операторы, 46
Локальная переменная, 73, 306

М

Манипулятор, 25, 363
 setprecision, 60, 63
Массив, 205, 210, 223
 без элементов, 221
 инициализация, 212
 размещение в памяти, 221
Медиана, 65
Метки защиты, 194, 202
Механизм раздельной компиляции, 91
Модификатор
 const, 191

Н

Наследование, 271, 295, 318
Неопределенное значение, 62
Низкоуровневые средства, 205
Ноль-строка, 31

О

Область видимости, 25
Обобщенные функции, 171
Объект, 30
 исключения, 78
Объявление, 343
 using, 50
 опережающее, 337
ООП, объектно-ориентированное
 программирование, 271
Операнды, 24
Оператор
 :, 25
 !=, 42, 53
 %, 69

- &&, 49
- ?:, 69
-], 47
- <<, 24
- <=, 53
- =, 46
- >>, 31
- delete, 220
- new, 220
- взятия адреса, 206
- вывода, 23
- дискремента, 138
- доступа по индексу, 231
- инкремента, 426, 130
- левоассоциативный, 24
- логическое И, 49
- логическое ИЛИ, 47
- неравенства, 42
- остатка, 69
- отрицания, 117
- перегруженный, 33, 231
- преобразования, 265, 268
- присваивания, 235
- присваивания, составной, 49
- проверки равенства, 46
- разрешения области
 - видимости, 25, 44, 192
- разыменования, 108, 206
- сложения, 33
- условный, 69
- Операторная функция, 231
- Операторы, 24, 358
 - логические, 46
 - отношений, 47
 - приоритет, 47
- Операции по умолчанию, 240
- Опережающее объявление, 337
- Описатель, 206, 344
- Определение, 26
 - переменной, 30, 35
 - функций, 76
 - типов, 73
- Отображение, 372

П

- Палиндром, 133
- Параметр-тип, 173
- Перегруженные операторы, 250
- Перегрузка, 80, 354
- Переменная, 30
 - локальная, 30
 - препроцессорная, 93
- Переопределение функций, 296
- Перечислимые типы, 354
- Полиморфизм, 277, 280
- Постфиксный оператор инкремента, 130
- Поток
 - cerr, 217
 - clog, 217
 - ввода-вывода, 223
- Предикат, 89
- Преобразование, 268, 353

- автоматическое, 255
- арифметические, 353
- определенное пользователем, 256
- Препроцессорная переменная, 93
- Префикс
 - std :, 23
- Приоритет, 55
 - операторов, 47
- Присваивание, 235
 - самоприсваивание, 236
- Пространство имен, 23, 26

Р

- Рекурсивный вызов, 165

С

- Свободный формат программы, 26
- Связывание
 - динамическое, 280, 296
 - статическое, 280
- Символьный литерал, 34, 350
- Синтезированные операции, 250
- Скобки
 - угловые, 22
 - фигурные, 22
- Сокращенное вычисление, 47
- Спецификатор, 345
 - класса памяти, 345
 - статического класса памяти static, 136
 - типа, 345
- Список, 111
- Ссылка, 79
- Стандартная библиотека, 22
- Стандартный заголовок, 22
- Стандартный поток
 - ошибок, 217
- Статический член, 297
- Статическое связывание, 280
- Строковый литерал, 26, 351
- Структура программы, 26
- Счетчик ссылок, 307

Т

- Тип, 24, 26
 - bool, 42, 56, 350
 - char, 35, 350
 - const_iterator, 106
 - double, 61
 - float, 61
 - ifstream, 218
 - int, 349
 - istream_iterator, 184
 - iterator, 106
 - list, 111, 126
 - long, 56
 - long int, 349
 - map<K, V>, 169
 - ofstream, 218
 - ostream_iterator, 184
 - pair, 156

pair<K, V>, 169
ptrdiff_t, 212
short, 56
short int, 349
size_t, 56, 210, 349
size_type, 68
streamsize, 60, 363
string, 35, 126
unsigned, 56
unsigned int, 349
unsigned long int, 349
unsigned short int, 349
value_type, 230
vector, 66, 73, 126
void, 150
void*, 266, 353
wchar_t, 35, 350
без знака, 45
булев, 350
значений с плавающей точкой, 352
символьный, 350

Типы, 348
перечислимые, 354
целочисленные, 349
целые, 349

Точки с запятой, 27

У

Угловые скобки, 22
Указатель, 206, 222
на функцию, 208
Управление копированием, 249
Управление памятью, 219, 224
автоматическое, 219
динамическое, 220
статическое, 220
Управляющие символы, 351
Условие, 57
Условный оператор, 69

Ф

Файловые потоки, 363
Фигурные скобки, 22, 27
Функции-члены, 202
Функция
empty, 140
Функция, 22
begin, 107
end, 107
equal, 133
getline, 118
insert, 122
isalnum, 136
isalpha, 138
isspace, 116, 132
main, 22, 27

max, 90
push_back, 67
search, 137
size(), 68
sort, 69
substr, 117
аргументы, 76
доступа, 196
виртуальная, 279
встраиваемая, 332
вызов по значению, 77
определение, 76
подставляемая, 332
список параметров, 76
transform, 141
remove_copy_if, 146
rand, 166
обобщенная, 171
операторная, 231
шаблонная, 172, 186
find, 178
copy, 179
replace, 180
reverse, 180
binary_search, 181
split, 185
strlen, 213
main, 216, 223
push_back, 230
back_inserter, 230
allocate, 244
deallocate, 244
construct, 244
destroy, 244
uninitialized_copy, 245
uninitialized_fill, 245
uninitialized_fill, 245
uncreate, 247
grow, 248
get, 259
unset, 259
min, 275
Функция-член, 191
константная, 193

Х

Хэш-таблица, 168
Хэш-функция, 168

Ш

Шаблонная специализация, 314, 315
Шаблонные
классы, 226, 249
параметры, 172
функции, 172, 186, 268

Научно-популярное издание

Эндрю Кёниг, Барбара Э. Му

Эффективное программирование на C++

Серия C++ *In-Depth*, т. 2

Литературный редактор	<i>Е.Д. Давидян</i>
Верстка	<i>О.В. Линник</i>
Художественный редактор	<i>Е.П. Дынник</i>
Корректоры	<i>Л.А. Гордиенко, Л.В. Коровкина, О.В. Мишутина</i>

Издательский дом "Вильямс".
101509, Москва, ул. Лесная, д. 43, стр. 1.
Изд. лиц. ЛР № 090230 от 23.06.99
Госкомитета РФ по печати.

Подписано в печать 28.10.2002. Формат 70×100/16.
Гарнитура Times. Печать офсетная.
Усл. печ. л. 31,0. Уч.-изд. л. 24,5.
Тираж 3500 экз. Заказ № 1502.

Отпечатано с диапозитивов в ФГУП "Печатный двор"
Министерства РФ по делам печати, телерадиовещания
и средств массовых коммуникаций.
197110, Санкт-Петербург, Чкаловский пр., 15.



Эффективное программирование на C++

Эндрю Кэниг и Барбара Му

Это — превосходная книга вводного характера, предлагающая особый практический подход к решению задач с помощью языка C++. В ней удивительно компактный формат удачно сочетается с широким охватом множества тем программирования на C++ — более широким по сравнению со всеми другими известными мне книгами подобного уровня.
— Дэй Бокс (Day Bock), член-учредитель C++-комитета ANSI/ISO

Авторы представили ясный и убедительное введение в программирование на языке C++, позволяющее студенту сразу же научиться писать нетривиальные программы.

— Стивен Клейнхаг (Stephen Clamage), Sun Microsystems Inc., председатель C++-комитета ANSI

Тот, кто освоит одну только эту книгу со всеми ее примерами и выполнит приведенные в ней упражнения, приобретет такие же навыки, которыми могут похвастаться многие профессиональные программисты.

— Джеффри Д. Олдем (Jeffrey D. Oldham), Оксфордский университет (Oxford University)

Почему книгу *Эффективное программирование на C++* стоит приобрести?

- Потому что изложение материала в ней начинается с рассмотрения самых полезных, а не самых примитивных понятий. И поэтому вы сразу же сможете приступить к написанию программ.
- Потому что в ней описываются реальные задачи и их решения, а не просто средства языка программирования. И поэтому вы не только поймете, что представляет собой то или иное языковое средство, но и узнаете как оно используется.
- Потому что здесь вы найдете описание возможностей как самого языка, так и стандартной библиотеки. И поэтому вы сможете использовать библиотечные средства с самых первых своих шагов на пути к вершинам программирования.

Авторы опробовали свой метод подачи материала на курсах интенсивного изучения языка C++ в Стэнфордском университете, и их студенты быстро научились писать серьезные программы прямо в учебных аудиториях.

Если вы еще только хотите научиться программировать на C++ или уже написали на этом языке несколько программ, но нуждаетесь в более глубоком освещении интересующих вас тем, выбранный авторами уникальный подход к освоению программирования на языке C++ делает книгу *Эффективное программирование на C++* совершенно необходимым дополнением вашей библиотеки.

Эндрю Кэниг — член отдела глобальных исследований в области программирования (лаборатория компании AT&T в г. Шеннон) и руководитель проекта комитета стандартов C++. Он программист с 30-летним стажем, причем 15 лет было им посвящено именно языку C++. Эндрю опубликовал более 150 статей о языке C++. Он является автором книги *C Traps and Pitfalls* и соавтором книги *Ruminations on C++*.

Барбара Му — независимый консультант с 20-летним опытом, приобретаемым ей в области разработки программного обеспечения. За 15 лет работы в компании AT&T она участвовала в создании одного из первых коммерческих продуктов, написанных на языке C++, руководила проектом создания первого компилятора языка C++ и возглавляла разработку Internet-предприятия сферы обслуживания WorldNet. Барбара — соавтор книги *Ruminations on C++*.

<http://www.aw.com/cseng/series/indepth/>

Фото на обложке: Спенсера Бейли (Spencer Bailey, Stone)

Издательский дом «Вильямс»
www.williamspublishing.com

ADDISON-WESLEY
Pearson Education

ISBN 5-8459-0350-5

